

emWin 开发指南

V1.2

本教程适用于正点原子所有 STM32 系列开发板

修订历史:

版本	日期	修改内容
V1.0	2022/06/20	第一次发布
V1.1	2022/07/17	修改图片大小
V1.2	2023/02/20	添加 RGB 移植内容



正点原子公司名称：广州市星翼电子科技有限公司

原子哥在线教学平台：www.yuanzige.com

开源电子网 / 论坛：www.openedv.com

正点原子官方网站：www.alientek.com

正点原子淘宝店铺：<https://openedv.taobao.com>

正点原子 B 站视频：<https://space.bilibili.com/394620890>

电话：020-38271790 传真：020-36773971

请下载原子哥 APP，数千讲视频免费学习，更快更流畅。

请关注正点原子公众号，资料发布更新我们会通知。



扫码下载“原子哥”APP



扫码关注正点原子公众号

第一章 emWin 介绍	16
1.1 emWin 简介	16
1.2. STemWin、emWin、 μ CGUI 三者之间的关系	16
1.3 emWin6.xx 版本和 6 以下版本的不同之处	16
1.4 MCU 移植 emWin 的硬件要求	17
第二章 emWin 裸机移植	18
2.1 移植准备工作	18
2.2 向工程添加文件	21
2.3 修改工程文件	22
2.4 触摸驱动移植	28
2.5 综合测试程序编写	32
2.6 下载验证	33
2.7 RGB 接口屏移植	34
2.8 使用 AC6 编译工程（扩展）	39
第三章 emWin 带操作系统移植	42
3.1 移植准备工作	42
3.2 向工程添加以及修改相应文件	42
3.3 添加应用程序	45
3.4 编写综合测试程序	54
3.5 下载验证	56
第四章 在 PC 上仿真 emWin	58
4.1 准备 emWin 仿真文件	58
4.2 打开 emWin 仿真文件	58
4.3 进行仿真	60
4.4 emWin 仿真文件介绍	62
4.4.1 Application 分组	62
4.4.2 Config 分组	64
4.4.3 GUI 分组	66
4.4.4 Sample 文件夹解析	66
第五章 文本显示	67
5.1 文本显示	67

5.2 文本显示 API 函数	67
5.3 文本显示实验	70
5.3.1 硬件设计	70
5.3.2 软件设计	70
5.3.3 下载验证	72
第六章 数值显示	74
6.1 数值显示 API 函数	74
6.2 显示十进制数值	78
6.3 显示浮点数值	79
6.4 二进制和十六进制数值	80
6.5 数值显示实验	81
6.5.1 硬件设计	81
6.5.2 软件设计	82
6.5.3 下载验证	83
第七章 2D 绘图	85
7.1 图形库基本 API 简介	85
7.1.1 画笔大小	85
7.1.2 基本绘图函数	85
7.1.3 Alpha 混色	91
7.2 其他图形绘制 API 函数	94
7.2.1 绘制线条	94
7.2.2 其他图形绘制函数	97
7.3 2D 绘图实验	104
7.3.1 硬件设计	104
7.3.2 软件设计	104
7.3.3 下载验证	110
第八章 显示位图	113
8.1 位图转换器	113
8.1.1 位图转换器简介	113
8.1.2 位图转换器的使用	113
8.2 位图绘制 API 简介	115
8.3 显示位图实验	117
8.3.1 硬件设计	117

8.3.2 软件设计	117
8.3.3 下载验证	119
第九章 颜色	120
9.1 逻辑颜色	120
9.2 预定义的基本颜色	121
9.3 调色板	121
9.4 颜色 API 函数	123
9.5 颜色显示实验	126
9.5.1 硬件设计	126
9.5.2 软件设计	126
9.5.3 下载验证	130
第十章 存储设备	131
10.1 存储设备简介	131
10.1.1 存储设备初窥	131
10.1.2 内存要求	133
10.2 存储设备实验	134
10.2.1 存储设备基本 API 函数	134
10.2.2 硬件设计	137
10.2.3 软件设计	138
10.2.4 下载验证	141
10.3 分段存储设备实验	141
10.3.1 分段存储简介	141
10.3.2 硬件设计	141
10.3.3 软件设计	142
10.3.4 下载验证	145
10.4 自动设备对象实验	145
10.4.1 自动设备对象简介	145
10.4.2 自动设备对象 API 函数	146
10.4.3 硬件设计	147
10.4.4 软件设计	148
10.4.5 下载验证	152
第十一章 窗口管理器	153
11.1 什么是窗口管理器	153

11.2 窗口术语说明	153
11.3 窗口回调机制	155
11.3.1 不使用回调渲染	155
11.3.2 使用回调渲染	155
11.3.3 背景窗口重绘和回调	156
11.3.4 无效化	156
11.3.5 渲染透明窗口	157
11.3.6 自动使用存储设备	157
11.4 消息	157
11.4.1 消息结构	157
11.4.2 消息类型	158
11.5 窗口重绘实验	159
11.5.1 硬件设计	161
11.5.2 软件设计	162
11.5.3 下载验证	165
11.6 窗口管理 API 函数实验	166
11.6.1 硬件设计	166
11.6.2 软件设计	166
11.6.3 下载验证	172
11.7 窗口裁剪实验	172
11.7.1 硬件设计	172
11.7.2 软件设计	173
11.7.3 下载验证	177
第十二章 GUIBulider 的使用	179
12.1 GUIBulider 简介	179
12.2 GUIBulider 使用步骤	181
12.3 GUIBulider 输出	186
12.4 修改 C 文件	190
12.5 GUIBulider 显示实验	191
12.5.1 硬件设计	191
12.5.2 软件设计	191
12.5.3 下载验证	196
第十三章 对话框	198

13.1 对话框基本原理	198
13.2 创建对话框	199
13.2.1 对话框资源表	199
13.2.2 对话框回调函数	199
13.2.3 初始化对话框	200
13.3 定义小工具功能	202
13.4 对话框 API 函数	204
第十四章 窗口小工具(控件)	206
14.1 小工具基础知识	206
14.1.1 可用小工具	206
14.1.2 了解重绘机制	208
14.1.3 如何使用小工具	208
14.2 通用控件 API 函数	209
14.2.1 用于控件的 WM 函数	209
14.2.2 常用 API 函数	210
14.2.3 用户绘制控件	213
第十五章 BUTTON 控件	215
15.1 BUTTON 控件简介	215
15.2 通知代码	215
15.3 键盘反应	215
15.4 BUTTON 常用 API 函数	216
15.5 BUTTON 控件基础实验	217
15.5.1 硬件设计	217
15.5.2 软件设计	218
15.5.3 下载验证	223
15.6 BUTTON 控件显示位图实验	223
15.6.1 制作位图 C 文件	223
15.6.2 GUIBulider 设计界面	224
15.6.3 硬件设计	228
15.6.4 软件设计	229
15.6.5 下载验证	233
第十六章 CHECKBOX 控件	234
16.1 通知代码	234

16.2 键盘反应	234
16.3 CHECKBOX 控件 API 函数	234
16.4 CHECKBOX 控件演示实验	237
16.4.1 硬件设计	237
16.4.2 软件设计	237
16.4.3 下载验证	242
第十七章 DROPDOWN 控件	243
17.1 DROPDOWN 控件简介	243
17.2 通知代码	243
17.3 键盘反应	244
17.4 DROPDOWN 控件 API 函数	244
17.5 DROPDOWN 控件演示实验	247
17.5.1 硬件设计	247
17.5.2 软件设计	248
17.5.2 下载验证	251
第十八章 EDIT 控件	252
18.1 EDIT 控件简介	252
18.2 通知代码	252
18.3 键盘反应	252
18.4 EDIT 控件 API 函数	253
18.5 EDIT 控件演示实验	258
18.5.1 硬件设计	258
18.5.2 软件设计	258
18.5.2 下载验证	263
第十九章 FRAMEWIN 控件	264
19.1 FRAMEWIN 控件简介	264
19.2 FRAMEWIN 控件 API 函数	265
19.3 FRAMEWIN 控件演示实验	270
19.3.1 硬件设计	270
19.3.2 软件设计	271
19.3.2 下载验证	276
第二十章 GRAPH 控件	278
20.1 GRAPH 控件简介	278

20.2 GRAPH 控件 API 函数.....	279
20.3 GRAPH 控件演示实验.....	284
20.3.1 硬件设计	284
20.3.2 软件设计	284
20.3.3 下载验证	298
第二十一章 HEADER 控件.....	299
21.1 HEADER 控件简介	299
21.2 通知代码	299
21.3 HEADER 控件 API 函数	299
21.4 HEADER 控件演示例程	300
21.4.1 硬件设计	300
21.4.2 软件设计	301
21.4.3 下载验证	305
第二十二章 ICONVIEW 控件.....	306
22.1 ICONVIEW 图标视图小部件简介	306
22.2 通知代码	307
22.3 键盘的反应	307
22.4 ICONVIEW 控件 API 函数	308
22.5 ICONVIEW 控件实验	311
22.5.1 硬件设计	311
22.5.2 软件设计	311
22.5.3 下载验证	316
第二十三章 SWIPELIST 控件.....	317
23.1 SWIPELIST 控件简介	317
23.2 通知代码	319
23.3 SWIPELIST 控件 API 函数	319
23.4 SWIPELIST 控件演示实验	321
23.4.1 硬件设计	321
23.4.2 软件设计	321
23.4.3 下载验证	325
第二十四章 TEXT 控件.....	326
24.1 TEXT 控件简介	326
24.2 通知代码	326

24.3 TEXT 控件 API 函数	327
24.4 TEXT 控件演示实验	328
24.4.1 硬件设计	328
24.4.2 软件设计	329
24.4.3 下载验证	332
第二十五章 LISTBOX 控件	334
25.1 LISTBOX 控件简介	334
25.2 通知代码	334
25.3 键盘反应	334
25.4 LISTBOX 控件 API 函数	335
25.5 LISTBOX 控件演示实验	337
25.5.1 硬件设计	337
25.5.2 软件设计	337
25.5.3 下载验证	343
第二十六章 LISTVIEW 控件	345
26.1 LISTVIEW 控件简介	345
26.2 通知代码	345
26.3 键盘反应	345
26.4 LISTVIEW 控件 API 函数	346
26.5 LISTVIEW 控件演示实验	348
26.5.1 硬件设计	348
26.5.2 软件设计	349
26.5.3 下载验证	355
第二十七章 LISTWHEEL 控件	357
27.1 LISTWHEEL 控件简介	357
27.2 通知代码	357
27.3 LISTWHEEL 控件 API 函数	358
27.4 LISTWHEEL 控件演示实验	360
27.4.1 硬件设计	360
27.4.2 软件设计	360
27.4.3 下载验证	363
第二十八章 RADIO 小工具	366
28.1 RADIO 控件简介	366

28.2 通知代码	366
28.3 键盘反应	366
28.4 RADIO 控件 API 函数	367
28.5 RADIO 控件演示实验	368
28.5.1 硬件设计	368
28.5.2 软件设计	368
28.5.3 下载验证	375
第二十九章 MULTIEDIT 小工具	376
29.1 MULTIEDIT 控件简介	376
29.2 通知代码	376
29.3 键盘反应	376
29.4 MULTIEDIT 控件 API 函数	377
29.5 MULTIEDIT 控件演示实验	378
29.5.1 硬件设计	378
29.5.2 软件设计	379
29.5.3 下载验证	384
第三十章 SLIDER 小工具	386
30.1 SLIDER 控件简介	386
30.2 通知代码	386
30.3 键盘反应	386
30.4 SLIDER 控件 API 函数	386
30.5 SLIDER 控件演示实验	388
30.5.1 硬件设计	388
30.5.2 软件设计	388
30.5.3 下载验证	395
第三十一章 字体	397
31.1 字体简介	397
31.2 字体类型	397
31.3 字体格式	398
31.3.1 C 文件格式	398
31.3.2 系统独立字体 (SIF) 格式	398
31.3.3 外部位图字体 (XBF) 格式	399
31.3.4 TrueType 字体(TTF)格式	399

31.4 声明自定义字体	400
31.5 选择字体	400
31.6 字体常用 API 函数	400
第三十二章 XBF 格式字体显示	404
32.1 XBF 格式字库制作	404
32.2 XBF 字体相关 API 函数	406
32.3 XBF 格式字库实验	408
32.3.1 硬件设计	408
32.3.2 软件设计	408
32.3.3 下载验证	418
第三十三章 TTF 格式字体显示	420
33.1 TTF 字体简介	420
33.2 TTF 字体相关 API 函数	420
33.3 TTF 字体显示实验	422
33.3.1 硬件设计	422
33.3.2 软件设计	422
33.3.3 下载验证	428
第三十四章 SIF 格式字体显示	429
34.1 SIF 格式字库制作	429
34.2 SIF 格式字体相关 API 函数	430
34.3 SIF 格式字体实验	430
34.3.1 硬件设计	430
34.3.2 软件设计	431
34.3.3 下载验证	439
第三十五章 汉字显示（外部存储器）	441
35.1 汉字显示简介	441
35.2 实验工程准备	441
35.3 字库制作	441
35.3.1 汉字字库制作	441
35.3.2 英文字符字库制作	442
35.4 汉字显示实验	443
35.4.1 硬件设计	443
35.4.2 软件设计	443

35.4.3 下载验证	449
第三十六章 BMP 图片显示	450
36.1 支持格式	450
36.2 BMP 解码 API 函数	450
36.3 BMP 显示实验	453
36.3.1 硬件设计	453
36.3.2 软件设计	454
36.3.3 下载验证	462
第三十七章 JPEG 图片显示	464
37.1 支持的 JPEG 压缩方法	464
37.2 RAM 使用要求	464
37.3 JPEG 解码 API 函数	465
37.4 JPEG 显示实验	467
37.4.1 硬件设计	467
37.4.2 软件设计	467
37.4.3 下载验证	476
第三十八章 GIF 图片显示	478
38.1 GIF 文件支持	478
38.2 GIF 解码 API 函数	478
38.3 GIF 显示实验	483
38.3.1 硬件设计	483
38.3.2 软件设计	484
38.3.3 下载验证	493
第三十九章 PNG 图片显示	495
39.1 基础工程准备	495
39.2 RAM 使用要求	496
39.3 PNG 解码 API 函数	496
39.4 PNG 显示实验	498
39.4.1 硬件设计	498
39.4.2 软件设计	498
39.4.3 下载验证	505
第四十章 软键盘	507
40.1 软键盘组成	507

40.2 软键盘设计	508
40.2.1 软键盘数据结构	508
40.2.2 分键盘的绘制	509
40.2.3 分键盘处理过程	512
40.2.4 软键盘父窗口	514
40.3 软键盘实验	518
40.3.1 硬件设计	518
40.3.2 软件设计	519
40.3.3 下载验证	521
第四十一章 多层显示(仅限 RGB)	523
41.1 多层显示简介	523
41.2 多层 API 函数	524
41.3 多层演示例程	525
41.3.1 硬件设计	525
41.3.2 软件设计	525
41.3.3 下载验证	532
第四十二章 AppWizard 设计器	533
42.1 AppWizard 使用	533
42.1.1 AppWizard 下载与安装	533
42.1.2 AppWizard 框架介绍	534
42.2 创建 AppWizard 工程流程	536
42.3 移植到工程	538

第一章 emWin 介绍

图形用户界面（GUI）是一种人与计算机通信和操作的界面，允许用户使用鼠标等输入设备操纵屏幕上的图标或菜单选项，以选择命令、调用文件、启动程序或执行其它一些日常任务。图形用户界面由多种控件及其相应的控制机制构成，在各种新式应用程序中都是标准化的，即相同的操作总是以同样的方式来完成，在图形用户界面，用户看到和操作的都是图形对象，应用的是计算机图形学的技术。在市面上，图形用户界面的种类非常多，比较常用的有：emWin、LVGL、MiniGUI、QT 等，本教程将基于 emWin 进行图形用户界面的开发。

本章分为以下几个部分：

- 1.1 emWin 简介
- 1.2 STemWin、emWin、μCGUI 三者之间的关系
- 1.3 emWin6.xx 版本和 6 以下版本的不同之处
- 1.4 MCU 移植 emWin 的硬件要求

1.1 emWin 简介

emWin 是由德国 SEGGER 公司开发，可为图形 LCD 设计提供高级支持，极大简化了 LCD 设计。在国内，很多嵌入式企业都会使用 emWin 来设计产品 UI，特别是在医疗和服务行业。

✧ **SEGGER 官方 emWin 用户手册下载：**

emWin 用户手册是英文版本，它的下载网址为 <https://www.segger.com/downloads/emwin>，如下图所示：

	Version	Date	File size
 emWin User Manual	V6.24	[2022-01-18]	19,225 KB

图 1.1.1 emWin 教程资料下载

在上图中，我们下载到的是被命名为 UM03001_emWin.pdf 的用户文档，当前最新版本是 6.24（**注意：**如果官方升级了新版本，大家使用最新版本即可，手册都是向下兼容的）。

✧ **emWin 中文版用户手册下载**

emWin 中文版用户手册是国人翻译的，虽然翻译的版本过低，但是对于初学者来说已经足够了。emWin 中文版用户手册存放路径：光盘资料 A 盘→6,软件资料→EMWIN 学习资料→emWin5.42 中文手册.pdf。

1.2. STemWin、emWin、μCGUI 三者之间的关系

在选择图形用户界面开发时，很多人会混淆 STemWin、emWin 和 μCGUI 之间的联系，其中，emWin 和 μCGUI 是一样的东西，只不过换了一个名称罢了。

STemWin 和 emWin 两者相差的是有没有“ST”前缀，它们的实现原理都是大同小异，因为 STemWin 是 SEGGER 公司授权给 ST（意法半导体）的图形用户界面，所以 STemWin 和 emWin 也是一样的图形界面库，只不过 STemWin 针对 ST 的微控制器做了专门的优化，例如 STemWin 在需要浮点处理的位置，专门为 ST 带有 FPU 的微控制器做了优化。

1.3 emWin6.xx 版本和 6 以下版本的不同之处

不同之处：emWin 6.0 版本进行了很大的更新，特别是推出了性能更强的图形化创建工具 AppWizard，AppWizard 工具所生成的代码可移植到 MCU 板子上。

相同之处：emWin6.0 向下兼容低版本。简单来说：如果用户使用了较低的 emWin 版本开发，也可以完全在高版本的 emWin 上运行。

1.4 MCU 移植 emWin 的硬件要求

打开 emWin 用户手册的 73 页，在这页中，讲述了 emWin 移植的硬件要求，如下图所示：

Small systems (no Window Manager)

- RAM: 100 Bytes
- Stack: 600 Bytes
- ROM: 10-25 KBytes (depending on the functionality used)

Big systems (including Window Manager and widgets)

- RAM: 2-20 KBytes *
- Stack: 1200-1800 bytes *
- ROM: 30-120 KBytes *

* Depending on the functionality used. The numbers above are only a rough estimation and could not reflect the exact requirements of an application. RAM requirement can be increased significantly when using large memory devices or when drawing PNG images. ROM requirements increase according to the number of fonts and/or images used in the application. All values are rough estimates and cannot be guaranteed. More details can be found in the chapter *Performance and Resource Usage* on page 2996.

图 1.4.1 emWin 移植硬件要求

从上图可以看到，emWin 移植的硬件要求分为两类：

- ✧ 小型系统硬件要求，该系统不能使用 emWin 图形界面提供的窗口管理器功能。
- ✧ 大型系统硬件要求，该系统可以使用 emWin 图形界面全部功能（窗口管理器和小控件）。

这两种系统硬件参数如下表所示：

硬件要求	小型系统	大型系统
微控制器	16/32 位	16/32 位
RAM（内存）	100 字节	2~20k 字节
Stack（堆栈）	600 字节	1200~1800 字节
ROM（flash）	10~25k 字节 （取决于所使用的功能）	30~120k 字节 （取决于所使用的功能）

表 1.4.1 emWin 移植的硬件要求

从上表可知：微处理器至少需要 16 位以上，所以传统的 51、52 单片机无法移植 emWin，它们都是 8 位的微处理器。接下来，我们来看一下正点原子 Mini 板是否满足 emWin 最低移植需求：它的 MCU 为 STM32F103RCT6，其主频率 72MHz，Flash 为 256K，SRAM 为 64K，显然 Mini 开发板的 MCU 可以满足 emWin 图形库的移植要求。

注意：上述大型系统硬件要求的数字只是粗略估计，不能反映实际的确切要求。当使用大型内存设备或绘制 PNG 图像时，RAM 需求会显著增加，ROM 需求会根据应用程序中使用的字体和图像的数量而增加。

第二章 emWin 裸机移植

本章笔者主要介绍 emWin 的无操作系统移植，即裸机移植，让读者了解如何将 emWin 移植到 STM32 的开发板。在移植之前，先声明一点，本教程适用于正点原子所有支持 emWin6.x 的 STM32 开发板，各个开发板移植 emWin 的不同之处，将会在教程中以表格或者红色字体的形式指出。

本章分为以下几个部分：

- 2.1 移植准备工作
- 2.2 向工程添加文件
- 2.3 修改工程文件
- 2.4 触摸驱动移植
- 2.5 综合测试程序编写
- 2.6 下载验证
- 2.7 RGB 接口屏移植
- 2.8 使用 AC6 编译工程（扩展）

2.1 移植准备工作

◇ 准备 emWin 软件包

由于 emWin 软件包不断地更新换代，而 STemWin 软件包始终停留在 5.44 版本，所以本教程不会使用 STemWin 软件包，如果大家想使用 STemWin 软件包，请看正点原子之前发布的 STemWin 教程且务必使能硬件 CRC 时钟，否则无法启动 STemWin。

emWin 软件包可以通过 MDK 软件的“pack installer”引导下载，目前下载的 emWin 版本是 emWin6.16c，本教程配套的例程正是使用该版本。下面是笔者通过“pack installer”引导下载 emWin 软件包的步骤：

第一步：打开 MDK，点击 pack installer  图标，如下图所示：

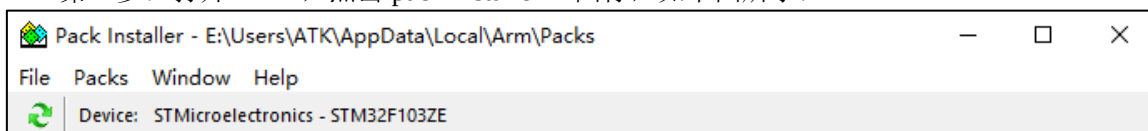


图 2.1.1 Pack installer 安装路径

第二步：找到 keil::MDK-Middleware 选项，点击 install/Update 安装 7.13.0 版本。该版本对应 emWin 为 6.16c，如下图所示：

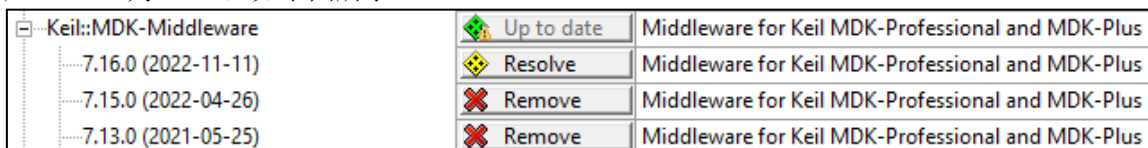


图 2.1.2 安装 7.13.0pack 包

第三步：安装完成之后，根据图 2.1.1 上方的路径打开，如下图所示：



图 2.1.3 MDK-Middleware 更新为最新版本

第四步：打开 7.13.0 文件夹，如下图所示：

<< Users > ATK > AppData > Local > Arm > Packs > Keil > MDK-Middleware > 7.13.0		
名称	类型	大小
Board	文件夹	
Doc	文件夹	
emWin	文件夹	
FileSystem	文件夹	
Network	文件夹	
Network_v6	文件夹	
USB	文件夹	
Utilities	文件夹	
Keil.MDK-Middleware.pdsc	PDSC 文件	269 KB

图 2.1.4 emWin 软件包

第五步：打开上图中的 emWin 文件夹，如下图所示：

<< ATK > AppData > Local > Arm > Packs > Keil > MDK-Middleware > 7.13.0 > emWin		
名称	修改日期	类型
Doc	2022/1/23 11:05	文件夹
Include	2022/1/23 11:05	文件夹
Lib	2022/1/23 11:05	文件夹
Sample	2022/1/23 11:05	文件夹
Tool	2022/1/23 11:05	文件夹

图 2.1.5 Keil5 的 emWin 文件路径

如果没有找到相应 emWin6.16c 软件包，可以在正点原子配套资料查找，该配套资料存放的路径为：光盘资料 A 盘→6,软件资料→3，emWin 学习资料→emWin 源码压缩包。

上图中，emWin 软件包包含五个文件夹，这些文件夹介绍如下表所示：

文件	说明
Doc	emWin 英文教程以及 AppWizard 设计器教程
Include	emWin 函数的头文件
Lib	emWin 的函数库
Sample	相关 emWin 实例以及相关驱动配置
Tool	一些 emWin 软件

表 2.1.1 emWin 软件包的各文件说明

精简版的 emWin 移植库只需要 Include、Lib、Sample 文件夹即可，其他文件夹可删除。

✧ 建立移植基础工程

在 emWin 移植之前，我们需要准备一个基础例程，最好该工程已经包含 LCD、MALLOC 和 LED 等驱动文件。在基础工程的 Middlewares 文件夹下新建 emWin 文件夹，接着在该文件夹下分别新建 Config、inc、Lib、OS 和 emWin_Demo 文件夹，这些文件夹介绍如下表所示：

文件夹	描述
Config	存放 emWin 配置文件
inc	存放 emWin 函数的头文件
Lib	存放 emWin 的函数库文件
OS	存放 GUI 的配置/系统相关外部设备文件
emWin_Demo	存放用户文件

表 2.1.2 新建 5 个文件夹

上表中的文件夹主要存放 emWin 软件包的源码和用户编写的源码。下面笔者分别讲解这五个文件夹到底添加哪些文件，如下步骤所示：**（注意：这些文件夹存放的文件是在图 2.1.5 的 emWin 软件包下获取）。**

第一步：Config 文件夹添加 emWin 配置文件，如下图所示：

<< 3, emWin例程 > emWin实验1 emWin无操作系统移植 > Middlewares > emWin > Config		
名称	类型	大小
 GUI_X_Touch_Analog.c	C 文件	4 KB
 GUIConf.c	C 文件	4 KB
 GUIConf.h	H 文件	4 KB
 GUIDRV_Lin_Template.c	C 文件	43 KB
 GUIDRV_Lin_Template.h	H 文件	2 KB
 GUIDRV_Template.c	C 文件	25 KB
 LCDConf.c	C 文件	7 KB
 LCDConf.h	H 文件	3 KB

图 2.1.6 Config 文件夹添加的文件

上图中，所有的文件都来自于 emWin 软件包，例如 GUIConf.c/h 和 LCDConf.c/h 文件在 emWin\Sample\Config 路径下获取，而 GUI_X_Touch_Analog.c 和 GUIDRV_Template.c 文件分别在 emWin\Sample\DisplayDriver 和 emWin\Sample\GUI_X 路径下获取。

第二步：inc 文件夹添加 emWin\Include 路径下的全部文件，如下图所示：

<< 3, emWin例程 > emWin实验1 emWin无操作系统移植 > Middlewares > emWin > inc		
名称	类型	大小
 AppWizard.h	H 文件	55 KB
 BUTTON.h	H 文件	11 KB
 CALENDAR.h	H 文件	8 KB
 CHECKBOX.h	H 文件	10 KB

图 2.1.7 inc 文件夹添加的文件

第三步：Lib 文件夹添加 emWin\Lib\ARM 路径下的全部文件，如下图所示：

<< 3, emWin例程 > emWin实验1 emWin无操作系统移植 > Middlewares > emWin > Lib		
名称	类型	大小
 GUI_CM0_L.lib	VisualStudio.lib....	7,459 KB
 GUI_CM3_L.lib	VisualStudio.lib....	7,490 KB
 GUI_CM4F_L.lib	VisualStudio.lib....	7,916 KB

图 2.1.8 Lib 文件夹添加的文件

正点原子 STM32 系列开发板使用到的是 GUI_CM3_L.lib 以及 GUI_CM4F_L.lib 文件，如果用户的 MCU 为 CM3 内核，则工程使用 GUI_CM3_L.lib 文件。GUI_CM4F_L.lib 文件只能使用在 CM4 以上的 MCU 内核。

第四步：OS 文件夹添加 emWin\Sample\GUI_X 路径下的 GUI_X.c 和 GUI_X_uCOS.c 文件，如下图所示：



图 2.1.9 OS 文件夹添加的文件

上图中的文件分别用在裸机 emWin 移植和操作系统 emWin 移植。

2.2 向工程添加文件

打开工程并在工程上添加四个工程管理项组，它们分别为 Middlewares/emWin/Config（添加配置文件）、Middlewares/emWin/Lib（存放 emWin 函数库）、Middlewares/OS（GUI 配置/系统相关外部设备）和 Middlewares/emWin/emWin_Demo（存放用户文件），如下图所示：

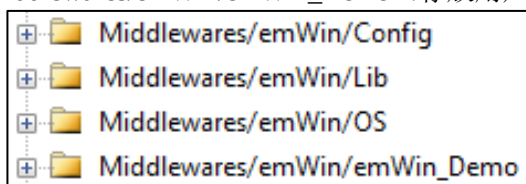


图 2.2.1 添加四个工程管理项组

这些分组所添加的文件，如下所示：

Middlewares/emWin/Config 组添加 emWin\Config 路径下的.c 文件：

C 文件	路径	说明
GUI_X_Touch_Analog.c	Config	触摸驱动
GUIConf.c		GUI 配置
GUIDRV_Template.c		设备驱动
LCDCConf.c		显示驱动配置

表 2.2.1 Middlewares/emWin/Config 工程管理项组添加文件的描述

Middlewares/emWin/Lib 组添加工程的 emWin\Lib 路径下的.c 文件（注意：请对应 MCU 来选择）：

Lib 文件夹	说明
GUI_CM3_L.lib	针对 M3 类型的 MCU（Mini、精英、战舰）
GUI_CM4F_L.lib	针对 M4 类型的 MCU（探索者、阿波罗 F4、水星 F4、DMF407.....）
	针对 M7 类型的 MCU（北极星、阿波罗/水星（H/F7.....）

表 2.2.2 根据 MCU 内核选择 Lib 文件

Middlewares/emWin/OS 组添加工程的 emWin\OS 路径下的.c 文件（注意：裸机移植请选择 GUI_X.c 文件，操作系统请选择 GUI_OS.c 文件）：

C 文件	说明	说明
GUI_X.c	裸机 emWin 移植	GUI 的配置/系统相关外部设备
GUI_OS.c	操作系统 emWin 移植	

表 2.2.3 Middlewares/emWin/OS 工程管理项组添加文件

本章讲解的是裸机 emWin 移植，则该分组添加 GUI_X.c 文件即可。

点击“Options for TargetC/C++”选项卡添加文件路径，如下图所示：

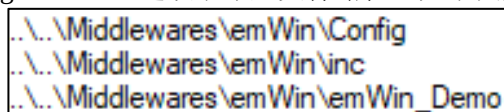


图 2.2.2 添加文件路径

2.3 修改工程文件

✧ 修改 GUIConf.h 文件

该文件主要定义是否支持操作系统、鼠标、窗口管理等配置项，如下源码所示：

```
#ifndef GUICONF_H
#define GUICONF_H
/*****
*
*      Multi layer/display support
*/
#define GUI_NUM_LAYERS          1              /* 显示的最大层数 */
/*****
*
*      Multi tasking support
*/
#define GUI_OS                  (0)            /* 0:不使用操作系统 */
/*****
*
*      Configuration of touch support
*/
#define GUI_SUPPORT_TOUCH      (1)            /* 0:不支持触摸 */
/*****
*
*      Default font
*/
#define GUI_DEFAULT_FONT      &GUI_Font6x8    /* 默认字体 */
/*****
*
*      Configuration of available packages
*/
#define GUI_SUPPORT_MOUSE      1              /* 支持鼠标 */
#define GUI_WINSUPPORT         1              /* 支持窗口管理 */
#define GUI_SUPPORT_MEMDEV     1              /* 支持存储设备 */
#define GUI_SUPPORT_DEVICES    1              /* 使用设备指针 */
#endif // Avoid multiple inclusion
```

由于本章讲解的是裸机 emWin 移植内容，所以 GUI_OS 配置项设置为 0，表示不使用操作系统，其他配置项根据需求启用。

✧ 修改 GUIConf.c 文件

GUIConf.c 文件主要定义 emWin 所使用的内存管理方式，该内存管理方式可使用自定义的 USE_EXRAM 宏定义来选择，如果该宏定义为 1，则使用外部 SRAM；如果宏定义为 0，则使用内部 SRAM。由于本教程兼容了正点原子多款 STM32 开发板，所以用户根据开发板来选择内存管理方式。用户选择方式如下表所示：

开发板	外部 RAM	USE_EXRAM 设置	GUI_NUMBYTES 设置
Mini 板	无	0	20*1024
精英板	无	0	25*1024
战舰板	有	1	500*1024
探索者	有	1	500*1024
阿波罗和水星 (F427/H743/F767)	有	1	8*1024*1024
北极星 (H750/F750)	有	1	8*1024*1024
Mini Pro H750	无	0	300*1024

表 2.3.1 根据开发板设置 USE_EXRAM 宏定义

这里小编建议大家使用内部 SRAM 的方式分配内存给 emWin 图形库，因为使用此方式，能加快 LCD 刷新速度，GUIConf.c 文件修改如下源码所示：

```
#include "GUI.h"
#include "../MALLOC/malloc.h"

#define USE_EXRAM      0                /* 根据开发板选择，0 为不使用外部 SRAM */
#define GUI_NUMBYTES (20*1024)         /* 根据开发板选择，设置 EMWIN 内存大小 */

void GUI_X_Config(void)
{
    #if (USE_EXRAM)
        /* 从外部 SRAM 中分配 GUI_NUMBYTES 字节的内存 */
        U32 *aMemory = mymalloc(SRAMEX, GUI_NUMBYTES);
    #else
        //
        // 32 bit aligned memory area
        //
        static U32 aMemory[GUI_NUMBYTES / 4];
    #endif

    //
    // Assign memory to emWin
    //
    GUI_ALLOC_AssignMemory(aMemory, GUI_NUMBYTES);
    //
    // Set default font
    //
    GUI_SetDefaultFont(GUI_FONT_6X8);
}
```

上述源码使用了一个条件编译，如果 USE_EXRAM 配置项为 0，则使用内部 SRAM 内存；如果 USE_EXRAM 配置项为 1，则使用外部 SRAM 内存。

✧ 修改 GUIDRV_Template.c 文件

该文件主要完成 emWin 的打点、读点以及填充等函数，该文件只需要修改四个函数，分别为 _SetPixelIndex、_GetPixelIndex、_FillRect 和 _DrawBitLine16BPP 函数。这些函数修改后如下所示：

```
/* 定义 LCD 的命令和数据位 */
#define UCGUI_LCD_CMD    LCD_BASE                /* 地址为 LCD 的 CMD */
#define UCGUI_LCD_DATA   (((LCD_BASE >> 1) + 1) << 1) /* 地址为 LCD 的 RAM */

static void _SetPixelIndex(GUI_DEVICE * pDevice,
                           int x, int y,
                           LCD_PIXELINDEX PixelIndex)
{
    #ifndef WIN32
        LCDSIM_SetPixelIndex(x, y, PixelIndex, pDevice->LayerIndex);
    #else
        //
        // Convert logical into physical coordinates (Dep. on LCDConf.h)
        //
        #if (LCD_MIRROR_X == 1) || (LCD_MIRROR_Y == 1) || (LCD_SWAP_XY == 1)
            int xPhys, yPhys;

            xPhys = LOG2PHYS_X(x, y);
            yPhys = LOG2PHYS_Y(x, y);
        #else
            #define xPhys x
            #define yPhys y
        #endif
        GUI_USE_PARA(pDevice);
        GUI_USE_PARA(x);
        GUI_USE_PARA(y);
    #endif
}
```

```

GUI_USE_PARA(PixelIndex);
{
    //
    // Write into hardware ... Adapt to your system
    //
    // TBD by customer...
    //
    lcd_draw_point(x,y,PixelIndex); /* 打点函数 */
}
#if (LCD_MIRROR_X == 0) && (LCD_MIRROR_Y == 0) && (LCD_SWAP_XY == 0)
    #undef xPhys
    #undef yPhys
#endif
#endif
}

static LCD_PIXELINDEX _GetPixelIndex(GUI_DEVICE * pDevice, int x, int y)
{
    LCD_PIXELINDEX PixelIndex;
#ifdef WIN32
    PixelIndex = LCDSIM_GetPixelIndex(x, y, pDevice->LayerIndex);
#else
    //
    // Convert logical into physical coordinates (Dep. on LCDConf.h)
    //
    #if (LCD_MIRROR_X == 1) || (LCD_MIRROR_Y == 1) || (LCD_SWAP_XY == 1)
        int xPhys, yPhys;
        xPhys = LOG2PHYS_X(x, y);
        yPhys = LOG2PHYS_Y(x, y);
    #else
        #define xPhys x
        #define yPhys y
    #endif
    GUI_USE_PARA(pDevice);
    GUI_USE_PARA(x);
    GUI_USE_PARA(y);
    {
        //
        // Write into hardware ... Adapt to your system
        //
        // TBD by customer...
        //
        PixelIndex = lcd_read_point(x,y); /* 读函数 */
    }
    #if (LCD_MIRROR_X == 0) && (LCD_MIRROR_Y == 0) && (LCD_SWAP_XY == 0)
        #undef xPhys
        #undef yPhys
    #endif
#endif
    return PixelIndex;
}

static void _FillRect(GUI_DEVICE * pDevice, int x0, int y0, int x1, int y1)
{
    int x;

    if (GUI_pContext->DrawMode & LCD_DRAWMODE_XOR)
    {
        for (; y0 <= y1; y0++)
        {
            for (x = x0; x <= x1; x++)
            {
                _XorPixel(pDevice, x, y0);
            }
        }
    }
}

```



```

    }
    else
    {
        lcd_fill(x0,y0,x1,y1,LCD_COLORINDEX); /* 填充区域 */
    }
}

static void _DrawBitLine16BPP(GUI_DEVICE * pDevice, int x, int y, U16 const
                             GUI_UNI_PTR * p, int xsize)
{
    LCD_PIXELINDEX pixel;
    /* RGB 显示屏 */
    if (lcdltdc.pwidth != 0)
    {
        for (;xsize > 0;xsize -- , x++ ,p++)
        {
            _SetPixelIndex(pDevice,x,y,*p);
        }
    }
    else /* MUCLCD 显示屏 */
    {
        lcd_set_cursor(x,y);
        *(__IO uint16_t *) (UCGUI_LCD_CMD) = lcddev.wramcmd; /* 写入颜色值 */

        for (;xsize > 0; xsize --, x++, p++)
        {
            pixel = *p;
            *(__IO uint16_t *) (UCGUI_LCD_DATA) = pixel;
        }
    }
}

```

为了兼容多款 LCD IC 的驱动，所以打点、读点和填充等函数可调用正点原子 LCD 驱动文件下的函数实现。

注意：由于 Mini 板采用管脚模拟 fsmc 接口，所以_DrawBitLine16BPP 函数不能使用上面的形式编写，该函数修改后如下源码所示：

```

static void _DrawBitLine16BPP(GUI_DEVICE * pDevice, int x, int y,
                             U16 const GUI_UNI_PTR * p,
                             int xsize)
{
    LCD_PIXELINDEX pixel;
    lcd_set_cursor(x,y);
    LCD_RS(0);
    LCD_CS(0);
    LCD_DATA_OUT(lcddev.wramcmd); /* 写指令 */
    LCD_WR(0);
    LCD_WR(1);
    LCD_CS(1);
    for (;xsize > 0; xsize--, x++, p++)
    {
        pixel = *p;
        lcd_wr_data(pixel); /* 写数据 */
    }
}

```

上述源码是针对正点原子 Mini 开发板，其他开发板跳过此处。

✧ 修改 LCDConf.c 文件

这个文件只有 LCD_X_Config 和 LCD_X_DisplayDriver 这 2 个函数。因为 emWin 自带一些 LCD 驱动 IC 的驱动程序，使用 LCD_X_Config 函数可直接使用 emWin 的 LCD 驱动 IC 的驱动程序，LCD_X_Config 函数代码如下所示：

```

#include "GUI.h"
#include "../BSP/LCD/lcd_ili93xx.h"
#include "../BSP/TOUCH/touch.h"
#include "GUIDRV_Template.h"

```

```
#include "GUIDRV_FlexColor.h"

/**
 * @brief      配置程序,用于创建显示驱动器件,设置颜色转换程序和显示尺寸
 * @param      无
 * @retval     无
 */
void LCD_X_Config(void)
{
    /* 创建显示驱动器件 */
    GUI_DEVICE_CreateAndLink(&GUIDRV_Template_API, GUICC_M565, 0, 0);
    LCD_SetSizeEx(0, lcddev.width, lcddev.height);
    LCD_SetVSizeEx(0, lcddev.width, lcddev.height);
}

/**
 * @brief      显示器驱动的回调函数
 * @param      LayerIndex:布局索引
 * @param      Cmd      :选项
 * @param      pData     :数据
 * @retval     无
 */
int LCD_X_DisplayDriver(unsigned LayerIndex, unsigned Cmd, void *pData)
{
    GUI_USE_PARA(LayerIndex);
    GUI_USE_PARA(Cmd);
    GUI_USE_PARA(pData);
    return 0;
}
```

上面代码中的 GUI_DEVICE_CreateAndLink 函数用来创建显示驱动器件，设置用于存取显示器的颜色转换程序，并将驱动器件连接到给定层的器件列表中。该函数的第一个形参指向结构体的指针，这个结构体就是显示器的驱动。因为 emWin 自带一些 LCD 驱动 IC 的驱动程序，因此这里可以选择相应的驱动程序，具体使用请查阅 emWin 用户手册的“显示驱动”章节；第二个参数是指定所使用的调色板，emWin 中有很多调色板可以使用，具体使用请查阅 emWin 手册的“颜色”章节，在这里我们使用的是 GUICC_M565 的调色板，也就是 RGB565，最后我们根据 LCD 尺寸来设置屏幕的大小。

✧ 提供 emWin 时基

首先把定时器驱动添加到 Drivers/BSP 工程管理项分组中，如下图所示：

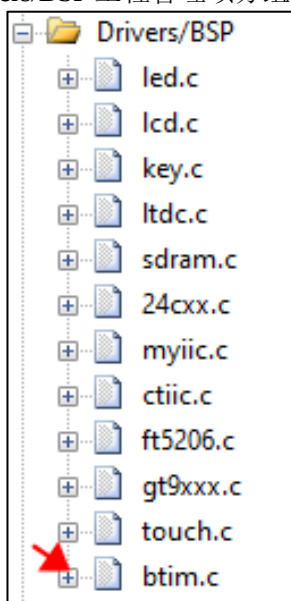


图 2.3.1 添加定时器驱动

打开 btim.c 文件，声明 GUI.h 和 OS_TimeMS 参数，如下图所示：

```
#include "GUI.h"

TIM_HandleTypeDef g_timx_handle;...../* 定时器参数句柄 */
extern __IO int32_t OS_TimeMS;
```

图 2.3.2 声明 GUI.h 和 OS_TimeMS 参数

定时器中断服务函数让 OS_TimeMS 参数加 1，如下图所示：

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == BTIM_TIMX_INT)
    {
        OS_TimeMS++;
    }
}
```

图 2.3.3 定时器中断服务让 OS_TimeMS 加 1

接着，我们在 main 函数中让定时器初始化，并设置定时周期为 1ms。

✧ 修改工程目标名称

本教程是以标准例程-HAL 库版本的内存管理实验工程为基础工程，内存管理实验工程的工程名为“MALLOC”，为了规范工程，笔者建议将工程的目标名修改为“emWin”或根据读者的实际场景进行修改，修改如下图所示：



图 2.3.4 修改工程目标名

✧ 添加 emWin 需要添加到内部 FLASH 的.o（注意：只针对北极星和 Mini Pro H750 开发板，其他开发板跳过此处）

因为我们的基础工程是 MALLOC 内存管理，所以有关分散加载的内容，这里就不再赘述，大家可以参考《STM32H750 开发指南》，但是针对 emWin 的移植，我们统一修改了分散加载文件的名称，命名为:qspi_code.scf，这样做是为了与其他的分散加载区分开来，因为针对 emWin 的移植我们需要多添加部分.o 到内部 FLASH，这部分添加内部 FLASH 的.o 不是每个 emWin 例程都需要加的，不同的实验需要的添加的.o 是不一样的，添加.o 过程如下所示：

✓ 打开分散加载文件，如下图所示：

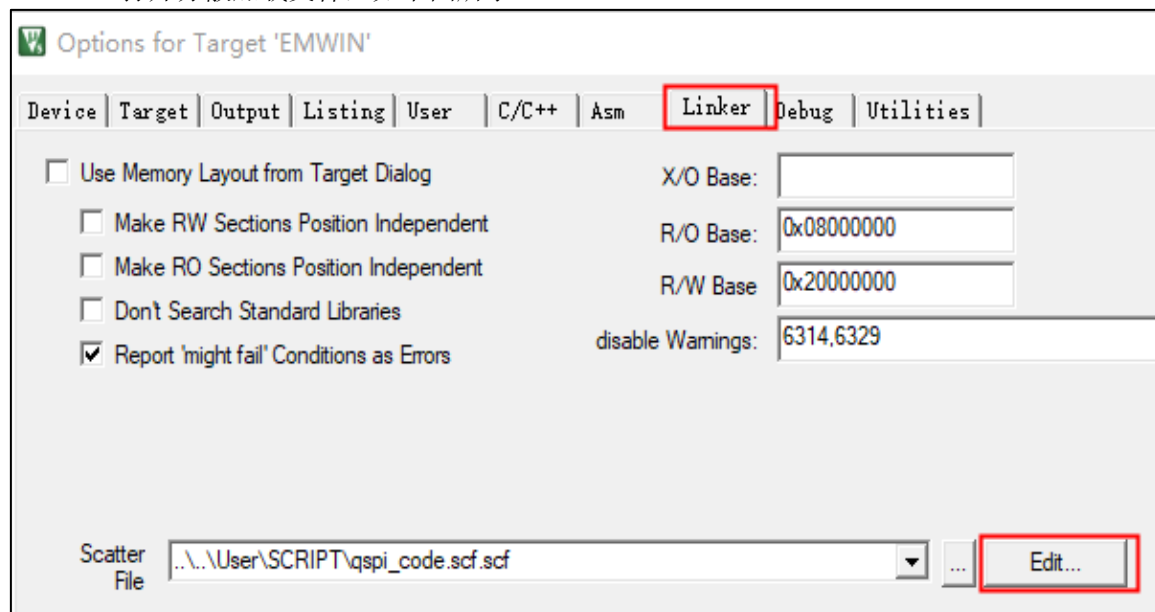


图 2.3.5 打开分散加载文件

✓ 添加 emWin 需要添加的.o，如下图所示：

```
rand.o
edit.o
hl_free.o
hl_init.o
hl_extend.o
hl_alloc.o
init_alloc.o
rt_ctype_table.o
rt_heap_descriptor_intlibspace.o
```

图 2.3.6 emWin 需要添加的.o

✧ 编写测试代码

到这一步 emWin 的移植基本已经完成，下面我们可以编写一个测试代码来测试一下移植是否成功，测试代码如下：

```
int main(void)
{
    /* 硬件初始化 */
    lcd_init();
    tp_dev.init();
    __HAL_RCC_CRC_CLK_ENABLE();
    GUI_Init();
    GUI_SetBkColor(GUI_BLUE);
    GUI_SetColor(GUI_YELLOW);
    GUI_Clear();
    GUI_SetFont(&GUI_Font24_ASCII);
    GUI_DispStringAt("Hello Word!", 0, 0);
    while(1);
}

/* 初始化 LCD */
/* 触摸屏初始化 */
/* 使能 CRC 时钟 */
/* emWin 初始化 */
/* 设置背景颜色 */
/* 设置颜色 */
/* 清屏 */
/* 设置字体 */
/* 在指定位置显示字符串 */
```

由于我们移植的不是 STemWin，而是 emWin，所以 CRC 时钟可选择开启。测试代码很简单，是在 LCD 上的 (0, 0) 坐标处显示 “Hello Word!” 字符串。

2.4 触摸驱动移植

emWin 触摸控制器是可选，因为它并不是运行 emWin 所必需的输入设备，简单来说，不需要触摸驱动也可以控制 UI 界面（例如键盘，按键等输入设备）。由于正点原子的 TFTLCD 以及 RGBLCD 都可以触摸，所以这一小节选择触摸控制器作为 emWin 的输入设备。添加触摸输入设备如下步骤所示：

✧ 修改和添加触摸屏底层驱动程序

首先将底层的触摸屏驱动添加到工程中，这里需要将 touch.c, gt9xxx.c, ctiic.c、ft5206.c、24cxx.c 和 myiic.c 这六个文件添加到工程的 Drivers/BSP 工程管理项中，如下图所示：

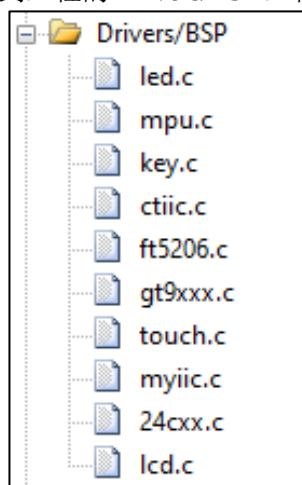


图 2.4.1 添加触摸驱动

注意：如果添加触摸驱动之后，编译报错，可能需要读者添加其他驱动文件。

✧ 修改 GUI_X_Touch_Analog.c 文件

此文件非常简单，需要读者调用函数获取显示屏的触摸值，提供给 emWin 系统处理，文件修改代码如下：

```
#include "GUI.h"
#include "../BSP/TOUCH/touch.h"
#include "../BSP/LCD/ILI93xx.h"
#include "../SYSTEM/usart/usart.h"

void GUI_TOUCH_X_ActivateX(void)
{
    /* XPT2046_WriteCMD(0x90); */
}

void GUI_TOUCH_X_ActivateY(void)
{
    /* XPT2046_WriteCMD(0xd0); */
}

int GUI_TOUCH_X_MeasureX(void)
{
    int32_t xvalue;
    uint16_t x, y;

    /* 判断电阻屏还是电容屏 */
    if ((tp_dev.touchtype & 0x80) == 0)
    {
        tp_dev.scan(0);

        if (tp_dev.sta & TP_PRES_DOWN) /* 触摸屏被按下 */
        {
            /* 电阻屏出现触摸没有那么顺畅的问题，
             请把 touch.c 文件下的 TP_ERR_RANGE 宏定义设置大一点 */
            tp_read_xy2(&x, &y);
            return (4095 - x);
        }
    }
    else /* 电容屏 */
    {
        tp_dev.scan(0);
        xvalue = tp_dev.x[0];
        return xvalue;
    }
    return 0;
}

int GUI_TOUCH_X_MeasureY(void)
{
    int32_t yvalue;
    uint16_t x, y;

    /* 判断电阻屏还是电容屏 */
    if ((tp_dev.touchtype & 0x80) == 0)
    {
        /* 解决快速按下的鼠标乱屏问题 */
        tp_dev.scan(0);

        if (tp_dev.sta & TP_PRES_DOWN) /* 触摸屏被按下 */
        {
            tp_read_xy2(&x, &y);
            return y;
        }
    }
}
```

```

else          /* 电容屏 */
{
    yvalue = tp_dev.y[0];
    return yvalue;
}
return 0;
}

```

可以看到，emWin 的触摸驱动文件 GUI_X_Touch_Analog.c 只提供四个函数，这些函数的描述如下表所示：

函数	描述
GUI_TOUCH_X_ActivateX()	获取 X 轴的值
GUI_TOUCH_X_ActivateY()	获取 Y 轴的值
GUI_TOUCH_X_MeasureX()	获取 X 轴的 AD 值
GUI_TOUCH_X_MeasureY()	获取 Y 轴的 AD 值

表 2.4.1 GUI_X_Touch_Analog.c 的四个函数描述

表 2.4.1 中前两个函数我们没有使用到，emWin 真正调用了 GUI_TOUCH_X_MeasureX 和 GUI_TOUCH_X_MeasureY 这两个函数来获取触摸屏按下时的 X 轴和 Y 轴的 AD 值，这里我们根据触摸类型来判断屏幕使用的是电容屏还是电阻屏，并分别作相应的处理，也就是调用不同的函数。注意：电容屏可直接输出坐标的 AD 值，而电阻屏需要转换得到坐标 AD 值。

✧ 修改 GUIConf.h 和 LCDConf.c 文件

将 GUIConf.h 文件中的 GUI_SUPPORT_TOUCH 定义为 1，使 emWin 系统支持触摸。在 LCDConf.c 文件的开始处添加如下代码：

```

/* 与触摸屏有关定义，根据实际情况填写 */
#define TOUCH_AD_TOP      160          /* 按下触摸屏的顶部，写下 y 轴模拟输入值。*/
#define TOUCH_AD_BOTTOM   3990        /* 按下触摸屏的底部，写下 y 轴模拟输入值。*/
#define TOUCH_AD_LEFT     160          /* 按下触摸屏的左侧，写下 x 轴模拟输入值。*/
#define TOUCH_AD_RIGHT    3990        /* 按下触摸屏的右侧，写下 x 轴模拟输入值。*/

```

以上定义了电阻屏触摸屏下左右边界的 AD 值，此值为实际测触摸屏得出来的，以通过串口输出查看，也可以使用 emWin 提供的代码来获取，这个代码如下所示：

```

void emwin_main(void)
{
    GUI_PID_STATE TouchState;
    int xPhys;
    int yPhys;
    GUI_Init();
    GUI_SetFont(&GUI_Font20_ASCII);
    GUI_CURSOR_Show();
    GUI_CURSOR_Select(&GUI_CursorCrossL);
    GUI_SetBkColor(GUI_WHITE);
    GUI_SetColor(GUI_BLACK);
    GUI_Clear();
    GUI_DispString("Measurement of\nA/D converter values");

    while (1)
    {
        GUI_TOUCH_GetState(&TouchState);          /* 以像素为单位获取触摸位置 */
        xPhys = GUI_TOUCH_GetxPhys();              /* 在 x 中得到 AD 测量结果 */
        yPhys = GUI_TOUCH_GetyPhys();              /* 在 y 中得到 AD 测量结果 */
        GUI_SetColor(GUI_BLUE);
        GUI_DispStringAt("Analog input:\n", 0, 40);
        GUI_GotoY(GUI_GetDispPosY() + 2);
        GUI_DispString("x:");
        GUI_DispDec(xPhys, 4);
        GUI_DispString(", y:");
        GUI_DispDec(yPhys, 4);
        GUI_SetColor(GUI_RED);
        GUI_GotoY(GUI_GetDispPosY() + 4);
        GUI_DispString("\nPosition:\n");
        GUI_GotoY(GUI_GetDispPosY() + 2);
    }
}

```

```
GUI_DispString("x:");
GUI_DispDec(TouchState.x,4);
GUI_DispString(", y:");
GUI_DispDec(TouchState.y,4);
delay_ms(50);
}
}
```

当点击触摸屏的时候，此函数将点击处的触摸屏 AD 值显示在 LCD 上，程序执行结果如下图所示，屏幕为竖屏，其中蓝色为原始的 AD 值，红色的为屏幕坐标值。

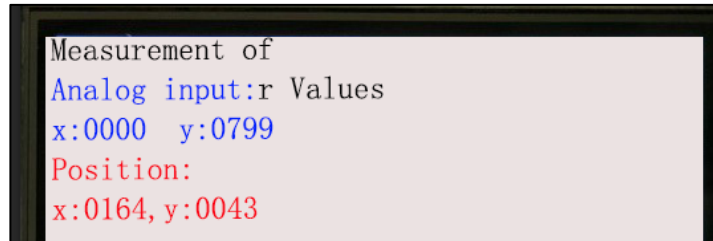


图 1.4.2 触摸屏 AD 值获取

最后修改 LCDConf.c 文件中的 LCD_X_Config 函数，修改后的函数如下所示：

```
void LCD_X_Config(void)
{
    /* 创建显示驱动器件 */
    GUI_DEVICE_CreateAndLink(&GUIDRV_Template_API, GUICC_M565, 0, 0);
    LCD_SetSizeEx(0, lcddev.width, lcddev.height);
    LCD_SetVSizeEx(0, lcddev.width, lcddev.height);
    /* 判断电阻屏还是电容屏 */
    if ((tp_dev.touchtype & 0x80) == 0)
    {
        /* 其他屏幕全部默认为 2.8 寸 320X240 */
        if (lcddev.dir == 0) /* 竖屏 */
        {
            GUI_TOUCH_SetOrientation(GUI_MIRROR_X);
            GUI_TOUCH_Calibrate(GUI_COORD_X, 0, lcddev.width, 155, 3903);
            GUI_TOUCH_Calibrate(GUI_COORD_Y, 0, lcddev.height, 188, 3935);
        }
        else /* 横屏 */
        {
            GUI_TOUCH_SetOrientation(GUI_SWAP_XY);
            GUI_TOUCH_Calibrate(GUI_COORD_X, 0, lcddev.width, 188, 3903);
            GUI_TOUCH_Calibrate(GUI_COORD_Y, 0, lcddev.height, 155, 3935);
        }
    }
    else /* 电容屏 */
    {
        if (lcddev.id == 0X1963) /* 1963 为 7 寸屏 800*480 */
        {
            /* 6368 3816 */
            /* GUI_TOUCH_SetOrientation(GUI_SWAP_XY|GUI_MIRROR_Y); */
            GUI_TOUCH_Calibrate(GUI_COORD_X, 0, 800, 0, 799);
            GUI_TOUCH_Calibrate(GUI_COORD_Y, 0, 480, 0, 479);
        } /* 0X5510 0X6804 为 3.5 寸 320x480 */
        else if (lcddev.id == 0X5310 || lcddev.id == 0X6804)
        {
            if (lcddev.dir == 0) /* 竖屏 */
            {
                GUI_TOUCH_SetOrientation(GUI_MIRROR_X);
            }
            else /* 横屏 */
            {
                GUI_TOUCH_SetOrientation(GUI_SWAP_XY);
            }
            GUI_TOUCH_Calibrate(GUI_COORD_X, 0, lcddev.width, 3931, 226);
            GUI_TOUCH_Calibrate(GUI_COORD_Y, 0, lcddev.height, 3812, 196);
        }
    }
}
```

```

    }
    else
    {
        GUI_TOUCH_Calibrate(GUI_COORD_X,0,lcddev.width,0,lcddev.width-1);
        GUI_TOUCH_Calibrate(GUI_COORD_Y,0,lcddev.height,0,lcddev.height-1);
    }
}

```

上述源码中，电容屏并没有使用上面测量四个 AD 值。因为电容屏直接读出来的点坐标就是 LCD 屏幕上相对应的点坐标，不需要校准，但是 GUI_TOUCH_Calibrate 函数的第 4,5 两个参数还是要写的，这两个参数本来是需要测量的，这里笔者就直接根据屏幕的大小写进去，如果屏幕 X 轴大小为 480，Y 轴大小为 800，则该函数 GUI_TOUCH_Calibrate 的参数就按照如下程序配置，如下所示：

```

GUI_TOUCH_Calibrate(GUI_COORD_X,0,480,0,479);
GUI_TOUCH_Calibrate(GUI_COORD_Y,0,800,0,799);

```

至此，触摸屏部分移植完成。

2.5 综合测试程序编写

emWin 移植完成以后，可移植 emWin 的官方 Demo 来测试移植是否成功，这个官方的 Demo 需要操作系统提供系统时钟，本章移植并没有使用操作系统，这里使用一个定时器来提供这个系统时钟，官方 Demo 中还需要定时调用 GUI_TOUCH_Exec 函数来定时处理触摸屏事件，定时器驱动的中断服务函数代码如下：

```

extern __IO int32_t OS_TimeMS;

void BTIM_TIMX_INT_IRQHandler(void)
{
    HAL_TIM_IRQHandler(&timx_handler); /*定时器回调函数*/
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim == (&timx_handler))
    {
        count_time++;
        OS_TimeMS++;
        if(count_time == 10)
        {
            GUI_TOUCH_Exec(); /* 每 10ms 调用一次，触发调用触摸驱动 */
            count_time = 0;
        }
    }
}

```

从定时器（bitm.c）的中断服务程序中可以看出，通过对 OS_TimeMS 这个全局变量加一来提供系统时钟，而 GUI_TOUCH_Exec 函数每 10ms 调用一次。

接下来将 emWin 软件包目录下 Sample\Application\GUIDemo 文件夹下的文件移植到工程当中，这个移植非常简单，只要将这个 GUIDemo 文件夹下的文件拷贝到工程的 Middlewares\emWin\emWin_Demo 路径下，并将这些文件添加到工程的 Middlewares/emWin/emWin_Demo 分组当中，添加完成后的分组如下图所示：

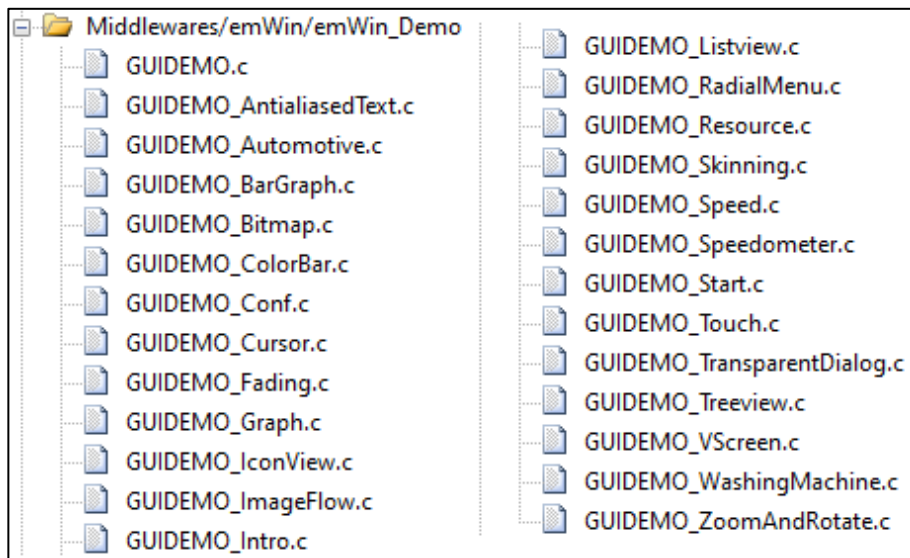


图 2.5.1 添加 Demo

注意：上图中的文件都来自于 emWin\Sample\Application\GUIDemo 文件夹下。

接下来编写 main 函数，在 main 函数中首先完成相应外设的初始化，接着初始化 emWin，初始化完成后调用 GUIDEMO_Main 函数运行官方 demo。

```
int main(void)
{
    /*****
    /* 硬件初始化 */
    *****/

    lcd_init();                /* 初始化 LCD */
    tp_dev.init();             /* 触摸屏初始化 */
    btim_timx_int_init(XXX,XXX); /* 定时 1ms 用来提供 emWin 心跳 */
    __HAL_RCC_CRC_CLK_ENABLE(); /* 使能 CRC 时钟 */
    GUI_Init();                /* emWin 初始化 */
    GUIDEMO_Main();
    while(1);
}
```

为了流畅的使用 emWin，可修改启动文件的栈大小，根据不同型号的主控芯片，指定修改启动文件的栈大小，具体的对应关系如下：

正点原子的 STM32 系列开发板类型	启动文件
STM32F1	startup_stm32f103xe.s
STM32F4	startup_stm32f407xx.s/startup_stm32f429xx.s
STM32F7	startup_stm32f767xx.s/startup_stm32f750xx.s
STM32H7	startup_stm32H743xx.s/startup_stm32H750xx.s

表 2.5.1 不同类型开发板与启动文件的对应关系

根据上表的内容，我们可在对应的启动文件修改栈大小，下面笔者以 CM3 微处理器内核为例，如下图所示：

```
Stack_Size EQU 0x00001000;如果使用STemWin的话这个值设置大一点
... AREA STACK, NOINIT, READWRITE, ALIGN=3
Stack_Mem SPACE Stack_Size
__initial_sp
```

图 2.5.2 修改后的启动文件栈大小

2.6 下载验证

至此，无操作系统的 emWin 移植完成，编译下载程序，程序运行如下图所示：

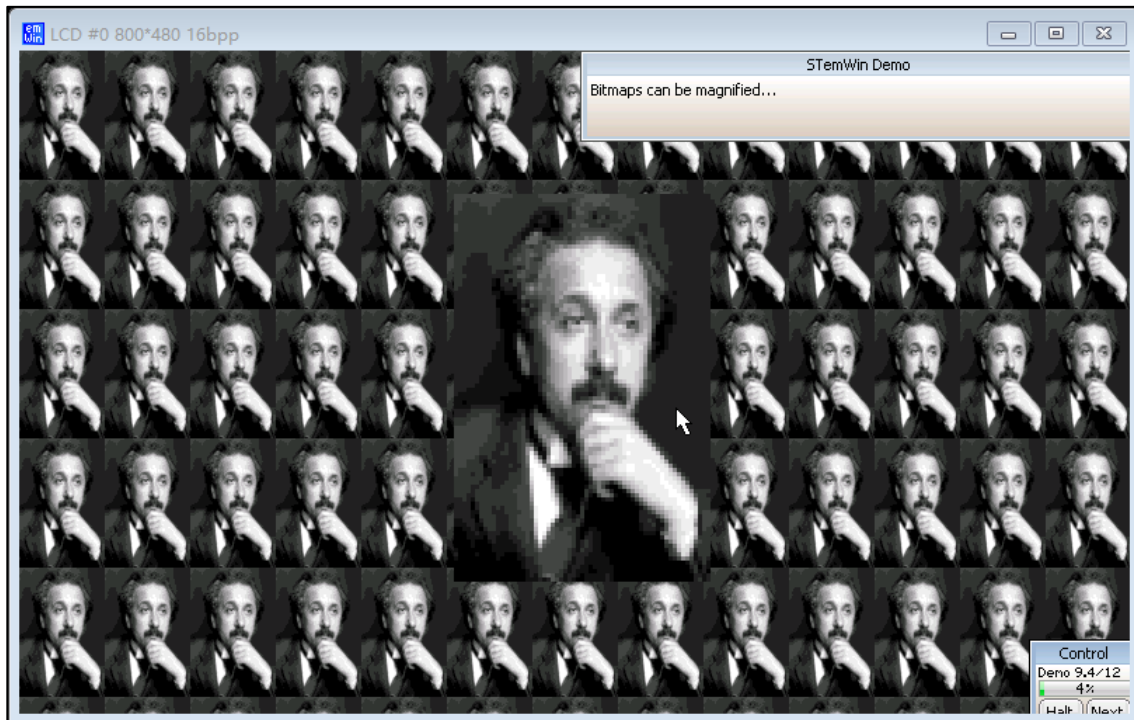


图 2.6.1 emWin 测试

2.7 RGB 接口屏移植

注：本小节的内容只适合阿波罗，北极星和水星开发板。

上面章节中的移植方法可以适用于任何液晶屏，只要提供打点，读点和填充函数，当然肯定也包括了正点原子的所有 RGB 接口的液晶屏。MCU 接口的液晶屏只能使用这种移植方法，但是 RGB 屏如果也使用这种方法的话虽然可以使用，但是完全没有发挥出 DMA2D 和 LTDC 的性能，两者的区别就是能用和好用之间的区别！所以在使用 RGB 接口 LCD 的时候需要专门针对 RGB 接口屏来移植 emWin，我们不用担心不熟悉 DMA2D，不知道如何发挥 MCU 在图形这一块的性能，因为 ST 已经给我们提供了一个它们官方移植好的例程，我们只需要将这个例程移植到我们自己的板子上就可以了，实际上用到的只有一个.c 文件。RGB 接口屏移植流程如下：

◇ 创建新 Project 工程

为了方便，我们在上小节移植工程的基础上创建一个工程来移植 RGB 屏版本的 emWin，点击图标 ，进入 Manage Project Items 对话框，在 Project Targets 栏重新创建一个 Project 并命名为“emWin (RGLCD)”，如下图所示：

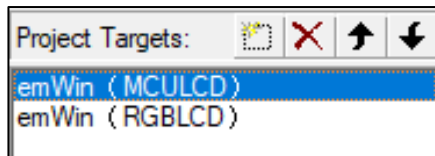


图 2.7.1 新建 Project 工程

通过上面的步骤我们就在一个工程中建立了两个 Project，这样做有什么好处呢？有时候我们在写程序的时候好几个工程中大多数文件都是通用的，只是其中一两个不同，如果每个都创建一个工程的话管理起来就很繁琐，相反，如果我们只创建一个工程，然后创建不同的 Project 来区分以前不同的工程，这样它们之间相同的文件就可以通用，各个 Project 管理自己独有的文件就可以了。移植 emWin 就是这种情况，上小节和本节之间的差别也仅仅是 2 个文件，所以在一个工程中创建两个 Project 是很好的解决方法。创建完成之后，我们怎么选择不同的 Project 工程呢？如下图所示那样，点击下拉列表的选项就可以选择不同的 Project 工程。

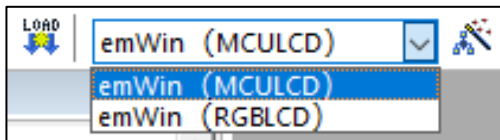


图 2.7.2 选择不同的 Project 工程

我们选择 emWin (RGBLCD) 工程，发现原工程没有什么变化，这是因为默认新建的 Project 和工程中原来存在的 Project 是相同的。我们接下来需要设置哪些文件不包含在新的 Project 工程中，比如在针对 RGB 屏的 emWin 移植中我们不需要 GUIDRV_Template.c 和 LCDConf.c 这两个文件，以 GUIDRV_Template.c 为例，如下图所示打开文件设置对话框：

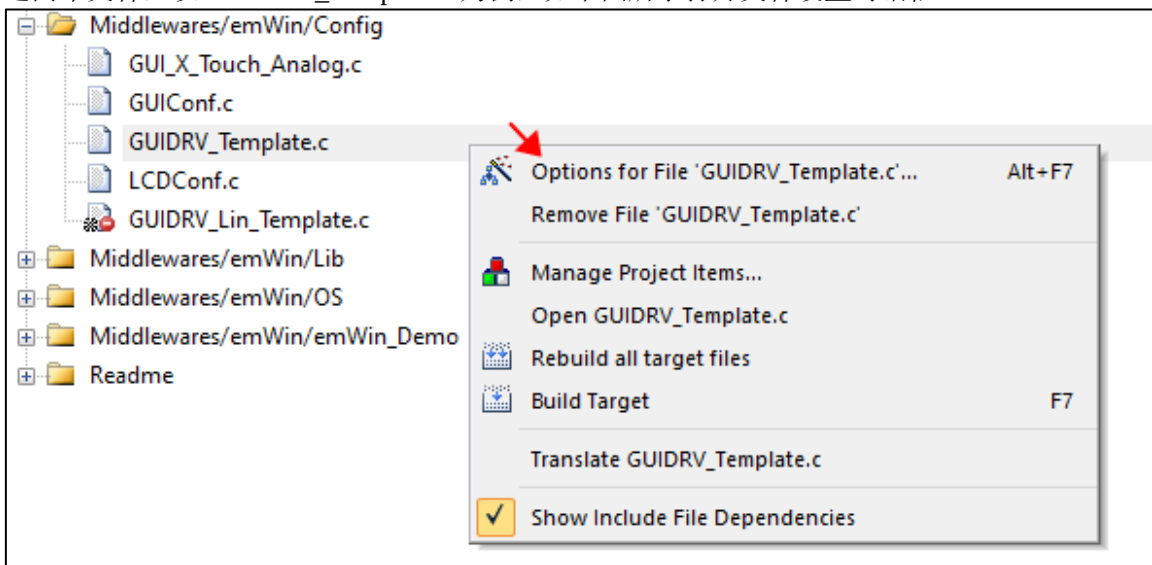


图 2.7.3 打开文件设置对话框

点击上图中的“Options for File ‘GUIDRV_Template.c’...”进入文件属性设置对话框，然后如下图那样不勾选“Include in Target Build”，表示工程编译时候此文件不会在编译器中编译，同样的方法设置文件 LCDConf.c 的文件属性。

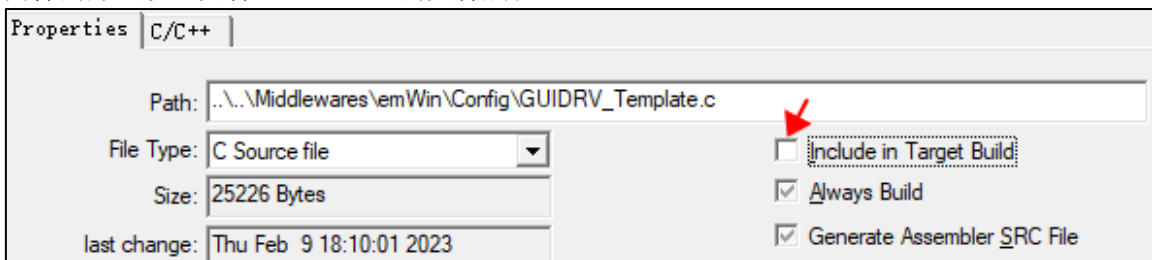


图 2.7.4 取消文件编译

✧ 添加 RGB 接口处理文件

上小节中，我们取消了 GUIDRV_Template.c 和 LCDConf.c 这两个文件的编译，那么这两个文件实现的功能由用户提供，如 emWin 的显示驱动和触摸校正等功能。这些功能怎么编写呢？ST 官方已经为用户考虑了这个问题，因此提供了 RGB 接口处理文件，实现了 emWin 显示驱动和触摸校正等功能，其中 emWin 显示驱动由 DMA2D 和 LTDC 组合编写。

打开 ST 官方网站，如下图所示那样下载 Cube 包，下载地址为：<https://www.st.com/zh/embedded-software/stm32cubef4.html>。

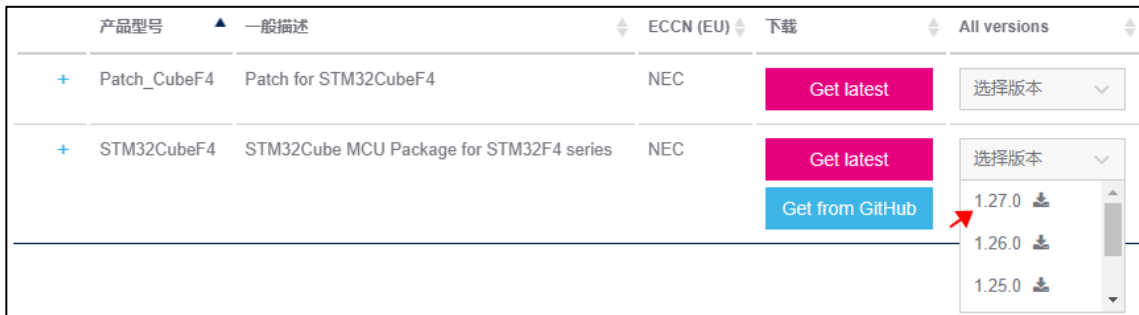


图 2.7.5 下载 F4 的 Cube 包

下载完成之后，打开 Cube 里面的 STemWin 例程，该例程的路径为：STM32Cube_FW_F4_V1.27.0\Projects\STM32F429I-Discovery\Applications\STemWin\STemWin_SampleDemo。这个工程就是 ST 官方移植好的 STemWin 的工程，这个工程里面有我们需要的针对 RGB 屏的移植文件，打开 STemWin_SampleDemo\STemWin\Target 路径，里面包含了 LCDConf_stm32f429i_disco_MB1075.c/h。这个文件就是我们需要的，将其复制到我们的移植工程，放到.\Middleware_s\emWin\Config 文件夹里面，并将其改名为 GUIDRV_Lin_Template.c/h，如下图所示：



图 2.7.6 RGB 接口处理文件

将上图中的 GUIDRV_Lin_Template.c 添加到 Middlewares/emWin/Config 分组当中，如下图所示：

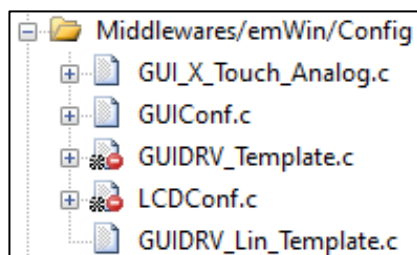


图 2.7.7 将 GUIDRV_Lin_Template.c 添加到分组

注：文件 GUIDRV_Lin_Template.c 是针对 RGB 显示屏的，因此在针对 MCU 屏的 Project 工程中需要将其“开除”在外，让编译器在 MCU 屏的 Project 工程中编译此文件，设置方法和上面一样。

✧ 修改 RGB 接口处理文件

ST 官方提供的 GUIDRV_Lin_Template.c 文件是针对某一个 RGB 显示屏，那么我们怎么把 GUIDRV_Lin_Template.c 文件适配正点原子多款 RGB 显示屏呢？下面我们对 GUIDRV_Lin_Template.c 文件进行修改，如下所示：

✓ 配置信息

```
#define XSIZE_PHYS 800 /* 最大支持 1280 * 800 */
```

```
#define YSIZE_PHYS            1280

#define NUM_BUFFERS            3            /* 使用多缓冲时的缓冲数量 */
#define NUM_VSCREENS          1            /* 使用虚拟屏幕是的虚拟屏幕数量 */

#define BK_COLOR GUI_DARKBLUE

#undef  GUI_NUM_LAYERS
#define GUI_NUM_LAYERS        1

#define COLOR_CONVERSION_0     GUICC_M565    /* 正点原子的 RGB 显示屏是 RGB565 */
#define DISPLAY_DRIVER_0       GUIDRV_LIN_16 /* 显示驱动器颜色深度 */
```

上面的宏定义了所使用的 RGB 屏幕的大小，这里根据自己实际所使用的屏幕大小来设置，接着就是多缓冲和虚拟屏幕的数量，宏 GUI_NUM_LAYERS 用来设定显示层数，由于 LTDC 支持双层显示，因此宏 GUI_NUM_LAYERS 最大就是 2，但是这里我们设置为 1，关于 emWin 的双层显示我们后面会有专门的讲解。宏 COLOR_CONVERSION_0 和 DISPLAY_DRIVER_0 用来设置第 0 层的颜色格式和驱动方式，我们平时默认使用的就是第 0 层，COLOR_CONVERSION_1 和 DISPLAY_DRIVER_1 和是第 1 层的颜色格式和现实驱动。

在第 0 层中我们支持 GUICC_M8888I、GUICC_M888、GUICC_M565、GUICC_M1555I 和 GUICC_M4444I 这种颜色格式，虽然 ST 官方移植的工程中还支持 GUICC_8666、GUICC_1616 I 和 GUICC_88666I 但是我在用 ST 官方的板子测试的时候发现使用这三中颜色格式的时候都会有颜色失真的情况，因此在我们的移植工程中将这三种颜色格式去掉了。我们在使用两层的时候通常都会用到 Alpha 通道，也就是透明功能，在 emWin 中，第 1 层只有 GUICC_M1555I 和 GUICC_8666_1 这两种颜色格式支持 Alpha，而 F767/F429/H743/H750/F750 又不支持 GUICC_8666_1，因此第 1 层只有 GUICC_M1555I 这一种颜色格式带 Alpha 了。接下来就是设置 RGB 屏两个层的缓冲区了，代码如下：

```
/* LCD 两层的缓冲区需要根据实际所使用的屏幕分辨率以及每层所使用的颜色格式来计算，
 * 最终确定最佳的缓冲区大小，这里默认设置最大分辨率为 1024*600，颜色格式为 8888，
 * 那么每层所需要的缓冲区就是 1280*800*4=4096000=0X3E8000，注意 mllloc.h 和 malloc.c
 * 中的外部 SDRAM 内存管理池地址和大小也需要做修改！！！！ */

#define LCD_LAYER0_FRAME_BUFFER ((uint32_t)0xC0000000) /* 第一层缓冲区 */
#define LCD_LAYER1_FRAME_BUFFER ((uint32_t)0xC0400000) /* 第二层缓冲区 */
```

添加 LTDC 中断服务函数，代码如下：

```
/**
 * @brief      LTDC 中断服务函数
 * @param      无
 * @retval     无
 */
void LTDC_IRQHandler(void)
{
    HAL_LTDC_IRQHandler(&g_ltdc_handle);
}
```

接下来就是重点修改函数 LCD_X_Config()，修改后的函数如下：

```
void LCD_X_Config(void)
{
    uint32_t i;

    LCD_LL_Init();

    /* .....省略部分代码..... */

    /* 设置第一层 */
    GUI_DEVICE_CreateAndLink(DISPLAY_DRIVER_0,COLOR_CONVERSION_0,0,0);
    LCD_GetSwapXYEx(0); /* 选中第 0 层 */

    if (lcddev.dir == 0) /* 竖屏 */
    {
```



```

        LCD_SetSizeEx(0, lcddev.height, lcddev.width);          /* 设置可见区尺寸 */
        LCD_SetVSizeEx(0, lcddev.height, lcddev.width * NUM_VSCREENS);
        GUI_SetOrientation(GUI_SWAP_XY|GUI_MIRROR_Y);           /* 设置为竖屏 */
    }
    else                                                         /* 横屏 */
    {
        LCD_SetSizeEx(0, lcddev.width, lcddev.height);          /* 设置可见区尺寸 */
        LCD_SetVSizeEx(0, lcddev.width, lcddev.height * NUM_VSCREENS);

        GUI_TOUCH_Calibrate(GUI_COORD_X, 0, lcddev.width, 0, lcddev.width - 1);
        GUI_TOUCH_Calibrate(GUI_COORD_Y, 0, lcddev.height, 0, lcddev.height - 1);

#ifdef GUI_NUM_LAYERS > 1
        /* 设置第二层 */
        GUI_DEVICE_CreateAndLink(DISPLAY_DRIVER_1, COLOR_CONVERSION_1, 0, 1);
        LCD_GetSwapXYEx(1);                                       /* 选中第 1 层 */

        if(lcddev.dir==0)                                         /* 竖屏 */
        {
            LCD_SetSizeEx(1, lcddev.height, lcddev.width);      /* 设置可见区尺寸 */
            LCD_SetVSizeEx(1, lcddev.height, lcddev.width * NUM_VSCREENS);
            GUI_SetOrientation(GUI_SWAP_XY|GUI_MIRROR_Y);         /* 设置为竖屏 */
        }
        else                                                      /* 横屏 */
        {
            LCD_SetSizeEx(1, lcddev.width, lcddev.height);       /* 设置可见区尺寸 */
            LCD_SetVSizeEx(1, lcddev.width, lcddev.height * NUM_VSCREENS);

            GUI_TOUCH_Calibrate(GUI_COORD_X, 0, lcddev.width, 0, lcddev.width - 1);
            GUI_TOUCH_Calibrate(GUI_COORD_Y, 0, lcddev.height, 0, lcddev.height - 1);
#endif
        /* .....省略部分代码..... */
    }
}

```

此函数一开始设置了每一层所使用的驱动、颜色格式、可见区尺寸和虚拟显示区大小，最后调用用户自定义的一些函数替代了 emWin 中的默认函数，这些用户自定义的函数就是 ST 针对 CM4、CM7 的图形性能所编写的，用来提高 emWin 在 RGB 接口屏幕上的显示性能。

另一个重要的函数 LCD_X_DisplayDriver 我们使用默认的 ST 编写好的默认函数就行了。

删除掉 HAL_LTDC_MspInit 和 HAL_LTDC_MspDeInit 这两个函数，因为 ST 官方移植的文件中还要初始化 RGB 屏幕的，但是我们会使用 ltdc_init 函数初始化 RGB 显示屏即可，所以这两个函数我们不需要。

下面就是对 LCD_LL_Init 函数进行修改。代码修改如下：

```

static void LCD_LL_Init(void)
{
    /* LTDC 中断，抢占优先级 1，子优先级 1 */
    HAL_NVIC_SetPriority(LTDC_IRQn, 1, 1);
    HAL_NVIC_EnableIRQ(LTDC_IRQn);
    HAL_LTDC_ProgramLineEvent(&g_ltdc_handle, 0);               /* 开启 LTDC 的行中断 */

    /* Configure the DMA2D default mode */
    g_dma2d_handle.Init.Mode          = DMA2D_R2M;
    g_dma2d_handle.Init.ColorMode     = DMA2D_RGB565;
    g_dma2d_handle.Init.OutputOffset = 0x0;

    g_dma2d_handle.Instance           = DMA2D;

    if (HAL_DMA2D_Init(&g_dma2d_handle) != HAL_OK)
    {
        while (1);
    }
}

```

```
ltcdc_clk_set(320,2,RCC_PLLSAIDIVR_4);          /* 设置像素时钟 40Mhz */
}
```

此函数就是对 DMA2D 进行初始化，并且开启 DMA2D 外设。

修改 LCD_LL_LayerInit 函数，兼容多款 RGB 显示屏，代码修改如下：

```
static void LCD_LL_LayerInit(uint32_t LayerIndex)
{
    /* 省略..... */

    if (LayerIndex < GUI_NUM_LAYERS)
    {
        /* 省略..... */
        layer_cfg.WindowX0 = 0;
        layer_cfg.WindowX1 = lcdltcdc.pwidth;      /* 设置显示屏宽度 */
        layer_cfg.WindowY0 = 0;
        layer_cfg.WindowY1 = lcdltcdc.pheight;     /* 设置显示屏高度 */
        layer_cfg.PixelFormat = LCD_LL_GetPixelFormat(LayerIndex);
        layer_cfg.FBStartAdress = layer_prop[LayerIndex].address;
        layer_cfg.Alpha = 255;
        layer_cfg.Alpha0 = 0;
        layer_cfg.Backcolor.Blue = 0;
        layer_cfg.Backcolor.Green = 0;
        layer_cfg.Backcolor.Red = 0;
        layer_cfg.BlendingFactor1 = LTDC_BLENDING_FACTOR1_PAxCA;
        layer_cfg.BlendingFactor2 = LTDC_BLENDING_FACTOR2_PAxCA;
        layer_cfg.ImageWidth = lcdltcdc.pwidth;    /* 设置显示屏宽度 */
        layer_cfg.ImageHeight = lcdltcdc.pheight;  /* 设置显示屏高度 */
        HAL_LTDC_ConfigLayer(&g_ltdc_handle, &layer_cfg, LayerIndex);
        /* 省略..... */
    }

    /* 省略..... */
}
```

注：上述添加注释的代码需要我们修改，使用 lcdltcdc 结构体的字段描述显示屏的宽度和高度，其他设置保持不变。

✧ 修改 malloc.c/h 文件

由于前面我们分配了 3M 给了 RGB 屏第 0 层做缓冲，同时还分配 3M 给了第 1 层做缓冲，所以归内存管理的 SDRAM 起始地址设置为 0XC0600000。

```
malloc.c
/* 外部 SDRAM 内存池,前面 2M 给 LTDC, 6M 给了 RGB 屏第一和第二层做缓冲(1280*800*2) */
static __align(32) uint8_t mem3base[MEM3_MAX_SIZE]
    __attribute__((at(0XC0600000)));

/* 外部 SRAM 内存池 MAP */
static MT_TYPE mem3mapbase[MEM3_ALLOC_TABLE_SIZE]
    __attribute__((at(0XC0600000 + MEM3_MAX_SIZE)));

malloc.h
/* mem3 内存参数设定.mem2 的内存池处于外部 SDRAM 里面 */
#define MEM3_BLOCK_SIZE          64          /* 内存块大小为 64 字节 */
#define MEM3_MAX_SIZE            20*1024*1024 /* 最大管理内存 20M */
#define MEM3_ALLOC_TABLE_SIZE    MEM3_MAX_SIZE/MEM3_BLOCK_SIZE /* 内存表大小 */
```

至此 RGB 屏基本就可以用了，如果还有什么问题的话就参考我们移植好的工程来修改，修改的时候重点注意一下 malloc.c、malloc.h 和 GUIDRV_Lin_Template.c 这三个文件。

2.8 使用 AC6 编译工程（扩展）

首先说明，本教程配套的实验工程全部采用 AC5 进行开发。

AC 是“ARM Compiler”的简称，是用于编译 ARM 处理器代码的编译工具链，AC 后面的 5 和 6 表示的是 AC 的版本号，在 Keil MDK 中集成了 AC5 和 AC6 编译工具链，这里简单地对比一下 AC5 和 AC6 的差异，如下表所示：

对比项	AC5	AC6	说明
中文支持	较好	较差	AC6 对中文支持极差，goto definition 无法使用，报错等
代码兼容性	较好	较差	AC6 对某些代码优化可能导致运行异常，需慢慢调试
编译速度	较慢	较快	AC6 编译速度比 AC5 快
语法检查	一般	严格	AC6 语法检查非常严格，代码严谨性较好

表 2.8.1 AC5&AC6 简单对比

虽然 AC6 的编译速度比较快，但是 AC5 在各方面的兼容性都比 AC6 要好，因此笔者推荐新手读者使用 AC5。当然读者可以自由地选择使用 AC5 或 AC6，正点原子的源码都是支持 AC6 的，只不过在 Keil MDK 的选项配置方面有些差异，具体差异如下表所示：

选项	AC5	AC6	说明
Target ->ARM Compiler	V5.xx	V6.xx	选择对应的编译工具

表 2.8.2 AC5&AC6 配置差异

通过以上配置正点原子的裸机程序已经能够正常使用 AC6 编译工具进行编译了，但是对于 FreeRTOS（emWin 最后一个例程用的是 FreeRTOS 移植 emWin）还需要稍作修改，需要修改四个地方，如下所示：

1. FreeRTOS 的 port.c 文件

在前面小节中，往工程添加 port.c 文件的路径为 FreeRTOS/portable/RVDS，这是在使用 AC5 编译工具的前提下，如果使用 AC6 编译工具，那么 port.c 文件的路径应该改为 FreeRTOS/portable/GCC，对于正点原子不同的 STM32 开发板，只要根据表 2.8.1 将对应的 port.c 文件添加到工程中即可。

2. 头文件路径

在前面小节中，添加了一个以 port.c 文件路径为路径的头文件路径，由于使用 AC6 编译工具时，使用了别的 port.c 文件，因此需要将原来以 port.c 文件路径为路径的头文件路径修改为新的 port.c 文件的路径为路径的头文件路径，修改后如下图所示（这里以正点原子的 STM32F4 系列开发板为例，其他类型的开发板类似）：

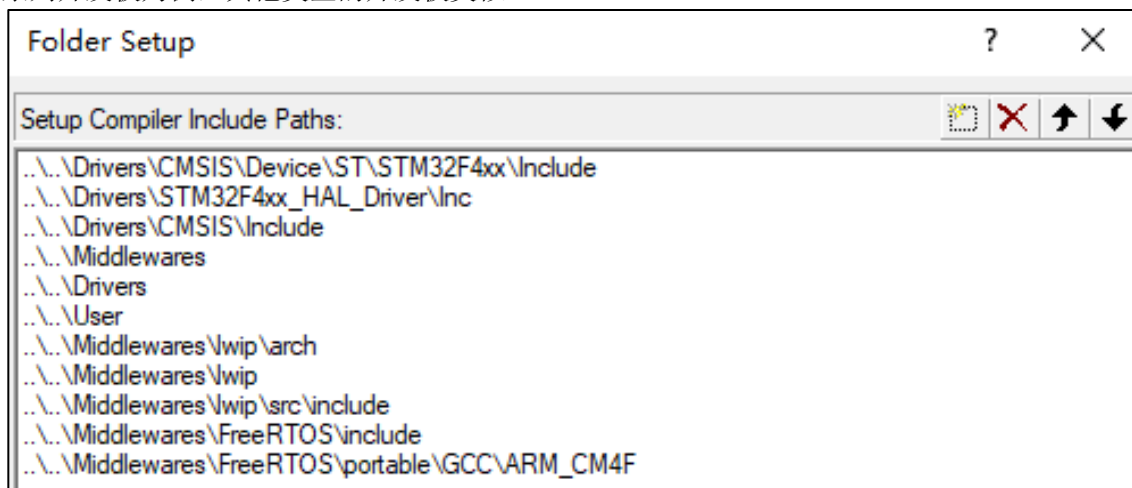


图 2.4.4.1 修改头文件路径

这里要注意的是，在切换了 Keil MDK 编译工具之后，最好完整地重新编译一遍工程，即 Rebuild all target files 而不是 Build Target，以免出现一些莫名的问题。

3. 设置 MiscControls

编译工程时，出现“.warning: illegal character encoding in string literal [-Winvalid-source-encoding]”警告，我们可在 MiscControls 选项中添加“-Wno-invalid-source-encoding”字段来屏蔽这个警告，如下图所示：

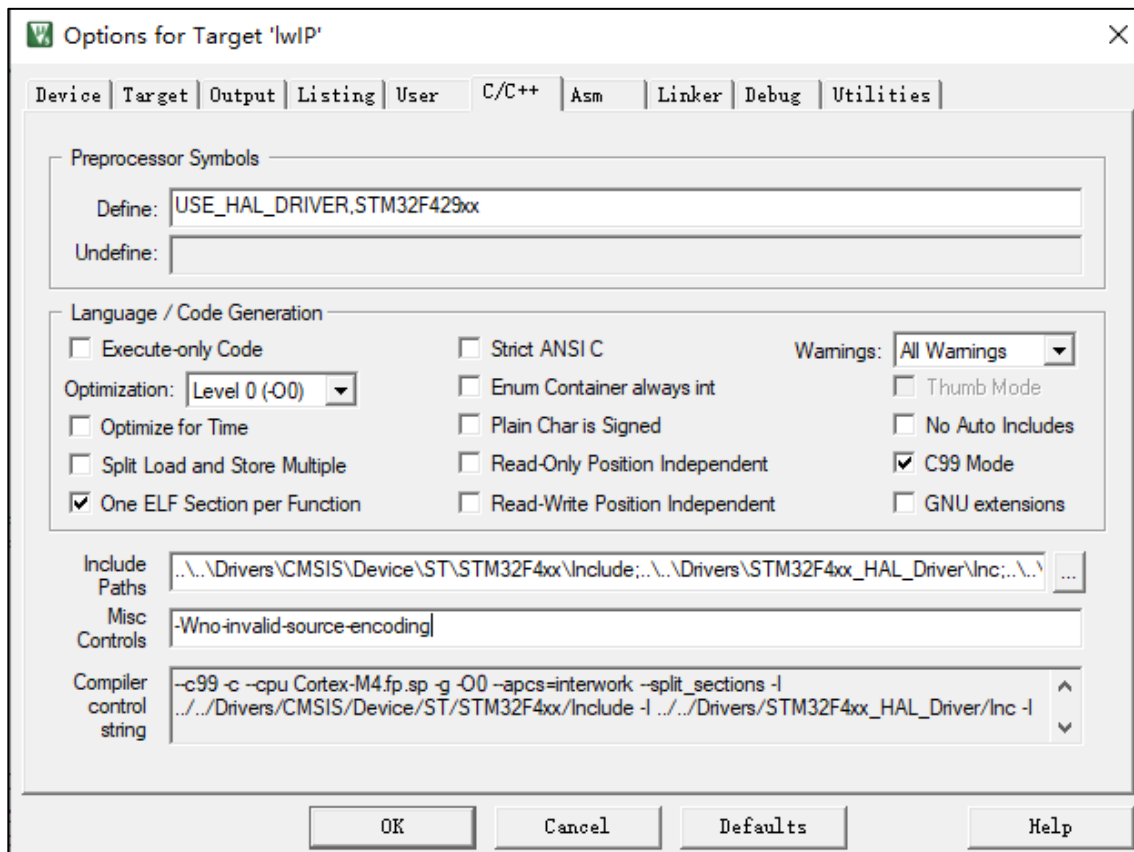


图 2.4.4.2 屏蔽警告

第三章 emWin 带操作系统移植

本章为大家讲解带操作系统的 emWin 移植，操作系统使用的是 uC-OS3，以后的例程中都是携带操作系统的 emWin 例程，至于 uC-OS3 的移植请参考正点原子《uC-OSIII 开发指南》教程，本章的移植例程是在上一章的移植例程上进行。

本章分为以下几个部分：

- 3.1 移植准备工作
- 3.2 向工程添加以及修改相应文件
- 3.3 综合测试程序编写
- 3.4 下载验证

3.1 移植准备工作

关于 uC-OS 操作系统的移植，请参考正点原子《uC-OSIII 开发指南》。注意：将 uC-OS3 的系统节拍 OS_CFG_TICK_RATE_HZ 配置项修改为 1000， $1/1000 = 0.001s$ 也就是 1ms，移植好 uC-OS3 后的工程分组，如下图所示：

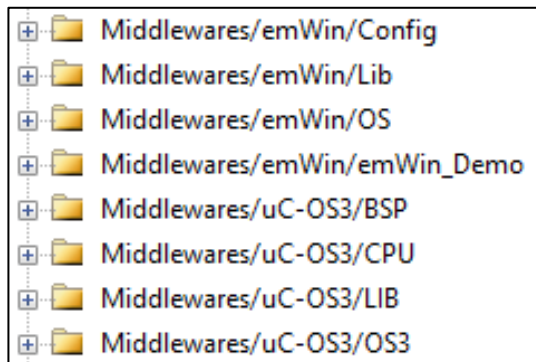


图 3.1.1 带 uC-OSIII 的工程分组

3.2 向工程添加以及修改相应文件

打开工程，把 Middlewares/emWin/OS 分组中的 GUI_X.c 文件移除，并添加 GUI_X_uCOS.c 文件，该文件是在 Middlewares\emWin\OS 路径下定义的，如图 3.2.1 所示：

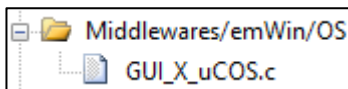


图 3.2.1 添加 GUI_X_uCOS.c 后的工程

注意：由于 GUI_X_uCOS.c 文件是 emWin 针对 uC-OSII 操作系统编写的配置文件，所以我们根据该文件的特性修改成 uC-OSIII 操作系统的配置文件，GUI_X_uCOS.c 文件修改步骤如下：

(1) 时间相关函数

```
int GUI_X_GetTime(void)
{
    OS_ERR err;
    return ((int)OSTimeGet(&err)); /* 获取系统时间,此处时间单元为 1ms */
}

/* GUI 延时函数 */
void GUI_X_Delay(int period)
{
    OS_ERR err;
    CPU_INT32U ticks;
```

```
ticks = (period * 1000) / OSCfg_TickRate_Hz;
OSTimeDly(ticks, OS_OPT_TIME_PERIODIC, &err); /* UCOSIII 延时采用周期模式 */
}
```

这两个函数分别提供 emWin 的时基和 emWin 内部延时函数。

(2) 信号量相关函数

```
void GUI_X_InitOS(void)
{
    OS_ERR err;
    /*创建初始值为 1 的信号量, 用于共享资源 */
    OSSemCreate (    (OS_SEM *          ) &DispSem,
                     (CPU_CHAR *       ) "Disp_SEM",
                     (OS_SEM_CTR       ) 1,
                     (OS_ERR *         ) &err);

    /* 创建初始值为 1 的信号量, 用于事件触发 */
    OSSemCreate (    (OS_SEM *          ) &EventSem,
                     (CPU_CHAR *       ) "Event_SEM",
                     (OS_SEM_CTR       ) 0,
                     (OS_ERR *         ) &err);
}

/*等待信号量 */
void GUI_X_Lock(void)
{
    OS_ERR err;
    OSSemPend(&DispSem, 0, OS_OPT_PEND_BLOCKING, 0, &err); /* 请求信号量 */
}

/*发送信号量 */
void GUI_X_Unlock(void)
{
    OS_ERR err;
    OSSemPost(&DispSem, OS_OPT_POST_1, &err); /* 发送信号量 */
}

/* 放回任务 ID, 此处返回的是任务优先级, 由于 UCOSIII 支持时间片
 * 轮转调度, 因此如果使用了时间片轮转调度功能的话有可能会出错! */
U32 GUI_X_GetTaskId(void)
{
    return ((U32) (OSTCBCurPtr->Prio)); /* 获取任务优先级, 也就是任务 ID */
}

void GUI_X_WaitEvent(void)
{
    OS_ERR err;
    OSSemPend(&EventSem, 0, OS_OPT_PEND_BLOCKING, 0, &err); /* 请求信号量 */
}

void GUI_X_SignalEvent(void)
{
    OS_ERR err;
    OSSemPost(&EventSem, OS_OPT_POST_1, &err); /* 发送信号量 */
}
```

这些函数分别创建信号量、上锁、解锁等操作, 这些信号量主要实现数据同步作用。

(3) 与键盘相关函数

下面这几个函数与键盘有关, 这几个函数很好理解。

```
static void CheckInit(void)
{
    if (KeyIsInitd == DEF_FALSE)
    {
        KeyIsInitd = DEF_TRUE;
        GUI_X_Init();
    }
}
```

```

void GUI_X_Init(void)
{
    OS_ERR err;
    /* 创建初始值为 1 的信号量 */
    OS_SemCreate (    (OS_SEM *           ) &KeySem,
                      (CPU_CHAR *        ) "Key_SEM",
                      (OS_SEM_CTR        ) 0,
                      (OS_ERR *          ) &err);
}

int GUI_X_GetKey(void)
{
    int r;

    r = KeyPressed;
    CheckInit();
    KeyPressed = 0;
    return (r);
}

int GUI_X_WaitKey(void)
{
    int r;
    OS_ERR err;
    CheckInit();

    if (KeyPressed == 0)
    {
        OSSemPend(&KeySem, 0, OS_OPT_PEND_BLOCKING, 0, &err); /* 请求信号量 */
    }

    r = KeyPressed;
    KeyPressed = 0;
    return (r);
}

void GUI_X_StoreKey(int k)
{
    OS_ERR err;
    KeyPressed = k;
    OSSemPost(&KeySem, OS_OPT_POST_1, &err); /* 发送信号量 */
}
    
```

上述源码可知，创建一个 KeySem 信号量，该信号量主要也是实现数据同步操作。

(4) 修改 GUI_Conf.c 文件

把该文件中宏定义 GUI_OS 设置为 1 表示支持操作系统，如下源码所示：

```

#ifndef GUICONF_H
#define GUICONF_H

/*****
 *
 *      Multi layer/display support
 */
#define GUI_NUM_LAYERS          1                /* 显示的最大层数 */

/*****
 *
 *      Multi tasking support
 */
#define GUI_OS                  (1)              /* 0:不使用操作系统 */

/*****
 *
 *      Configuration of touch support
 */
    
```

```
#define GUI_SUPPORT_TOUCH            (1)                /* 0:不支持触摸 */

/*****
 *
 *      Default font
 */
#define GUI_DEFAULT_FONT              &GUI_Font6x8      /* 默认字体 */

/*****
 *
 *      Configuration of available packages
 */
#define GUI_SUPPORT_MOUSE             1                  /* 支持鼠标 */
#define GUI_WINSUPPORT                1                  /* 支持窗口管理 */
#define GUI_SUPPORT_MEMDEV            1                  /* 支持存储设备 */
#define GUI_SUPPORT_DEVICES           1                  /* 使用设备指针 */

#endif // Avoid multiple inclusion
```

3.3 添加应用程序

移植好 emWin 之后，当然要测试一下移植是否成功。在本步骤中，一共需要修改 1 个文件并添加 4 个文件，修改的 1 个文件为 main.c，添加的 4 个文件为 emwin_demo.c、emwin_demo.h、uc-os3_demo.c 和 uc-os3_demo.h。对于 main.c 主要是在 main 函数中完成一些硬件的初始化，最后调用 uc-os3_demo.c 文件中的 uc_os3_demo 函数。uc-os3_demo.c 则是用于编写 uC-OS3 的相关应用程序代码。而 emwin_demo.c 则是用于编写 emWin 的相关应用程序代码。

注意：uc-os3_demo.c、uc-os3_demo.h 文件保存在 User 文件夹下，而 emwin_demo.c、emwin_demo.h 文件保存在 Middlewares\emWin\emWin_Demo 文件夹下。

1. main.c

由于正点原子的 STM32 系列开发板众多，这里以正点原子战舰开发板为例为读者进行演示，读者可以根据自己所移植的目标开发板在本教程的配套例程源码的《emWin 实验 3 文本显示》中查看对应的 main.c 文件。正点原子战舰开发板《emWin 实验 3 文本显示》中的 main.c 文件如下所示：

```
/**
 ****
 * @file      main.c
 * @author    正点原子团队 (ALIENTEK)
 * @version   V1.0
 * @date      2020-04-28
 * @brief     emwin 实验
 * @license   Copyright (c) 2020-2032, 广州市星翼电子科技有限公司
 ****
 * @attention
 *
 * 实验平台:正点原子 STM32F103 开发板
 * 在线视频:www.yuanzige.com
 * 技术论坛:www.openedv.com
 * 公司网址:www.alientek.com
 * 购买地址:openedv.taobao.com
 *
 ****
 */

#include "./SYSTEM/sys/sys.h"
#include "./SYSTEM/usart/usart.h"
#include "./SYSTEM/delay/delay.h"
#include "./USMART/usmart.h"
#include "./BSP/LED/led.h"
#include "./BSP/LCD/lcd.h"
```

```
#include "../BSP/KEY/key.h"
#include "../BSP/SRAM/sram.h"
#include "../MALLOC/malloc.h"
#include "../BSP/TIMER/btim.h"
#include "../BSP/TOUCH/touch.h"
#include "emwin_demo.h"

int main(void)
{
    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(RCC_PLL_MUL9); /* 设置时钟, 72Mhz */
    delay_init(72); /* 延时初始化 */
    usart_init(115200); /* 串口初始化为 115200 */
    usmart_dev.init(72); /* 初始化 USMART */
    led_init(); /* 初始化 LED */
    lcd_init(); /* 初始化 LCD */
    key_init(); /* 初始化按键 */
    sram_init(); /* SRAM 初始化 */
    my_mem_init(SRAMIN); /* 初始化内部 SRAM 内存池 */
    my_mem_init(SRAMEX); /* 初始化外部 SRAM 内存池 */
    tp_dev.init(); /* 触摸屏初始化 */
    uc_os3_demo(); /* 创建任务, 并执行 emwin 例程 */

    while (1);
}
```

可以看到, 在 main.c 文件中只包含了一个 main 函数, main 函数主要就是完成了一些外设的初始化, 如串口、LED、LCD、按键等, 并在最后调用了 uc_os3_demo 函数。

2. uc-os3_demo.c

```
/**
*****
* @file      uc-os3_demo.c
* @author    正点原子团队 (ALIENTEK)
* @version   V1.0
* @date      2022-03-12
* @brief     emwin 文本显示实验
* @license   Copyright (c) 2020-2032, 广州市星翼电子科技有限公司
*****
* @attention
*
* 实验平台: 正点原子 STM32F103 开发板
* 在线视频: www.yuanzige.com
* 技术论坛: www.openedv.com
* 公司网址: www.alientek.com
* 购买地址: openedv.taobao.com
*
*****
*/

#include "uc-os3_demo.h"
#include "os.h"
#include "cpu.h"
#include "GUI.h"
#include "atk_emwin.h"
#include "WM.h"

/* uC-OS3 任务设置 */
/* START_TASK 任务 配置
 * 包括: 任务句柄 任务优先级 堆栈大小 创建任务
 */
#define START_TASK_PRIO    3 /* 任务优先级 */
```

```

#define START_STK_SIZE      128                /* 任务堆栈大小 */
OS_TCB StartTaskTCB;                          /* 任务控制块 */
CPU_STK START_TASK_STK[START_STK_SIZE];       /* 任务堆栈 */
void start_task(void *p_arg);                  /* 任务函数 */

/* USER_TASK 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */
#define USER_TASK_PRIO      4                /* 任务优先级 */
#define USER_STK_SIZE       1024             /* 任务堆栈大小 */
OS_TCB User_TaskTCB;                          /* 任务控制块 */
CPU_STK USER_TASK_STK[USER_STK_SIZE];        /* 任务堆栈 */
void user_task(void *p_arg);

/* TOUCH_TASK 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */
#define TOUCH_TASK_PRIO     5                /* 任务优先级 */
#define TOUCH_STK_SIZE      128             /* 任务堆栈大小 */
OS_TCB Touch_TaskTCB;                          /* 任务控制块 */
CPU_STK TOUCH_TASK_STK[TOUCH_STK_SIZE];       /* 任务堆栈 */
void touch_task(void *p_arg);

/* LED0_TASK 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */
#define LED0_TASK_PRIO      6                /* 任务优先级 */
#define LED0_STK_SIZE       128             /* 任务堆栈大小 */
OS_TCB Led0_TaskTCB;                          /* 任务控制块 */
CPU_STK LED0_TASK_STK[LED0_STK_SIZE];        /* 任务堆栈 */
void led0_task(void *p_arg);

/**
 * @brief      执行 emwin 例程
 * @param      无
 * @retval     无
 */
void uc_os3_demo(void)
{
    OS_ERR err;

    /* 关闭所有中断 */
    CPU_IntDis();

    /* 初始化 uC/OS-III */
    OSInit(&err);

    /* 创建开始任务 */
    OSTaskCreate( (OS_TCB *      )&StartTaskTCB,          /* 任务控制块 */
                  (CPU_CHAR *    )"start task",          /* 任务名字 */
                  (OS_TASK_PTR   )start_task,             /* 任务函数 */
                  (void *        )0,                     /* 任务函数的参数 */
                  (OS_PRIO       )START_TASK_PRIO,        /* 任务优先级 */
                  (CPU_STK *      )&START_TASK_STK[0],    /* 任务堆栈基地址 */
                  (CPU_STK_SIZE  )START_STK_SIZE / 10,    /* 任务堆栈深度限位 */
                  (CPU_STK_SIZE  )START_STK_SIZE,         /* 任务堆栈大小 */
                  (OS_MSG_QTY    )0,
                  (OS_TICK       )0,
                  (void *        )0,                     /* 用户补充的存储区 */
                  (OS_OPT        )OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR,
                  (OS_ERR *      )&err);                 /* 存放该函数错误时的返回值 */
}

```

```

/* 开始任务调度 */
OSStart(&err);

for (;;)
{
    /* 不会进入这里 */
}

/**
 * @brief      start_task
 * @param      p_arg : 传入参数(未用到)
 * @retval     无
 */
void start_task(void *p_arg)
{
    p_arg = p_arg;

    OS_ERR err;
    CPU_INT32U cnts;

    /* 初始化 CPU 库 */
    CPU_Init();

    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U) (HAL_RCC_GetSysClockFreq() / OSCfg_TickRate_Hz);
    OS_CPU_SysTickInit(cnts);

    /* 开启时间片调度, 时间片设为默认值 */
    OSSchedRoundRobinCfg(OS_TRUE, 0, &err);

    GUI_Init();                                /* emWin 初始化 */

    /* 创建 User 任务 */
    OSTaskCreate( (OS_TCB *) &User_TaskTCB,
                  (CPU_CHAR *) "User task",
                  (OS_TASK_PTR) user_task,
                  (void *) 0,
                  (OS_PRIO) USER_TASK_PRIO,
                  (CPU_STK *) &USER_TASK_STK,
                  (CPU_STK_SIZE) USER_STK_SIZE / 10,
                  (CPU_STK_SIZE) USER_STK_SIZE,
                  (OS_MSG_QTY) 0,
                  (OS_TICK) 0,
                  (void *) 0,
                  (OS_OPT) OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR,
                  (OS_ERR *) &err);

    /* 创建 Led0 任务 */
    OSTaskCreate( (OS_TCB *) &Led0_TaskTCB,
                  (CPU_CHAR *) "led0_task",
                  (OS_TASK_PTR) led0_task,
                  (void *) 0,
                  (OS_PRIO) LED0_TASK_PRIO,
                  (CPU_STK *) &LED0_TASK_STK,
                  (CPU_STK_SIZE) LED0_STK_SIZE / 10,
                  (CPU_STK_SIZE) LED0_STK_SIZE,
                  (OS_MSG_QTY) 0,
                  (OS_TICK) 0,
                  (void *) 0,
                  (OS_OPT) OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR,
                  (OS_ERR *) &err);

    /* 创建 Touch 任务 */

```



```

OSTaskCreate( (OS_TCB *      ) &Touch_TaskTCB,
              (CPU_CHAR *    ) "Touch task",
              (OS_TASK_PTR   ) touch_task,
              (void *        ) 0,
              (OS_PRIO       ) TOUCH_TASK_PRIO,
              (CPU_STK *     ) &TOUCH_TASK_STK,
              (CPU_STK_SIZE  ) TOUCH_STK_SIZE / 10,
              (CPU_STK_SIZE  ) TOUCH_STK_SIZE,
              (OS_MSG_QTY    ) 0,
              (OS_TICK       ) 0,
              (void *        ) 0,
              (OS_OPT        ) OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR,
              (OS_ERR *      ) &err);

/* 删除 Start Task */
OSTaskDel((OS_TCB *)0, &err);
}

/**
 * @brief      执行 emwin 例程
 * @param      p_arg : 传入参数 (未用到)
 * @retval     无
 */
void user_task(void *p_arg)
{
    p_arg = p_arg;
    OS_ERR err;
    emwin_main();

    while(1)
    {
        OSTimeDly(5, OS_OPT_TIME_DLY, &err);    /* 延时 5ticks */
    }
}

/**
 * @brief      emwin 触摸
 * @param      p_arg : 传入参数 (未用到)
 * @retval     无
 */
void touch_task(void *p_arg)
{
    p_arg = p_arg;
    OS_ERR err;

    while(1)
    {
        GUI_TOUCH_Exec();
        OSTimeDly(5, OS_OPT_TIME_DLY, &err);    /* 延时 5ticks */
    }
}

/**
 * @brief      代表系统在运行中
 * @param      p_arg : 传入参数 (未用到)
 * @retval     无
 */
void led0_task(void *p_arg)
{
    OS_ERR err;
    p_arg = p_arg;

    while (1)
    {
        LED0_TOGGLE();
    }
}

```

```

        OSTimeDly(500, OS_OPT_TIME_DLY, &err);    /* 延时 500ticks */
    }
}

```

对于 uc-os3_demo.c 文件，这里先简单的介绍一下这个文件的代码结构，这个文件的代码结构可分为 6 个部分，分别是包含头文件、uC-OS3 相关配置、全局变量及自定义函数、应用程序入口函数、开始任务入口函数、其他任务入口函数，接下来分别地介绍以上这几个部分的代码。

(1) 包含头文件

包含的头文件分成两个部分，分别为 uC-OS3 头文件和其他头文件，如下所示：

```

#include "./SYSTEM/usart/usart.h"
#include "./BSP/LED/led.h"
#include "./BSP/LCD/lcd.h"
#include "uc-os3_demo.h"
#include "os.h"
#include "cpu.h"
#include "GUI.h"
#include "GUIDEMO.h"
#include "WM.h"

```

(2) uC-OS3 相关配置

uC-OS3 的配置主要包括所创建 uC-OS3 任务的相关定义（任务优先级、任务堆栈大小、任务句柄、任务函数）以及 uC-OS3 相关变量（信号量、事件、列表、软件定时器等）的定义，如下所示：

```

/* uC-OS3 任务设置 */
/* START_TASK 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */
#define START_TASK_PRIO    3                /* 任务优先级 */
#define START_STK_SIZE     128              /* 任务堆栈大小 */
OS_TCB StartTaskTCB;                      /* 任务控制块 */
CPU_STK START_TASK_STK[START_STK_SIZE];   /* 任务堆栈 */
void start_task(void *p_arg);              /* 任务函数 */

/* USER_TASK 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */
#define USER_TASK_PRIO    4                /* 任务优先级 */
#define USER_STK_SIZE     1024             /* 任务堆栈大小 */
OS_TCB User_TaskTCB;                      /* 任务控制块 */
CPU_STK USER_TASK_STK[USER_STK_SIZE];    /* 任务堆栈 */
void user_task(void *p_arg);

/* TOUCH_TASK 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */
#define TOUCH_TASK_PRIO    5                /* 任务优先级 */
#define TOUCH_STK_SIZE     128             /* 任务堆栈大小 */
OS_TCB Touch_TaskTCB;                     /* 任务控制块 */
CPU_STK TOUCH_TASK_STK[TOUCH_STK_SIZE];   /* 任务堆栈 */
void touch_task(void *p_arg);

/* LED0_TASK 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */
#define LED0_TASK_PRIO     6                /* 任务优先级 */
#define LED0_STK_SIZE     128              /* 任务堆栈大小 */
OS_TCB Led0_TaskTCB;                      /* 任务控制块 */
CPU_STK LED0_TASK_STK[LED0_STK_SIZE];     /* 任务堆栈 */
void led0_task(void *p_arg);

```

(3) 应用程序入口函数

这部分就是函数 `uc_os3_demo`，函数 `uc_os3_demo` 一开始就是创建开始任务，最后开启 uC-OS3 系统任务调度，如下所示：

```
/**
 * @brief      执行 emwin 例程
 * @param      无
 * @retval     无
 */
void uc_os3_demo(void)
{
    OS_ERR err;

    /* 关闭所有中断 */
    CPU_IntDis();

    /* 初始化 uC/OS-III */
    OSInit(&err);

    /* 创建开始任务 */
    OSTaskCreate( (OS_TCB *) &StartTaskTCB,          /* 任务控制块 */
                  (CPU_CHAR *) "start_task",          /* 任务名字 */
                  (OS_TASK_PTR) start_task,           /* 任务函数 */
                  (void *) 0,                          /* 任务函数的参数 */
                  (OS_PRIO) START_TASK_PRIO,          /* 任务优先级 */
                  (CPU_STK *) &START_TASK_STK[0],     /* 任务堆栈基地址 */
                  (CPU_STK_SIZE) START_STK_SIZE / 10, /* 任务堆栈深度限位 */
                  (CPU_STK_SIZE) START_STK_SIZE,      /* 任务堆栈大小 */
                  (OS_MSG_QTY) 0,
                  (OS_TICK) 0,
                  (void *) 0,                          /* 用户补充的存储区 */
                  (OS_OPT) OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR,
                  (OS_ERR *) &err);                   /* 存放该函数错误时的返回值 */

    /* 开始任务调度 */
    OSStart(&err);

    for (;;)
    {
        /* 不会进入这里 */
    }
}
```

(4) 开始任务入口函数

这部分就是开始任务的入口函数，开始任务主要用于创建或初始化特定实验中使用到的一些硬件外设和 uC-OS3 相关的软件（信号量、事件、列表、软件定时器等）以及创建其他用于实验演示的任务，如下所示：

```
/**
 * @brief      start_task
 * @param      p_arg : 传入参数(未用到)
 * @retval     无
 */
void start_task(void *p_arg)
{
    p_arg = p_arg;

    OS_ERR err;
    CPU_INT32U cnts;

    /* 初始化 CPU 库 */
    CPU_Init();

    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U) (HAL_RCC_GetSysClockFreq() / OSCfg_TickRate_Hz);
```

```

OS_CPU_SysTickInit(cnts);

/* 开启时间片调度，时间片设为默认值 */
OSSchedRoundRobinCfg(OS_TRUE, 0, &err);

GUI_Init(); /* emWin 初始化 */

/* 创建 User 任务 */
OSTaskCreate( (OS_TCB *) &User_TaskTCB,
              (CPU_CHAR *) "User task",
              (OS_TASK_PTR) user_task,
              (void *) 0,
              (OS_PRIO) USER_TASK_PRIO,
              (CPU_STK *) &USER_TASK_STK,
              (CPU_STK_SIZE) USER_STK_SIZE / 10,
              (CPU_STK_SIZE) USER_STK_SIZE,
              (OS_MSG_QTY) 0,
              (OS_TICK) 0,
              (void *) 0,
              (OS_OPT) OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR,
              (OS_ERR *) &err);

/* 创建 Led0 任务 */
OSTaskCreate( (OS_TCB *) &Led0_TaskTCB,
              (CPU_CHAR *) "led0 task",
              (OS_TASK_PTR) led0_task,
              (void *) 0,
              (OS_PRIO) LED0_TASK_PRIO,
              (CPU_STK *) &LED0_TASK_STK,
              (CPU_STK_SIZE) LED0_STK_SIZE / 10,
              (CPU_STK_SIZE) LED0_STK_SIZE,
              (OS_MSG_QTY) 0,
              (OS_TICK) 0,
              (void *) 0,
              (OS_OPT) OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR,
              (OS_ERR *) &err);

/* 创建 Touch 任务 */
OSTaskCreate( (OS_TCB *) &Touch_TaskTCB,
              (CPU_CHAR *) "Touch task",
              (OS_TASK_PTR) touch_task,
              (void *) 0,
              (OS_PRIO) TOUCH_TASK_PRIO,
              (CPU_STK *) &TOUCH_TASK_STK,
              (CPU_STK_SIZE) TOUCH_STK_SIZE / 10,
              (CPU_STK_SIZE) TOUCH_STK_SIZE,
              (OS_MSG_QTY) 0,
              (OS_TICK) 0,
              (void *) 0,
              (OS_OPT) OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR,
              (OS_ERR *) &err);

/* 删除 Start Task */
OSTaskDel((OS_TCB *) 0, &err);
}
    
```

(5) 其他任务入口函数

这部分就包含了实验中用于演示的任务入口函数，如下所示：

```

/**
 * @brief      执行 emwin 例程
 * @param      p_arg : 传入参数(未用到)
 * @retval     无
 */
void user_task(void *p_arg)
{
    
```

```

    p_arg = p_arg;
    OS_ERR err;
    emwin_main();

    while(1)
    {
        OSTimeDly(5, OS_OPT_TIME_DLY, &err);    /* 延时 5ticks */
    }
}

/**
 * @brief      emwin 触摸
 * @param      p_arg : 传入参数(未用到)
 * @retval     无
 */
void touch_task(void *p_arg)
{
    p_arg = p_arg;
    OS_ERR err;

    while(1)
    {
        GUI_TOUCH_Exec();
        OSTimeDly(5, OS_OPT_TIME_DLY, &err);    /* 延时 5ticks */
    }
}

/**
 * @brief      代表系统在运行中
 * @param      p_arg : 传入参数(未用到)
 * @retval     无
 */
void led0_task(void *p_arg)
{
    OS_ERR err;
    p_arg = p_arg;

    while (1)
    {
        LED0_TOGGLE();
        OSTimeDly(500, OS_OPT_TIME_DLY, &err);    /* 延时 500ticks */
    }
}

```

user_task 任务函数调用函数 emwin_main 执行 emWin 例程，该函数在 emwin_demo.c 文件定义。touch_task 任务函数调用函数 GUI_TOUCH_Exec 刷新触摸坐标。led0_task 任务函数主要表示系统是否在运行中。本教程配套的实验例程都会按着这个代码结构来进行实验代码的编写，建议读者先熟悉这个代码结构。

3. uc-os_demo.h

```

#ifndef __UC_OS3_DEMO_H
#define __UC_OS3_DEMO_H

void uc_os3_demo(void);

#endif

```

uc-os_demo.h 这个文件很简单，就是将函数 uc_os3_demo 导出给其他 C 源文件调用。

4. emwin_demo.c

该文件主要由用户编写的 emWin 程序，如下所示：

```

/**
 * @brief      emwin_main(emwin 入口)
 * @param      无
 * @retval     无
 */

```

```
void emwin_main(void)
{
    /* 编写 emWin 代码 */
}
```

5. emwin_demo.h

```
#ifndef _EMWIN_DEMO_H
#define _EMWIN_DEMO_H
#include "../SYSTEM/sys/sys.h"

void emwin_main(void); /* emwin 例程入口 */
#endif
```

emwin_demo.h 这个文件很简单，就是将函数 emwin_main 导出给其他 C 源文件调用。

3.4 编写综合测试程序

本实验主要测试 emWin 文本 API 函数使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用 GUIDEMO_Main 函数执行 emWin 程序，该函数主要运行官方的代码。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 2 uC-OS3 操作系统移植》。

(1) 任务设置

```
/* START_TASK 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */
#define START_TASK_PRIO    3          /* 任务优先级 */
#define START_STK_SIZE     128        /* 任务堆栈大小 */
OS_TCB StartTaskTCB;                /* 任务控制块 */
CPU_STK START_TASK_STK[START_STK_SIZE]; /* 任务堆栈 */
void start_task(void *p_arg);        /* 任务函数 */

/* USER_TASK 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */
#define USER_TASK_PRIO     4          /* 任务优先级 */
#define USER_STK_SIZE      1024       /* 任务堆栈大小 */
OS_TCB User_TaskTCB;          /* 任务控制块 */
CPU_STK USER_TASK_STK[USER_STK_SIZE]; /* 任务堆栈 */
void user_task(void *p_arg);

/* TOUCH_TASK 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */
#define TOUCH_TASK_PRIO    5          /* 任务优先级 */
#define TOUCH_STK_SIZE     512        /* 任务堆栈大小 */
OS_TCB Touch_TaskTCB;          /* 任务控制块 */
CPU_STK TOUCH_TASK_STK[TOUCH_STK_SIZE]; /* 任务堆栈 */
void touch_task(void *p_arg);
```

由于 emWin 的程序使用的堆栈比较大，所以任务的堆栈尽量设置大一些。

(2) 任务创建

```
/**
 * @brief      start_task
 * @param      p_arg : 传入参数 (未用到)
 * @retval     无
```

```

*/
void start_task(void *p_arg)
{
    p_arg = p_arg;

    OS_ERR err;
    CPU_INT32U cnts;

    /* 初始化 CPU 库 */
    CPU_Init();

    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U) (HAL_RCC_GetSysClockFreq() / OSCfg_TickRate_Hz);
    OS_CPU_SysTickInit(cnts);

    /* 开启时间片调度，时间片设为默认值 */
    OSSchedRoundRobinCfg(OS_TRUE, 0, &err);

    GUI_Init(); /* emWin 初始化 */
    /* 创建 User 任务 */
    OSTaskCreate( (OS_TCB *) &User_TaskTCB,
                  (CPU_CHAR *) "User task",
                  (OS_TASK_PTR) user_task,
                  (void *) 0,
                  (OS_PRIO) USER_TASK_PRIO,
                  (CPU_STK *) &USER_TASK_STK,
                  (CPU_STK_SIZE) USER_STK_SIZE / 10,
                  (CPU_STK_SIZE) USER_STK_SIZE,
                  (OS_MSG_QTY) 0,
                  (OS_TICK) 0,
                  (void *) 0,
                  (OS_OPT) OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR,
                  (OS_ERR *) &err);

    /* 创建 Touch 任务 */
    OSTaskCreate( (OS_TCB *) &Touch_TaskTCB,
                  (CPU_CHAR *) "Touch task",
                  (OS_TASK_PTR) touch_task,
                  (void *) 0,
                  (OS_PRIO) TOUCH_TASK_PRIO,
                  (CPU_STK *) &TOUCH_TASK_STK,
                  (CPU_STK_SIZE) TOUCH_STK_SIZE / 10,
                  (CPU_STK_SIZE) TOUCH_STK_SIZE,
                  (OS_MSG_QTY) 0,
                  (OS_TICK) 0,
                  (void *) 0,
                  (OS_OPT) OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR,
                  (OS_ERR *) &err);

    /* 创建 Touch 任务 */
    OSTaskCreate( (OS_TCB *) &Touch_TaskTCB,
                  (CPU_CHAR *) "Touch task",
                  (OS_TASK_PTR) touch_task,
                  (void *) 0,
                  (OS_PRIO) TOUCH_TASK_PRIO,
                  (CPU_STK *) &TOUCH_TASK_STK,
                  (CPU_STK_SIZE) TOUCH_STK_SIZE / 10,
                  (CPU_STK_SIZE) TOUCH_STK_SIZE,
                  (OS_MSG_QTY) 0,
                  (OS_TICK) 0,
                  (void *) 0,
                  (OS_OPT) OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR,
                  (OS_ERR *) &err);

    CPU_CRITICAL_EXIT(); /* 退出临界区 */
    OSTaskDel((OS_TCB *) 0, &err); /* 删除 start_task 任务自身 */
}

```

(3) 任务实现

```
/**
 * @brief      执行 emwin 例程
 * @param      p_arg : 传入参数 (未用到)
 * @retval     无
 */
void user_task(void *p_arg)
{
    OS_ERR err;
    p_arg = p_arg;
    GUIDEMO_Main();

    while (1)
    {
        OSTimeDlyHMSM(0,0,0,5,OS_OPT_TIME_PERIODIC,&err); /* 延时 5ms */
    }
}

/**
 * @brief      emwin 触摸
 * @param      p_arg : 传入参数 (未用到)
 * @retval     无
 */
void touch_task(void *p_arg)
{
    OS_ERR err;
    p_arg = p_arg;
    GUI_CURSOR_Show();

    while (1)
    {
        GUI_TOUCH_Exec();
        GUI_Delay(5); /* 延时 5ms */
    }
}

/**
 * @brief      代表系统在运行中
 * @param      p_arg : 传入参数 (未用到)
 * @retval     无
 */
void led0_task(void *p_arg)
{
    OS_ERR err;
    p_arg = p_arg;

    while (1)
    {
        LED0_TOGGLE();
        OSTimeDly(500, OS_OPT_TIME_DLY, &err); /* 延时 500ticks */
    }
}
```

在上一章中，我们使用定时器来为 emWin 提供系统时钟和处理触摸事件。在本章中，我们使用 uC-OS3 系统来为 emWin 提供系统时钟，对于触摸事件的处理可建立一个任务来完成，因此这里就不需要使用定时器了。

3.5 下载验证

代码编译成功之后，下载代码到我们的开发板上，LCD 显示如下图所示：

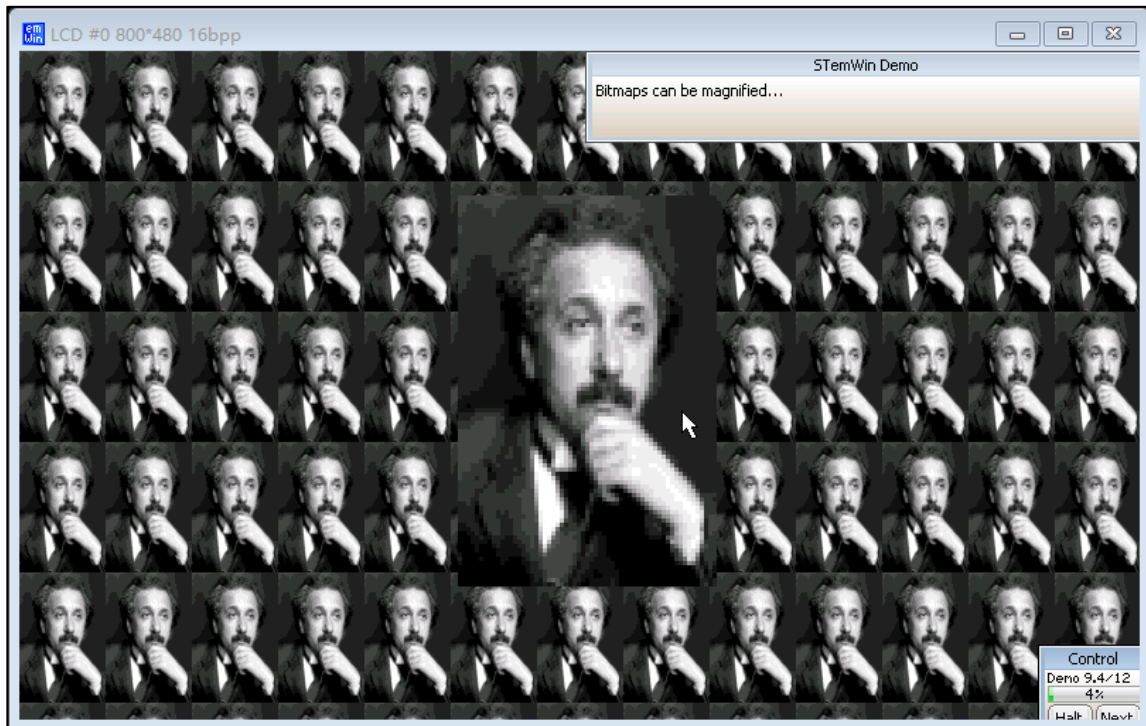


图 3.5.1 程序运行效果图

第四章 在 PC 上仿真 emWin

Segger 官方提供给用户很多学习例程，这些例程涵盖了 emWin 的所有功能，用户想学习 emWin 图形库，其实就是学习这些例程，但是这些例程都是基于 PC 的，也就是说在 PC 上仿真运行。因此学会如何在 PC 上仿真运行 emWin 非常重要！再者，学会了在 PC 上仿真 emWin，即使我们没有开发板也完全可以学习如何使用 emWin，这些例程需要什么软件可以仿真呢？我们常用的软件有 Visual Studio2019、VC++等软件。

本章分为以下几个部分：

- 4.1 准备 emWin 仿真文件
- 4.2 打开 emWin 仿真文件
- 4.3 进行仿真
- 4.4 emWin 仿真文件介绍

4.1 准备 emWin 仿真文件

虽然 ST 官方并没有提供 STemWin 的 PC 仿真文件，但是 STemWin 是基于 emWin 而诞生的，只是针对 ST 的 MCU 做了简单的修改，因此用户完全可以使用 emWin 的 PC 仿真文件。emWin 的仿真文件可在 Segger 官网下载，下载地址为：<https://www.segger.com/downloads/emwin>，打开这个地址之后如下图所示：

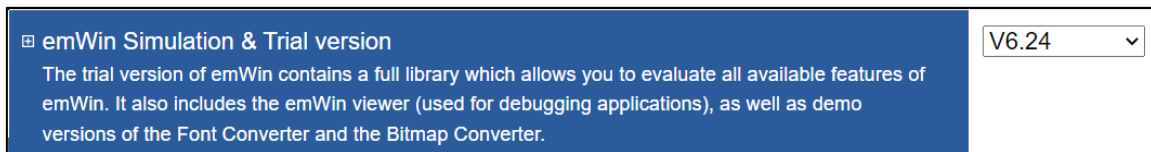


图 4.1.1 emWin 下载界面

由于 segger 官网下载需要进行注册才能下载，所以笔者已经把仿真文件放置在光盘当中，该文件获取路径为：“6，软件资料→3，emWin 学习资料→emWin6.22 PC 仿真文件”。

4.2 打开 emWin 仿真文件

emWin 模拟需要 Microsoft Visual C++（6.00 或更高版本）的 IDE，笔者使用的是 VS2019 也就是 Visula Studio 2019，关于 VS2019 软件的安装这里不做讲解，默认大家都已经安装好了 VS2019。接下来我们分为以下几步讲解。

1. 在某盘根目录下新建一个文件夹 emWin6.22 simulate，比如 C 盘、D 盘等建立文件夹。将 SeggerEval_WIN32_MSVC_MinGW_GUI_V622.zip 解压的 PC 仿真文件并拷贝到 emWin6.22 simulate 文件夹中，注意，不能出现中文路径！解压后的文件目录如下图所示：

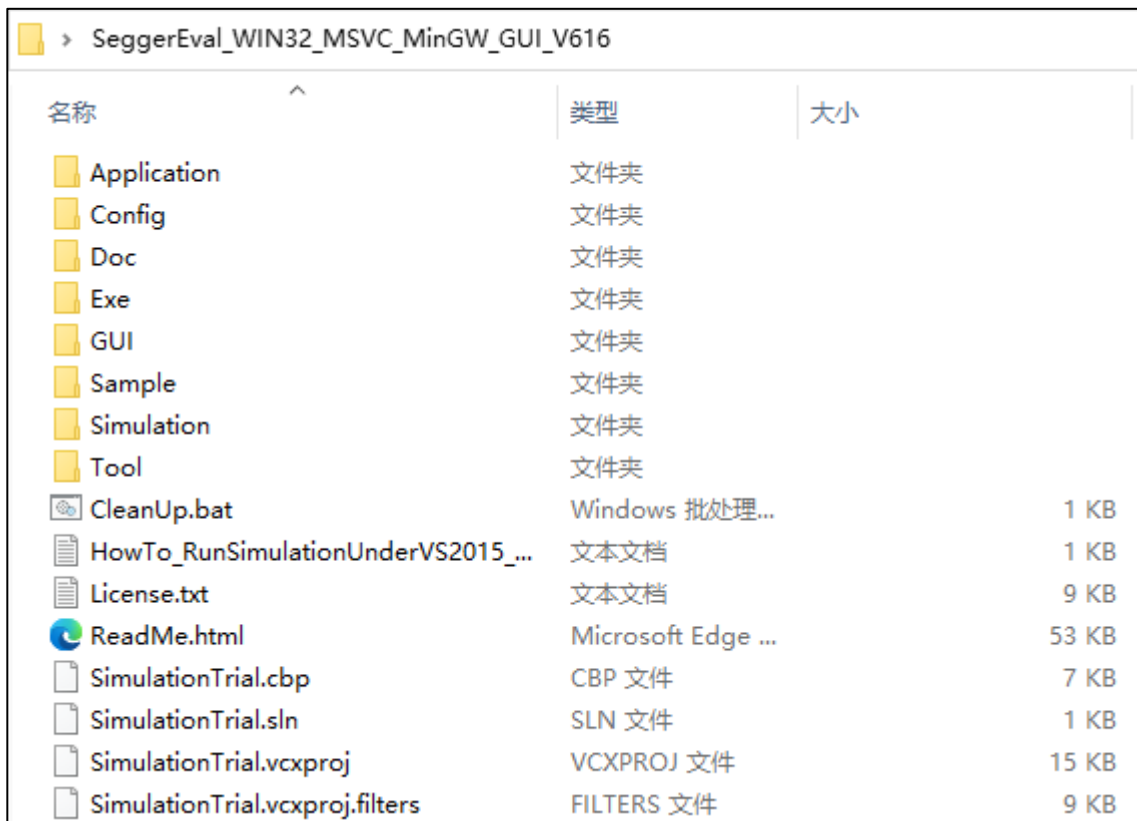


图 4.2.1 解压后的 emWin6.22 仿真文件

2. 点击 SimulationTrial.sln 打开仿真工程。
3. 仿真工程结构如下图所示：



图 4.2.2 打开后的仿真文件

上图中仿真工程一共分为 5 个分组：Application，Config，GUI，Simulation 和外部依赖项分组，这些分组介绍如下表所示：

组	描述
Application	官方的演示程序源代码
Config	emWin 以及 LCD 的配置文件，包含 GUIConf.c，GUIConf.h，LCDConf.c，LCDConf.h 和 SIMConf.c 这 5 个文件
GUI	emWin 的库文件和库的头文件
Simulation	这个是配置 emWin 的运行环境等等，我们不用管这个东西
外部依赖项	依赖的库文件

表 4.2.1 仿真文件的描述

4.3 进行仿真

对于 emWin 仿真工程我们分为以下几步讲解：

第一步：重定目标解决方案

点击项目→重定目标解决方案，如下图所示：



图 4.3.1 重定目标解决方案

点击确定重定向项目，如下图所示：



图 4.3.2 确定安装版本

第二步：设置附加依赖项

工程项目→右键→选择属性，如下图所示：



图 4.3.3 属性设置

点击链表器→输入→附加依赖项，在该附加依赖项填写 `legacy_stdio_definitions.lib` 文件库，设置完毕记得点击确定。接着设置忽略所有默认库选项为否，然后添加两个忽略特定默认库 `LIBC.lib` 和 `LIBCMDT.lib`，最后确定保存。

打开 emWin 仿真工程后，按下键盘的 F5 键或者如下图红框所示的地方编译并运行整个工程。



图 4.3.4 编译并运行工程

编译完成后运行的是 emWin 自带的综合演示 Demo，也就是 Application 分组中的.c和.h文件，运行如下图所示：



图 4.3.7 emWin 仿真工程运行

上图的演示 Demo 就是 Application 分组中的.c 和.h 文件，这些文件并不适合初学者学习，读者可以先从基本功能开始，最后再去学习高级的应用。

4.4 emWin 仿真文件介绍

4.4.1 Application 分组

本节主要讲解 Application 分组下的文件，如下图所示：

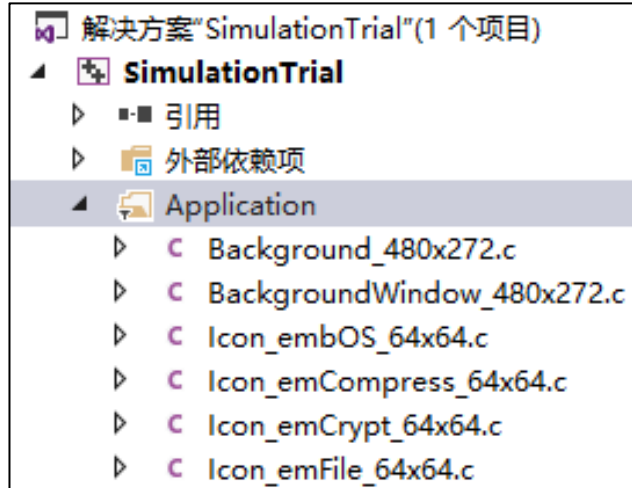


图 4.4.1.1 Application 文件

在上一节中看到的绚丽的界面就是由这些.C 文件构成的，但是这些例程比较高级，不利于初学者学习，当我们对 emWin 有了一定的了解后再来看这些文件就会轻松很多了。

用户怎么样查看 emWin 的控件例程呢，首先在工程新建 APP_DEMO 文件，添加完成之后如下图所示：



图 4.4.1.2 添加一个 APP_Demo 文件

由于 emWin 的基础例程在 SeggerEval_WIN32_MSVC_MinGW_GUI_V622\Sample 文件夹下，所以需要用户把这些例程添加到工程当中。

APP_Demo→右键→添加→现有项→sample/Tutorial 文件中的.c 文件，如下图所示：

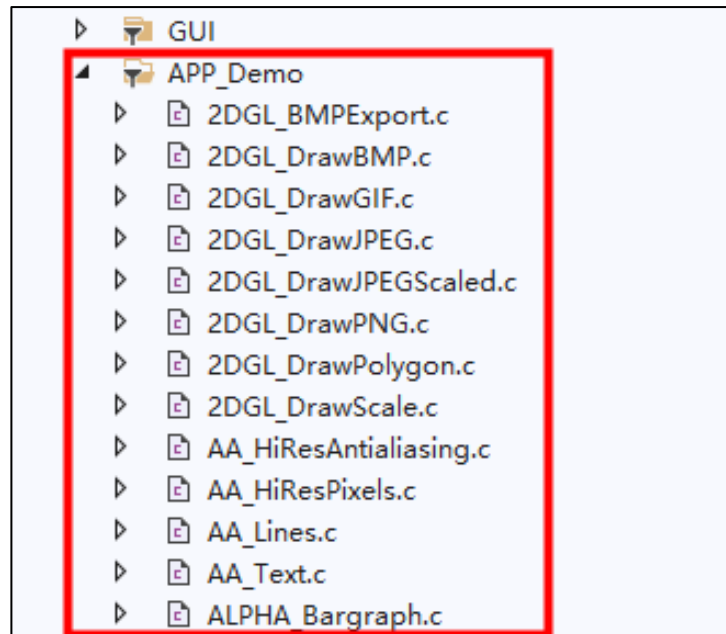


图 4.4.1.3 添加 emWin 例程

由于添加了 SeggerEval_WIN32_MSVC_MinGW_GUI_V614\Sample\Tutorial 文件中的.c 文件，而这些.c 文件每一个都是独立的例程，VS2019 不可能同时编译多个例程，所以对例程进行排除编译，防止并行编译。如下图所示：



图 4.4.1.4 配置 APP_Demo 文件下的.c 文件配置

全部选择，接着右键→属性，选择从生成中排除，这样例程就不会影响到其他的例程编译了，例程排除之后如下图所示：

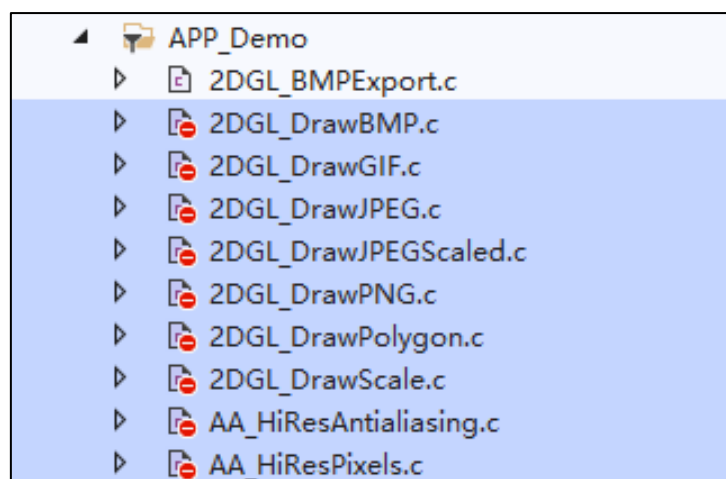


图 4.4.1.5 APP_Demo 文件中例程文件的配置

从上图可知：我们只保留 2DGL_BMPExport.c 例程运行。

如果用户想运行 APP_Demo 文件下的某个例程，则先把 Application 文件下的全部.c 文件排除编译，如下图所示：

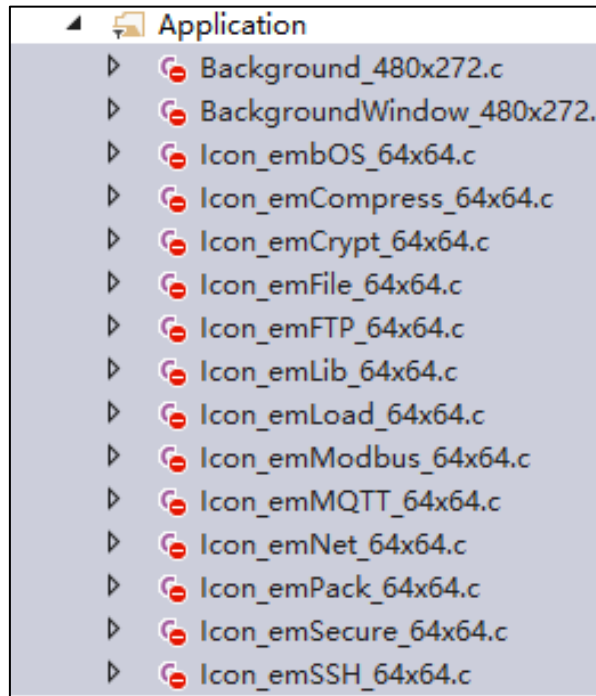


图 4.4.1.6 Application 文件下的.c 文件的配置

最后选中编译的例程，按下 F5 或者在 vs2019 编译按钮，如下图所示：



图 4.4.1.7 运行 2DGL_BMPEXport.c 例程

注意：如果继续编译其他例程，需要把当前例程文件从生成中排除，再进行其他例程调试。

4.4.2 Config 分组

GUIConf.c 文件主要是给 emWin 分配内存的，代码如下：

```
#include "GUI.h"
//
```



```
// Define the available number of bytes available for the GUI
//
#define GUI_NUMBYTES 0x280000

void GUI_X_Config(void)
{
    //
    // 32 位对齐内存区
    //
    static U32 aMemory[GUI_NUMBYTES / 4];
    //
    // 分配内存给 emWin
    //
    GUI_ALLOC_AssignMemory(aMemory, GUI_NUMBYTES);
}
```

从上面的代码可以看出，GUIConf.c 文件里面只有一个 GUI_X_Config 函数，这个函数就是给 emWin 分配内存，宏 GUI_NUMBYTES 定义了分配的内存大小，默认为 0X280000，也就是 2.8M 的内存，最后调用 GUI_ALLOC_AssignMemory 函数将这 2.8M 的内存分配给 emWin。

GUIConf.h 文件主要配置以及裁剪 emWin，比如是否支持操作系统、触摸、鼠标、窗口和存储设备等配置项，代码如下：

```
#define GUI_NUM_LAYERS 16 // Maximum number of available layers

/*****
 *
 *      Multi tasking support
 */
#define GUI_OS (1) // Compile with multitasking support

/*****
 *      Configuration of available packages
 */
#define GUI_SUPPORT_TOUCH (1) // Support a touch screen (req. win-manager)
#define GUI_SUPPORT_MOUSE (1) // Support a mouse
#define GUI_WINSUPPORT (1) // Window manager package available
#define GUI_SUPPORT_MEMDEV (1) // Memory devices available

/*****
 *      Configuration of window manager
 */
#define WM_SUPPORT_NOTIFY_VIS_CHANGED (1)

/*****
 *
 *      Default font
 */
```

```
#define GUI_DEFAULT_FONT    &GUI_Font6x8
#endif /* Avoid multiple inclusion */
```

LCDConf.c 文件主要用来设置模拟显示屏大小，配置显示驱动和颜色转换格式等配置，如下所示：

```
#define XSIZE_PHYS          480
#define YSIZE_PHYS          272
```

修改上面这两个宏可以更改仿真时模拟显示屏大小，这个很常用，其他配置项默认即可。

LCDConf.h 文件里面没有东西，SIMConf.c 是在仿真 emWin 的时候对 PC 环境的一些特定设置，我们不需要了解。

4.4.3 GUI 分组

GUI 分组下的文件就是 emWin 的真身，这个文件里面是 emWin 源码的 lib 文件和相应的头文件，emWin 已经不提供底层源码，都是以 lib 文件形式给出。在 PC 版的仿真文件中 GUI.lib 只适用于 PC 平台，移植 emWin 到开发板不能使用这个 lib 文件！segger 官方有专门针对 Cortex-M 系列 MCU 的 lib 文件。由于这两个 lib 文件里面的 API 函数都是一样的，因此在 PC 上仿真通过的代码是完全可以可以在开发板上运行。

4.4.4 Sample 文件夹解析

该文件夹是 APP_DEMO 针对 emWin 各个小控件、功能做的演示例程，我们学习 emWin 主要就是学习这个文件里面代码！相对与 Application 文件，这个文件中的代码比较简单易懂，如下图所示：

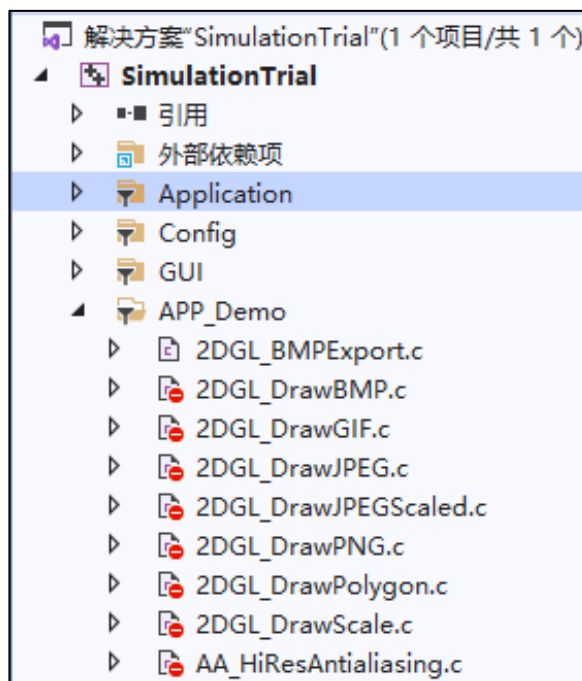


图 4.4.4.1 emWin 部分演示例程

可以看出 Sample 中的例程是非常多的，涵盖了 emWin 的所有内容，本教程后面的 emWin 例程就是移植和讲解的这些文件，因此使用我们的 emWin 开发指南即使没有板子也是可以学习。

第五章 文本显示

本章，主要讲解 emWin 的文本显示，它是 emWin 最基本的功能，我们只掌握几个典型的函数即可实现 emWin 文本显示。

本章分为以下几个部分：

- 5.1 文本显示
- 5.2 文本显示 API 函数
- 5.3 重点 API 函数简介
- 5.4 文本显示实验

5.1 文本显示

emWin 文本显示相对简单，只要设置好文本的背景颜色、前景以及字体大小就可以调用文本 API 函数进行显示了。例如使用文本显示接口显示一段字符串可调用函数 GUI_DisString 显示，如下源码所示：

```
GUI_DisString("Hello world!");
```

上述源码是将字符串“Hello world!”显示在 LCD 显示屏(0,0)坐标上。

另外，emWin 文本可支持控制字符，控制字符是指字符代码小于 32 的字符。控制字符被定义为 ASCII 代码的一部分，如下表所示：

字符代码	ASCII 代码	C	描述
10	LF	\n	换行： 当前文本位置改变至下一行的开始。默认为：X = 0， Y += 字体 - 距离（单位：像素）
13	CR	\r	回车： 当前文本位置改变至当前行的开始。默认为：X = 0

表 5.1.1 emWin 中可使用的控制字符

控制符\n 和\r 非常实用，换行符的用法在字符串中非常方便。换行符可以作为字符串的一部分，这样，字符串就可以拆分为多行，且只需调用一个函数就能显示，换行符设置如下所示：

```
GUI_DisString("Hello \n world!");
```

5.2 文本显示 API 函数

emWin 针对文本显示提供了很多的 API 函数，这些函数并不是全部都要掌握，读者会使用常用的文本显示函数即可。文本显示 API 函数如下表所示：

函数	描述
显示文本的函数	
GUI_DisCEOL()	清除从当前位置到末尾的当前行
GUI_DisChar()	在当前位置显示单个字符
GUI_DisCharAt()	在指定位置显示单个字符
GUI_DisChars()	按指定次数显示字符
GUI_DisNextLine()	将光标移至下一行的开始
GUI_DisString()	在当前位置显示字符串
GUI_DisStringAt()	在指定位置显示字符串
GUI_DisStringAtCEOL()	在指定位置显示字符串，并清除至行末

GUI_DispStringHCenterAt()	在指定位置水平居中显示字符串
GUI_DispStringInRect()	在指定的矩形区域中显示字符串
GUI_DispStringInRectEx()	在指定的矩形区域中显示字符串，并可旋转
GUI_DispStringInRectWrap()	在指定的矩形区域中显示字符串，并可自动换行
GUI_DispStringInRectWrapEx()	在指定矩形中的显示字符串，并可旋转
GUI_DispStringLength()	在当前位置显示指定字符数的字符串
GUI_ShowMissingCharacters()	此函数允许显示丢失的字符
GUI_WrapGetNumLines()	为指定自动换行模式获取文本行数
选择文本绘制模式	
GUI_GetTextMode()	返回当前文本模式
GUI_SetTextMode()	设置文本绘制模式
GUI_SetTextStyle()	设置要使用的文本样式
GUI_SetClearTextRectMode()	启用或禁用文本矩形模式
选择文本对齐模式	
GUI_GetTextAlign()	返回当前文本对齐模式
GUI_SetLBorder()	设置换行后的左边界
GUI_SetTextAlign()	设置文本对齐模式
设置当前文本位置	
GUI_GotoX()	设置 X 的坐标
GUI_GotoXY()	设置 X 和 Y 坐标
GUI_GotoY()	设置 Y 的坐标
返回当前文本位置	
GUI_GetDispPosX()	返回当前的 x 位置
GUI_GetDispPosY()	返回当前的 y 位置
清除窗口或部分窗口的例程	
GUI_Clear()	清除活动窗口
GUI_DispCEOL()	清除从当前文本位置到行末的显示内容

表 5.2.1 文本显示 API 函数

表 5.2.1 是文本显示函数相关描述。接下来，我们来讲解文本显示常用的函数。

(1) GUI_SetTextMode 函数

设置文本的模式，该函数的原型如下所示：

```
int GUI_SetTextMode(int Mode);
```

该函数的相关形参如下表所示：

参数	描述	
Mode	GUI_TM_NORMAL	设置文本为正常模式，默认设置改值为 0
	GUI_TM_XOR	设置文本反转模式
	GUI_TM_TRANS	设置文本透明模式
	GUI_TM_REV	设置文本反相模式

表 5.2.2 GUI_SetTextMode 函数的形参描述

返回值：文本模式。

由上述可知，emWin 具有多种文本绘制模式，这些绘制模式可组合方式传入 Mode 形参中。这些模式的描述如下表所示：

文本模式	描述
正常文本	GUI_TEXTMODE_NORMAL 或 0：正常显示文本
反转文本	GUI_TEXTMODE_RE：反转显示文本，通常的黑底白字显示方式将变为白底黑字显示
透明文本	GUI_TEXTMODE_TRANS：透明文本。透明文本表示文本写在屏幕上已经

	可见的内容之上。不同之处在于，屏幕上原有的内容仍然可见，而在正常文本中，背景会替换为当前选择的背景色
异或文本	GUI_TEXTMODE_XOR：异或模式显示文本。通常情况下，用白色绘制的(实际字符)显示是反相的。如果背景色是黑色，效果与默认模式(正常文本)是一样的。如果背景是白色，输出与反转文本一样。如果使用彩色，反相的像素由下式计算：新像素颜色=颜色的值 - 实际像素颜色 - 1
透明反转文本	GUI_TEXTMODE_TRANS GUI_TEXTMODE_REV：透明反转文本。它与透明文本一样，它不会覆盖背景，而且和反转文本一样，该文本会反转显示

表 5.2.3 文本模式描述

(2) GUI_SetTextAlign 函数

设置文本的对齐模式，该函数的原型如下所示：

```
int GUI_SetTextAlign(int Align);
```

该函数的相关形参如下表所示：

参数	描述
Align	设置文本对齐模式，可以水平和垂直对齐
	水平对齐模式
	GUI_TA_LEFT X 轴方向左对齐（默认）
	GUI_TA_HCENTER X 轴方向居中
	GUI_TA_RIGHT X 轴方向右对齐
	垂直对齐模式
	GUI_TA_TOP 再字符 Y 轴方向顶部对齐（默认）
	GUI_TA_VCENTER Y 轴方向居中
	GUI_TA_BOTTOM 在字符 Y 轴底部像素线对齐

表 5.2.4 GUI_SetTextAlign 函数相关形参描述

返回值：所选的文本对齐模式。

(3) GUI_DispCEOL 函数

清除窗口，从当前文本位置到行末的内容，高度为当前字体高度。此函数原型如下所示：

```
void GUI_DispCEOL(void);
```

返回值：无。

(4) GUI_DispStringInRectWrap 函数

在 LCD 指定的矩形区域内显示字符串，此函数原型如下所示：

```
void GUI_DispStringInRectWrap(const char * s, GUI_RECT * pRect, int TextAlign,
                               GUI_WRAPMODE WrapMode);
```

该函数的形参如下表所示：

参数	描述
s	显示的字符串
pRect	要写入的客户端窗口矩形区域（单位：像素）
TextAlign	可以使用“OR”操作进行组合，垂直对齐和水平对齐标志使用
	水平对齐：GUI_TA_LEFT、GUI_TA_HCENTER、GUI_TA_RIGHT
	垂直对齐：GUI_TA_TOP、GUI_TA_VCENTER、GUI_TA_BOTTOM
WrapMode	换行模式
	GUI_WRAPMODE_NONE 自动换行
	GUI_WRAPMODE_WORD 根据字对文本进行自动换行
	GUI_WRAPMODE_CHAR 根据字符对文本进行自动换行

表 5.2.5 GUI_DispStringInRectWrap 函数相关形参描述

返回值：无。

用户一般使用函数 GUI_DispStringInRectWrap 时，需要定义一个矩形区域，接着调用该函数在矩形区域显示字符串，如下源码所示：

```
GUI_WRAPMODE aWm[] = { GUI_WRAPMODE_NONE, GUI_WRAPMODE_CHAR, GUI_WRAPMODE_WORD };
GUI_RECT Rect = {10, 10, 59, 59};
char acText[] = "ALIENTEK";

void emwin_main(void)
{
    GUI_SetTextMode(GUI_TM_TRANS);      /* 设置文本透明模式 */
    for (int i = 0; i < 3; i++)
    {
        GUI_SetColor(GUI_BLUE);          /* 设置区域颜色为蓝色 */
        GUI_FillRectEx(&Rect);           /* 设置填充的区域 */
        GUI_SetColor(GUI_WHITE);         /* 设置字体颜色 */
        GUI_DispStringInRectWrap(acText, &Rect, GUI_TA_LEFT, aWm[i]);
        Rect.x0 += 60;
        Rect.x1 += 60;
    }
}
```

上述源码可知，首先我们设置文本模式为透明模式，接着进入 3 次 for 循环，每一次 for 循环都调用函数 GUI_DispStringInRectWrap 在指定区域显示字符串，并设置不同的换行模式。

5.3 文本显示实验

5.3.1 硬件设计

1. 例程功能

本实验主要测试 emWin 文本 API 函数使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用 emwin_main 函数执行 emWin 程序，该函数绘画了两条线条，在这两条线条中间绘画多个不同文本模式的字符串，依次纵向排列，然后在它们底部绘画了三个区域，最后在这些区域按不同的换行模式显示 “This example demonstrates text wrapping” 字符串。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 3 文本显示》。

5.3.2 软件设计

5.3.2.1 程序流程图

本实验的程序流程图，如下图所示：

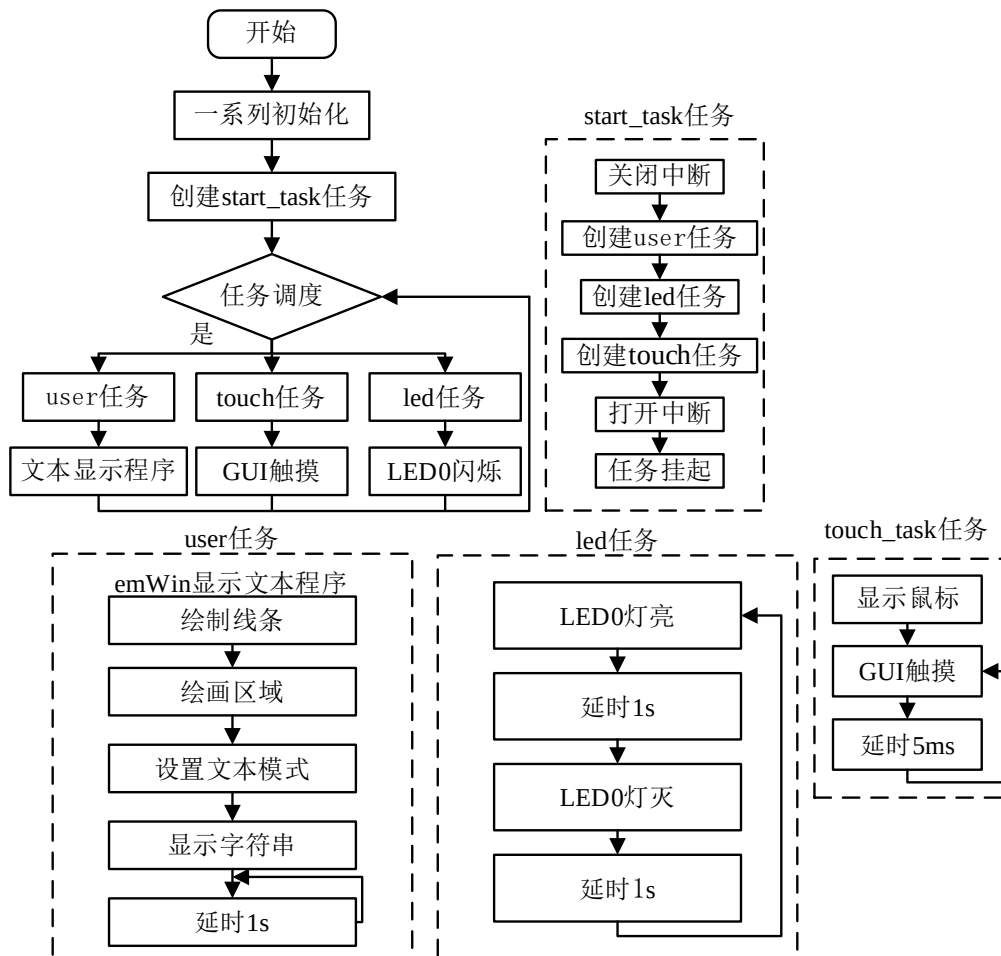


图 5.3.2.1.1 文本显示实验流程图

5.3.2.2 程序解析

本小节着重讲解本实验相关的部分。如下源码所示：

```

void emwin_main(void)
{
    int i;
    char acText[] = "This example demonstrates text wrapping";
    GUI_RECT Rect = {200, 240, 259, 299}; /* 定义矩形显示区域 */
    GUI_WRAPMODE aWm[] = { GUI_WRAPMODE_NONE,
                           GUI_WRAPMODE_CHAR,
                           GUI_WRAPMODE_WORD};

    GUI_SetBkColor(GUI_BLUE); /* 设置背景颜色 */
    GUI_Clear(); /* 清屏 */
    GUI_SetFont(&GUI_Font24_ASCII); /* 设置字体 */
    GUI_SetColor(GUI_YELLOW); /* 设置前景色(如文本，画线等颜色) */
    GUI_DispString("HELLO WORD!");
    GUI_SetFont(&GUI_Font8x16); /* 设置字体 */
    GUI_SetPenSize(10); /* 设置笔大小 */
}

```

```

GUI_SetColor(GUI_RED);           /* 红色字体 */
GUI_DrawLine(300, 50, 500, 130); /* 绘线 */
GUI_DrawLine(300, 130, 500, 50); /* 绘线 */
GUI_SetBkColor(GUI_BLACK);       /* 设置黑色背景 */
GUI_SetColor(GUI_WHITE);         /* 设置字体颜色为白色 */
GUI_SetTextMode(GUI_TM_NORMAL);  /* 正常模式 */
GUI_DispStringHCenterAt("GUI_TM_NORMAL", 400, 50);
GUI_SetTextMode(GUI_TM_REV);     /* 反转文本 */
GUI_DispStringHCenterAt("GUI_TM_REV", 400, 66);
GUI_SetTextMode(GUI_TM_TRANS);   /* 透明文本 */
GUI_DispStringHCenterAt("GUI_TM_TRANS", 400, 82);
GUI_SetTextMode(GUI_TM_XOR);     /* 异或文本 */
GUI_DispStringHCenterAt("GUI_TM_XOR", 400, 98);
GUI_SetTextMode(GUI_TM_TRANS | GUI_TM_REV); /* 透明反转文本 */
GUI_DispStringHCenterAt("GUI_EM_TRANS|GUI_TM_REV", 400, 114);
GUI_SetTextMode(GUI_TM_TRANS);   /* 透明文本 */
for (i = 0; i < 3; i++)
{
    GUI_SetColor(GUI_WHITE);
    GUI_FillRectEx(&Rect);
    GUI_SetColor(GUI_BLACK);
    /* 在当前窗口指定的矩形区域内显示字符串 (并可自动换行) */
    GUI_DispStringInRectWrap(acText, &Rect, GUI_TA_LEFT, aWm[i]);
    Rect.x0 += 70;
    Rect.x1 += 70;
}
}

```

该函数绘画了两条大线条，在这两条大线条中间绘画多个不同文本模式的字符串，依次纵向排列，然后在它们底部绘画了三个区域，最后在这些区域按不同的换行模式显示 “This example demonstrates text wrapping” 字符串。

5.3.3 下载验证

把代码编译并下载到开发板中，如下图所示：

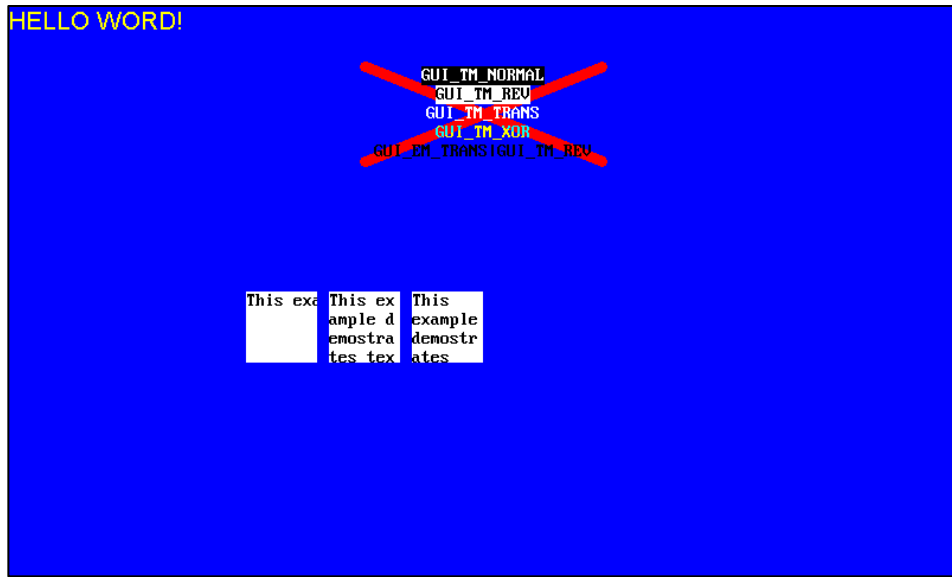


图 5.3.3.1 实例运行效果图

注意：图 5.3.3.1 为 4.3 寸屏（800 * 480）上的演示效果，2.8 或 3.5 寸屏会显示其中一部分。

本章内容为最基础的，大家务必掌握常用 API 函数的使用，本章只列举了几个函数做简单的讲解，其他没有列举出来的 API 函数大家可以参考 emWin 中文手册来学习。

总结：

假设背景颜色为蓝色，文本背景：黑，文本颜色为白色。

✧ 正常文本:背景：黑，文本：白色。

✧ 反转文本:背景：白色，文本：黑。

✧ 透明文本:不需要背景色了，仅使用前景色显示文本。

✧ 异或文本:新像素颜色 = 支持的颜色数 - 实际像素颜色 - 1（简单来说：白色取反就是黑色，黑色取反就是白色）。

✧ 透明反转文本:不需要背景色了，文本：黑。

第六章 数值显示

本章，为大家讲解 emWin 的数值显示，包含二进制、十进制、十六进制和浮点数值显示，这些显示都是调用函数实现的，但是调用时，必须了解这些函数的限定事项，才能正确地显示数值。

本章分为以下几个部分：

- 6.1 数值显示 API 函数
- 6.2 显示十进制数值
- 6.3 显示浮点数值
- 6.4 二进制和十六进制数值
- 6.5 数值显示实验

6.1 数值显示 API 函数

emWin 数值显示的 API 函数和 emWin 文本显示函数一样具有非常多的接口函数，对于这些函数我们只掌握常用的接口函数即可，emWin 数值显示 API 函数如下表所示：

API 函数	描述
显示十进制数值	
GUI_DisDec()	当前位置显示指定字符数的十进制数值
GUI_DisDecAt()	指定位置显示指定字符数的十进制数值
GUI_DisDecMin()	当前位置显示最小字符数的十进制数值
GUI_DisDecShift()	当前位置显示指定字符数、带小数点的十进制长数值
GUI_DisDecSpace()	当前位置显示指定字符数的十进制数值，用空格代替首位的
GUI_DisSDec()	当前位置显示指定字符数的十进制数值并显示符号
GUI_DisSDecShift()	当前位置显示指定字符数、带小数点的十进制长数值并显示号
显示浮点数值	
GUI_DisFloat()	显示指定字符数的浮点数值
GUI_DisFloatFix()	显示指定小数点右边位数的浮点数值
GUI_DisFloatMin()	显示最小字符数的浮点数值
GUI_DisSFloatFix()	显示指定小数点右边位数的浮点数值并显示符号
GUI_DisSFloatMin()	显示最小字符数的浮点数值并显示符号
显示二进制数值	
GUI_DisBin()	在当前位置显示二进制数值
GUI_DisBinAt()	在指定位置显示二进制数值
显示十六进制数值	
GUI_DisHex()	在当前位置显示十六进制数值
GUI_DisHexAt()	在指定位置显示十六进制数值
emWin 版本	
GUI_GetVersionString()	返回 emWin 的当前版本

表 6.1.1 数值显示 API 函数

以上是表 6.1.1 的基本绘制函数相关信息。下面我们来讲解 emWin 数值显示常用的函数。

(1) GUI_DisDec 函数

在当前位置显示指定字符数的十进制数值，该函数的原型如下所示：

```
void GUI_DisDec(I32 v, U8 Len);
```

该函数的相关形参如下表所示：

参数	描述
v	显示的数值。最小值为-2147483648（-2 ³¹ ）；最大值为 2147483647（2 ³¹ -1）
Len	显示的位数（最大为 10）

表 6.1.2 GUI_DisDec 函数的相关形参描述

返回值：无。

(2) GUI_DisDecAt 函数

在指定位置显示指定字符数的十进制数值，该函数的原型如下所示：

```
void GUI_DisDecAt( I32 v, I16P x, I16P y, U8 Len);
```

该函数的相关形参如下表所示：

参数	描述
v	显示的数值。最小值为-2147483648（-2 ³¹ ）；最大值为 2147483647（2 ³¹ -1）
x	要写入的客户端窗口 X 位置（单位：像素）
y	要写入的客户端窗口 Y 位置（单位：像素）
Len	显示的位数（最大为 10）

表 6.1.3 GUI_DisDecAt 函数的相关形参描述

返回值：无。

(3) GUI_DisDecMin 函数

在当前位置显示最小字符数的十进制数值，该函数的原型如下所示：

```
void GUI_DisDecMin(I32 v);
```

该函数的相关形参如下表所示：

参数	描述
v	显示的数值。最小值为-2147483648（-2 ³¹ ）；最大值为 2147483647（2 ³¹ -1）

表 6.1.4 GUI_DisDecMin 函数的相关形参描述

返回值：无。

(4) GUI_DisDecShift 函数

在当前位置显示指定字符数、带小数点的十进制数值，该函数的原型如下所示：

```
void GUI_DisDecShift(I32 v, U8 Len, U8 Shift);
```

该函数的相关形参如下表所示：

参数	描述
v	显示的数值。最小值为-2147483648（-2 ³¹ ）；最大值为 2147483647（2 ³¹ -1）。
Len	显示的位数（最大为 10）
Shift	小数点右边显示的位数

表 6.1.5 GUI_DisDecShift 函数的相关形参描述

返回值：无。

(5) GUI_DisDecSpace 函数

在当前位置显示指定字符数的十进制数值，该函数的原型如下所示：

```
void DisDecSpace(I32 v, U8 MaxDigits);
```

该函数的相关形参如下表所示：

参数	描述
v	显示的数值，最小值为-2147483648（-2 ³¹ ）；最大值为 2147483647（2 ³¹ -1）
Len	显示的位数，包括首位空格。显示的最大位数为 10（不包括首位空格）

表 6.1.6 GUI_DisDecSpace 函数的相关形参描述

返回值：无。

(6) GUI_DisDec 函数

在当前位置显示指定字符数的十进制数值并显示符号，该函数的原型如下所示：

```
void GUI_DispsDec(I32 v, U8 Len);
```

该函数的相关形参如下表所示：

参数	描述
v	显示的数值，最小值为-2147483648（-2 ³¹ ）；最大值为 2147483647（2 ³¹ -1）
Len	显示的位数（最大为 10）

表 6.1.7 GUI_DispsDec 函数的相关形参描述

返回值：无。

(7) GUI_DispsDecShift 函数

当前位置显示指定字符数、带小数点的十进制长数值并显示符号，该函数的原型如下所示：

```
void GUI_DispsDecShift(I32 v, U8 Len, U8 Shift);
```

该函数的相关形参如下表所示：

参数	描述
v	显示的数值，最小值为-2147483648（-2 ³¹ ）。最大值为 2147483647（2 ³¹ -1）。
Len	显示的位数（如果设置 Shift 最大 8 位；未设置 Shift 最大 9 位）
Shift	小数点右边显示的位数

表 6.1.8 GUI_DispsDecShift 函数的相关形参描述

返回值：无。

(8) GUI_DispsFloat 函数

显示指定字符数的浮点数值，该函数的原型如下所示：

```
void GUI_DispsFloat(float v, char Len);
```

该函数的相关形参如下表所示：

参数	描述
v	显示的数值，最小值为-2147483648（-2 ³¹ ）。最大值为 2147483647（2 ³¹ -1）
Len	显示的位数（最大为 10）

表 6.1.9 GUI_DispsFloat 函数的相关形参描述

返回值：无。

(10) GUI_DispsFloatFix 函数

显示指定小数点右边位数的浮点数值，该函数的原型如下所示：

```
void GUI_DispsFloatFix(float v, char Len, char Decs);
```

该函数的相关形参如下表所示：

参数	描述
v	在当前窗口的当前文本位置处，使用当前字体显示指定总字符数和小数点右边字符数的浮点数值
Len	显示的位数（最大为 10）
Decs	小数点右边显示的位数

表 6.1.10 GUI_DispsFloatFix 函数的相关形参描述

返回值：无。

(11) GUI_DispsFloatMin 函数

显示最小字符数的浮点数值，该函数的原型如下所示：

```
void GUI_DispsFloatMin(float v, char Fract);
```

该函数的相关形参如下表所示：

参数	描述
v	显示的数值，最小值为 1.2E-38；最大值为 3.4E38
Fract	显示最少的字符数

表 6.1.11 GUI_DisFloatMin 函数的相关形参描述

返回值：无。

(12) GUI_DisFloatFix 函数

显示指定小数点右边位数的浮点数值并显示符号，该函数的原型如下所示：

```
void GUI_DisFloatFix(float v, char Len, char Decs);
```

该函数的相关形参如下表所示：

参数	描述
v	显示的数值，最小值为 1.2E-38；最大值为 3.4E38
Len	显示的位数（最大为 10）
Decs	小数点右边显示的位数

表 6.1.12 GUI_DisFloatFix 函数的相关形参描述

返回值：无。

(13) GUI_DisFloatMin 函数

显示最小字符数的浮点数值并显示符号，该函数的原型如下所示：

```
void GUI_DisFloatMin(float v, char Fract);
```

该函数的相关形参如下表所示：

参数	描述
v	显示的数值，最小值为 1.2E-38；最大值为 3.4E38
Fract	显示最少的字符数

表 6.1.13 GUI_DisFloatMin 函数的相关形参描述

返回值：无。

(14) GUI_DisBin 函数

在当前位置显示二进制数值，该函数的原型如以所示：

```
void GUI_DisBin(U32 v, U8 Len);
```

该函数的相关形参如下表所示：

参数	描述
v	显示的数值（32 位）
Len	显示的数字位数（包括首位的 0）

表 6.1.14 GUI_DisBin 函数的相关形参描述

返回值：无。

(15) GUI_DisBinAt 函数

在指定位置显示二进制数值，该函数的原型如下所示：

```
void GUI_DisBinAt(U32 v, I16P x, I16P y, U8 Len);
```

该函数的相关形参如下表所示：

参数	描述
v	显示的数值（16 位）
x	要写入的客户端窗口 X 位置（单位：像素）
y	要写入的客户端窗口 Y 位置（单位：像素）
Len	显示的数字位数（包括首位的 0）

表 6.1.15 GUI_DisBinAt 函数的相关形参描述

返回值：无。

(16) GUI_DisHex 函数

在当前位置显示十六进制数值，该函数的原型如下所示：

```
void GUI_DisHex(U32 v, U8 Len);
```

该函数的相关形参如下表所示：

参数	描述
v	显示的数值（16 位）
Len	显示的数字位数

表 6.1.16 GUI_DisPHex 函数的相关形参描述

返回值：无。

(17) GUI_DisPHexAt 函数

在指定位置显示十六进制数值，该函数的原型如下所示：

```
void GUI_DisPHexAt(U32 v, I16P x, I16P y, U8 Len);
```

该函数的相关形参如下表所示：

参数	描述
v	显示的数值（16 位）
x	要写入的客户端窗口 X 位置（单位：像素）
y	要写入的客户端窗口 Y 位置（单位：像素）
Len	显示的数字位数

表 6.1.17 GUI_DisPHexAt 函数的相关形参描述

返回值：无。

6.2 显示十进制数值

emWin 有关十进制数值显示的 API 函数有多种，这些函数都有各自的特点，接下来分别地介绍 emWin 十进制数值显示函数的区别，如下所示：

```
void emwin_main(void)
{
    GUI_Init();
    GUI_SetBkColor(GUI_BLUE);
    GUI_Clear();
    GUI_SetFont(&GUI_Font16_ASCII);
    GUI_DisPStringHCenterAt("ALIENEX NUM DISPLAY",160,0);
    GUI_SetColor(GUI_RED);
    GUI_SetFont(&GUI_Font13_ASCII);

    /* 十进制数值 API 函数测试 */
    GUI_DisPStringAt("GUI_DisPDec(): ",0,20);
    GUI_DisPDec(123,4);
    GUI_DisPString(" ");
    GUI_DisPDec(-123, 4);

    GUI_DisPStringAt("GUI_DisPDecAt(): ", 0, 35);
    GUI_DisPDecAt(100,100,35,3);
    GUI_DisPDecAt(-564, 130, 35, 4);

    GUI_DisPStringAt("GUI_DisPDecShift(): ", 0, 50);
    GUI_DisPDecShift(1236,5,2);
    GUI_DisPString(" ");
    GUI_DisPDecShift(-1236, 6, 2);
}
```

```
GUI_DispStringAt("GUI_DispDecSpace(): ", 0, 65);
GUI_DispDecSpace(3265, 5);

GUI_DispStringAt("GUI_DispSDec(): ", 0, 80);
GUI_DispSDec(200, 4);

GUI_DispStringAt("GUI_DispSDecShift(): ", 0, 95);
GUI_DispSDecShift(12345, 7, 3);
while (1);
}
```

从上述源码可知：我们调用了多个十进制数值函数显示数值，这些函数根据自身的特性对数值进行转换。程序运行结果如下图所示：



图 6.2.1 十进制数值显示

从上图可知：GUI_DispDec 和 GUI_DispDecAt 函数支持首位为 0 的格式，比如，要显示十进制数是“123”，如果设置该函数显示位数为 4，数值位数小于显示位数，则显示出来的是“0123”，其首位用 0 来填充；如果数值为负数，则在该数值前显示一个“-”号。简单来说，这些函数将正负号和小数点也算在数值位数里面。函数 GUI_DispDecShift 显示指定字符数、带小数点的十进制数值，DispDecSpace、DispSDec 和 DispSDecShift 函数不做讲解，大家可以查看本章的 6.1 小节相关介绍。

6.3 显示浮点数值

浮点显示 API 函数演示程序如下：

```
void emwin_main(void)
{
    float f = 123.45678;
    GUI_Init();
    GUI_SetBkColor(GUI_BLUE);
    GUI_Clear();
    GUI_SetFont(&GUI_Font16_ASCII);
    GUI_DispStringHCenterAt("ALIEN EK NUM DISPLAY", 160, 0);
    GUI_SetColor(GUI_RED);
    GUI_SetFont(&GUI_Font13_ASCII);
    /* 浮点数值显示 API 函数演示 */
    GUI_DispStringAt("GUI_DispFloat():\n", 0, 20);
    GUI_DispFloat(f, 9);
}
```

```
GUI_GotoX(100);
GUI_DispFloat(-f, 9);
GUI_DispStringAt("GUI_DispFloatFix():\n", 0, 45);
GUI_DispFloatFix(f, 9, 2);
GUI_GotoX(100);
GUI_DispFloatFix(f, 9, 2);
GUI_DispStringAt("GUI_DispSFloatFix():\n", 0, 70);
GUI_DispSFloatFix(f, 9, 2);
GUI_GotoX(100);
GUI_DispSFloatFix(-f, 9, 2);
GUI_DispStringAt("GUI_DispFloatMin():\n", 0, 95);
GUI_DispFloatMin(f, 3);
GUI_GotoX(100);
GUI_DispFloatMin(-f, 3);
GUI_DispStringAt("GUI_DispSFloatMin():\n", 0, 120);
GUI_DispSFloatMin(f, 3);
GUI_GotoX(100);
GUI_DispSFloatMin(-f, 3);
}
```

程序下载完成之后，运行结果如下图所示：



图 6.3.1 浮点数值 API 函数演示

从上图可知：GUI_DispFloat 函数会把符号也算入位数，然后对数值的最后一位进行四舍五入，最后得出实际显示的数值；GUI_DispFloatFix 函数是根据第二和第三个形参来决定显示实际的数值，如果第三个形参设置为 2，则该数值的小数点位数只能显示两位，其次数值根据第二个形参来显示总位数；如果该数值位数小于总位数，则该显示的数值前面补 0。其他的浮点显示函数也是一样，它们都是根据自身的形参来决定实际显示数值。

6.4 二进制和十六进制数值

emWin 为用户提供了一套进制转换函数，该进制转换函数是由十进制数值转换成二进制或者十六进制数值，极大的方便用户开发。接下来我们分别地介绍进制转换函数的使用，如下源码所示：

```
void emwin_main(void)
{
    int dec = 20;
```



```
GUI_Init();
GUI_SetBkColor(GUI_BLUE);
GUI_Clear();
GUI_SetFont(&GUI_Font16_ASCII);
GUI_DispStringHCenterAt("ALIENEK BIN&HEX NUM DISPLAY",160,0);
GUI_SetColor(GUI_RED);
GUI_SetFont(&GUI_Font13_ASCII);
/* 二进制数值显示 API 函数演示 */
GUI_DispStringAt("GUI_DispBin():      ",0,20);
GUI_DispBin(dec,6);
GUI_DispStringAt("GUI_DispBinAt():  ",0,35);
GUI_DispBinAt(dec,87,35,6);
/* 十六进制数值显示 API 函数演示 */
GUI_DispStringAt("GUI_DispHex():    ",0,50);
GUI_DispHex(dec,2);
GUI_DispStringAt("GUI_DispHexAt(): ",0,65);
GUI_DispHexAt(dec,87,65,2);
while (1);
}
```

关于二进制和十六进制数值的显示，emWin 只提供了 4 个 API 函数，我们掌握它们的其中一个即可，程序运行结果如下图所示：

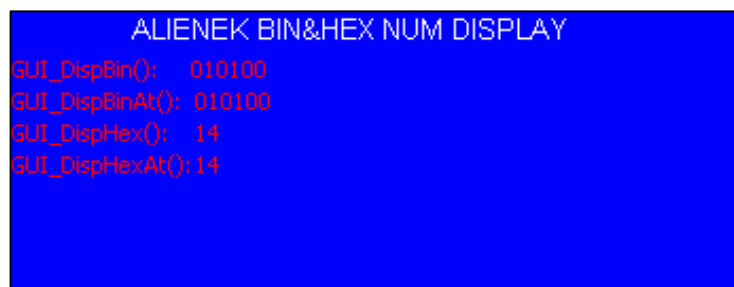


图 6.4.1 二进制和十六进制数值显示

从上图可知：我们调用 GUI_DispBin() 和 GUI_DispBinAt 函数把十进制数值转成二进制，这些函数的唯一的区别在于是否包含指定位置，而 GUI_DispHex() 和 GUI_DispHexAt 函数是把十进制数值转成十六进制，这两个函数的区别也是和上述一样，GUI_DispHexAt 函数的形参包含指定显示位置。

6.5 数值显示实验

6.5.1 硬件设计

1. 例程功能

本实验主要测试 emWin 数值显示 API 函数使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用 emwin_main 函数执行 emWin 程序，该函数主要测试数值显示相关 API 函数，如浮点、十进制传二进制、十进制传十六进制等测试。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。该实验的实验工程，请参考《emWin 实验 4 数值显示》。

6.5.2 软件设计

6.5.2.1 程序流程图

本实验的程序流程图，如下图所示：

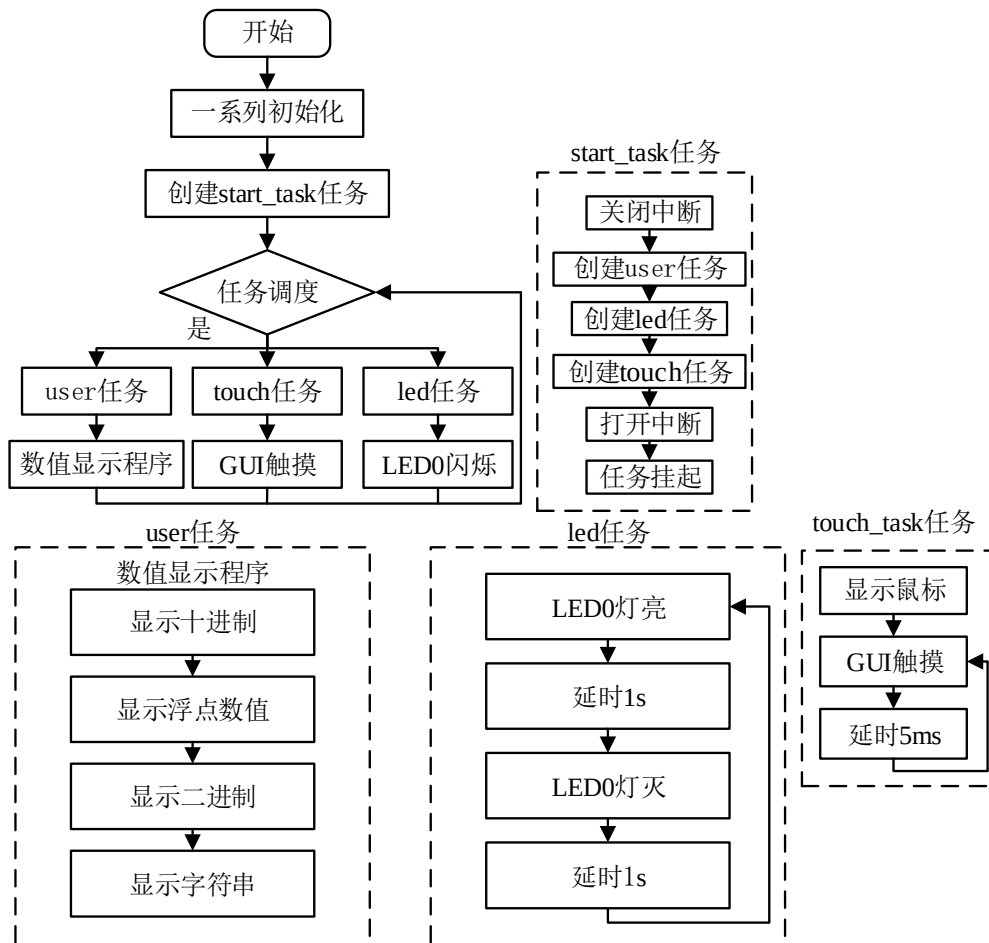


图 6.5.2.1.1 数值显示实验流程图

6.5.2.2 程序解析

本小节着重讲解本实验相关的部分。如下源码所示：

```

void emwin_main(uint16_t x,uint16_t y)
{
    char* emwinversion;                /* emwin 版本号 */
    GUI_SetBkColor(GUI_BLUE);
    GUI_Clear();
    GUI_SetTextMode(GUI_TM_TRANS);    /* 透明显示模式 */
}
    
```

```
GUI_SetFont(&GUI_Font24_ASCII);
GUI_SetColor(GUI_YELLOW);
GUI_DispStringHCenterAt("ALIENTEK NUM DISPLAY DEMO",400,10);

GUI_SetColor(GUI_RED);
GUI_SetFont(&GUI_Font20_ASCII);

GUI_DispStringAt("GUI_DispDecAt():",x,y);
GUI_DispDecAt(200,x+150,y,3);          /* 显示数值 200 */

GUI_DispStringAt("GUI_DispFloat():",x,y+20);
GUI_GotoXY(x+150,y+20);                /* 移动光标 */
GUI_DispFloat(49.35,5);                /* 显示浮点数 49.35 */
GUI_DispString("  ");
GUI_DispFloat(-0.345,6);               /* 显示浮点数-0.345 */

GUI_DispStringAt("GUI_DispBinAt():",x,y+40);
GUI_DispBinAt(8,x+150,y+40,4);        /* 显示十进制数 8 的二进制值 */

GUI_DispStringAt("GUI_DispHex():",x,y+60);
GUI_GotoXY(x+150,y+60);                /* 移动光标 */
GUI_DispHex(100,3);                    /* 显示十进制数 100 的 16 进制值 */
GUI_DispStringAt("GUI_DispHexAt():",x,y+80);
GUI_DispHexAt(100,x+150,y+80,3);      /* 在指定位置显示十进制数 100 的 16 进制值 */
GUI_DispStringAt("GUI_GetVersionString():",x,y+100);
emwinversion = (char*)GUI_GetVersionString(); /* 获取当前 emwin 版本号 */
GUI_DispStringAt(emwinversion,x+200,y+100); /* 显示 emwin 版本号 */
}
```

上述源码主要讲解 emWin 数值显示函数的使用，这些函数无需全部掌握，读者掌握常用的显示函数即可。

6.5.3 下载验证

程序在开发板上的运行结果如下图所示（此处使用的是 4.3 寸 800*480 屏幕）：



图 6.5.3.1 显示效果图

总结：

显示十进制相关的函数：数值的首位不能是 0、数值位数大于数值位数大于（前补 0）、数值位数小于数值位数（错误）。

显示二进制相关的函数：如果位数大于数值位数（前补 0），否则显示实际位数。

显示十六进制相关的函数：如果位数大于数值位数（前补 0），否则显示实际位数。

显示浮点数值相关的函数：数值的首位不能是 0（会错误显示）。

第七章 2D 绘图

emWin 提供有完整的 2D 图形库，对于大多数的应用程序是已经完全足够了，这些 2D 图形库基于快速高效的算法绘制直线、曲线、多边形、饼图、二维码和条形码等图形。

本章分为以下几个部分：

- 7.1 图形库基本 API 简介
- 7.2 其他图形绘制 API 函数
- 7.3 2D 绘图实验

7.1 图形库基本 API 简介

7.1.1 画笔大小

在绘制图形时，我们常常需要不同粗细的线条来绘制图形，因此 emWin 提供了 2 个关于画笔大小的 API 函数，这些函数分别为 GUI_GetPenSize 和 GUI_SetPenSize 函数，一个是获取当前画笔的大小，而另一个是修改画笔的大小，这些函数原型如下所示：

(1) GUI_GetPenSize 函数

获取画笔大小，其原型如下所示：

```
U8 GUI_GetPenSize(void);
```

返回值：为当前的画笔大小。

(2) GUI_SetPenSize 函数

设置画笔大小，其原型如下所示：

```
U8 GUI_SetPenSize(U8 PenSize);
```

该函数的相关形参如下表所示：

参数	描述
PenSize	使用的画笔大小（单位：像素）

表 7.1.1.1 GUI_SetPenSize 函数相关形参描述

返回值：之前的画笔大小。

这两个函数可在绘制图形时，调整画笔的大小让图形按照自己理想的大小绘画。

7.1.2 基本绘图函数

基本绘制图形函数是允许在显示器上的任何位置绘制单独的点、水平线条、垂直线条和形状，这些函数在所有的绘制模式下都可使用。emWin 基本绘制图形函数如下表所示：

函数	描述
GUI_Clear()	清除当前窗口
GUI_ClearRect()	为矩形区域填充背景颜色
GUI_CopyRect()	复制显示器中的一个矩形区域
GUI_DrawGradientH()	绘制用水平颜色梯度填充的矩形
GUI_DrawGradientHEX()	从 GUI_RECT 指针绘制水平颜色梯度
GUI_DrawGradientV()	绘制用垂直颜色梯度填充的矩形
GUI_DrawGradientVEX()	从 GUI_RECT 指针绘制一个垂直的颜色梯度
GUI_DrawGradientRoundedH()	绘制用水平颜色梯度填充的圆角矩形
GUI_DrawGradientRoundedHEX()	绘制一个用水平颜色渐变填充圆角的矩形

GUI_DrawGradientRoundedV()	绘制用垂直颜色梯度填充的圆角矩形
GUI_DrawGradientRoundedVEx()	从 GUI_RECT 指针绘制一个带有圆角的垂直颜色渐变
GUI_DrawGradientMH()	这两个函数绘制一个多颜色梯度水平或垂直
GUI_DrawGradientMV()	
GUI_DrawGradientMHEx()	GUI_RECT 指针绘制一个多颜色梯度水平或垂直
GUI_DrawGradientMVEx()	
GUI_DrawPixel()	绘制单个像素
GUI_DrawPoint()	绘制点
GUI_DrawRect()	绘制矩形
GUI_DrawRectEx()	绘制矩形
GUI_DrawRoundedFrame()	绘制圆角框
GUI_DrawRoundedFrameEx	在当前窗口的指定位置绘制具有指定宽度的圆角框架
GUI_DrawRoundedRect()	绘制圆角矩形
GUI_DrawRoundedRectEx()	在当前窗口的指定位置绘制带有圆角的矩形
GUI_FillRect()	绘制填充的矩形
GUI_FillRectEx()	绘制填充的矩形
GUI_FillRoundedRect()	绘制填充的圆角矩形
GUI_FillRoundedRectEx()	在当前窗口的指定位置绘制带有圆角的填充矩形
GUI_InvertRect()	倒转矩形区域

表 7.1.2.1 基本绘制函数

以下是基本图形绘制函数相关描述。接下来，我们讲解基本图形绘制常用的函数。

(1) GUI_ClearRect 函数

在当前窗口的指定位置清除一个矩形区域，该函数的原型如下所示：

```
void GUI_ClearRect(int x0, int y0, int x1, int y1);
```

该函数的相关形参如下表所示：

参数	描述
X0	左上 X 坐标
Y0	左上 Y 坐标
X1	右下 X 坐标
Y1	右下 Y 坐标

表 7.1.2.2 GUI_ClearRect 函数的相关形参描述

返回值：无。

(2) GUI_CopyRect 函数

将给定矩形区域的内容复制到指定位置，其函数的原型如下所示：

```
void GUI_CopyRect(int x0, int y0, int x1, int y1, int xSize, int ySize);
```

该函数的相关形参如下表所示：

参数	描述
X0	源矩形的左上角 X 位置
Y0	源矩形的左上角 Y 位置
X1	目标矩形的左上角 X 位置
Y1	目标矩形的左上角 Y 位置
xSize	目标矩形的宽度
ySize	矩形的高度

表 7.1.2.3 GUI_CopyRect 函数相关形参描述

返回值：无。

(3) GUI_DrawGradientH 函数

绘制水平颜色梯度填充的矩形，其函数的原型如下所示：

```
void GUI_DrawGradientH(int x0, int y0, int x1, int y1,
    GUI_COLOR Color0, GUI_COLOR Color1);
```

该函数的相关形参如下表所示：

参数	描述
X0	左上 X 坐标
Y0	左上 Y 坐标
X1	右下 X 坐标
Y1	右下 Y 坐标
Color0	矩形最左侧要绘制的颜色
Color1	矩形最右侧要绘制的颜色

表 7.1.2.4 GUI_DrawGradientH 函数相关形参描述

返回值：无。

(4) GUI_DrawGradientHEx 函数

根据 GUI_RECT 指针绘制水平颜色梯度，该函数的原型如下所示：

```
void GUI_DrawGradientVEx(const GUI_RECT * pRect, GUI_COLOR Color0,
    GUI_COLOR Color1);
```

该函数的相关形参如下表所示：

参数	描述
pRect	指向 GUI_RECT 结构的指针
Color0	绘制在矩形最顶边的颜色
Color1	绘制在矩形最底边的颜色

表 7.1.2.5 GUI_DrawGradientHEx 函数的相关形参描述

返回值：无。

(5) GUI_DrawGradientV 函数

绘制用垂直颜色梯度填充的矩形，该函数的原型如下所示：

```
void GUI_DrawGradientV(int x0, int y0, int x1, int y1,
    GUI_COLOR Color0, GUI_COLOR Color1);
```

该函数的相关形参如下表所示：

参数	描述
X0	左上 X 坐标
Y0	左上 Y 坐标
X1	右下 X 坐标
Y1	右下 Y 坐标
Color0	矩形最顶侧要绘制的颜色
Color1	矩形最底侧要绘制的颜色

表 7.1.2.6 GUI_DrawGradientH 函数相关形参描述

返回值：无。

(6) GUI_DrawGradientVEx 函数

绘制用垂直颜色梯度填充的矩形，该函数的原型如下所示：

```
void GUI_DrawGradientVEx(const GUI_RECT *pRect,
    GUI_COLOR Color0, GUI_COLOR Color1);
```

该函数的相关形参如下表所示：

参数	描述
X0	指向 GUI_RECT 结构的指针
Color0	矩形最顶侧要绘制的颜色

Color1	矩形最底侧要绘制的颜色
--------	-------------

表 7.1.2.7 GUI_DrawGradientH 函数相关形参描述

返回值：无。

(7) GUI_DrawGradientRoundedH 函数

绘制用水平颜色梯度填充的圆角矩形，该函数的原型如下所示：

```
void GUI_DrawGradientRoundedH(int x0, int y0, int x1, int y1,
                               int rd, GUI_COLOR Color0, GUI_COLOR Color1);
```

该函数的相关形参如下表所示：

参数	描述
x0	左上 X 坐标
y0	左上 Y 坐标
x1	右下 X 坐标
y1	右下 Y 坐标
rd	要用于圆角的半径
Color0	矩形最左侧要绘制的颜色
Color1	矩形最右侧要绘制的颜色

表 7.1.2.8 GUI_DrawGradientRoundedH 函数的相关形参描述

返回值：无。

(8) GUI_DrawGradientRoundedHEX 函数

绘制一个用水平颜色渐变填充圆角的矩形，其函数原型如下所示：

```
void GUI_DrawGradientRoundedHEX(const GUI_RECT * pRect, int rd,
                                  GUI_COLOR Color0, GUI_COLOR Color1);
```

该函数的相关形参如下表所示：

参数	描述
pRect	指向 GUI_RECT 结构的指针
rd	要用于圆角的半径
Color0	矩形最左侧要绘制的颜色
Color1	矩形最右侧要绘制的颜色

表 7.1.2.9 GUI_DrawGradientRoundedHEX 函数相关形参描述

返回值：无。

(9) GUI_DrawGradientRoundedV 函数

绘制用垂直颜色梯度填充的圆角矩形，其函数的原型如下所示：

```
void GUI_DrawGradientRoundedV(int x0, int y0, int x1, int y1, int rd,
                                GUI_COLOR Color0, GUI_COLOR Color1);
```

该函数的形参如下表所示：

参数	描述
x0	左上 X 坐标
y0	左上 Y 坐标
x1	右下 X 坐标
y1	右下 Y 坐标
rd	圆角使用的半径
Color0	矩形最顶侧要绘制的颜色
Color1	矩形最底侧要绘制的颜色

表 7.1.2.10 GUI_DrawGradientRoundedV 函数相关形参描述

返回值：无。

(10) GUI_DrawGradientRoundedVEx 函数

根据 GUI_RECT 指针绘制一个带有圆角的垂直颜色渐变，该函数的原型如下所示：

```
void GUI_DrawGradientRoundedVEx(    const GUI_RECT * pRect,int rd,
                                     GUI_COLOR Color0,GUI_COLOR Color1);
```

其相关形参如下表所示：

参数	描述
pRect	指向 GUI_RECT 结构的指针
rd	圆角使用的半径
Color0	矩形最顶侧要绘制的颜色
Color1	矩形最顶侧要绘制的颜色

表 7.1.2.11 GUI_DrawGradientRoundedVEx 函数相关形参描述

返回值：无。

(11) GUI_DrawGradientMH、GUI_DrawGradientMV 函数

绘制一个多颜色水平或垂直梯度，该函数的原型如下所示：

```
void GUI_DrawGradientMH(int x0,int y0,int y1,
                        GUI_GRADIENT_INFO * pGradientInfo,
                        int NumColors);
void GUI_DrawGradientMV(int x0,int y0,int x1,
                        GUI_GRADIENT_INFO * pGradientInfo,
                        int NumColors);
```

这两个函数的形参是一样的，如下表所示：

参数	描述
x0	左上角的 X 坐标
y0	左上角 Y 坐标
x1	右下方的 X 坐标
y1	右下角 Y 坐标
pGradientInfo	指向 GUI_GRADIENT_INFO 结构的指针
NumColors	pGradientInfo 中的颜色数目

表 7.1.2.12 GUI_DrawGradientMH 函数、GUI_DrawGradientMV 函数相关形参描述

返回值：无。

注意：GUI_GRADIENT_INFO 结构的成员 Pos 用于定义颜色的开始和结束位置，它还定义了梯度的大小。

该函数使用方式如下：

```
GUI_GRADIENT_INFO aInfo[] = {
    {0, GUI_RED},
    {60, GUI_GREEN},
    {120, GUI_BLUE},}; /* 产生一个大小为 120 像素的渐变 */

void emwin_main(void)
{
    GUI_Init();
    GUI_DrawGradientMH(0, 0, 120, &aInfo[0], GUI_COUNTOF(aInfo));

    while (1)
    {
```

```

        GUI_Delay(100);
    }
}

```

上述的源码主要讲解 GUI_GRADIENT_INFO 结构体的成员变量如何设置，该结构主要定义颜色的开始和结束的位置，根据这些成员变量绘画多颜色的水平或垂直梯度。

前面我们已经讲解了多个 2D 图形绘画函数，这些函数的使用我们就以一个简单的实例来讲解，程序代码如下：

```

void emwin_main(void)
{
    GUI_Init();
    GUI_SetBkColor(GUI_BLUE);
    GUI_Clear();
    GUI_SetFont(&GUI_Font16_ASCII);
    GUI_DispStringHCenterAt("ALIENK BIN&HEX NUM DISPLAY", 160, 0);
    GUI_SetColor(GUI_RED);
    GUI_SetFont(&GUI_Font13_ASCII);
    /* 基本绘制例程函数演示 */
    GUI_SetBkColor(GUI_YELLOW);
    GUI_ClearRect(5, 20, 95, 110); /* 此函数使用背景颜色填充 */
    GUI_SetBkColor(GUI_BLUE);
    /* 绘制水平颜色梯度填充的矩形 */
    GUI_DrawGradientH(100, 20, 190, 110, 0X4117BB, 0XC6B6F5);
    /* 绘制垂直颜色梯度填充的圆角矩形 */
    GUI_DrawGradientRoundedV(195, 20, 285, 110, 25, 0X4117BB, 0XC6B6F5);
    GUI_DrawRect(5, 115, 95, 205); /* 在指定位置绘制矩形 */
    GUI_FillRect(100, 115, 190, 205); /* 在指定位置绘制填充的矩形区域 */
    GUI_FillRoundedRect(195, 115, 285, 205, 25); /* 在指定位置绘制填充的圆角矩形 */
    while(1);
}

```

上面程序展示了几个基本绘制函数的使用，在 PC 模拟器上的运行效果如下图所示：

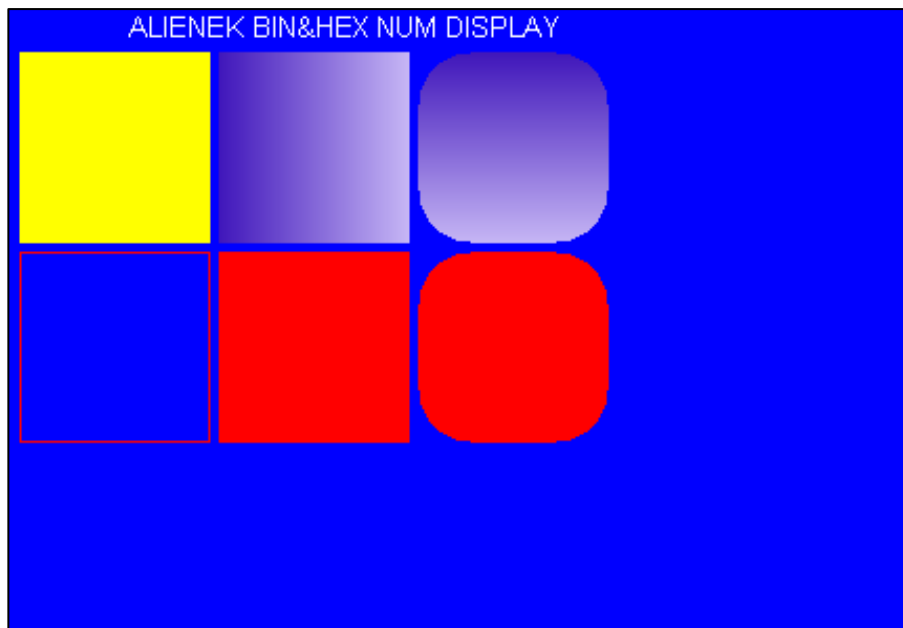


图 7.1.2.1 基本绘制函数演示

7.1.3 Alpha 混色

Alpha 混合是一种合并前景图像与背景来创建半透明效果的方法。Alpha 值决定可见的像素数以及背景透露的像素数。emWin 能够使用 2 种不同的颜色格式（虽然说 emWin 在 5.30 版本之后，会支持 ARGB 和 ABGR 两种颜色格式，但是 MDK 提供的 emWin 库仅支持 ABGR 颜色格式），Alpha 混合完全自动执行，唯一需要做的就是使用函数 GUI_EnableAlpha 启用 Alpha 混合功能。

emWin 的 32 位颜色信息：

- ✧ 0~7 位：红色。
- ✧ 8~15 位：绿色。
- ✧ 16~23 位：蓝色。
- ✧ 24~31 位：Alpha 混色。

从上述可知，如果用户想使用 Alpha 混色，则必须操作颜色位的 24~31 位；如果 Alpha 值为 0，则表示不透明；如果 Alpha 值为 255，则表示完全透明。Alpha 的 API 函数如下表所示：

函数	描述
GUI_EnableAlpha()	启用 Alpha
GUI_PreserveTrans()	启用透明保存
GUI_RestoreUserAlpha()	恢复之前的用户
GUI_SetAlpha()	设置 alpha 值
GUI_SetUserAlpha()	设置其他值，以用于计算要使用的实际

表 7.1.3.1 Alpha 混色 API 函数

以下是表 7.1.3.1 的 Alpha 混色函数的相关信息。

(1) GUI_EnableAlpha 函数

启用或禁用自动 Alpha 混合，该函数的原型如下源码所示：

```
unsigned GUI_EnableAlpha(unsigned OnOff);
```

其函数的相关形参如下表所示：

参数	描述
OnOff	1: 启用自动 Alpha 混合, 0: 禁用

表 7.1.3.2 GUI_EnableAlpha 函数的相关形参描述

返回值: 旧状态。

(2) GUI_PreserveTrans 函数

将帧缓冲器的内容与项目颜色进行混合, 该函数的原型如下源码所示:

```
unsigned GUI_PreserveTrans(unsigned OnOff);
```

其函数的相关形参如下表所示:

参数	描述
OnOff	1 启用透明保护, 0 禁用

表 7.1.3.3 GUI_PreserveTrans 函数相关形参描述

返回值: 旧状态。

(3) GUI_RestoreUserAlpha 函数

恢复用户 alpha 混合之前的状态, 该函数的原型如下源码所示:

```
U32 GUI_RestoreUserAlpha(GUI_ALPHA_STATE * pAlphaState);
```

其函数的相关形参如下表所示:

参数	描述
pAlphaState	指向要还原的前一个状态信息的 GUI_ALPHA_STATE 结构的指针

表 7.1.3.4 GUI_RestoreUserAlpha 函数的相关形参描述

返回值: 当前用户 alpha 值。

(4) GUI_SetAlpha 函数

后续绘制操作启用软件 Alpha 混合, 该函数的原型如下源码所示:

```
unsigned GUI_SetAlpha(U8 Value);
```

其函数的相关形参如下表所示:

参数	描述
Value	用于所有后续绘制操作的 Alpha 值。默认为 0, 表示无 Alpha 混合。

表 7.1.3.5 GUI_SetAlpha 函数相关形参描述

返回值: Alpha 混合之前使用的值。

(5) GUI_SetUserAlpha 函数

设置其他值以用于计算要使用的实际 Alpha 值。实际 Alpha 值的计算方式如下:

$$\text{Alpha} = \text{AlphaFromObject} + ((255 - \text{AlphaFromObject}) * \text{UserAlpha}) / 255$$

该函数的原型如下源码所示:

```
U32 GUI_SetUserAlpha(GUI_ALPHA_STATE * pAlphaState, U32 UserAlpha);
```

其函数相关形参如下表所示:

参数	描述
pAlphaState	指向用于保存当前状态的 GUI_ALPHA_STATE 结构的指针。
UserAlpha	要会用的值。

表 7.1.3.6 GUI_SetUserAlpha 函数相关形参描述

返回值: 之前的用户 Alpha 值。

在 GUI_SetUserAlpha 函数的形参 GUI_ALPHA_STATE 结构如下表所示:

数据类型	成员	描述
U32	UserAlpha	要使用的 alpha 值。

表 7.1.3.7 GUI_ALPHA_STATE 结构描述

下面我们以简单的实例来讲解 Alpha 混色函数的使用, 如下源码所示:

```
void emwin_main(void)
```

```
{
    GUI_Init();
    GUI_SetBkColor(GUI_WHITE);
    GUI_Clear();
    GUI_SetFont(&GUI_Font16_ASCII);
    GUI_SetColor(GUI_BLACK);
    GUI_DispStringHCenterAt("ALIENEK ALPHA DISPLAY", 160, 0);
    GUI_EnableAlpha(1);          /* 开启 Alpha */
    GUI_SetColor(GUI_BLACK);
    GUI_DispStringHCenterAt("Alphablending", 45, 61);
    GUI_SetAlpha(40);            /* 设置 Alpha 值 */
    GUI_SetColor(GUI_RED);
    GUI_FillRect(0, 20, 49, 69);
    GUI_SetAlpha(80);            /* 设置 Alpha 值 */
    GUI_SetColor(GUI_GREEN);
    GUI_FillRect(20, 40, 69, 89);
    GUI_SetAlpha(40);            /* 设置 Alpha 值 */
    GUI_SetColor(GUI_BLUE);
    GUI_FillRect(40, 60, 89, 109);
    GUI_EnableAlpha(0);          /* 关闭 Alpha */
    while (1);
}
```

从上述源码可知，我们调用函数 GUI_EnableAlpha 开启 Alpha 混色，然后调用 GUI_FillRect 填充三个红色、蓝色、绿色矩形，最后我们调用函数 GUI_EnableAlpha 关闭 Alpha 混色，程序效果图如下图所示：

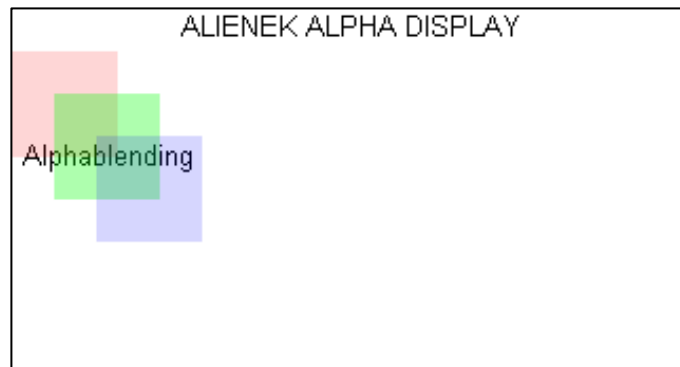


图 7.1.3.1 Alpha 混色演示

上图中，我们填充了三个矩形，它们的背景颜色分别为红、绿、蓝，这三个矩形的交集处会以混合颜色显示，这些混合颜色是开启 Alpha 混色的效果。

7.2 其他图形绘制 API 函数

7.2.1 绘制线条

在平常的使用中，最常用的绘制函数是绘制从一点到另一点的线的函数，我们在很多地方需要绘制线条，emWin 中提供了很多关于绘制线条的 API 函数，我们可使用这些函数方便的绘制出各种的线条，这些 API 函数如下表所示：

函数	描述
GUI_DrawHLine()	绘制水平线
GUI_DrawLine()	绘制从某个指定起点到指定终点的线条(绝对坐标)
GUI_DrawLineRel()	绘制从当前位置到按 X 和 Y 距离指定的终点的线条（相对坐标）
GUI_DrawLineTo()	绘制从当前位置到指定终点的线条
GUI_DrawPolyLine()	绘制折线
GUI_DrawVLine()	绘制垂直线
GUI_GetLineStyle()	返回当前的线条样式
GUI_MoveRel()	相对于其当前位置移动线条指针
GUI_MoveTo()	将线条指针移动到给定位置
GUI_SetLineStyle()	设置当前的线条样式

表 7.2.1.1 绘制线条 API 函数

以上是表 7.2.1.1 的基本绘制线条函数的相关信息。接下来，我们来讲解 emWin 基本绘制线条常用的函数。

(1) GUI_DrawHLine 函数

从某个起点到某个终点绘画线条(绘画水平线)，该函数的原型如下源码所示：

```
void GUI_DrawHLine(int y, int x0, int x1);
```

其函数的相关形参如下表所示：

参数	描述
y	Y 坐标
x0	X 起始坐标
x1	X 结束坐标

表 7.2.1.2 GUI_DrawHLine 函数的相关形参描述

如果 $x1 < x0$ ，则不会显示任何内容。

(2) GUI_DrawLine 函数

根据两个坐标点绘画线条，该函数的原型如下源码所示：

```
void GUI_DrawLine(int x0, int y0, int x1, int y1);
```

其函数的相关形参如下表所示：

参数	描述
x0	X 起始坐标
y0	Y 起始坐标
x0	X 起始坐标
x1	X 结束坐标

表 7.2.1.3 GUI_DrawLine 函数的相关形参描述

(3) GUI_DrawLineRel 函数

从当前(X,Y) 的位置到按 X 和 Y 距离指定的终点绘画条线(相对坐标)，该函数的原型如下源码所示：

```
void GUI_DrawLineRel(int dx, int dy);
```

其函数的相关形参如下表所示：

参数	描述
dx	X 方向到要绘制的线的终点的距离
dy	Y 方向到要绘制的线的终点的距离

表 7.2.1.4 GUI_DrawLineRel 函数的相关形参描述

(4) GUI_DrawLineTo 函数

从当前(X,Y) 位置到按 X 和 Y 坐标绘画线条，该函数的原型如下源码所示：

```
void GUI_DrawLineTo(int x, int y);
```

其函数的相关形参如下表所示：

参数	描述
x	X 结束坐标
y	Y 结束坐标

表 7.2.1.5 GUI_DrawLineTo 函数的相关形参描述

(5) GUI_DrawPolyLine 函数

用线条连接预定义的点列表，该函数的原型如下源码所示：

```
void GUI_DrawPolyLine(const GUI_POINT * pPoint, int NumPoints, int x, int y);
```

其函数的相关形参如下表所示：

参数	描述
pPoint	指向要显示的多边形的指针。
NumPoints	在点列表中指定的点数。
x	初始 X 坐标
y	初始 Y 坐标

表 7.2.1.6 GUI_DrawPolyLine 函数的相关形参描述

使用该函数时，折线的起点和终点可以不需要一样的数值。

(6) GUI_DrawVLine 函数

从某个起点到某个终点绘画线条(绘画垂直线)，该函数的原型如下源码所示：

```
void GUI_DrawVLine( int x, int y0, int y1 );
```

其函数的相关形参如下表所示：

参数	描述
x	X 坐标
y0	Y 起始坐标
y1	Y 结束坐标

表 7.2.1.7 GUI_DrawVLine 函数的相关形参描述

如果 $y1 < y0$ ，则不会显示任何内容。

(7) GUI_MoveRel 函数

当前位置移动当前的线条指针，该函数的原型如下源码所示：

```
void GUI_MoveRel( int dx, int dy);
```

其函数的相关形参如下表所示：

参数	描述
dx	X 方向移动的距离
dy	Y 方向移动的距离

表 7.2.1.8 GUI_MoveRel 函数的相关形参描述

(8) GUI_MoveTo 函数

将当前的线条指针移动到给定位置，该函数的原型如下源码所示：

```
void GUI_MoveTo( int x, int y);
```

其函数的相关形参如下表所示：

参数	描述
x	X 新的坐标
y	Y 新的坐标

表 7.2.1.9 GUI_MoveTo 函数的相关形参描述

(9) GUI_SetLineStyle 函数

设置 GUI_DrawLine 函数当前使用的线条样式，该函数的原型如下源码所示：

```
U8 GUI_SetLineStyle(U8 LineStyle);
```

其函数的相关形参如下表所示：

参数	描述	
LineStyle	要使用的新线条样式	
	GUI_LS_SOLID	按实线样式绘制线条（默认）
	GUI_LS_DASH	按虚线样式绘制线条
	GUI_LS_DOT	按圆点样式绘制线条
	GUI_LS_DASHDOT	按虚线和圆点交替的样式绘制线条
	GUI_LS_DASHDOT	按虚线和双点交替的样式绘制线条

表 7.2.1.10 GUI_SetLineStyle 函数的相关形参描述

返回值：之前 GUI_DrawLine 函数使用的线条样式。

下面我们在 PC 上仿真展示一下这些函数的实际效果，程序如下：

```
const GUI_POINT Points[] =
{
    {10,220},
    {56,100},
    {102,220},
    {148,100},
    {194,220}
};

void emwin_main(void)
{
    GUI_Init();
    GUI_SetBkColor(GUI_BLUE);
    GUI_Clear();
    GUI_SetFont(&GUI_Font16_ASCII);
    GUI_SetColor(GUI_RED);

    GUI_DispStringHCenterAt("ALIENEK DRAW LINE DISPLAY", 160, 0);

    GUI_DrawHLine(30,10,250);          /* 绘制一条水平线 */

    GUI_SetPenSize(3);
    GUI_DrawLine(10,50,250,50);        /* 绘制一条指定起点和终点的直线 */
    GUI_SetPenSize(1);

    GUI_SetLineStyle(GUI_LS_DASH);      /* 设置线条样式为虚线 */
}
```



```
GUI_DrawLine(10, 70, 250, 70); /* 绘制一条指定起点和终点的直线 */
GUI_SetLineStyle(GUI_LS_SOLID); /* 设置线条样式为实线 */

GUI_DrawVLine(300,30,239); /* 绘制垂直线*/

GUI_SetPenSize(2);
GUI_DrawPolyLine(&Points,GUI_COUNTOF(Points),0,0); /* 连接预定的点列表*/

while (1);
}
```

从上述源码可知，首先我们调用函数 GUI_DrawHLine 绘画水平线，然后根据画笔的大小和线条的样式（虚线和实线等）绘画线条，最后调用函数 GUI_DrawPolyLine 根据点的列表绘画线条。程序效果图如下图所示：

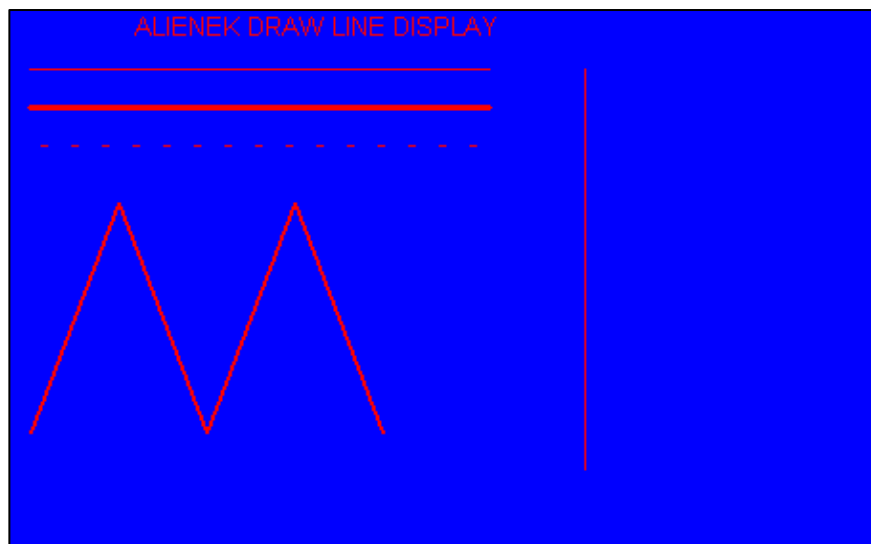


图 7.2.1.1 线条绘制演示

7.2.2 其他图形绘制函数

除了绘画图形和线条之外，emWin 还提供了其他图形绘制函数，比如绘制多边形、圆、椭圆、弧线、线图和饼图等 API 函数，如下表所示：

函数	描述
绘制多边形	
GUI_DrawPolygon()	绘制多边形的轮廓
GUI_EnlargePolygon()	扩展多边形
GUI_FillPolygon()	绘制填充的多边形
GUI_MagnifyPolygon()	放大多边形
GUI_RotatePolygon()	按指定角度旋转多边形
绘制圆	
GUI_DrawCircle()	绘制圆的轮廓
GUI_FillCircle()	绘制填充的圆
绘制椭圆	
GUI_DrawEllipse()	绘制椭圆的轮廓

GUI_DrawEllipseXL()	画一个大椭圆的轮廓
GUI_FillEllipse()	绘制填充的椭圆
绘制弧线	
GUI_DrawArc()	绘制弧线
绘制曲线	
GUI_DrawGraph()	绘制曲线
绘制饼图	
GUI_DrawPie()	绘制圆形扇区
绘制条形码	
GUI_BARCODE_Draw()	绘制条形码
GUI_BARCODE_GetXSize()	返回给定条形码的 x 大小，而不绘制它
绘制二维码	
GUI_QR_Create()	绘制的 QR-code 位图
GUI_QR_CreateFramed()	绘制的 QR-code 位图并添加一个模块大小的白色框架
GUI_QR_Delete()	释放用于 QR-code 的内存
GUI_QR_Draw()	在给定的像素位置绘制给定的 QR-code
GUI_QR_GetInfo()	返回包含给定 QT-code 信息的结构

表 7.2.2.1 其他图形绘制 API 函数

下面我们根据上表的函数进行分类讲解常用的图形绘画函数。

1. 绘制多边形

(1) GUI_DrawPolygon 函数

按点列表定义绘画多边形，该函数的原型如下源码所示：

```
void GUI_DrawPolygon( const GUI_POINT * pPoint, int NumPoints, int x, int y);
```

其函数的相关形参如下表所示：

参数	描述
pPoint	指向要显示的多边形的指针
NumPoints	在点列表中指定的点数
x	初始 X 坐标
y	初始 Y 坐标

表 7.2.2.2 GUI_DrawPolygon 函数的相关形参描述

通过将终点连接到起点，绘制的多边形将自动闭合。

(2) GUI_EnlargePolygon 函数

按指定的长度(像素)全方位扩展多边形，该函数的原型如下源码所示：

```
void GUI_EnlargePolygon( GUI_POINT * pDest, const GUI_POINT * pSrc,
                        int NumPoints, int Len);
```

其函数的相关形参如下表所示：

参数	描述
pDest	指向目标多边形的指针
pSrc	指向源多边形的指针
NumPoints	在点列表中指定的点数
Len	扩展多边形的长度（像素）

表 7.2.2.3 GUI_EnlargePolygon 函数的相关形参描述

使用该函数时，必须确保点的点数组等于或大于源数组。

(3) GUI_FillPolygon 函数

按点列表的定义填充的多边形，该函数的原型如下源码所示：

```
void GUI_FillPolygon( const GUI_POINT * pPoint, int NumPoints, int x, int y);
```

其函数的相关形参如下表所示：

参数	描述
pPoint	指向要显示和填充的多边形的指针
NumPoints	在点列表中指定的点数
x	初始 X 坐标
y	初始 Y 坐标

表 7.2.2.4 GUI_FillPolygon 函数的相关形参描述

通过将终点连接到起点，绘制的多边形将自动闭合。终点无需接触到多边形的轮廓。通过多边形的各个 y 位置绘制一条或多条水平直线，可对多边形进行渲染。默认情况下，一个 y 位置绘制水平直线使用的最大点数量为 12(意味着每个 y 位置需要 6 条直线)。如果需要增加此值，可使用宏 GUI_FP_MAXCOUNT 设置最大点数量。

(4) GUI_MagnifyPolygon 函数

指定系数放大多边形，该函数的原型如下源码所示：

```
void GUI_MagnifyPolygon( GUI_POINT * pDest, const GUI_POINT * pSrc,
                        int NumPoints,int Mag);
```

其函数的相关形参如下表所示：

参数	描述
pDest	指向目标多边形的指针
pSrc	指向源多边形的指针
NumPoints	在点列表中指定的点数
Mag	用于放大多边形的系数

表 7.2.2.5 GUI_MagnifyPolygon 函数的相关形参描述

(5) GUI_RotatePolygon 函数

指定角度旋转多边形，该函数的原型如下源码所示：

```
void GUI_RotatePolygon(GUI_POINT * pDest, const GUI_POINT * pSrc,
                       int NumPoints,float Angle);
```

其函数的相关形参如下表所示：

参数	描述
pDest	指向目标多边形的指针
pSrc	指向源多边形的指针
NumPoints	在点列表中指定的点数
Angle	用于旋转多边形的角度（弧度）

表 7.2.2.6 GUI_RotatePolygon 函数的相关形参描述

2. 绘制圆

(1) GUI_DrawCircle 函数

指定位置绘制圆，该函数的原型如下源码所示：

```
void GUI_DrawCircle(int x0, int y0, int r);
```

其函数的相关形参如下表所示：

参数	描述
x0	客户端窗口的圆心的 X 位置（像素）。
y0	客户端窗口的圆心的 Y 位置（像素）。
r	圆的半径（直径的一半）。必须为正数。

表 7.2.2.7 GUI_DrawCircle 函数的相关形参描述

(2) GUI_FillCircle()

指定位置填充圆形，该函数的原型如下源码所示：

```
void GUI_FillCircle(int x0, int y0, int r);
```

其函数的相关形参如下表所示:

参数	描述
x0	客户端窗口的圆心的 X 位置 (像素)
y0	客户端窗口的圆心的 Y 位置 (像素)
r	圆的半径 (直径的一半)

表 7.2.2.8 GUI_FillCircle 函数的相关形参描述

3. 绘制椭圆

(1) GUI_DrawEllipse 函数

指定位置绘制椭圆, 该函数的原型如下源码所示:

```
void GUI_DrawEllipse(int x0, int y0, int rx, int ry);
```

其函数的相关形参如下表所示:

参数	描述
x0	客户端窗口的圆心的 X 位置 (像素)
y0	客户端窗口的圆心的 Y 位置 (像素)
rx	椭圆的 X 半径 (直径的一半)
ry	椭圆的 Y 半径 (直径的一半)

表 7.2.2.9 GUI_DrawEllipse 函数的相关形参描述

(2) GUI_DrawEllipseXL 函数

绘制一个大椭圆, 该函数的原型如下源码所示:

```
void GUI_DrawEllipseXL(int x0,int y0,int rx,int ry);
```

其函数的相关形参如下表所示:

参数	描述
x0	客户端窗口中圆心的 x 位置(以像素为单位)
y0	客户端窗口中圆心的 y 位置(以像素为单位)
rx	椭圆的 x 半径(直径的一半)
ry	椭圆的 y 半径(直径的一半)

表 7.2.2.10 GUI_DrawEllipseXL 函数的相关形参描述

(3) GUI_FillEllipse 函数

指定位置填充一个椭圆, 该函数的原型如下源码所示:

```
void GUI_FillEllipse(int x0, int y0, int rx, int ry);
```

其函数的相关形参如下表所示:

参数	描述
x0	客户端窗口中圆心的 x 位置(以像素为单位)
y0	客户端窗口中圆心的 y 位置(以像素为单位)
rx	椭圆的 x 半径(直径的一半)
ry	椭圆的 y 半径(直径的一半)

表 7.2.2.11 GUI_FillEllipse 函数的相关形参描述

4. 绘制弧线

(1) GUI_DrawArc 函数

指定位置绘制弧线。该函数的原型如下源码所示:

```
void GUI_DrawArc(int xCenter, int yCenter, int rx, int ry, int a0, int a1);
```

其函数的相关形参如下表所示:

参数	描述
xCenter	客户端窗口中心的水平位置 (像素)

yCenter	客户端窗口中心的垂直位置（像素）
rx	X 半径（像素）
ry	Y 半径（像素）
a0	起始角度（度）
a1	结束角度（度）

表 7.2.2.12 GUI_FillEllipse 函数的相关形参描述

5. 绘制曲线

(1) GUI_DrawGraph 函数

绘制曲线，该函数的原型如下源码所示：

```
void GUI_DrawGraph(I16 * paY, int NumPoints, int x0, int y0);
```

其函数的相关形参如下表所示：

参数	描述
paY	指向包含曲线 Y 值的数组
NumPoints	要显示的 Y 值的数量
x0	X 的起始点
y0	Y 的起始点

表 7.2.2.13 GUI_DrawGraph 函数的相关形参描述

6. 绘制饼图

(1) GUI_DrawPie 函数

指定位置填充一个椭圆，该函数的原型如下源码所示：

```
void GUI_FillEllipse(int x0, int y0, int rx, int ry);
```

其函数的相关形参如下表所示：

参数	描述
x0	客户端窗口中圆心的 x 位置（以像素为单位）
y0	客户端窗口中圆心的 y 位置（以像素为单位）
rx	椭圆的 x 半径(直径的一半)，必须是正数
ry	椭圆的 y 半径(直径的一半)，必须是正数

表 7.2.2.14 GUI_FillEllipse 函数的相关形参描述

7. 绘制饼图

(1) GUI_DrawPie 函数

绘制圆形扇区，该函数的原型如下源码所示：

```
void GUI_DrawPie(int x0, int y0, int r, int a0, int a1, int Type);
```

其函数的相关形参如下表所示：

参数	描述
x0	客户端窗口的圆心的 X 位置（像素）
y0	客户端窗口的圆心的 Y 位置（像素）
r	圆的半径（直径的一半）
a0	起始角度（度）
a1	结束角度（度）
Type	留待将来使用，应为 0

表 7.2.2.15 GUI_FillEllipse 函数的相关形参描述

8. 绘制条形码

(1) GUI_BARCODE_Draw 函数

指定位置绘制给定类型的条形码，该函数的原型如下源码所示：

```
int GUI_BARCODE_Draw( int xPos, int yPos, int ModuleSize, int ySize, int Type,
```

```
const char *sBarcode);
```

其函数的相关形参如下表所示:

参数	描述	
xPos	使用 x 位置	
yPos	使用 y 位置	
ModuleSize	一个模块的大小, 即最小像素条的宽度	
ySize	条形码的 y 大小。不能低于 ModuleSize 大小的三倍	
Type	GUI_BARCODE_ITF	它可以显示偶数数量的数字
	GUI_BARCODE_128	可以显示所有 128 个 ASCII 字符的 Code128
sBarcode	包含要用于条形码的数据的字符串。ITF 条码只能显示数字	

表 7.2.2.16 GUI_BARCODE_Draw 函数的相关形参描述

返回值: 0 成功, -1 失败。

(2) GUI_BARCODE_GetXSize 函数

返回条形码的 x 大小, 该函数的原型如下源码所示:

```
int GUI_BARCODE_GetXSize( int Type, int ModuleSize, const char *sBarcode);
```

其函数的相关形参如下表所示:

参数	描述	
Type	要绘制的条形码的类型。	
	GUI_BARCODE_ITF	它可以显示偶数数量的数字
	GUI_BARCODE_128	可以显示所有 128 个 ASCII 字符的 Code128
ModuleSize	一个模块的大小, 即最小像素条的宽度	
sBarcode	包含要用于条形码的数据的字符串。ITF 条码只能显示数字	

表 7.2.2.17 GUI_BARCODE_GetXSize 函数的相关形参描述

返回值: 返回条形码大小。

9. 绘制二维码

(1) GUI_QR_Create 函数

创建一个二维码位图, 其函数的原型如下源码所示:

```
GUI_HMEM GUI_QR_Create(const char *pText, int PixelSize, int EccLevel,  
int Version);
```

该函数的相关参数描述如下表所示:

参数	描述	
pText	用于二维码的 UTF-8 文本	
PixelSize	二维码的像素大小	
EccLevel	要使用的纠错电平	
	GUI_QR_ECLEVEL_L	大约 7%或更少的错误可以被纠正
	GUI_QR_ECLEVEL_M	大约 15%或更少的错误可以被纠正
	GUI_QR_ECLEVEL_Q	大约 25%或更少的错误可以被纠正
	GUI_QR_ECLEVEL_H	大约 30%或更少的错误可以被纠正
Version	设置为 0(推荐), 大小将自动计算	

表 7.2.2.18 GUI_QR_Create 函数相关形参描述

返回值: 成功: 二维码句柄, 0 为错误。

(2) GUI_QR_CreateFramed 函数

创建一个二维码位图, 可以用 GUI_QR_Draw()绘制。将添加一个模块大小的白色框架。

该函数的原型如下源码所示:

```
GUI_HMEM GUI_QR_CreateFramed( const char *pText, int PixelSize,  
int EccLevel, int Version);
```

该函数的相关参数描述如下表所示：

参数	描述
pText	用于二维码的 UTF-8 文本
PixelSize	二维码的像素大小
EccLevel	要使用的纠错电平。关于 QR 码如以下所示：
	GUI_QR_ECLEVEL_L 大约 7%或更少的错误可以被纠正
	GUI_QR_ECLEVEL_M 大约 15%或更少的错误可以被纠正
	GUI_QR_ECLEVEL_Q 大约 25%或更少的错误可以被纠正
	GUI_QR_ECLEVEL_H 大约 30%或更少的错误可以被纠正
Version	设置为 0(推荐)，大小将自动计算

表 7.2.2.19 GUI_QR_CreateFramed 函数相关形参描述

返回值：成功：二维码句柄，0 为错误。

(3) GUI_QR_Delete 函数

删除二维码，该函数原型如下源码所示：

```
void GUI_QR_Delete(GUI_HMEM hQR);
```

返回值：无。

(4) GUI_QR_Draw 函数

绘制二维码，该函数原型如下源码所示：

```
void GUI_QR_Draw(GUI_HMEM hQR, int xPos, int yPos);
```

该函数的相关参数描述如下表所示：

参数	描述
hQR	要绘制的二维码的句柄
xPos	x 位置
yPos	y 位置

表 7.2.2.20 GUI_QR_Draw 函数相关形参描述

返回值：无。

(5) GUI_QR_GetInfo 函数

返回二维码的信息结构，该函数的原型如下源码所示：

```
void GUI_QR_GetInfo(GUI_HMEM hQR, GUI_QR_INFO *pInfo);
```

该函数的相关参数描述如下表所示：

参数	描述
hQR	二维码的句柄
pInfo	GUI_QR_INFO 的结构体的指针

表 7.2.2.21 GUI_QR_GetInfo 函数的参数描述

GUI_QR_INFO 的结构，如下源码所示：

```
typedef struct
{
    int Version; /* 二维码的版本 */
    int Width; /* 宽度 */
    int Size; /* 像素的大小 */
} GUI_QR_INFO
```

emWin 官方的仿真文件给我们提供了很多这些 API 函数的例程，大家可以运行一下这些程序，观察一下这些函数的运行结果，这里就不一一给大家展示了。

7.3 2D 绘图实验

7.3.1 硬件设计

1. 例程功能

本实验主要测试 emWin2D 绘图 API 函数使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用 emwin_main 函数执行 emWin 程序，该函数主要测试 2D 图形相关 API 函数，如绘制多边形、圆形、条形码和二维码等测试。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 5 2D 绘图》。

7.3.2 软件设计

7.3.2.1 程序流程图

本实验的程序流程图如下图所示：

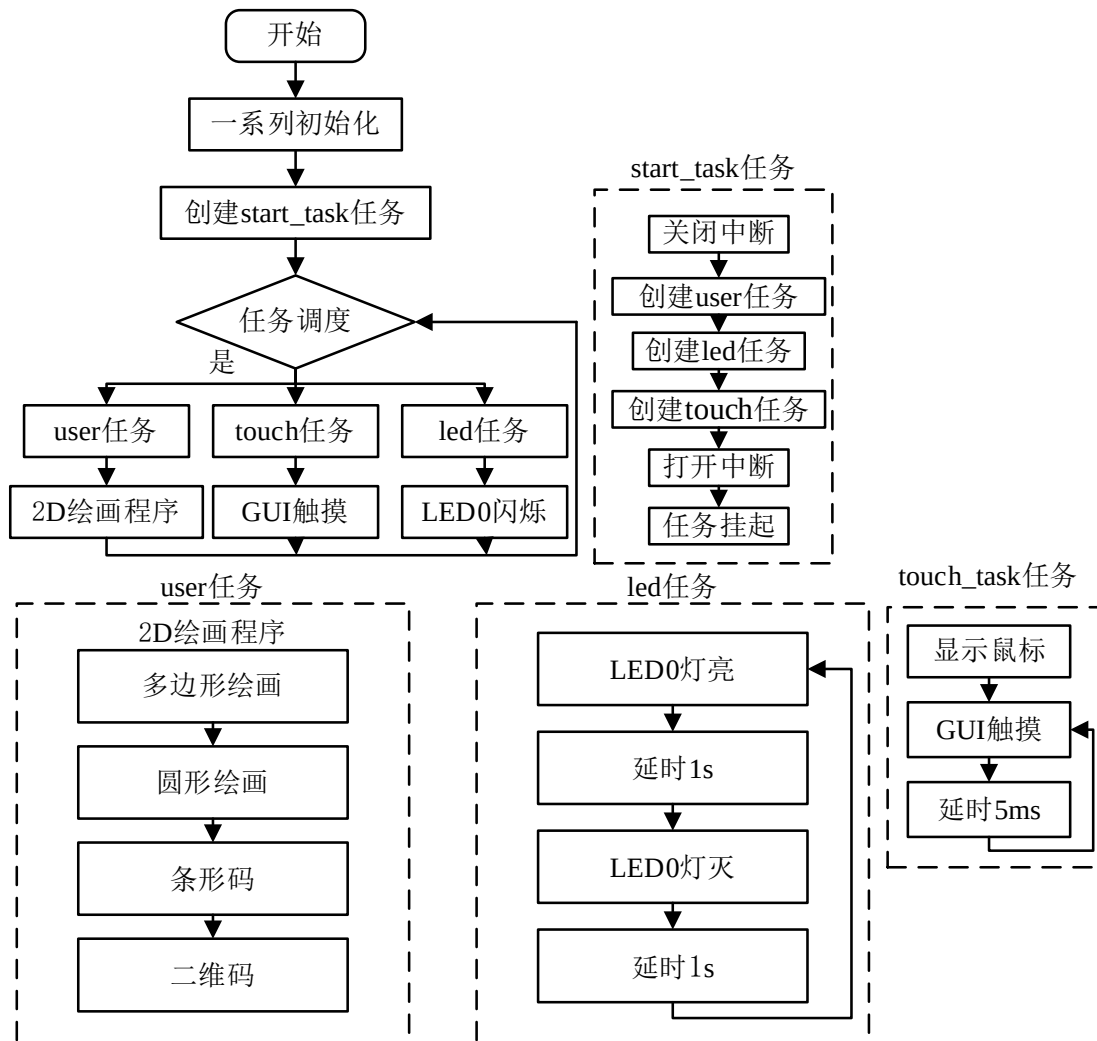


图 7.3.2.1.1 2D 绘画实验流程图

7.3.2.2 程序解析

这里我们编写了一个综合演示例程在开发板上运行演示 emWin 的 2D 绘图 API 函数，实验完整工程为：[emWin 实验 5 2D 绘图](#)，此工程可以直接下载到开发板里面观察运行结果，本程序是基于 800 * 480 分辨率屏幕的，如果使用 2.8 或者 3.5 寸屏的话可能显示不完全，这一点大家要注意一下。下面我们分别地讲解 emwin_demo.c 文件实现 2D 图形的函数，这些 2D 图形的函数如下所示：

(1) 绘制多边形、圆形等图形，如下源码所示：

```

/**
 * @brief      2D 图形绘制
 * @param      无
 * @return     无
 */
void emwin_display_2d(void)
{
    int i;

    GUI_SetBkColor(GUI_BLUE); /* 设置背景颜色 */
    
```

```

GUI_Clear();

GUI_SetColor(GUI_YELLOW);
GUI_SetFont(&GUI_Font24_ASCII);
GUI_DispStringHCenterAt("ALIENTEK 2D DISPALY", 400, 10);

GUI_SetColor(GUI_RED);
GUI_SetFont(&GUI_Font16_ASCII);

GUI_SetBkColor(GUI_GREEN);
GUI_ClearRect(10, 50, 100, 150);    /* 在一个矩形区域填充绿色背景 */
GUI_SetBkColor(GUI_BLUE);           /* 背景改回蓝色 */
/* 绘制用水平颜色梯度填充的矩形 */
GUI_DrawGradientH(110, 50, 210, 150, 0X4117BB, 0XC6B6F5);
/* 绘制用垂直颜色梯度填充的矩形 */
GUI_DrawGradientV(220, 50, 320, 150, 0X4117BB, 0XC6B6F5);
/* 绘制水平颜色梯度填充的圆角矩形 */
GUI_DrawGradientRoundedH(330, 50, 430, 150, 25, 0X4117BB, 0XC6B6F5);
/* 绘制垂直颜色梯度填充的圆角矩形 */
GUI_DrawGradientRoundedV(440, 50, 540, 150, 25, 0X4117BB, 0XC6B6F5);

GUI_DrawRect(550, 50, 650, 150);    /* 在当前窗口中指定的位置绘制矩形 */
GUI_FillRect(660, 50, 760, 150);    /* 在当前窗口中指定位置绘制填充的矩形区域 */

GUI_SetPenSize(5);                   /* 设置画笔颜色,单位像素点 */
GUI_DrawLine(10, 160, 110, 260);    /* 绘制线条 */
/* 绘制圆,无法绘制半径大于180的圆 */
for (i = 0; i < 50; i += 3) GUI_DrawCircle(200, 220, i);
/* 在指定位置绘制指定尺寸的填充的圆,无法绘制半径大于180的圆 */
GUI_FillCircle(320, 220, 50);

GUI_SetPenSize(2);
GUI_SetColor(GUI_CYAN);
/* 在指定位置绘制指定尺寸的椭圆的轮廓 无法处理大于180的 rx/ry 参数 */
GUI_DrawEllipse(450, 220, 70, 50);
GUI_SetColor(GUI_MAGENTA);
GUI_FillEllipse(450, 220, 65, 45); /* 在指定位置绘制指定尺寸的填充的椭圆 */

emwin_drawarcscale();
emwin_draw_graph();
}

```

上述源码主要绘画矩形、圆角矩形、圆形等多个图形，最后我们调用 `emwin_drawarcscale` 以及 `emwin_draw_graph` 函数绘画仪表和线图。

(2) Alpha 混合显示，如下源码所示：

```
/**
 * @brief      Alpha 混合显示
 * @param      无
 * @return     无
 */
void emwin_alpha_display(void)
{
    GUI_EnableAlpha(1);          /* 使能 Alpha 功能 */
    GUI_SetBkColor(GUI_WHITE);   /* 设置背景颜色 */
    GUI_Clear();

    GUI_SetTextMode(GUI_TM_TRANS); /* 文本透明显示 */
    GUI_SetColor(GUI_BLACK);
    GUI_SetFont(&GUI_Font32_1);
    GUI_DispStringHCenterAt("ALIENTEK ALPHA TEST", 400, 220);

    GUI_SetAlpha(0x40);          /* 设置 alpha 值 0x00-0xFF */
    GUI_SetColor(GUI_RED);
    GUI_FillRect(100, 50, 300, 250); /* 绘制红色矩形 */

    GUI_SetAlpha(0x80);
    GUI_SetColor(GUI_GREEN);
    GUI_FillRect(200, 150, 400, 350); /* 绘制绿色矩形 */

    GUI_SetAlpha(0xC0);
    GUI_SetColor(GUI_BLUE);
    GUI_FillRect(300, 250, 500, 450); /* 绘制蓝色矩形 */

    GUI_SetAlpha(0x80);
    GUI_SetColor(GUI_YELLOW);
    GUI_FillRect(400, 150, 600, 350); /* 绘制黄色矩形 */

    GUI_SetAlpha(0x40);
    GUI_SetColor(GUI_BROWN);
    GUI_FillRect(500, 50, 700, 250); /* 绘制棕色矩形 */
    GUI_SetAlpha(0);                /* 关闭 alpha 通道 */
}
```

此函数主要测试 Alpha 混色，我们调用绘画矩阵函数绘画矩阵，而这些矩阵交集的地方以混合颜色显示，这些混合颜色是开启 Alpha 混色的效果。

(3) 绘制弧线制作仪表，如下源码所示：

```
/**
 * @brief      绘制弧线 绘制简易仪表盘
```

```

* @param      无
* @return     无
*/
void emwin_drawarcscale(void)
{
    int x0 = 630;
    int y0 = 380;
    int i;
    char ac[4];
    GUI_SetPenSize( 5 );
    GUI_SetTextMode(GUI_TM_TRANS);
    GUI_SetFont(&GUI_FontComic18B_ASCII);
    GUI_SetColor(GUI_BLACK);
    GUI_DrawArc( x0, y0, 150, 150, -30, 210 );

    for (i = 0; i <= 23; i++)
    {
        float a = (-30 + i * 10) * 3.1415926 / 180;
        int x = -141 * cos(a) + x0;
        int y = -141 * sin(a) + y0;

        if (i % 2 == 0)
            GUI_SetPenSize( 5 );
        else
            GUI_SetPenSize( 4 );

        GUI_DrawPoint(x, y);

        if (i % 2 == 0)
        {
            x = -123 * cos(a) + x0;
            y = -130 * sin(a) + y0;
            sprintf(ac, "%d", 10 * i);
            GUI_SetTextAlign(GUI_TA_VCENTER);
            GUI_DispStringHCenterAt(ac, x, y);
        }
    }
}

```

此函数主要绘制了一个圆弧并在该圆弧基础上绘制相应的点和数值，最后该圆弧整体类似于一个仪表效果。

(4) 绘制线图，如下源码所示：

```

/**
* @brief      绘制线图

```

```

    * @param      无
    * @return      无
    */
void emwin_draw_graph(void)
{
    I16 aY[400];
    int i;

    for (i = 0; i < GUI_COUNTOF(aY); i++)    aY[i] = rand() % 100;

    GUI_ClearRect(0, 300, 400, 400); /* 每次显示前先清除显示区域 */
    GUI_DrawGraph(aY, GUI_COUNTOF(aY), 0, 300);
}

```

此函数主要调用函数 GUI_DrawGraph 绘制线图。

(5) 绘制多边形，如下源码所示：

```

/**
 * @brief      绘制多边形
 * @param      无
 * @return      无
 */
void emwin_draw_polygon(void)
{
    int i, Mag, magy = 50, Count = 4;
    GUI_SetBkColor(GUI_BLUE);
    GUI_SetColor(GUI_YELLOW);
    GUI_Clear();

    GUI_SetTextMode(GUI_TM_TRANS); /* 透明显示 */
    GUI_SetFont(&GUI_Font24_ASCII); /* 设置字体 */
    GUI_DispStringHCenterAt("ALIENTEK PLAYGON DISPLAY", 400, 10);

    GUI_SetColor(GUI_WHITE);
    GUI_SetDrawMode(GUI_DM_XOR); /* 设置绘制模式 */
    GUI_FillPolygon(aPoints, GUI_COUNTOF(aPoints), 140, 110);

    for (i = 1; i < 10; i++)
    {
        GUI_EnlargePolygon(aEnlargedPoints, aPoints,
            GUI_COUNTOF(aPoints), i * 5); /* 放大坐标 */
        /* 绘制放大坐标后的多边形 */
        GUI_FillPolygon(aEnlargedPoints, GUI_COUNTOF(aPoints), 140, 110);
    }
}

```

```

GUI_SetDrawMode(GUI_DM_NORMAL); /* 恢复默认的绘制模式 */

GUI_SetColor(GUI_GREEN);

for (Mag = 1; Mag <= 4; Mag *= 2, Count /= 2)
{
    int magx = 600;
    /* 按指定系数放大多边形 */
    GUI_MagnifyPolygon(aMagnifiedPoints, aPoints,
        GUI_COUNTOF(aPoints), Mag);

    for (i = Count; i > 0; i--, magx += 40 * Mag)
    {
        /* 绘制多边形 */
        GUI_FillPolygon(aMagnifiedPoints, GUI_COUNTOF(aPoints), magx, magy);
    }

    magy += 20 * Mag;
}

GUI_SetFont(&GUI_Font8x16);
GUI_DispStringAt("Polygons of arbitrary shape ", 100, 300);
GUI_DispStringAt("in any color", 220, 320);
GUI_SetColor(GUI_DARKRED);
GUI_FillPolygon (&aPointArrow[0], 7, 200, 400); /* 绘制多边形 */
}

```

上述源码主要调用本章节 7.2.2 小节图形函数绘制多边形。

7.3.3 下载验证

程序在开发板上的运行结果如图 7.3.3.1、7.3.3.2、7.3.3.3 所示（此处使用的是 4.3 寸 800*480 屏幕）：

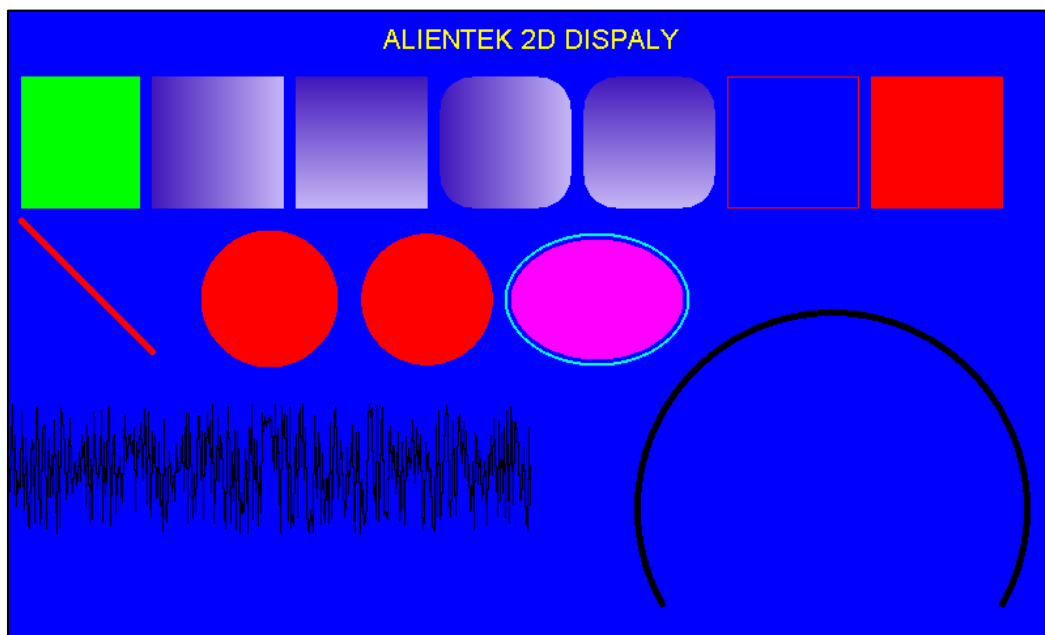


图 7.3.3.1 2D 绘图

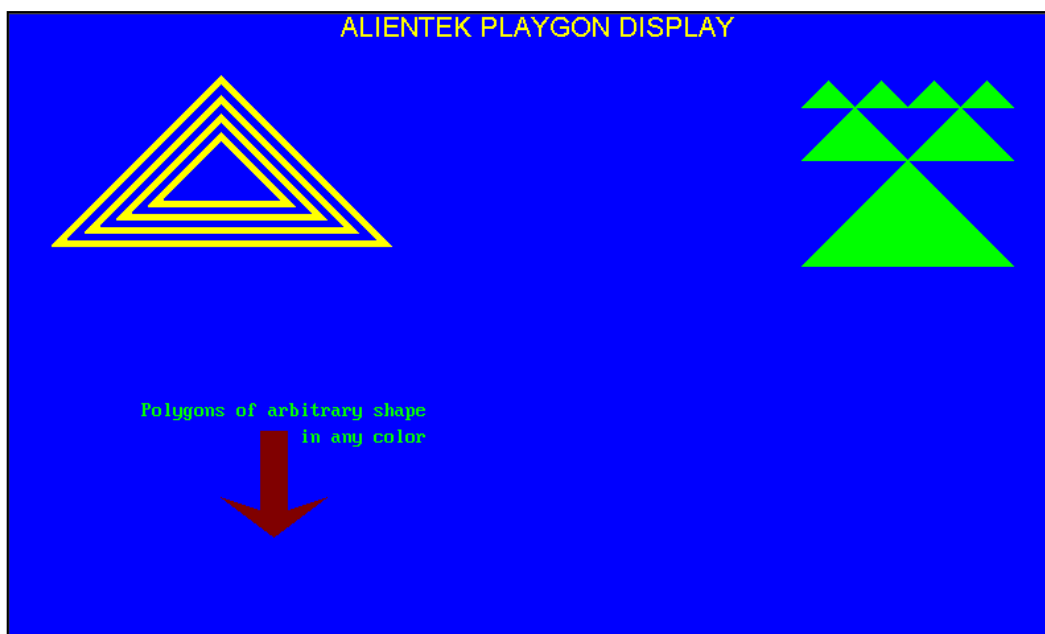


图 7.3.3.2 多边形绘制

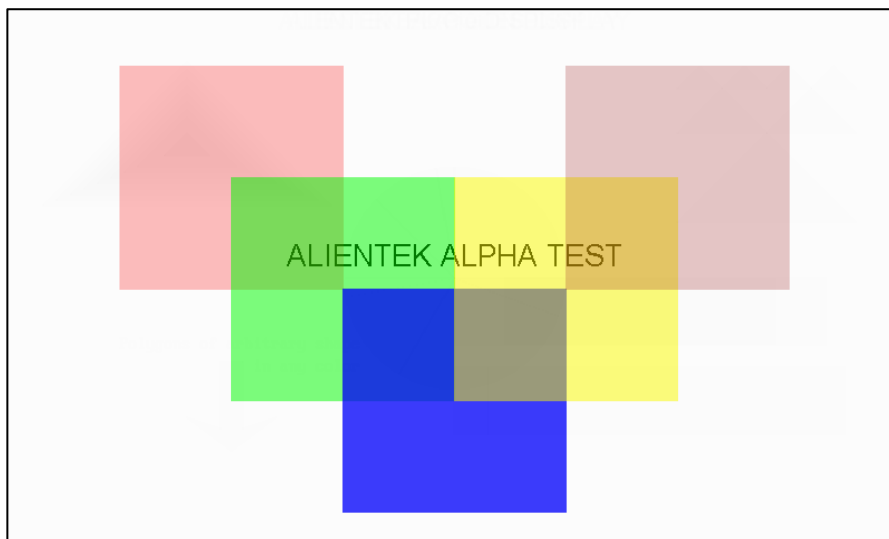


图 7.3.3.3 Alpha 测试

总结：

使用 Alpha 步骤：

方法一：

GUI_EnableAlpha(1)使能 Alpha 混合。

GUI_SetUserAlpha(&AlphaState, 0xC0)设置 Alpha 值。

GUI_RestoreUserAlpha(&AlphaState)恢复之前得用户设定值。

方法二：

GUI_EnableAlpha(1)使能 Alpha 混合。

GUI_SetColor((0x40uL << 24) | GUI_RED);

GUI_RestoreUserAlpha(&AlphaState)恢复之前得用户设定值。

第八章 显示位图

本章，我们讲解一下如何使用 emWin 位图转换器将图片转换为 C 文件，接着调用 emWin 中的相应函数来显示这个 C 文件的位图。本章暂时不讲解如何在屏幕上直接显示 GIF、JPEG、BMP 和 PNG 等格式的图片，这些格式图片笔者会在以后的章节中讲解。

本章分为以下几个部分：

- 8.1 位图转换器
- 8.2 位图绘制 API 简介
- 8.3 显示位图实验

8.1 位图转换器

8.1.1 位图转换器简介

位图转换器的主要作用是将位图从 PC 格式转换为 C 文件。emWin 可使用的位图在 C 文件中通常定义为 GUI_BITMAP 结构。这些结构会非常大，手动生成这些位图很费时并且效率很低。因此建议使用位图转换器，自动从位图生成 C 文件。它的另一项有用功能是将图像另存为 C 流文件。它与常规 C 文件相比，其优势在于这些数据流可位于需要将 C 文件置于的可寻址 CPU 区域的任何程序中的任何位置。还具有色彩转换功能，因此可以减小所生成的 C 代码。通常可以减少每个像素的位数以减少内存消耗。位图转换器显示已转换的图像。位图转换器可执行很多简单的功能，包括缩放尺寸、水平或垂直裁剪位图、旋转以及转换位图索引或色彩。位图转换器这个软件是在 STemWin 软件包里面获取的，它命名为 BmpCvtST.exe。因为该软件的是由 ST 提供的，所以这个软件包并不在我们移植的 emWin 软件包里面。请大家在 ST 官方下载 5.44 版本的 STemWin 软件包下的 Software 文件夹获取如下图所示：

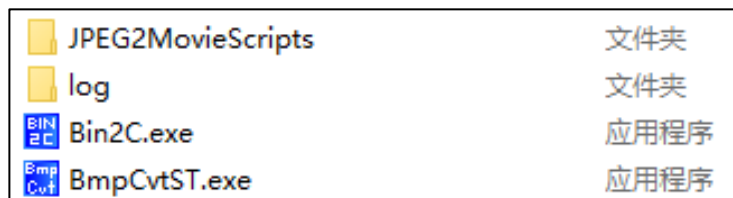


图 8.1.1.1 位图转换器

上图中 BmpCvtST.exe 是位图转换器，该软件是把图片转成 C 数组格式的图片。

8.1.2 位图转换器的使用

虽然 BmpCvtST.exe 位图转换器是针对 STemWin 图形界面库开发的，但是 STemWin 和 emWin 图形界面库是一个大同小异的东西，所以 emWin 也具备使用该软件转换的 C 数组文件。下面笔者讲解位图转换器使用步骤：

第一步：打开 BmpCvtST.exe 位图转换器如下图所示：

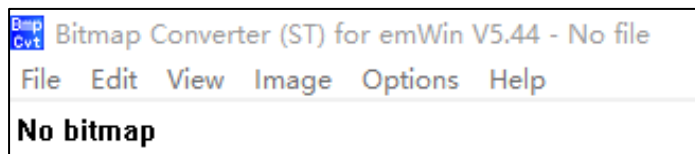


图 8.1.2.1 位图转换器

第二步：向位图转换器中加载图片，我们直接把图片拖到此软件里面，也可以点击位图转换器的“File→Open”选项。这里笔者选择正点原子的 logo 图片，格式为 PNG，打开以后如下图所示：

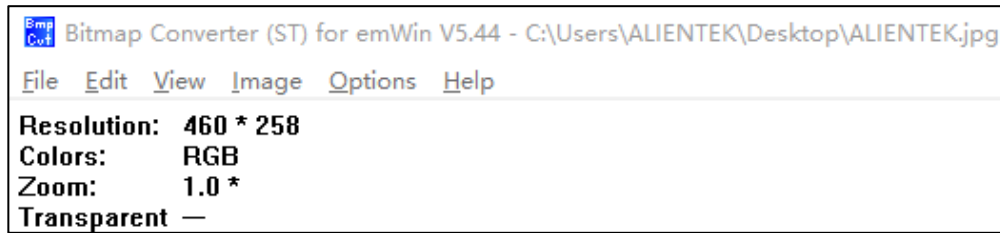


图 8.1.2.2 向位图转换器中加载图片

第三步：色彩转换，选择所需的调色板，点击“Image→Convert to”选项，我们可以看到，有很多种格式可以选择，这里我们选择“Best palette”选项，也就是“最佳调色板”，大家也可以尝试选择其他的格式。转换位图颜色格式的主要原因是为了减少内存消耗。最常用的方法是使用“最佳调色板”选项，它可定制特定位图的调色板，只包含图像中要用的颜色。为了使调色板尽可能小，同时还要完全支持图像的全彩位图，此选项尤其有用。在位图转换器中打开位图文件后，只需从菜单中选择 Image/Convert Into/Best palette 选项。如果需要保持透明性，请选择 Image/Convert Into/Best palette + transparency 选项。对于某些应用，可能使用固定颜色调色板更高效，请在 Image/Convert Into 菜单下选择。例如，图片支持四种灰度的显示器上显示全彩模式的位图。此时使图像保持为原始格式会浪费内存，因为在显示器上只显示为四种灰度。将全彩位图转换为四灰度、2bpp 位图可获得最高效率。

第四步：将位图存为 C 文件，点击“File→Save As”选项保存，注意这里选择存储为.C 文件（设置好文件的保存路径），如下图所示：

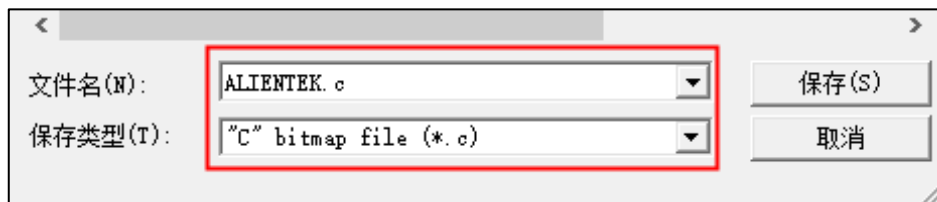


图 8.1.2.3 保存为.c 文件

第五步：选择为.C 文件以后，点击保存，会弹出如下图所示画面，让我们选择格式，这里我们选择“High color (565) 选项”。

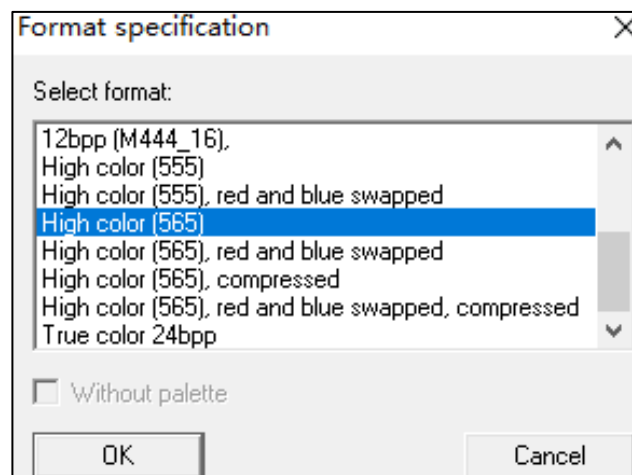


图 8.1.2.4 选择格式

第六步：选择完格式后点击“OK”，这样就在你之前设置好的文件夹下生成了一个.C 文件，将这个 C 文件拷贝到我们的工程中，我们在接下来的实验中就要使用这个 C 文件。把该 C 文件放在这个路径下：emWin 实验 6 绘制位图→Middlewares→emWin→emWin_Demo，打开以后如下图所示：

emWin实验6 绘制位图 > Middlewares > emWin > emWin_Demo		
名称	类型	大小
ALIENTEK.c	C 文件	159 KB
emwin_demo.c	C 文件	3 KB
emwin_demo.h	C++ Header file	1 KB

8.1.2.5 将位图 C 文件放入工程中

至此，我们已经把图片转换成 emWin 可读取的 C 数组文件，emWin 如何显示该 C 数组的图片数据呢？下面笔者重点讲解一下 emWin 位图绘制函数。

8.2 位图绘制 API 简介

emWin 的位图绘制函数只提供了五个 API 函数，这些函数如下表所示：

函数	描述
GUI_DrawBitmap()	绘制位图
GUI_DrawBitmapEx()	绘制缩放的位图
GUI_DrawBitmapHWAAlpha()	硬件 Alpha 混合支持的系统上绘制带 Alpha 混合信息的位图
GUI_DrawBitmapMag()	绘制放大的位图
GUI_SetAlphaMask8888()	这只对绘制未压缩的真颜色位图起作用 (GUI_DRAW_BMP8888 和 GUI_DRAW_BMPM888I)

表 8.2.1 位图绘制 API 函数

我们最常用的就是 GUI_DrawBitmap 和 GUI_DrawBitmapEx 这两个函数。在演示实例中，我们主要讲解这两个函数的使用。下面我们分别地讲解上述的函数的原型和形参，如下所示：

(1) GUI_DrawBitmap 函数

在当前窗口的指定位置绘制位图图像，该函数原型如下源码所示：

```
void GUI_DrawBitmap(const GUI_BITMAP * pBitmap,int x0,int y0);
```

该函数的相关形参如下表所示：

参数	描述
pBitmap	指向要显示的位图
x0	显示器重位图左上角的 X 位置
y0	显示器重位图左上角的 Y 位置

表 8.2.2 GUI_DrawBitmap 函数相关形参描述

注意：图片数据的解析从第一个字节的最高有效位（msb）开始位流，新行启终在偶数字节地址开始，就像位图的第 n 行偏移 n*BytesPerLine 开始，在客户区的任何位置都能看到位图。

(2) GUI_DrawBitmapEx 函数

在显示器中缩放位图，该函数的原型如下源码所示：

```
void GUI_DrawBitmapEx( const GUI_BITMAP * pBitmap, int x0, int y0,
                        int xCenter,int yCenter,
                        int xMag,int yMag);
```

该函数的相关形参如下表所示：

参数	描述
pBitmap	指向要显示的位图
x0	显示器的定位点 X 位置
y0	显示器的定位点 Y 位置
xCenter	位图中定位点 X 位置
yCenter	位图中定位点 Y 位置
xMag	x 方向的比例因子，单位：1/1000
yMag	y 方向的比例因子，单位：1/1000

表 8.2.3 GUI_DrawBitmapEx 函数相关形参描述

注意：该函数不能用于绘制 RLE 压缩的位图和格式为 A565 的位图。

在上面两个函数中，位图信息都是用结构体 GUI_BITMAP 来定义的，我们使用位图转换器生成的 C 文件最终就是将图片信息放到结构体里面，包括图片的数据，尺寸，调色板信息等。

(3) GUI_DrawBitmapHWAlpha 函数

具有硬件 Alpha 混合支持的多层系统上绘制带 Alpha 信息的位图，该函数的原型如下源码所示：

```
void GUI_DrawBitmapHWAlpha(const GUI_BITMAP * pBM, int x0, int y0);
```

该函数的相关形参如下表所示：

参数	描述
pBM	指向要显示的位图的指针
x0	显示器中位图左上角的 X 位置
y0	显示器中位图左上角的 Y 位置

表 8.2.4 GUI_DrawBitmapHWAlpha 函数相关形参描述

(4) GUI_DrawBitmapMag 函数

放大显示器中的位图，该函数的原型如下源码所示：

```
void GUI_DrawBitmapMag(const GUI_BITMAP * pBM, int x0, int y0, int XMul, int YMul);
```

该函数的相关形参如下表所示：

参数	描述
pBM	指向要显示的位图的指针
x	显示器中位图左上角的 X 位置
y	显示器中位图左上角的 Y 位置
XMul	X 方向的放大系数
YMul	Y 方向的放大系数

表 8.2.5 GUI_DrawBitmapHWAlpha 函数相关形参描述

(5) GUI_SetAlphaMask8888 函数

用来设置用于绘制非压缩真彩色位图像素的附加掩码，该函数的原型如下源码所示：

```
void GUI_SetAlphaMask8888( U32 OrMask, U32 AndMask);
```

该函数的相关形参如下表所示：

参数	描述
OrMask	与每个像素进行 OR 组合的值
AndMask	与每个像素进行 AND 组合的值

表 8.2.6 GUI_DrawBitmapHWAlpha 函数相关形参描述

8.3 显示位图实验

8.3.1 硬件设计

1. 例程功能

本实验主要测试 emWin 显示位图 API 函数使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用 emwin_main 函数执行 emWin 程序，该函数显示一张位图，然后调用函数 GUI_DrawBitmapEx 根据比例因子设置位图的大小。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 6 绘制位图》。

8.3.2 软件设计

8.3.2.1 程序流程图

本实验的程序流程图，如下图所示：

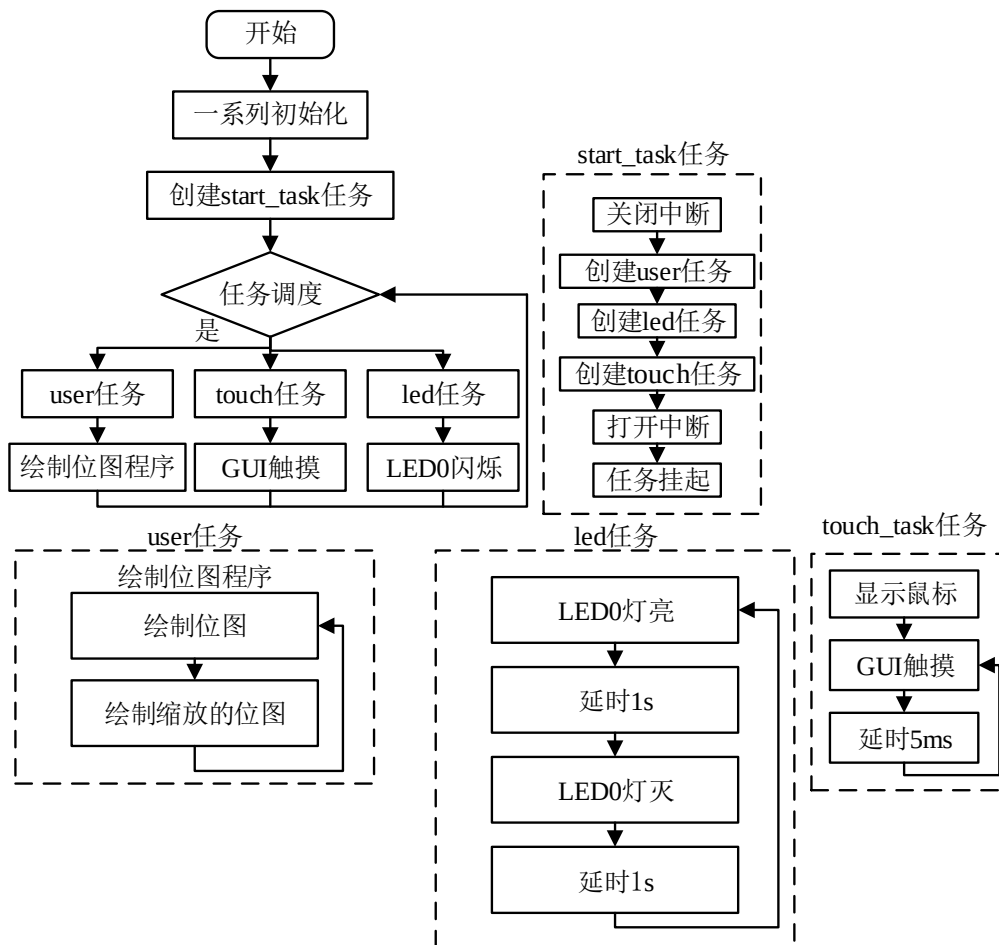


图 8.3.2.1.1 绘制位图实验流程图

8.3.2.2 程序解析

这里我们编写了一个综合演示例程在开发板上运行演示 emWin 的位图显示功能，实验完整工程为：**emWin 实验 6 绘制位图**，打开 emwin_demo.c，程序源码如下：

```
/**
 * @brief      显示 C 文件格式的位图
 * @param      无
 * @return     无
 */
void emwin_draw_bitmap(void)
{
    GUI_SetBkColor(GUI_BLUE);
    GUI_SetColor(GUI_YELLOW);
    GUI_Clear();

    GUI_SetFont(&GUI_Font24_ASCII);
    GUI_SetTextMode(GUI_TM_TRANS);          /* 透明显示 */
    GUI_DispStringHCenterAt("ALIENTEK BITMAP DISPLAY",400,0 );
    GUI_DrawBitmap(&bmALIENTEK,295,194);    /* 绘制正点原子图标 */
}

/**
 * @brief      在显示器中缩放位图
 * @param      Xmag:X 方向的比例因子，单位为千分之
 * @param      Ymag:Y 方向的比例因子，单位为千分之
 * @return     无
 */
void emwin_zoom_bitmap(int Xmag,int Ymag)
{
    GUI_SetBkColor(GUI_BLUE);
    GUI_Clear();
    GUI_DrawBitmapEx(&bmALIENTEK,400,240,105,46,Xmag,Ymag); /*按照比例绘制位图*/
}
```

在上面程序中有两个函数 emwin_draw_bitmap 和 emwin_zoom_bitmap，emwin_draw_bitmap 函数主要展示了 GUI_DrawBitmap 函数的使用，该函数主要绘制位图，而 emwin_zoom_bitmap 主要展示了 GUI_DrawBitmapEx 的使用，该函数根据缩放因子决定位图的大小。程序非常简单，我们在打开 uc-os3_demo.c 文件，该 emwin_main 函数在 user 任务函数调用。

```
void emwin_main (void)
{
    OS_ERR err;
    p_arg = p_arg;
    int Xmag = 0 ,Ymag = 0;          /* 缩放因子,单位 千分之一 */
    emwin_draw_bitmap();
    OSTimeDlyHMSM(0,0,2,0,OS_OPT_TIME_PERIODIC,&err); /* 延时 2s */
}
```

```
while(1)
{
    Xmag += 100;
    Ymag += 100;
    if(Xmag >= 2000)
    {
        Xmag = 100;
    }
    if(Ymag >= 2000)
    {
        Ymag = 100;
    }
    emwin_zoom_bitmap(Xmag,Ymag); /* 绘制缩放的位图 */
    GUI_Delay(500);
}
```

此函数一开始先调用 `emwin_draw_bitmap` 函数显示正点原子的 logo，延时 2s 以后进入 `while(1)` 循环中，循环调用 `emwin_zoom_bitmap` 函数显示由小到大的正点原子的 logo，每次 X,Y 的比例因子增加 100%，也就是十分之一。

8.3.3 下载验证

程序运行结果如图 8.3.3.1 所示，**注意：**此程序是基于 800*480 分辨率 LCD 的，如果是在正点原子的 2.8 或者 3.5 寸屏幕上会显示不完全，稍微改动一下程序就可以了。



图 8.3.3.1 位图显示

第九章 颜色

emWin 支持黑色/白色、灰度（具有不同强度的单色）和彩色显示器。同一用户程序可用于任何的显示器，只有 LCD 显示器的配置才需要进行更改。emWin 中颜色分为逻辑颜色和物理颜色，逻辑颜色是应用程序处理的颜色，逻辑颜色始终按 RGB 值进行定义，是一个 24 位的值，每种颜色包含 8 位。物理颜色是可用显示器实际显示的颜色，按逻辑颜色映射到物理颜色。

本章分为以下几个部分：

- 9.1 逻辑颜色
- 9.2 预定义的基本颜色
- 9.3 调色板
- 9.4 颜色显示实验

9.1 逻辑颜色

emWin 的逻辑颜色是应用程序处理的颜色，低于 emWin5.30 的版本仅支持 ABGR，也就是我们操作 emWin 时候，它按照 BGR 格式处理写入的颜色值，这样可以解释了初学者为什么写入的颜色值（0x0000FF）显示出来的是红色，本应该绘画出来的是蓝色。其中根本原因在于这个 BGR 格式上，因为我们开发时候，大多数使用 emWin 预定义好的基本颜色，例如显示红色就调用 GUI_RED，导致很多初学者误认为这个 GUI_RED 是以 RGB 格式设置的数值。32 位的 ABGR 颜色格式如下所示：

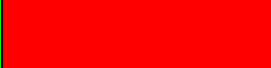
Alpha	BLUE	GREEN	RED
0x00 表示完全不透明 0xFF 表示完全透明			
Bit[31:24]占用 8 位	Bit[23:16]占用 8 位	Bit[15:8]占用 8 位	Bit[7:0]占用 8 位

表 9.1.1 emWin 的 ABGR 颜色格式

上表中的 31:24 位表示 Alpha 透明通道，如果我们使用这个透明通道，则必须调用 GUI_EnableAlpha 函数开启 Alpha。从 emWin5.30 之后为什么又推出了 ARGB 颜色格式呢？主要因为越来越多的显示屏控制器采用这种颜色格式了，比如高性能的 ST 芯片就是这种颜色格式。32 位 ARGB 颜色格式如下所示：

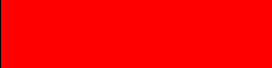
Alpha	RED	GREEN	BLUE
0x00 表示完全不透明 0xFF 表示完全透明			
Bit[31:24]占用 8 位	Bit[23:16]占用 8 位	Bit[15:8]占用 8 位	Bit[7:0]占用 8 位

表 9.1.2 emWin 的 ARGB 颜色格式

既然我们的 LCD 控制器是 ARGB 格式的，跟 emWin 默认的 ABGR 格式不兼容，岂不是每次显示前需要颜色格式转换，实际上 emWin 确实这样干的。虽然这样的转换会很大程度上影响 LCD 的性能，但是开启多缓冲、DMA2D 颜色格式硬件转换在一定程度上降低影响。

问题来了，我们怎么使用 ARGB 格式呢？我们只需要在 GUIConf.h 文件下把 GUI_USE_ARGB 宏定义置 1（0 表示使用 ABGR 格式，1 表示使用 ARGB 格式）可开启 ARGB 颜色格式。值得注意的是：MDK 提供的 emWin 库文件（本教程使用该库来开发）**仅支持 ABGR 格式**，而 ST 提供了 ARGB 和 ABGR 两种格式库（由于版本过低，本教程不采用此库）。

9.2 预定义的基本颜色

在 emWin 中预定义一些标准颜色，这些预定义的颜色都用宏定义来表示，我们可以使用这些预定义的基本颜色来设置背景色和前景色，如果需要其他颜色的话我们就要自己定义了，这些预定义的颜色如下表所示：

颜色宏定义	颜色	RGB 值
GUI_BLUE		0x00FF0000
GUI_GREEN		0x0000FF00
GUI_RED		0x000000FF
GUI_CYAN		0x00FFFF00
GUI_MAGENTA		0x00FF00FF
GUI_YELLOW		0x0000FFFF
GUI_LIGHTBLUE		0x00FF8080
GUI_LIGHTGREEN		0x0080FF80
GUI_LIGHTRED		0x008080FF
GUI_LIGHTCYAN		0x00FFFF80
GUI_LIGHTMAGENTA		0x00FF80FF
GUI_LIGHTYELLOW		0x0080FFFF
GUI_DARKBLUE		0x00800000
GUI_DARKGREEN		0x00008000
GUI_DARKRED		0x00000080
GUI_DARKCYAN		0x00808000
GUI_DARKMAGENTA		0x00800080
GUI_DARKYELLOW		0x00008080
GUI_WHITE		0x00FFFFFF
.....		
GUI_BLACK		0x00000000
GUI_BROWN		0x002A2AA5
GUI_ORANGE		0x0000A5FF

表 9.2.1 emWin 预定义颜色

9.3 调色板

emWin 中内置了很多固定的调色板模式如下表所示：

标识符	可用的颜色数	掩码
GUICC_1	2 (黑白)	0x01
GUICC_2	4 (灰度)	0x03
GUICC_4	16 (灰度)	0x0F
GUICC_5	32 (灰度)	0x1F
GUICC_111	8	BGR
GUICC_M111	8	RGB
GUICC_222	64	BBGGRR
GUICC_M222	64	RRGGBB
GUICC_233	256	BBGGRRRR
GUICC_M233	256	RRGGGBBB
GUICC_323	256	BBBGRRRR
GUICC_M323	256	RRRGGBBB
GUICC_332	256	BBBGGRRR
GUICC_M332	256	RRRGGBBB
GUICC_44412	4096	0000BBBBGGGRRRRR
GUICC_M444_12	4096	0000RRRRGGGGBBBB
GUICC_444121	4096	BBBBGGGRRRRR0000
GUICC_44416	4096	0BBBB0GGGG0RRRR0
GUICC_M44416	4096	0RRRR0GGGG0BBBB0
GUICC_555	32768	0BBBBBGGGGGRRRRR
GUICC_M555	32768	0RRRRRGGGGGBBBBB
GUICC_556	65536	BBBBBGGGGGRRRRRR
GUICC_M556	65536	RRRRRGGGGGBBBBBB
GUICC_565	65536	BBBBBGGGGGRRRRR
GUICC_M565	65536	RRRRRGGGGGBBBBBB
GUICC_655	65536	BBBBBGGGGGRRRRR
GUICC_M655	65536	RRRRRGGGGGBBBBBB
GUICC_666	262144	BBBBBBGGGGGRRRRRR
GUICC_M666	262144	RRRRRGGGGGBBBBBB
GUICC_666_9	262144	000000BBBBBBGGG000000GGGRRRRRR
GUICC_M666_9	262144	000000RRRRRRGGG000000GGGBBBBBB
GUICC_822216	256	0xFF
GUICC_84444	240	0xFF
GUICC_8666	232	0xFF
GUICC_86661	233 (232 + 透明)	0xFF
GUICC_888	16777216	BBBBBBBGGGGGGGRRRRRRR

表 9.3.1 emWin 内置调色板

关于表 9.2.1 中固定调色板详细的说明可以参考 emWin 中文手册，在前面讲解 emWin 移植的时候我们详细的讲解了 LCDConf_FlexColor_Template.c 文件中的函数 LCD_X_Config，在这个函数中我们调用了 GUI_DEVICE_CreateAndLink 函数来设置调色板，该调色板我们选择的是 GUICC_M565 这个调色板。

如果没有固定调色板模式与色彩转换的需求相匹配，则可以使用定制调色板模式，利用此模式就可使用应用程序定义的色彩转换，本教程不讲解此模式，感兴趣的同学可以参考 emWin 中文手册中此模式的使用方法。

9.4 颜色 API 函数

有关颜色的 API 函数如下表所示：

例程	描述
基本颜色函数	
GUI_GetBkColor()	返回当前背景色
GUI_GetBkColorIndex()	返回当前背景色的索引
GUI_GetColor()	返回当前前景色
GUI_GetColorIndex()	返回当前前景色的索引
GUI_GetDefaultColor()	返回当前设置的默认前景色
GUI_GetDefaultBkColor()	返回当前设置的默认背景颜色
GUI_SetBkColor()	设置当前背景色
GUI_SetBkColorIndex()	设置当前背景色的索引
GUI_SetColor()	设置当前前景色
GUI_SetColorIndex()	设置当前前景色的索引
GUI_SetDefaultColor()	设置默认的前景色
GUI_SetDefaultBkColor()	设置默认背景色
索引和颜色转换	
GUI_CalcColorDist()	返回
GUI_CalcVisColorError()	将差值返回给下一可用颜色
GUI_Color2Index()	将色彩转换为颜色索引
GUI_Color2VisColor()	返回最接近的可用颜色
GUI_ColorIsAvailable()	检查给定的颜色是否可用
GUI_Index2Color()	将颜色索引转换为颜色

表 9.4.1 颜色 API 函数

以上是表 9.4.1 的调色板函数的相关信息。接下来，我们来讲解颜色显示常用的函数。

(1) GUI_GetBkColor 函数

获取当前背景色，该函数原型如下源码所示：

```
GUI_COLOR GUI_GetBkColor(void);
```

返回值：当前背景色。

(2) GUI_GetBkColorIndex 函数

获取当前背景色的索引，该函数原型如下源码所示：

```
int GUI_GetBkColorIndex(void);
```

返回值：当前背景色索引。

(3) GUI_GetColor 函数

获取当前前景色，该函数原型如下源码所示：

```
GUI_COLOR GUI_GetColor(void);
```

返回值：当前前景色。

(4) GUI_GetColorIndex 函数

获取当前前景色的索引，该函数原型如下源码所示：

```
int GUI_GetColorIndex(void);
```

返回值：当前前景色的索引。

(5) GUI_GetDefaultColor 函数

获取默认前景色的当前设置，该函数原型如下源码所示：

```
GUI_COLOR GUI_GetDefaultColor(void);
```

返回值：默认前景色。

(6) GUI_GetDefaultBkColor 函数

获取默认背景色的当前设置，该函数原型如下源码所示：

```
GUI_COLOR GUI_GetDefaultBkColor(void);
```

返回值：当前背景色。

(7) GUI_SetBkColor 函数

设置当前背景色，该函数原型如下源码所示：

```
GUI_COLOR GUI_SetBkColor(GUI_COLOR Color);
```

该函数的相关形参如下表所示：

参数	描述
Color	背景的颜色，24 位 RGB 值

表 9.4.3 GUI_SetBkColor 函数相关形参描述

返回值：选定的背景色。

(8) GUI_SetBkColorIndex 函数

设置当前背景色的索引，该函数原型如下源码所示：

```
int GUI_SetBkColorIndex(int Index);
```

该函数的相关形参如下表所示：

参数	描述
Index	要使用颜色的索引

表 9.4.4 GUI_SetBkColorIndex 函数相关形参描述

返回值：要使用颜色的索引。

(9) GUI_SetColor 函数

设置当前前景色，该函数原型如下源码所示：

```
void GUI_SetColor(GUI_COLOR Color);
```

该函数的相关形参如下表所示：

参数	描述
Color	前景的颜色 24 位 RGB 值

表 9.4.5 GUI_SetColor 函数相关形参描述

返回值：选定的前景色。

(10) GUI_SetColorIndex 函数

设置当前前景色的索引，该函数原型如下源码所示：

```
void GUI_SetColorIndex(int Index);
```

该函数的相关形参如下表所示：

参数	描述
Index	要使用颜色的索引

表 9.4.6 GUI_SetColorIndex 函数相关形参描述

返回值：选定的前景色索引。

(11) GUI_SetDefaultColor 函数

设置默认前景色，该函数原型如下源码所示：

```
void GUI_SetDefaultColor(GUI_COLOR Color);
```

该函数的相关形参如下表所示：

参数	描述
Color	前景的颜色，24 位 RGB 值

表 9.4.7 GUI_GetBkColor 函数相关形参描述

(12) GUI_SetDefaultBkColor 函数

设置默认背景色，该函数原型如下源码所示：

```
void GUI_SetDefaultBkColor(GUI_COLOR Color);
```

该函数的相关形参如下表所示：

参数	描述
Color	背景的颜色，24 位 RGB 值

表 9.4.8 GUI_SetDefaultBkColor 函数相关形参描述

(13) GUI_CalcColorDist 函数

计算 2 种颜色之间的差值。差值应通过红色、绿色和蓝色分量差值的平方和进行计算，该计算方式如下所示：

$$\text{差值} = (\text{Red1} - \text{Red0})^2 + (\text{Green1} - \text{Green0})^2 + (\text{Blue1} - \text{Blue0})^2$$

该函数原型如下源码所示：

```
U32 GUI_CalcColorDist(GUI_COLOR Color0, GUI_COLOR Color1);
```

该函数的相关形参如下表所示：

参数	描述
Color0	第一种颜色的 RGB 值
Color1	第二种颜色的 RGB

表 9.4.9 GUI_CalcColorDist 函数相关形参描述

返回值：如上所述的差值。

(14) GUI_CalcVisColorError 函数

计算与下一可用颜色的差值，请参考 GUI_CalcColorDist 函数，该函数原型如下源码所示：

```
U32 GUI_CalcVisColorError(GUI_COLOR Color);
```

该函数的相关形参如下表所示：

参数	描述
Color	要计算的颜色的 RGB 值

表 9.4.10 GUI_CalcVisColorError 函数相关形参描述

返回值：与下一可用颜色的差值。

(15) GUI_Color2Index 函数

获取指定 RGB 颜色值的索引，该函数原型如下源码所示：

```
int GUI_Color2Index(GUI_COLOR Color);
```

该函数的相关形参如下表所示：

参数	描述
Color	要转换的颜色的 RGB 值

表 9.4.11 GUI_Color2Index 函数相关形参描述

返回值：颜色索引。

(16) GUI_Color2VisColor 函数

将系统的下一可用颜色作为 RGB 颜色值返回，该函数原型如下源码所示：

```
GUI_COLOR GUI_Color2VisColor(GUI_COLOR Color);
```

该函数的相关形参如下表所示：

参数	描述
Color	颜色的 RGB 值

表 9.4.12 GUI_Color2VisColor 函数相关形参描述

返回值：最接近的可用颜色的 RGB 颜色值。

(17) GUI_ColorIsAvailable 函数

检查给定的颜色是否可用，该函数原型如下源码所示：

```
char GUI_ColorIsAvailable(GUI_COLOR Color);
```

该函数的相关形参如下表所示：

参数	描述
Color	颜色的 RGB 值

表 9.4.13 GUI_ColorIsAvailable 函数相关形参描述

返回值：颜色的 RGB 值。

(18) GUI_Index2Color 函数

获取指定索引的 RGB 颜色值，该函数原型如下源码所示：

```
int GUI_Index2Color(int Index);
```

该函数的相关形参如下表所示：

参数	描述
Index	要转换颜色的索引

表 9.4.14 GUI_Index2Color 函数相关形参描述

返回值：RGB 颜色值

在这些 API 函数中我们最常使用的就是 GUI_SetColor 和 GUI_SetBkColor 这两个函数，它们是用来设置前景色和背景色。

9.5 颜色显示实验

9.5.1 硬件设计

1. 例程功能

本实验主要测试 emWin 颜色 API 函数使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数主要调用函数 GUI_DrawGradientH 创建了 13 个渐变梯形，这些图形根据_aBarData 数组绘制相应的颜色。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 7 颜色条绘制》。

9.5.2 软件设计

9.5.2.1 程序流程图

本实验的程序流程图，如下图所示：

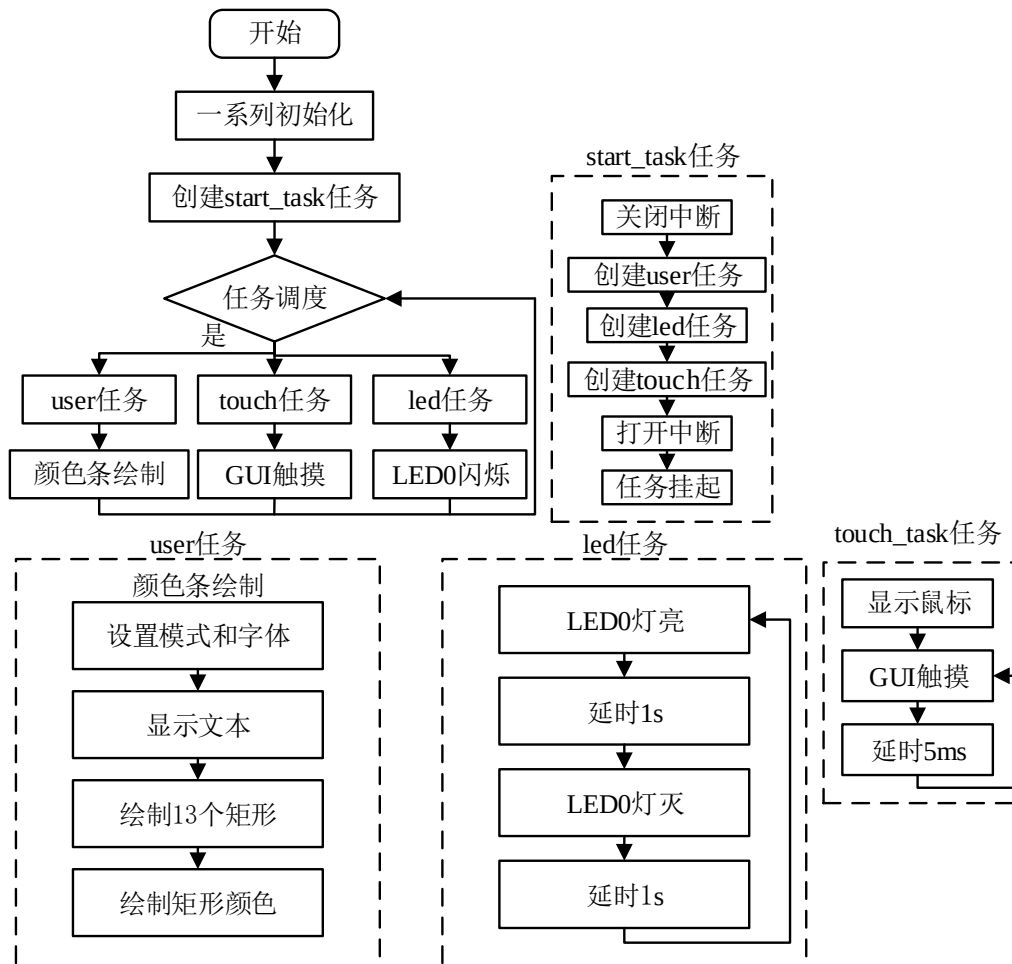


图 9.5.2.1.1 颜色条绘制实验流程图

9.5.2.2 程序解析

这里我们编写了一个颜色条绘制的例程，实验完整工程为：**emWin 实验 7 颜色条绘制**，打开 emwin_demo.c 文件，例程源码如下：

```

#define X_START 60 /* 颜色条开始 x 坐标 */
#define Y_START 40 /* 颜色条开始 y 坐标 */

typedef struct
{
    int NumBars; /* 颜色的数量 */
    GUI_COLOR Color; /* 颜色 */
    const char * s; /* 颜色的名称 */
} BAR_DATA;

static const BAR_DATA _aBarData[] = {
    { 2, GUI_RED, "Red" },
    { 2, GUI_GREEN, "Green" },
    { 2, GUI_BLUE, "Blue" },

```

```

    { 1, GUI_WHITE , "Grey" },
    { 2, GUI_YELLOW , "Yellow" },
    { 2, GUI_CYAN , "Cyan" },
    { 2, GUI_MAGENTA, "Magenta" },
};

/* 定义两种颜色分别为黑色与白色 */
static const GUI_COLOR _aColorStart[] = { GUI_BLACK, GUI_WHITE };

/**
 * @brief      显示颜色条
 * @param      无
 * @return     无
 */
static void emwin_show_color_bar(void)
{
    GUI_RECT Rect;
    int yStep;
    int i;
    int j;
    int xSize;
    int ySize;
    int NumBars;          /* 要显示的颜色条数量 2+2+2+1+2+2+2=13 */
    int NumColors;        /* 要显示的颜色数量，这里有 7 个 */

    /* 获取显示屏尺寸 */
    xSize = LCD_GetXSize();
    ySize = LCD_GetYSize();

    /* 获取颜色条的数量，NumColors 等于 7 */
    NumColors = GUI_COUNTOF(_aBarData);

    for (i = NumBars = 0, NumBars = 0; i < NumColors; i++)
    {
        NumBars += _aBarData[i].NumBars;          /* 获取_aBarData 的颜色的数量 */
    }

    yStep = (ySize - Y_START) / NumBars;          /* 获得显示颜色的平均值 */

    /* 显示文本 */
    Rect.x0 = 0;
    Rect.x1 = X_START - 1;
    Rect.y0 = Y_START;

```



```

GUI_SetFont(&GUI_Font8x16);

for (i = 0; i < NumColors; i++)
{
    Rect.y1 = Rect.y0 + yStep * _aBarData[i].NumBars - 1;
    /* 显示文本 */
    GUI_DispStringInRect(_aBarData[i].s, &Rect, GUI_TA_LEFT|GUI_TA_VCENTER);
    Rect.y0 = Rect.y1 + 1;
}

/* 显示颜色 */
Rect.x0 = X_START;
Rect.x1 = xSize - 1;
Rect.y0 = Y_START;

for (i = 0; i < NumColors; i++)
{
    for (j = 0; j < _aBarData[i].NumBars; j++)
    {
        Rect.y1 = Rect.y0 + yStep - 1;
        GUI_DrawGradientH(Rect.x0, Rect.y0, Rect.x1, Rect.y1,
                           _aColorStart[j],
                           _aBarData[i].Color); /* 显示颜色 */
        Rect.y0 = Rect.y1 + 1;
    }
}
}

```

上述函数主要调用 GUI_DrawGradientH 函数绘制颜色条并往 Y 轴方向移动绘制。

```

/**
 * @brief      emwin_main(emwin 入口)
 * @param      无
 * @return     无
 */
void emwin_main (void)
{
    GUI_SetBkColor(GUI_BLUE);
    GUI_SetColor(GUI_YELLOW);
    GUI_Clear();

    GUI_SetFont(&GUI_Font24_ASCII);
    GUI_SetTextMode(GUI_TM_TRANS); /* 透明显示 */
    GUI_DispStringHCenterAt("COLOR_BAR TEST!", 400, 0);
}

```

```
emwin_show_color_bar();          /*显示颜色条 */  
}
```

上述函数主要设置背景以及字体相关属性，最后我们调用 emwin_show_color_bar 函数绘制颜色条。

9.5.3 下载验证

在上述的程序解析的源码比较简单，就是在 LCD 上绘制多条颜色条，程序运行效果如图 9.3.3.1 所示：



图 9.5.3.1 颜色条实验

第十章 存储设备

存储设备可在各种情况下使用，主要用于防止在绘制重叠项目时出现显示器闪烁。其基本思想很简单，不使用存储设备时，绘制操作直接写入显示器，屏幕在执行绘制操作时随时更新，从而在进行各种更新时使屏幕闪烁。例如，如果要在背景中绘制一个位图，在前景中绘制一些透明文本，应首先绘制位图，然后绘制文本，效果将是文本出现闪烁。但是，如果在此过程中使用存储设备，则所有绘制操作都在存储器中执行。仅在所有操作都完成后才将最终结果显示在屏幕上，其优点是没有闪烁。差别可归纳如下：如果不使用存储设备，则可以看到一步步的绘制操作效果，缺点是会出现显示器闪烁。使用存储设备时，一次可见到所有例程的效果，就象单次操作一样，不能实际看见中间步骤。如上所述，使用存储设备的优点是完全消除了显示器的闪烁，这也是通常所期望的。

本章分为以下几个部分：

10.1 存储设备简介

10.2 存储设备实验

10.3 分段存储设备实验

10.4 自动设备对象实验

10.1 存储设备简介

10.1.1 存储设备初窥

要使用存储设备的话首先需要将 GUIConf.h 文件中的宏 GUI_WINSUPPORT 定义为 1，如下所示：

```
#define GUI_WINSUPPORT      (1)          /* 窗口管理 */
```

使用存储设备的主要原因就是可以防止屏幕闪烁，下面的示例展示了存储设备是如何来消除屏幕闪烁的，存储设备防止屏幕闪烁流程如下图所示：

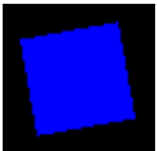
API 函数	无存储设备	有存储设备
第 0 步：初始状态		
第 1 步：GUI_Clear()		
第 2 步：GUI_DrawPolygon()		
第 3 步：GUI_DispString()		
第 4 步： GUI_MEMDEV_CopyToLCD() (仅在使用存储设备时)		

图 10.1.1.1 存储设备流程

上面的示例展示了一下同一操作在使用和不使用存储设备时的屏幕截图。两种情况下的目的是相同的：旋转工件并分别标志旋转角度（此处为 10 度）。第一种情况下不使用存储设备，屏幕必须清除，然后在新位置重绘多边形，并写入带新标志的字符串。第二种情况下(使用存储设备)，在存储器中执行相同的操作，但屏幕此时不更新。仅在调用 GUI_MEMDEV_CopyToLCD 例程时出现更新，并且此更新一次就反映所有操作。

窗口管理器可与存储设备完美搭配，有关窗口管理器会在后面的章节讲解，每个窗口都有一个标记，告诉窗口管理器是否应使用存储设备进行渲染。此标记可以在创建窗口时指定，也可在任何时候进行设置/重设。如果为特定窗口设置了存储设备标记，则 WM 在绘制窗口时自动使用存储设备，它会在绘制窗口前创建一个存储设备，然后在绘制操作完成后将其删除。如果有足够的内存可用，会将整个窗口装入 WM 所创建存储设备的内存中。如果没有足够的内存可用于将整个窗口装入一个存储设备中，则 WM 使用“分段”来绘制窗口，有关“分段”

的内容在接下来的小节中说明。用于绘制操作的内存仅在绘制操作期间分配，如果在绘制（重绘）窗口时没有足够的内存可用，则不使用存储设备重绘窗口。

10.1.2 内存要求

使用存储设备的时候势必会耗费内存的，而所需要的内存字节数取决于存储设备的色彩深度以及是否需要支持透明。对于透明支持我们可以分为两种情况讲解，如下所示：

(1) 无透明支持的内存使用情况

无透明支持的存储设备所需内存如下表所示（**注意：**所需内存取决于色彩深度）：

存储设备的色彩深度	系统色彩深度 (LCD_BITSPERPIXEL)	内存使用
1bpp	1bpp	1 字节/8 像素： (XSIZE+7)/8*YSIZE
8bpp	2、4 和 8bpp	XSIZE*YSIZE
16bpp	12 和 16bpp	2 字节/像素 XSIZE*YSIZE*2
32bpp	18、24 和 32bpp	4 字节/像素： XSIZE*YSIZE*4

表 10.1.2.1 无透明支持需要的内存

加入要创建 X 方向 111 像素、Y 方向 33 像素的存储设备，它还应兼容使用 12bpp 色彩深度的显示器并支持透明。所需的字节数计算如下：

$$\text{所需字节数} = (111 * 2 + (111 + 7) / 8) * 33 = 7788 \text{ 字节}$$

(2) 有透明支持的内存使用情况

如果存储设备要支持透明性，则每 8 像素需要一个额外字节来进行内部管理，所需内存如下表所示：

存储设备的色彩深度	系统色彩深度 (LCD_BITSPERPIXEL)	内存使用
1bpp	1bpp	2 字节/8 像素： (XSIZE+7)/8*YSIZE*2
8bpp	2、4 和 8bpp	1 字节/像素+1 字节/8 像素： (XSIZE+(XSIZE+7)/8)*YSIZE
16bpp	12 和 16bpp	2 字节/像素+1 字节/8 像素： (XSIZE*2+(XSIZE+7)/8)*YSIZE
32bpp	18、24 和 32bpp	4 字节/像素+1 字节/8 像素： (XSIZE*4+(XSIZE+7)/8)*YSIZE

表 10.1.2.2 有透明支持所需的内存

要创建 X 方向 200 像素、Y 方向 50 像素的存储设备，它还应兼容 4bpp 色彩深度的显示器并支持透明。所需的字节数计算如下：

$$\text{所需字节数} = (200 + (200 + 7) / 8) * 50 = 11250 \text{ 字节}$$

使用存储设备通常不会显著影响性能。使用存储设备时，驱动的工作就更容易了：只需将位图传输给显示控制器。在具有较慢驱动的系统（例如通过串口连接的显示器），如果使用存储设备性能会更高；在具有快速驱动的系统（如内存映射的显示存储器、GUIDRV_Lin 以及其他），使用存储设备会牺牲某些性能。如果需要“分段”，则绘制窗口所用的时间将随着分段数增加而增加。存储设备可用的内存越多，性能也越好。

10.2 存储设备实验

10.2.1 存储设备基本 API 函数

函数	描述
GUI_MEMDEV_Clear()	将存储设备内容标志为未更改
GUI_MEMDEV_CopyFromLCD()	将 LCD 内容复制到存储设备
GUI_MEMDEV_CopyToLCD()	将存储设备内容复制到 LCD
GUI_MEMDEV_CopyToLCDAA()	复制存储设备内容并抗锯齿处理
GUI_MEMDEV_CopyToLCDAt()	复制存储设备内容到 LCD 的指定位置
GUI_MEMDEV_Create()	创建存储设备（第一步）
GUI_MEMDEV_CreateEx()	创建具有额外创建标记的存储设备
GUI_MEMDEV_CreateFixed()	创建具有给定色彩深度的存储设备
GUI_MEMDEV_Delete()	释放存储设备使用的内存
GUI_MEMDEV_DrawPerspectiveX()	将透视变形的给定存储设备绘制到当前选定设备
GUI_MEMDEV_GetDataPtr()	返回指向数据直接操作区域的指针
GUI_MEMDEV_GetXSize()	返回存储设备的 X 尺寸(宽度)
GUI_MEMDEV_GetYSize()	返回存储设备的 Y 尺寸(高度)
GUI_MEMDEV_MarkDirty()	将某矩形区域标志为脏污
GUI_MEMDEV_ReduceYSize()	减小存储设备的 Y 尺寸
GUI_MEMDEV_Rotate()	旋转并缩放存储设备，并将结果写入使用“最近邻居”法的存储设备
GUI_MEMDEV_RotateHQ()	旋转并缩放存储设备，并将结果写入使用“高品质”法的存储设备
GUI_MEMDEV_Select()	选择内存设备作为绘图操作的目标
GUI_MEMDEV_SetOrg()	改变存储设备在 LCD 上的原点
GUI_MEMDEV_Write()	将存储设备的内容写入存储设备
GUI_MEMDEV_WriteAlpha()	将存储设备的内容写入使用 Alpha 混合处理的存储设备
GUI_MEMDEV_WriteAlphaAt()	将存储设备的内容写入使用给定位置和 Alpha 混合处理的存储设备
GUI_MEMDEV_WriteAt()	将存储设备的内容写入存储设备的给定位置
GUI_MEMDEV_WriteEx()	将存储设备的内容写入使用 Alpha 混合处理和缩放的存储设备
GUI_MEMDEV_WriteExAt()	将存储设备的内容写入使用 Alpha 混合处理和缩放的存储设备的给定位置
GUI_SelectLCD()	选择作为绘制操作目标的

表 10.2.1 存储设备 API 函数

接下来，我们讲解一下，比较常用的存储设备的 API 函数，其他相关的 API 函数，大家可以看 emWin 手册。

(1) GUI_MEMDEV_CopyToLCD 函数

将存储设备内容复制到 LCD，该函数原型如下源码所示：

```
void GUI_MEMDEV_CopyToLCD(GUI_MEMDEV_Handle hMem);
```

该函数的相关形参如下表所示：

参数	描述
hMem	内存设备的句柄

表 10.2.2 GUI_MEMDEV_CopyToLCD 函数相关形参描述

(2) GUI_MEMDEV_Create 函数

创建内存设备，该函数原型如下源码所示：

```
GUI_MEMDEV_Handle GUI_MEMDEV_Create(int x0, int y0, int xSize, int ySize);
```

该函数的相关形参如下表所示：

参数	描述
x0	内存设备的 X 位置
y0	内存设备的 Y 位置
xSize	内存设备的 X 尺寸
ySize	内存设备的 Y 尺寸

表 10.2.3 GUI_MEMDEV_Create 函数相关形参描述

返回值：已创建内存设备的句柄。函数失败时返回值为 0。

(3) GUI_MEMDEV_CreateEx 函数

创建内存设备，该函数原型如下源码所示：

```
GUI_MEMDEV_Handle GUI_MEMDEV_CreateEx(int x0, int y0, int xSize,
                                        int xSize, int Flags);
```

该函数的相关形参如下表所示：

参数	描述
x0	内存设备的 X 位置
y0	内存设备的 Y 位置
xSize	内存设备的 X 尺寸
ySize	内存设备的 Y 尺寸
Flags	参数 Flags 的允许值
	GUI_MEMDEV_HASTRANS(推荐) 默认：使用透明性标记创建内存设备，该标记确保正确绘制背景
	GUI_MEMDEV_NOTRANS 创建内存设备，无透明性。用户必须确保正确绘制背景。这样可将内存设备用于非矩形区域。另一优势是速度较高：使用此标记可加速内存设备约 30 - 50%

表 10.2.4 GUI_MEMDEV_CreateEx 函数相关形参描述

返回值：已创建内存设备的句柄，函数失败时返回值为 0。

(4) GUI_MEMDEV_CreateFixed 函数

创建具有专用颜色深度颜色转换的内存设备，该函数原型如下源码所示：

```
GUI_MEMDEV_Handle GUI_MEMDEV_CreateFixed(int x0, int y0,
                                           int xSize, int ySize, int Flags,
                                           const tLCDDEV_APIList *pMemDevAPI,
                                           const LCD_API_COLOR_CONV *pColorConvAPI);
```

该函数的相关形参如下表所示：

参数	描述
x0	内存设备的 X 位置
y0	内存设备的 Y 位置
xSize	内存设备的 X 尺寸
ySize	内存设备的 Y 尺寸
Flags	参数 Flags 的允许值
	GUI_MEMDEV_HASTRANS 默认：使用透明性标记创建内存设备，该标记确保正确绘制背景
	GUI_MEMDEV_NOTRANS 创建内存设备，无透明性。用户必须确

		保正确绘制背景。这样可将内存设备用于非矩形区域。另一优势是速度较高：使用此标记可加速内存设备约 30 - 50%
pMemDevAPI	定义内存设备的颜色深度，单位 bpp 。内存设备的颜色深度应等于或大于颜色转换函数要求的位数。	
	GUI_MEMDEV_APILIST_1	创建颜色深度为 1bpp 的内存设备（每 8 像素 1 字节）用于指定颜色转换要求 1bpp
	GUI_MEMDEV_APILIST_8	创建颜色深度为 8bpp 的内存设备（每像素 1 字节）用于指定颜色转换要求 8bpp 或更低
	GUI_MEMDEV_APILIST_16	创建颜色深度为 16bpp 的内存设备（每像素 1U16）用于指定颜色转换要求高于 8bpp 时（高色彩模式）
	GUI_MEMDEV_APILIST_32	创建颜色深度为 32bpp 的内存设备（每像素 1U32）用于指定颜色转换要求高于 16bpp 时（真彩色模式）
pColorConvAPI	此参数定义所需的颜色转换	
	GUICC_1	固定调色板模式 1（黑色/白色）
	GUICC_2	固定调色板模式 2（4 级灰度）
	GUICC_4	固定调色板模式 4（16 级灰度）
	GUICC_565	固定调色板模式 565
	GUICC_M565	固定调色板模式 M565
	GUICC_8666	固定调色板模式 M565
	GUICC_888	固定调色板模式 888
	GUICC_8888	固定调色板模式 8888

表 10.2.5 GUI_MEMDEV_CreateFixed 函数相关形参描述

返回值：已创建内存设备的句柄，函数失败时返回值为 0。

(5) GUI_MEMDEV_Delete 函数

删除内存设备，该函数原型如下源码所示：

```
void GUI_MEMDEV_Delete(GUI_MEMDEV_Handle MemDev);
```

该函数的相关形参如下表所示：

参数	描述
MemDev	要删除的内存设备的句柄

表 10.2.6 GUI_MEMDEV_Delete 函数相关形参描述

返回值：无。

(6) GUI_MEMDEV_Select 函数

激活内存设备（如果句柄为 0，则激活 LCD），该函数原型如下源码所示：

```
GUI_MEMDEV_Handle GUI_MEMDEV_Select(GUI_MEMDEV_Handle hMemDev);
```

该函数的相关形参如下表所示：

参数	描述
hMemDev	内存设备的句柄

表 10.2.7 GUI_MEMDEV_Select 函数相关形参描述

返回值：先前选择的设备。如果选择了显示器则为 0。

(7) GUI_MEMDEV_Write 函数

给定内存设备的内容写入当前选定的内存设备中，该函数原型如下源码所示：

```
void GUI_MEMDEV_Write(GUI_MEMDEV_Handle hMem);
```


该函数的相关形参如下表所示：

参数	描述
hMem	内存设备的句柄

表 10.2.8 GUI_MEMDEV_Write 函数相关形参描述

(8) GUI_MEMDEV_WriteAt 函数

给定内存设备的内容写入当前选定内存设备的指定位置中，该函数原型如下源码所示：

```
void GUI_MEMDEV_WriteAt(GUI_MEMDEV_Handle hMem, int x, int y);
```

该函数的相关形参如下表所示：

参数	描述
hMem	内存设备的句柄
x	X 方向位置
y	Y 方向位置

表 10.2.9 GUI_MEMDEV_WriteAt 函数相关形参描述

(9) GUI_MEMDEV_WriteEx 函数

使用 Alpha 混合和缩放将给定内存设备的内容写入当前选定内存设备的位置(0,0)，该函数原型如下源码所示：

```
void GUI_MEMDEV_WriteEx(GUI_MEMDEV_Handle hMem, int xMag, int yMag, int Alpha);
```

该函数的相关形参如下表所示：

参数	描述
hMem	内存设备的句柄
xMag	X 轴的缩放系数*1000
yMag	Y 轴的缩放系数*1000
Alpha	Alpha 混合因子（0 – 255）

表 10.2.10 GUI_MEMDEV_WriteEx 函数相关形参描述

上述的函数，可以查看官方的中文手册或者英文手册，目前 ST 的手册是 5.4.2 版本是中文的（该中文手册在资料盘(A 盘)→6，软件资料→3，emWin 学习资料可以找到），如果下载 6.1 以上的 emWin 手册都是英文手册，有些函数是一样的，不过高版本会有新增的功能。

存储设备的使用方法：

- ✧ 创建存储设备（GUI_MEMDEV_Create()/GUI_MEMDEV_CreateFixed()）。
- ✧ 要把显示的东西放到那个存储设备（GUI_MEMDEV_Select()）。
- ✧ 绘制显示操作（用户绘画的函数）。
- ✧ 选中显示设备（GUI_MEMDEV_Select(0)为 LCD）。
- ✧ 开始显示（GUI_MEMDEV_Write()/GUI_MEMDEV_CopyToLCD）。

10.2.2 硬件设计

1. 例程功能

本实验主要测试 emWin 存储设备 API 函数使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数创建存储设备，然后把绘制内容复制到存储设备当中，其次调用函数 GUI_MEMDEV_CopyToLCDAt 把存储设备的

内容从内存复制到 LCD 的指定位置，最后在屏幕的左边是没有使用存储设备的绘制内容，而屏幕的右边的绘制内容使用了存储设备绘制。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 8 存储设备》。

10.2.3 软件设计

10.2.3.1 程序流程图

本实验的程序流程图，如下图所示：

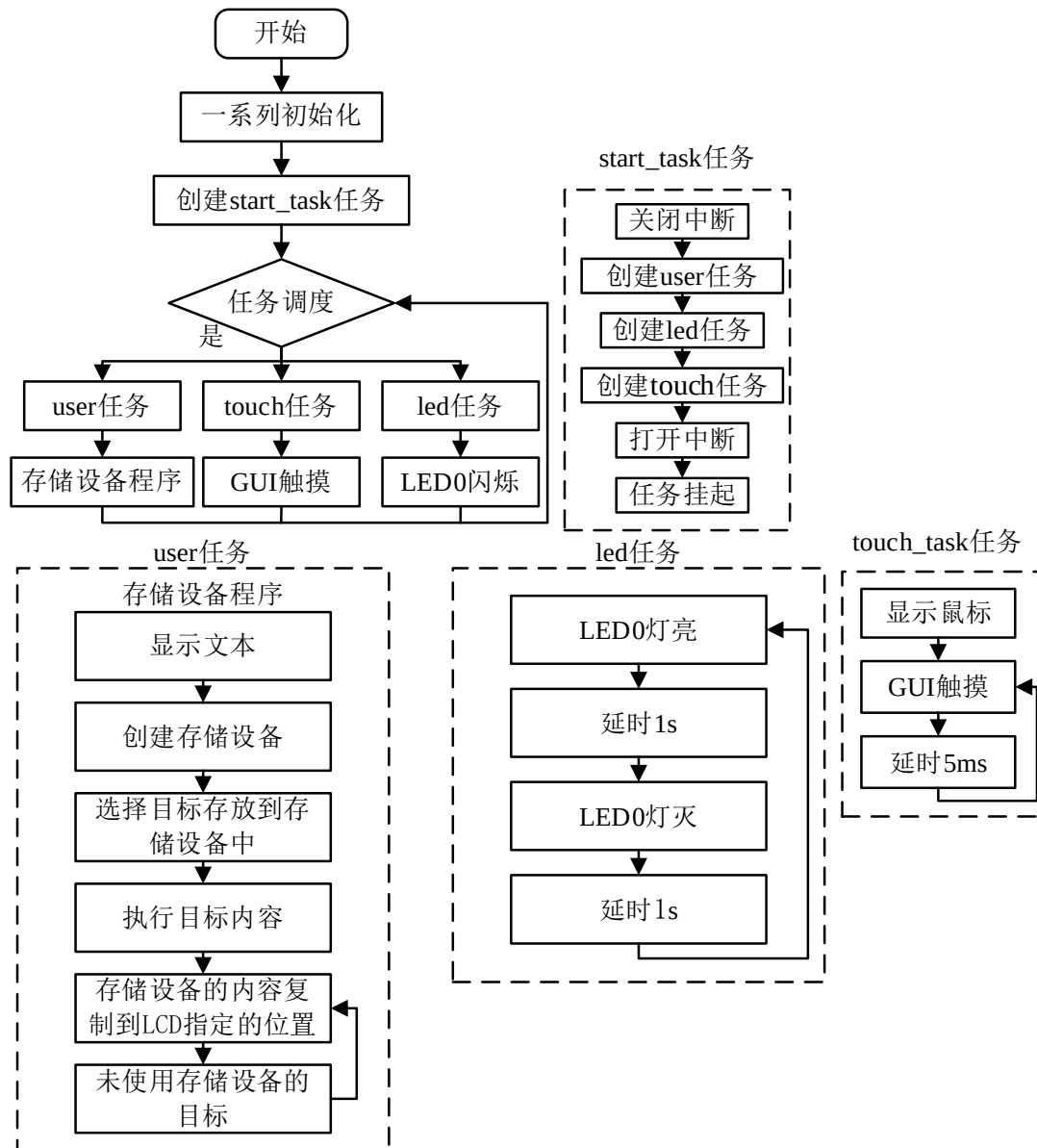


图 10.2.3.1.1 存储设备实验流程图

10.2.3.2 程序解析

下面程序延时了一下使用和不使用存储设备的区别，实验完整工程为：**emWin 实验 8 存储设备**，打开 `emwin_demo.c` 文件，源码如下：

```
static GUI_RECT Rect = {200, 150, 300, 210};

/**
 * @brief      未使用存储设备绘制
 * @param      Delay : 延时
 * @return     无
 */
static void emwin_draw(int Delay)
{
    GUI_SetPenSize(5);
    GUI_SetColor(GUI_RED);
    GUI_DrawLine(Rect.x0 + 3, Rect.y0 + 3, Rect.x1 - 3, Rect.y1 - 3);
    GUI_Delay(Delay);
    GUI_SetColor(GUI_GREEN);
    GUI_DrawLine(Rect.x0 + 3, Rect.y1 - 3, Rect.x1 - 3, Rect.y0 + 3);
    GUI_Delay(Delay);
    GUI_SetColor(GUI_WHITE);
    GUI_SetFont(&GUI_FontComic24B_ASCII);
    GUI_SetTextMode(GUI_TM_TRANS);
    GUI_DispStringInRect("Closed", &Rect, GUI_TA_HCENTER | GUI_TA_VCENTER);
    GUI_Delay(Delay);
}

/**
 * @brief      emwin_main(emwin 入口)
 * @param      无
 * @return     无
 */
void emwin_main (void)
{
    GUI_MEMDEV_Handle hMem;    /* 存储设备的句柄 */
    int i;
    GUI_SetBkColor(GUI_BLUE);
    GUI_Clear();
    GUI_SetColor(GUI_YELLOW);
    GUI_SetFont(&GUI_Font24_ASCII);
    GUI_DispStringHCenterAt("MEMDEV_MemDev - Sample", 400, 0);
    GUI_SetFont(&GUI_Font20_ASCII);
    GUI_DispStringHCenterAt("Shows the advantage
                            of using a\\nmemorydevice", 400, 50);
    GUI_SetFont(&GUI_Font16_ASCII);
}
```

```
GUI_DispStringHCenterAt("Draws the picture\
                        nwithouta\nmemorydevice",250, 100);
GUI_DispStringHCenterAt("Draws the picture\
                        nusing a\nmemory device",550, 100);

/* 创建存储设备 */
hMem = GUI_MEMDEV_Create(Rect.x0, Rect.y0,
                        Rect.x1 - Rect.x0, Rect.y1 - Rect.y0);

GUI_MEMDEV_Select(hMem);          /* 选择 hMem 作为绘制操作目标的存储设备 */
emwin_draw(0);                    /* 执行要显示的内容复制到存储设备中 */
GUI_MEMDEV_Select(0);             /* 选中 LCD */
while (1)
{
    for (i = 0; i < 3; i++)
    {
        GUI_Delay(250);
        GUI_ClearRect( LCD_GetXSize() - Rect.x1, Rect.y0, LCD_GetXSize(),
                        Rect.y1);    /* 清除选中的屏幕 */
        GUI_Delay(250);
        /* 将存储设备的内容从内存复制到 LCD 的指定位置 */
        GUI_MEMDEV_CopyToLCDAt(hMem, LCD_GetXSize() - Rect.x1, Rect.y0);
    }
    GUI_Delay(500);
    /* 未使用存储设备 */
    emwin_draw(400);
    GUI_Delay(400);
    GUI_ClearRect(0, 150, 799, 210);
}
}
```

在上述的源码中，创建存储设备与选择存储设备是有规律的，如下源码所示：

```
hMem = GUI_MEMDEV_Create(Rect.x0, Rect.y0,
                        Rect.x1 - Rect.x0,
                        Rect.y1 - Rect.y0);

GUI_MEMDEV_Select(hMem);          /* 选择 hMem 作为绘制操作目标的存储设备 */
_Draw(0);                        /* 执行要显示的内容复制到存储设备中 */
GUI_MEMDEV_Select(0);             /* 选中 LCD */
```

第一：创建存储设备 GUI_MEMDEV_Create。

第二：选择存储设备作为绘制的存储设备 GUI_MEMDEV_Select。

第三：需要绘制的内容（自己的绘制源码）。

第四：绘制的内容在 LCD 显示 GUI_MEMDEV_Select。

第五：GUI_MEMDEV_CopyToLCDAt将存储设备的内容从内存复制到LCD的指定位置。

上面程序中，我们显示通过显示红线、绿线、“Closed”这三个元素来展示使用存储设备的区别，在屏幕的右侧我们使用了存储设备，可以看到红线、绿线、“Closed”这三个元素是同时出来的，左边的是红线、绿线、“Closed”依次出现的。

注意： 如果不需要存储设备的话，保留上述第二步即可，其他不需要。

10.2.4 下载验证

程序运行效果如下图所示，**注意：** 此程序是在 4.3 寸，800*480 分辨率上调试的，如果在 2.8 或者 3.5 寸屏幕上会显示不完全，请大家自行稍微修改一下程序。

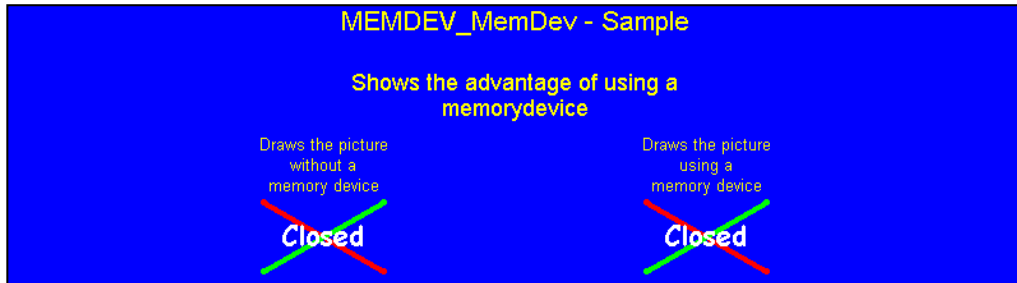


图 10.2.4.1 存储设备效果图

10.3 分段存储设备实验

10.3.1 分段存储简介

前面我们说过，使用存储设备的最大好处就是防止屏幕闪烁，使用存储设备的时候首先通过执行指定绘制函数来填充存储设备，填充后，将存储设备中的内容绘制到 LCD 中。但是有时候没有用于一次性存储整个输出区域的内存可用，此时分段存储设备就派上用场了。分段存储设备将绘制区域分成多段，每段包含当前可用内存可包含的最多行。

分段存储设备就一个 API 函数 GUI_MEMDEV_Draw，函数描述如下：

GUI_MEMDEV_Draw 函数

防止显示器闪烁，该函数的原型如下源码所示：

```
int GUI_MEMDEV_Draw(GUI_RECT* pRect, GUI_CALLBACK_VOID_P* pfDraw,
                    void* pData, int NumLines, int Flags);
```

该函数的形参如下表所示：

参数	描述	
pRect	所使用 LCD 区域的 GUI_RECT 结构的指针	
pfDraw	执行绘制的回调函数的指针	
pData	用作回调函数参数的数据结构体的指针	
NumLines	0(推荐)或存储设备的行数	
Flags	GUI_MEMDEV_HASTRANS(推荐)	默认：使用透明性标记创建存储设备，该标记确保正确绘制背景
	GUI_MEMDEV_NOTRANS	创建存储设备，无透明性。用户必须确保正确绘制背景，仅应用于优化目的

表 10.3.1.1 函数参数描述表

返回值： 成功时为 0，例程失败则为 1。

注意： 如果参数 NumLines 为 0，则每段的行数由该函数自动计算。然后函数通过移动存储设备的原点，逐段迭代输出区域。

10.3.2 硬件设计

1. 例程功能

本实验主要测试 emWin 分段存储设备 API 函数使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数创建了一个旋转多边形，该旋转多边形绘制在屏幕上需要 USE_BANDING_MEMDEV 旋转，如果这个宏定义为 1，则这个旋转的多边形绘制由分段存储函数绘制，该图形绘制时不会出现闪烁；如果该宏定义为 0，则直接绘制旋转多边形，该图形绘制时会闪烁。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 9 分段存储设备》。

10.3.3 软件设计

10.3.3.1 程序流程图

本实验的程序流程图，如下图所示：

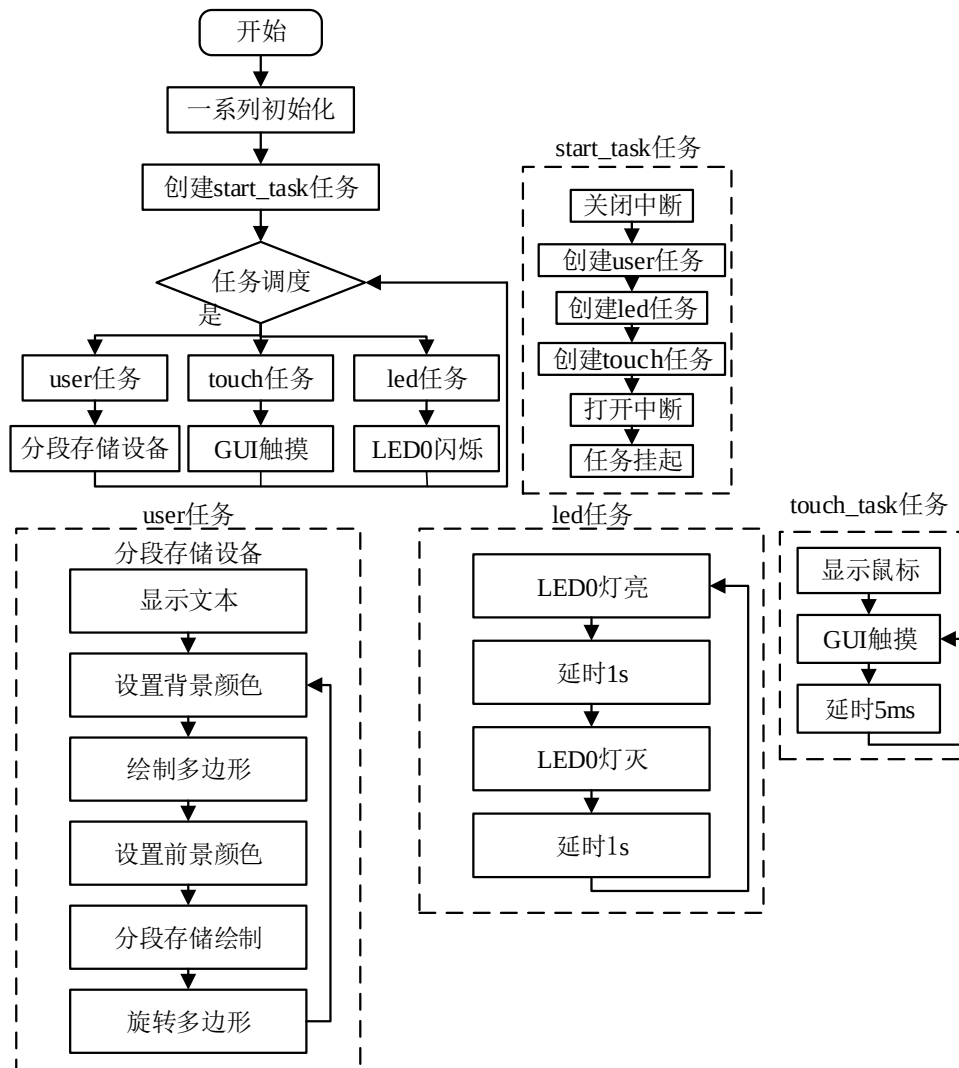


图 10.3.3.2.1 分段存储设备实验流程图

10.3.3.2 程序解析

以下程序演示了分段存储设备的效果，实验完整工程为：**emWin 实验 9 分段存储设备**，打开 emwin_demo.c 文件，程序源码如下：

```
#define USE_BANDING_MEMDEV (1) /* 定义使用分段存储 */
#define SIZE_OF_ARRAY(Array) (sizeof(Array) / sizeof(Array[0]))

/* 多边形点坐标 */
static const GUI_POINT aPoints[] = {
    {-50, 0},
    {-10, 10},
    { 0, 50},
    { 10, 10},
    { 50, 0},
    { 10,-10},
    { 0,-50},
    {-10,-10}
};

typedef struct
{
    int XPos_Poly;
    int YPos_Poly;
    int XPos_Text;
    int YPos_Text;
    GUI_POINT aPointsDest[8];
} tDrawItContext;

/**
 * @brief      绘画多边形
 * @param      无
 * @return     无
 */
static void emwin_draw_it(void * pData)
{
    tDrawItContext * pDrawItContext = (tDrawItContext *)pData;
    GUI_Clear();
    GUI_SetFont(&GUI_Font8x8);
    GUI_SetTextMode(GUI_TM_TRANS);

    /* 绘制前景 */
    GUI_SetColor(GUI_GREEN);
    GUI_FillRect(pDrawItContext->XPos_Text,
                pDrawItContext->YPos_Text - 25,
```

```

        pDrawItContext->XPos_Text + 100,
        pDrawItContext->YPos_Text - 5);

/* 绘制多边形 */
GUI_SetColor(GUI_BLUE);
GUI_FillPolygon(pDrawItContext->aPointsDest,
                SIZE_OF_ARRAY(aPoints),160,120);

/* 绘制前景 */
GUI_SetColor(GUI_RED);
GUI_FillRect(220 - pDrawItContext->XPos_Text,
             pDrawItContext->YPos_Text + 5,
             220 - pDrawItContext->XPos_Text + 100,
             pDrawItContext->YPos_Text + 25);
}

/**
 * @brief      emwin_main(emwin 入口)
 * @param      无
 * @return     无
 */
void emwin_main (void)
{
    tDrawItContext DrawItContext;
    int i, swap=0;
    GUI_SetBkColor(GUI_BLACK);
    GUI_Clear();
    GUI_SetColor(GUI_YELLOW);
    GUI_SetFont(&GUI_Font24_ASCII);
    GUI_DispStringHCenterAt("MEMDEV_Banding - Sample", 160, 5);
    GUI_SetFont(&GUI_Font16_ASCII);
    GUI_DispStringHCenterAt("Banding memory device\nwithout flickering",160,40);
    DrawItContext.XPos_Poly = 160;
    DrawItContext.YPos_Poly = 120;
    DrawItContext.YPos_Text = 116;

    while (1)
    {
        swap = ~swap;
        for (i = 0; i < 220; i++)
        {
            float angle = i * 3.1415926 / 55;
            DrawItContext.XPos_Text = (swap) ? i : 220 - i;

```



```

/* 旋转多边形 */
GUI_RotatePolygon(DrawItContext.aPointsDest, aPoints,
                  SIZE_OF_ARRAY(aPoints), angle);

#ifdef USE_BANDING_MEMDEV
{
    GUI_RECT Rect = {0, 70, 320, 170};
    /* 使用分段存储绘制 */
    GUI_MEMDEV_Draw(&Rect, &DrawIt, &DrawItContext, 0, 0);
}
#else
{
    /* 如果没有开启分段存储的话就使用普通绘制方式 */
    emwin_draw_it((void *)&DrawItContext);
}
#endif
}
}
}

```

上面程序中两个重要的函数 `emwin_draw_it` 和 `_DemoBandingMemdev`，`emwin_draw_it` 函数用来绘制绿色矩形的背景、多边形和红色矩形的前景。`emwin_banding_memdev` 函数里面有一个条件编译，当宏定义 `USE_BANDING_MEMDEV` 定义为 1 的话就会使用分段存储设备，当该宏定义 `USE_BANDING_MEMDEV` 定义为 0 的话就不使用分段存储设备，大家可以通过修改该宏定义 `USE_BANDING_MEMDEV` 来观察使用和不使用分段存储设备两种情况下的区别。

10.3.4 下载验证

程序运行效果如下图所示：

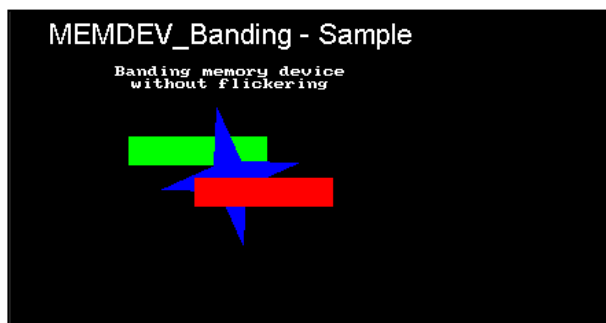


图 10.3.4.1 分段存储设备运行效果图

10.4 自动设备对象实验

10.4.1 自动设备对象简介

有时候我们只需要更新 LCD 的一小部分内容，如指针，针对这种情况，emWin 提供了另外一种解决方法——自动设备对象。自动设备对象以分段存储设备为基础，对于一次仅更新显示器一小部分的应用而言可能更高效。该设备会自动识别包含固定对象的显示器区域，以及包

含必须更新的移动或更改对象的区域。第一次调用绘制函数时，会绘制所有项目，每一次后续调用都将仅更新移动或更改对象使用的空间，实际绘制操作使用分段存储设备，但仅限在必要空间内。使用自动设备对象的主要优势（相对于使用分段存储设备而言）是计算时间减少，因为它不始终更新整个显示器。

10.4.2 自动设备对象 API 函数

自动设备对象 API 函数如下表所示：

函数	描述
GUI_MEMDEV_CreateAuto()	创建自动设备对象
GUI_MEMDEV_DeleteAuto()	删除自动设备对象
GUI_MEMDEV_DrawAuto()	使用 GUI_AUTODEV 对象进行绘制

表 10.4.2.1 自动设备对象 API 函数

(1) GUI_MEMDEV_CreateAuto 函数

创建自动设备对象，该函数原型如下源码所示：

```
int GUI_MEMDEV_CreateAuto(GUI_AUTODEV * pAutoDev);
```

该函数的相关形参如下表所示：

参数	描述
pAutoDev	指向 GUI_AUTODEV 对象的指针

表 10.4.2.2 GUI_MEMDEV_CreateAuto 函数相关形参描述

返回值：当前为 0，为以后使用保留。

(2) GUI_MEMDEV_DeleteAuto 函数

删除自动设备对象，该函数原型如下源码所示：

```
void GUI_MEMDEV_DeleteAuto(GUI_AUTODEV * pAutoDev);
```

该函数的相关形参如下表所示：

参数	描述
pAutoDev	指向 GUI_AUTODEV 对象的指针

表 10.4.2.3 GUI_MEMDEV_DeleteAuto 函数相关形参描述

(3) GUI_MEMDEV_DrawAuto 函数

分段内存设备执行指定绘制函数，该函数原型如下源码所示：

```
int GUI_MEMDEV_DrawAuto(GUI_AUTODEV * pAutoDev, GUI_AUTODEV_INFO * pAutoDevInfo,
                        GUI_CALLBACK_VOID_P * pfDraw, void * pData);
```

该函数的相关形参如下表所示：

参数	描述
pAutoDev	指向 GUI_AUTODEV 对象的指针
pAutoDevInfo	指向 GUI_AUTODEV_INFO 对象的指针
pfDraw	指向要执行的用户定义绘制函数的指针
pData	指向传递给绘制函数的数据结构体的指针

表 10.4.2.4 GUI_MEMDEV_DrawAuto 函数相关形参描述

自动设备对象使用方法如下：

1. 先调用 GUI_MEMDEV_CreateAuto 函数创建一个自动设备对象。
2. 函数 GUI_MEMDEV_DrawAuto 参数 1 就是我们在上一步中创建的自动设备对象，参数 2 是一个结构体 GUI_AUTODEV_INFO，这个结构体如下：

```
typedef struct
{
```

```
char DrawFixed;
char IsMeasurement;
} GUI_AUTODEV_INFO;
```

如果必须绘制所有项目，则 **DrawFixed** 设置为 1；如果仅绘制移动或者更改对象，则设置为 0。使用此功能的话可以按照如下方式：

```
/* 此结构体必须包含 GUI_AUTODEV_INFO 类型变量，此结构体中也可以添加其他用户需要的参数 */
typedef struct{
    /* 结构体 GUI_AUTODEV_INFO 变量,函数 GUI_MEMDEV_DrawAuto() 要用到这个变量 */
    GUI_AUTODEV_INFO AutoDevInfo;
    ...          /* 其他用户所需参数 */
} PARAM;
/* 绘制函数 */
static void Draw(void *p)
{
    PARAM * pParam = (PARAM*)p;
    if (pParam->AutoDevInfo.DrawFixed)
    {
        ...      /* 绘制固定的背景 */
    }
    /* 绘制移动的内容，比如鼠标指针等 */
    if (pParam->AutoDevInfo.DrawFixed)
    {
        ...      /* 绘制固定的前景 */
    }
}

void emwin_main(void)
{
    PARAM Param;
    GUI_AUTODEV AutoDev; /* 定义一个自动设备对象*/
    /* 设置或者定义其他用户所需内容，比如设置字体，颜色，显示一些提示信息等 */
    ...
    GUI_MEMDEV_CreateAuto(&AutoDev);          /* 创建自动设备对象 AutoDev */
    GUI_MEMDEV_DrawAuto(&AutoDev,             /* 使用分段存储设备执行指定绘制例程 */
                        &Param.AutoDevInfo, &Draw, &Param);
    GUI_MEMDEV_DeleteAuto(&AutoDev);          /* 删除自动设备对象 */
}
```

在下一小节的自动设备对象演示例程中，大家就可以看到我们基本就是按照这个顺序来写程序的。

10.4.3 硬件设计

1. 例程功能

本实验主要测试 emWin 自动设备对象 API 函数使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数创建了一个旋转多边形，该旋转多边形绘制在屏幕上由函数 GUI_MEMDEV_DrawAuto 的第三个形参传入回调函数 emwin_draw 进行绘画，如果我们不使用自动设备对象方法，可直接调用函数 emwin_draw 绘画即可。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 10 自动设备对象》。

10.4.4 软件设计

10.4.4.1 程序流程图

本实验的程序流程图，如下图所示：

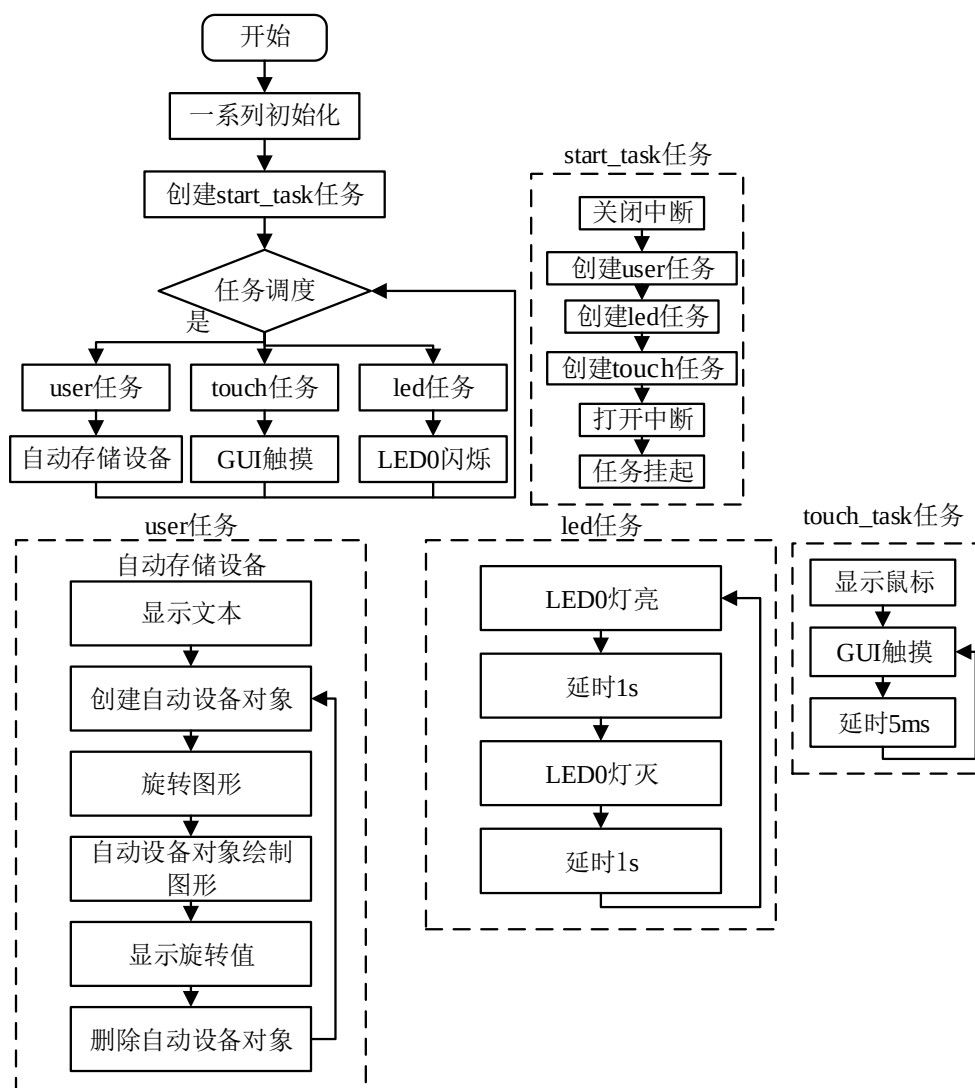


图 10.4.4.1.1 自动存储设备实验流程图

10.4.4.2 程序解析

以下程序演示了自动设备对象，实验完整工程为：**emWin 实验 10 自动设备对象**，打开 emwin_demo.c 文件，程序源码如下：

```
#define countof(Obj)    (sizeof(Obj) / sizeof(Obj[0]))
#define DEG2RAD          (3.1415926f / 180)
#define MAG              4

static const GUI_COLOR ColorsScaleR140[] = {
    0x000000, 0x00AA00, 0xFFFFFf, 0x0000AA,
    0x00FF00, 0xAEAEAE, 0x737373, 0xD3D3D3,
    0xDFDFDF, 0xBBDFBB, 0x6161DF, 0x61DF61,
    0xBBBDDF, 0xC7C7C7, 0x616193
};

/* 调色板 */
static const GUI_LOGPALETTE PalScaleR140 = {
    15,                /* 条目的数量 */
    0,                 /* 无透明度 */
    &ColorsScaleR140[0]
};

static const unsigned char acScaleR140[] = {
    /* 限于篇幅原因，此处省略掉了数组中的内容，至于数组中的具体内容大家可以查阅实验 */
};

static const GUI_BITMAP bmScaleR140 = {
    200,                /* XSize */
    73,                 /* YSize */
    100,                /* BytesPerLine */
    4,                  /* BitsPerPixel */
    acScaleR140,        /* Pointer to picture data (indices) */
    &PalScaleR140        /* Pointer to palette */
};

/* 多边形的外形 */
static const GUI_POINT _aNeedle[] = {
    { MAG * ( 0), MAG * ( 0 + 125) },
    { MAG * (-3), MAG * (-15 + 125) },
    { MAG * (-3), MAG * (-65 + 125) },
    { MAG * ( 3), MAG * (-65 + 125) },
    { MAG * ( 3), MAG * (-15 + 125) },
};
```

```
};

/* PARAM, 包含绘图所需信息的结构体 */
typedef struct {
    GUI_AUTODEV_INFO    AutoDevInfo;
    GUI_POINT            aPoints[7];    /* Polygon data */
    float                Angle;
} PARAM;

/**
 * @brief        获取角度,这里只是模拟。在实际使用中这个角度应该是一个实际值
 * @param        tDiff : 传入时间参数
 * @return        无
 */
static float emwin_get_angle(int tDiff)
{
    if (tDiff < 15000)
    {
        return 225 - 0.006 * tDiff ;
    }
    tDiff -= 15000;

    if (tDiff < 7500)
    {
        return 225 - 90 + 0.012 * tDiff ;
    }
    return 225;
}

/**
 * @brief        绘图
 * @param        p : PARAM
 * @return        无
 */
static void emwin_draw(void * p)
{
    PARAM * pParam;
    pParam = (PARAM *)p;
    /* 显示背景 */
    if (pParam->AutoDevInfo.DrawFixed)
    {
        GUI_ClearRect (60,80 + bmScaleR140.YSize, 60 + bmScaleR140.XSize-1,180);
        GUI_DrawBitmap(&bmScaleR140, 60, 80);
    }
}
```

```

    }

    /* 移动指针 */
    GUI_SetColor(GUI_WHITE);
    /* 绘制实心无锯齿多边形 */
    GUI_AA_FillPolygon(pParam->aPoints, countof(_aNeedle), MAG * 160, MAG * 220);
    /* 显示前景色 */
    if (pParam->AutoDevInfo.DrawFixed)
    {
        GUI_SetTextMode(GUI_TM_TRANS);
        GUI_SetColor(GUI_RED);
        GUI_SetFont(&GUI_Font24B_ASCII);
        GUI_DispStringHCenterAt("RPM / 1000", 160, 140);
    }
}

/**
 * @brief      emwin_main(emwin 入口)
 * @param      无
 * @return     无
 */
void emwin_main (void)
{
    GUI_AUTODEV AutoDev;          /* Object for banding memory device */
    PARAM      Param;             /* Parameters for drawing routine */
    int        Cnt;
    int        tDiff;
    int        t0;

    /* 显示信息 */
    GUI_SetBkColor(GUI_BLACK);
    GUI_Clear();
    GUI_SetColor(GUI_WHITE);
    GUI_SetFont(&GUI_Font24_ASCII);
    GUI_DispStringHCenterAt("MEMDEV_AutoDev - Sample", 160, 5);
    GUI_SetFont(&GUI_Font8x16);
    GUI_DispStringHCenterAt("Scale using GUI_AUTODEV-object", 160, 50);
    /* 使能高分辨率抗锯齿 */
    GUI_AA_EnableHiRes();
    GUI_AA_SetFactor(MAG);
    while (1)
    {
        t0 = GUI_GetTime();
        GUI_MEMDEV_CreateAuto(&AutoDev);          /* 创建自动设备对象 */
        for (Cnt = 0; (tDiff = GUI_GetTime() - t0) < 24000; Cnt++)

```

```
{
    /* 按照指定角度旋转多边形 */
    Param.Angle = _GetAngle(tDiff) * DEG2RAD;
    GUI_RotatePolygon(Param.aPoints, _aNeedle, countof(_aNeedle),
        Param.Angle);

    /* 使用自动设备对象绘制图形 */
    GUI_MEMDEV_DrawAuto(&AutoDev, &Param.AutoDevInfo,
        &emwin_draw, &Param);
}

/* 显示 milliseconds / picture 图片 */
GUI_SetColor(GUI_WHITE);
GUI_SetFont(&GUI_Font8x16);
GUI_DispStringHCenterAt("Milliseconds / picture:", 160, 200);
GUI_SetTextAlign(GUI_TA_CENTER);
GUI_SetTextMode(GUI_TM_NORMAL);
GUI_DispNextLine();
GUI_GotoX(160);
GUI_DispFloatMin((float)tDiff / (float)Cnt, 2);

/* 删除自动设备对象 */
GUI_MEMDEV_DeleteAuto(&AutoDev);
GUI_Delay(3000);
GUI_ClearRect(0, 70, 319, 239);
}
}
```

上面程序基本就是按照我们上一小节中讲到的思路和方法来编写的。一开始我们先定义了一个位图 `bmScaleR140`，因为实验的背景色就是一个汽车的速度表盘，是固定图案，因此这个图案可以按照绘制位图的方式来绘制，通过调用函数 `GUI_DrawBitmap` 来绘制位图，其他部分程序大家就可以按照上一小节讲到的思路分析一下。

10.4.5 下载验证

程序运行结果如下图所示：

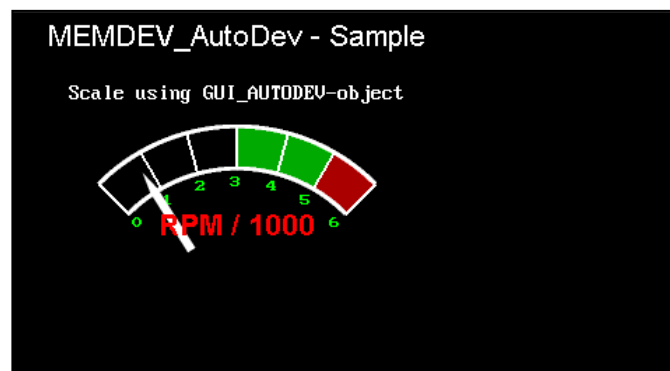


图 10.4.5.1 自动设备对象运行效果图

第十一章 窗口管理器

大家在使用电脑的时候对于窗口应该是非常熟悉的了，窗口是用户界面中最重要的部分，它是屏幕上与一个应用程序相对应的矩形区域，包括框架和客户区，是用户与产生该窗口的应用程序之间的可视界面。每当用户开始运行一个应用程序时，应用程序就创建并显示一个窗口；当用户操作窗口中的对象时，程序会作出相应反应。用户通过关闭一个窗口来终止一个程序的运行；通过选择相应的应用程序窗口来选择相应的应用程序。emWin 同样提供了窗口管理器的功能，本章我们就讲解一下 emWin 的窗口管理。

本章分为以下几个部分：

- 11.1 什么是窗口管理器
- 11.2 窗口术语说明
- 11.3 窗口回调机制
- 11.4 消息
- 11.5 窗口重绘实验
- 11.6 窗口管理 API 函数实验
- 11.7 窗口剪裁实验

11.1 什么是窗口管理器

窗口就是屏幕上的一个个矩形区域，窗口管理器就是这些矩形区域的管理者，它控制窗口的大小、位置和提供用户去操作这些窗口的方法。简单来讲，窗口管理器提供一系列的函数，利用这些函数很容易地对特定窗口进行创建、移动和调整大小等操作。

11.2 窗口术语说明

窗口是由其原点（左上角的 X 和 Y 坐标）以及 X 和 Y 值（分别为宽度和高度）定义的矩形，emWin 中的窗口有如下特点：

- ✧ 是矩形的
- ✧ 具有 Z 位置
- ✧ 可以隐藏或显示
- ✧ 可具有有效和无效区域
- ✧ 可以透明或不透明
- ✧ 可以具有或不具有回调例程

(1) 活动窗口

当前正用于绘制操作的窗口称为活动窗口，不一定就是最上面的窗口。

(2) 回调例程

回调例程由用户程序定义，指示在特定事件出现时图形系统调用特定的函数。它们通常用于在窗口内容更改时自动重绘窗口。

(3) 子窗口/父窗口，同属窗口

子窗口是相对于其他窗口（称为父窗口）定义的。只要父窗口移动，其子窗口就会相应移动。子窗口始终完全包含在其父窗口中，并在必要时会被裁剪。具有相同父窗口的多个子窗口被视为“同属”窗口。

(4) 客户区

窗口的客户区就是其可用区域。如果窗口包含边框或标题栏，则客户区是内部的矩形区域。如果没有这种边框，则客户区的坐标与窗口自身的坐标相同。

(5) 裁剪，裁剪区域

裁剪是将输出限制为一个窗口或窗口一部分的过程。窗口的裁剪区域是其可见区域。它是窗口区域减去被更高 Z 轴阶层的同属窗口遮挡的区域，然后减去没有放入父窗口可见区域的任何部分。

(6) 坐标

坐标通常是 2 维坐标，以像素单位表示，坐标由 2 个值组成。第一个值指定水平分量（也称为 x 坐标），第二个值指定垂直分量（也称为 y 坐标）。

(7) 桌面坐标

桌面坐标是桌面窗口的坐标。显示器的左上角位置（原点）为 (0,0)。

(8) 桌面窗口

桌面窗口由窗口管理器自动创建，并且始终覆盖整个显示区域。它始终是最底层的窗口，在没有定义其他窗口时，它是默认（活动）窗口。所有窗口都是桌面窗口的后代窗口（子窗口、孙窗口等）。

(9) 句柄

创建新窗口后，WM 会分配一个称为句柄的唯一标识符。句柄用于在该特定窗口中执行的其他任何操作。

(10) 隐藏/显示窗口

隐藏的窗口不可见，但是仍然存在（有一个句柄）。创建窗口时，如果不指定创建标记，默认情况下是隐藏的。显示窗口使其可见，隐藏窗口则使其不可见。

(11) 父坐标

父坐标是与父窗口相关的窗口坐标。窗口的左上角位置（原点）为 (0,0)。

(12) 透明性

具有透明区域的窗口包含不与窗口其余部分一起重绘的区域。这些区域就像其背后窗口“透过”它们显示一样。在此情况下，在有透明区域的窗口之前重绘背后窗口非常重要。WM 自动按正确的顺序进行重绘。

(13) 有效化/无效化

有效窗口是不需要重绘的完全更新窗口。无效窗口不会反映所有更新，因此需要完全或部分重绘。作出的更改影响了特定窗口时，WM 将该窗口标记为无效。下次窗口重绘时（手动或通过回调例程），将进行验证。

(14) 窗口坐标

窗口坐标是窗口的坐标。窗口的左上角位置（原点）为 (0,0)。

(15) Z 位置，底部/顶部

尽管窗口显示在以 X 和 Y 表示的二维屏幕上，但是 WM 也管理所谓的 Z 位置或深度坐标--虚拟的第三维上的位置，该坐标确定从背景到前景的位置。各窗口因此可在其他窗口之上或之下出现。将某窗口设置为底部，会将该窗口置于其所有同属窗口（如果有）的“底部”；设置为顶部，则将其置于其同属窗口的“顶部”。创建窗口时，如果不指定创建标记，默认情况下设置为顶部。

以上关于窗口的术语非常重要，请大家一定要牢记和理解！

11.3 窗口回调机制

回调机制是 emWin 的一个重要的特性和功能，WM 可在有或无回调例程（回调函数）时使用，大多情况下最好使用回调。emWin 为窗口和窗口对象（小工具）提供回调机制的根本概念是一个事件驱动系统。因为在大多数窗口式系统中，其原理是控制流不仅仅是从用户程序到图形系统，还要再通过用户程序提供的回调例程返回用户程序。此机制通常称为“好莱坞原则”（“不要打电话给我们，我们会给你打电话的！”），窗口管理器需要它的主要目的是触发窗口重绘。这与传统编程相反，但是它能利用窗口管理器的无效化逻辑。

11.3.1 不使用回调渲染

回调例程不是必须使用的，但是如果这样做，WM 会失去管理窗口重绘（更新）的能力。也可能混合使用，例如让有些窗口使用回调，有些不使用。当然，如果窗口不使用回调机制，则由应用程序负责更新其内容。

11.3.2 使用回调渲染

如果要使用回调渲染的话，那么在调用 WM_CreateWindow 函数创建窗口之前必须先定义一个回调函数，然后在调用 WM_CreateWindow 函数创建窗口的时候将这个回调函数作为参数，如下源码所示：

```
void callback(WM_MESSAGE * pMsg);
```

该函数的形参如下表所示：

参数	描述
pMsg	指向类型 WM_MESSAGE 的数据结构指针

表 11.3.2.1 callback 函数形参描述

回调函数里面具体做什么样的处理取决于接收到的消息（pMsg）的类型，一般情况下我们收到的 pMsg 里面会包含多种类型的消息，那么我们就可以使用 switch 语句来对不同的消息做出不同的处理，pMsg 里面至少会有 WM_PAINT，下一节中我们就会详细的讲解 pMsg 的消息类型。

(1) WM_PAINT 消息

窗口收到 WM_PAINT 消息时，应重绘自身，将此消息发送到窗口前。

(2) 非透明窗口（默认！）必须重绘其整个无效区域

最简单的方式是重新着色窗口的整个区域。WM 的裁剪机制确保了仅重绘无效区域。为了加速绘制过程，仅重绘无效区域非常有用。另外，透明窗口不必重绘整个无效区域；它可让窗口区域部分不受影响。此不受影响的区域会变成透明。WM 发送 WM_PAINT 消息到透明窗口之前，下面的区域已经重绘（通过发送一条 WM_PAINT 消息到下面窗口）。

(3) 注意：处理 WM_PAINT 时，不得执行某些操作

处理 WM_PAINT 消息时，回调例程除了重绘窗口的内容外，不得执行其它操作。处理 WM_PAINT 事件时，下列函数不能被调用：

- ✧ WM_SelectWindow。
- ✧ WM_Paint。
- ✧ WM_DeleteWindow。
- ✧ WM_CreateWindow。

如果用户更改窗口属性的其他任何函数也不能调用 WM_Move 和 WM_Resize 等函数。

(4) 重写回调函数

在 emWin 中控件和窗口的默认行为是在其回调函数中定义的。如果必须更改控件的行为，或者需要增强窗口的功能以满足客户的需求，建议重写内部回调函数。这是通过以下几个简单步骤完成的：

第一步：创建自定义回调函数，实现该步使用以下函数：

```
void Callback(WM_MESSAGE * pMsg);
```

第二步：消息（实现对某种消息的响应）由于自定义回调函数不需要处于自有可能的消息，所以建议使用 switch/case 条件。这使得可以轻松地添加或删除一个消息特定的代码，而不会影响其他消息。参数 pMsg 包含了消息的 ID（pMsg->MsgId）。

第三步：处理默认回调，确保所有未由自定义回调函数处理的消息由内部（默认）回调函数处理。推荐的方法是使用 switch/case 条件的 default 语句来调用内部回调函数。内部回调函数对每个窗口类型都不同。控件的内部回调函数被命名为<WIDGET>_Callback()。所有其他类型的窗口使用 WM_DefaultProc 函数进行消息处理，如下源码所示：

```
switch (pMsg->MsgId)
{
    case WM_CREATE:
        .
        .
        .
        break;
    case WM_PAINT:
        break;
    case WM_SIZE:
        .
        .
        .
        break;
    default:
        <WIDGET>_Callback(pMsg);
}
```

第四步：设置要使用的自定义回调函数，最后一步要做的是设置窗口或控件使用的新创建的回调函数。简单调用 WM_SetCallback 即可。

11.3.3 背景窗口重绘和回调

初始化窗口管理器期间，会创建一个包含整个 LCD 区域的窗口作为背景窗口，此窗口的句柄为 WM_HBKWIN。WM 不会自动重绘背景窗口的区域，因为没有默认的背景颜色，也就是说如果创建了另一个窗口然后将其删除，则删除的窗口仍然可见。需要指定函数 WM_SetDesktopColor 以便为重绘背景窗口设置颜色，也可设置回调函数来处理此问题，如果如前所述创建然后删除了一个窗口，则回调例程将触发 WM 来确认背景窗口不再有效，并自动重绘它。

11.3.4 无效化

无效化窗口或窗口的一部分会告诉 WM 该窗口的无效区域在下次调用 GUI_Exec 或 GUI_Delay 时应重绘。emWin 的无效化例程不会重绘窗口的无效部分，只管理窗口的无效区域。

✧ 窗口的无效区域

对于每个窗口，WM 只使用一个矩形来存储包含整个无效区域的最小矩形。例如，如果左上角的一小部分和右下角的一小部分变为无效，则整个窗口即无效。

✧ 使用无效化的原因

使用窗口无效化而非立即重绘每个窗口的优点是只绘制窗口一次，即使其被无效化多次。例如，如果窗口的多个属性需要更改（如背景颜色、字体和窗口大小），与所有属性都更改后重绘一次窗口相比，每个属性更改后立即重绘窗口需要更多的次数。

✧ 重绘无效窗口

函数 `GUI_Exec` 重绘所有无效窗口，这通过向每个无效窗口发送一条或多条 `WM_PAINT` 消息完成。

11.3.5 渲染透明窗口

如果需要绘制透明窗口，则 `WM` 会自动确保在透明窗口收到 `WM_PAINT` 消息前绘制窗口的背景。方法为，在向透明窗口发送 `WM_PAINT` 消息前，首先重绘透明窗口无效区域下面的所有窗口区域。为确保窗口管理器能执行透明窗口的重绘，有必要响应 `WM_PAINT` 消息并重绘窗口，否则，不能保证透明窗口的外观是正确的。使用透明窗口比使用不透明窗口需要更多消耗 CPU。如果需要考虑性能问题，尝试避免使用透明窗口或许是一个选择。

11.3.6 自动使用存储设备

窗口管理器的默认特性是向每个需要重绘的窗口发送一条 `WM_PAINT`，这会导致闪变效应，为抑制这些每个窗口的闪变效应，可为重绘操作自动使用存储设备。方法为，在创建窗口时设置标记 `WM_CF_MEMDEV`、使用函数 `WM_SetCreateFlags` 设置默认创建标记，或使用函数 `WM_EnableMemdev`。`WM` 然后将 `WM_PAINT` 消息输出重定向到存储设备中，再复制到显示器中。如果整个窗口的内存不够，会自动使用分段，存储设备只是临时使用，在绘制操作完成后会移除。

注意：11.2 节中的这些关于窗口的基础知识非常重要，请大家一定要理解！

11.4 消息

上一节讲到在窗口的回调函数中根据不同的消息相应的做出不同的处理，本节我们就讲解一下回调函数中的参数——消息（`pMsg`）。

11.4.1 消息结构

回调例程被调用时，它会收到以其 `pMsg` 参数指定的消息，此消息实际上是一个 `WM_MESSAGE` 数据结构，如下所示：

```
struct WM_MESSAGE
{
    int MsgId;           /* 消息类型 */
    WM_HWIN hWin;        /* 目标窗口*/
    WM_HWIN hWinSrc;     /* 源窗口*/

    union
    {
        const void * p;  /* 消息特定的数据指针 */
        int v;           /* 消息特定的数据数值 */
        GUI_COLOR Color; /* 颜色 */
    }
}
```

```
} Data;  
}
```

结构体 WM_MESSAGE 各个元素的含义如下表所示：

数据类型	元素	描述
int	MsgId	消息类型
WM_HWIN	hWin	目标窗口
WM_HWIN	hWinSrc	源窗口
void*	Data.p	数据指针
int	Data.v	数据值

表 11.4.1.1 WM_MESSAGE 结构体各个元素含义

11.4.2 消息类型

emWin 中窗口的消息类型分为 4 种，见下表所示：

消息 ID (MsgId)	描述
系统定义的消息	
WM_CREATE	窗口创建后立即发送
WM_DELETE	要删除窗口前发送
WM_GET_ID	发送到窗口，请求窗口的 Id
WM_INIT_DIALOG	创建对话框后立即发送到对话框窗口
WM_KEY	按下某个键后发送到当前包含焦点的窗口
WM_MOVE	窗口移动后立即发送到窗口
WM_NOTIFY_PARENT	告知父窗口，其子窗口中发生了某些改变
WM_NOTIFY_VIS_CHANGED	窗口可见性更改时发送到窗口
WM_PAINT	窗口变为无效并应重绘时，发送到窗口
WM_POST_PAINT	最后一条 WM_PAINT 消息处理后发送到窗口
WM_PRE_PAINT	第一条 WM_PAINT 消息发出前发送到窗口
WM_SET_FOCUS	获得或失去输入焦点时发送到窗口
WM_SET_ID	发送到窗口以更改窗口 ID
WM_SIZE	窗口大小更改后发送到窗口
WM_TIMER	定时器到期后发送到窗口
指针输入设备 (PID) 消息	
WM_MOUSEOVER	指针输入设备接触到窗口轮廓时发送到窗口，仅在启用鼠标支持时发送
WM_MOUSEOVER_END	指针输入设备已移出窗口轮廓时发送到窗口。仅在启用鼠标支持时发送
WM_PID_STATE_CHANGED	按下状态已更改时，发送到指针输入设备指向的窗口
WM_TOUCH	指针输入设备接触到处于按下状态的窗口轮廓时发送到窗口
WM_TOUCH_CHILD	输入设备已接触到子窗口时发送到父窗口
通知代码	
WM_NOTIFICATION_CHILD_DELETED	此通知消息将在某窗口被删除前，从该窗口发送到其父窗口
WM_NOTIFICATION_CLICKED	此通知消息将在点击窗口后发送
WM_NOTIFICATION_LOST_FOCUS	此通知消息将在窗口失去焦点时发送
WM_NOTIFICATION_MOVED_OUT	此通知消息将在指针移出窗口并点击时发送
WM_NOTIFICATION_RELEASED	此通知消息将在被点击且已被释放时发送

WM_NOTIFICATION_SCROLL_CHANGED	此通知消息将在附加的 SCROLLBAR（滚动条）小工具的滚动位置更改时发送
WM_NOTIFICATION_SCROLLBAR_ADDED	此通知消息在将对象滚动时发送
WM_NOTIFICATION_SEL_CHANGED	此通知消息将在小工具选择已更改时发送
WM_NOTIFICATION_VALUE_CHANGED	此通知消息将在小工具的特定值已更改时发送
用户定义的消息	
WM_USER	应用程序可使用 WM_USER 常数来定义私人消息，通常形式为 WM_USER+X，其中 X 为整数值

表 11.4.2.1 窗口的消息类型

表 11.4.2.1 中这些消息在使用窗口小工具的时候会被常常用到，下一小节我们讲解其中一部分消息的使用方法。

11.5 窗口重绘实验

emWin 提供的窗口管理器函数非常多，这里笔者把它们分成四类 API 函数，如下所示：

1. 窗口的创建及删除函数，如下表所示：

函数	描述
WM_Activate()	激活窗口管理器
WM_AttachWindow()	将窗口附加到一个新的父窗口
WM_AttachWindowAt()	在给定位置将窗口附加到一个新的父窗口
WM_BroadcastMessage()	发送消息到所有现有窗口
WM_BringToBottom()	将窗口放在其同属窗口后面
WM_BringToTop()	将窗口放在其同属窗口前面
WM_ClrHasTrans()	清除“有透明性”标记
WM_CreateWindow()	创建窗口
WM_CreateWindowAsChild()	创建子窗口
WM_Deactivate()	去激活窗口管理器
WM_DefaultProc()	句柄消息的默认例程
WM_DeleteWindow()	删除窗口
WM_DetachWindow()	断开窗口与其父窗口的关联
WM_DisableWindow()	将小工具状态设置为禁用

表 11.5.1 窗口创建及删除函数

2. 获取窗口属性及操作函数，如下表所示：

函数	描述
WM_EnableWindow()	将窗口状态设置为启用（默认）
WM_Exec()	通过执行回调重绘无效窗口（所有工作）
WM_Exec1()	通过执行一个回调重绘一个无效窗口（仅一项工作）
WM_ForEachDesc()	迭代窗口的所有后代
WM_GetActiveWindow()	返回活动窗口的句柄
WM_GetCallback()	返回窗口回调函数的指针
WM_GetClientRect()	返回活动窗口的尺寸
WM_GetClientRectEx()	返回窗口的尺寸
WM_GetDesktopWindow()	返回桌面窗口的窗口句柄
WM_GetDesktopWindowEx()	返回指定桌面窗口的窗口句柄
WM_GetDialogItem()	返回对话框项目（小工具）的窗口句柄
WM_GetFirstChild()	返回窗口第一个子窗口的句柄
WM_GetFocussedWindow()	返回具有输入焦点的窗口的句柄

WM_GetHasTrans()	返回“有透明性”标记的当前值
WM_GetInvalidRect()	返回给定窗口的无效矩形
WM_GetNextSibling()	返回指定页面的窗口句柄
WM_GetOrgX()	返回活动窗口 X 方向的原点
WM_GetOrgY()	返回活动窗口 Y 方向的原点
WM_GetParent()	返回窗口的父窗口的句柄。
WM_GetPrevSibling()	返回窗口前一同属窗口的句柄
WM_GetStayOnTop()	返回“保持在顶部”标记的当前值
WM_GetUserData()	检索窗口的用户数据
WM_GetWindowOrgX()	返回窗口 X 方向的原点
WM_GetWindowOrgY()	返回窗口 Y 方向的原点
WM_GetWindowRect()	返回活动窗口的屏幕坐标
WM_GetWindowRectEx()	返回窗口的屏幕坐标
WM_GetWindowSizeX()	返回窗口的水平尺寸（宽度）
WM_GetWindowSizeY()	返回窗口的垂直尺寸（高度）
WM_HasCaptured()	检查给定窗口是否已捕获鼠标和触摸屏输入
WM_HasFocus()	检查给定窗口是否具有输入焦点
WM_HideWindow()	使窗口不可见
WM_InvalidateArea()	使显示器的某些部分无效
WM_InvalidateRect()	使部分窗口无效
WM_InvalidateWindow()	使窗口无效
WM_IsCompletelyCovered()	检查窗口是否完全被覆盖
WM_IsCompletelyVisible()	检查窗口是否完全可见
WM_IsEnabled()	返回指定页面的启用状态
WM_IsVisible()	返回表示窗口是否可见的值
WM_IsWindow()	确定指定句柄是否是有效的窗口句柄
WM_MakeModal()	将窗口变成“模态”窗口

表 11.5.2 获取窗口属性及操作函数

3. 窗口移动及通知相关的函数，如下表所示：

WM_MoveChildTo()	设置窗口在窗口坐标中的位置
WM_MoveTo()	设置窗口在桌面坐标中的位置
WM_MoveWindow()	将窗口移动到其他位置
WM_NotifyParent()	WM_NOTIFY_PARENT 消息到给定窗口的父窗口
WM_Paint()	立即绘制或重绘窗口
WM_PaintWindowAndDescs()	立即绘制给定窗口和所有后代窗口
WM_ReleaseCapture()	停止捕获鼠标和触摸屏输入
WM_ResizeWindow()	改变窗口尺寸
WM_SelectWindow()	设置要用于绘制操作的活动窗口
WM_SendMessage()	向窗口发送消息
WM_SendMessageNoPara()	向窗口发送无参数的消息
WM_SendToParent()	将给定消息发送到给定窗口的父窗口
WM_SetDesktopColor()	设置桌面窗口颜色
WM_SetDesktopColorEx()	设置给定桌面的桌面窗口颜色
WM_SetCallback()	设置窗口的回调例程
WM_SetCapture()	开始捕获鼠标和触摸屏输入
WM_SetCreateFlags()	设置在创建新窗口时用作默认的标记
WM_SetFocus()	将输入焦点设置到指定窗口
WM_SetHasTrans()	设置有透明性标记

WM_SetId()	将 WM_SET_ID 消息发送到给定窗口
WM_SetpfPollPID()	设置 WM 轮询 PID 而要调用的函数
WM_SetSize()	设置窗口的尺寸
WM_SetWindowPos()	设置窗口的尺寸和位置
WM_SetXSize()	设置窗口的尺寸 X 尺寸
WM_SetYSize()	设置窗口的尺寸 Y 尺寸
WM_SetStayOnTop()	设置保持在顶部标记
WM_SetTransState()	设置或清除 WM_CF_HASTRANS 和 WM_CF_CONST_OUTLINE 标记
WM_SetUserClipRect()	临时缩小裁剪区域
WM_SetUserData()	设置给定窗口的用户数据
WM_ShowWindow()	使窗口可见
WM_Update()	绘制给定窗口的无效部分
WM_UpdateWindowAndDescs()	给定窗口的无效部分和所有后代窗口的无效部分
WM_ValidateRect()	有效化部分窗口
WM_ValidateWindow()	有效化窗口
存储设备支持（可选）	
WM_DisableMemdev()	禁用存储设备用于重绘
WM_EnableMemdev()	启用存储设备用于重绘
定时器相关	
WM_CreateTimer()	创建向窗口发送 WM_TIMER 消息的定时器

表 11.5.3 窗口移动及通知相关的函数

4. 窗口定时及获取和设置属性相关函数，如下表所示：

函数	描述
WM_DeleteTimer()	删除定时器
WM_GetTimerId()	获取给定定时器的 ID
WM_RestartTimer()	重启定时器
WM_GetClientWindow()	返回客户端窗口的句柄
WM_GetId()	返回小工具的 ID
WM_GetInsideRect()	返回活动窗口减去边界后的尺寸
WM_GetInsideRectEx()	返回窗口减去边界后的尺寸
WM_GetScrollPosH()	返回窗口水平滚动条的滚动位置
WM_GetScrollPosV()	返回窗口垂直滚动条的滚动位置
WM_GetScrollState()	获取滚动条小工具的状态
WM_SetScrollPosH()	设置窗口水平滚动条的滚动位置
WM_SetScrollPosV()	设置窗口垂直滚动条的滚动位置
WM_SetScrollState()	设置滚动条小工具的状态

表 11.5.4 窗口定时及获取和设置属性相关函数

WM 的许多函数都有窗口句柄为参数，使用句柄时请遵守以下规则：

(1) 窗口句柄可以为 0。在此情况下，函数立即返回，除非函数的文档另有规定。

(2) 如果窗口句柄为 != 0，则应是一个有效句柄。WM 不会检查给定的句柄是否有效。如果向函数提供了无效句柄，则函数会失败，甚至崩溃。

11.5.1 硬件设计

1. 例程功能

本实验主要测试 emWin 背景回调函数的作用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数创建一个窗口，如果该窗口不设置背景回调函数，则该窗口移动时会有残影；如果该窗口设置了回调函数，则该窗口移动时不会出现残影。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 11 WM 重绘背景》。

11.5.2 软件设计

11.5.2.1 程序流程图

本实验的程序流程图，如下图所示：

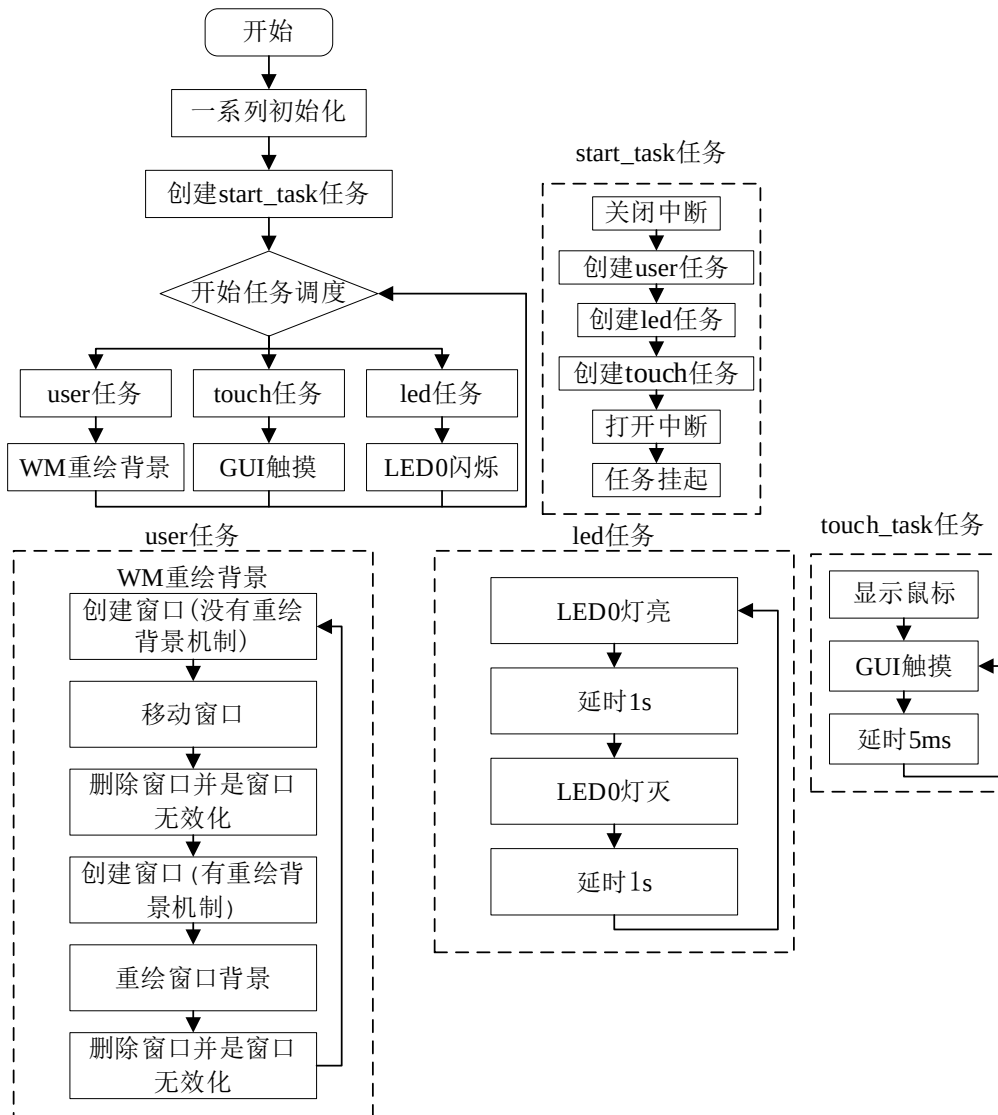


图 11.5.2.1.1 WM 重绘背景实验流程图

11.5.2.2 程序解析

以下程序演示了窗口管理的基本使用方法，实验完整工程为：**emWin 实验 11 WM 重绘背景**，打开 emwin_demo.c 文件，程序源码如下：

```
/**
 * @brief      背景窗口回调函数
 * @param      pMsg : 窗口
 * @return     无
 */
static void _cbBkWindow(WM_MESSAGE *pMsg)
{
    switch (pMsg->MsgId) /* 根据不同的消息类型处理 */
    {
        case WM_PAINT: /* WM_PAINT 消息，窗口重绘自身 */
            GUI_ClearRect(0, 50, 319, 239);
            break;
        default:
            WM_DefaultProc(pMsg); /* 此函数可处理未处理的消息 */
    }
}

/**
 * @brief      窗口回调函数
 * @param      pMsg : 窗口
 * @return     无
 */
static void _cbWindow(WM_MESSAGE *pMsg)
{
    GUI_RECT Rect;
    switch (pMsg->MsgId)
    {
        case WM_PAINT:
            WM_GetInsideRect(&Rect); /* 返回活动窗口减去边界后的尺寸 */
            GUI_SetBkColor(GUI_RED); /* 红色背景色 */
            GUI_SetColor(GUI_YELLOW); /* 黄色 */
            GUI_ClearRectEx(&Rect); /* 清除 Rect 区域, 既绘制红色区域 */
            GUI_DrawRectEx(&Rect); /* 绘制 Rect 区域, 既绘制黄色框框 */
            GUI_SetColor(GUI_BLACK); /* 黑色 */
            GUI_SetFont(&GUI_Font16_ASCII); /* 设置字体 */
            GUI_DispStringHCenterAt("Foreground Window", 75, 40);
            break;
        default:
            WM_DefaultProc(pMsg);
    }
}
```

```

}

/**
 * @brief      移动窗口
 * @param      pText : 传入文本
 * @return     无
 */
static void emwin_move_window(const char *pText)
{
    WM_HWIN hWnd;
    int i;
    /* 创建前景窗口 */
    hWnd = WM_CreateWindow(10, 50, 150, 100, WM_CF_SHOW, _cbWindow, 0);
    GUI_Delay(500);
    /* 移动前景窗口 */
    for (i = 0; i < 40; i++)
    {
        WM_MoveWindow(hWnd, 2, 2); /* 移动窗口 */
        GUI_Delay(10);
    }
    /* 删除窗口之前显示文本 */
    if (pText)
    {
        GUI_DispStringAt(pText, 5, 50);
        GUI_Delay(2500);
    }
    /* 删除前景窗口 */
    WM_DeleteWindow(hWnd); /* 删除窗口 */
    WM_Invalidate(WM_HBKWIN); /* 背景窗口无效 */
    GUI_Exec();
}

/**
 * @brief      emwin_main(emwin 入口)
 * @param      无
 * @return     无
 */
void emwin_main (void)
{
    WM_CALLBACK *_cbOldBK;
    GUI_SetBkColor(GUI_BLACK); /* 设置背景颜色 */
    GUI_Clear();
    GUI_SetColor(GUI_WHITE); /* 白色 */

```

```
GUI_SetFont(&GUI_Font24_ASCII);          /* 设置字体 */
GUI_DispStringHCenterAt("WM_Redraw-Sample", 160, 5);
GUI_SetFont(&GUI_Font16B_ASCII);          /* 设置字体 */
while (1)
{
    emwin_move_window("Background has not been redraw"); /* 移动窗口 */
    /* 清除背景 */
    GUI_ClearRect(0, 50, 319, 239);
    GUI_Delay(1000);
    /* 为背景窗口设置回调函数 */
    _cbOldBK = WM_SetCallback(WM_HBKWIN, _cbBkWindow);
    /* 移动窗口 */
    emwin_move_window("Background has been redraw");
    /* 窗口 WM_HBKWIN 的回调函数重新设置为上一个回调函数 */
    WM_SetCallback(WM_HBKWIN, _cbOldBK);
}
}
```

上述_cbOldBK 函数为背景回调函数，主要作用重绘窗口，其次 emwin_move_window()为移动窗口，最后_cbBkWindow()主要作用是窗口的重绘。

11.5.3 下载验证

本例程给我们展示了一下如何使用回调函数重绘窗口，大家一定要学会回调函数重绘窗口的方法，这个是使用窗口的基础，程序效果如图 11.5.3.1 和 11.5.3.2 所示，图 11.5.3.1 展示的是背景窗口 WM_HBKWIN 未使用回调函数重绘的效果图，11.5.3.2 展示的是背景窗口 WM_HBKWIN 使用了回调函数重绘的效果图。

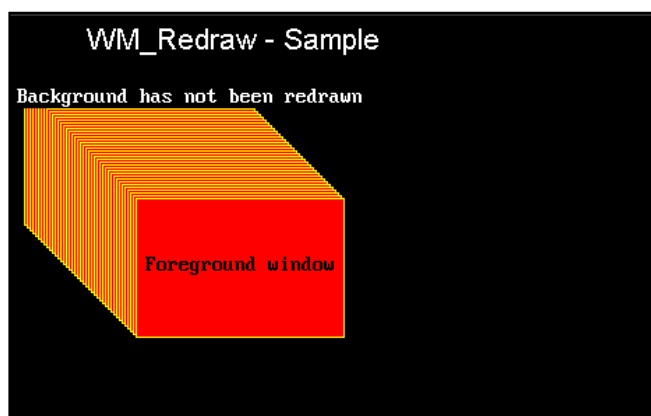


图 11.5.3.1 窗口 WM_HBKWIN 未使用回调函数重绘

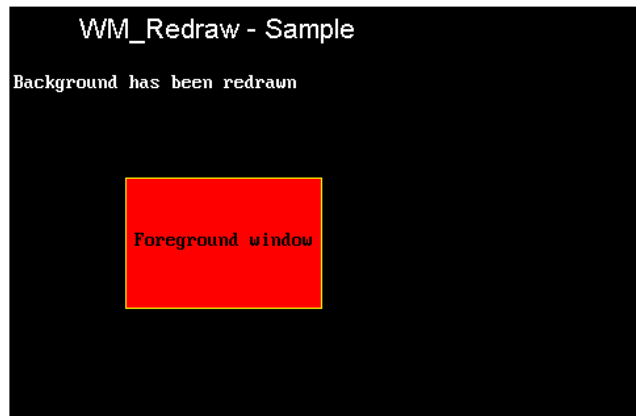


图 11.5.3.2 窗口 WM_HBKWIN 使用回调函数重绘

11.6 窗口管理 API 函数实验

11.6.1 硬件设计

1. 例程功能

本实验主要测试 emWin 窗口 API 函数使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数主要测试相关 emWin 窗口的函数使用，比如两个窗口交集区域的处理、窗口移动等操作。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 12 WM 函数使用》。

11.6.2 软件设计

11.6.2.1 程序流程图

本实验的程序流程图，如下图所示：

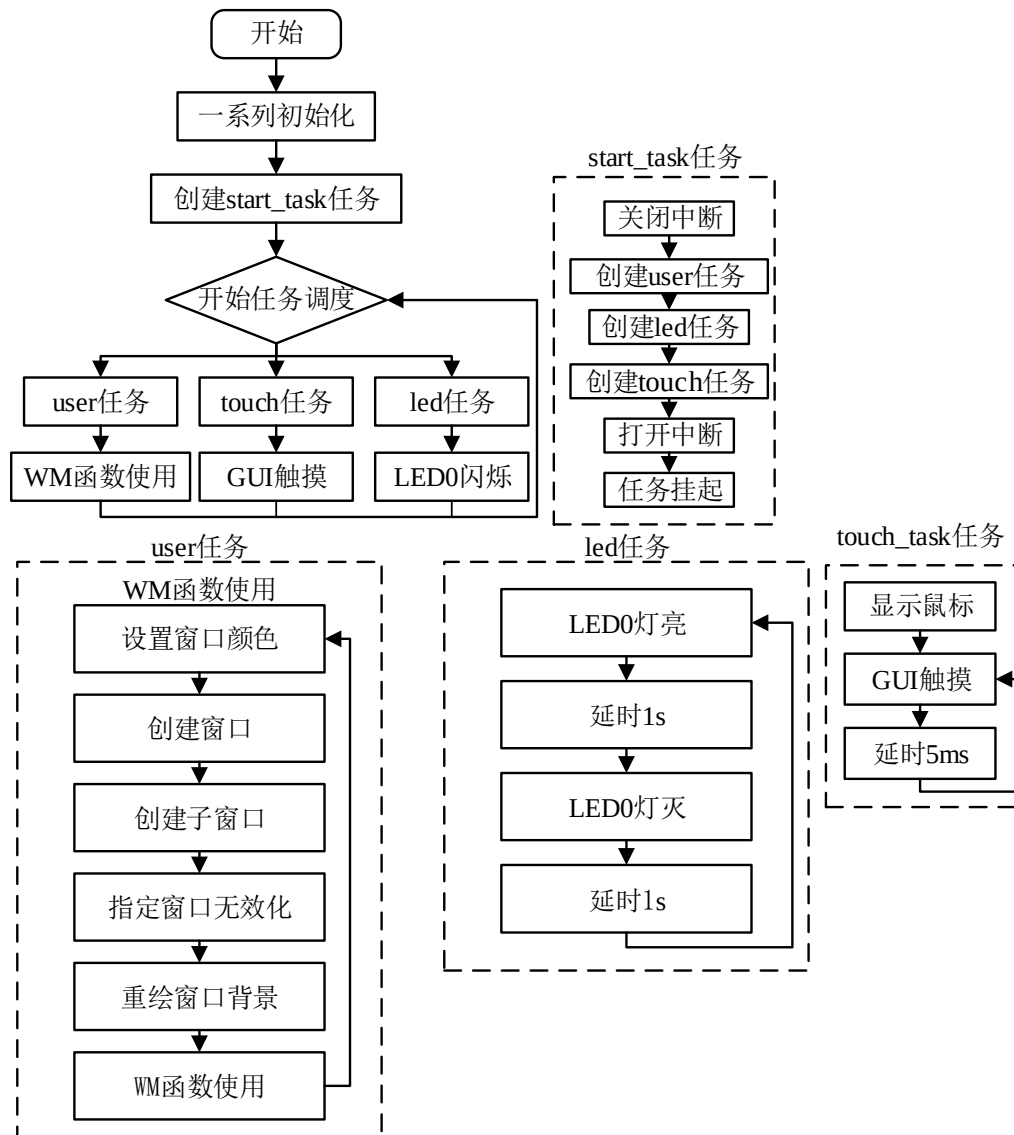


图 11.6.2.1 WM 函数使用实验流程图

11.6.2.2 程序解析

在 11.5 节中的表 11.5.1 中讲解了很多关于窗口的 API 函数，本小节的程序就演示一下部分 API 函数的使用，实验完整工程为：[emWin 实验 12 WM 函数使用](#)，打开 emwin_demo.c 文件，由于本例程代码比较多，下面我们只是截取了其中几个比较重要的给大家讲解一下，如下源码所示：

```

/**
 * @brief      向背景窗口发送一个消息并使背景窗口无效
 *              背景窗口的回调函数就会显示发送的消息
 * @param      pStr : 要发送的字符串
 * @return     无
 */
static void emwin_change_info_text(char *pStr)
{

```

```

    WM_MESSAGE Message;
    Message.MsgId = MSG_CHANGE_TEXT;
    Message.Data.p = pStr;

    /* 这里调用的是函数 WM_SendMessage() 向 WM_HBKWIN 发送消息 Message,
    此函数的第二个参数为 WM_MESSAGE 类型的, 所以需要打包一个 WM_MESSAGE 的变量。
    另一个向指定窗口发送消息的函数是 WM_SendMessageNoPara(),
    此函数的第二个参数为 int 类型, 我们在使用 WM_SendMessageNoPara()
    函数发送消息的时候可以直接发送消息 ID */
    WM_SendMessage(WM_HBKWIN, &Message); /* 向背景窗口 WM_HBKWIN 发送一条消息 */
    /* 调用函数 WM_InvalidateWindow() 使背景窗口 WM_HBKWIN 无效,
    只有使窗口无效化了, 才能在下次调用 GUI_Exec 函数或者包含
    GUI_Exec() 的其他函数的时候才会重绘窗口 */
    WM_InvalidateWindow(WM_HBKWIN);
}

/**
 * @brief      背景窗口的回调函数
 * @param      pMsg: 窗口
 * @return     无
 */
static void _cbBkWindow(WM_MESSAGE *pMsg)
{
    switch (pMsg->MsgId)
    {
        /* MSG_CHANGE_TEXT 类型的消息, 将消息中的数据 Data.p 拷贝到数组 _acInfoText 中去 */
        case MSG_CHANGE_TEXT:
            strcpy(_acInfoText, (char const *)pMsg->Data.p);

        case WM_PAINT: /* WM_PAINT 消息 */
            GUI_SetBkColor(GUI_BLACK);
            GUI_Clear();
            GUI_SetColor(GUI_WHITE);
            GUI_SetFont(&GUI_Font24_ASCII);
            GUI_DispStringHCenterAt("WindowManager - Sample", 160, 5);
            GUI_SetFont(&GUI_Font8x16);
            GUI_DispStringAt(_acInfoText, 5, 40);
            break;

        default:
            WM_DefaultProc(pMsg);
    }
}

```



```

/**
 * @brief      窗口 1 回调函数
 * @param      pMsg:窗口
 * @return     无
 */
static void _cbWindow1(WM_MESSAGE *pMsg)
{
    GUI_RECT Rect;
    int x;
    int y;

    switch (pMsg->MsgId)
    {
        case WM_PAINT:
            WM_GetInsideRect(&Rect);                /* 返回客户区的坐标 */
            GUI_SetBkColor(_WindowColor1);
            GUI_SetColor(_FrameColor1);
            GUI_ClearRectEx(&Rect);                  /* 填充一个矩形区域 */
            GUI_DrawRectEx(&Rect);                  /* 绘制一个矩形 */
            GUI_SetColor(GUI_WHITE);
            GUI_SetFont(&GUI_Font24_ASCII);
            x = WM_GetWindowSizeX(pMsg->hWin);      /*获取窗口的 x 轴大小 */
            y = WM_GetWindowSizeY(pMsg->hWin);      /*获取窗口的 y 轴大小 */
            GUI_DispStringHCenterAt("Window 1", x / 2, (y / 2) - 12);
            break;

        default:
            WM_DefaultProc(pMsg);
    }
}

/**
 * @brief      窗口 2 回调函数
 * @param      pMsg:窗口
 * @return     无
 */
static void _cbWindow2(WM_MESSAGE *pMsg)
{
    GUI_RECT Rect;
    int x;
    int y;

    switch (pMsg->MsgId)

```

```

{
    case WM_PAINT:
        WM_GetInsideRect(&Rect);                /* 返回客户区的坐标 */
        GUI_SetBkColor(_WindowColor2);
        GUI_SetColor(_FrameColor2);
        GUI_ClearRectEx(&Rect);                  /* 填充一个矩形区域 */
        GUI_DrawRectEx(&Rect);                    /* 绘制一个矩形 */
        GUI_SetColor(GUI_WHITE);
        GUI_SetFont(&GUI_Font24_ASCII);
        x = WM_GetWindowSizeX(pMsg->hWin);      /* 获取窗口的 x 轴大小 */
        y = WM_GetWindowSizeY(pMsg->hWin);      /* 获取窗口的 y 轴大小 */
        GUI_DispStringHCenterAt("Window 2", x / 2, (y / 4) - 12);
        break;

    default:
        WM_DefaultProc(pMsg);
}
}

/**
 * @brief      子窗口的回调函数
 * @param      pMsg:窗口
 * @return     无
 */
static void _cbChild(WM_MESSAGE *pMsg)
{
    GUI_RECT Rect;
    int x;
    int y;

    switch (pMsg->MsgId)
    {
        case WM_PAINT:
            WM_GetInsideRect(&Rect);
            GUI_SetBkColor(_ChildColor);
            GUI_SetColor(_ChildFrame);
            GUI_ClearRectEx(&Rect);
            GUI_DrawRectEx(&Rect);
            GUI_SetColor(GUI_RED);
            GUI_SetFont(&GUI_Font24_ASCII);
            x = WM_GetWindowSizeX(pMsg->hWin);
            y = WM_GetWindowSizeY(pMsg->hWin);
            GUI_DispStringHCenterAt("Child window", x / 2, (y / 2) - 12);
    }
}

```

```

        break;

        default:
            WM_DefaultProc(pMsg);
    }
}

/**
 * @brief      窗口 1 的另一个回调函数
 * @param      pMsg:窗口
 * @return     无
 */
static void _cbDemoCallback1(WM_MESSAGE *pMsg)
{
    int x, y;

    switch (pMsg->MsgId)
    {
        case WM_PAINT:
            GUI_SetBkColor(GUI_GREEN);
            GUI_Clear();
            GUI_SetColor(GUI_RED);
            GUI_SetFont(&GUI_FontComic18B_1);
            x = WM_GetWindowSizeX(pMsg->hWin);
            y = WM_GetWindowSizeY(pMsg->hWin);
            GUI_DispStringHCenterAt("Window 1\nanother Callback",
                                    x / 2, (y / 2) - 18);

            break;

        default:
            WM_DefaultProc(pMsg);
    }
}

/**
 * @brief      窗口 2 的另一个回调函数
 * @param      pMsg:窗口
 * @return     无
 */
static void _cbDemoCallback2(WM_MESSAGE *pMsg)
{
    int x, y;
    switch (pMsg->MsgId)

```

```
{
    case WM_PAINT:
        GUI_SetBkColor(GUI_MAGENTA);
        GUI_Clear();
        GUI_SetColor(GUI_YELLOW);
        GUI_SetFont(&GUI_FontComic18B_1);
        x = WM_GetWindowSizeX(pMsg->hWin);
        y = WM_GetWindowSizeY(pMsg->hWin);
        GUI_DispStringHCenterAt("Window 2\nanother Callback", x / 2, \
                                (y / 4) - 18);

        break;
    default:
        WM_DefaultProc(pMsg);
}
}
```

11.6.3 下载验证

这个演示例程展示了窗口 API 函数的使用，大家一定要仔细阅读，观察不同 API 函数的作用和效果，演示例程效果如下图所示。注意，此例程可以在 2.8，3.5，4.3，7 寸屏幕上显示。

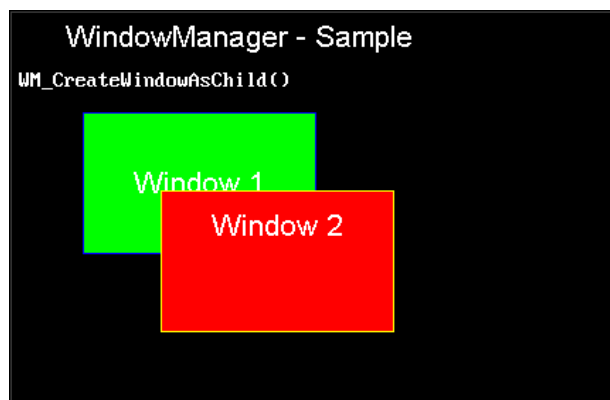


图 11.6.3.1 窗口 API 函数展示效果图

11.7 窗口裁剪实验

11.7.1 硬件设计

1. 例程功能

本实验主要测试 emWin 窗口 API 函数使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数主要测试窗口裁剪。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 13 WM 裁剪》。

11.7.2 软件设计

11.7.2.1 程序流程图

本实验的程序流程图，如下图所示：

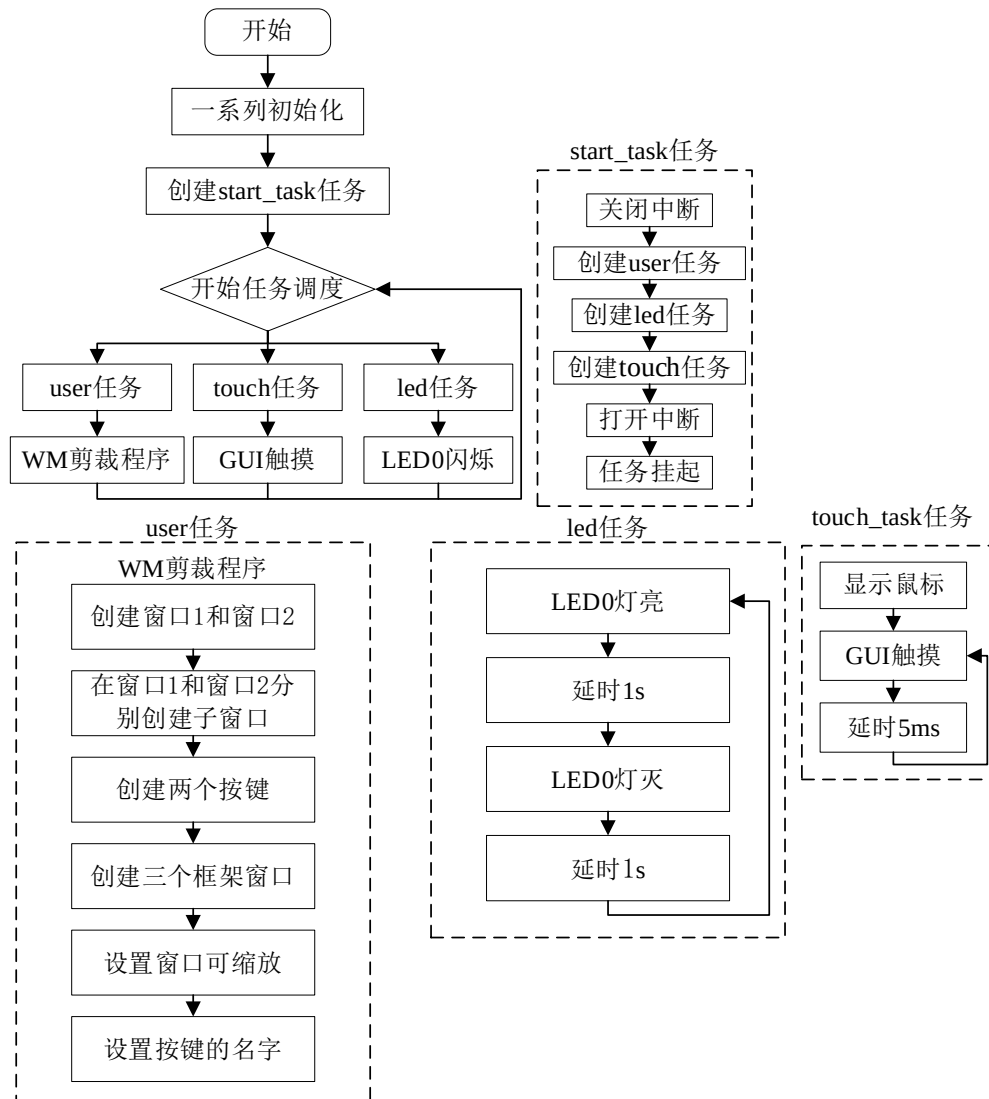


图 11.7.2.1.1 WM 裁剪实验流程图

11.7.2.2 程序解析

下面的例程也是 segger 官方的例程，为 WM_LateClipping.c，这个例程中展示了窗口的裁剪，在这个例程中使用到了 BUTTON 和 FRAMEWIN 两个控件，我们可以先不用管这两个东西，后面章节中我们会讲到这两个控件的使用方法，实验完整工程为：[emWin 实验 13 WM 裁剪](#)。

```
static WM_HWIN _hWin1;
static WM_HWIN _hWin2;
static WM_HWIN _hBut1;
```

```
static WM_HWIN _hBut2;
static int _PaintCount1;
static int _PaintCount2;

static GUI_COLOR _aColors[] =
{
    GUI_BLUE, GUI_YELLOW, GUI_RED, GUI_LIGHTCYAN, GUI_DARKRED, GUI_DARKRED
};

/**
 * @brief      背景窗口回调函数
 * @param      pMsg : WM 窗口
 * @return     无
 */
static void _cbBkWin(WM_MESSAGE *pMsg)
{
    switch (pMsg->MsgId)
    {
        case WM_PAINT:
            GUI_SetBkColor(GUI_BLACK);
            GUI_Clear();
            GUI_SetColor(0x0060FF);
            GUI_DispStringAt("PaintCount (Early):", 0, 0);
            GUI_DispDecAt(_PaintCount1, 120, 0, 5);
            GUI_SetColor(0x00FFC0);
            GUI_DispStringAt("PaintCount (Late):", 0, 12);
            GUI_DispDecAt(_PaintCount2, 120, 12, 5);
            break;

        case WM_NOTIFY_PARENT:
            if (pMsg->Data.v == WM_NOTIFICATION_RELEASED) /* 按钮被释放时 */
            {
                if (pMsg->hWinSrc == _hBut1) /* button1 被释放 */
                {
                    WM_InvalidateWindow(_hWin1); /* 窗口 1 失效 */
                    WM_InvalidateWindow(_hWin2); /* 窗口 2 失效 */
                }
                else if (pMsg->hWinSrc == _hBut2) /* button2 被释放 */
                {
                    _PaintCount1 = 0;
                    _PaintCount2 = 0;
                    WM_InvalidateWindow(pMsg->hWin); /* 主窗口失效 */
                }
            }
    }
}
```

```

    }
    break;
default:
    WM_DefaultProc(pMsg);
}
}

```

上述函数可知：当我们按下窗口按键时，使窗口 1 和窗口 2 无效导致重绘窗口。

```

/**
 * @brief      框架窗口 1 回调函数
 * @param      pMsg : WM 窗口
 * @return     无
 */
static void _cbFrameWin1(WM_MESSAGE *pMsg)
{
    switch (pMsg->MsgId)
    {
        case WM_PAINT:
            GUI_SetBkColor(_aColors[_PaintCount1 % 6]); /* 设置背景颜色 */
            GUI_Clear();
            GUI_SetColor(0x0060FF);
            GUI_FillCircle(25, 25, 15);
            GUI_SetColor(GUI_BLACK);
            GUI_DrawCircle(25, 25, 15);
            _PaintCount1++;
            WM_InvalidateWindow(WM_HBKWIN); /* 背景窗口失效 */
            break;

        default:
            WM_DefaultProc(pMsg);
    }
}

/**
 * @brief      框架窗口 2 回调函数
 * @param      pMsg : WM 窗口
 * @return     无
 */
static void _cbFrameWin2(WM_MESSAGE *pMsg)
{
    switch (pMsg->MsgId)
    {
        case WM_PAINT:
            GUI_SetBkColor(_aColors[_PaintCount2 % 6]);

```

```

        GUI_Clear();
        GUI_SetColor(0x00FFC0);
        GUI_FillCircle(25, 25, 15);
        GUI_SetColor(GUI_BLACK);
        GUI_DrawCircle(25, 25, 15);
        _PaintCount2++;
        WM_InvalidateWindow(WM_HBKWIN);
        break;

    default:
        WM_DefaultProc(pMsg);
    }
}

```

上述函数可知：该函数为窗口 1 和窗口 2 的回调函数，主要功能为重绘窗口。

```

/**
 * @brief      emwin_main(emwin 入口)
 * @param      无
 * @return     无
 */
void emwin_main (void)
{
    WM_HWIN hWin0;
    WM_HWIN hWin1;
    WM_HWIN hWin2;
    WM_HWIN hFrame1;
    WM_HWIN hFrame2;
    WM_HWIN hClient1;
    WM_HWIN hClient2;

    WM_SetCallback(WM_HBKWIN, _cbBkWin);
    /*创建框架窗口 1,可移动 */
    hFrame1 = FRAMEWIN_CreateEx(10, 30, 140, 140, 0, WM_CF_SHOW,
    FRAMEWIN_CF_MOVEABLE, 0, "Early Clipping", _cbFrameWin1);
    /*创建框架窗口 2,可移动 */
    hFrame2 = FRAMEWIN_CreateEx(170, 30, 140, 140, 0, WM_CF_SHOW,
    FRAMEWIN_CF_MOVEABLE, 0, "Late Clipping", _cbFrameWin2);
    hClient1 = WM_GetClientWindow(hFrame1);      /* 返回框架 1 的客户端窗口句柄 */
    hClient2 = WM_GetClientWindow(hFrame2);      /* 返回框架 2 的客户端窗口句柄 */
    _hWin1 = WM_CreateWindowAsChild(0, 0, WM_GetWindowSizeX(hClient1),
    WM_GetWindowSizeY(hClient1),
    hClient1, WM_CF_SHOW, _cbFrameWin1, 0);
    _hWin2 = WM_CreateWindowAsChild(0, 0, WM_GetWindowSizeX(hClient2),
    WM_GetWindowSizeY(hClient2),

```



```

        hClient2, WM_CF_SHOW | WM_CF_LATE_CLIP,
        _cbFrameWin2, 0);

_hBut1 = BUTTON_CreateEx(10, 210, 140, 20, 0, WM_CF_SHOW, 0, 1);
_hBut2 = BUTTON_CreateEx(170, 210, 140, 20, 0, WM_CF_SHOW, 0, 2);
hWin0 = FRAMEWIN_CreateEx(60, 80, 40, 40, 0, WM_CF_SHOW | WM_CF_STAYONTOP,
        FRAMEWIN_CF_MOVEABLE, 0, "Top 0", _cbTop);
hWin1 = FRAMEWIN_CreateEx(220, 80, 40, 40, 0, WM_CF_SHOW | WM_CF_STAYONTOP,
        FRAMEWIN_CF_MOVEABLE, 0, "Top 1", _cbTop);
hWin2 = FRAMEWIN_CreateEx(140, 170, 40, 40, 0, WM_CF_SHOW | WM_CF_STAYONTOP,
        FRAMEWIN_CF_MOVEABLE, 0, "Top 2", _cbTop);

FRAMEWIN_SetResizable(hWin0, 1);    /* 窗口设置为可缩放 */
FRAMEWIN_SetResizable(hWin1, 1);    /* 窗口设置为可缩放 */
FRAMEWIN_SetResizable(hWin2, 1);    /* 窗口设置为可缩放 */

BUTTON_SetText(_hBut1, "Invalidate"); /* 设置按钮的名字 */
BUTTON_SetText(_hBut2, "Reset counts");
while (1)
{
    GUI_Delay(50);
}
}

```

上述函数可知：调用函数 `FRAMEWIN_CreateEx()` 创建两个窗口框架，然后在其窗口调用 `WM_CreateWindowAsChild` 函数创建子窗口。

11.7.3 下载验证

这个例程我们重点学习回调函数的使用和消息的处理！关于 `FRAMEWIN` 和 `BUTTON` 控件的创建、使用可以先不用管，例程运行结果如下图所示：



图 11.7.3.1 例程运行效果图

第十二章 GUIBulider 的使用

本章笔者介绍一个使用 emWin 做界面的时候常使用到的一款“神器”：GUIBulider，使用这款软件就不需要我们自己用 C 语言编写界面了，可以在 GUIBulider 中设计好界面，然后导出 C 程序。GUIBulider 是 emWin 官方推出的软件，每个版本的 emWin 都有其对应版本的 GUIBulider 软件，控件非常齐全，熟练使用 GUIBulider 在使用 emWin 设计 GUI 界面的时候会起到事半功倍的效果。

本章分为以下几个部分：

- 12.1 GUIBulider 简介
- 12.2 GUIBulider 使用步骤
- 12.3 GUIBulider 输出
- 12.4 修改 C 文件
- 12.5 GUIBulider 显示实验

12.1 GUIBulider 简介

我们在 segger 官网或者 ST 官网下载 emWin 或 STemWin 软件包的时，该软件包里面已经包含了 GUIBulider 这个软件，这里以 emWin 为例，按照如下路径打开 GUIBulider 软件，小编的路径为：AppData\Local\Arm\Packs\Keil\MDK-Middleware\7.13.0\emWin\Tool\，如下图所示：

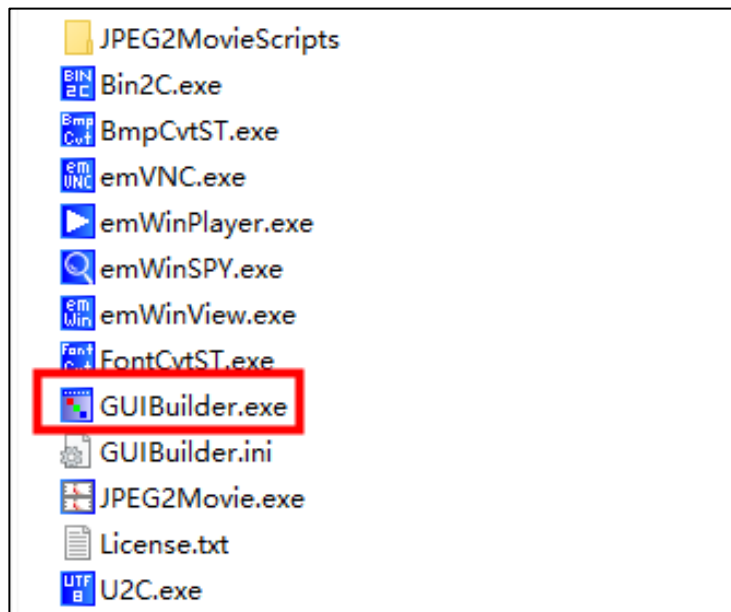


图 12.1.1 GUIBuilder 位置

打开以后如下图所示：

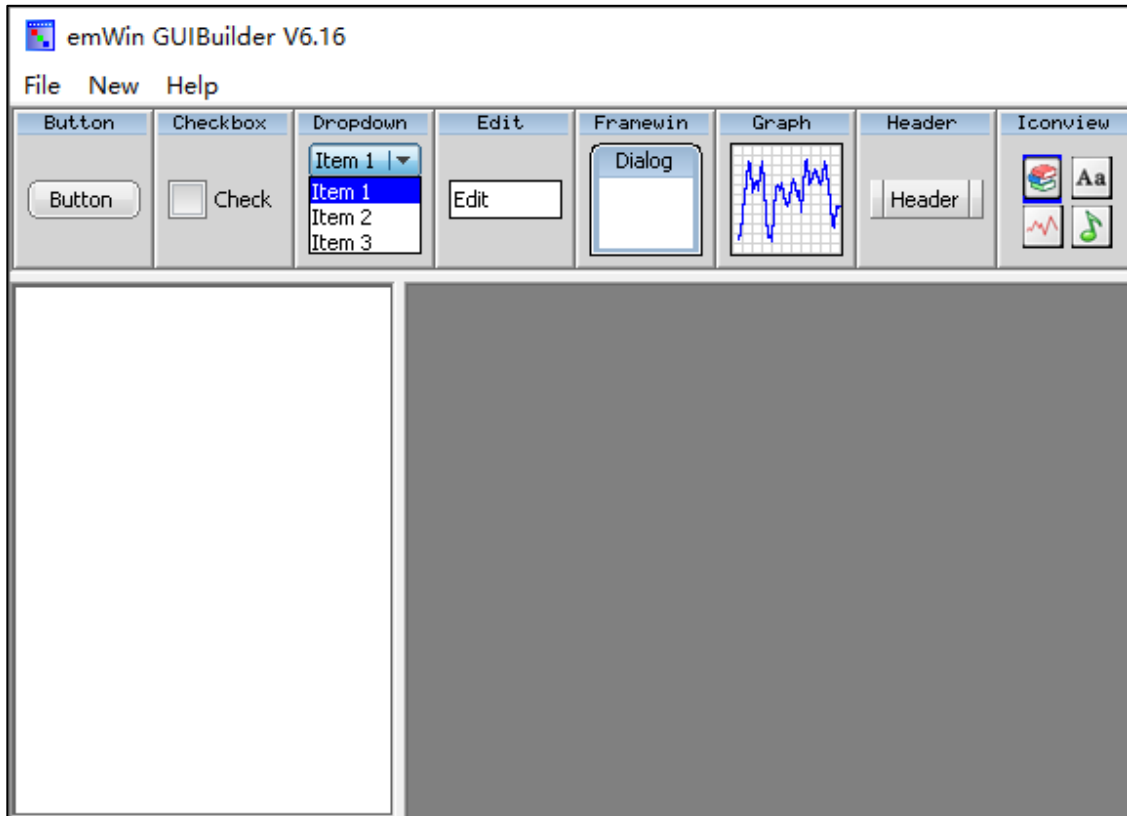


图 12.1.2 GUIBulider

上述图中，GUIBulider 可以分为四个模块框架，这些模块框架分别为小工具框架、对象数框架、编辑区框架以及属性框架如下图所示：

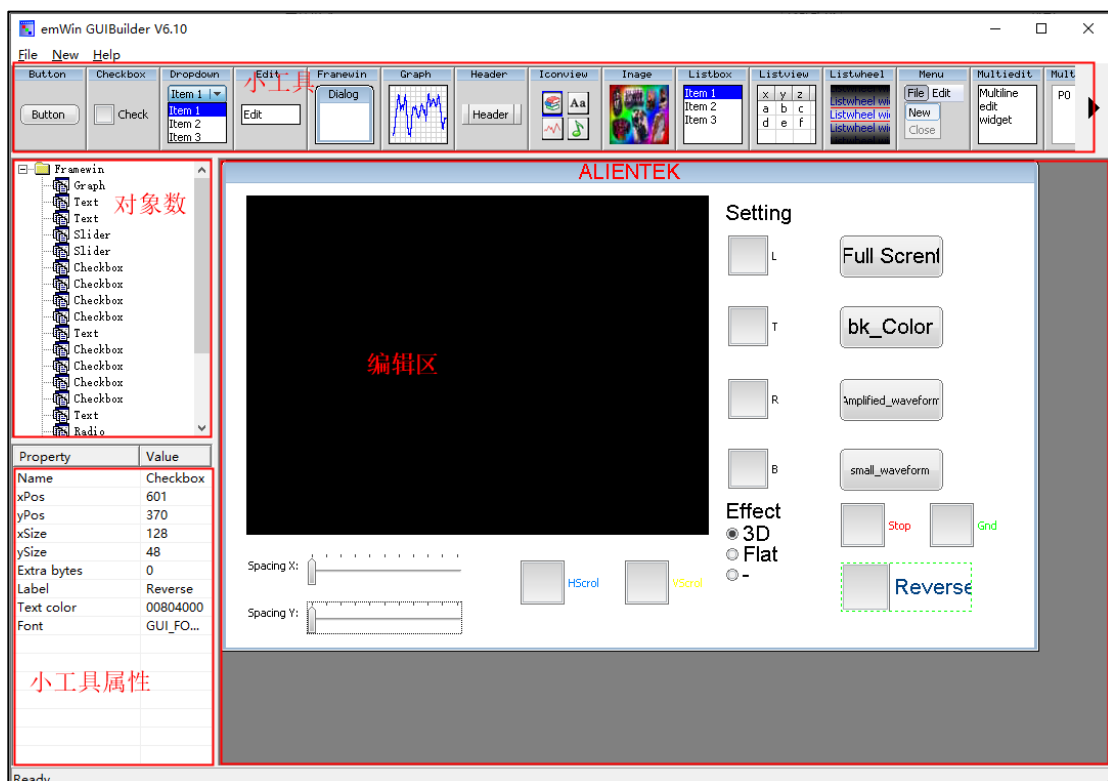


图 12.1.3 GUIBulider 用于界面各要素

下面笔者根据上图的 GUIBulider 界面模块框架的信息来讲解一下各个模块框架的作用，如下所示：

(1) 小工具选择栏

该选择栏包含 GUIBulider 所有可用的小工具，只要单击所需小工具上的选择栏或将其拖入编辑器区域内，即可进行添加。

(2) 对象树

该区域显示所有当前加载的对话框及其子小工具，只要单击相应条目，即可选择小工具。

(3) 小工具属性

它显示每个小工具的属性，并可以用于编辑属性。

(4) 编辑器

编辑器窗口显示当前所选的对话框，他可用于放置对话框及其小工具，并调整他们的大小。

12.2 GUIBulider 使用步骤

第一步：创建对话框，每个对话框都需要一个有效的父小工具，目前能够做父小工具有两个：Framewin（框架窗口小工具）和 Window（窗口小工具），更多情况下我们都是创建的 Framewin 小工具，点击如下图中的控件，创建一个 Framewin。

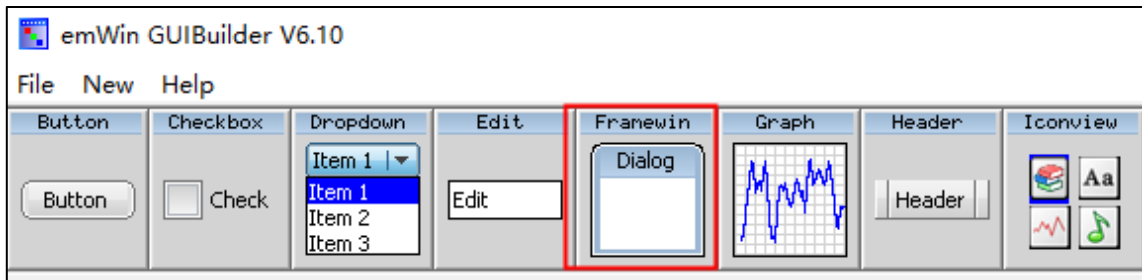


图 12.2.1 选择 Framewin

第二步：添加完成 Framewin 小工具之后的 GUIBulider 界面如下图所示：

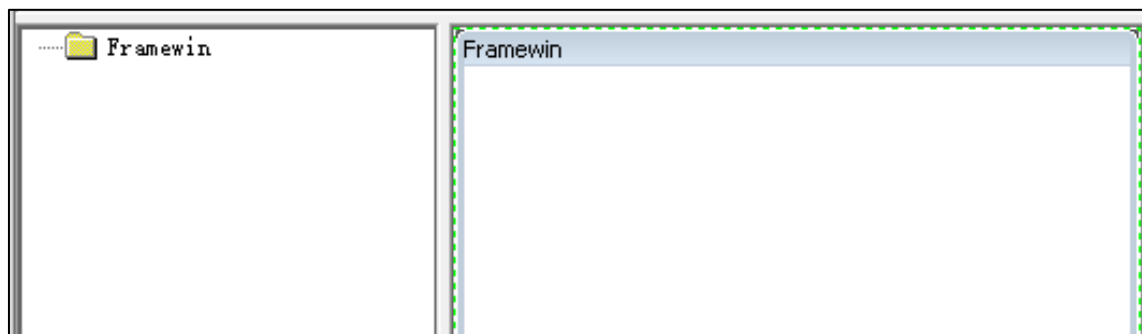


图 12.2.2 添加完成 Framewin

第三步：设置 Framewin 参数，设置参数如下：

(1) Framewin 控件坐标为 (0,0)，即空间左上角在 LCD 上的坐标为 (0,0)，如下图所示：

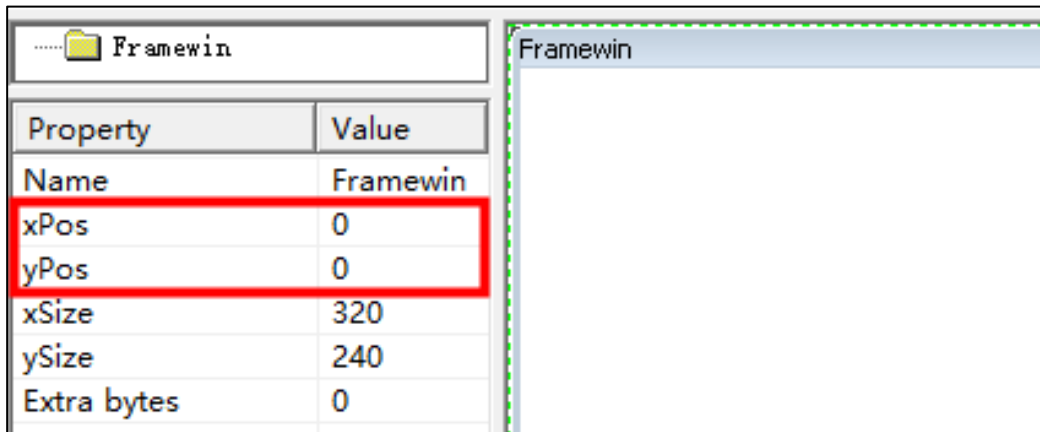


图 12.2.3 设置 Framewin 起始坐标

(2) 控件 X 轴大小为 800，Y 轴大小为 480（注意此处是按照 800*480 分辨率来设置的，大家如果使用的其他分辨率的屏请按照自己的屏幕尺寸设置控件大小，不要超过屏幕尺寸就可以。这里设置为 800*480，将来显示控件的时候就会全屏显示），如下图所示：

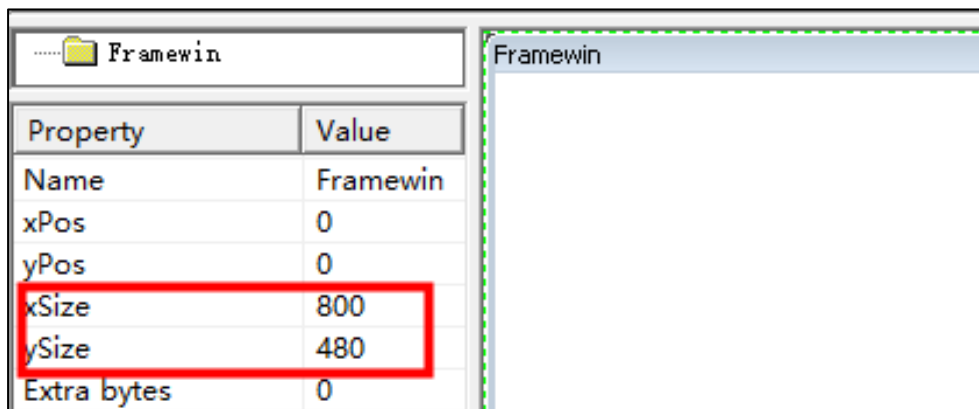


图 12.2.4 设置 Framewin 窗口大小

(3) 设置标题栏高度为 40，标题栏名字设置为“正点原子 GUIBulider”，字体设置为“GUI_FONT_32B_ASCII”，名字的对齐方式为水平和垂直居中，字体为红色。

(4) 设置标题栏的高度，在控件上点击鼠标右键，会出现很多设置选项，如下图所示，这里我们选择“Set title height”。

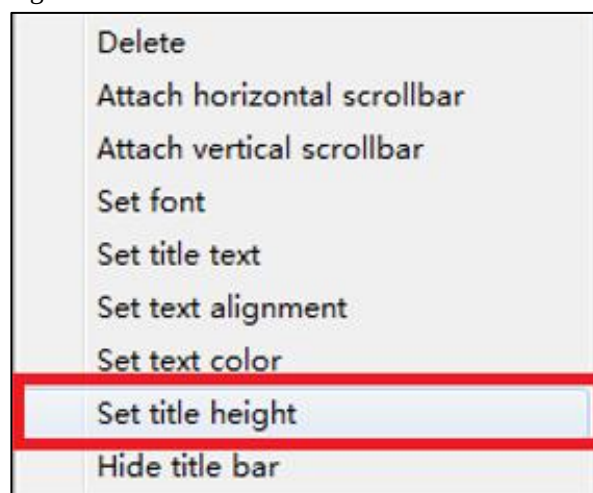


图 12.2.5 设置标题栏高度

(5) 当我们选择完成后就会在 GUIBulider 左下方的小工具属性栏出现“Title height”设置选项，默认为 14，这里我们修改为 40，如下图所示：

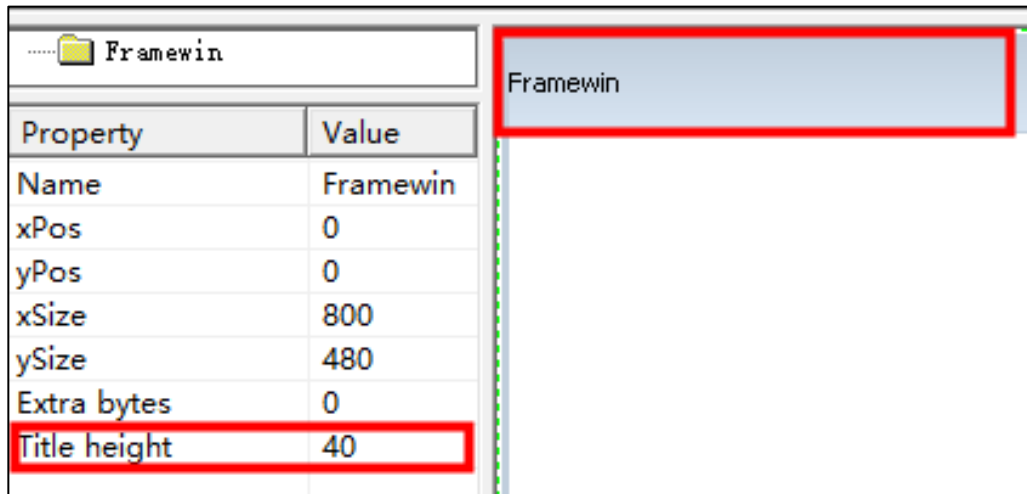


图 12.2.6 设置 Framewin 窗口标题高度

(6) 设置标题栏名字，在控件上点击右键并选择“Set title text”，设置标题栏名字为“ALIENTEK GUIBulider”，如下图所示：

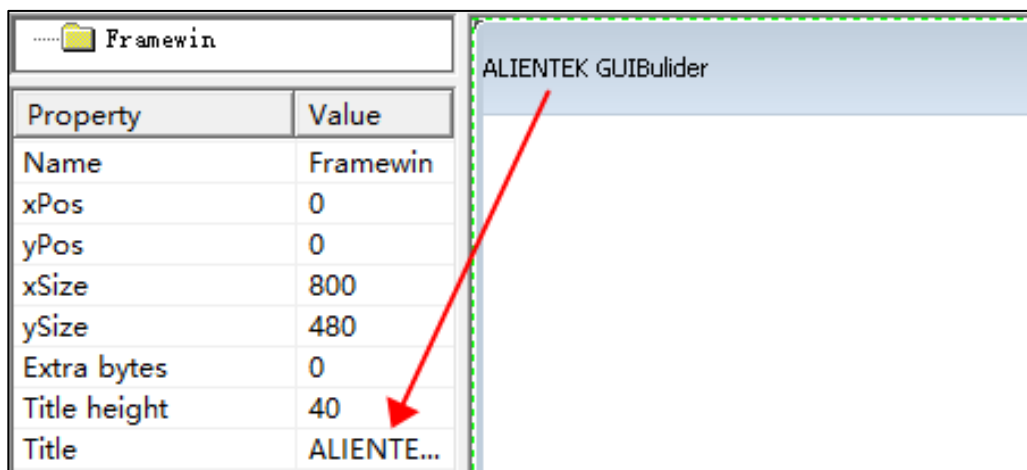


图 12.2.7 设置 Framewin 窗口标题名字

(7) 设置标题栏名字的字体系为“GUI_FONT_32B_ASCII”，控件上点击鼠标右键，选择“Set font”，出现如下图所示对话框，选择 GUI_FONT_32B_ASCII。

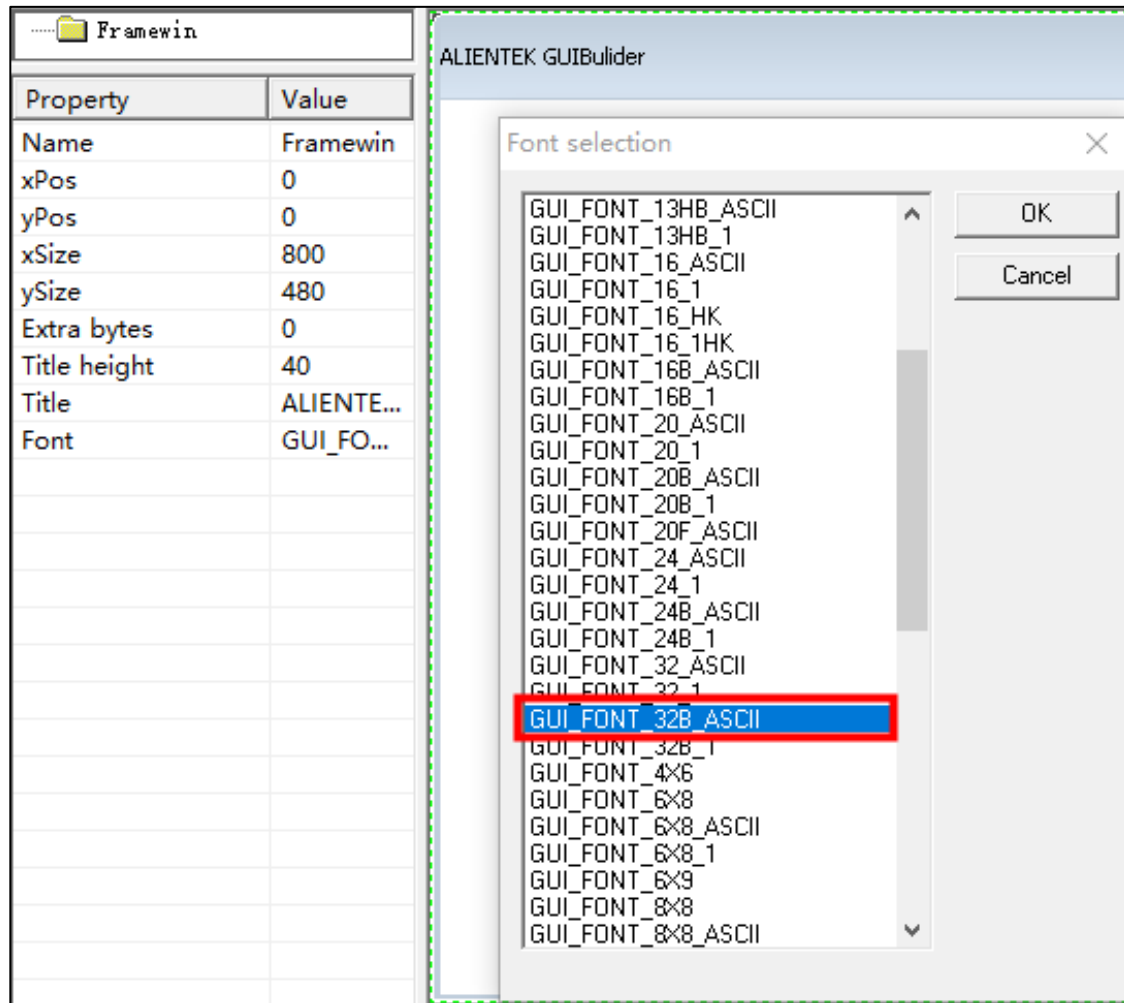


图 12.2.8 设置 Framewin 窗口标题字体

(8) 设置标题栏名字的对齐方式为水平和垂直居中，鼠标右键选择“Set text alignment”，在出现的对话框中选择自己想要的对齐方式，这里我们选择水平和居中对齐如下图所示：

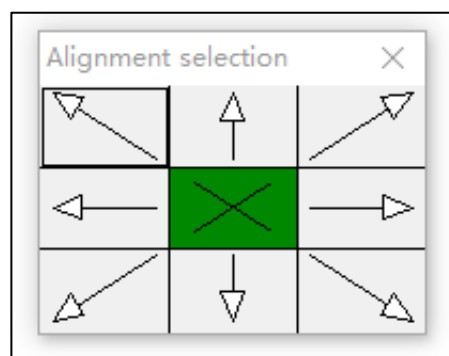


图 12.2.9 设置 Framewin 窗口标题对齐

(9) 全部都设置完成以后 Framewin 控件就如下图所示：

..... Framewin	
Property	Value
Name	Framewin
xPos	0
yPos	0
xSize	800
ySize	480
Extra bytes	0
Title height	40
Title	ALIENTE...
Font	GUI_FO...
Align	HV
Text color	000000FF

图 12.2.10 设置 Framewin 窗口标题颜色

第四步：我们可在 Framewin 窗口中添加一个 Listview 控件，一个 Edit 控件，两个 Button 控件和一个 Scrollbar 控件，添加完成后如下图所示：

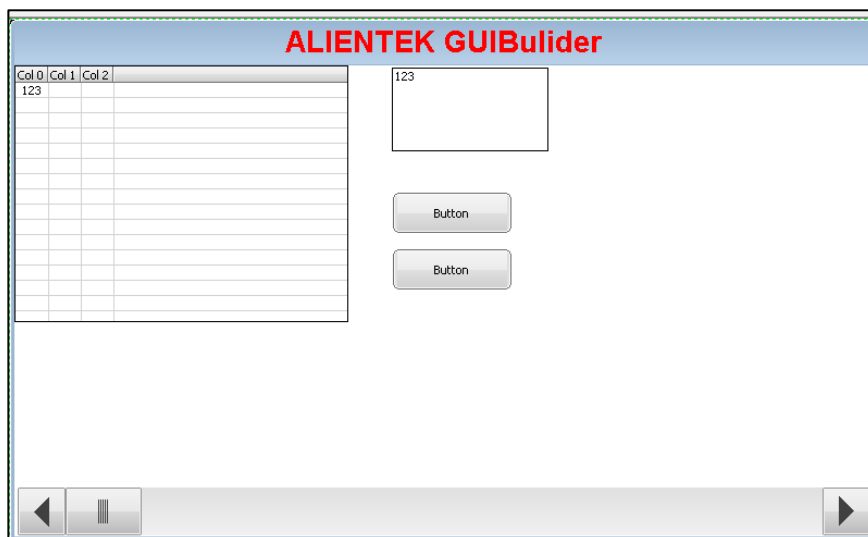


图 12.2.11 设计完成

第五步：保存设计，生成 C 文件，点击 File→Save，出现提示对话框，点击“确定”，这样在 GUIBulider 所在的文件夹下就会有一个.c 文件生成，如下图所示，FramewinDLG.c 就是我们通过 GUIBulider 设计并生成的界面代码。

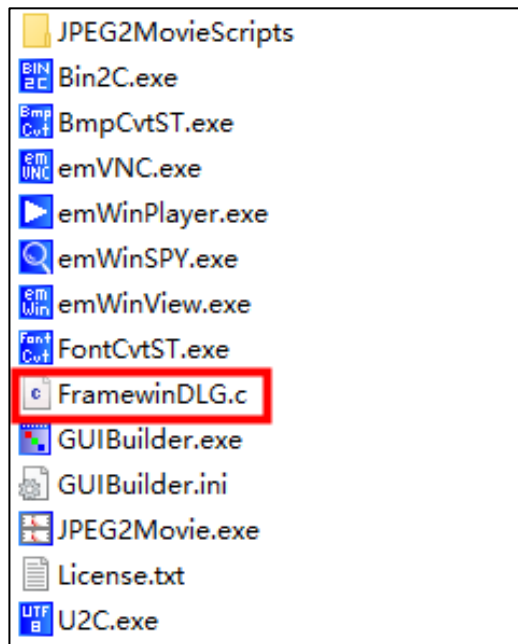


图 12.2.12 生成 C 文件

12.3 GUIBulider 输出

上一小节我们已经提到过了，GUIBulider 最终输出的就是 C 语言，以前的 GUIBulider 一旦关闭了 GUIBulider 软件就再也不能打开之前的设计，只能通过修改 C 文件修改代码，这也是 GUIBulider 的一大缺点！，但是，现在的 GUIBulider6.1 以上的版本已经解决了这个缺陷了，直接 **File**→**open** 打开文件即可打开之前的设计。我们将 **FrameWinDLG.c** 文件拷贝到工程中，拷贝到的路径为：目标工程/Middlewares→emWin→emWin_Demo 文件夹下并把该文件重命名为 **emwin_demo.c**，如下图所示，实验完整工程为：**emWin 实验 14 GUIBulider 使用**。

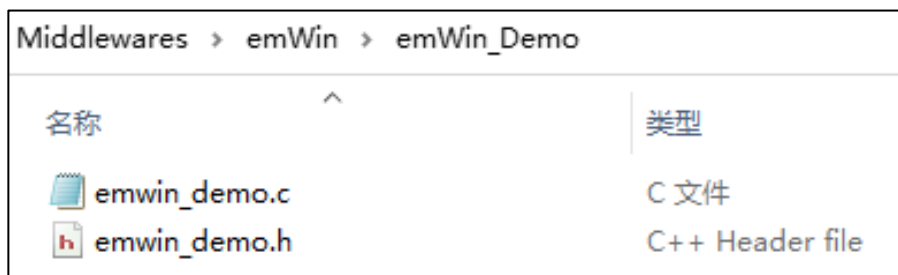


图 12.3.1 GUIBulider 生成的 C 文件拷贝到工程中

生成的 C 文件如下，限于篇幅原因，这里删除了生成的 C 文件中的注释性语言。

```
#include "DIALOG.h"

#define ID_FRAMEWIN_0      (GUI_ID_USER + 0x01)
#define ID_EDIT_0          (GUI_ID_USER + 0x02)
#define ID_LISTVIEW_0      (GUI_ID_USER + 0x03)
#define ID_BUTTON_0        (GUI_ID_USER + 0x04)
#define ID_BUTTON_1        (GUI_ID_USER + 0x05)
#define ID_SCROLLBAR_0     (GUI_ID_USER + 0x06)

static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] =
```

```

{
{ FRAMEWIN_CreateIndirect,"Framewin",ID_FRAMEWIN_0,3,-2,800, 480, 0, 0x64, 0 },
{ EDIT_CreateIndirect, "Edit",ID_EDIT_0,410, 20, 350, 250, 0,0x64, 0 },
{ LISTVIEW_CreateIndirect, "Listview", ID_LISTVIEW_0,20, 20,350,250,0,0x0, 0 },
{ BUTTON_CreateIndirect,"Button", ID_BUTTON_0, 93,290, 150, 45, 0, 0x0, 0 },
{ BUTTON_CreateIndirect, "Button", ID_BUTTON_1, 506, 290, 150, 45, 0, 0x0, 0 },
{ SCROLLBAR_CreateIndirect,"Scrollbar", ID_SCROLLBAR_0, 9,373, 772,49,0,0x0, 0 },
};

/* 对话框回调函数 */
static void _cbDialog(WM_MESSAGE * pMsg)
{
    WM_HWIN hItem;
    int NCode;
    int Id;
    switch (pMsg->MsgId)
    {
        case WM_INIT_DIALOG:
            /* 初始化 Framewin 小工具 */
            hItem = pMsg->hWin;
            FRAMEWIN_SetTitleHeight(hItem, 40);
            FRAMEWIN_SetText(hItem, "ALIENTEK BUIBulider");
            FRAMEWIN_SetFont(hItem, GUI_FONT_32B_ASCII);
            FRAMEWIN_SetTextColor(hItem, 0x000000FF);
            FRAMEWIN_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
            /* 初始化 Edit 小工具 */
            hItem = WM_GetDialogItem(pMsg->hWin, ID_EDIT_0);
            EDIT_SetText(hItem, "123");
            /* 初始化 Listview 小工具 */
            hItem = WM_GetDialogItem(pMsg->hWin, ID_LISTVIEW_0);
            LISTVIEW_AddColumn( hItem, 30, "Name", GUI_TA_HCENTER |
                                GUI_TA_VCENTER);
            LISTVIEW_AddColumn( hItem, 30, "Year", GUI_TA_HCENTER |
                                GUI_TA_VCENTER);
            LISTVIEW_AddColumn( hItem, 30, "Num", GUI_TA_HCENTER |
                                GUI_TA_VCENTER);
            LISTVIEW_AddRow(hItem, NULL);
            LISTVIEW_SetGridVis(hItem, 1);
            LISTVIEW_SetFont(hItem, GUI_FONT_20_ASCII);
            LISTVIEW_SetItemText(hItem, 0, 0, "ZhangSan");
            LISTVIEW_SetAutoScrollH(hItem, 1);
            LISTVIEW_SetAutoScrollV(hItem, 1);
            LISTVIEW_SetHeaderHeight(hItem, 30);
    }
}

```

```

LISTVIEW_SetItemText(hItem, 1, 0, "26");
LISTVIEW_SetItemText(hItem, 2, 0, "20150328001");
LISTVIEW_SetRowHeight(hItem, 25);
/* 初始化 BUTTON0 小工具 */
hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_0);
BUTTON_SetText(hItem, "OK");
BUTTON_SetFont(hItem, GUI_FONT_COMIC24B_1);
/* 初始化 BUTTON1 小工具 */
hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_1);
BUTTON_SetText(hItem, "CANCEL");
BUTTON_SetFont(hItem, GUI_FONT_COMIC24B_1);
break;
case WM_NOTIFY_PARENT:
    Id = WM_GetId(pMsg->hWinSrc);
    NCode = pMsg->Data.v;
    switch(Id)
    {
        case ID_EDIT_0: /* Edit 发送的通知消息 */
            switch(NCode)
            {
                case WM_NOTIFICATION_CLICKED:
                    break;
                case WM_NOTIFICATION_RELEASED:
                    break;
                case WM_NOTIFICATION_VALUE_CHANGED:
                    break;
            }
            break;
        case ID_LISTVIEW_0: /* Listview 发送的通知消息 */
            switch(NCode)
            {
                case WM_NOTIFICATION_CLICKED:
                    break;
                case WM_NOTIFICATION_RELEASED:
                    break;
                case WM_NOTIFICATION_SEL_CHANGED:
                    break;
            }
            break;
        case ID_BUTTON_0: /* Button0 发送的通知消息 */
            switch(NCode)
            {
                case WM_NOTIFICATION_CLICKED:

```

```

        break;
        case WM_NOTIFICATION_RELEASED:
            break;
    }
    break;
case ID_BUTTON_1: /* Button1 发送的通知消息 */
    switch(NCode)
    {
        case WM_NOTIFICATION_CLICKED:
            break;
        case WM_NOTIFICATION_RELEASED:
            break;
    }
    break;
case ID_SCROLLBAR_0: /* Scrollbar 发送的通知消息 */
    switch(NCode)
    {
        case WM_NOTIFICATION_CLICKED:
            break;
        case WM_NOTIFICATION_RELEASED:
            break;
        case WM_NOTIFICATION_VALUE_CHANGED:
            break;
    }
    break;
    }
    break;
default:
    WM_DefaultProc(pMsg);
    break;
    }
}

WM_HWIN CreateFramewin(void);

WM_HWIN CreateFramewin(void)
{
    WM_HWIN hWin;
    hWin = GUI_CreateDialogBox(_aDialogCreate, GUI_COUNTOF(_aDialogCreate),
        _cbDialog, WM_HBKWIN, 0, 0);
}

```

从以上代码可以看出 GUIBulider 生成的只是一个初始化的代码，仅仅只是一个程序框架，至于这些控件的具体功能需要我们自己编写程序的！这些控件的具体作用不是本节的内容，会在以后章节讲解的。

12.4 修改 C 文件

生成的 C 文件中包含了很多自定义代码区域，如下所示：

```
// USER START (选择性插入自己的代码)
// USER END
```

我们可以在这些自定义代码区域加上自己的代码，比如初始化控件，换肤，定义控件的功能等，为了给大家提前展示一下 emWin 强大的换肤功能，开启 emWin 的换肤功能很简单，只要在 main.c 中的 emWin 任务函数中加入了一下程序就可以了。

```
/* 更换皮肤 */
BUTTON_SetDefaultSkin(BUTTON_SKIN_FLEX);
CHECKBOX_SetDefaultSkin(CHECKBOX_SKIN_FLEX);
DROPDOWN_SetDefaultSkin(DROPDOWN_SKIN_FLEX);
FRAMEWIN_SetDefaultSkin(FRAMEWIN_SKIN_FLEX);
HEADER_SetDefaultSkin(HEADER_SKIN_FLEX);
MENU_SetDefaultSkin(MENU_SKIN_FLEX);
MULTIPAGE_SetDefaultSkin(MULTIPAGE_SKIN_FLEX);
PROGBAR_SetDefaultSkin(PROGBAR_SKIN_FLEX);
RADIO_SetDefaultSkin(RADIO_SKIN_FLEX);
SCROLLBAR_SetDefaultSkin(SCROLLBAR_SKIN_FLEX);
SLIDER_SetDefaultSkin(SLIDER_SKIN_FLEX);
SPINBOX_SetDefaultSkin(SPINBOX_SKIN_FLEX);
```

新建一个 emwin_demo.h 文件，在这个 h 文件里面声明一下函数 CreateFramewin，这样我们就可以在其他文件调用了。

修改 main.c 中 user 的任务函数 user_task 如下所示，编译完成下载到开发板中，显示结果如图 12.4.1 所示：

```
/**
 * @brief      执行 emwin 例程
 * @param      p_arg : 传入参数 (未用到)
 * @retval     无
 */
void user_task(void *p_arg)
{
    OS_ERR err;
    p_arg = p_arg;
    CreateFramewin();

    while (1)
    {
        OSTimeDlyHMSM(0,0,0,5,OS_OPT_TIME_PERIODIC,&err); /* 延时 5ms */
    }
}
```

}

图 12.4.1 是使用换肤以后的效果图，注意这是在 800*480 分辨率上显示的，如果在 320*240 分辨率的 LCD 上显示会显示不完全！实验完整工程为：[emWin 实验 14 GUIBulider 使用](#)。

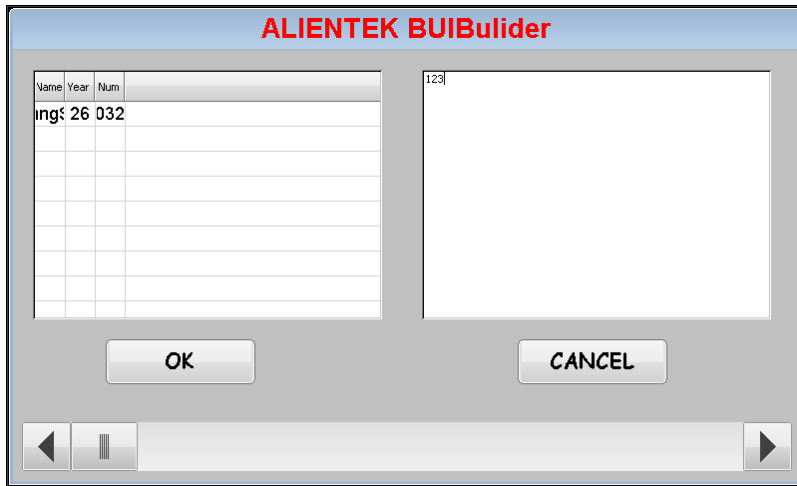


图 12.4.1 换肤后的 LCD 显示效果图

12.5 GUIBulider 显示实验

12.5.1 硬件设计

1. 例程功能

本实验主要测试 GUIBulider 软件的使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数主要测试 GUIBulider 软件生成的.c 文件是否可在 MCU 下运行。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 14 GUIbulider 使用》。

12.5.2 软件设计

12.5.2.1 程序流程图

本实验的程序流程图，如下图所示：

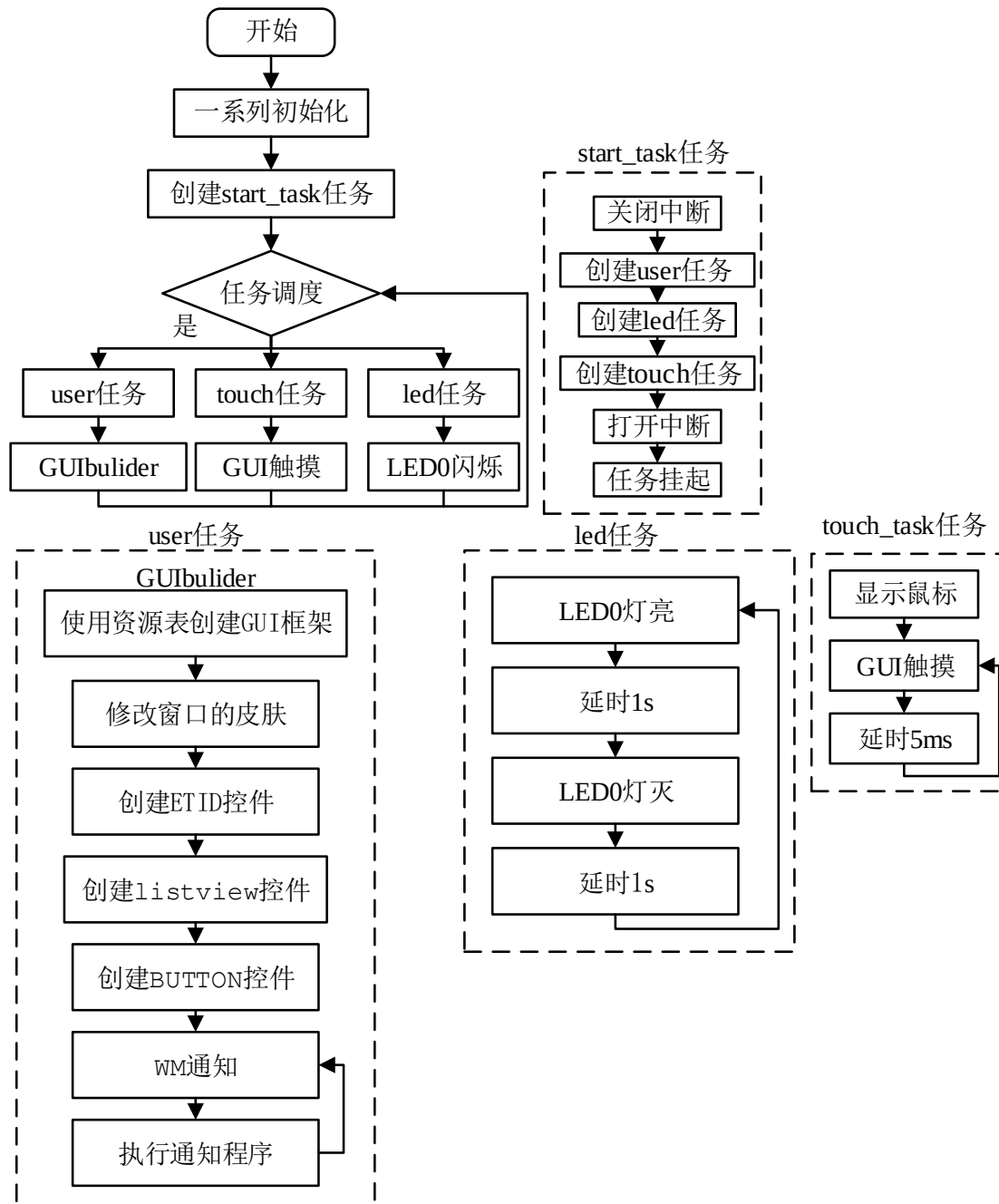


图 12.5.2.1.1 GUIBulider 的使用实验流程图

12.5.2.2 程序解析

使用 GUIBulider.exe 会自动生成资源表和一个框架，我们只需要编程控件实现什么功能即可，如下源码所示：

```

/* GUI 资源表 */
static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] =
{
    { FRAMEWIN_CreateIndirect, "Framewin", ID_FRAMEWIN_0, 0, 0, 800, 480, 0, 0x64,0},
    { EDIT_CreateIndirect, "Edit", ID_EDIT_0, 410, 20, 350, 250, 0, 0x64, 0},
    { LISTVIEW_CreateIndirect, "Listview", ID_LISTVIEW_0, 20, 20, 350, 250, 0, 0x0,0},

```



```
{ BUTTON_CreateIndirect, "Button", ID_BUTTON_0, 93, 290, 150, 45, 0, 0x0, 0},
{ BUTTON_CreateIndirect, "Button", ID_BUTTON_1, 506, 290, 150, 45, 0, 0x0, 0},
{ SCROLLBAR_CreateIndirect, "Scrollbar", ID_SCROLLBAR_0, 9, 373, 772, 49, 0, 0x0, 0},
};
```

上述的源码 GUIBulider.exe 自动生成的资源表，可以在里面添加控件等。

emWin 程序入口，如下源码所示：

```
/**
 * @brief      emwin_main(emwin 入口)
 * @param      无
 * @return     无
 */
void emwin_main (void)
{
    GUI_CreateDialogBox( _aDialogCreate, GUI_COUNTOF(_aDialogCreate),
                        _cbDialog, WM_HBKWIN, 0, 0);
}
```

上述的源码中，使用资源表生成相关的控件，_cbDialog 为回调函数，该回调函数如以下源所示：

```
/**
 * @brief      回调函数
 * @param      无
 * @return     无
 */
static void _cbDialog(WM_MESSAGE *pMsg)
{
    WM_HWIN hItem;
    int      NCode;
    int      Id;

    switch (pMsg->MsgId)
    {
        case WM_INIT_DIALOG: /* Framewin 初始化 */
            hItem = pMsg->hWin;
            FRAMEWIN_SetTitleHeight(hItem, 40);
            FRAMEWIN_SetText(hItem, "ALIENTEK BUIBulider");
            FRAMEWIN_SetFont(hItem, GUI_FONT_32B_ASCII);
            FRAMEWIN_SetTextColor(hItem, 0x000000FF);
            FRAMEWIN_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);

            /* Edit 控件初始化 */
            hItem = WM_GetDialogItem(pMsg->hWin, ID_EDIT_0);
            EDIT_SetText(hItem, "123");
    }
```

```

/* Listview 控件初始化 */
hItem = WM_GetDialogItem(pMsg->hWin, ID_LISTVIEW_0);
LISTVIEW_AddColumn(hItem, 30, "Name", GUI_TA_HCENTER |
                    GUI_TA_VCENTER);
LISTVIEW_AddColumn(hItem, 30, "Year", GUI_TA_HCENTER |
                    GUI_TA_VCENTER);
LISTVIEW_AddColumn(hItem, 30, "Num", GUI_TA_HCENTER |
                    GUI_TA_VCENTER);
LISTVIEW_AddRow(hItem, NULL);
LISTVIEW_SetGridVis(hItem, 1);
LISTVIEW_SetFont(hItem, GUI_FONT_20_ASCII);
LISTVIEW_SetItemText(hItem, 0, 0, "ZhangSan");
LISTVIEW_SetAutoScrollH(hItem, 1);
LISTVIEW_SetAutoScrollV(hItem, 1);
LISTVIEW_SetHeaderHeight(hItem, 30);
LISTVIEW_SetItemText(hItem, 1, 0, "26");
LISTVIEW_SetItemText(hItem, 2, 0, "20150328001");
LISTVIEW_SetRowHeight(hItem, 25);

/* Button 初始化 */
hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_0);
BUTTON_SetText(hItem, "OK");
BUTTON_SetFont(hItem, GUI_FONT_COMIC24B_1);

/* Button 初始化 */
hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_1);
BUTTON_SetText(hItem, "CANCEL");
BUTTON_SetFont(hItem, GUI_FONT_COMIC24B_1);

break;

case WM_NOTIFY_PARENT:
    Id = WM_GetId(pMsg->hWinSrc);
    NCode = pMsg->Data.v;

    switch (Id)
    {
        case ID_EDIT_0: /* Edit 发送通知 */
            switch (NCode)
            {
                case WM_NOTIFICATION_CLICKED: /* 通知被点击 */
                    /* 执行的功能..... */
                    break;
            }
        break;
    }

```

```
        case WM_NOTIFICATION_RELEASED: /* 通知被释放 */
            /* 执行的功能..... */
            break;

        case WM_NOTIFICATION_VALUE_CHANGED: /* 值被改变 */
            /* 执行的功能..... */
            break;
    }

    break;

case ID_LISTVIEW_0:
    switch (NCode)
    {
        case WM_NOTIFICATION_CLICKED:
            break;

        case WM_NOTIFICATION_RELEASED:
            break;

        case WM_NOTIFICATION_SEL_CHANGED:
            break;
    }
    break;
case ID_BUTTON_0:
    switch (NCode)
    {
        case WM_NOTIFICATION_CLICKED:
            break;

        case WM_NOTIFICATION_RELEASED:
            break;
    }
    break;
case ID_BUTTON_1:
    switch (NCode)
    {
        case WM_NOTIFICATION_CLICKED:
            break;

        case WM_NOTIFICATION_RELEASED:
            break;
    }
    break;
```

```

        case ID_SCROLLBAR_0:
            switch (NCode)
            {
                case WM_NOTIFICATION_CLICKED:
                    break;
                case WM_NOTIFICATION_RELEASED:
                    break;
                case WM_NOTIFICATION_VALUE_CHANGED:
                    break;
            }
            break;
        }
        break;
    default:
        WM_DefaultProc(pMsg);
        break;
    }
}

```

上述源码的 WM_INIT_DIALOG 是 emWin 的初始化对话框，简单来说：用户定义的控件都在这里初始化，而 WM_NOTIFY_PARENT 是 emWin 通知父类的消息类型。

12.5.3 下载验证

程序运行结果如下图所示：

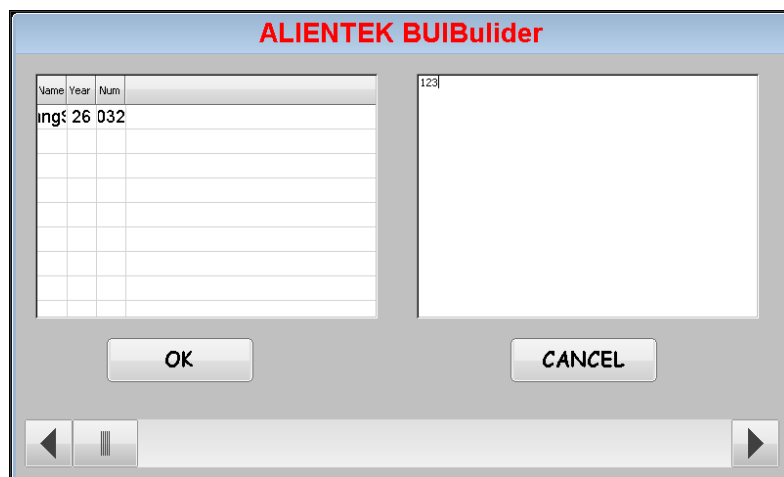


图 12.5.3.1 换肤后的 LCD 显示效果图

图 12.5.3.1 是换肤后的显示效果图，类似于 WIN7 的效果，我们可注释掉换肤部分的程序，看一下如果不换肤的话会显示成什么样子的，关闭换肤功能后的显示效果如图 12.5.3.2 所示：

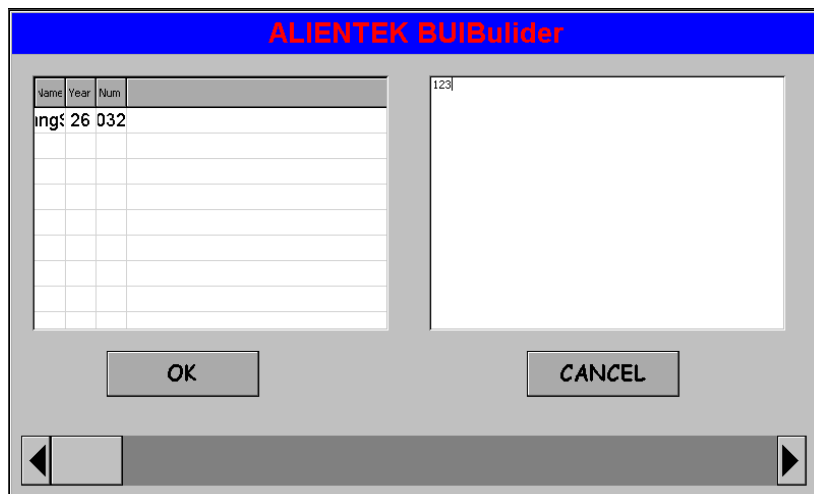


图 12.5.3.2 未换肤的显示效果

第十三章 对话框

虽然说小工具可以创建并独立使用，是因为它们本质上就是窗口，但是，通常项目开发时需要使用对话框，在它的基础上使用小工具，简单来讲：对话框是一个包含一个或多个小工具的窗口。对话框通常是一个窗口，它在出现时会要求用户输入信息，它可能包含多个小工具并要求用户根据各种选择来提供信息或者以消息框的形式提供信息（比如向用户提供注意事项或警告）和一个“确定”按钮。其实在上一章讲解 GUIBuilder 的时候就使用到了对话框，使用 GUIBuilder 的制作 GUI 界面的时候基本都是用了对话框，本章我们就详细的讲解一下什么是对话框。

本章分为以下几个部分：

13.1 对话框基本原理

13.2 创建对话框

13.3 定义小工具功能

13.4 对话框 API 函数

13.1 对话框基本原理

(1) 输入焦点

窗口管理器能记住用户使用触摸屏、鼠标、键盘或用其他方式最终所选择的窗口或窗口对象。该窗口会收到键盘输入消息时，即具有输入焦点。追踪输入焦点的主要原因是为了确定键盘命令的发送目的地。具有输入焦点的窗口会接收由键盘所生成的事件。如果要将对对话框内的输入焦点移至下一个焦点对话框项目，可以使用 GUI_KEY_TAB 键。如果要向后移动，则可以使用 GUI_KEY_BACKTAB。

(2) 阻塞式和非阻塞式对话框

对话框窗口可以分为阻塞式和非阻塞式，阻塞式对话框会阻塞执行的线程，默认情况下，它有输入焦点，用户必须先关闭它，线程才能继续执行。阻塞式对话框不会同时禁用所显示的其他对话框，换言之，阻塞式对话框并非模式对话框。如果对话框为阻塞式，则表示只有在对话框关闭后，所使用的函数 GUI_ExecDialogBox 或 GUI_ExecCreatedDialog 才会返回值。而非阻塞式对话框则不会阻塞调用的线程——在它为可见时，可允许任务继续运行。创建对话框后，函数会立即返回值。

需要注意的是，切勿从回调函数中调用阻塞式函数，否则可能会导致应用程序故障！

(3) 对话框过程函数

对话框就是一个窗口，它接收消息的方式与系统中其他所有窗口一样。大多数消息由对话框的窗口回调程序自动处理，而其他消息则传递到建立对话框时所指定的回调程序，这便称为对话框过程函数。

(4) 对话框消息

发送到对话框过程函数的两种附加消息为：WM_INIT_DIALOG/WM_NOTIFY_PARENT。在显示对话框前，WM_INIT_DIALOG 消息会立即发送到对话框过程函数，对话框过程函数通常使用该消息来初始化小工具，并执行其他任何会影响对话框外观的初始化任务。而另一个消息 WM_NOTIFY_PARENT 则通过对话框的子窗口发送到对话框，通知任何事件的父窗口以确保同步化。通过子窗口发送的事件取决于其类型，并针对每个类型的小工具单独记录。

13.2 创建对话框

创建对话框需要两个基本要素：资源表和对话框过程；前者定义所要包括的小工具，后者定义小工具的初始值及其行为。一旦具备这两个要素，则只需进行单个函数调用 `GUI_CreateDialogBox` 或 `GUI_ExecDialogBox` 就能创建对话框。

13.2.1 对话框资源表

对话框可以基于阻塞（`GUI_ExecDialogBox` 函数）或非阻塞（`GUI_CreateDialogBox` 函数）方式创建。必须首先定义一个资源表，以指定在对话框中所要包括的所有小工具，一般我们使用 `GUIBuilder` 设计界面的时候选择了对话框的话就会在最终生成的 C 文件中自动生成资源表，下面的示例定义了一个资源表：

```
static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] =
{
    { FRAMEWIN_CreateIndirect, "Dialog", 0, 30, 5, 260, 230, FRAMEWIN_CF_MOVEABLE, 0 },
    { BUTTON_CreateIndirect, "OK", GUI_ID_OK, 100, 5, 60, 20 },
    { BUTTON_CreateIndirect, "Cancel", GUI_ID_CANCEL, 100, 30, 60, 20 },
    { TEXT_CreateIndirect, "LText", 0, 10, 55, 48, 15, TEXT_CF_LEFT },
    { TEXT_CreateIndirect, "RText", 0, 10, 80, 48, 15, TEXT_CF_RIGHT },
    { EDIT_CreateIndirect, NULL, GUI_ID_EDIT0, 60, 55, 100, 15, 0, 50 },
    { EDIT_CreateIndirect, NULL, GUI_ID_EDIT1, 60, 80, 100, 15, 0, 50 },
    { TEXT_CreateIndirect, "Hex", 0, 10, 100, 48, 15, TEXT_CF_RIGHT },
    { EDIT_CreateIndirect, NULL, GUI_ID_EDIT2, 60, 100, 100, 15, 0, 6 },
    { TEXT_CreateIndirect, "Bin", 0, 10, 120, 48, 15, TEXT_CF_RIGHT },
    { EDIT_CreateIndirect, NULL, GUI_ID_EDIT3, 60, 120, 100, 15 },
    { LISTBOX_CreateIndirect, NULL, GUI_ID_LISTBOX0, 10, 10, 60, 40 },
    { CHECKBOX_CreateIndirect, NULL, GUI_ID_CHECK0, 10, 140, 0, 0 },
    { CHECKBOX_CreateIndirect, NULL, GUI_ID_CHECK1, 30, 140, 0, 0 },
    { SLIDER_CreateIndirect, NULL, GUI_ID_SLIDER0, 60, 140, 180, 20 },
    { SLIDER_CreateIndirect, NULL, GUI_ID_SLIDER1, 10, 170, 230, 30 },
    { DROPDOWN_CreateIndirect, NULL, GUI_ID_DROPDOWN0, 170, 10, 80, 60, 0, 3 },
    { DROPDOWN_CreateIndirect, NULL, GUI_ID_DROPDOWN1, 170, 60, 80, 60, 0, 3 }
};
```

对话框中所要包括的任何小工具都必须使用 `<WIDGET>_CreateIndirect` 函数来间接创建。

13.2.2 对话框回调函数

前面说过了对话框也是窗口的，创建对话框也是需要回调函数，通过回调函数来初始化对话框中其他小工具和定义这些小工具的行为，回调函数如下所示：

```
/* 对话框回调函数 */
static void _cbDialog(WM_MESSAGE * pMsg)
{
    switch (pMsg->MsgId)
    {
        case WM_INIT_DIALOG:
```

```
/* 其他消息... */

defauyt:
    WM_DefaultProc(pMsg);
}
}
```

对于该示例，对话框显示时会有下列代码行：

```
GUI_ExecDialogBox(_aDialogCreate, GUI_COUNTOF(_aDialogCreate),
    _cbCallback, 0, 0, 0);
```

在使用函数 GUI_CreateDialogBox 或 GUI_ExecDialogBox 创建对话框的时候需要将回调函数作为参数。使用上面资源表创建完成以后的对话框如下图所示：

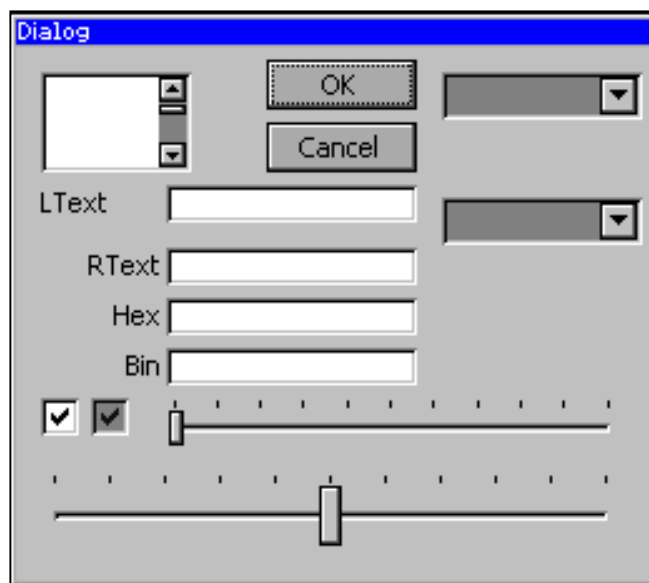


图 13.2.2.1 创建好的对话框

创建对话框后，所有资源表中的小工具都将可见。尽管这些小工具在上面的屏幕截图中可见，但它们是以“空”的形式出现的。这是因为对话框过程函数尚未包含初始化单个元素的代码。小工具的初始值、由它们所引起的行为以及它们之间的交互作用都需要在对话框过程中进行定义。

13.2.3 初始化对话框

图 13.2.2.1 中只展示的只是创建好的对话框，但是对话框还没有设置初始化值，接下来我们就要使用各个小工具的初始值对其进行初始化，接收到 WM_INIT_DIALOG 消息时做小工具的初始化，下面的程序展示了这个过程：

```
static void _cbCallback(WM_MESSAGE * pMsg)
{
    WM_HWIN hEdit0;
    WM_HWIN hEdit1;
    WM_HWIN hEdit2;
    WM_HWIN hEdit3;
    WM_HWIN hListBox;
```



```

WM_HWIN hDropd0;
WM_HWIN hDropd1;
WM_HWIN hWin= pMsg->hWin;
switch (pMsg->MsgId)
{
    case WM_INIT_DIALOG: /* WM_INIT_DIALOG 消息, 初始化所有小控件 */
        hEdit0 = WM_GetDialogItem(hWin, GUI_ID_EDIT0);
        hEdit1 = WM_GetDialogItem(hWin, GUI_ID_EDIT1);
        hEdit2 = WM_GetDialogItem(hWin, GUI_ID_EDIT2);
        hEdit3 = WM_GetDialogItem(hWin, GUI_ID_EDIT3);
        hListBox = WM_GetDialogItem(hWin, GUI_ID_LISTBOX0);
        hDropd0 = WM_GetDialogItem(hWin, GUI_ID_DROPDOWN0);
        hDropd1 = WM_GetDialogItem(hWin, GUI_ID_DROPDOWN1);
        /* 初始化所有小控件 */
        /* use EDIT_B */
        EDIT_SetText(hEdit0, "EDIT widget 0");
        EDIT_SetText(hEdit1, "EDIT widget 1");
        EDIT_SetTextAlign(hEdit1, GUI_TA_LEFT);
        EDIT_SetHexMode(hEdit2, 0x1234, 0, 0xffffffff);
        EDIT_SetBinMode(hEdit3, 0x1234, 0, 0xffff);
        LISTBOX_SetText(hListBox, _apListBox);
        WM_DisableWindow (WM_GetDialogItem(hWin, GUI_ID_CHECK1));
        CHECKBOX_Check( WM_GetDialogItem(hWin, GUI_ID_CHECK0));
        CHECKBOX_Check( WM_GetDialogItem(hWin, GUI_ID_CHECK1));
        SLIDER_SetWidth( WM_GetDialogItem(hWin, GUI_ID_SLIDER0), 5);
        SLIDER_SetValue( WM_GetDialogItem(hWin, GUI_ID_SLIDER1), 50);
        SCROLLBAR_CreateAttached(hListBox, SCROLLBAR_CF_VERTICAL);
        DROPDOWN_AddString(hDropd0, "Item 0");
        DROPDOWN_AddString(hDropd0, "Item 1");
        DROPDOWN_AddString(hDropd0, "Item 2");
        DROPDOWN_AddString(hDropd1, "Item 0");
        DROPDOWN_AddString(hDropd1, "Item 1");
        DROPDOWN_AddString(hDropd1, "Item 2");
        DROPDOWN_AddString(hDropd1, "Item 3");
        DROPDOWN_AddString(hDropd1, "Item 4");
        /* use EDIT_E */
        break;
    default:
        WM_DefaultProc(pMsg);
}
}
    
```

上述程序中使用 EDIT_B 和使用 EDIT_E 包围的部分就是对话框回调函数收到消息 WM_INIT_DIALOG 后初始化对话框中其他的小工具，至于具体调用的初始化函数我们这里不用管，在以后讲解各个控件的时候会详细的讲解这些 API 函数，初始化完以后的对话框如下图所示：

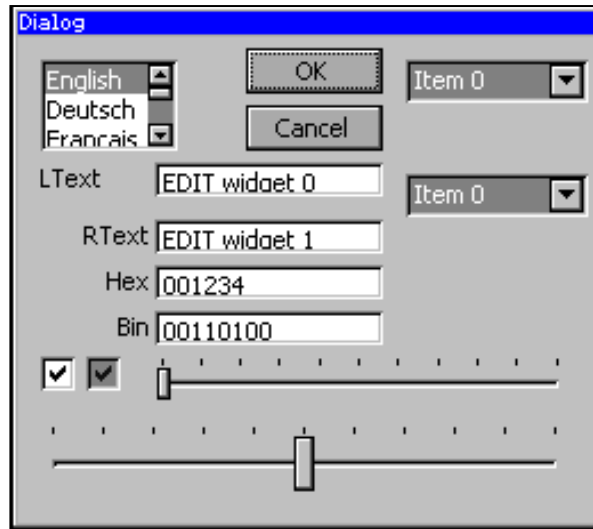


图 13.2.3.1 初始化对话框小工具

13.3 定义小工具功能

一旦对话框得到初始化，则剩下的所有工作便是向对话框过程函数添加代码，这样可添加小工具的代码，从而使其能充分操作。继续同一个示例，最终的对话框过程函数如下所示：

```
static void _cbCallback(WM_MESSAGE * pMsg)
{
    WM_HWIN hEdit0;
    WM_HWIN hEdit1;
    WM_HWIN hEdit2;
    WM_HWIN hEdit3;
    WM_HWIN hListBox;
    WM_HWIN hDropd0;
    WM_HWIN hDropd1;
    WM_HWIN hWin;
    hWin = pMsg->hWin;      /* 窗体的控制块 */

    switch (pMsg->MsgId)    /* 窗口消息的 ID */
    {
        case WM_INIT_DIALOG: /* WM_INIT_DIALOG 消息，用于初始化小工具 */
            /* 根据小工具 ID 获取小工具控制块 */
            hEdit0 = WM_GetDialogItem(hWin, GUI_ID_EDIT0);
            hEdit1 = WM_GetDialogItem(hWin, GUI_ID_EDIT1);
            hEdit2 = WM_GetDialogItem(hWin, GUI_ID_EDIT2);
            hEdit3 = WM_GetDialogItem(hWin, GUI_ID_EDIT3);
            hListBox = WM_GetDialogItem(hWin, GUI_ID_LISTBOX0);
            hDropd0 = WM_GetDialogItem(hWin, GUI_ID_DROPDOWN0);
```

```

hDropd1 = WM_GetDialogItem(hWin, GUI_ID_DROPDOWN1);

/* 初始化小工具 */
EDIT_SetText(hEdit0, "EDIT widget 0");
EDIT_SetText(hEdit1, "EDIT widget 1");
EDIT_SetTextAlign(hEdit1, GUI_TA_LEFT);
EDIT_SetHexMode(hEdit2, 0x1234, 0, 0xffffffff);
EDIT_SetBinMode(hEdit3, 0x1234, 0, 0xffff);
LISTBOX_SetText(hListBox, _apListBox);
WM_DisableWindow(WM_GetDialogItem(hWin, GUI_ID_CHECK1));
CHECKBOX_Check(WM_GetDialogItem(hWin, GUI_ID_CHECK0));
CHECKBOX_Check(WM_GetDialogItem(hWin, GUI_ID_CHECK1));
SLIDER_SetWidth(WM_GetDialogItem(hWin, GUI_ID_SLIDER0), 5);
SLIDER_SetValue(WM_GetDialogItem(hWin, GUI_ID_SLIDER1), 50);
SCROLLBAR_CreateAttached(hListBox, SCROLLBAR_CF_VERTICAL);
DROPDOWN_AddString(hDropd0, "Item 0");
DROPDOWN_AddString(hDropd0, "Item 1");
DROPDOWN_AddString(hDropd0, "Item 2");
DROPDOWN_AddString(hDropd1, "Item 0");
DROPDOWN_AddString(hDropd1, "Item 1");
DROPDOWN_AddString(hDropd1, "Item 2");
DROPDOWN_AddString(hDropd1, "Item 3");
DROPDOWN_AddString(hDropd1, "Item 4");
break;

case WM_KEY:
    switch (((WM_KEY_INFO*) (pMsg->Data.p))->Key)
    {
        case GUI_KEY_ESCAPE:
            GUI_EndDialog(hWin, 1);
            break;

        case GUI_KEY_ENTER:
            GUI_EndDialog(hWin, 0);
            break;
    }
    break;

case WM_NOTIFY_PARENT:
    Id = WM_GetId(pMsg->hWinSrc);           /* 获取小控件 ID 号 */
    NCode = pMsg->Data.v;                   /* 小工具消息类型 */
    switch (NCode)
    {
        case WM_NOTIFICATION_RELEASED:      /* 按键被释放 */
            if (Id == GUI_ID_OK)           /* OK 按键 */
            {

```

```

        GUI_EndDialog(hWin, 0);
    }

    if (Id == GUI_ID_CANCEL)      /* CANCEL 按钮 */
    {
        GUI_EndDialog(hWin, 1);
    }

    break;

    default:
        WM_DefaultProc(pMsg);
    }
}

```

13.4 对话框 API 函数

表 13.4.1 列出了有关对话框的 API 函数：

函数	描述
GUI_CreateDialogBox()	创建非阻塞式对话框
GUI_ExecCreatedDialog()	执行已创建的对话框
GUI_ExecDialogBox()	创建并执行对话框
GUI_EndDialog()	结束对话框

表 13.4.1 对话框 API 函数

(1) GUI_CreateDialogBox 函数

创建对话框，该函数原型如下源码所示：

```

WM_HWIN GUI_CreateDialogBox(const GUI_WIDGET_CREATE_INFO * paWidget,
                             int NumWidgets, WM_CALLBACK * cb,
                             WM_HWIN hParent, int x0, int y0);

```

该函数的相关形参如下表所示：

参数	描述
paWidget	定义对话框中所要包含的控件的资源表的指针
NumWidgets	对话框中所包含的控件的总数
cb	应用程序特定回调函数（对话框过程函数）的指针
hParent	父窗口的句柄（0 表示没有父窗口）
x0	对话框相对于父窗口的 X 轴位置
y0	对话框相对于父窗口的 Y 轴位置

表 13.4.2 GUI_CreateDialogBox 函数相关形参描述

返回值：创建的对话框的句柄。这个句柄可用于访问资源表中的第一个控件。这必须是一个 FRAMEWIN 或 WINDOW 控件。

注意：参数 cb 用作客户窗口的回调函数。如果要为对话框窗口设置回调函数，函数 WM_SetCallback 应使用由 GUI_CreateDialogBox 返回的句柄。客户窗口的句柄可用传递函数 GUI_CreateDialogBox 返回的句柄的函数 WM_GetClientWindow 来确定。

(2) GUI_ExecCreatedDialog 函数

执行已创建的对话框，该函数原型如下源码所示：

```

int GUI_ExecCreatedDialog(WM_HWIN hDialog);

```

该函数的相关形参如下表所示：

参数	描述
hDialog	对话框的句柄

表 13.4.3 GUI_ExecCreatedDialog 函数相关形参描述

返回值：返回传递给 GUI_EndDialog 函数的值。

注意：这是一个阻塞式函数。这个函数直到使用函数 GUI_EndDialog 关闭对话框才返回。WM_CF_SHOW 标志被设置，所以要到窗口管理器下一次执行才绘制对话框。

(3) GUI_ExecDialogBox 函数

创建并执行对话框，该函数原型如下源码所示：

```
int GUI_ExecDialogBox( const GUI_WIDGET_CREATE_INFO * paWidget,
                      int NumWidgets,
                      WM_CALLBACK * cb,
                      WM_HWIN hParent,
                      int x0,
                      int y0);
```

该函数的相关形参如下表所示：

参数	描述
paWidget	定义对话框中所要包含的控件的资源表的指针
NumWidgets	对话框中所包含的控件的总数
cb	应用程序特定回调函数（对话框过程函数）的指针
hParent	父窗口的句柄（0 表示没有父窗口）
x0	对话框相对于父窗口的 X 轴位置
y0	对话框相对于父窗口的 Y 轴位置

表 13.4.4 GUI_ExecDialogBox 函数相关形参描述

返回值：返回传递给 GUI_EndDialog 函数的值。

注意：该函数实际调用函数 GUI_CreateDialogBox 和 GUI_ExecCreatedDialog。这是阻塞式函数，所以要直到使用函数 GUI_EndDialog 关闭对话框才返回。WM_CF_SHOW 标志被设置，所以要到窗口管理器下一次执行才绘制对话框。

(4) GUI_EndDialog 函数

结束（关闭）对话框。对话框及其子窗口将从内存删除，该函数原型如下源码所示：

```
void GUI_EndDialog(WM_HWIN hDialog, int r);
```

该函数的相关形参如下表所示：

参数	描述
hDialog	对话框的句柄
r	如果非阻塞式对话框已经关闭，则忽略该值

表 13.4.5 GUI_EndDialog 函数相关形参描述

注意：在下一次调用函数 WM_Exec（也在 GUI_Delay 中调用）之后，句柄 hDialog 就不再有效。如果非阻塞式对话框已经关闭，则该函数与函数 WM_DeleteWindow 的工作方式相同。

关于对话框我们就讲解到这里，本章的内容非常重要，在我们制作好的 UI 界面的时候几乎都要用到对话框的，尤其是用 GUIBulider 的时候，所以请大家务必掌握对话框的使用方法。

第十四章 窗口小工具(控件)

小工具是具有对象类型属性的各种窗口，在窗口界被称为控件，它是组成用户界面的重要元素。它们可自动对某些事件作出反应，例如，按下某按钮后，它可以不同状态显示。小工具需要创建，具有可在存续期间随时更改的属性，并通常在不再需要时被删除。正如窗口一样，小工具通过其创建函数返回的句柄进行引用，小工具要求使用窗口管理器，小工具创建后，可像其他任何窗口一样处理它。WM 确保了在必要时随时正确显示（并重绘）它，小工具不是编写应用程序或用户界面所必需的，但它们可大大简化编程。

本章分为以下几个部分：

14.1 小工具基础知识

14.2 通用控件 API 函数

14.1 小工具基础知识

14.1.1 可用小工具

下表列出了目前 6.16 版本 emWin 支持的控件如下表所示：

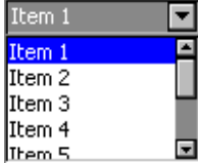
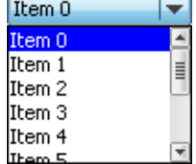
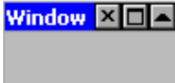
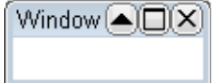
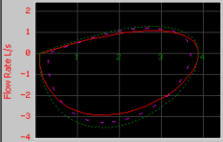
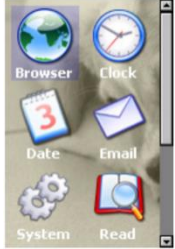
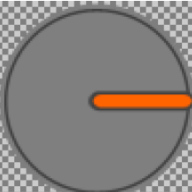
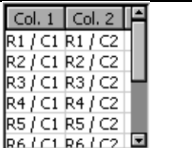
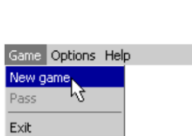
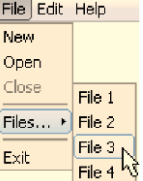
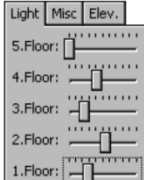
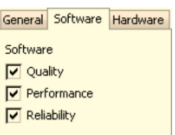
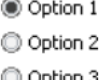
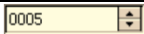
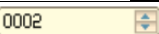
控件名字	屏幕截图 (经典)	屏幕截图 (皮肤设置)	描述
BUTTON			可按下的按钮，按钮上可显示文本或者位图
CHECKBOX			可被选中或取消选中的复选框
DROPDROWN			下来列表，按下后会打开一个列表框
EDIT			单行编辑字段，提示用户键入数字或文本
FRAMEWIN			框架窗口，创建典型的 GUI 外观
GRAPH			图形小工具，用于显示曲线或测量值
HEADER			标题控件，用于管理各栏
ICONVIEW			图标视图小工具，对基于图标的平台非常有用

IMAGE			图像控件，自动显示各种格式的图片
KNOB			旋钮控件，可用于调整不可计数的数值
LISTBOX			突出显示用户选择项目的列表框
LISTVIEW			Listview 小工具用于创建表
LISTWHEEL			Listwheel 小工具，可使用指针输入设备移动或加速数据
MENU			菜单小工具可用于创建水平或垂直菜单
MULTIEDIT			Multedit 小工具可用于编辑多行文本
MULTIPAGE			Multipage 小工具用于创建具有多个页面的对话框
PROGBAR			用于可视化的进度条
RADIO			可以选择的单选按钮，一次只能选择一个按钮
SCROLLBAR			水平或者垂直的滚动条
SLIDER			用于更改值的滚动条
SPINBOX			用于改变一个数值
TEXT			静态文本控件，一般用于对话框

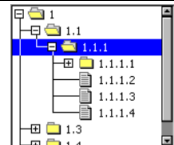
SWIPELIST			用于创建可通过在触屏上滑动手指（或任何其他 PID）来移动的可滑动列表的滑动列表控件
TREEVIEW			用于管理分层列表的树形图形小工具

表 14.1.1.1 emWin 可用控件

14.1.2 了解重绘机制

小工具根据其属性绘制自身，此操作在调用 WM_Exec、GUI_Exec 或 GUI_Delay 时执行。在多任务环境中，通常由后台任务来调用 WM_Exec 并更新小工具（所有其他具有回调函数的窗口）。小工具的属性更改后，该小工具的窗口（部分窗口）会被标记为无效，但不会立即重绘，因此，代码段的执行速度会非常快。重绘由 WM 在稍后执行，或者通过为该小工具调用 WM_Paint 强制执行（直到重绘所有窗口时调用 WM_Exec）。

14.1.3 如何使用小工具

假设我们要显示一个进度条，所需的只是以下代码：

```
PROGBAR_Handle hProgBar; /* 定义一个进度条句柄 */
GUI_DispStringAt("Progress bar", 100, 20);
/* 调用 PROGBAR_Create 函数创建一个进度条 */
hProgBar = PROGBAR_Create(100, 40, 100, 20, WM_CF_SHOW);
```

创建完成以后窗口管理器会在调用函数 WM_Exec 会自动绘制此小工具。每种类型的小工具都有若干能够修改其外观的成员函数，小工具创建后，可通过调用其成员函数之一更改其属性，这些函数使用小工具的句柄作为第一个自变量。要使上述创建的进度条显示 45% 并将条颜色从默认设置（深灰/浅灰）更改为绿色/红色，可使用以下代码段：

```
PROGBAR_SetBarColor(hProgBar, 0, GUI_GREEN);
PROGBAR_SetBarColor(hProgBar, 1, GUI_RED);
PROGBAR_SetValue(hProgBar, 45);
```

修改完以后的进度条属性如下图所示：

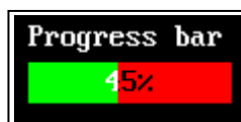


图 14.1.3.1 修改进度条属性

下面我们分为四个部分来讲解小工具想知识。

(1) 默认配置

所有小工具都具有一个或多个配置宏，它们定义各种默认设置（如所使用的字体和颜色）。在后续章节中详细讲解控件的时候就会列出了每个控件的可用配置选项。

(2) 小工具如何通信

小工具通常作为子窗口创建，父窗口可为任何窗口类型，甚至是另一种小工具，为了确保同步，无论何时父窗口的任何子项有任何事件发生，通常都应通知父窗口。有事件发生时，子窗口小工具通过发送 WM_NOTIFY_PARENT 消息与其父窗口通信，作为消息一部分发送的通知代码取决于事件。大多数小工具都有定义不同类型事件的一个或多个通知代码，每种小工具可用的通知代码（如果有）在其各自章节中列出。

(3) 皮肤设置

小工具的外观可使用其各自的成员函数进行修改，其中一些支持皮肤设置。如果对某小工具使用了皮肤设置，则“皮肤”将决定该小工具的外观，且一些成员函数将失效。

(4) 小工具的动态内存使用

在嵌入式应用中，由于分裂效应，通常都不太希望使用动态内存，可使用许多不同策略来避免此情况，但只要内存区域被应用程序中的指针引用，这些策略的使用都有局限性。因此，emWin 使用不同的方法：所有对象（以及运行时存储的所有数据）都存储在被句柄引用的内存区域中。这使它能够在运行时重新分配已分配的内存区域，因此避免了使用指针时会发生的长期分配问题。所以所有小工具都使用句柄引用。

14.2 通用控件 API 函数

14.2.1 用于控件的 WM 函数

由于小工具本质上就是窗口，所以它们与任何窗口管理器 API 例程都兼容。小工具的句柄用作 hWin 参数，且小工具的处理方式与其他任何窗口都相同。最常用于控件的 WM 函数如下表所示：

函数	描述
WM_DeleteWindow()	删除窗口
WM_DisableMemdev()	禁止使用存储设备进行重绘
WM_EnableMemdev()	启用存储设备用于重绘
WM_InvalidateWindow()	使窗口无效
WM_Paint()	立即绘制或重绘窗口

表 14.2.1.1 用于控件的常用 WM 函数

(1) WM_DeleteWindow 函数

删除指定窗口，该函数原型如下源码所示：

```
void WM_DeleteWindow(WM_HWIN hWin);
```

该函数的相关形参如下表所示：

参数	描述
hWin	窗口句柄

表 14.2.1.2 WM_DeleteWindow 函数相关形参描述

注意：删除窗口前，它接收一条 WM_DELETE 消息。此消息通常用于删除所使用的任何对象（小工具），并释放窗口动态分配的内存。如果指定的窗口具有任何已存在的子窗口，则在删除窗口前自动删除这些子窗口。因此不必单独删除子窗口。窗口删除前，会发送一条 WM_NOTIFICATION_CHILD_DELETED 消息到其父窗。

(2) WM_DisableMemdev 函数

断开窗口与其父窗口的关联。该函数原型如下源码所示：

```
void WM_DetachWindow(WM_HWIN hWin);
```

该函数的相关形参如下表所示：

参数	描述
hWin	窗口句柄

表 14.2.1.3 WM_DisableWindow 函数相关形参描述

注意：禁用的控件通常显示为灰色，且不接受来自用户的输入。当然，实际外观可能会有所不同（取决于 widget/configuration settings 等）。禁用的窗口将不会接收以下消息：WM_TOUCH、WM_TOUCH_CHILD、WM_PID_STATE_CHANGED 和 WM_MOUSEOVER。

(3) WM_EnableMemdev 函数

将指定的窗口设置为启用状态。启用的窗口接收来自 WM 的指针输入设备（PID）消息（触摸、鼠标、操纵手柄 ...），该函数原型如下源码所示：

```
void WM_EnableWindow(WM_Handle hObj);
```

该函数的相关形参如下表所示：

参数	描述
hObj	窗口句柄

表 14.2.1.4 WM_EnableWindow 函数相关形参描述

(4) WM_InvalidateWindow 函数

使指定窗口无效，该函数原型如下源码所示：

```
void WM_InvalidateWindow (WM_Handle hObj);
```

该函数的相关形参如下表所示：

参数	描述
hObj	窗口句柄

表 14.2.1.5 WM_InvalidateWindow 函数相关形参描述

(5) WM_Paint 函数

立即绘制或重绘指定窗口，该函数原型如下源码所示：

```
void WM_Paint(WM_HWIN hWin);
```

该函数的相关形参如下表所示：

参数	描述
hWin	窗口句柄

表 14.2.1.6 WM_Paint 函数相关形参描述

14.2.2 常用 API 函数

表 14.2.2.1 按字母顺序列出了可用的小工具相关函数，这些函数对于所有控件都是通用的，在此列出以避免重复，每个小工具可用的额外成员函数在稍后各章节中介绍。

函数	描述
<WIDGET>_Callback()	默认回调函数
<WIDGET>_CreateIndirect()	用于对话框中的自动创建
<WIDGET>_CreateUser()	使用额外字节作为用户数据创建小工具
<WIDGET>_GetUserData()	检索用<WIDGET>_SetUserData 设置的数据
<WIDGET>_SetUserData()	设置小工具的额外数据
WIDGET_GetDefaultEffect()	返回用于小工具的默认效果
WIDGET_SetDefaultEffect()	设置用于小工具的默认效果
WIDGET_SetEffect()	设置用于给定小工具的效果

表 14.2.2.1 控件通用 API 函数

<WIDGET>表示的是小工具的名称，其中函数<WIDGET>_CreateIndirect()尤为重要，因为在使用对话框的时候就是使用的这个函数来创建各个小工具的。

(1) <WIDGET>_Callback 函数

覆盖回调函数内部使用的控件的默认回调函数，该函数原型如下源码所示：

```
void <WIDGET>_Callback(WM_MESSAGE * pMsg);
```

该函数的相关形参如下表所示：

参数	描述
pMsg	指向要显示的位图

表 14.2.2.2 <WIDGET>_Callback 函数相关形参描述

(2) <WIDGET>_CreateIndirect 函数

创建要在对话框中使用的控件，该函数原型如下源码所示：

```
<WIDGET>_Handle <WIDGET>_CreateIndirect(
    const GUI_WIDGET_CREATE_INFO * pCreateInfo,
    WM_HWIN hParent,
    int x0,
    int y0,
    WM_CALLBACK * cb);
```

该函数的相关形参如下表所示：

参数	描述
pCreateInfo	指向 GUI_WIDGET_CREATE_INFO 结构的指针
hParent	父窗口的句柄
x0	控件最左侧的像素（在父坐标中）
y0	控件最顶端的像素（在父坐标中）
cb	指向回调函数的指针

表 14.2.2.3 <WIDGET>_CreateIndirect 函数相关形参描述

GUI_WIDGET_CREATE_INFO 结构如下源码所示：

```
typedef struct {
    GUI_WIDGET_CREATE_FUNC * pfCreateIndirect; /* 创建函数 */
    const char * pName; /* 文本(不是用于所有小部件) */
    I16 Id; /* 小部件的窗口 ID */
    I16 x0, y0, xSize, ySize; /* 小部件的大小和位置 */
    I16 Flags; /* 特定于小部件的标志(或 0) */
    I32 Para; /* 特定于部件的参数(或 0) */
    U32 NumExtraBytes; /* 可用的额外字节数 */
    /* with <WIDGET>_SetUserData &<WIDGET>_GetUserData */
} GUI_WIDGET_CREATE_INFO;
```

小工具标记和参数是可选的，并随小工具类型不同而不同。每个小工具可用的标记和参数（如果有）将在后续章节中讲解。

(3) <WIDGET>_CreateUser 函数

使用额外字节作为用户数据创建控件。此函数与相应控件的<WIDGET>_CreateEx 函数在每一方面都相似，除了额外参数 NumExtraBytes 以外，该函数原型如下源码所示：

```
<WIDGET>_Handle <WIDGET>_CreateUser(int x0, int y0, ...,
    int Id,int NumExtraBytes);
```

该函数的相关形参如下表所示：

参数	描述
NumExtraBytes	要分配的额外字节数

表 14.2.2.4 <WIDGET>_CreateUser 函数相关形参描述

返回值：已创建控件的句柄; 0 如果函数失败。

(4) <WIDGET>_GetUserData 函数

获取<WIDGET>_SetUserData 函数设置的数据，该函数原型如下源码所示：

```
int <WIDGET>_GetUserData ( <WIDGET>_Handle hObj, void * pDest, int NumBytes);
```

该函数的相关形参如下表所示：

参数	描述
hObj	控件的句柄
pDest	指向缓冲区的指针
NumBytes	要读取的字节数

表 14.2.2.5 <WIDGET>_GetUserData 函数相关形参描述

返回值：已读取的字节数。

(5) <WIDGET>_SetUserData 函数

设置控件的额外数据，该函数原型如下源码所示：

```
int <WIDGET>_SetUserData( <WIDGET>_Handle hObj, void * pDest, int NumBytes);
```

该函数的相关形参如下表所示：

参数	描述
hObj	指向要显示的位图
pDest	指向缓冲区的指针
NumBytes	要写入的字节数

表 14.2.2.6 <WIDGET>_SetUserData 函数相关形参描述

(6) WIDGET_GetDefaultEffect 函数

获取用于控件的默认效果，该函数原型如下源码所示：

```
const WIDGET_EFFECT * WIDGET_GetDefaultEffect(void);
```

返回值：此函数的结果是 WIDGET_EFFECT 结构的指针。

(7) WIDGET_SetDefaultEffect 函数

设置用于控件的默认效果，该函数原型如以下所示：

```
const WIDGET_EFFECT * WIDGET_SetDefaultEffect(const WIDGET_EFFECT *pEffect);
```

该函数的相关形参如下表所示：

参数	描述
pEffect	指向 WIDGET_EFFECT 结构的指针
	&WIDGET_Effect_3D
	&WIDGET_Effect_None
	&WIDGET_Effect_Simple

表 14.2.2.7 WIDGET_SetDefaultEffect 函数相关形参描述

(8) WIDGET_SetEffect 函数

指向 WIDGET_EFFECT 结构的指针，该函数原型如下源码所示：

```
void WIDGET_SetEffect(WM_HWIN hObj, const WIDGET_EFFECT *pEffect);
```

该函数的相关形参如下表所示：

参数	描述
hObj	控件的句柄
pEffect	指向 WIDGET_EFFECT 结构的指针

表 14.2.2.8 WIDGET_SetEffect 函数相关形参描述

14.2.3 用户绘制控件

一些小工具支持自画功能，例如 LISTBOX 控件小工具的用户绘制模式激活后，将调用 WIDGET_DRAW_ITEM_FUNC 类型的应用程序定义函数来绘制该控件（项目）。应用程序定义的自画函数的原型应按如下定义：

```
int WIDGET_DRAW_ITEM_FUNC(const WIDGET_ITEM_DRAW_INFO *pDrawItemInfo)
```

该函数的形参如下表所示：

参数	描述		
pDrawItemInfo	指向 WIDGET_ITEM_DRAW_INFO 结构的指针		
	hWin(WM_HWIN)	小工具的句柄	
	Cmd (int)	元素 Cmd 的允许值	
		WIDGET_ITEM_GET_XSIZE	该函数返回给定项目的 x 尺寸（宽度），单位为像素
		WIDGET_ITEM_GET_YSIZE	该函数返回给定项目的 y 尺寸（高度），单位为像素
		WIDGET_ITEM_DRAW	函数在指定位置绘制指定项目
		WIDGET_DRAW_BACKGROUND	要绘制的小工具的背景
		WIDGET_DRAW_OVERLAY	目的是可以在小工具之上绘制一些覆盖项目
	ItemIndex (int)	要绘制项目的以零为基准的索引	
	x0 (int)	用于绘制项目的 x0 窗口坐标	
	y0 (int)	用于绘制项目的 y0 窗口坐标	
	x1	用于绘制项目的 x1 窗口坐	
y1	用于绘制项目的 y1 窗口坐标。		

表 14.2.3.1 WIDGET_DRAW_ITEM_FUNC 函数相关形参描述

返回值：取决于给定命令。

对命令的反应：该函数必须对在 WIDGET_ITEM_DRAW_INFO 结构中给定的命令作出反应，以下 2 种方式之一执行：

- ✧ 调用特定控件提供的适当默认函数（例如，LISTBOX_OwnerDraw）。
- ✧ 提供作出相应反应的代码。

命令：该函数支持并应对以下所列命令作出反应，如上所描述，应为所有未处理函数调用默认自画函数，这可减小代码大小（例如，如果高度与默认高度相同），并确保代码与将来的软件新版本和引入的其他命令保持兼容如下表所示：

参数	描述
WIDGET_ITEM_GET_XSIZE	返回给定项目的 X 尺寸，以像素为单位
WIDGET_ITEM_GET_YSIZE	返回给定项目的 Y 尺寸（高度），以像素为单位
WIDGET_ITEM_DRAW	绘制给定项目，WIDGET_ITEM_DRAW_INFO 结构的 x0 和 y0 指定项目在窗口坐标中的位置。该项目必须填充其整个

	矩形，该矩形由提供给函数的起始位置 x0、y0 以及函数反应命令 WIDGET_ITEM_GET_YSIZE 、 WIDGET_ITEM_GET_XSIZE 返回的尺寸定义。此矩形区域的任何部分都不得留空，它不能绘制到此矩形外，因为在调用该函数之前已设置了裁剪矩
--	--

表 14.2.3.2 相关命令描述

第十五章 BUTTON 控件

从本章开始我们就讲解 emWin 中各个控件，首先讲解的就是最常用的按钮控件，按钮控件通常用作触摸屏的主要用户界面元素，如果按钮有输入焦点，它还会对键 GUI_KEY_SPACE 和 GUI_KEY_ENTER 作出反应。

本章分为以下几个部分：

15.1 BUTTON 控件简介

15.2 通知代码

15.3 键盘反应

15.4 BUTTON 常用 API 函数

15.5 BUTTON 控件基础实验

15.6 BUTTON 控件显示位图实验

15.1 BUTTON 控件简介

按钮小部件通常用作触摸屏的主要用户界面元素。如果按钮有输入焦点，它还会对键 GUI_KEY_SPACE 和作出反应 GUI_KEY_ENTER。按钮可以用文本显示，如下图所示，也可以用位图显示。



图 15.1.1 默认按钮显示效果

我们可以通过设置按钮的皮肤改变按钮的显示效果，设置按钮皮肤以后显示如下图所示：



图 15.1.2 设置皮肤后的按钮显示效果

15.2 通知代码

以下事件是按钮小工具作为 WM_NOTIFY_PARENT 消息的一部分发送给其父窗口的，我们通常根据以下通知代码来编写按钮的功能的，尤其是 GUIBulider 生成的代码中，如下表列出了可能的通知代码。

消息	描述
WM_NOTIFICATION_CLICKED	按钮已被点击
WM_NOTIFICATION_RELEASED	按钮已被释放
WM_NOTIFICATION_MOVED_OUT	按钮已被点击，且指针已移出按钮并且没有释放

表 15.2.1 BUTTON 通知代码

15.3 键盘反应

如果小工具具有输入焦点，则它将对下列各键做出反应，如下表列出了按键反应：

按键	反应
GUI_KEY_ENTER	此类键按下后，按钮认为它被按下并立即释放，然后作出相应反应
GUI_KEY_SPACE	此类键按下后，状态改变为已按下。键释放后，状态改变为未按下

表 15.3.1 BUTTON 的按键反应

15.4 BUTTON 常用 API 函数

表 15.4.1 列举出了 BUTTON 的 API 函数：

函数	描述
BUTTON_Create()	创建 BUTTON 小工具
BUTTON_CreateAsChild()	将 BUTTON 小工具创建为子窗口
BUTTON_CreateEx()	创建 BUTTON 小工具
BUTTON_CreateIndirect()	从资源表项创建 BUTTON 小工具
BUTTON_CreateUser()	使用额外字节作为用户数据创建 BUTTON 小工具
BUTTON_GetBitmap()	返回指向 BUTTON 位图的指针
BUTTON_GetBkColor()	返回 BUTTON 的背景色
BUTTON_GetDefaultBkColor()	返回 BUTTON 小工具的默认背景色
BUTTON_GetDefaultFont()	返回 BUTTON 小工具的默认字体
BUTTON_GetDefaultTextAlign()	返回 BUTTON 小工具的默认文本对齐方式
BUTTON_GetDefaultTextColor()	返回 BUTTON 小工具的默认文本颜色
BUTTON_GetFont()	返回 BUTTON 小工具字体的指针
BUTTON_GetText()	检索指定 BUTTON 的文本
BUTTON_GetTextAlign()	返回 BUTTON 文本的对齐方式
BUTTON_GetTextColor()	返回指定 BUTTON 的文本颜色
BUTTON_GetUserData()	检索用 BUTTON_SetUserData 设置的数据
BUTTON_IsPressed()	返回表示按钮是否按下的值
BUTTON_SetBitmap()	设置显示 BUTTON 时使用的位图
BUTTON_SetBitmapEx()	设置显示 BUTTON 时使用的位图
BUTTON_SetBkColor()	设置按钮的背景色
BUTTON_SetBMP()	设置显示 BUTTON 时使用的位图
BUTTON_SetBMPEX()	设置显示 BUTTON 时使用的位图
BUTTON_SetDefaultBkColor()	设置 BUTTON 小工具的默认背景色
BUTTON_SetDefaultFocusColor()	设置按钮小部件的默认焦点矩形颜色
BUTTON_SetDefaultFont()	设置 BUTTON 小工具的默认字体
BUTTON_SetDefaultTextAlign()	设置 BUTTON 小工具的默认文本对齐方式
BUTTON_SetDefaultTextColor()	设置 BUTTON 小工具的默认文本颜色
BUTTON_SetFocusColor()	设置用于呈现按钮小部件的焦点矩形的颜色
BUTTON_SetFocussable()	设置接收输入焦点的能力
BUTTON_SetFont()	选择文本的字体
BUTTON_SetFrameColor()	设置按钮框架的颜色
BUTTON_SetPressed()	将按钮的状态设置为按下或未按下
BUTTON_SetReactOnLevel()	将所有 BUTTON 小工具设置为对水平做出反应
BUTTON_SetReactOnTouch()	设置所有按钮小部件对触摸事件作出反应
BUTTON_SetStreamedBitmap()	设置显示 BUTTON 小工具时使用的位图
BUTTON_SetStreamedBitmapEx()	设置显示 BUTTON 小工具时使用的位图
BUTTON_SetText()	设置文本
BUTTON_SetTextAlign()	设置 BUTTON 文本的对齐方式
BUTTON_SetTextColor()	设置文本的颜色
BUTTON_SetTextOffset()	根据当前的文本对齐方式设置，调节按钮文本的位置
BUTTON_SetUserData()	设置 BUTTON 小工具的额外数据

表 15.4.1 BUTTON 控件 API 函数

表 15.4.1 是 BUTTON 控件函数相关描述。接下来，我们来讲解 BUTTON 控件常用的函数。

(1) BUTTON_CreateEx 函数

指定位置创建指定尺寸的 BUTTON 控件，该函数原型如下源码所示：

```
BUTTON_Handle BUTTON_CreateEx(int x0, int y0, int xSize, int ySize,
                               WM_HWIN hParent, int WinFlags, int ExFlags,
                               int Id);
```

该函数的相关形参如下表所示：

参数	描述
x0	控件最左侧的像素（在父坐标中）
y0	控件最顶端的像素（在父坐标中）
xSize	控件的水平尺寸（以像素为单位）
ySize	控件的垂直尺寸（以像素为单位）
hParent	父窗口的句柄，如果为 0，则 BUTTON 控件将是桌面（顶级窗口）的子项
WinFlags	窗口创建标记，为了使控件立即可见，通常为 WM_CF_SHOW
ExFlags	未使用，保留供日后使用
Id	控件的窗口 ID

表 15.4.2 BUTTON_CreateEx 函数相关形参描述

返回值：已创建 BUTTON 控件的句柄；函数失败时为 0。

(2) BUTTON_SetFont 函数

设置按钮控件字体，该函数原型如下源码所示：

```
void BUTTON_SetFont(BUTTON_Handle hObj, const GUI_FONT *pFont);
```

该函数的相关形参如下表所示：

参数	描述
hObj	BUTTON 控件的句柄
pFont	指向字体的指针

表 15.4.3 BUTTON_SetFont 函数相关形参描述

(3) BUTTON_SetText 函数

设置要在按钮控件上显示的文本，该函数原型如下源码所示：

```
int BUTTON_SetText(BUTTON_Handle hObj, const char *s);
```

该函数的相关形参如下表所示：

参数	描述
hObj	BUTTON 控件的句柄
s	要显示的文本

表 15.4.4 BUTTON_SetText 函数相关形参描述

返回值：0 成功，1 错误。

15.5 BUTTON 控件基础实验

15.5.1 硬件设计

1. 例程功能

本实验主要使用 GUIBulider 软件设置两个 BUTTON 小工具，这两个 BUTTON 小工具分别对开发板上的 LED0 和 BEEP 状态进行翻转。在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 `emwin_main` 执行 emWin 程序，该函数主要测试 BUTTON 小工具的使用，当用户按下界面上的 LED 按键位图时，系统会对开发板 LED0 的状态进行翻转；当用户按下界面上的 BEEP 按键位图时，系统会对开发板上的蜂鸣器的状态进行翻转。

touch_task 任务：主要调用 emWin 的 `GUI_TOUCH_Exec` 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 15 BUTTON 使用》。

15.5.2 软件设计

15.5.2.1 程序流程图

本实验的程序流程图，如下图所示：

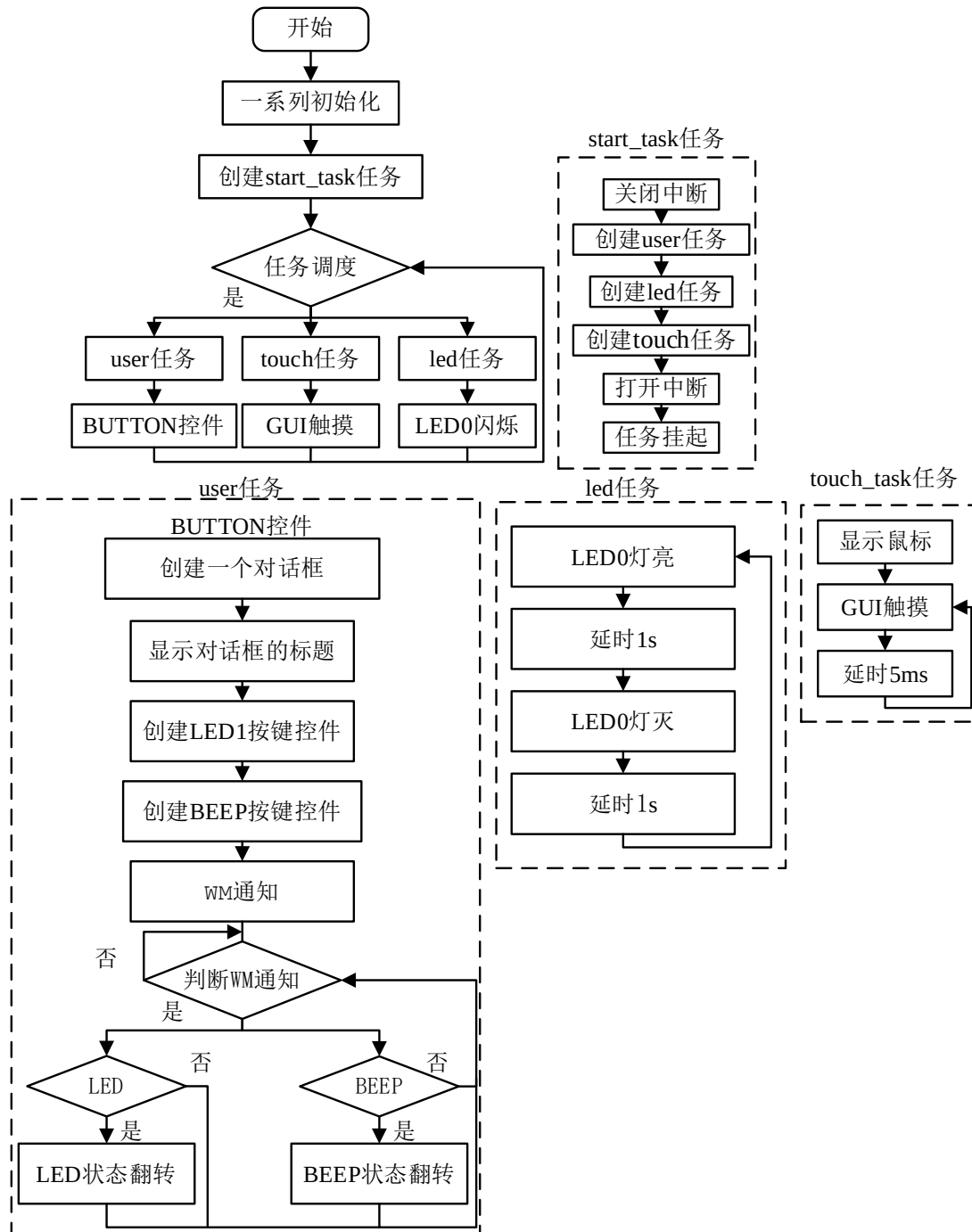


图 15.5.2.1.1 BUTTON 控件使用实验流程图

15.5.2.2 程序解析

我们用 GUIBulider 创建一个如图 15.5.2.3.1 所示的 GUI 界面，此界面包括一个 Frame，两个 BUTTON，两个 BUTTON 用来控制开发板上的 LED1 和蜂鸣器的开关，界面分辨率为 800*480，大家也可以根据自己所使用的 LCD 来设置合适的界面大小。



图 15.5.2.3.1 按钮显示效果图

实验完整工程为：**emWin 实验 15 BUTTON 使用**，打开 emwin_demo.c 文件，源码如下：

(1) GUIBulider 生成的资源表

```
/* 对话框资源表 */
static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] =
{
{ FRAMEWIN_CreateIndirect, "Framewin", ID_FRAMEWIN_0, 0, 0, 800, 480, 0, 0x64, 0 },
{ BUTTON_CreateIndirect, "Button", ID_BUTTON_0, 300, 122, 150, 50, 0, 0x0, 0 },
{ BUTTON_CreateIndirect, "Button", ID_BUTTON_1, 300, 251, 150, 50, 0, 0x0, 0 },
};
```

从上述的源码中，可以看出使用 GUIBulider 软件创建了三个控件，一个 Framewin 和两个 Button 控件。

(2) 建立对话框

```
/**
 * @brief      创建一个对话框
 * @param      无
 * @return     无
 */
WM_HWIN emwin_create_framewin(void)
{
    WM_HWIN hWin;

    hWin = GUI_CreateDialogBox(_aDialogCreate, GUI_COUNTOF(_aDialogCreate),
                                _cbDialog, WM_HBKWIN, 0, 0);

    return hWin;
}

/**
 * @brief      emwin_main(emwin 入口)
```

```

* @param      无
* @return     无
*/
void emwin_main(void)
{
    WM_HWIN hWin;
    hWin = emwin_create_framewin();

    while (1)
    {
        GUI_Delay(100);
    }
}

```

在上述源码可以看出调用函数 GUI_CreateDialogBox 创建资源表中的控件。

(3) 对话框的回调函数

```

/**
* @brief      对话框回调函数
* @param      无
* @return     无
*/
static void _cbDialog(WM_MESSAGE *pMsg)
{
    WM_HWIN hItem;
    int      NCode;
    int      Id;

    switch (pMsg->MsgId)
    {
        case WM_INIT_DIALOG:
            /* 初始化对话框 */
            hItem = pMsg->hWin;
            FRAMEWIN_SetTitleHeight(hItem, 30);
            FRAMEWIN_SetText(hItem, "ALIENTE BUTTON USE");
            FRAMEWIN_SetFont(hItem, GUI_FONT_24_ASCII);
            FRAMEWIN_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
            FRAMEWIN_SetTextColor(hItem, 0x0000FFFF);

            /* 初始化 BUTTON0 */
            hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_0);
            BUTTON_SetFont(hItem, GUI_FONT_24_ASCII);
            BUTTON_SetText(hItem, "LED1");

            /* 初始化 BUTTON1 */

```

```

        hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_1);
        BUTTON_SetText(hItem, "BEEP");
        BUTTON_SetFont(hItem, GUI_FONT_24_ASCII);
        break;

    case WM_NOTIFY_PARENT:
        Id      = WM_GetId(pMsg->hWinSrc);
        NCode = pMsg->Data.v;

        switch (Id)
        {
            case ID_BUTTON_0: /* BUTTON_0 的通知代码，控制 LED1 */
                switch (NCode)
                {
                    case WM_NOTIFICATION_CLICKED:
                        break;

                    case WM_NOTIFICATION_RELEASED: /* 按钮被按下并释放 */
                        LED1_TOGGLE(); /* LED1 反转 */
                        break;
                }
                break;

            case ID_BUTTON_1: /* BUTTON_1 的通知代码，控制 BEEP */
                switch (NCode)
                {
                    case WM_NOTIFICATION_CLICKED:
                        break;

                    case WM_NOTIFICATION_RELEASED:
                        BEEP_TOGGLE(); /* 蜂鸣器反转 */
                        break;
                }
                break;
        }
        break;
    default:
        WM_DefaultProc(pMsg);
        break;
}
}

```

上述源码 WM_INIT_DIALOG 对控件进行初始化，而 WM_NOTIFY_PARENT 为通知消息，使用函数 NCode = pMsg->Data.v 来判断 LCD 界面的两个按键的触发信号，如果 LED1 按下时，开发板的 LED1 亮，如果 BEEP 按下时，开发板上的蜂鸣器会相应的动作。

15.5.3 下载验证

编译并下载到开发板中，结果显示如图 15.5.3.1 所示，图 15.5.3.1 是设置了皮肤的，请大家注意了。



图 15.5.3.1 按钮显示效果图

通过按下 LED1 和 BEEP 按钮来控制开发板上 LED1 和蜂鸣器的打开和关闭。

15.6 BUTTON 控件显示位图实验

上一小节我们讲解了如何使用 emWin 的 BUTTON 控件，有时候我们希望在按钮上显示一些图片以区分打开和关闭，比如上一小节中打开 LED，使用 emWin 自带的 BUTTON 控件的时候打开和关闭 LED 时 BUTTON 的状态都是一样的，不能很只管的判断出此时 LED 灯的状态，需要我们去查看 LED 的亮灭才能判断。emWin 提供了在 BUTTON 上显示位图的 API 函数，那么我们就可以在打开和关闭的时候通过显示不同的位图来区分开、关这两种状态，本节就讲解一下如何在 BUTTON 上显示位图。

15.6.1 制作位图 C 文件

首先我们肯定需要制作两个位图文件，当关闭的时候我们在 BUTTON 上显示图 15.6.1.1 所示图片，打开的时候显示图 15.6.1.2 所示图片，这两种图片的格式均为 BMP。



图 15.6.1.1 OFF



图 15.6.1.2 ON

在第七章我们就已经讲过了位图转换器的使用方法，本节我们在回顾一下，首先打开 OFF 图片，打开后如图 15.6.1.3 所示：

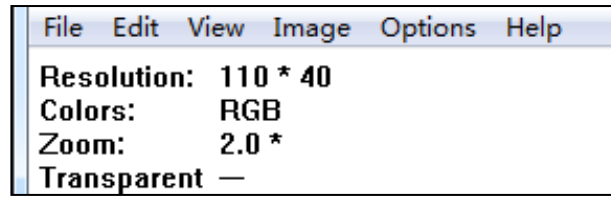


图 15.6.1.3 使用位图转换器打开 OFF 图片

设置色彩转换，点击 Image->Convert to->Best palette，设置完成以后如图 15.6.1.4 所示：

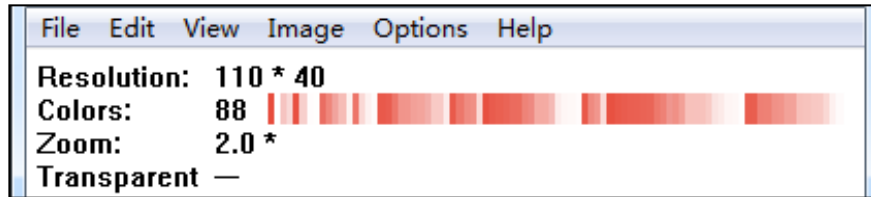


图 15.6.1.4 设置色彩转换

设置为色彩转换格式以后就可以保存为 C 文件了，点击 File->Save As，提示如图 15.6.1.5 所示对话框，设置好保存的文件类型和保存路径，我这里是保存在桌面上，注意图 15.6.1.5 红框中的保存类型！

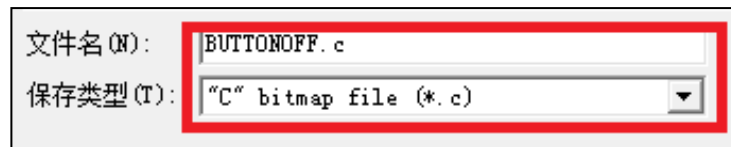


图 15.6.1.5 保存 C 文件

点击图 15.6.1.5 中的保存以后又会跳出图 15.6.1.6 所示对话框，用来设置生成的格式，这里选择“High color(565)”。

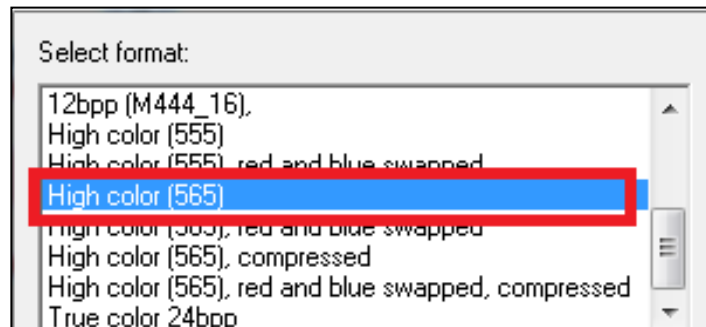


图 15.6.1.6 设置格式

设置完图 14.6.1.6 中的格式以后点击“OK”按钮，这样就会在刚刚设置的保存路径下生成一个 C 文件，这个 C 文件就是位图转换器生成的位图文件，我们一会要用这个位图文件，用同样的方法着 ON 图片的位图文件。

15.6.2 GUIBulider 设计界面

用 GUIBulider 设计如下图所示的界面：

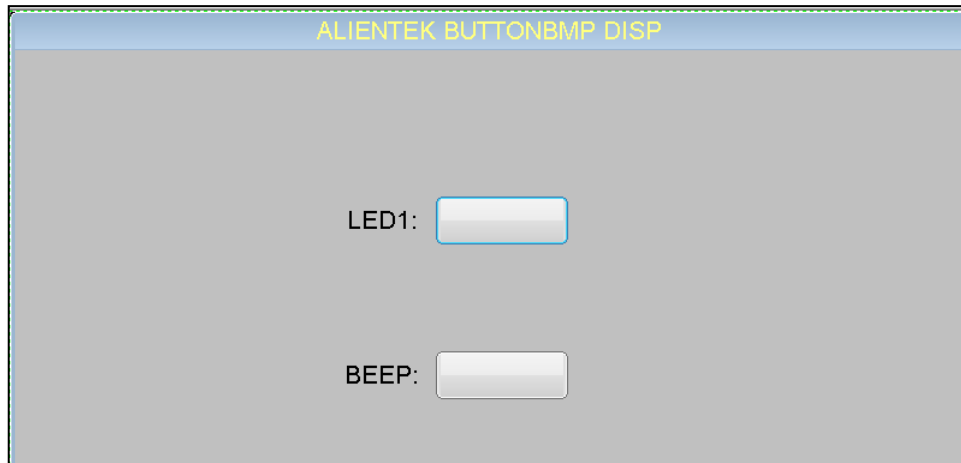


图 15.6.2.1 界面设计

图 15.6.2.1 用 GUIBulider 设计出的界面中使用了一个我们还没有讲解过的控件：TEXT，我们用 TEXT 控件显示“LED1:”和“BEEP:”这两个提示信息。图中的按钮还是使用的 emwin 默认的按钮类型，要想修改为位图就需要修改生成的 C 文件，注意：BUTTON 控件上没有显示字符，GUIBulider 只是生成一个大体的程序框架而已，生成的 C 程序如下所示，为了节省篇幅，这里删除了部分注释行。

```

/*****
*
*      _aDialogCreate
*/
static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] = {
    {FRAMEWIN_CreateIndirect, "Framewin", ID_FRAMEWIN_0, 0, 0, 800, 480, 0, 0x64, 0 },
    {BUTTON_CreateIndirect, "Button", ID_BUTTON_0, 350, 122, 110, 40, 0, 0x0, 0 },
    {BUTTON_CreateIndirect, "Button", ID_BUTTON_1, 350, 251, 110, 40, 0, 0x0, 0 },
    {TEXT_CreateIndirect, "Text", ID_TEXT_0, 266, 132, 80, 20, 0, 0x64, 0 },
    {TEXT_CreateIndirect, "Text", ID_TEXT_1, 266, 261, 80, 20, 0, 0x64, 0 },
};
/*****
*
*      _cbDialog
*/
static void _cbDialog(WM_MESSAGE * pMsg)
{
    WM_HWIN hItem;
    int      NCode;
    int      Id;
    switch (pMsg->MsgId)
    {
        case WM_INIT_DIALOG:
            //
            // Initialization of 'Framewin'

```

```
//
hItem = pMsg->hWin;
FRAMEWIN_SetTitleHeight(hItem, 30);
FRAMEWIN_SetText(hItem, "ALIENTEK BUTTONBMP DISP");
FRAMEWIN_SetFont(hItem, GUI_FONT_24_ASCII);
FRAMEWIN_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
FRAMEWIN_SetTextColor(hItem, 0x0000FFFF);
//
// Initialization of 'Button'
//
hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_0);
BUTTON_SetText(hItem, "");
//
// Initialization of 'Button'
//
hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_1);
BUTTON_SetText(hItem, "");
//
// Initialization of 'Text'
//
hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_0);
TEXT_SetFont(hItem, GUI_FONT_32_ASCII);
TEXT_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
TEXT_SetText(hItem, "LED1:");

hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_1);
TEXT_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
TEXT_SetFont(hItem, GUI_FONT_32_ASCII);
TEXT_SetText(hItem, "BEEP:");
break;
case WM_NOTIFY_PARENT:
    Id      = WM_GetId(pMsg->hWinSrc);
    NCode = pMsg->Data.v;
    switch(Id)
    {
        case ID_BUTTON_0: // Notifications sent by 'Button'
            switch(NCode)
            {
                case WM_NOTIFICATION_CLICKED:
                    break;
                case WM_NOTIFICATION_RELEASED:
                    break;
            }
    }
}
```

```

        break;
    case ID_BUTTON_1: // Notifications sent by 'Button'
        switch(NCode)
        {
            case WM_NOTIFICATION_CLICKED:
                break;
            case WM_NOTIFICATION_RELEASED:
                break;
        }
        break;
    }
    break;
default:
    WM_DefaultProc(pMsg);
    break;
}
}

WM_HWIN emwin_create_framewin(void);

WM_HWIN emwin_create_framewin(void)
{
    WM_HWIN hWin;
    hWin = GUI_CreateDialogBox(_aDialogCreate, GUI_COUNTOF(_aDialogCreate)
        , _cbDialog, WM_HBKWIN, 0, 0);

    return hWin;
}

```

上面程序就是用 GUIBulider 生成的模板文件，接下来我们就要在这个模板文件中修修改改、缝缝补补的完成本次实验。新建 `buttonbmp.c` 文件将生成的位图文件都放进去，整理好的位图文件如下所示：

```

#include "buttonbmp.h"
#include <stdlib.h>

static GUI_CONST_STORAGE unsigned short _acBUTTONOFF[] = {
    /* 数组数据 */
};

static GUI_CONST_STORAGE unsigned short _acBUTTONON[] = {
    /* 数组数据 */
};

GUI_CONST_STORAGE GUI_BITMAP bmBUTTONOFF = {
    110, /* xSize */
    40, /* ySize */
    220, /* BytesPerLine */

```

```

16,                                /* BitsPerPixel */
(unsigned char *)_acBUTTONOFF,      /* Pointer to picture data */
NULL,                              /* Pointer to palette */
GUI_DRAW_BMP565
};

GUI_CONST_STORAGE GUI_BITMAP bmBUTTONON = {
110,                                /* xSize */
40,                                 /* ySize */
220,                               /* BytesPerLine */
16,                                /* BitsPerPixel */
(unsigned char *)_acBUTTONON,      /* Pointer to picture data */
NULL,                              /* Pointer to palette */
GUI_DRAW_BMP565
};

```

值得我们注意一点就是位图的大小与按键的大小一致，不然可能显示一部分位图。在 buttonbmp.c 文件就是将前面生成的那两个位图 C 文件复制到一个 C 文件里面，很简单，这里就不讲解了，上面为了篇幅原因没贴出数组 _acBUTTONOFF[] 和 _acBUTTONON 中的数据，新建 buttonbmp.h 文件，添加如下代码：

```

#ifndef _BUTTONBMP_H
#define _BUTTONBMP_H
#include "sys.h"
#include "GUI.h"
#ifndef GUI_CONST_STORAGE
#define GUI_CONST_STORAGE const
#endif
extern GUI_CONST_STORAGE GUI_BITMAP bmBUTTONOFF;
extern GUI_CONST_STORAGE GUI_BITMAP bmBUTTONON;
#endif

```

buttonbmp.h 文件主要是声明了两个 GUI_BITMAP 结构体变量：bmBUTTONOFF 和 bmBUTTONON，这样我们就可以在其他文件中调用这两个结构体变量，可以简单的理解为这两个结构体变量就是图片 OFF 和 ON。

15.6.3 硬件设计

1. 例程功能

本实验主要测试 GUIBulider 软件的使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数主要测试 GUIBulider 软件生成的.c 文件是否可在 MCU 下运行。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 16 BUTTON 位图显示》。

15.6.4 软件设计

15.6.4.1 程序流程图

本实验的程序流程图，如下图所示：

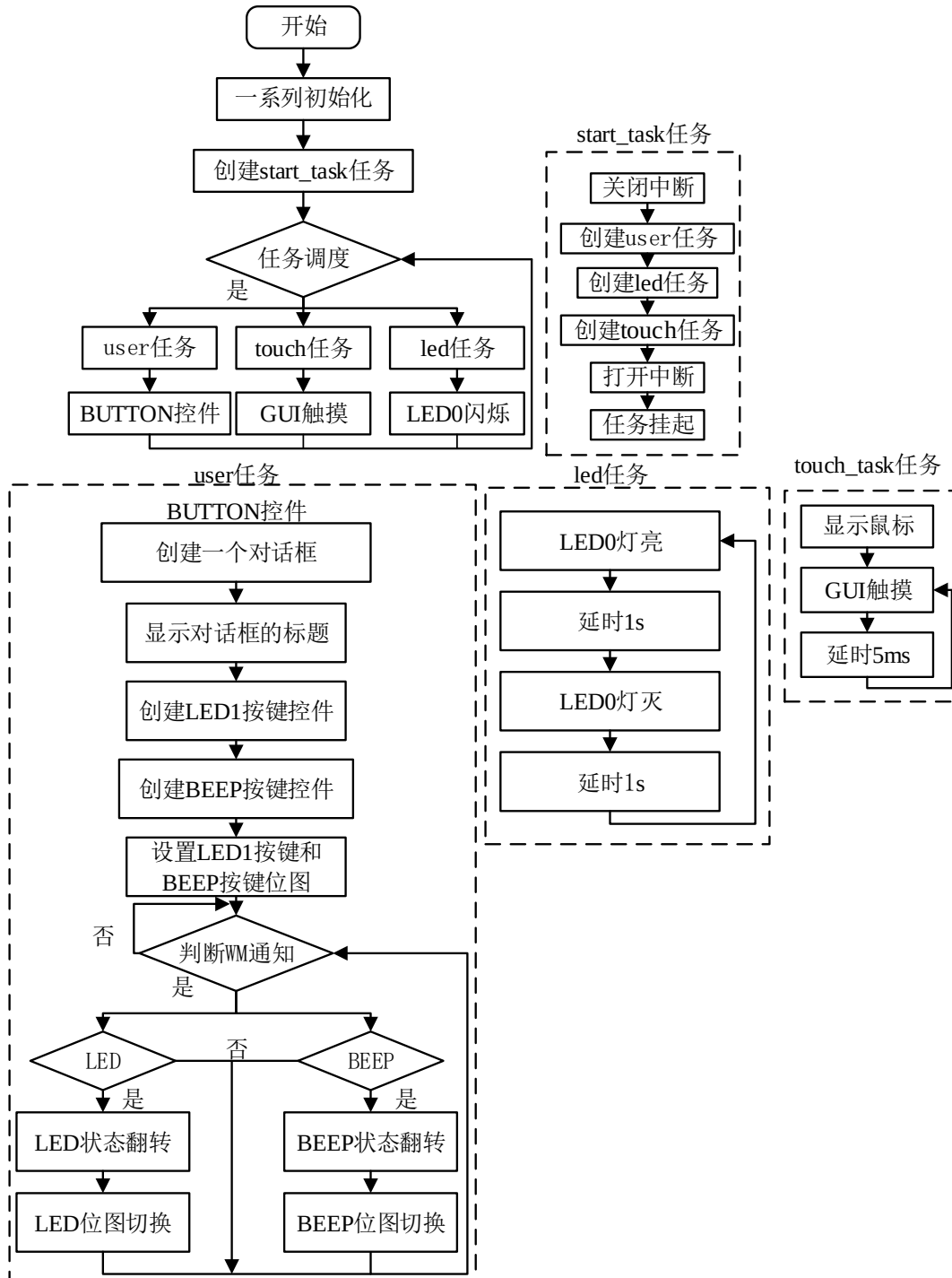


图 15.6.4.1.1 BUTTON 位图实验流程图

15.6.4.2 程序解析

本实验完整工程为：**emWin 实验 16 BUTTON 位图显示**，打开 emwin_demo.c 文件，这个文件就是在 GUIBulider 生成的 C 文件基础上修改的，打开以后程序源码如下：

(1) GUIBulider 生成的资源表

```
/* 对话框资源表 */
static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] =
{
    { FRAMEWIN_CreateIndirect, "Framewin", ID_FRAMEWIN_0, 0, 0, 800, 480, 0, 0x64, 0 },
    { BUTTON_CreateIndirect, "Button", ID_BUTTON_0, 350, 122, 110, 40, 0, 0x0, 0 },
    { BUTTON_CreateIndirect, "Button", ID_BUTTON_1, 350, 251, 110, 40, 0, 0x0, 0 },
    { TEXT_CreateIndirect, "Text", ID_TEXT_0, 266, 132, 80, 20, 0, 0x64, 0 },
    { TEXT_CreateIndirect, "Text", ID_TEXT_1, 266, 261, 80, 20, 0, 0x64, 0 },
};
```

(2) 建立对话框

```
/**
 * @brief      创建一个对话框
 * @param      无
 * @return     无
 */
WM_HWIN emwin_create_framewin(void)
{
    WM_HWIN hWin;
    buttonbmp_tab[0] = bmBUTTONOFF;
    buttonbmp_tab[1] = bmBUTTONON;
    hWin = GUI_CreateDialogBox(_aDialogCreate, GUI_COUNTOF(_aDialogCreate),
                               _cbDialog, WM_HBKWIN, 0, 0);

    return hWin;
}

/**
 * @brief      emwin_main(emwin 入口)
 * @param      无
 * @return     无
 */
void emwin_main (void)
{
    WM_HWIN hWin;
    hWin = emwin_create_framewin();

    while (1)
    {
        GUI_Delay(100);
    }
}
```

```
}
```

在上述的源码中，调用函数 GUI_CreateDialogBox()使用资源表创建对话框。

(3) 对话框的回调函数

```
/**
 * @brief      对话框回调函数
 * @param      pMsg
 * @return     无
 */
static void _cbDialog(WM_MESSAGE *pMsg)
{
    WM_HWIN hItem;
    int      NCode;
    int      Id;
    static uint8_t ledflag = 0;
    static uint8_t beepflag = 0;

    switch (pMsg->MsgId)
    {
        case WM_PAINT:
            GUI_SetBkColor(GUI_WHITE);
            GUI_Clear();
            break;

        case WM_INIT_DIALOG:
            /* 初始化对话框 */
            hItem = pMsg->hWin;
            FRAMEWIN_SetTitleHeight(hItem, 30);
            FRAMEWIN_SetText(hItem, "ALIENTEK BUTTONBMP DISP");
            FRAMEWIN_SetFont(hItem, GUI_FONT_24_ASCII);
            FRAMEWIN_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
            FRAMEWIN_SetTextColor(hItem, 0x0000FFFF);

            /* 初始化 BUTTON0 */
            hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_0);
            BUTTON_SetBitmapEx(hItem, 0, &buttonbmp_tab[0], 0, 0);
            BUTTON_SetText(hItem, "");

            /* 初始化 BUTTON1 */
            hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_1);
            BUTTON_SetBitmapEx(hItem, 0, &buttonbmp_tab[0], 0, 0);
            BUTTON_SetText(hItem, "");

            /* 初始化 TEXT0 */
    }
```

```

hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_0);
TEXT_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
TEXT_SetFont(hItem, GUI_FONT_32_ASCII);
TEXT_SetText(hItem, "LED1:");

/* 初始化 TEXT1 */
hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_1);
TEXT_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
TEXT_SetFont(hItem, GUI_FONT_32_ASCII);
TEXT_SetText(hItem, "BEEP:");
break;

case WM_NOTIFY_PARENT:
    Id      = WM_GetId(pMsg->hWinSrc);
    NCode   = pMsg->Data.v;

    switch (Id)
    {
        case ID_BUTTON_0: /* BUTTON_0 的通知代码, 控制 LED1 */
            hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_0);

            switch (NCode)
            {
                case WM_NOTIFICATION_CLICKED:
                    break;

                case WM_NOTIFICATION_RELEASED: /* 按钮被按下并释放 */
                    LED1_TOGGLE(); /* LED1 反转 */
                    ledflag = ~ledflag;
                    BUTTON_SetBitmapEx(hItem, 0, ledflag ?
                                        &buttonbmp_tab[1] :
                                        &buttonbmp_tab[0], 0, 0);
                    break;
            }
            break;

        case ID_BUTTON_1: /* BUTTON_1 的通知代码, 控制 BEEP */
            hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_1);

            switch (NCode)
            {
                case WM_NOTIFICATION_CLICKED:
                    break;
            }
    }

```



```

        case WM_NOTIFICATION_RELEASED:
            BEEP_TOGGLE(); /* 蜂鸣器反转 */
            beepflag = ~beepflag;
            BUTTON_SetBitmapEx(hItem, 0, beepflag ?
                                &buttonbmp_tab[1] :
                                &buttonbmp_tab[0], 0, 0);

            break;

    }
    break;

}
break;

default:
    WM_DefaultProc(pMsg);
    break;

}
}

```

ID_BUTTON_0 和 ID_BUTTON_0 控件都调用函数 BUTTON_SetBitmapEx()来获取位图，当 ID_BUTTON_0 和 ID_BUTTON_0 控件其中一个按下时，beepflag ? &buttonbmp_tab[1] :&buttonbmp_tab[0]三目运算会自动判断切换位图。

15.6.5 下载验证

编译并下载到开发板中，显示如下图所示：

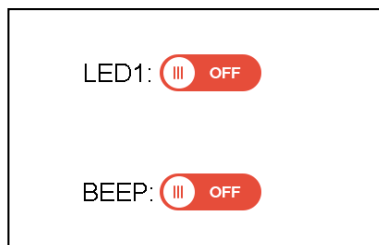


图 15.6.5.1 显示效果图

点击 LED1 和 BEEP 的开关，显示如下图所示，并且 LED1 点亮，蜂鸣器响。

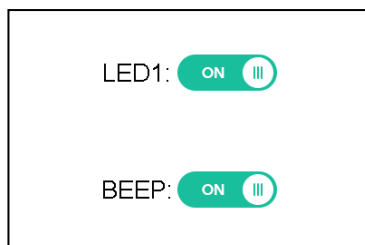


图 15.6.5.2 打开 LED1 和 BEEP

本例程所使用的 LCD 屏幕分辨率为 800*480，如果在其他低于此分辨率的 LCD 上运行此例程的话可能导致显示不完全，请自行修改控件大小和位置以适应所使用的 LCD 屏。

第十六章 CHECKBOX 控件

复选框是选择各种不同选项的最常用小工具之一，用户可选中或取消选中复选框，并且可一次选中任意个框。使用键盘界面时，被聚焦的复选框的状态可使用<SPACE>键切换，禁用的框显示为灰色。

本章分为以下几个部分：

16.1 通知代码

16.2 键盘反应

16.3 CHECKBOX 控件 API 函数

16.4 CHECKBOX 控件演示实验

16.1 通知代码

表 16.1.1 列举出了复选框小工具作为 WM_NOTIFY_PARENT 消息的一部分发送给其父窗口的消息类型：

消息	描述
WM_NOTIFICATION_CLICKED	复选框已被点击
WM_NOTIFICATION_RELEASED	复选框已被释放
WM_NOTIFICATION_MOVED_OUT	复选框已被点击，且指针已移出框并没有释放
WM_NOTIFICATION_VALUE_CHANGED	复选框的状态已改变

表 16.1.1 CHECKBOX 通知代码

16.2 键盘反应

如果 CHECKBOX 控件具有输入焦点，则它将对表 16.2.1 所列的各键做出反应：

按键	反应
GUI_KEY_SPACE	切换小工具的选中状态

表 16.2.1 CHECKBOX 的键盘反应

16.3 CHECKBOX 控件 API 函数

表 16.3.1 列举出了 CHECKBOX 控件 API 函数：

函数	描述
CHECKBOX_Check()	将复选框的状态设置为选中（弃用）
CHECKBOX_Create()	创建 CHECKBOX 小工具 (弃用)
CHECKBOX_CreateEx()	创建 CHECKBOX 小工具
CHECKBOX_CreateIndirect()	从资源表项创建 CHECKBOX 小工具
CHECKBOX_CreateUser()	使用额外字节作为用户数据创建 CHECKBOX 小工具
CHECKBOX_GetDefaultBkColor()	返回 CHECKBOX 小工具文本的默认背景
CHECKBOX_GetDefaultFont()	返回用于显示 CHECKBOX 小工具文本的默认字体
CHECKBOX_GetDefaultSpacing()	返回 CHECKBOX 小工具文本和框之间的默认间
CHECKBOX_GetDefaultTextAlign()	返回文本对齐方式
CHECKBOX_GetDefaultTextColor()	返回文本颜色
CHECKBOX_GetState()	返回复选框的当前状态
CHECKBOX_GetText()	返回复选框的文本

CHECKBOX_GetUserData()	检索使用 CHECKBOX_SetUserData 设置的数据
CHECKBOX_IsChecked()	返回复选框的当前状态(选中或未选中)
CHECKBOX_SetBkColor()	设置给定 CHECKBOX 小工具的背景色
CHECKBOX_SetBoxBkColor()	设置复选框区域的背景色
CHECKBOX_SetDefaultBkColor()	设置 CHECKBOX 小工具的默认背景色
CHECKBOX_SetDefaultFocusColor()	设置 CHECKBOX 小工具的默认焦点矩形颜色
CHECKBOX_SetDefaultFont()	设置用于显示 CHECKBOX 小工具文本的默认字体
CHECKBOX_SetDefaultImage()	设置复选框被选中后要显示的默认图像
CHECKBOX_SetDefaultSpacing()	设置 CHECKBOX 小工具文本和框之间的默认间距
CHECKBOX_SetDefaultTextAlign()	设置用于显示复选框文本的默认对齐方式
CHECKBOX_SetDefaultTextColor()	设置文本颜色
CHECKBOX_SetFocusColor()	设置聚焦框的颜色
CHECKBOX_SetFont()	设置复选框字体
CHECKBOX_SetImage()	设置复选框被选中后要显示的图像
CHECKBOX_SetNumStates()	设置复选框的可能状态的数量（2 或 3）
CHECKBOX_SetSpacing()	设置框和复选框文本之间的间距
CHECKBOX_SetState()	设置 CHECKBOX 小工具的状态
CHECKBOX_SetText()	设置 CHECKBOX 小工具的文本
CHECKBOX_SetTextAlign()	设置用于显示 CHECKBOX 小工具文本的对齐方式
CHECKBOX_SetTextColor()	设置用于显示 CHECKBOX 小工具文本的颜色
CHECKBOX_SetUserData()	设置 CHECKBOX 小工具的额外数据
CHECKBOX_Uncheck()	将复选框的状态设置为未选中（弃用）

表 16.3.1 CHECKBOX 控件 API 函数

表 16.3.1 是 CHECKBOX 控件函数相关描述。接下来，我们来讲解 CHECKBOX 控件常用的函数。

(1) CHECKBOX_CreateEx 函数

指定位置创建指定尺寸的 CHECKBOX 控件，该函数原型如下源码所示：

```
CHECKBOX_Handle CHECKBOX_CreateEx(int x0, int y0, int xSize, int ySize,
                                   WM_HWIN hParent,
                                   int WinFlags,
                                   int ExFlags, int Id);
```

该函数的相关形参如下表所示：

参数	描述
x0	控件最左侧的像素（在父坐标中）
y0	控件最顶端的像素（在父坐标中）
xSize	控件的水平尺寸（以像素为单位）
ySize	控件的垂直尺寸（以像素为单位）
hParent	父窗口的句柄
WinFlags	窗口创建标记。为了使控件立即可见，通常为 WM_CF_SHOW
ExFlags	未使用，保留供日后使用
Id	控件的窗口 ID

表 16.3.2 CHECKBOX_CreateEx 函数相关形参描述

返回值：已创建 CHECKBOX 控件的句柄；函数失败时为 0。

注意：如果参数 xsize 或 ysize 为 0，则将使用默认复选标记位图的尺寸（11x11 像素）加上效果尺寸作为复选框的默认尺寸。如果所需的复选框尺寸不同于默认尺寸，则可使用函数

CHECKBOX_SetImage 来设置用户定义的复选标记图像；如果要与该控件一同显示复选框文本，则尺寸应足够大到可显示“框+文本+框和文本之间的间距”。

(2) CHECKBOX_SetBkColor 函数

设置用于显示复选框背景的背景色，该函数原型如下源码所示：

```
void CHECKBOX_SetBkColor(CHECKBOX_Handle hObj, GUI_COLOR Color);
```

该函数的相关形参如下表所示：

参数	描述
hObj	CHECKBOX 控件的句柄
Color	用于绘制背景的颜色

表 16.3.3 CHECKBOX_SetBkColor 函数相关形参描述

注意：如果复选框应在透明模式下操作，则应使用 GUI_INVALID_COLOR。

(3) CHECKBOX_SetBoxBkColor 函数

设置框区域的背景色，该函数原型如下源码所示：

```
GUI_COLOR CHECKBOX_SetBoxBkColor(CHECKBOX_Handle hObj, GUI_COLOR Color, int Index);
```

该函数的相关形参如下表所示：

参数	描述	
hObj	CHECKBOX 控件的句柄	
Color	用于绘制背景的颜色或者在透明模式下操作的 GUI_INVALID_COLOR	
Index	参数 Index 的允许值	
	CHECKBOX_CI_DISABLED	用于禁用状态的背景色
	CHECKBOX_CI_ENABLED	用于启用状态的背景色

表 16.3.4 CHECKBOX_SetBoxBkColor 函数相关形参描述

返回值：参数 Index 的允许值

(4) CHECKBOX_SetState 函数

设置给定复选框控件的新状态，该函数原型如下源码所示：

```
void CHECKBOX_SetState(CHECKBOX_Handle hObj, unsigned State);
```

该函数的相关形参如下表所示：

参数	描述	
hObj	CHECKBOX 控件的句柄	
State	参数 State 的允许值	
	0	取消选中
	1	选中
	2	第三种状态

表 16.3.5 CHECKBOX_SetState 函数相关形参描述

(5) CHECKBOX_SetText 函数

设置复选框旁显示的可选文本，该函数原型如下源码所示：

```
void CHECKBOX_SetText(CHECKBOX_Handle hObj, const char * pText);
```

该函数的相关形参如下表所示：

参数	描述
hObj	CHECKBOX 控件的句柄
pText	指向要显示在复选框旁边的文本的指针

表 16.3.6 CHECKBOX_SetText 函数相关形参描述

注意：点击复选框旁文本的作用与点击框内效果相同。

16.4 CHECKBOX 控件演示实验

16.4.1 硬件设计

1. 例程功能

本实验主要讲解 CHECKBOX 小工具的使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数主要创建一个 window 界面并在其上面创建多个 CHECKBOX 小工具、“OK”和“Cancel”按键小工具，这些 CHECKBOX 小工具有些可以被选中，而有些被禁用了。当用户按下“OK”以及“Cancel”按键时，系统会调用 GUI_EndDialog 函数关闭 window 窗口并开启 3 秒的定时器，如果定时器时间到来，则系统重新打开 window 窗口。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 17 CHECKBOX 小工具》。

16.4.2 软件设计

16.4.2.1 程序流程图

本实验的程序流程图，如下图所示：

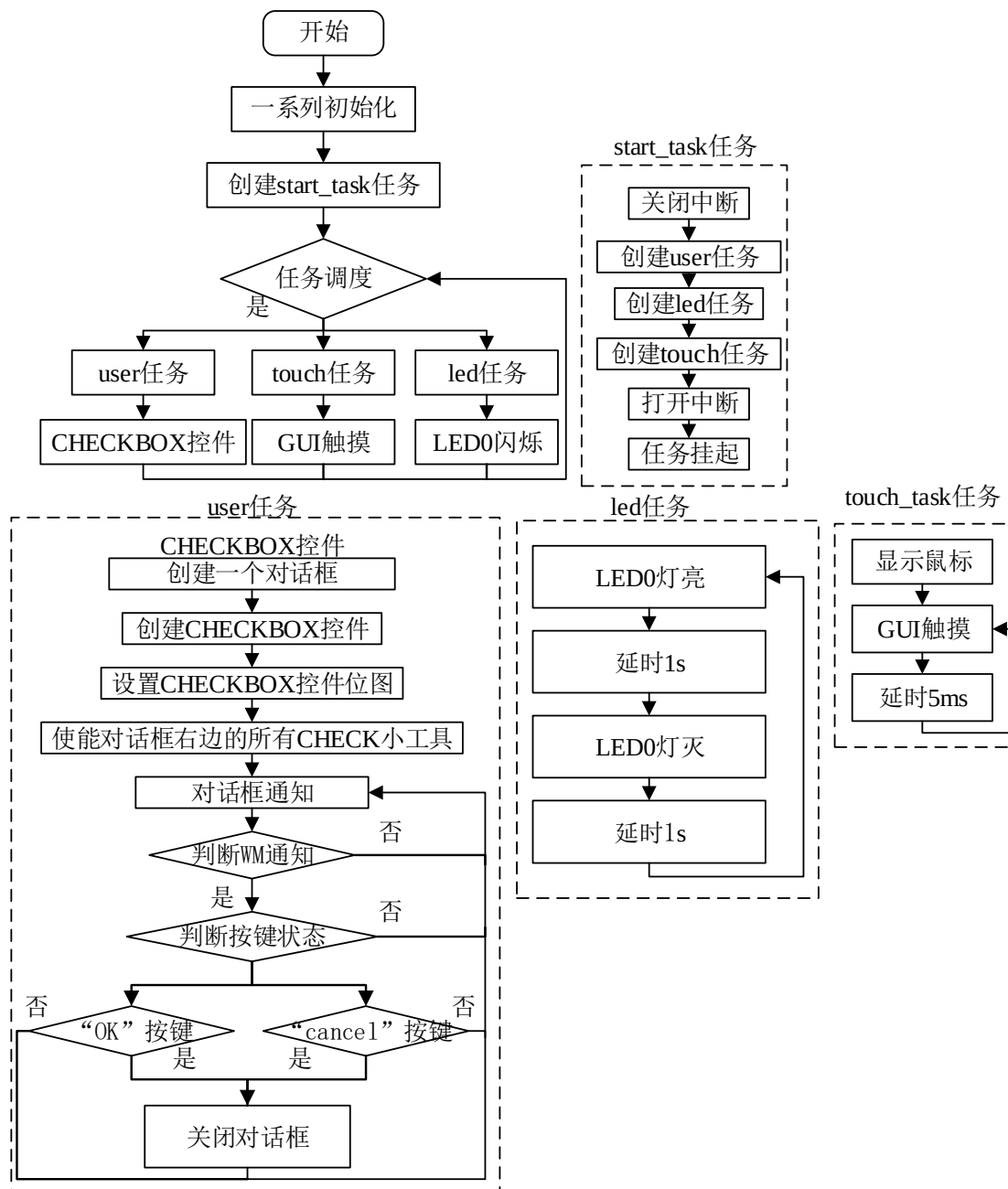


图 16.4.2.1.1 CHECKBOX 控件实验流程图

16.4.2.2 程序解析

Segger 官方提供了一个关于 CHECKBOX 控件的演示例程，这里我们将这个例程移植到开发板中，学习一下 CHECKBOX 控件的使用方法，本实验完整工程为：**emWin 实验 17 CHECKBOX 小工具**。在 emWin_Demo 文件夹里面有 5 个文件，分别为：BitmapBar.c、BitmapCheckXL.c、BitmapCheckXXL.c、emwin_demo.c 和 emwin_demo.h。其中前三个里面定义了本实验需要用到的位图，emwin_demo.c 文件是我们本次实验的重点，打开 emwin_demo.c 文件，程序源码如下：

(1) GUIBulider 生成的资源表

```
static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] =
```

```
{
{FRAMEWIN_CreateIndirect,"Checkbox sample",0,0,10,300,220,FRAMEWIN_CF_MOVEABLE},
{ TEXT_CreateIndirect,      "Enabled:",          0,          5,  10, 120,  0 },
{ CHECKBOX_CreateIndirect, 0,          GUI_ID_CHECK0,    5,  30, 120,  0 },
{ CHECKBOX_CreateIndirect, 0,          GUI_ID_CHECK1,    5,  60, 120,  0 },
{ CHECKBOX_CreateIndirect, 0,          GUI_ID_CHECK2,    5,  90, 120, 20 },
{ CHECKBOX_CreateIndirect, 0,          GUI_ID_CHECK3,    5, 125, 120, 26 },
{ TEXT_CreateIndirect,      "Disabled:",          0,          150, 10, 120,  0 },
{ CHECKBOX_CreateIndirect, 0,          GUI_ID_CHECK4,   150,  30, 120,  0 },
{ CHECKBOX_CreateIndirect, 0,          GUI_ID_CHECK5,   150,  60, 120,  0 },
{ CHECKBOX_CreateIndirect, 0,          GUI_ID_CHECK6,   150,  90, 120, 26 },
{ CHECKBOX_CreateIndirect, 0,          GUI_ID_CHECK7,   150, 125, 120, 26 },
{ BUTTON_CreateIndirect,    "OK",          GUI_ID_OK,       10, 165,  80, 30 },
{ BUTTON_CreateIndirect,    "Cancel", GUI_ID_CANCEL,    200, 165,  80, 30 },
};
```

(2) 建立对话框

```
/**
 * @brief      emwin_main(emwin 入口)
 * @param      无
 * @return     无
 */
void emwin_main (void)
{
    GUI_CURSOR_Show();
#ifdef GUI_SUPPORT_MEMDEV
    WM_SetCreateFlags(WM_CF_MEMDEV);
#endif
    WM_SetDesktopColor(GUI_BLACK);
    while (1)
    {
        GUI_ExecDialogBox(_aDialogCreate, GUI_COUNTOF(_aDialogCreate),
                          &_cbCallback, 0, 0, 0);

        GUI_Delay(500);
    }
}
```

(3) 对话框的回调函数

```
/**
 * @brief      回调函数
 * @param      pMsg:消息
 * @return     无
 */
static void _cbCallback(WM_MESSAGE *pMsg)
{
}
```

```

int i;
int NCode, Id;
WM_HWIN hDlg, hItem;
hDlg = pMsg->hWin;

switch (pMsg->MsgId)
{
    case WM_INIT_DIALOG:
        /* 获取 CHECKBOX 的句柄 */
        hItem = WM_GetDialogItem(hDlg, GUI_ID_CHECK0);

        /* WM_INIT_DIALOG 消息下，用来初始化 CHECKBOX 控件，
           因为一共有 8 个 CHECKBOX，所以要循环设置 8 次 */
        for (i = 0; i < 8; i++)
        {
            int Index = i % 4;
            /* 根据 i 的不同获取各个 CHECKBOX 控件的句柄 */
            hItem = WM_GetDialogItem(hDlg, GUI_ID_CHECK0 + i);
            CHECKBOX_SetText(hItem, _apLabel[Index]);

            switch (Index)
            {
                /* 一共有四排 CHECKBOX 小工具，每排两个，8 个
                   CHECKBOX 的排序是按照从上到下，从左到右 */
                case 1:
                    /* 调用 CHECKBOX_SetNumStates 函数设置复选框的可选状态的数量，
                       默认情况下复选框有 2 种状态，选中 (1) 和取消选中 (0)，
                       使用函数 CHECKBOX_SetNumStates() 可将可能状态增加到 3，
                       如果复选框处于第三种状态，则该函数返回 2。 */
                    CHECKBOX_SetNumStates(hItem, 3);
                    CHECKBOX_SetImage(hItem, &_abmBar[0],
                                      CHECKBOX_BI_INACTIV_3STATE);
                    CHECKBOX_SetImage(hItem, &_abmBar[1],
                                      CHECKBOX_BI_ACTIV_3STATE);
                    CHECKBOX_SetState(hItem, 2);
                    break;

                case 2:
                    CHECKBOX_SetState(hItem, 1);
                    CHECKBOX_SetImage(hItem, &_abmXL[0],
                                      CHECKBOX_BI_INACTIV_CHECKED);
                    CHECKBOX_SetImage(hItem, &_abmXL[1],
                                      CHECKBOX_BI_ACTIV_CHECKED);
            }
        }
    }

```



```

        CHECKBOX_SetFont(hItem, &GUI_FontComic18B_ASCII);
        break;

    case 3:
        CHECKBOX_SetState(hItem, 1);
        CHECKBOX_SetImage(hItem, &_abmXXL[0],
                           CHECKBOX_BI_INACTIV_CHECKED);
        CHECKBOX_SetImage(hItem, &_abmXXL[1],
                           CHECKBOX_BI_ACTIV_CHECKED);
        CHECKBOX_SetFont(hItem, &GUI_FontComic24B_ASCII);
        break;

    }

    if (i >= 4)
    {
        WM_DisableWindow(hItem); /* 能对话框右边的所有 CHECK 小工具 */
    }

    break;

case WM_NOTIFY_PARENT:
    Id = WM_GetId(pMsg->hWinSrc); /* 小工具 ID */
    NCode = pMsg->Data.v; /* 通知代码 */
    switch (NCode)
    {
        case WM_NOTIFICATION_RELEASED:
            if (Id == GUI_ID_OK)
            {
                GUI_EndDialog(hDlg, 0); /* 关闭对话框 */
            }

            if (Id == GUI_ID_CANCEL)
            {
                GUI_EndDialog(hDlg, 1); /* 关闭对话框 */
            }

            break;

        }

        break;
default:
    WM_DefaultProc(pMsg);
    }
}

```

16.4.3 下载验证

编译并下载到开发板中，LCD 显示如下图所示。

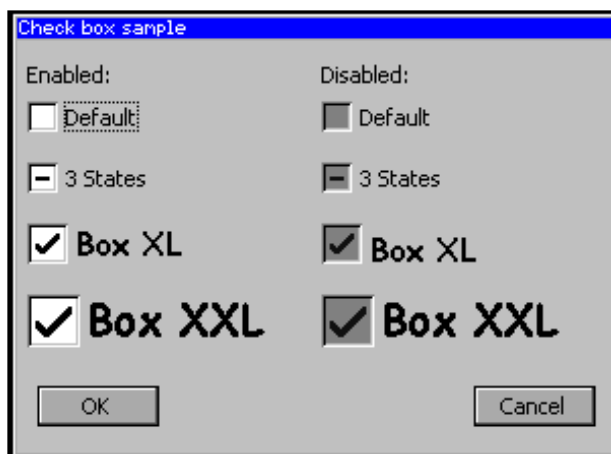


图 16.4.3.1 LCD 显示结果

点按 LCD 触摸屏上复选框，观察结果和我们设置的是否一致。

第十七章 DROPDOWN 控件

DROPDOWN 小工具用于从具有若干栏的列表中选择元素，它以非打开状态显示当前选择的项目，如果用户打开 DROPDOWN 小工具，就会出现一个选择新项目的 LISTBOX。

本章分为以下几个部分：

17.1 DROPDOWN 控件简介

17.2 通知代码

17.3 键盘反应

17.4 DROPDOWN 控件 API 函数

17.5 DROPDOWN 控件演示实验

17.1 DROPDOWN 控件简介

DROPDOWN 控件在打开和关闭情况下分别如图 17.1.1 和 17.1.2 所示：

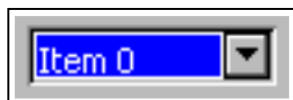


图 17.1.1 DROPDOWN 关闭状态

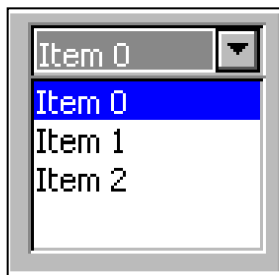


图 17.1.2 DROPDOWN 打开状态

DROPDOWN 可以换肤，换肤之后的小工具如下图所示：

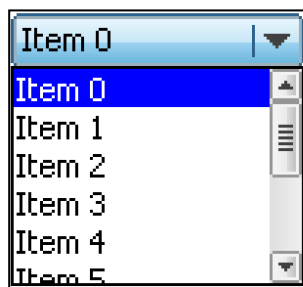


图 17.1.3 换肤后的 DROPDOWN 控件

17.2 通知代码

表 17.2.1 列举出了 DROPDOWN 小工具作为 WM_NOTIFY_PARENT 消息的一部分发送给其父窗口的消息类型：

消息	描述
WM_NOTIFICATION_CLICKED	已单击小工具
WM_NOTIFICATION_RELEASED	已释放小工具

WM_NOTIFICATION_MOVED_OUT	已单击小工具，并且指针已移出小工具，但没有释放
WM_NOTIFICATION_SCROLL_CHANGED	打开的下拉列表小工具的可选滚动条的滚动
WM_NOTIFICATION_SEL_CHANGED	下拉列表的选择已更改

表 17.2.1 DROPDOWN 通知代码

17.3 键盘反应

如果 DROPDOWN 控件具有输入焦点，则它将对表 16.3.1 所列的各键做出反应：

按键	反应
GUI_KEY_ENTER	从打开的下拉列表中选择项目，然后关闭列表
GUI_KEY_SPACE	打开下拉列表

表 16.3.1 DROPDOWN 键盘反应

17.4 DROPDOWN 控件 API 函数

表 17.4.1 列举出了 DROPDOWN 控件 API 函数：

函数	描述
DROPDOWN_AddString()	给 DROPDOWN 列表添加元素
DROPDOWN_Collapse()	关闭下拉列表
DROPDOWN_Create()	创建 DROPDOWN 小工具
DROPDOWN_CreateEx()	创建 DROPDOWN 小工具
DROPDOWN_CreateIndirect()	从资源表项创建 DROPDOWN 小工具
DROPDOWN_CreateUser()	额外字节作为用户数据创建 DROPDOWN 小工具
DROPDOWN_DecSel()	减小选定范围
DROPDOWN_DecSelExp()	递减处于展开状态的选取项
DROPDOWN_DeleteItem()	删除 DROPDOWN 列表的项目
DROPDOWN_Expand()	打开下拉列表
DROPDOWN_GetDefaultFont()	返回用于创建 DROPDOWN 小工具的默认字体
DROPDOWN_GetItemDisabled()	返回给定项目的状态
DROPDOWN_GetListbox()	返回处于展开状态的附加 DROPDOWN 的句柄
DROPDOWN_GetNumItems()	返回下拉列表中项目的数量
DROPDOWN_GetSel()	返回当前选定元素的数量
DROPDOWN_GetSelExp()	返回处于展开状态的当前选定元素的数量
DROPDOWN_GetUserData()	检索使用 DROPDOWN_SetUserData() 设置的数据
DROPDOWN_IncSel()	增加选定范围
DROPDOWN_IncSelExp()	递增处于展开状态的选取项
DROPDOWN_SetAutoScroll()	启用在下拉列表中自动使用滚动条
DROPDOWN_SetBkColor()	设置背景颜色
DROPDOWN_SetColor()	设置 DROPDOWN 小工具的箭头和按钮的颜色
DROPDOWN_SetDefaultColor()	DROPDOWN 小工具的箭头和按钮的默认颜色
DROPDOWN_SetDefaultFont()	设置 DROPDOWN 小工具的默认字体
DROPDOWN_SetDefaultScrollbarColor()	设置下拉列表中可选滚动条的默认颜色
DROPDOWN_SetFont()	设置给定 DROPDOWN 小工具的字体
DROPDOWN_SetItemDisabled()	设置给定项目的状态
DROPDOWN_SetItemSpacing()	设置下拉列表中各项目之间的间距
DROPDOWN_SetScrollbarColor()	设置下拉列表中滚动条的颜色
DROPDOWN_SetSel()	设置当前选定内容

DROPDOWN_SetSelExp()	设置处于展开状态的当前选择
DROPDOWN_SetTextAlign()	设置用于显示关闭状态下拉文本的对齐方式
DROPDOWN_SetTextColor()	设置给定 DROPDOWN 小工具的文本颜色
DROPDOWN_SetTextHeight()	设置用于显示关闭状态下拉文本的矩形的高度
DROPDOWN_SetUserData()	设置 DROPDOWN 小工具的额外数据

表 17.4.1 DROPDOWN 控件 API 函数

表 17.4.1 是 DROPDOWN 控件函数相关描述。接下来，我们来讲解 DROPDOWN 控件常用的函数。

(1) DROPDOWN_AddString 函数

DROPDOWN 控件添加新元素，该函数原型如下源码所示：

```
void DROPDOWN_AddString( DROPDOWN_Handle hObj, const char *s);
```

该函数的相关形参如下表所示：

参数	描述
hObj	DROPDOWN 控件的句柄
s	指向要添加的字符串的指针

表 17.4.2 DROPDOWN_AddString 函数相关形参描述

(2) DROPDOWN_Collapse 函数

关闭 DROPDOWN 控件的下拉列表，该函数原型如下源码所示：

```
void DROPDOWN_Collapse(DROPDOWN_Handle hObj);
```

该函数的相关形参如下表所示：

参数	描述
hObj	DROPDOWN 控件的句柄

表 17.4.3 DROPDOWN_Collapse 函数相关形参描述

(3) DROPDOWN_CreateEx 函数

在指定位置创建指定尺寸的 DROPDOWN 控件，该函数原型如下源码所示：

```
DROPDOWN_Handle DROPDOWN_CreateEx( int x0, int y0, int xSize, int ySize,
                                     WM_HWIN hParent,
                                     int WinFlags,
                                     int ExFlags,
                                     int Id);
```

该函数的相关形参如下表所示：

参数	描述
x0	控件最左侧的像素（在父坐标中）
y0	控件最顶端的像素（在父坐标中）
xSize	控件的水平尺寸（以像素为单位）
ySize	处于打开状态的控件的垂直尺寸（单位：像素）
hParent	父窗口的句柄。如果为 0，则新的 DROPDOWN 控件将是桌面（顶级窗口）的子项
WinFlags	窗口创建标记。为了使控件立即可见，通常为 WM_CF_SHOW
ExFlags	ExFlags 选项
	0
	DROPDOWN_CF_AUTOSCROLLBAR
	DROPDOWN_CF_UP

Id	控件的窗口 ID。
----	-----------

表 17.4.4 DROPDOWN_Collapse 函数相关形参描述

返回值：已创建的 DROPDOWN 控件的句柄；函数失败时为 0。

(4) DROPDOWN_DeleteItem 函数

删除 DROPDOWN 控件的给定项目，该函数原型如下源码所示：

```
void DROPDOWN_DeleteItem(DROPDOWN_Handle hObj, unsigned int Index);
```

该函数的相关形参如下表所示：

参数	描述
hObj	DROPDOWN 控件的句柄
Index	DROPDOWN 控件的句柄

表 17.4.5 DROPDOWN_DeleteItem 函数相关形参描述

注意：DROPDOWN 控件的句柄。

(5) DROPDOWN_Expand 函数

删除 DROPDOWN 控件的给定项目，该函数原型如下源码所示：

```
void DROPDOWN_Expand(DROPDOWN_Handle hObj);
```

该函数的相关形参如下表所示：

参数	描述
hObj	DROPDOWN 控件的句柄

表 17.4.6 DROPDOWN_Expand 函数相关形参描述

(6) DROPDOWN_SetAutoScroll 函数

启用在 DROPDOWN 控件中自动使用垂直滚动条，该函数原型如下源码所示：

```
void DROPDOWN_SetAutoScroll(DROPDOWN_Handle hObj, int OnOff);
```

该函数的相关形参如下表所示：

参数	描述
hObj	DROPDOWN 控件的句柄
OnOff	参数 OnOff 的允许值
	0 禁用自动使用滚动条
	1 启用自动使用滚动条

表 17.4.7 DROPDOWN_Expand 函数相关形参描述

注意：如果启用，DROPDOWN 控件将检查是否所有元素都能放入列表框中。如果未启用，则将添加垂直滚动条。

(7) DROPDOWN_SetBkColor 函数

设置给定 DROPDOWN 控件的背景色，该函数原型如下源码所示：

```
void DROPDOWN_SetBkColor(DROPDOWN_Handle hObj, unsigned int Index,
    GUI_COLOR Color);
```

该函数的相关形参如下表所示：

参数	描述
hObj	DROPDOWN 控件的句柄
Index	参数 Index 的允许值
	DROPDOWN_CI_UNSEL 未选取的元素
	DROPDOWN_CI_SEL 选取的元素，无焦点
	DROPDOWN_CI_SELFOCUS 选取的元素，有焦点
Color	按钮颜色

表 17.4.8 DROPDOWN_Expand 函数相关形参描述

(8) DROPDOWN_SetColor 函数

设置给定 DROPDOWN 控件的按钮或箭头的颜色，该函数原型如下源码所示：

```
void DROPDOWN_SetColor(DROPDOWN_Handle hObj, unsigned int Index,
                        GUI_COLOR Color);
```

该函数的相关形参如下表所示：

参数	描述	
hObj	DROPDOWN 控件的句柄	
Index	参数 Index 的允许值	
	DROPDOWN_CI_ARROW	按钮内小箭头的颜色
	DROPDOWN_CI_BUTTON	按钮颜色
Color	按钮颜色	

表 17.4.9 DROPDOWN_SetColor 函数相关形参描述

(9) DROPDOWN_SetListHeight 函数

设置用于 DROPDOWN 控件展开列表的像素高度，该函数原型如下源码所示：

```
int DROPDOWN_SetListHeight(DROPDOWN_Handle hObj, unsigned Height);
```

该函数的相关形参如下表所示：

参数	描述
hObj	DROPDOWN 控件的句柄
Height	要使用的高度

表 17.4.10 DROPDOWN_SetListHeight 函数相关形参描述

(10) DROPDOWN_SetScrollbarWidth 函数

设置给定 DROPDOWN 控件的列表使用的滚动条的宽度，该函数原型如下源码所示：

```
void DROPDOWN_SetScrollbarWidth(DROPDOWN_Handle hObj, unsigned Width);
```

该函数的相关形参如下表所示：

参数	描述
hObj	DROPDOWN 控件的句柄
Width	给定 DROPDOWN 控件的下拉列表使用的滚动条的宽度

表 17.4.11 DROPDOWN_SetScrollbarWidth 函数相关形参描述

(11) DROPDOWN_SetItemSpacing 函数

设置 DROPDOWN 控件项目下面的额外间距，该函数原型如下源码所示：

```
void DROPDOWN_SetItemSpacing(DROPDOWN_Handle hObj, unsigned Value);
```

该函数的相关形参如下表所示：

参数	描述
hObj	DROPDOWN 控件的句柄
Value	用于 DROPDOWN 控件项目之间额外间距的像素数量

表 17.4.12 DROPDOWN_SetItemSpacing 函数相关形参描述

17.5 DROPDOWN 控件演示实验

17.5.1 硬件设计

1. 例程功能

本实验主要讲解 DROPDOWN 小工具的使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数主要创建一个 window 界面并在其上面创建一个 DROPDOWN 小工具，在该控件上添加多个选项，如"China"、"Cambodia"、"Japan"、"Australien"和"Canada"等选项。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 18 DROPDOWN 小工具》。

17.5.2 软件设计

17.5.2.1 程序流程图

本实验的程序流程图，如下图所示：

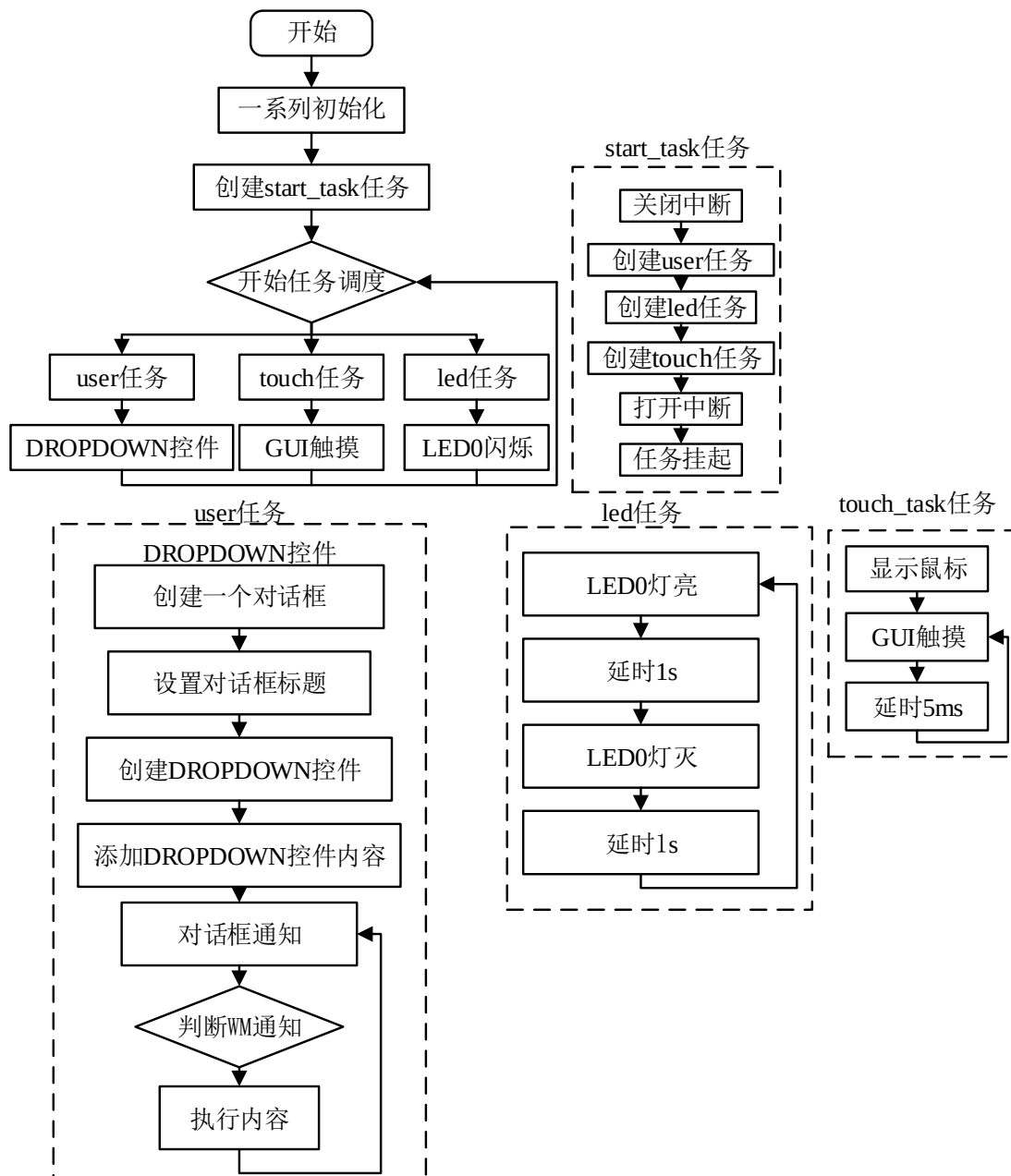


图 17.5.2.1.1 DROPDOWN 控件实验流程图

17.5.2.2 程序解析

本实验完整工程为：**emWin 实验 18 DROPDOWN 小工具**，打开 emwin_demo.c，程序源码如下：

(1) GUIBulider 生成的资源表

```
#define ID_FRAMEWIN_0 (GUI_ID_USER + 0x02)
#define ID_DROPDOWN_0 (GUI_ID_USER + 0x04)
/* 对话框资源表 */
static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] =
{
    { FRAMEWIN_CreateIndirect, "Framewin", ID_FRAMEWIN_0, 0, 0, 320, 240,
      FRAMEWIN_CF_MOVEABLE, 0x64, 0 },
    { DROPDOWN_CreateIndirect, "Dropdown", ID_DROPDOWN_0, 88, 24, 109, 60, 0, 0x0, 0 },
};
```

资源表中，创建了框架窗口和 DROPDOWN 控件。

(2) 建立对话框

```
/**
 * @brief      emwin_main(emwin 入口)
 * @param      无
 * @return     无
 */
void emwin_main (void)
{
    WM_HWIN hWin;
    hWin = GUI_CreateDialogBox(_aDialogCreate, GUI_COUNTOF(_aDialogCreate),
                              _cbDialog, WM_HBKWIN, 0, 0);

    while (1)
    {
        GUI_Delay(100);
    }
}
```

此函数主要调用函数 GUI_CreateDialogBox 根据资源表和回调函数创建一个对话框。对话框的回调函数如下所示：

(3) 对话框的回调函数

```
/**
 * @brief      对话框回调函数
 * @param      无
 * @return     无
 */
static void _cbDialog(WM_MESSAGE *pMsg)
{
    WM_HWIN hItem;
    int      NCode;
```

```

int      Id;

switch (pMsg->MsgId)
{
    case WM_INIT_DIALOG:
        hItem = pMsg->hWin;
        /* 初始化 FRAMEWIN */
        FRAMEWIN_SetText(hItem, "DROPDOWN ");
        FRAMEWIN_SetFont(hItem, GUI_FONT_16_ASCII);
        FRAMEWIN_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
        /* 初始化 DROPDOWN */
        hItem = WM_GetDialogItem(pMsg->hWin, ID_DROPDOWN_0);
        DROPDOWN_SetFont(hItem, GUI_FONT_13H_ASCII);
        DROPDOWN_SetAutoScroll(hItem, 1); /* 启用自动使用滚动条 */
        DROPDOWN_AddString(hItem, "China"); /* 向 DROPDOWN 控件中添加新元素 */
        DROPDOWN_AddString(hItem, "Cambodia");
        DROPDOWN_AddString(hItem, "Japan");
        DROPDOWN_AddString(hItem, "Australien");
        DROPDOWN_AddString(hItem, "Canada");
        break;

    case WM_NOTIFY_PARENT:
        Id      = WM_GetId(pMsg->hWinSrc);
        NCode = pMsg->Data.v;

        switch (Id)
        {
            case ID_DROPDOWN_0: /* DROPDOWN 通知代码 */
                switch (NCode)
                {
                    case WM_NOTIFICATION_CLICKED:
                        break;

                    case WM_NOTIFICATION_RELEASED:
                        break;

                    case WM_NOTIFICATION_SEL_CHANGED:
                        break;
                }
                break;
        }
        break;
}

```

```
default:
    WM_DefaultProc(pMsg);
    break;
}
}
```

本例程比较简单，Segger 官方提供了一个 DROPDOWN 的例程：WIDGET_Dropdown.c，大家可以使用模拟器仿真运行一个这个例程，这个例程中详细的展示了 DROPDOWN 的 API 函数的使用。

17.5.2 下载验证

编译并下载到开发板中，如下图所示：



图 17.5.2.1 LCD 显示结果

第十八章 EDIT 控件

EDIT 控件通常用来作为输入文本的主要用户界面，也可使用编辑字段以二进制、十进制或十六进制模式输入值。

本章分为以下几个部分：

- 18.1 EDIT 控件简介
- 18.2 通知代码
- 18.3 键盘反应
- 18.4 EDIT 空间 API 函数
- 18.5 EDIT 控件演示实验

18.1 EDIT 控件简介

EDIT 控件如图 18.1.1 所示：



图 18.1.1 EDIT 控件

该控件主要作为文本输入的小工具，例如键盘控件所输入的字符可以在该控件显示。

18.2 通知代码

表 18.2.1 列举出了 EDIT 小工具作为 WM_NOTIFY_PARENT 消息的一部分发送给其父窗口的消息类型：

消息	描述
WM_NOTIFICATION_CLICKED	已单击小工具
WM_NOTIFICATION_RELEASED	已释放小工具
WM_NOTIFICATION_MOVED_OUT	已单击小工具，并且移出小工具，但没有释放
WM_NOTIFICATION_VALUE_CHANGED	编辑小工具的值(内容)已更改

表 18.2.1 EDIT 控件通知代码

18.3 键盘反应

如果小工具具有输入焦点，则它将对下列各键做出反应，如下表所示：

按键	反应
GUI_KEY_UP	上升当前字符。如果当前字符(光标下的字符)为“A”，则变为“B”
GUI_KEY_DOWN	下降当前字符。例如，如果当前字符为为“B”，则变为“A”
GUI_KEY_RIGHT	将光标向右移动一个字符
GUI_KEY_LEFT	将光标向左移动一个字符
GUI_KEY_BACKSPACE	删除光标前的字符
GUI_KEY_DELETE	删除当前字符
GUI_KEY_INSERT	此键在 GUI_EDIT_MODE_OVERWRITE

表 18.3.1 EDIT 控件按键反应

18.4 EDIT 控件 API 函数

表 18.4.1 按字母顺序列出了可用的 EDIT 控件相关例程：

函数	描述
EDIT_AddKey()	添加按键输入
EDIT_Create()	创建 EDIT 小工具
EDIT_CreateAsChild()	创建 EDIT 小工具，作为子窗口
EDIT_CreateEx()	创建 EDIT 小工具
EDIT_CreateIndirect()	从资源表项创建 EDIT 小工具
EDIT_CreateUser()	使用额外字节作为用户数据创建 EDIT 小工具
EDIT_EnaBleBlink()	启用/禁用闪烁光
EDIT_GetCursorCharPos()	返回光标位置的字符编号
EDIT_GetCursorPixelPos()	返回光标的像素位置
EDIT_GetDefaultBkColor()	返回默认背景颜色
EDIT_GetDefaultFont()	返回默认的字体
EDIT_GetDefaultTextAlign()	返回默认的文本对齐方式
EDIT_GetDefaultTextColor()	返回默认文本颜色
EDIT_GetFloatValue()	按浮点值返回当前值
EDIT_GetNumChars()	返回给定编辑小工具的字符数
EDIT_GetText()	获取用户输入
EDIT_GetUserData()	检索用 EDIT_SetUserData 设置的数据
EDIT_GetValue()	返回当前值
EDIT_SetBinMode()	启用二进制编辑模式
EDIT_SetBkColor()	设置编辑字段的背景色
EDIT_SetCursorAtChar()	将编辑小工具光标设置到指定的字符位置
EDIT_SetCursorAtPixel()	将编辑小工具光标设置到指定的像素位置
EDIT_SetDecMode()	启用十进制编辑模式
EDIT_SetDefaultBkColor()	设置默认背景色
EDIT_SetDefaultFont()	设置用于编辑字段的默认字体
EDIT_SetDefaultTextAlign()	设置编辑字段的默认文本对齐方式
EDIT_SetDefaultTextColor()	设置默认文本颜色
EDIT_SetFloatMode()	启用浮点编辑模式
EDIT_SetFloatValue()	设置使用浮点编辑模式时的浮点值
EDIT_SetFont()	选择文本的字体
EDIT_SetHexMode()	启用十六进制编辑模式
EDIT_SetInsertMode()	启用或禁用插入模式
EDIT_SetMaxLen()	设置编辑字段的最大字符数
EDIT_SetpfAddKeyEx()	设置添加字符时要调用的函数
EDIT_SetSel()	设置当前选定内容
EDIT_SetText()	设置文本
EDIT_SetTextAlign()	设置编辑字段的文本对齐方式
EDIT_SetTextColor()	设置文本的颜色
EDIT_SetTextMode()	将小工具的编辑模式设置回文本模式
EDIT_SetValue()	设置当前值
EDIT_SetUlongMode()	启用不带符号的长十进制编辑模式
EDIT_SetUserData()	设置 EDIT 小工具的额外数据
GUI_EditBin()	编辑当前光标位置处的二进制值
GUI_EditDec()	编辑当前光标位置处的十进制值

GUI_EditHex()	编辑当前光标位置处的十六进制值
GUI_EditString()	编辑当前光标位置处的字符串

表 18.4.1 EDIT 控件 API 函数

表 18.4.1 是 EDIT 控件函数相关描述。接下来，我们来讲解 EDIT 控件常用的函数。

(1) EDIT_AddKey 函数

指定的 EDIT 控件添加用户输入，该函数原型如下源码所示：

```
void EDIT_AddKey(EDIT_Handle hObj, int Key);
```

该函数的相关形参如下表所示：

参数	描述
hObj	EDIT 控件的句柄
Key	要添加的字符

表 18.4.2 EDIT_AddKey 函数相关形参描述

注意：指定字符被添加到 EDIT 控件的用户输入中。如果要清除最后一个字符，可使用键 GUI_KEY_BACKSPACE。如果已达到最大字符数，则不会添加其他字符。

(2) EDIT_CreateAsChild 函数

创建 EDIT 控件，该控件在窗口中作为子窗口，该函数原型如下源码所示：

```
EDIT_Handle EDIT_CreateAsChild(int x0, int y0, int xSize, int ySize,
                               WM_HWIN hParent, int Id,
                               int Flags, int MaxLen);
```

该函数的相关形参如下表所示：

参数	描述
x0	编辑框控件相对于父窗口的 X 位置
y0	编辑框控件相对于父窗口的 Y 位置
xsize	编辑框控件的水平尺寸（单位：像素）
ysize	编辑框控件的垂直尺寸（单位：像素）
hParent	父窗口的句柄
Id	控件的窗口 ID
Flags	通常为 WM_CF_SHOW
MaxLen	最大字符数

表 18.4.3 EDIT_CreateAsChild 函数相关形参描述

返回值：已创建的 EDIT 控件的句柄；函数失败时为 0。

(3) EDIT_CreateEx 函数

在指定位置创建指定尺寸的 EDIT 控件，作为子窗口，该函数原型如下源码所示：

```
EDIT_Handle EDIT_CreateEx(int x0, int y0,
                          int xSize, int ySize,
                          WM_HWIN hParent, int WinFlags,
                          int ExFlags, int Id, int MaxLen);
```

该函数的相关形参如下表所示：

参数	描述
x0	编辑框控件相对于父窗口的 X 位置
y0	编辑框控件相对于父窗口的 Y 位置
xsize	编辑框控件的水平尺寸（单位：像素）
ysize	编辑框控件的垂直尺寸（单位：像素）
hParent	父窗口的句柄。如果为 0，则新的 EDIT 控件将是桌面（顶级窗口）的子项
WinFlags	窗口创建标记。为了使控件立即可见，通常为 WM_CF_SHOW

ExFlags	未使用，保留供日后使用
Id	控件的窗口 ID
MaxLen	最大字符数

表 18.4.4 EDIT_CreateEx 函数相关形参描述

返回值：已创建的 EDIT 控件的句柄；函数失败时为 0。

(4) EDIT_EnableBlink 函数

启用/禁用闪烁光标，该函数原型如下源码所示：

```
void EDIT_EnableBlink(EDIT_Handle hObj,
                      int Period,
                      int OnOff);
```

该函数的相关形参如下表所示：

参数	描述
hObj	EDIT 控件的句柄
Period	闪烁周期
OnOff	1：启用闪烁；0：禁用闪烁

表 18.4.5 EDIT_EnableBlink 函数相关形参描述

返回值：已创建的 EDIT 控件的句柄；函数失败时为 0。

(5) EDIT_EnableInversion 函数

默认情况下通过反转当前编辑的字符来绘制。如果需要具有专用文本和背景色的光标，可以关闭反转，并使用函数 EDIT_SetTextColor()和 EDIT_SetBkColor()设置光标颜色，该函数原型如下源码所示：

```
int EDIT_EnableInversion(EDIT_Handle hObj,
                         int OnOff);
```

该函数的相关形参如下表所示：

参数	描述
hObj	EDIT 控件的句柄
OnOff	1：启用反转；0：禁用反转

表 18.4.6 EDIT_EnableInversion 函数相关形参描述

返回值：先前使用的模式。

(6) EDIT_SetBinMode 函数

启用 EDIT 控件的二进制编辑模式。可在给定范围内修改给定数值，该函数原型如下所示：

```
void EDIT_SetBinMode(EDIT_Handle hObj,
                     U32 Value,
                     U32 Min, U32 Max);
```

该函数的相关形参如下表所示：

参数	描述
hObj	EDIT 控件的句柄
Value	要修改的值
Min	最小值
Max	最大值

表 18.4.7 EDIT_SetBinMode 函数相关形参描述

(7) EDIT_SetBkColor 函数

设置 EDIT 控件的背景色，该函数原型如下源码所示：

```
void EDIT_SetBkColor( EDIT_Handle hObj,
                     unsigned int Index,
```

```
GUI_COLOR Color);
```

该函数的相关形参如下表所示：

参数	描述	
hObj	EDIT 控件的句柄	
Index	参数 Index 的允许值	
	EDIT_CI_CURSOR	用于光标的背景色
	EDIT_CI_DISABLED	用于禁用状态的背景色
	EDIT_CI_ENABLED	用于启用状态的背景色
Color	要设置的颜色	

表 18.4.8 EDIT_SetBkColor 函数相关形参描述

(8) EDIT_SetCursorAtChar 函数

将 EDIT 控件光标设置到指定的字符位置，该函数原型如下源码所示：

```
void EDIT_SetCursorAtChar( EDIT_Handle hObj,
                           int xPos);
```

该函数的相关形参如下表所示：

参数	描述
hObj	EDIT 控件的句柄
xPos	要将光标设置到的字符位置

表 18.4.9 EDIT_SetCursorAtChar 函数相关形参描述

字符位置按如下参数确定：

- 0：第一个（最左）字符的左边。
- 1：第一个和第二个字符之间。
- 2：第二个和第三个字符之间，如此类推。

(9) EDIT_SetDecMode 函数

启用编辑框控件的十进制编辑模式。可在给定范围内修改给定数值，该函数原型如下所示：

```
void EDIT_SetDecMode( EDIT_Handle hEdit,
                     I32 Value,
                     I32 Min,
                     I32 Max,
                     int Shift,
                     U8 Flags);
```

该函数的相关形参如下表所示：

参数	描述
hEdit	EDIT 控件的句柄
Value	要修改的值
Min	最小值
Max	最大值
Shift	如果 >0，指定小数点的位置
Flags	参数 Flags（“OR”组合）的允许值
	GUI_EDIT_NORMAL 在正常模式下编辑，符号仅在 Value 为负值时显示
	GUI_EDIT_SIGNED 显示 “+” 和 “-” 号

表 18.4.10 EDIT_SetDecMode 函数相关形参描述

(10) EDIT_SetFloatMode 函数

启用编辑框控件的浮点编辑模式。可在给定范围内修改给定数值，该函数原型如下所示：

```
void EDIT_SetFloatMode(EDIT_Handle hObj, float Value, float Min,
```



```
float Max, int Shift, U8 Flags);
```

该函数的相关形参如下表所示:

参数	描述	
hObj	EDIT 控件的句柄	
Value	要修改的值	
Min	最小值	
Max	最大值	
Shift	小数点后的位数	
Flags	参数 Flags ("OR"组合) 的允许值	
	GUI_EDIT_NORMAL	在正常模式下编辑, 符号仅在 Value 为负值时显示
	GUI_EDIT_SIGNED	显示 “+” 和 “-” 号
	GUI_EDIT_SUPPRESS_LEADING_ZEROES	不显示前导零

表 18.4.11 EDIT_SetFloatMode 函数相关形参描述

(11) EDIT_SetFloatValue 函数

可用于设置 EDIT 控件的浮点数值, 该函数原型如下源码所示:

```
void EDIT_SetFloatValue(EDIT_Handle hObj, float Value);
```

该函数的相关形参如下表所示:

参数	描述
hObj	EDIT 控件的句柄
Value	EDIT 控件的新的浮点数值

表 18.4.12 EDIT_SetFloatValue 函数相关形参描述

(12) EDIT_SetHexMode 函数

启用编辑框控件的十六进制编辑模式。可在给定范围内修改给定值, 该函数原型如下所示:

```
void EDIT_SetHexMode(EDIT_Handle hObj,
                     U32 Value,
                     U32 Min,
                     U32 Max);
```

该函数的相关形参如下表所示:

参数	描述
hObj	EDIT 控件的句柄
Value	要修改的值
Min	最小值
Max	最大值

表 18.4.13 EDIT_SetHexMode 函数相关形参描述

(13) EDIT_SetText 函数

设置要在 EDIT 控件中显示的文本, 该函数原型如下源码所示:

```
void EDIT_SetText(EDIT_Handle hObj,
                  const char * s)
```

该函数的相关形参如下表所示:

参数	描述
hObj	EDIT 控件的句柄
s	要显示的文本

表 18.4.14 EDIT_SetText 函数相关形参描述

18.5 EDIT 控件演示实验

18.5.1 硬件设计

1. 例程功能

本实验主要讲解 DROPDOWN 小工具的使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数主要创建一个 window 界面并在其上面创建一个 DROPDOWN 和 BUTTON 小工具，DROPDOWN 控件显示“DROPDOWN”字符串，当用户按下 BUTTON 控件时，系统会关闭对话框并开启定时 3s，定时时间到达之后重新打开对话框。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 19 EDIT 小工具》。

18.5.2 软件设计

18.5.2.1 程序流程图

本实验的程序流程图，如下图所示：

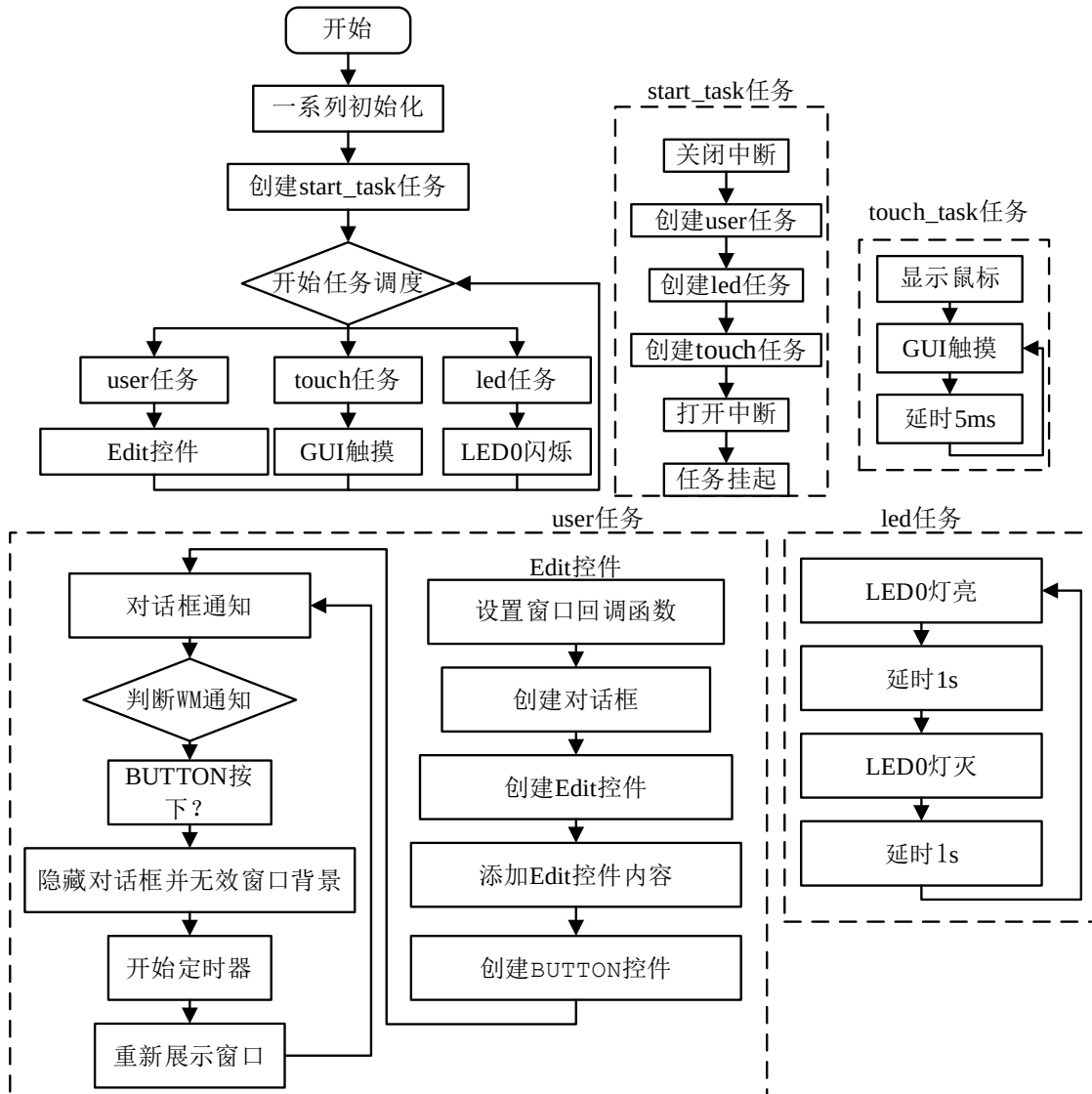


图 18.5.2.1.1EDIT 控件实验流程图

18.5.2.2 程序解析

本实验完整工程为：**emWin 实验 19 EDIT 小工具**，打开 emwin_demo.c，程序源码如下：

(1) GUIBulider 生成的资源表

```

#define WM_APP_SHOW_TEXT    (WM_USER +0)
#define TEXT_MAXLEN         40

WM_HWIN DialoghWin;
/* 对话框资源表 */
static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] =
{
    {FRAMEWIN_CreateIndirect, "Framewin", 0, 55, 90, 200,110,FRAMEWIN_CF_MOVEABLE},
    { EDIT_CreateIndirect, "Edit", GUI_ID_EDIT0, 24, 10, 145, 25, 0, 15},
    { BUTTON_CreateIndirect, "Ok", GUI_ID_OK, 65, 50, 70, 30}
};

```

上述源码是有 GUIBulider 软件生成得到的资源表，该资源表是创建对话框必备的数组。

(2) 建立对话框

```
/**
 * @brief      emwin_main(emwin 入口)
 * @param      无
 * @return     无
 */
void emwin_main (void)
{
    WM_SetCallback(WM_HBKWIN, _BkCallback);
    DialoghWin = GUI_CreateDialogBox(_aDialogCreate,
                                    GUI_COUNTOF(_aDialogCreate),
                                    _cbDialog, WM_HBKWIN, 0, 0);

    while (1)
    {
        GUI_Delay(10);
    }
}
```

此函数为 emWin 程序的入口，该函数主要设置窗口 _BkCallback 的回调函数和创建一个对话框，该对话框根据资源表和 _cbDialog 回调函数创建的。这两个回调函数如下所示：

(3) 窗口回调函数

```
/**
 * @brief      背景窗口的回调函数
 * @param      无
 * @return     无
 */
static void _BkCallback(WM_MESSAGE *pMsg)
{
    static WM_HWIN hWin;
    static WM_HWIN hEdit;
    static U8 text;
    char buffer[TEXT_MAXLEN];

    switch (pMsg->MsgId)
    {
        case WM_PAINT:
            GUI_SetBkColor(GUI_DARKGRAY);
            GUI_Clear();
            GUI_SetFont(&GUI_Font24_ASCII);
            GUI_DispStringHCenterAt("WIDGET_Edit - Sample", 160, 5);
            GUI_SetFont(&GUI_Font8x16);
    }
```

```

        if (text)
        {
            hEdit = WM_GetDialogItem(DialoghWin, GUI_ID_EDIT0);
            GUI_DispStringHCenterAt("The string you have
                                    modified is:",160,90);

            EDIT_GetText(hEdit, buffer, TEXT_MAXLEN);
            GUI_DispStringHCenterAt(buffer, 160, 110);
        }
        else
        {
            GUI_DispStringHCenterAt("Use keyboard to modify string",160,90);
        }
        break;

    case WM_APP_SHOW_TEXT:
        hWin = pMsg->hWinSrc;
        WM_HideWindow(hWin); /* 隐藏对话框 */
        text = 1;
        WM_InvalidateWindow(WM_HBKWIN); /* 背景窗口无效 */
        WM_CreateTimer(WM_HBKWIN, 0, 3000, 0);
        break;

    case WM_TIMER:
        text = 0;
        WM_InvalidateWindow(WM_HBKWIN);
        WM_ShowWindow(hWin);
        break;

    default:
        WM_DefaultProc(pMsg);
        break;
    }
}

/**
 * @brief      对话框对调函数
 * @param      pMsg
 * @return     无
 */
static void _cbDialog(WM_MESSAGE *pMsg)
{
    WM_HWIN hItem;
    int      NCode;

```

```

int      Id;
WM_MESSAGE Msg;

switch (pMsg->MsgId)
{
    case WM_INIT_DIALOG:
        /* 初始化 FRAMEWIN */
        hItem = pMsg->hWin;
        FRAMEWIN_SetText(hItem, "EDIT USER");
        FRAMEWIN_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
        FRAMEWIN_SetFont(hItem, GUI_FONT_16B_ASCII);
        /* 初始化 EDIT */
        hItem = WM_GetDialogItem(pMsg->hWin, GUI_ID_EDIT0);
        EDIT_EnableBlink(hItem, 500, 1);
        EDIT_SetText(hItem, "Hello Word!");
        EDIT_SetFont(hItem, &GUI_Font20_ASCII);
        EDIT_SetTextAlign(hItem, GUI_TA_LEFT | GUI_TA_VCENTER);
        break;
    case WM_NOTIFY_PARENT:
        Id      = WM_GetId(pMsg->hWinSrc);
        NCode = pMsg->Data.v;
        switch (Id)
        {
            case GUI_ID_EDIT0: /* EDIT 控件通知消息 */
                switch (NCode)
                {
                    case WM_NOTIFICATION_CLICKED:
                        break;
                    case WM_NOTIFICATION_RELEASED:
                        break;
                    case WM_NOTIFICATION_VALUE_CHANGED:
                        break;
                }
                break;
            case GUI_ID_OK:
                switch (NCode)
                {
                    case WM_NOTIFICATION_CLICKED:
                        break;

                    case WM_NOTIFICATION_RELEASED:
                        Msg.MsgId = WM_APP_SHOW_TEXT;
                        Msg.hWinSrc = pMsg->hWin;
                }
            }
        }
    }

```

```
        WM_SendMessage(WM_HBKWIN, &Msg);  
        break;  
    }  
}  
break;  
default:  
    WM_DefaultProc(pMsg);  
    break;  
}  
}
```

_BkCallback 回调函数主要用来重绘背景，而_cbDialog 回调函数主要设置对话框的控件以及根据消息类型触发相应的动作。

18.5.2 下载验证

编译并下载到开发板中，如下图所示：



图 18.5.2.1 运行结果

注意：这里我们没有做输入法，所以没办法在开发板上输入字符串，大家可以自行将本例程移植到模拟器中，在 PC 上模拟，通过键盘输入字符串，然后点击 OK 键，所输入的字符串就会显示在背景窗口中，如下图所示：



图 18.5.2.2 背景窗口中显示 EDIT 中的字符串

第十九章 FRAMEWIN 控件

框架窗口为我们的应用提供一个类似 PC 应用程序的窗口外观，这些窗口由周围框架、标题栏和用户区组成，标题栏的颜色改变以显示窗口是否激活，本章讲解一下 emWin 的框架小工具的使用。

本章分为以下几个部分：

19.1 FRAMEWIN 控件简介

19.2 FRAMEWIN 控件 API 函数

19.3 FRAMEWIN 控件演示实验

19.1 FRAMEWIN 控件简介

FRAMEWIN 控件的外观如下图所示：



图 19.1.1 FrameWin 控件

我们可以预定义的按钮附加到标题栏中，比如最大化、最小化和关闭按钮，如下图所示：

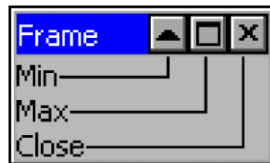


图 19.1.2 FrameWin 控件添加预定义按钮

皮肤设置，如下图所示：

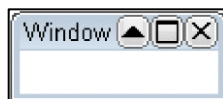


图 19.1.3 FrameWin 皮肤设置

FrameWin 框架的结构如下图所示：

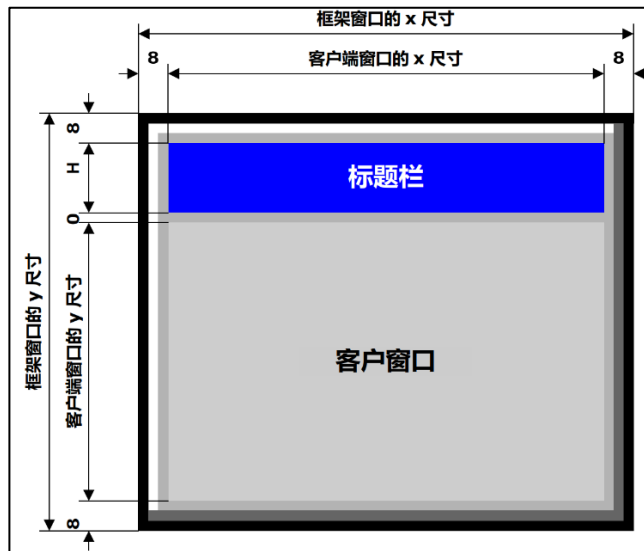


图 19.1.4 FrameWin 框架

框架窗口实际上由 2 种窗口组成：主窗口和子窗口，子窗口称为客户端窗口，处理回调函数时意识到此点非常重要：存在具有 2 种不同回调函数的 2 种窗口。创建子窗口时，这些子窗口通常作为客户端窗口的子窗口而创建；它们的父窗口因此是客户端窗口。图 19.1.4 中各个参数的含义如下表所示：

参数	描述
B	框架窗口的边框尺寸，默认的边框尺寸为
H	标题栏的高度，取决于标题所用字体的尺寸
D	标题栏和客户端窗口的间距
标题栏	标题栏是框架窗口的一部分，不是单独的窗口
客户端窗口	客户端窗口是一个单独的窗口，作为框架窗口的子窗口创建

表 19.1.1 FrameWin 窗口参数描述

19.2 FRAMEWIN 控件 API 函数

函数	描述
FRAMEWIN_AddButton()	在标题栏中添加一个按钮
FRAMEWIN_AddCloseButton()	在标题栏中添加一个“关闭”按钮
FRAMEWIN_AddMaxButton()	在标题栏中添加一个“最大化”按钮
FRAMEWIN_AddMenu()	向框架窗口添加一个“菜单”按钮
FRAMEWIN_AddMinButton()	在标题栏中添加一个“最小化”按钮
FRAMEWIN_Create()	创建一个 FRAMEWIN 小工具
FRAMEWIN_CreateAsChild()	创建一个 FRAMEWIN 小工具作为子窗口
FRAMEWIN_CreateEx()	创建一个 FRAMEWIN 小工具
FRAMEWIN_CreateIndirect()	从资源表项创建 FRAMEWIN 小工具
FRAMEWIN_CreateUser()	额外字节作为用户数据创建一个 FRAMEWIN 小工具
FRAMEWIN_GetActive()	返回框架窗口是否处于激活状态
FRAMEWIN_GetBarColor()	返回标题栏的颜色
FRAMEWIN_GetBorderSize()	返回边框的尺寸
FRAMEWIN_GetDefaultBarColor()	返回标题栏的默认颜色
FRAMEWIN_GetDefaultBorderSize()	返回默认的边框尺寸
FRAMEWIN_GetDefaultClientColor()	返回客户区的默认颜色

FRAMEWIN_GetDefaultFont()	返回标题栏使用的默认字体
FRAMEWIN_GetDefaultTextColor()	返回标题的默认文本颜色
FRAMEWIN_GetDefaultTitleHeight()	返回标题栏的默认尺寸
FRAMEWIN_GetFont()	返回标题文本所用的字体
FRAMEWIN_GetText()	返回标题文本
FRAMEWIN_GetTextAlign()	返回标题文本的对齐方式
FRAMEWIN_GetTitleHeight()	返回标题栏的高度
FRAMEWIN_GetUserData()	检索用 FRAMEWIN_SetUserData() 设置的数据
FRAMEWIN_IsMinimized()	返回表示框架窗口是否最小化的值
FRAMEWIN_IsMaximized()	返回表示框架窗口是否最大化的值
FRAMEWIN_Maximize()	将框架窗口放大到父窗口的尺寸
FRAMEWIN_Minimize()	隐藏框架窗口的客户区
FRAMEWIN_OwnerDraw()	用于绘制标题栏的默认函数
FRAMEWIN_Restore()	恢复最小化或最大化的框架窗口
FRAMEWIN_SetActive()	设置框架窗口的状态（弃用）
FRAMEWIN_SetBarColor()	设置标题栏的颜色
FRAMEWIN_SetBorderSize()	设置框架窗口的边框尺寸
FRAMEWIN_SetClientColor()	设置客户区的颜色
FRAMEWIN_SetDefaultBarColor()	设置标题栏的默认颜色
FRAMEWIN_SetDefaultBorderSize()	设置默认的边框尺寸
FRAMEWIN_SetDefaultClientColor()	设置客户区的默认颜色
FRAMEWIN_SetDefaultFont()	设置标题栏使用的默认字体
FRAMEWIN_SetDefaultTextColor()	设置标题的默认文本颜色
FRAMEWIN_SetDefaultTitleHeight()	设置标题栏的默认高度
FRAMEWIN_SetFont()	选择标题文本所用的字体
FRAMEWIN_SetMoveable()	将框架窗口设置为可移动/不可移动状态
FRAMEWIN_SetOwnerDraw()	启用要自画的框架窗口
FRAMEWIN_SetResizable()	将框架窗口设置为可缩放状态
FRAMEWIN_SetText()	设置标题文本
FRAMEWIN_SetTextAlign()	设置标题文本的对齐方式
FRAMEWIN_SetTextColor()	设置标题文本的颜色
FRAMEWIN_SetTextColorEx()	设置标题文本的颜色
FRAMEWIN_SetTitleHeight()	设置标题栏的高度
FRAMEWIN_SetTitleVis()	设置标题栏的可见性标记
FRAMEWIN_SetUserData()	设置 FRAMEWIN 小工具的额外数据

表 19.2.1 FRAMEWIN 控件 API 函数

表 19.2.1 是 FRAMEWIN 控件函数相关描述。接下来，我们来讲解 FRAMEWIN 控件常用的函数。

(1) FRAMEWIN_AddButton 函数

将按钮添加到 FRAMEWIN 控件的标题栏，该函数原型如下源码所示：

```
WM_HWIN FRAMEWIN_AddButton(FRAMEWIN_Handle hObj, int Flags, int Off, int Id);
```

该函数的相关形参如下表所示：

参数	描述	
hObj	FRAMEWIN 控件的句柄	
Flags	参数 Flags 的允许值	
	FRAMEWIN_BUTTON_LEFT	在左侧创建 BUTTON
	FRAMEWIN_BUTTON_RIGHT	在右侧创建 BUTTON

Off	用于创建 BUTTON 控件的 X 偏移
Id	BUTTON 控件的 ID

表 19.2.2 FRAMEWIN_AddButton 函数相关形参描述

返回值：BUTTON 控件的句柄。

功能示例：

之前	之后
	

(2) FRAMEWIN_AddCloseButton 函数

“关闭”按钮添加到框架窗口的标题栏，该函数原型如下源码所示：

```
WM_HWIN FRAMEWIN_AddCloseButton(FRAMEWIN_Handle hObj, int Flags, int Off);
```

该函数的相关形参如下表所示：

参数	描述	
hObj	FRAMEWIN 控件的句柄	
Flags	参数 Flags 的允许值	
	FRAMEWIN_BUTTON_LEFT	在左侧创建 BUTTON
	FRAMEWIN_BUTTON_RIGHT	在右侧创建 BUTTON
Off	用于创建 BUTTON 控件的 X 偏移	

表 19.2.3 FRAMEWIN_AddCloseButton 函数相关形参描述

返回值：“关闭”按钮的句柄。

注意：用户按下“关闭”按钮时，框架窗口及其所有子窗口都将被删除。

功能示例：

之前	之后
	

(3) FRAMEWIN_AddMaxButton 函数

将“最大化”按钮添加到框架窗口的标题栏，该函数原型如下源码所示：

```
WM_HWIN FRAMEWIN_AddMaxButton(FRAMEWIN_Handle hObj, int Flags, int Off);
```

该函数的相关形参如下表所示：

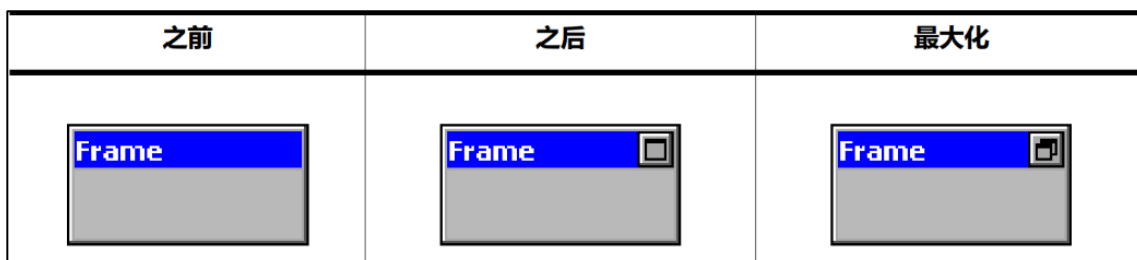
参数	描述	
hObj	FRAMEWIN 控件的句柄	
Flags	参数 Flags 的允许值	
	FRAMEWIN_BUTTON_LEFT	在左侧创建 BUTTON
	FRAMEWIN_BUTTON_RIGHT	在右侧创建 BUTTON
Off	用于创建 BUTTON 控件的 X 偏移	

表 19.2.4 FRAMEWIN_AddMaxButton 函数相关形参描述

返回值：“最大化”按钮的句柄。

注意：用户第一次按“最大化”按钮时，框架窗口将放大到父窗口的尺寸。第二次按该按钮时，将框架窗口缩小到原来尺寸并恢复到原来位置。

功能示例：



(4) FRAMEWIN_AddMenu 函数

给框架窗口添加给定菜单，该函数原型如下源码所示：

```
void FRAMEWIN_AddMenu(FRAMEWIN_Handle hObj, WM_HWIN hMenu);
```

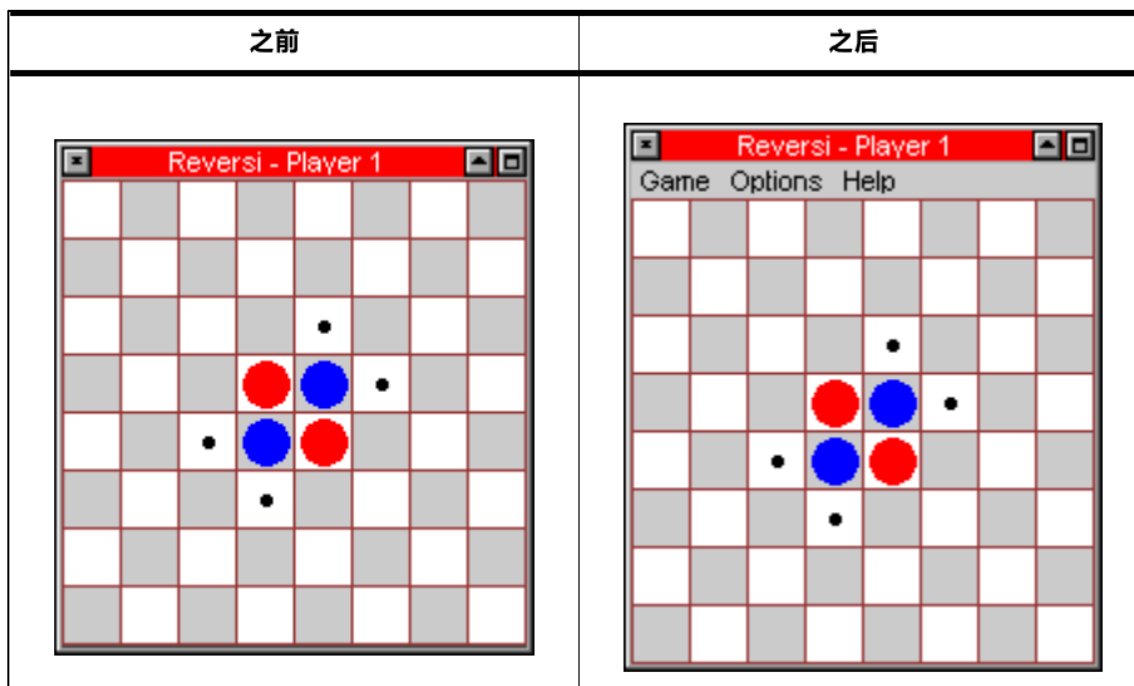
该函数的相关形参如下表所示：

参数	描述
hObj	框架窗口的句柄
hMenu	要添加的菜单控件的句柄

表 19.2.5 FRAMEWIN_AddMenu 函数相关形参描述

注意：添加的菜单作为框架窗口的子项附加。如果已用回调函数创建了框架窗口，则该函数确保将 WM_MENU 消息传递到框架窗口的客户端窗口。

功能示例：



(5) FRAMEWIN_AddMinButton 函数

将“最小化”按钮添加到框架窗口的标题栏，该函数原型如下源码所示：

```
WM_HWIN FRAMEWIN_AddMinButton(FRAMEWIN_Handle hObj,
                                int Flags,
                                int Off);
```

该函数的相关形参如下表所示：

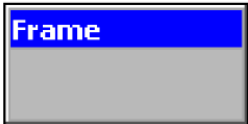
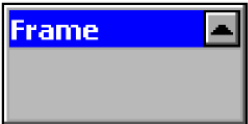
参数	描述	
hObj	FRAMEWIN 控件的句柄	
Flags	参数 Flags 的允许值	
	FRAMEWIN_BUTTON_LEFT	在左侧创建 BUTTON
	FRAMEWIN_BUTTON_RIGHT	在右侧创建 BUTTON
Off	用于创建 BUTTON 控件的 X 偏移	

表 19.2.6 FRAMEWIN_AddMinButton 函数相关形参描述

返回值：“最小化”按钮的句柄。

注意：用户第一次按“最小化”按钮时，框架窗口的客户区将隐藏，只有标题栏可见。第二次按该按钮时，框架窗口将恢复到原来尺寸。

功能示例：

之前	之后	最小化窗口
		

(6) FRAMEWIN_CreateEx 函数

在指定位置创建指定尺寸的 FRAMEWIN 控件，该函数原型如下源码所示：

```
FRAMEWIN_Handle FRAMEWIN_CreateEx( int x0, int y0,
                                     int xSize, int ySize,
                                     WM_HWIN hParent, int WinFlags,
                                     int ExFlags, int Id, const char * pTitle,
                                     WM_CALLBACK * cb);
```

该函数的相关形参如下表所示：

参数	描述	
x0	控件最左侧的像素（在父坐标中）	
y0	控件最顶端的像素（在父坐标中）	
xSize	控件的水平尺寸（以像素为单位）	
ySize	控件的垂直尺寸（以像素为单位）	
hParent	父窗口的句柄，如果为 0，则新的 FRAMEWIN 控件将是桌面的子项	
WinFlags	窗口创建标记，为了使控件立即可见，通常为 WM_CF_SHOW	
ExFlags	参数 ExFlags 的允许值	
	0	无功能
	FRAMEWIN_CF_MOVEABLE	将新框架窗口设置为可移动状态
Id	控件的窗口 ID	
pTitle	显示在标题栏中的标题	
cb	指向用户回调函数的指针，此用户回调函数由客户端窗口的窗口回调函数	

表 19.2.7 FRAMEWIN_CreateEx 函数相关形参描述

返回值：已创建的 FRAMEWIN 控件的句柄；函数失败时为 0。

(7) FRAMEWIN_Maximize 函数

将框架窗口放大到父窗口的尺寸，该函数原型如下源码所示：

```
void FRAMEWIN_Maximize(FRAMEWIN_Handle hObj);
```

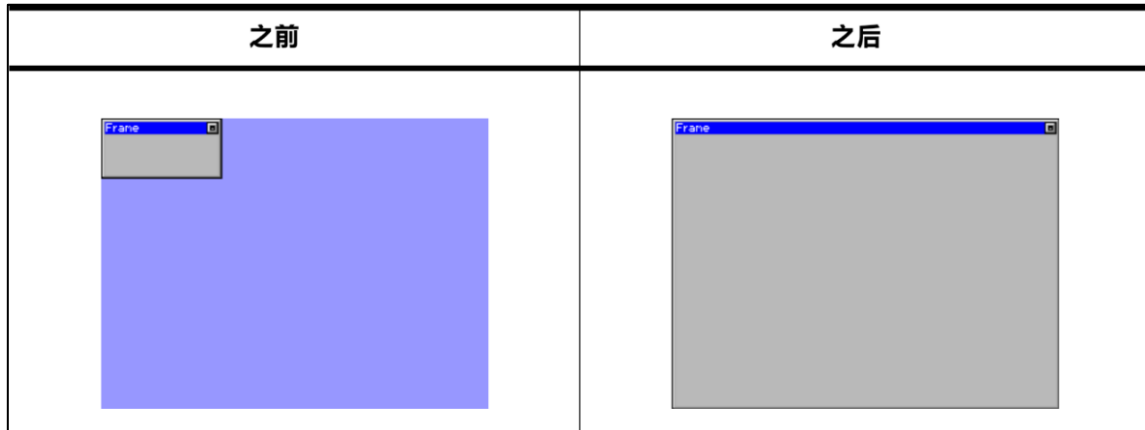
该函数的相关形参如下表所示：

参数	描述
hObj	FRAMEWIN 控件的句柄

表 19.2.8 FRAMEWIN_Maximize 函数相关形参描述

注意：调用此函数时，框架窗口的操作与用户按“最大化”按钮时相同。框架窗口将放大到父窗口尺寸。

功能示例：



(8) FRAMEWIN_Minimize 函数

隐藏给定框架窗口的客户区，该函数原型如下源码所示：

```
void FRAMEWIN_Minimize(FRAMEWIN_Handle hObj);
```

该函数的相关形参如下表所示：

参数	描述
hObj	FRAMEWIN 控件的句柄

表 19.2.9 FRAMEWIN_Minimize 函数相关形参描述

注意：调用此函数时，框架窗口的操作与用户按“最小化”按钮时相同。框架窗口的客户区将隐藏，只有标题栏可见。

19.3 FRAMEWIN 控件演示实验

19.3.1 硬件设计

1. 例程功能

本实验主要讲解 FRAMEWIN 小工具的使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数主要创建一个 FRAMEWIN 界面并在其上面添加删除、放大以及缩小按键，当用户按下不同的按键时，系统会做相应的动作。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 20 FRAMEWIN 小工具》。

19.3.2 软件设计

19.3.2.1 程序流程图

本实验的程序流程图，如下图所示：

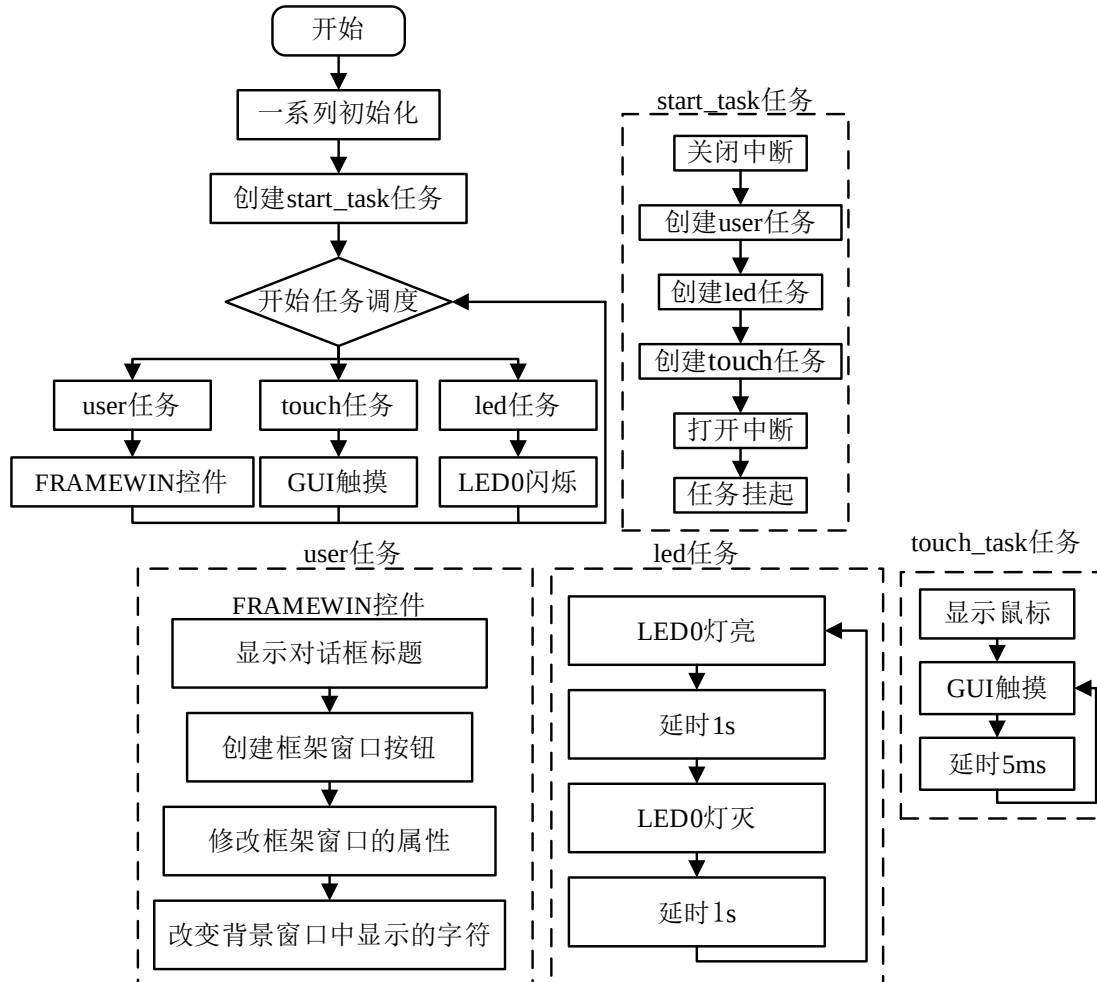


图 19.3.2.1.1 FRAMEWIN 控件实验流程图

19.3.2.2 程序解析

本实验完整工程为：**emWin 实验 20 FRAMEWIN 小工具**，本例程移植的 Segger 官方的例程：打开 emwin_demo.c，程序源码如下：

```

#define SPEED 1200

/* MSG_CHANGE_MAIN_TEXT 为用户自定义消息，用于改变要显示的字符串 */
#define MSG_CHANGE_MAIN_TEXT (WM_USER+0)

static FRAMEWIN_Handle _hFrame;
static WM_CALLBACK      *_pcbOldFrame;
static char             _acMainText[100];
static int              _LockClose = 1;

/**

```

```

* @brief      改变背景窗口中显示的字符
* @param      pStr   : 字符串
* @param      Delay  : 延时
* @return     无
*/

static void emwin_change_main_text(char *pStr, int Delay)
{
    WM_MESSAGE Message;
    /* 消息 ID 为 MSG_CHANGE_MAIN_TEXT, 并且将要传递的消息的首地址传递给 Data.p */
    Message.MsgId = MSG_CHANGE_MAIN_TEXT;
    Message.Data.p = pStr;
    GUI_Delay(Delay);
    /* 向背景窗口 WM_HBKWIN 发送消息 MSG_CHANGE_MAIN_TEXT */
    WM_SendMessage(WM_HBKWIN, &Message);
    WM_InvalidateWindow(WM_HBKWIN);
    GUI_Delay(Delay / 3);
}

/**
* @brief      子窗口回调函数
* @param      pMsg   : 消息
* @return     无
*/

static void _cbChild(WM_MESSAGE *pMsg)
{
    WM_HWIN hWin = (FRAMEWIN_Handle)(pMsg->hWin);

    switch (pMsg->MsgId)
    {
        case WM_PAINT:
            GUI_SetBkColor(GUI_WHITE);
            GUI_SetColor(GUI_BLACK);
            GUI_SetFont(&GUI_FontComic24B_ASCII);
            GUI_SetTextAlign(GUI_TA_HCENTER | GUI_TA_VCENTER);
            GUI_Clear();
            GUI_DispStringHCenterAt("Client window", WM_GetWindowSizeX(hWin) 2,
                                    WM_GetWindowSizeY(hWin) / 2);

            break;

        default:
            WM_DefaultProc(pMsg);
    }
}

```



```

/**
 * @brief      FrameWin 控件回调函数
 * @param      pMsg  : 消息
 * @return     无
 */
static void _cbFrame(WM_MESSAGE *pMsg)
{
    switch (pMsg->MsgId)
    {
        case WM_NOTIFY_PARENT:
            if (pMsg->Data.v == WM_NOTIFICATION_RELEASED)
            {
                int Id = WM_GetId(pMsg->hWinSrc);

                if (Id == GUI_ID_CLOSE) /* 点击了 FrameWin 的“关闭”按钮 */
                {
                    /* 如果 FrameWin 已经关闭了的话就直接返回，
                     否则的话将 _hFrame 置 0，标记框架已经关闭了 */
                    if (_LockClose) return;

                    _hFrame = 0;
                }
            }
            break;
    }

    if (_pcbOldFrame) (*_pcbOldFrame)(pMsg);
}

/**
 * @brief      背景窗口 WM_HBKWIN 回调函数
 * @param      pMsg  : 消息
 * @return     无
 */
static void _cbBkWindow(WM_MESSAGE *pMsg)
{
    switch (pMsg->MsgId)
    {
        /* 背景窗口 WM_HBKWIN 收到消息 MSG_CHANGE_MAIN_TEXT */
        case MSG_CHANGE_MAIN_TEXT:
            /* 将消息 MSG_CHANGE_MAIN_TEXT 中要显示的字符串拷贝到数组 _acMainText 中 */
            strcpy(_acMainText, pMsg->Data.p);
    }
}

```

```

        /* 使目标窗口无效化, 这里也就是 WM_HBKWIN */
        WM_InvalidateWindow(pMsg->hWin);
        break;

    case WM_PAINT:
        GUI_SetBkColor(GUI_BLACK);
        GUI_Clear();
        GUI_SetColor(GUI_WHITE);
        GUI_SetFont(&GUI_Font24_ASCII);
        GUI_DispStringHCenterAt("WIDGET_FrameWin - Sample", 160, 5);
        GUI_SetFont(&GUI_Font8x16);
        /* 显示数组_acMainText 中的字符串 */
        GUI_DispStringHCenterAt(_acMainText, 160, 40);
        GUI_SetFont(&GUI_Font6x8);
        GUI_DispStringHCenterAt("The function FRAMEWIN_Create creates
                                boththe\n"frame window and
                                the client window.", 160, 190);

        break;

    default:
        WM_DefaultProc(pMsg);
    }
}

/**
 * @brief      Framewin 演示
 * @param      无
 * @return     无
 */
static void emwin_framewin(void)
{
    int i;
    char acInfoText[] = "-- sec to play with window";
    WM_HWIN hChild;

    WM_SetCallback(WM_HBKWIN, _cbBkWindow); /* 为背景设置回调函数 */

    emwin_change_main_text("FRAMEWIN_Create", SPEED);
    /* 创建一个框架窗口 */
    _hFrame = FRAMEWIN_Create("Frame window", 0, WM_CF_SHOW, 30, 60, 260, 125);
    FRAMEWIN_SetTitleHeight(_hFrame, 20);
    FRAMEWIN_SetFont(_hFrame, &GUI_Font16B_ASCII);

```

```

_pcbOldFrame = WM_SetCallback(_hFrame, _cbFrame); /* 为框架窗口设置回调函数 */

hChild = WM_GetClientWindow(_hFrame); /* 获取子窗口的句柄 */

WM_SetCallback(hChild, _cbChild); /* 设置子窗口的回调函数 */

FRAMEWIN_SetMoveable(_hFrame, 1); /* 设置框架窗口为可移动的 */
/* 给框架窗口创建几个按钮 */
FRAMEWIN_AddCloseButton(_hFrame, FRAMEWIN_BUTTON_RIGHT, 0);
FRAMEWIN_AddMaxButton(_hFrame, FRAMEWIN_BUTTON_RIGHT, 2);
FRAMEWIN_AddMinButton(_hFrame, FRAMEWIN_BUTTON_RIGHT, 2);

/*修改框架窗口的属性 */
emwin_change_main_text("FRAMEWIN_SetActive", SPEED);
FRAMEWIN_SetActive(_hFrame, 1); /* 激活框架_hFrame */
emwin_change_main_text("FRAMEWIN_SetFont", SPEED);
/* 设置_hFrame 的字体为 GUI_Font24_ASCII */
FRAMEWIN_SetFont(_hFrame, &GUI_Font24_ASCII);
/* 设置框架标题栏的高度为 25 */
FRAMEWIN_SetTitleHeight(_hFrame, 25);
emwin_change_main_text("FRAMEWIN_SetTextColor", SPEED);
/* 设置框架的文本颜色为 GUI_YELLOW */
FRAMEWIN_SetTextColor(_hFrame, GUI_YELLOW);
emwin_change_main_text("FRAMEWIN_SetTextAlign", SPEED);
/* 设置框架文本的对齐方式 */
FRAMEWIN_SetTextAlign(_hFrame, GUI_TA_HCENTER);
emwin_change_main_text("FRAMEWIN_Minimize", SPEED);
/* 设置框架最小化 */
FRAMEWIN_Minimize(_hFrame);
emwin_change_main_text("FRAMEWIN_Maximize", SPEED);
/* 调用函数 FRAMEWIN_Maximize() 设置框架最大化 */
FRAMEWIN_Maximize(_hFrame);
emwin_change_main_text("FRAMEWIN_Restore", SPEED);
/* 调用函数 FRAMEWIN_Restore() 恢复到原来的尺寸和位置 */
FRAMEWIN_Restore(_hFrame);
emwin_change_main_text("FRAMEWIN_SetTitleVis", SPEED);

for (i = 0; i < 5; i++)
{
    /* 调用函数 FRAMEWIN_SetTitleVis() 设置框架标题栏不可见 */
    FRAMEWIN_SetTitleVis(_hFrame, 0);
    GUI_Delay(200);
    /* 调用函数 FRAMEWIN_SetTitleVis() 设置框架标题栏可见 */

```

```

    FRAMEWIN_SetTitleVis(_hFrame, 1);
    GUI_Delay(200);
}

_LockClose = 0;

for (i = 250; (i > 0) && _hFrame; i--)
{
    acInfoText[0] = '0' + ((i + 9) / 100);
    acInfoText[1] = '0' + (((i + 9) / 10) % 10);
    emwin_change_main_text(acInfoText, 0);
    GUI_Delay(100);
}

/* 如果_hFrame 为 1 的话说明没有框架，那么在这里就删除掉框架窗口 */
if (_hFrame)
{
    /* 调用函数 FRAMEWIN_Delete() 关闭框架 */
    emwin_change_main_text("FRAMEWIN_Delete", SPEED);
}
else
{
    emwin_change_main_text("", 50);
}
}

/**
 * @brief      emwin_main(emwin 入口)
 * @param      无
 * @return     无
 */
void emwin_main (void)
{
    WM_EnableMemdev(WM_HBKWIN);

    while (1)
    {
        emwin_framewin();
    }
}

```

19.3.2 下载验证

编译并下载到开发板中，如下图所示：



图 19.3.2.1 演示结果

大家可以下载到开发板中点击框架中的“关闭”，“最大化”和“最小化”按钮来观察框架的变化。

第二十章 GRAPH 控件

GRAPH 控件可在屏幕上以波形的方式来显示一些数据，比如温度、电压、电机转速等等，它通过波形使我们更直观的观察数据的变化，数据如何变为波形得？例如我们可以通过打点的方式，在屏幕上绘制出采集到的数据值，它们连接起来就是波形了。但是这种方法很原始，操作很不方便，所以 emWin 给我们提供了一个控件来完成波形的绘制，那就 GRAPH 控件，本章我们来学习 GRAPH 控件的使用方法。

本章分为以下几个部分：

20.1 GRAPH 控件简介

20.2 GRAPH 控件 API 函数

20.3 GRAPH 控件演示实验

20.1 GRAPH 控件简介

GRAPH 控件用来可视化数据得，典型的应用场景就是显示数据的测量值，可以同时显示多条数据线，也可以用竖屏和垂直刻度来标记数据线，比如显示温度值的时候，X 轴作为时间轴，Y 轴作为温度值轴。如果我们要显示的数据值不能容纳进可见区域的话，GRAPH 控件可以生成滚动条，借用滚动条来查看那些不能纳入可见区域的数据，GRAPH 控件示意图如下图所示：

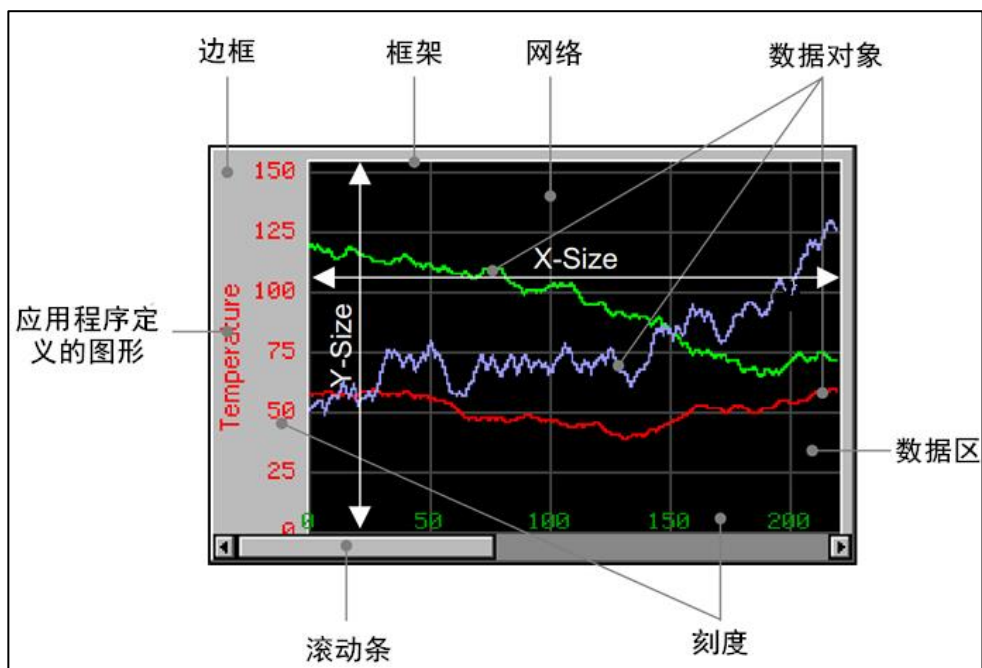


图 20.1.1 GRAPH 控件

根据上图所示，我们来讲解一下 GRAPH 控件各个组成部分如下表所示：

控件成员	描述
边框	边框为可选部分，是 GRAPH 控件的一部分
框架	数据区四周的细线
网络	显示在数据区的背景中的网格

数据区	网络和数据对象的显示区域
数据对象	每组数据的曲线
应用程序定义的图形	应用程序定义的一种回调函数，可用于绘制任何应用程序定义的文本或图形
滚动条	控件可以选择是否使用垂直和水平滚动条，通过滚动条可以查看大于控件可视区域的数据值
刻度	X 轴和 Y 轴的刻度
X-Size	数据区的 X 轴尺寸
Y-Size	数据区的 Y 轴尺寸

表 20.1.1 GRAPH 控件组成部分

下面我们来讲解一下 GRAPH 控件的创建步骤，该步骤如下所示：

- (1) 创建图形小工具并设置属性。
- (2) 创建数据对象。
- (3) 将数据对象附加到图形小工具中。
- (4) 创建可选的刻度对象
- (5) 将刻度对象附加到图形小工具中。

上述内容就是 GRAPH 控件创建的步骤，下面我们来介绍 GRAPH 控件的图表类型，emWin 的 GRAPH 控件提供了两种数据曲线类型 XY 类型和 YT 类型，这两种类型曲线如下图所示：

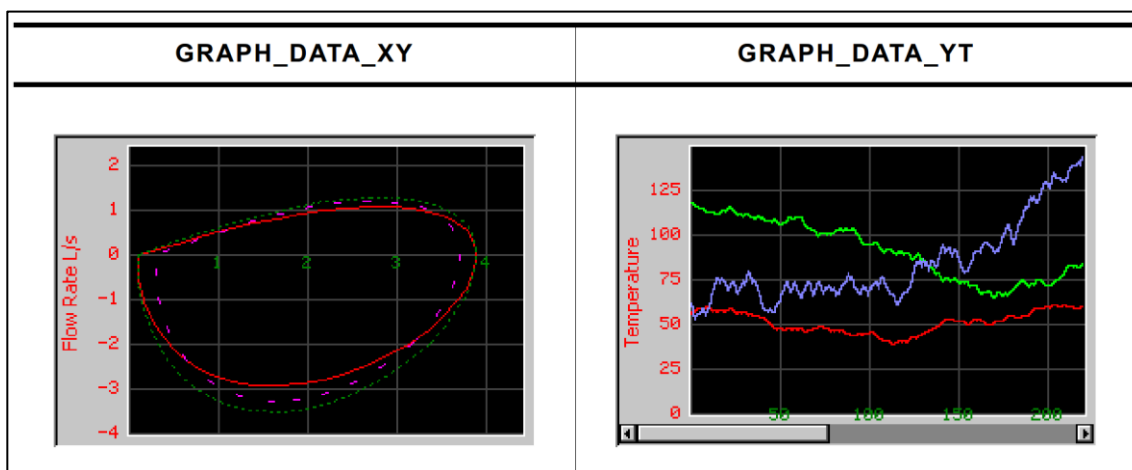


图 20.1.2 XY 和 YT 类型曲线

根据上图可知：emWin 的 GRAPH 控件图表类型具有两种形式，下面我们分别来讲解这两种类型。

(1) GRAPH_DATA_XY

XY 类型用于显示点阵组成的曲线，绘制出的数据线为折线，这个类型的曲线典型应用是绘制函数图形，比如函数 $Y=2X+1$ ，不同的 X 值对应不同的 Y 值，我们可以使用 XY 曲线画出 X 和 Y 之间的关系。

(2) GRAPH_DATA_YT

此类型用于显示图形上每个 X 位置都有一个 Y 值的数据曲线，典型的应用就是显示测量值连续变化的曲线，比如显示温度，速度，ADC 值等数据。

20.2 GRAPH 控件 API 函数

表 20.2.1 按字母顺序列出了可用的 GRAPH 控件相关 API 函数：

函数	描述
----	----

常用函数	
GRAPH_AttachData()	将数据对象附加到 GRAPH 小工具
GRAPH_AttachScale()	将刻度对象附加到 GRAPH 小工具
GRAPH_CreateEx()	创建 GRAPH 小工具
GRAPH_CreateIndirect()	从资源表项创建 GRAPH 小工具
GRAPH_CreateUser()	使用额外字节作为用户数据创建 GRAPH 小工具
GRAPH_DetachData()	从 GRAPH 小工具分离数据对象
GRAPH_DetachScale()	从 GRAPH 小工具分离刻度对象
GRAPH_GetUserData()	检索由 GRAPH_SetUserData 设置的数据
GRAPH_SetBorder()	设置边框的尺寸（上、下、左、右）
GRAPH_SetColor()	设置 GRAPH 小工具的颜色
GRAPH_SetGridDistX()	设置水平网格间距
GRAPH_SetGridDistY()	设置垂直网格间距
GRAPH_SetGridFixedX()	固定 X 轴方向的网格
GRAPH_SetGridVis()	启用网格绘制
GRAPH_SetLineStyleH()	设置水平网格线的线型
GRAPH_SetLineStyleV()	设置垂直网格线的线型
GRAPH_SetUserData()	设置 GRAPH 小工具的额外数据
GRAPH_SetUserDraw()	设置用户回调函数
GRAPH_SetVSizeX()	设置 GRAPH 小工具的水平范围
GRAPH_SetVSizeY()	设置 GRAPH 小工具的垂直范围
GRAPH_DATA_YT 相关 API 函数	
GRAPH_DATA_YT_AddValue()	向 GRAPH_DATA_YT 对象添加数据项
GRAPH_DATA_YT_Clear()	清除 GRAPH_DATA_YT 对象的所有数据项
GRAPH_DATA_YT_Create()	创建 GRAPH_DATA_YT 对象
GRAPH_DATA_YT_Delete()	删除 GRAPH_DATA_YT 对象
GRAPH_DATA_YT_MirrorX()	镜像 x 轴。
GRAPH_DATA_YT_SetAlign()	设置给定 GRAPH_DATA_YT 对象的对齐方式
GRAPH_DATA_YT_SetOffY()	设置用于绘制数据的垂直偏移
GRAPH_DATA_XY 相关 API 函数	
GRAPH_DATA_XY_AddPoint()	向 GRAPH_DATA_XY 对象添加一个点
GRAPH_DATA_XY_Create()	创建 GRAPH_DATA_XY 对象
GRAPH_DATA_XY_Delete()	删除 GRAPH_DATA_XY 对象
GRAPH_DATA_XY_SetLineStyle()	设置用于绘制数据的线型
GRAPH_DATA_XY_SetOffX()	设置用于绘制数据的水平偏移
GRAPH_DATA_XY_SetOffY()	设置用于绘制数据的垂直偏移
GRAPH_DATA_XY_SetOwnerDraw()	设置自回调函数
GRAPH_DATA_XY_SetPenSize()	设置绘制数据所用的画笔尺寸
刻度相关 API 函数	
GRAPH_SCALE_Create()	创建 GRAPH_SCALE 对象
GRAPH_SCALE_Delete()	删除 GRAPH_SCALE 对象
GRAPH_SCALE_SetFactor()	设置用于计算从像素转换到所需单位的计算因子
GRAPH_SCALE_SetFixed()	避免刻度滚动
GRAPH_SCALE_SetFont()	设置用于绘制编号的字体
GRAPH_SCALE_SetNumDecs()	设置小数部分的位数
GRAPH_SCALE_SetOff()	设置添加到编号的可选偏移
GRAPH_SCALE_SetPos()	设置刻度的水平或垂直位置
GRAPH_SCALE_SetTextColor()	设置刻度的文本颜色

GRAPH_SCALE_SetTickDist()	设置刻度线之间的像素距离
---------------------------	--------------

表 20.2.1 GRAPH 控件 API 函数

GRAPH 控件的 API 函数比较多，下面我们讲解几个比较重点的函数：

(1) GRAPH_DATA_YT_Create 函数

创建 GRAPHZ_DATA_YT 类型的数据对象，此数据对象要求 X 轴上的每个点在 Y 轴上都有一个值，通常用于和时间相关的图形，该函数原型如下源码所示：

```
GRAPH_DATA_Handle GRAPH_DATA_YT_Create(GUI_COLOR Color,
                                         unsigned MaxNumItems,
                                         I16 * pData,unsigned NumItems);
```

函数参数描述如下表所示：

参数	描述
Color	绘制数据要使用的颜色
MaxNumItems	数据项的最大数
pData	指向要添加到对象的数据的指针，该指针应指向一组 I16 值
NumItems	要添加的数据项数

表 20.2.2 GRAPH_DATA_YT_Create 函数相关参数描述

返回值：创建成功时数据对象的句柄，否则为 0。

注意：最后一个数据项显示在数据区的最右列。如果数据对象包含的数据多于图形小工具数据区可显示的数据，则使用函数 GRAPH_SetVSizeX 来显示滚动条，从而可在较大的数据对象中滚动。附加到图形小工具后，就不需要使用应用程序来删除数据对象，在删除图形小工具时会自动删除。

(2) GRAPH_AttachData 函数

将数据对象附加到 GRAPH 控件中，该函数原型如下源码所示：

```
void GRAPH_AddGraph(GRAPH_Handle hObj, GRAPH_DATA_Handle hData)
```

函数参数如下表所示：

参数	描述
hObj	小工具的句柄
hData	要添加到小工具的数据对象的句柄，数据对象应由 GRAPH_DATA_YT_Create 或 GRAPH_DATA_XY_Create 创建

表 20.2.3 GRAPH_AddGraph 函数相关参数描述

返回值：无。

功能示例：



(3) GRAPH_SCALE_Create 函数

创建刻度对象，该函数原型如下源码所示：

```
GRAPH_SCALE_Handle GRAPH_SCALE_Create(int Pos, int Align,
                                       unsigned Flags, unsigned TickDist);
```

函数参数如下表所示：

参数	描述
Pos	相对于图形小工具的左边/顶边的位置
TextAlign	用于绘制编号的文本对齐方式
Flags	设置创建水平或垂直刻度，GRAPH_SCALE_CF_HORIZONTAL 为水平刻度，GRAPH_SCALE_CF_VERTICAL 为垂直刻度
TickDist	从一个刻度线到下一刻度线的距离

表 20.2.4 GRAPH_SCALE_Create 函数相关参数描述

返回值：创建成功时刻度对象的句柄，否则返回 0。

注意：水平刻度对象从数据区的左边开始向右边进行标记，垂直刻度对象从顶部开始向底部进行标记，其中第一个位置是零点。参数 TickDist 指定编号之间的距离。参数 Pos 指定从图形小工具顶边到刻度文本的垂直距离，以像素为单位，请注意，实际文本位置也取决于用参数 TextAlign 指定的文本对齐方式。刻度对象为数据区内的每个位置绘制一个编号。对于水平刻度，有一个例外：如果第一个位置是 0，则在此位置不绘制编号。

(4) GRAPH_AttachScale 函数

将刻度对象添加到 GRAPH 控件中，该函数原型如下源码所示：

```
void GRAPH_AttachScale(GRAPH_Handle hObj, GRAPH_SCALE_Handle hScale)
```

函数参数如下表所示：

参数	描述
hObj	小工具句柄
hScale	要添加的刻度的句柄

表 20.2.5 GRAPH_AttachScale 函数相关参数描述

返回值：无。

功能示例：



(5) GRAPH_SCALE_SetFactor 函数

用于计算刻度编号的因子，该函数原型如下源码所示：

```
float GRAPH_SCALE_SetFactor(GRAPH_SCALE_Handle hScaleObj, float Factor)
```

函数参数如下表所示：

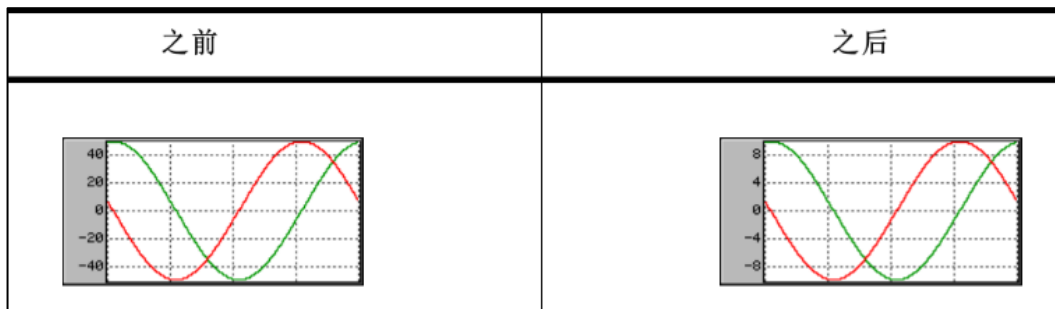
参数	描述
hScaleObj	刻度对象的句柄
Factor	要用于计算编号的因子

表 20.2.6 GRAPH_SCALE_SetFactor 函数相关参数描述

返回值：返回刻度编号。

注意：此函数用于计算刻度值的因子，默认情况下，我们在给 GRAPH 标志水平或者垂直刻度的时候默认是根据像素值计算的，比如 GRAPH 的可视区域高为 300 像素点，那么垂直刻度范围默认就是 0~300，但是我们实际上要用到的刻度范围是 0~3000 的时候该怎么办？这个时候就可以调用函数 GRAPH_SCALE_SetFactor 来设置计算因子为 10 即可。

功能示例：



(6) GRAPH_SCALE_SetOff 函数

用于在正负方向上“移动”的刻度对象的偏移，该函数原型如下源码所示：

```
int GRAPH_SCALE_SetOff(GRAPH_SCALE_Handle hScaleObj, int Off)
```

函数参数如下表所示：

参数	描述
hScaleObj	刻度对象的句柄
Off	用于绘制刻度的偏移

表 20.2.7 GRAPH_SCALE_SetOff 函数相关参数描述

返回值：返回偏移量。

注意：对于水平刻度对象默认是从可视区的左边开始向右边进行标记的，垂直刻度对象是从底部开始向顶部进行标记的，这时候 0 点就在左下角，有时候我们不希望 0 点在左下角。比如显示波形的时候，0 点在垂直刻度的中央，可以显示负值和正值，这个时候就可以调用函数 GRAPH_SCALE_SetOff 设置垂直刻度的偏移。

功能示例：



(7) GRAPH_DATA_YT_AddValue 函数

向 GRAPH_DATA_YT 对象添加新数据项，该函数原型如下源码所示：

```
void GRAPH_DATA_YT_AddValue(GRAPH_DATA_Handle hDataObj,  
                             I16 Value)
```

函数参数如下表所示：

参数	描述
hDataObj	数据对象的句柄
Value	要添加到数据对象的值

表 20.2.8 GRAPH_DATA_YT_AddValue 函数相关参数描述

返回值：无。

注意：给定的数据值添加到数据对象中，如果该数据对象“已满”，即意味着它包含的数据项与创建时在参数 MaxNumItems 中指定的项数相同，在添加新值前会首先移动一个数据项，因此，向“已满”对象添加数据项时，第一个数据项被移出。

20.3 GRAPH 控件演示实验

20.3.1 硬件设计

1. 例程功能

本实验主要讲解 GRAPH 小工具的使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数主要创建一个 window 界面并在其上面添加多个控件，例如按键、图表、滑块控件等，当用户按下 bk_Color 按键时，系统会切换图表部件的背景颜色；当用户滑动 Spacing X 滑块时，系统对图表控件的 X 坐标放大与缩小，其他的控件也是一样，它们都是操作图表控件相应属性的。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 21 GRAPH 小工具》。

20.3.2 软件设计

20.3.2.1 程序流程图

本实验的程序流程图，如下图所示：

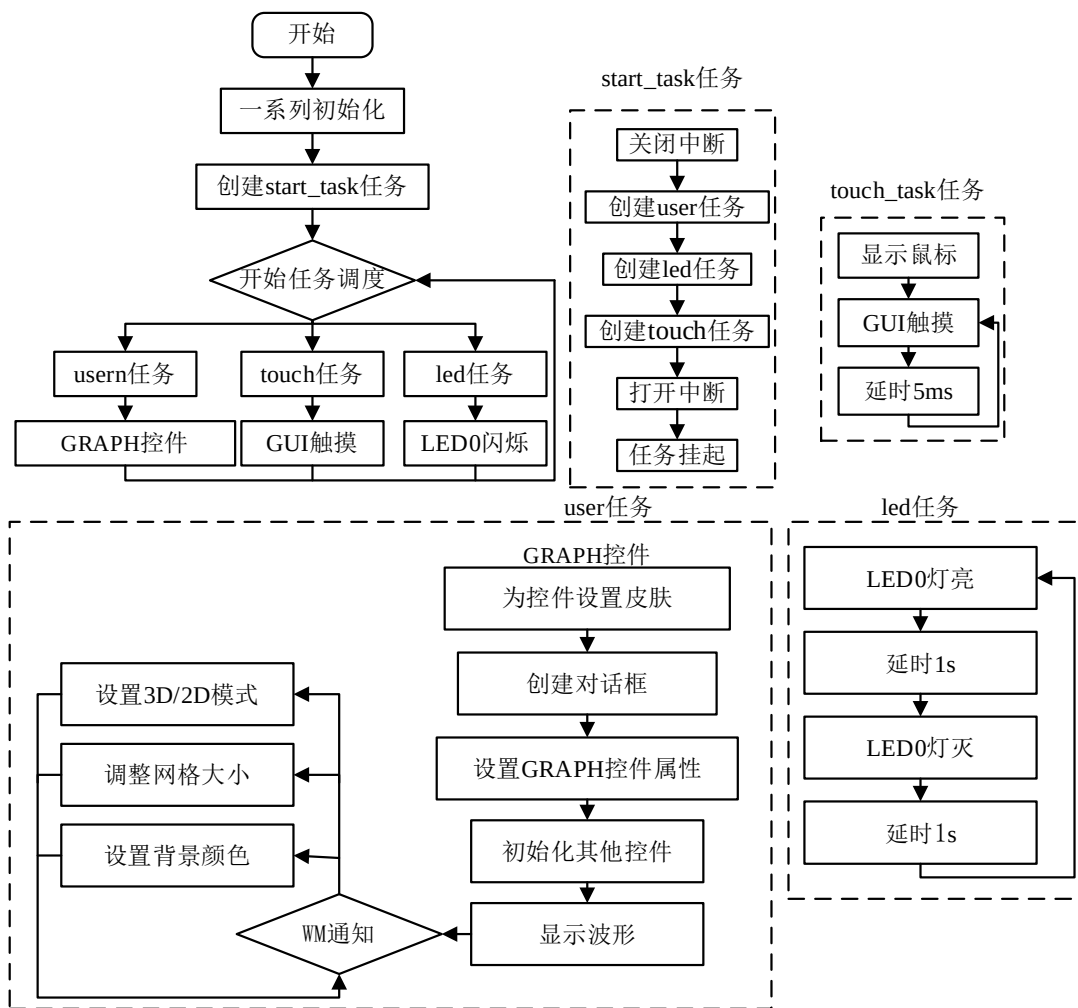


图 20.3.2.1.1 GRAPH 控件实验流程图

19.3.2.2 程序解析

本章我们使用 GRAPH 控件来显示 STM32 内部 AD 采集到的电压值波形，采用 YT 类型，实现的功能如下：

1. 数据区大小为 452*331。
2. 一条数据曲线来显示采集到的 AD 值。
3. 一次最大可以显示 600 个数据值。
4. 显示水平和垂直刻度值。
5. 可以控制波形和网隔放大与缩小。
6. 边框的设置。

首先用 GUIBulider 搭建一个雏形，修改生成的代码使其满足我们的设计要求，代码比较长，我们节选其中重点程序做讲解。打开 **emWin 实验 21 GRAPH 小工具**，工程中的 emwin_demo.c 和 emwin_demo.h 就是本章的实验源码，emwin_demo.h 定义了一个结构体 _oscill_dev，该结构体如下所示：

```
typedef struct
{
    int *buffer; /* AD 数据存放区*/
}
```

```
uint8_t      adflag;                /* AD 采集是否完成标志 */
int  dataxsize, dataysize;          /* 示波器数据区 x,y 大小 */
uint8_t      linecolorindex, backcolorindex; /* 线条颜色索引值和背景颜色索引值 */
GUI_COLOR linecolor, backcolor;     /* 线条颜色和背景颜色 */
GRAPH_DATA_Handle graphdata;        /* GRAPH_DATA 的数据句柄 */
int  div;                            /* 一格代表几秒 */
float resolut;                       /* 每个 AD 原始值代表多大的电压 (单位为 mv) */
} _oscill_dev;

extern _oscill_dev oscilldev;
```

结构体 _oscill_dev 很简单，定义了本例程中的一些属性。下面我们分别讲解 emwin_demo.c 文件的源码，如下所示：

(1) 定义控件皮肤

```
/* 自定义皮肤专用结构体 */
BUTTON_SKINFLEX_PROPS  ButtonSkin;    /* 按键皮肤 */
RADIO_SKINFLEX_PROPS   RadioSkin;     /* 单选控件 */
SPINBOX_SKINFLEX_PROPS SpinBoxSkin;    /* 微调框皮肤 */
SLIDER_SKINFLEX_PROPS  SliderSkin;     /* 滑块皮肤 */
MULTIPAGE_SKINFLEX_PROPS MultipageSkin; /* 分页皮肤 */
FRAMEWIN_SKINFLEX_PROPS FramewinSkin;  /* 框架窗口皮肤 */
```

(2) GUIBulider 生成的资源表

```
/* 资源表 */
static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] = {
{ FRAMEWIN_CreateIndirect, "Framewin", ID_FRAMEWIN_0, 1, 0, 800, 480, 0, 0x64, 0 },
{ GRAPH_CreateIndirect, "Graph", ID_GRAPH_0, 20, 12, 452, 331, 0, 0x0, 0 },
{ TEXT_CreateIndirect, "Text", ID_TEXT_0, 22, 366, 80, 20, 0, 0x64, 0 },
{ TEXT_CreateIndirect, "Text", ID_TEXT_1, 22, 412, 80, 20, 0, 0x64, 0 },
{ SLIDER_CreateIndirect, "Slider", ID_SLIDER_0, 79, 408, 152, 32, 0, 0x0, 0 },
{ SLIDER_CreateIndirect, "Slider", ID_SLIDER_1, 79, 361, 152, 32, 0, 0x0, 0 },
{ CHECKBOX_CreateIndirect, "Checkbox", ID_CHECKBOX_0, 601, 312, 82, 43, 0, 0x0, 0 },
{ CHECKBOX_CreateIndirect, "Checkbox", ID_CHECKBOX_1, 288, 368, 82, 43, 0, 0x0, 0 },
{ CHECKBOX_CreateIndirect, "Checkbox", ID_CHECKBOX_2, 688, 312, 82, 43, 0, 0x0, 0 },
{ CHECKBOX_CreateIndirect, "Checkbox", ID_CHECKBOX_3, 390, 368, 82, 43, 0, 0x0, 0 },
{ TEXT_CreateIndirect, "Text", ID_TEXT_2, 488, 17, 97, 30, 0, 0x64, 0 },
{ CHECKBOX_CreateIndirect, "Checkbox", ID_CHECKBOX_4, 491, 51, 100, 39, 0, 0x0, 0 },
{ CHECKBOX_CreateIndirect, "Checkbox", ID_CHECKBOX_5, 491, 120, 100, 39, 0, 0x0, 0 },
{ CHECKBOX_CreateIndirect, "Checkbox", ID_CHECKBOX_6, 491, 191, 100, 39, 0, 0x0, 0 },
{ CHECKBOX_CreateIndirect, "Checkbox", ID_CHECKBOX_7, 491, 259, 100, 39, 0, 0x0, 0 },
{ TEXT_CreateIndirect, "Text", ID_TEXT_3, 488, 308, 97, 30, 0, 0x64, 0 },
{ RADIO_CreateIndirect, "Radio", ID_RADIO_0, 487, 334, 73, 69, 0, 0x1403, 0 },
{ BUTTON_CreateIndirect, "Button", ID_BUTTON_1, 600, 120, 101, 41, 0, 0x0, 0 },
{ BUTTON_CreateIndirect, "Button", ID_BUTTON_2, 600, 191, 101, 41, 0, 0x0, 0 },
{ BUTTON_CreateIndirect, "Button", ID_BUTTON_3, 600, 259, 101, 41, 0, 0x0, 0 },
```

```
{ CHECKBOX_CreateIndirect, "Checkbox", ID_CHECKBOX_8, 601, 370, 128, 48, 0, 0x0, 0 },
};
```

上述源码为 emWin 资源表的定义，这个资源表是根据 GUIBulider 软件搭建的，如图 19.3.2.1 所示：



图 19.3.2.2.1 资源表组成的界面

(3) 建立对话框

```
/**
 * @brief      emwin_main
 * @param      无
 * @return     无
 */
void emwin_main(void)
{
    WM_HWIN hDlg;
    WM_HWIN _hGraph;
    WM_HWIN ClientWin;
    int Graph_xSize, Graph_ySize;

    _hGraph = 0;

    Widget_SetSkin(); /* 设置控件皮肤 */

    WM_SetDesktopColor(GUI_BLACK);
#ifdef GUI_SUPPORT_MEMDEV
    WM_SetCreateFlags(WM_CF_MEMDEV);
#endif
}
```

```

hDlg = GUI_CreateDialogBox(_aDialogCreate, GUI_COUNTOF(_aDialogCreate),
                           _cbCallback, 0, 0, 0);

ClientWin = WM_GetDialogItem(hDlg, ID_GRAPH_0);
Graph_xSize = WM_GetWindowSizeX(ClientWin);
Graph_ySize = WM_GetWindowSizeY(ClientWin);

oscilldev.dataxsize = Graph_xSize - BORDER_LEFT - BORDER_RIGHT - 3;
oscilldev.dataysize = Graph_ySize - BORDER_TOP - BORDER_BOTTOM - 3;
oscilldev.buffer = mymalloc(SRAMEX, oscilldev.dataxsize * 4); /* 申请内存 */

while (1)
{
    if (!Craph_Stop) /* 判断是否停止 */
    {
        if (!_hGraph)
        {
            _hGraph = WM_GetDialogItem(hDlg, ID_GRAPH_0); /* 获取句柄 */
        }

        _AddValues(); /* 显示数据 */
    }
    GUI_Delay(100);
    GUI_Exec();
}
}

```

从上述的源码可以看出，我们调用函数 `GUI_CreateDialogBox` 根据资源表创建对话框，函数 `Widget_SetSkin` 为各个控件设置皮肤。

(4) 控件设置皮肤

```

/**
 * @brief      为控件设置皮肤 (除关闭按钮和 Knob 控件外)
 * @param      无
 * @retval     无
 */
void emwin_widget_set_skin()
{
    /* 按钮控件 (针对没有通过 BUTTON_DrawSkin 函数绘制皮肤的按钮) */
    BUTTON_GetSkinFlexProps(&ButtonSkin, BUTTON_SKINFLEX_PI_ENABLED);
    ButtonSkin.aColorFrame[0] = SkinColor[ColorIndex].Color2;
    ButtonSkin.aColorFrame[1] = SkinColor[ColorIndex].Color2;
    ButtonSkin.aColorFrame[2] = SkinColor[ColorIndex].Color2;
    ButtonSkin.aColorUpper[0] = SkinColor[ColorIndex].Color2;
    ButtonSkin.aColorUpper[1] = SkinColor[ColorIndex].Color2;
    ButtonSkin.aColorLower[0] = SkinColor[ColorIndex].Color2;
}

```



```

ButtonSkin.aColorLower[1] = SkinColor[ColorIndex].Color2;
BUTTON_SetSkinFlexProps(&ButtonSkin, BUTTON_SKINFLEX_PI_ENABLED);

BUTTON_GetSkinFlexProps(&ButtonSkin, BUTTON_SKINFLEX_PI_PRESSED);
ButtonSkin.aColorFrame[0] = SkinColor[ColorIndex].Color2;
ButtonSkin.aColorFrame[1] = SkinColor[ColorIndex].Color2;
ButtonSkin.aColorFrame[2] = SkinColor[ColorIndex].Color3;
ButtonSkin.aColorUpper[0] = SkinColor[ColorIndex].Color3;
ButtonSkin.aColorUpper[1] = SkinColor[ColorIndex].Color3;
ButtonSkin.aColorLower[0] = SkinColor[ColorIndex].Color3;
ButtonSkin.aColorLower[1] = SkinColor[ColorIndex].Color3;
BUTTON_SetSkinFlexProps(&ButtonSkin, BUTTON_SKINFLEX_PI_PRESSED);

/* 单选控件 */
RADIO_GetSkinFlexProps(&RadioSkin, RADIO_SKINFLEX_PI_UNPRESSED);
RadioSkin.aColorButton[0] = SkinColor[ColorIndex].Color1;
RadioSkin.aColorButton[1] = SkinColor[ColorIndex].Color1;
RadioSkin.aColorButton[2] = SkinColor[ColorIndex].Color1;
RadioSkin.aColorButton[3] = SkinColor[ColorIndex].Color1;
RadioSkin.ButtonSize = 10;
RADIO_SetSkinFlexProps(&RadioSkin, RADIO_SKINFLEX_PI_UNPRESSED);

RADIO_GetSkinFlexProps(&RadioSkin, RADIO_SKINFLEX_PI_PRESSED);
RadioSkin.aColorButton[0] = SkinColor[ColorIndex].Color2;
RadioSkin.aColorButton[1] = SkinColor[ColorIndex].Color2;
RadioSkin.aColorButton[2] = SkinColor[ColorIndex].Color2;
RadioSkin.ButtonSize = 10;
RADIO_SetSkinFlexProps(&RadioSkin, RADIO_SKINFLEX_PI_PRESSED);

/* 升降按钮控件(emWin 错误:Disabled 实际上是设置 Pressed 的皮肤) */
SPINBOX_GetSkinFlexProps(&SpinBoxSkin, SPINBOX_CI_DISABLED);
SpinBoxSkin.aColorFrame[0] = SkinColor[ColorIndex].Color2;
SpinBoxSkin.aColorFrame[1] = SkinColor[ColorIndex].Color2;
SpinBoxSkin.aColorUpper[0] = SkinColor[ColorIndex].Color3;
SpinBoxSkin.aColorUpper[1] = SkinColor[ColorIndex].Color3;
SpinBoxSkin.aColorLower[0] = SkinColor[ColorIndex].Color3;
SpinBoxSkin.aColorLower[1] = SkinColor[ColorIndex].Color3;
SpinBoxSkin.ColorArrow = SkinColor[ColorIndex].Color5;
SpinBoxSkin.ColorBk = SkinColor[ColorIndex].Color5;
SpinBoxSkin.ColorText = SkinColor[ColorIndex].Color2;
SPINBOX_SetSkinFlexProps(&SpinBoxSkin, SPINBOX_CI_DISABLED);

SPINBOX_GetSkinFlexProps(&SpinBoxSkin, SPINBOX_CI_ENABLED);
    
```

```

SpinBoxSkin.aColorFrame[0] = SkinColor[ColorIndex].Color2;
SpinBoxSkin.aColorFrame[1] = SkinColor[ColorIndex].Color2;
SpinBoxSkin.aColorUpper[0] = SkinColor[ColorIndex].Color2;
SpinBoxSkin.aColorUpper[1] = SkinColor[ColorIndex].Color2;
SpinBoxSkin.aColorLower[0] = SkinColor[ColorIndex].Color2;
SpinBoxSkin.aColorLower[1] = SkinColor[ColorIndex].Color2;
SpinBoxSkin.ColorArrow = SkinColor[ColorIndex].Color5;
SpinBoxSkin.ColorBk = SkinColor[ColorIndex].Color5;
SpinBoxSkin.ColorText = SkinColor[ColorIndex].Color2;
SPINBOX_SetSkinFlexProps(&SpinBoxSkin, SPINBOX_CI_ENABLED);

SPINBOX_GetSkinFlexProps(&SpinBoxSkin, SPINBOX_CI_PRESSED);
SpinBoxSkin.aColorFrame[0] = SkinColor[ColorIndex].Color2;
SpinBoxSkin.aColorFrame[1] = SkinColor[ColorIndex].Color2;
SpinBoxSkin.aColorUpper[0] = SkinColor[ColorIndex].Color2;
SpinBoxSkin.aColorUpper[1] = SkinColor[ColorIndex].Color2;
SpinBoxSkin.aColorLower[0] = SkinColor[ColorIndex].Color2;
SpinBoxSkin.aColorLower[1] = SkinColor[ColorIndex].Color2;
SpinBoxSkin.ColorArrow = SkinColor[ColorIndex].Color5;
SpinBoxSkin.ColorBk = SkinColor[ColorIndex].Color5;
SpinBoxSkin.ColorText = SkinColor[ColorIndex].Color2;
SPINBOX_SetSkinFlexProps(&SpinBoxSkin, SPINBOX_CI_PRESSED);

/* 滑块控件 */
SLIDER_GetSkinFlexProps(&SliderSkin, SLIDER_SKINFLEX_PI_UNPRESSED);
SliderSkin.aColorFrame[0] = SkinColor[ColorIndex].Color2;
SliderSkin.aColorFrame[1] = SkinColor[ColorIndex].Color2;
SliderSkin.aColorInner[0] = SkinColor[ColorIndex].Color2;
SliderSkin.aColorInner[1] = SkinColor[ColorIndex].Color2;
SliderSkin.aColorShaft[0] = SkinColor[ColorIndex].Color1;
SliderSkin.aColorShaft[1] = SkinColor[ColorIndex].Color1;
SliderSkin.aColorShaft[2] = SkinColor[ColorIndex].Color2;
SLIDER_SetSkinFlexProps(&SliderSkin, SLIDER_SKINFLEX_PI_UNPRESSED);

SLIDER_GetSkinFlexProps(&SliderSkin, SLIDER_SKINFLEX_PI_PRESSED);
SliderSkin.aColorFrame[0] = SkinColor[ColorIndex].Color3;
SliderSkin.aColorFrame[1] = SkinColor[ColorIndex].Color3;
SliderSkin.aColorInner[0] = SkinColor[ColorIndex].Color3;
SliderSkin.aColorInner[1] = SkinColor[ColorIndex].Color3;
SliderSkin.aColorShaft[0] = SkinColor[ColorIndex].Color1;
SliderSkin.aColorShaft[1] = SkinColor[ColorIndex].Color1;
SliderSkin.aColorShaft[2] = SkinColor[ColorIndex].Color2;
SLIDER_SetSkinFlexProps(&SliderSkin, SLIDER_SKINFLEX_PI_PRESSED);
    
```

```

/* 框架窗口 */
FRAMEWIN_GetSkinFlexProps(&FramewinSkin, FRAMEWIN_CI_ACTIVE);
FramewinSkin.aColorTitle[0] = SkinColor[ColorIndex].Color3;
FramewinSkin.aColorTitle[1] = SkinColor[ColorIndex].Color3;
FramewinSkin.aColorFrame[0] = SkinColor[ColorIndex].Color3;
FramewinSkin.aColorFrame[1] = SkinColor[ColorIndex].Color3;
FramewinSkin.aColorFrame[2] = SkinColor[ColorIndex].Color3;
FRAMEWIN_SetSkinFlexProps(&FramewinSkin, FRAMEWIN_CI_ACTIVE);

FRAMEWIN_GetSkinFlexProps(&FramewinSkin, FRAMEWIN_CI_INACTIVE);
FramewinSkin.aColorTitle[0] = SkinColor[ColorIndex].Color2;
FramewinSkin.aColorTitle[1] = SkinColor[ColorIndex].Color2;
FramewinSkin.aColorFrame[0] = SkinColor[ColorIndex].Color2;
FramewinSkin.aColorFrame[1] = SkinColor[ColorIndex].Color2;
FramewinSkin.aColorFrame[2] = SkinColor[ColorIndex].Color2;
FRAMEWIN_SetSkinFlexProps(&FramewinSkin, FRAMEWIN_CI_INACTIVE);
}

```

(5) 对话框的回调函数

```

/**
 * @brief      回调函数
 * @param      pMsg : 消息
 * @retval     无
 */
static void _cbCallback(WM_MESSAGE *pMsg)
{
    WM_HWIN hItem;
    int      NCode;
    int      Id;
    int      Value;
    WM_HWIN  hDlg;

    switch (pMsg->MsgId)
    {
        case WM_INIT_DIALOG:
            hItem = pMsg->hWin;
            /* 获取 ID_GRAPH_0 控件的句柄 */
            hDlg = WM_GetDialogItem(hItem, ID_GRAPH_0);
            /* 小控件的初始化代码, 已省略 .....*/
        case WM_NOTIFY_PARENT:
            Id      = WM_GetId(pMsg->hWinSrc);
            NCode   = pMsg->Data.v;
    }
}

```

```

switch (Id)
{
    case ID_SLIDER_0:
        switch (NCode)
        {
            case WM_NOTIFICATION_VALUE_CHANGED:
                hItem = WM_GetDialogItem(pMsg->hWin, ID_GRAPH_0);
                Value = SLIDER_GetValue(pMsg->hWinSrc) * 10;
                GRAPH_SetGridDistX(hItem, Value);
                GRAPH_SCALE_SetTickDist(hScaleH, Value);
                break;
        }
        break;

    case ID_SLIDER_1:
        switch (NCode)
        {
            case WM_NOTIFICATION_VALUE_CHANGED:
                hItem = WM_GetDialogItem(pMsg->hWin, ID_GRAPH_0);
                Value = SLIDER_GetValue(pMsg->hWinSrc) * 5;
                GRAPH_SetGridDistY(hItem, Value);
                GRAPH_SCALE_SetTickDist(hScaleV, Value);
                break;
        }
        break;

    case ID_CHECKBOX_0:
        switch (NCode)
        {
            case WM_NOTIFICATION_VALUE_CHANGED:
                Craph_Stop ^= 1;
                break;
        }
        break;

    case ID_CHECKBOX_1:
        switch (NCode)
        {
            case WM_NOTIFICATION_VALUE_CHANGED:
                hItem = WM_GetDialogItem(pMsg->hWin, ID_GRAPH_0);

                if (CHECKBOX_IsChecked(WM_GetDialogItem(pMsg->hWin,
                                                            ID_CHECKBOX_1)))

```

```

        {
            GRAPH_SetVSizeX(hItem, 500);
        }
        else
        {
            GRAPH_SetVSizeX(hItem, 0);
        }
        break;
    }
    break;

case ID_CHECKBOX_2:
    switch (NCode)
    {
        case WM_NOTIFICATION_VALUE_CHANGED:
            hItem = WM_GetDialogItem(pMsg->hWin, ID_GRAPH_0);
            GRAPH_SetGridVis(hItem, CHECKBOX_IsChecked
                (WM_GetDialogItem(pMsg->hWin, ID_CHECKBOX_2)));
            break;
    }
    break;

case ID_CHECKBOX_3:
    switch (NCode)
    {
        case WM_NOTIFICATION_VALUE_CHANGED:
            hItem = WM_GetDialogItem(pMsg->hWin, ID_GRAPH_0);

            if (CHECKBOX_IsChecked(WM_GetDialogItem(pMsg->hWin,
                ID_CHECKBOX_3)))
            {
                GRAPH_SetVSizeY(hItem, 400);
            }
            else
            {
                GRAPH_SetVSizeY(hItem, 0);
            }
            break;
    }
    break;

case ID_CHECKBOX_4:
case ID_CHECKBOX_5:

```

```

case ID_CHECKBOX_6:
case ID_CHECKBOX_7:
    switch (NCode)
    {
        case WM_NOTIFICATION_VALUE_CHANGED:
            hItem = WM_GetDialogItem(pMsg->hWin, ID_GRAPH_0);
            GRAPH_SetBorder(hItem,
                CHECKBOX_IsChecked(WM_GetDialogItem(pMsg->hWin,
                    ID_CHECKBOX_4)) * 40,
                CHECKBOX_IsChecked(WM_GetDialogItem(pMsg->hWin,
                    ID_CHECKBOX_5)) * 5,
                CHECKBOX_IsChecked(WM_GetDialogItem(pMsg->hWin,
                    ID_CHECKBOX_6)) * 5,
                CHECKBOX_IsChecked(WM_GetDialogItem(pMsg->hWin,
                    ID_CHECKBOX_7)) * 5);
            break;
    }
    break;

case ID_RADIO_0:
    switch (NCode)
    {
        case WM_NOTIFICATION_VALUE_CHANGED:
            hItem = WM_GetDialogItem(pMsg->hWin, ID_GRAPH_0);

            switch (RADIO_GetValue(pMsg->hWinSrc))
            {
                case 0:
                    WIDGET_SetEffect(hItem, &WIDGET_Effect_3D);
                    break;

                case 1:
                    WIDGET_SetEffect(hItem,
                        &WIDGET_Effect_Simple);
                    break;

                case 2:
                    WIDGET_SetEffect(hItem, &WIDGET_Effect_None);
                    break;
            }
            break;
    }
    break;

```

```

case ID_BUTTON_1:
    switch (NCode)
    {
        case WM_NOTIFICATION_RELEASED:
            /* 返回对话的对象句柄 */
            hItem = WM_GetDialogItem(pMsg->hWin, ID_GRAPH_0);

            if (color > 3)
            {
                color = 0;
            }
            /*设置 GRAPH 背景颜色 */
            GRAPH_SetColor(hItem, _aColor[color], GRAPH_CI_BK);
            color ++;
            break;
    }
    break;

case ID_BUTTON_2:
    switch (NCode)
    {
        case WM_NOTIFICATION_RELEASED:
            oscilldev.div -= 30;

            if (oscilldev.div < 10) oscilldev.div = 10;

            __HAL_TIM_DISABLE(&timx_handler);/* TIM6 失能 */
            btim_timx_int_init(oscilldev.div, 72 - 1);
            break;
    }
    break;

case ID_BUTTON_3:
    switch (NCode)
    {
        case WM_NOTIFICATION_RELEASED:
            oscilldev.div += 30;

            if (oscilldev.div > 1000)oscilldev.div = 100;

            __HAL_TIM_DISABLE(&timx_handler);/* TIM6 失能 */
            btim_timx_int_init(oscilldev.div, 72 - 1);
    }
    break;

```

```

        break;
    }
    break;

case ID_CHECKBOX_8:
    switch (NCode)
    {
        case WM_NOTIFICATION_VALUE_CHANGED:
            WM_GetDlgItem(pMsg->hWin, ID_GRAPH_0);
            if (CHECKBOX_IsChecked(
                WM_GetDlgItem(pMsg->hWin, ID_CHECKBOX_8)))
            {
                GRAPH_DATA_YT_SetAlign(oscilldev.graphdata,
                                        GRAPH_ALIGN_LEFT);
                GRAPH_DATA_YT_MirrorX (oscilldev.graphdata,
                                        1);
            }
            else
            {
                GRAPH_DATA_YT_SetAlign(oscilldev.graphdata,
                                        GRAPH_ALIGN_RIGHT);
                GRAPH_DATA_YT_MirrorX (oscilldev.graphdata,
                                        0);
            }
            break;
        }
    }
    break;

default:
    WM_DefaultProc(pMsg);
    break;
}
}

```

从上述源码可知：当用户触发不同的小控件时，系统会根据用户触发的小工具执行相应的动作。

(6) 数值显示到 GRAPH 对象中

```

/**
 * @brief      这个例程根据以前的添加值计算新的值
               并将它们添加到 GRAPH_DATA 对象中
 * @param      无

```



```

    * @retval      无
    */
static void emwin_add_values(void)
{
    int i;

    if (oscilldev.adflag == 1)
    {
        for (i = 0; i < oscilldev.dataxsize; i++)
        {
            /* 向 GRAPH 图形小工具添加数据,注意要缩 20 倍,因为垂直坐标扩大(缩放)20 倍,
            并且还要加 200,因为垂直坐标上移了 200 个像素点 */
            GRAPH_DATA_YT_AddValue(oscilldev.graphdata,
                                    oscilldev.buffer[i] 50 + 80);
        }

        memset(oscilldev.buffer, 0, oscilldev.dataxsize); /* 清空 buffer */
        oscilldev.adflag = 0;
    }
}

```

此函数主要把 oscilldev.buffer 的数据加载到 GRAPH 图形小工具当中。

(7) 定时器设置

```

/**
 * @brief      回调函数,定时器中断服务函数调用
 * @param      无
 * @retval      无
 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    static int count=0;

    if (htim == (&timx_handler))
    {
        /* buffer 为空,并且允许进行 AD 采集 */
        if((oscilldev.buffer!=NULL) && (oscilldev.adflag==0))
        {
            /* 电压值,因为单位 mv */
            oscilldev.buffer[count] = (int)((float)adc_get_result(ADC_ADCX_CHY)
                                             *oscilldev.resolut);

            count++;

            if(count > oscilldev.dataxsize) /* 一次数据采集完成 */
            {

```

```
count = 0;
oscilldev.adflag = 1;      /* 标记 AD 采集完成 */
}
}
}
}
```

注意！我们用的是定时器 6 的中断服务函数，采集过程很简单，采集 600 个 AD 数据保存到 oscilldev.buffer 中供 GRAPH 使用，到这里例程就讲解完了。

20.3.3 下载验证

编译并下载例程到开发板中，LCD 显示如图 20.3.3.1 所示：

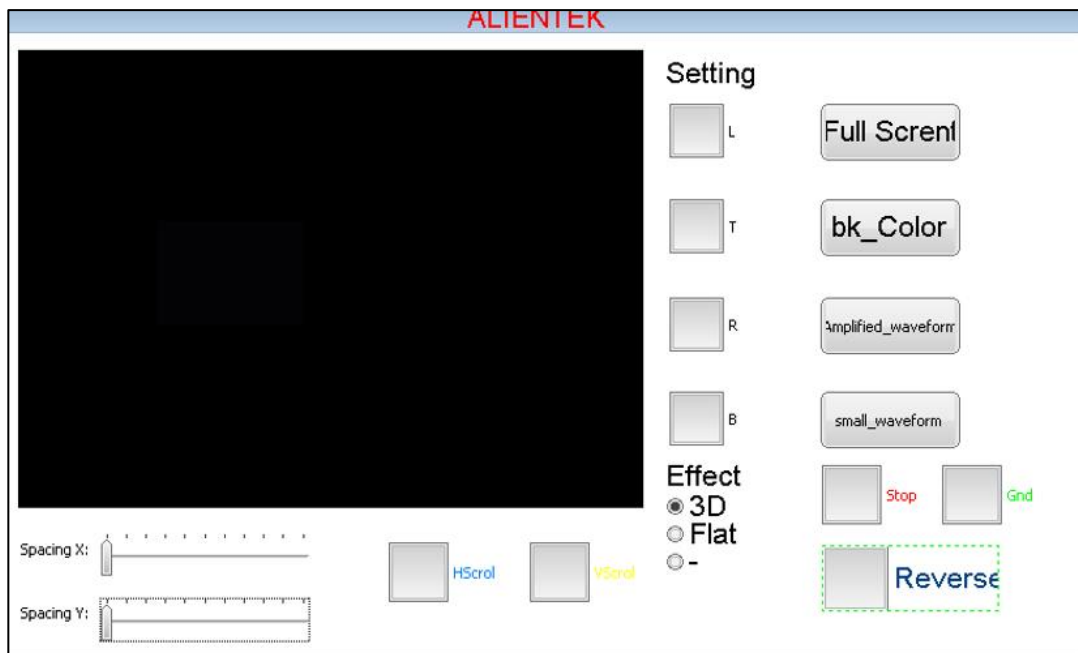


图 20.3.3.1 LCD 显示 GRAPH 控件

使用杜邦线把 PA1 与 PA4 连接起来，显示正弦波，如果杜邦线 PA1 与 PA8 连接，我们得显示方波。

第二十一章 HEADER 控件

有时候我们需要用表格来统计和显示数据，在用表格显示数据的时候肯定是需要表头来标记表格的每一栏是什么东西，emWin 提供了这样的一个控件来标记表格的列的，这个控件叫做 HEADER 控件，本章我们就来学习一下 HEADER 控件的使用方法。

本章分为以下几个部分：

21.1 HEADER 控件简介

21.2 通知代码

21.3 HEADER 控件 API 函数

21.4 HEADER 控件演示实验

21.1 HEADER 控件简介

emWin 的 HEADER 控件用来标记表格的列，HEADER 控件示意图如下图 21.1.1 所示：



图 21.1.1 HEADER 控件

控件中的宽度可以调节的，如果使用了触摸屏的话就可以在触摸屏上滑动来调整各列的宽度，当光标出现并且在分隔线附近的时候光标就会变成信号，这个时候就可以在当前位置拖动分隔线了，如果使用了表格的话表格的宽度也会相应的改变，注意：不仅仅是 HEADER 控件的宽度改变。

在 emWin 中，每一个控件都可以设置皮肤，HEADER 控件皮肤设置效果图如下图所示：



图 21.1.2 HEADER 控件皮肤设置

21.2 通知代码

HEADER 小工具会作为 WM_NOTIFY_PARENT 消息的一部分发送给父窗口，通知代码如下表所示：

消息	描述
WM_NOTIFICATION_CLICKED	以单击小工具
WM_NOTIFICATION_RELEASED	以释放小工具
WM_NOTIFICATION_MOVED_OUT	以单击小工具，并且指针已经移出小工具，有释放

表 21.2.1 HEADER 通知代码

21.3 HEADER 控件 API 函数

表 21.3.1 按字母顺序列出了可用的 HEADER 控件相关 API 函数：

函数	描述
HEADER_AddItem()	在右侧增加一个项目
HEADER_Create()	创建 HEADER 小工具（弃用）
HEADER_CreateAttached()	创建附加于窗口的 HEADER 小工具
HEADER_CreateEx()	创建 HEADER 小工具

HEADER_CreateIndirect()	从资源表项创建 HEADER 小工具
HEADER_CreateUser()	使用额外字节作为用户数据创建 HEADER 小工具
HEADER_GetDefaultBkColor()	返回默认背景色
HEADER_GetDefaultBorderH()	返回水平间距值
HEADER_GetDefaultBorderV()	返回垂直间距值
HEADER_GetDefaultCursor()	返回指向默认光标的指针
HEADER_GetDefaultFont()	返回指向默认字体的指针
HEADER_GetDefaultTextColor()	返回默认文本颜色
HEADER_GetHeight()	返回小工具的高度
HEADER_GetItemWidth()	返回项目的宽度
HEADER_GetNumItems()	返回项目数
HEADER_GetUserData()	检索使用 HEADER_SetUserData 设置的数据
HEADER_SetBitmap()	设置显示给定项目时使用的位图
HEADER_SetBitmapEx()	设置显示给定项目时使用的位图
HEADER_SetBkColor()	设置小工具的背景色
HEADER_SetBMP()	设置显示给定项目时使用的位图
HEADER_SetBMPEX()	设置显示给定项目时使用的位图
HEADER_SetDefaultBkColor()	设置默认背景色
HEADER_SetDefaultBorderH()	设置水平间距的默认值
HEADER_SetDefaultBorderV()	设置垂直间距的默认值
HEADER_SetDefaultCursor()	设置默认光标
HEADER_SetDefaultFont()	设置默认字体
HEADER_SetDefaultTextColor()	设置默认文本颜色
HEADER_SetDragLimit()	将拖动项目限制设置为开或关
HEADER_SetFont()	设置小工具的字体
HEADER_SetHeight()	设置小工具的高度
HEADER_SetItemText()	设置给定项目的文本
HEADER_SetItemWidth()	设置给定项目的宽度
HEADER_SetStreamedBitmap()	设置显示给定项目时使用的位图
HEADER_SetStreamedBitmapEx()	设置显示给定项目时使用的位图
HEADER_SetTextAlign()	设置给定项目的对齐方式
HEADER_SetTextColor()	设置小工具的文本颜色
HEADER_SetUserData()	设置 HEADER 小工具的额外数据

表 21.3.1 HEADER 控件 API 函数

HEADER 控件的 API 函数都比较简单，可以看一下 emWin 官方的手册，再者 HEADER 控件很少单独拿出来用，一般都是作为 LISTVIEW 控件的一部分来使用的。

21.4 HEADER 控件演示例程

21.4.1 硬件设计

1. 例程功能

本实验主要讲解 HEADER 小工具的使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数主要创建一个 window 界面并在其上面添加 HEADER 控件，我们在 HEADER 控件上添加了三个菜单栏，它们分别是“Red”、“Green”以及“Blue”菜单栏，当我们滑动某一个菜单栏时，它们下方的颜色框跟着菜单栏缩放。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 22 HEADER 小工具》。

21.4.2 软件设计

21.4.2.1 程序流程图

本实验的程序流程图，如下图所示：

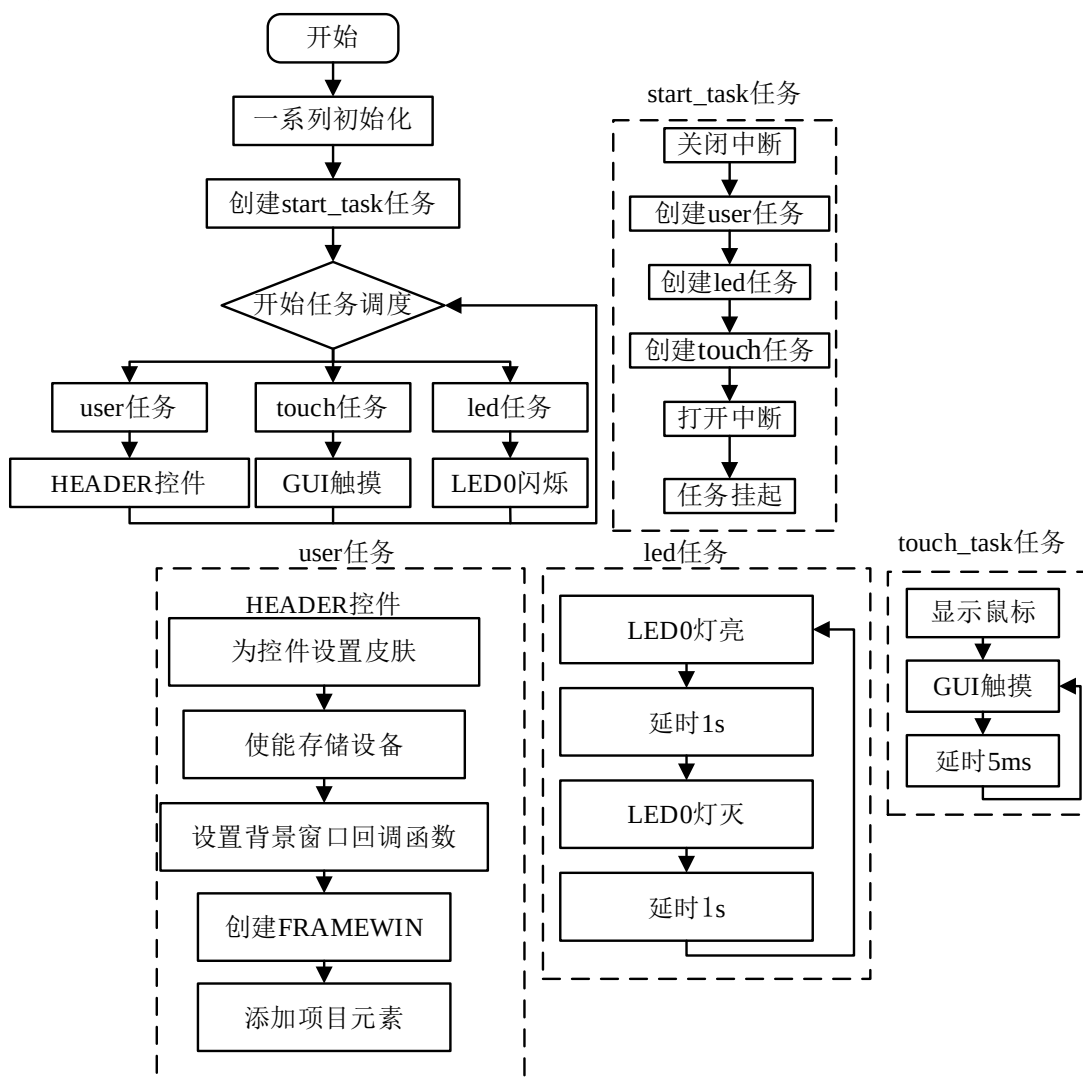


图 21.4.2.1.1 HEADER 控件实验流程图

21.4.2.2 程序解析

Segger 官方例程 WIDGET_Header.c 给我们演示了 HEADER 控件各个 API 函数的使用方法，我们可以将这个例程移植到开发板上来学习 HEADER 控件 API 函数的使用。打开 [emWin 实验](#)

22 HEADER 小工具工程中的文件 emwin_demo.c，函数 emwin_main 就是本章的演示函数，代码如下：

```
/**
 * @brief      emwin_main(emwin 入口)
 * @param      无
 * @retval     无
 */
void emwin_main (void)
{
    WM_EnableMemdev(WM_HBKWIN);          /* 使能存储设备 */
    WM_SetCreateFlags(WM_CF_MEMDEV);
    WM_SetCallback(WM_HBKWIN, _cbBkWindow); /* 设置背景窗口回调函数 */
    GUI_CURSOR_Show();

    while (1)                            /* Demo 演示*/
    {
        emwin_header_frameWin();
        emwin_headerWin();
        emwin_header();
    }
}
```

函数 emwin_main 里面的 while() 循环里面的三个函数就是 HEADER 的演示例程，分别为在 FrameWin 中使用 HEADER、在 Window 中使用 HEADER 和单独使用 HEADER 小控件，这三个函数如下所示：

```
/**
 * @brief      在 FRAMEWIN 控件中使用 header
 * @param      无
 * @retval     无
 */
static void emwin_header_frameWin(void)
{
    FRAMEWIN_Handle hFrameWin;
    emwin_change_main_text("HEADER control inside a FRAMEWIN");
    /* 创建 FRAMEWIN */
    hFrameWin = FRAMEWIN_Create("Title", _cbWindow, WM_CF_SHOW, 10, 80, 300, 140);
    FRAMEWIN_SetActive(hFrameWin, 1); /* 激活 FRAMEWIN */
    /* 创建附加于 FRAMEWIN 的 HEADER 小工具 */
    _hHeader = HEADER_CreateAttached(WM_GetClientWindow(hFrameWin), 1234, 0);
    emwin_demo();
    WM_DeleteWindow(hFrameWin);
}

/**
```

```

* @brief      在窗口中使用 header
* @param      无
* @retval     无
*/
static void emiwn_header_win(void)
{
    WM_HWIN hWin;
    emwin_change_main_text("HEADER control inside a window");
    /* 创建窗口 */
    hWin = WM_CreateWindow(10, 80, 300, 140, WM_CF_SHOW, _cbWindow, 0);
    /* 创建附加于窗口的 HEADER 小工具 */
    _hHeader = HEADER_CreateAttached(hWin, 1234, 0);
    emwin_demo();
    WM_DeleteWindow(hWin); /* 删除窗口 */
}

/**
* @brief      直接使用 HEADER
* @param      无
* @retval     无
*/
static void emwin_header(void)
{
    emwin_change_main_text("HEADER control without parent");
    /* 创建 HEADER */
    _hHeader = HEADER_Create(10, 80, 300, 0, 0, 1234, WM_CF_SHOW, 0);
    emiwn_demo();
    WM_DeleteWindow(_hHeader);
}

```

从上面的三个函数可以看出，这三个演示例程本质上都是调用函数 `emwin_demo` 来演示 HEADER 的 API 函数，不同之处就是在 FrameWin 或者 Window 中使用 HEADER 的话需要创建 FrameWin 或者 Window 小控件，然后 HEADER 作为 FrameWin 或 Window 的子窗口使用，最后调用函数 `emwin_demo` 来演示，该函数如下所示：

```

/**
* @brief      演示
* @param      无
* @retval     无
*/
static void emwin_demo(void)
{
    int Cnt = 10;
    char acInfoText[] = "-- sec to play with header control";
    emwin_change_main_text("HEADER_ADDItem");
}

```

```

/* 添加项目 Red */
HEADER_AddItem(_hHeader, 100, "Red", GUI_TA_VCENTER | GUI_TA_HCENTER);
/* 添加项目 Green */
HEADER_AddItem(_hHeader, 0, "Green", GUI_TA_VCENTER | GUI_TA_HCENTER);
/* 添加项目 :- */
HEADER_AddItem(_hHeader, 0, ":-", GUI_TA_VCENTER | GUI_TA_HCENTER);
GUI_Delay(750);

emwin_change_main_text("HEADER_SetItemWidth");          /* 调整项目 1 宽度 */
HEADER_SetItemWidth(_hHeader, 1, 60);
GUI_Delay(750);

emwin_change_main_text("HEADER_SetItemText");           /* 调整项目 2 宽度 */
HEADER_SetItemWidth(_hHeader, 2, 100);
HEADER_SetItemText(_hHeader, 2, "Blue");
GUI_Delay(750);

emwin_change_main_text("HEADER_SetFont");
HEADER_SetFont(_hHeader, &GUI_Font8x8);                /* 设置字体 */
GUI_Delay(750);

emwin_change_main_text("HEADER_SetHeight");             /* 设置高度 */
HEADER_SetHeight(_hHeader, 50);
GUI_Delay(750);

emwin_change_main_text ("HEADER_SetTextColor");
HEADER_SetTextColor(_hHeader, GUI_YELLOW);              /* 设置字体颜色 */
GUI_Delay(750);

emwin_change_main_text("HEADER_SetBkColor");
HEADER_SetBkColor(_hHeader, GUI_DARKGRAY);              /* 设置背景颜色 */
GUI_Delay(750);
emwin_change_main_text("HEADER_SetTextAlign");
HEADER_SetTextAlign(_hHeader, 0, GUI_TA_HCENTER);       /* 设置对齐方式 */

while (Cnt > 0)
{
    acInfoText[0] = '0' + (Cnt / 10);
    acInfoText[1] = '0' + (Cnt-- % 10);
    emwin_change_main_text(acInfoText);
    GUI_Delay(1000);
}
}
    
```


可以看出 `emwin_demo()` 调用了很多跟 HEADER 控件有关的 API 函数，这些函数都比较简单，`emwin_demo.c` 文件中的其他函数就不讲解了，剩下的都是一些绘制、回调函数之类的，大家自行看一下。

21.4.3 下载验证

编译并下载例程到开发板中，LCD 显示如下图所示：

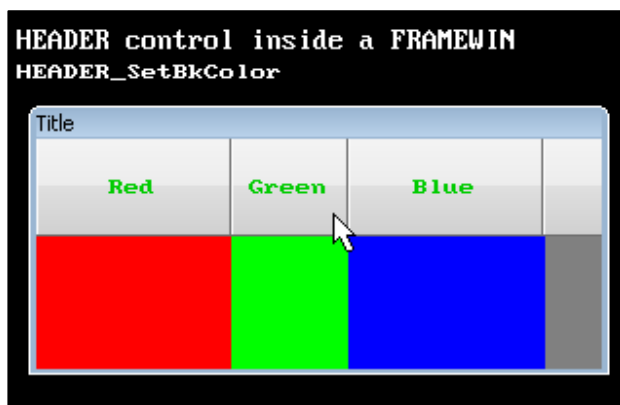


图 21.4.3.1 LCD 显示结果

第二十二章 ICONVIEW 控件

我们在使用电脑或者手机的时候可以通过在桌面上点击指定的图标打开我们要用的 APP，使用起来非常方便。emWin 中也有这样的控件来完成打开不同 APP 的功能，它就是 ICONVIEW 控件，可以通过选择不同的图标打开不同的 APP，本章我们就学习 ICONVIEW 控件的使用。

本章分为以下几个部分：

22.1 ICONVIEW 控件简介

22.2 通知代码

22.3 ICONVIEW 控件 API 函数

22.4 ICONVIEW 控件演示例程

22.1 ICONVIEW 图标视图小部件简介

ICONVIEW 控件也叫图标视图控件，在智能手机、PDA 等等这类手持设备上常常需要用到这种控件，它显示一系列的图标，每个图标都有文本标注。ICONVIEW 控件支持透明和 Alpha 混合处理，这样它的背景就可以显示其他的信息，比如背景图片。当选中图标会用纯色或 Alpha 混合后的颜色突出显示，ICONVIEW 控件也支持滚动条，这样当所有图标不能一次都显示出的时候就可以使用滚动条，ICONVIEW 控件如下图所示：

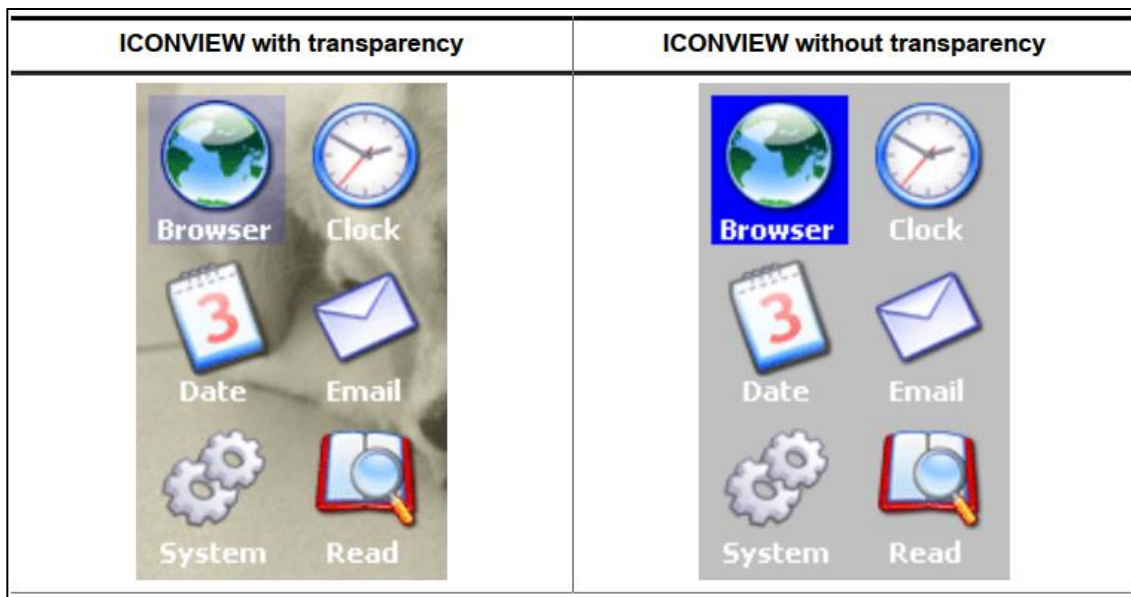


图 22.1.1 ICONVIEW 控件

下面我们来讲解一下 ICONVIEW 控件配置选项和预定义 ID 的知识点。

1. 配置选项

ICONVIEW 控件的配置选项主要根据宏定义设置该控件的选项，如下表所示：

宏指令	默认	描述
ICONVIEW_BKCOLOR0_DEFAULT	GUI_WHITE	背景颜色，未选中状态
ICONVIEW_BKCOLOR1_DEFAULT	GUI_BLUE	背景颜色，选定状态
ICONVIEW_TEXTCOLOR0_DEFAULT	GUI_WHITE	文本颜色，未选择的状态
ICONVIEW_TEXTCOLOR1_DEFAULT	GUI_WHITE	文本颜色，选择的状态

ICONVIEW_FONT_DEFAULT	GUI_Font13_1	用于绘制标签的字体
ICONVIEW_FRAMEX_DEFAULT	5	图标之间和小部件的左右边框之间的空闲空间
ICONVIEW_FRAMEY_DEFAULT	5	图标和小部件的顶部和底部边界之间的空闲空间
ICONVIEW_SPACEX_DEFAULT	5	图标之间的自由水平空间
ICONVIEW_SPACEY_DEFAULT	5	图标之间的自由垂直空间
ICONVIEW_ALIGN_DEFAULT	GUI_TA_HCENTER GUI_TA_BOTTOM	用于绘制标签的默认对齐方式

表 22.1.1 ICONVIEW 控件的配置选项

2. 预定义的 ID

下面的符号定义了可以用来区分 ICONVIEW 小控件的 id:

```
#define GUI_ID_ICONVIEW0 0x250
#define GUI_ID_ICONVIEW1 0x251
#define GUI_ID_ICONVIEW2 0x252
#define GUI_ID_ICONVIEW3 0x253
#define GUI_ID_ICONVIEW4 0x254
#define GUI_ID_ICONVIEW5 0x255
#define GUI_ID_ICONVIEW6 0x256
#define GUI_ID_ICONVIEW7 0x257
#define GUI_ID_ICONVIEW8 0x258
```

从上述可知: emWin 的 ICONVIEW 小控件的 ID 是从 0x250 开始, 如果用户想定义预定义 ID 可选择 0x250 之后的 ID。

22.2 通知代码

WM_NOTIFY_PARENT 消息的一部分, 它们是从 ICONVIEW 小部件发送到父窗口, 如下表所示:

消息	描述
WM_NOTIFICATION_CLICKED	小部件已被点击
WM_NOTIFICATION_CLICKED	小部件已经释放
WM_NOTIFICATION_CLICKED	小部件已被单击且已移出小部件区域没有释放
WM_NOTIFICATION_CLICKED	可选滚动条的滚动位置已被更改
WM_NOTIFICATION_SEL_CHANGED	小部件的选择已经更改

表 22.1.1 ICONVIEW 小部件消息类型

22.3 键盘的反应

如果小部件有输入焦点, 则它会对键作出哪些反应呢?, 如下表所示:

key	反应
GUI_KEY_RIGHT	将选择移动到下一个图标
GUI_KEY_LEFT	将所选内容移动到上一个图标
GUI_KEY_DOWN	向下移动选区
GUI_KEY_UP	向上移动选区
GUI_KEY_HOME	将选择移动到第一个图标
GUI_KEY_END	将选择移动到最后一个图标

表 22.3.1 ICONVIEW 小部件键盘反应

22.4 ICONVIEW 控件 API 函数

表 22.4.1 按字母顺序列出了可用的 ICONVIEW 控件相关 API 函数：

函数	描述
ICONVIEW_AddBitmapItem()	向小部件添加一个新的位图图标
ICONVIEW_AddStreamedBitmapItem()	向小部件添加一个新的流位图图标
ICONVIEW_CreateEx()	创建一个 ICONVIEW 小部件
ICONVIEW_CreateIndirect()	从资源表条目创建一个 ICONVIEW 小部件
ICONVIEW_CreateUser()	使用额外的字节作为用户数据创建一个 ICONVIEW 小部件
ICONVIEW_DeleteItem()	删除现有项
ICONVIEW_EnableStreamAuto()	向小部件添加一个新的流位图图标
ICONVIEW_GetBkColor()	返回小部件的背景颜色
ICONVIEW_GetFont()	返回小部件的字体
ICONVIEW_GetItemBitmap()	指定 ICONVIEW 项的位图
ICONVIEW_GetItemText()	指定 ICONVIEW 项的文本
ICONVIEW_GetItemUserData()	从特定项检索以前存储的用户数据
ICONVIEW_GetNumItems()	返回给定的 ICONVIEW 小部件
ICONVIEW_GetReleasedItem()	这个函数返回已发布的 ICONVIEW 项的索引
ICONVIEW_GetSel()	返回当前选中图标的以零为基础的索引
ICONVIEW_GetTextColor()	返回小部件的文本颜色
ICONVIEW_GetUserData()	使用 ICONVIEW_SetUserData 检索数据集
ICONVIEW_InsertBitmapItem()	向小部件插入一个新的位图图标
ICONVIEW_InsertStreamedBitmapItem()	向小部件插入一个新的流位图图标
ICONVIEW_OwnerDraw()	绘制 ICONVIEW 小部件的默认函数
ICONVIEW_SetBitmapItem()	设置特定对象使用的位图
ICONVIEW_SetBkColor()	设置小部件的背景颜色
ICONVIEW_SetFont()	设置用于绘制图标标签的字体
ICONVIEW_SetFrame()	设置小部件边界和图标之间的框架大小
ICONVIEW_SetIconAlign()	设置图标对齐方式
ICONVIEW_SetItemText()	设置特定项的文本
ICONVIEW_SetItemUserData()	将用户数据存储到特定的项中
ICONVIEW_SetOwnerDraw()	设置一个自定义的绘图函数
ICONVIEW_SetSel()	设置当前选择
ICONVIEW_SetSpace()	设置 x 或 y 方向上图标之间的空格
ICONVIEW_SetStreamedBitmapItem()	设置特定项目使用的流位图
ICONVIEW_SetTextAlign()	设置项目文本的文本对齐方式
ICONVIEW_SetTextColor()	设置用于绘制标签的颜色
ICONVIEW_SetUserData()	设置一个 ICONVIEW 小部件的额外数据
ICONVIEW_SetWrapMode()	设置用于给定 ICONVIEW 小部件的包装模式
组定义	
ICONVIEW_alignment flags	ICONVIEW_SetIconAlign 中使用的对齐标志
ICONVIEW_color indexes	ICONVIEW 小部件使用的颜色索引
ICONVIEW_create flags	创建用于 ICONVIEW 小部件的标志

表 22.4.1 ICONVIEW 控件 API 函数

ICONVIEW 控件的 API 函数比较多，我们讲解几个比较重点的函数：

(1) ICONVIEW_CreateEx 函数

在指定位置创建一个指定大小的 ICONVIEW 小部件，该函数的原型如下源码所示：

```
ICONVIEW_Handle ICONVIEW_CreateEx( int x0,int y0,
                                   int xSize,int ySize,
                                   WM_HWIN hParent,int WinFlags,
                                   int ExFlags,int Id,
                                   int xSizeItems,int ySizeItems);
```

该函数的相关形参如下表所示：

参数	描述
x0	小部件在父坐标中的最左边像素
y0	小部件在父坐标中的最顶部像素
xSize	小部件的水平大小(以像素为单位)
ySize	小部件的垂直大小(以像素为单位)
hParent	父窗口的句柄。如果 0，新的小部件将是桌面的一个子窗口(顶级窗口)
WinFlags	窗口创建的旗帜。通常是 WM_CF_SHOW
ExFlags	ICONVIEW create flags
Id	小部件的窗口 ID
xSizeItems	以像素为单位的水平图标大小
ySizeItems	垂直图标大小(像素)

表 22.4.2 ICONVIEW_CreateEx 函数的形参描述

返回值：新小部件的句柄，如果函数失败，则为 0。

(2) ICONVIEW_AddBitmapItem 函数

向小部件添加一个新的位图图标，如下图所示：

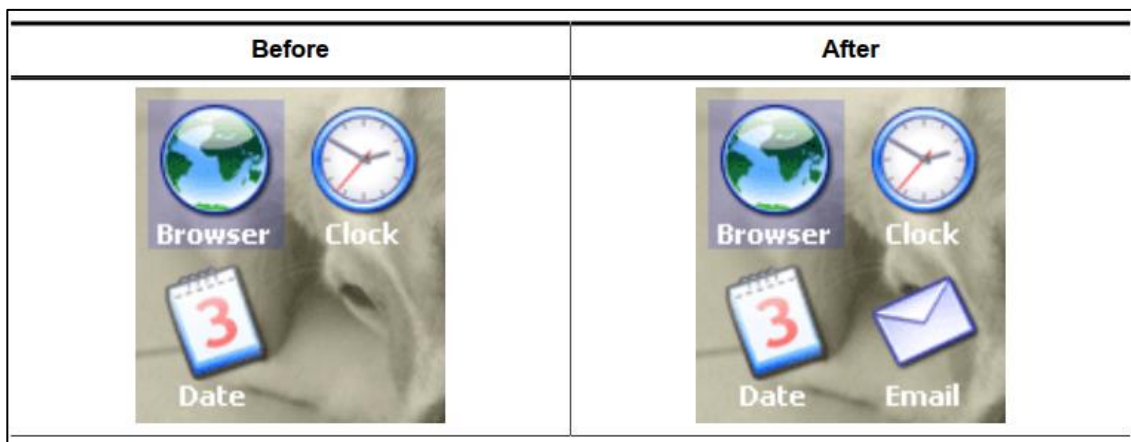


图 22.4.1 添加一个位图图标

该函数的原型如下源码所示：

```
int ICONVIEW_AddBitmapItem( ICONVIEW_Handle hObj, const GUI_BITMAP * pBitmap,
                           const char * pText);
```

该函数的相关形参如下表所示：

参数	描述
hObj	ICONVIEW 小部件的句柄
pBitmap	指向用于绘制图标的位图结构的指针
pText	用于标记图标的文本

表 22.4.3 ICONVIEW_AddBitmapItem 函数的形参描述

返回值：0：失败，非 0：成功。

注意：位图指针需要保持有效。

(3) ICONVIEW_SetIconAlign 函数

设置图标对齐方，如图 22.4.2 所示：

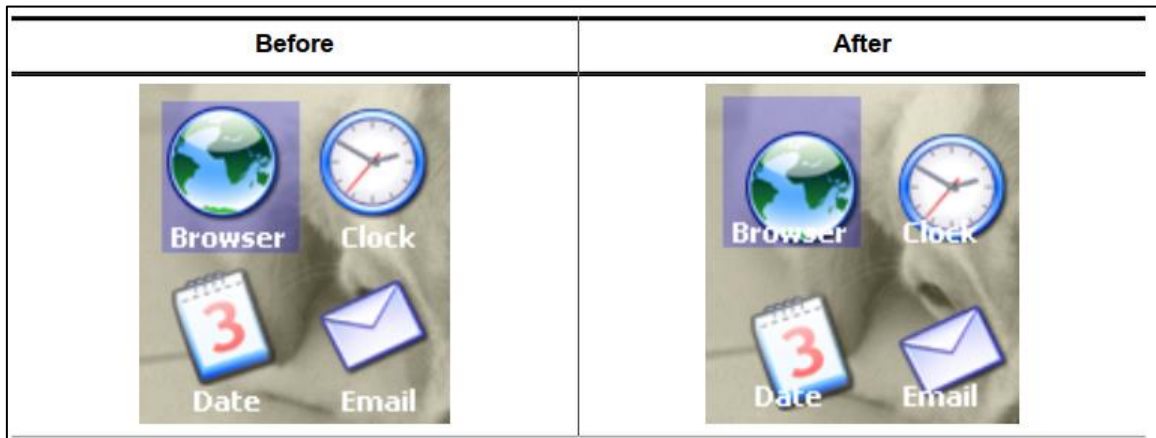


图 22.4.2 设置图标对齐

该函数的原型如下源码所示：

```
void ICONVIEW_SetIconAlign(ICONVIEW_Handle hObj, int IconAlign);
```

该函数的相关形参如下表所示：

参数	描述	
hObj	ICONVIEW 小部件的句柄	
IconAlign	水平对齐	
	ICONVIEW_IA_LEFT	坐标左对齐
	ICONVIEW_IA_HCENTER	圆心 X 坐标
	ICONVIEW_IA_RIGHT	对齐的右坐标
	垂直对齐	
	ICONVIEW_IA_TOP	与字符顶部对齐 y 位置
	ICONVIEW_IA_VCENTER	中心坐标 (默认)
	ICONVIEW_IA_BOTTOM	y 轴位置与字体的底部像素线对齐

表 22.4.4 ICONVIEW_SetIconAlign 函数的形参描述

(4) ICONVIEW_SetSpace 函数

设置 x 或 y 方向上图标之间的间距，如下图所示：

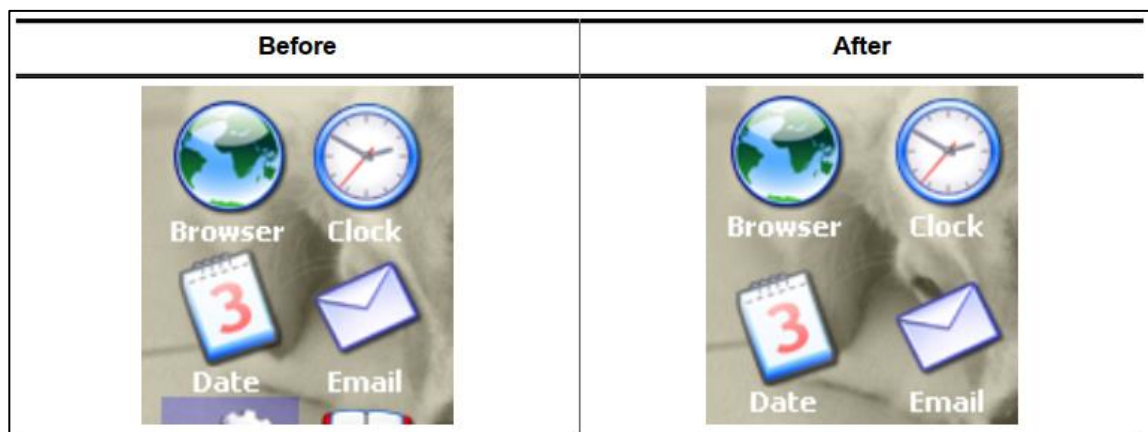


图 22.4.3 设置图标间的间距

该函数的原型如下源码所示：

```
void ICONVIEW_SetSpace(ICONVIEW_Handle hObj, int Coord, int Value);
```


该函数的相关形参如下表所示：

参数	描述	
hObj	ICONVIEW 小部件的句柄	
Coord	间距参数	
	GUI_COORD_X	x 轴方向
	GUI_COORD_Y	y 轴方向
Value	要设置的距离	

表 22.4.5 ICONVIEW_SetSpace 函数的形参描述

22.5 ICONVIEW 控件实验

22.5.1 硬件设计

1. 例程功能

本实验主要讲解 ICONVIEW 小工具的使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数主要创建一个 window 界面并在其上面添加多个 ICONVIEW 控件，当我们按下某一个图标时，系统弹出一个 window 界面，我们可以在这个界面下添加我们的功能。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 23 ICONVIEW 小工具》。

注意：本实验不支持 Mini、精英、战舰、DMF407、探索者开发板运行。

22.5.2 软件设计

22.5.2.1 程序流程图

本实验的程序流程图，如下图所示：

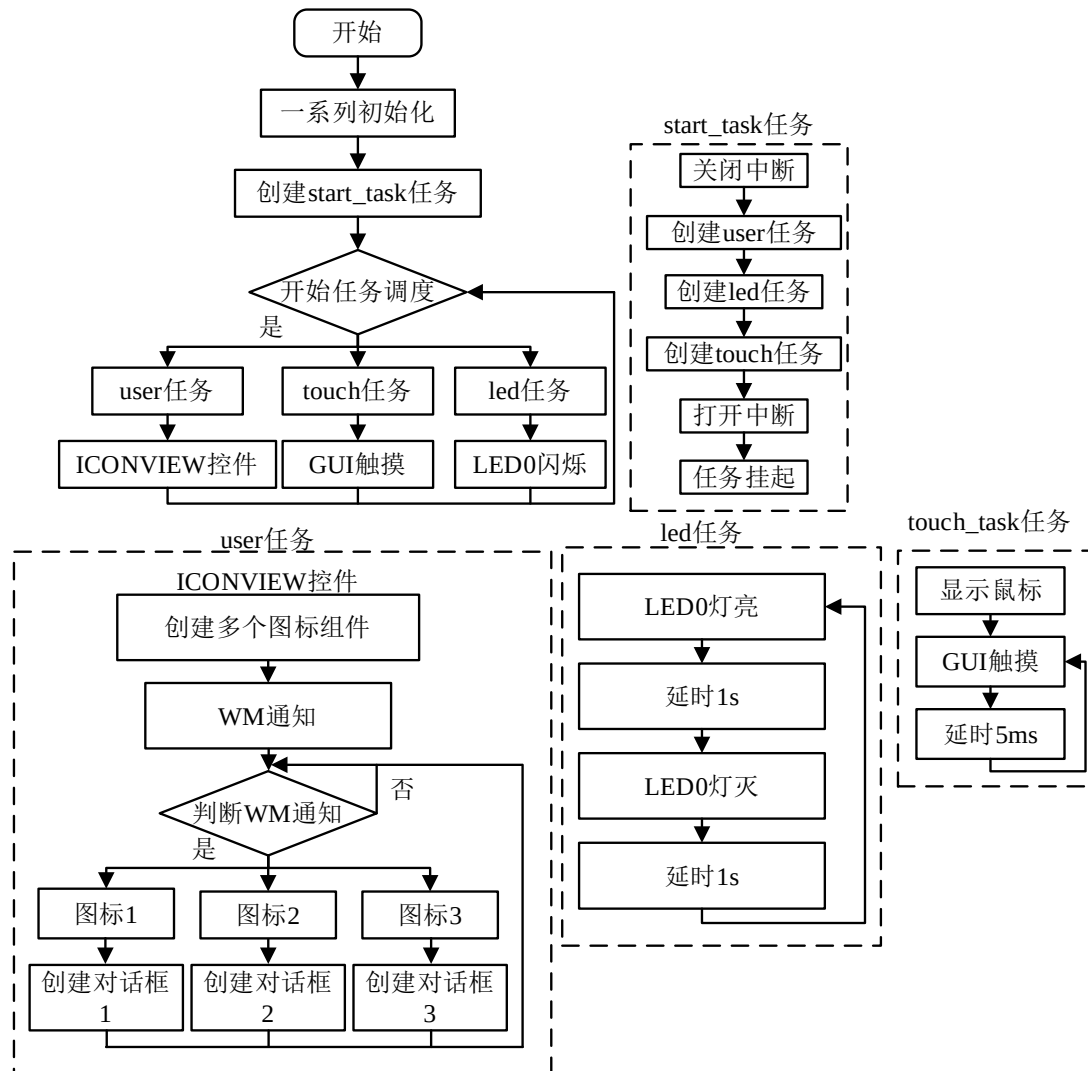


图 22.5.2.1.1 ICONVIEW 控件使用实验流程图

22.5.2.2 程序解析

打开 **emWin 实验 23 ICONVIEW 小工具** 工程中的文件 `emwin_demo.c`，这个文件的代码就是本实验的主要源码，这些源码如下所示：

(1) 定义图片源

```

/* app 图标描述结构体 */
typedef struct{
    const GUI_BITMAP    *pBitmap;    /* app 图标 */
    const char          *pText;      /* app 图标名字 */
}BITMAP_ITEM;

/* ICONVIEW 显示的 APP 图标 */
static const BITMAP_ITEM BitmapItemTab1[]={
    {&bmbag,              "Bag"},
    {&bmbbook,            "Book"},
    {&bmbbookshelf,       "Booksh"},

```



```

    {&bmbrowser,      "Brower"},
    {&bmcalendar,     "Calendar"},
    {&bmcipboard,     "Clipboard"},
    {&bcompass,       "Compass"},
    {&bcompose,       "Cmpose"},
    {&bmdocument,    "Cument"},
    {&bmfolder,       "Folder"},
    {&bmmail,         "Mail"},
    {&bmpaint,        "Paint"},
    {&bmpin,          "Pin"},
    {&bmsettings,     "Set"},
};

```

上述源码关于位图制作的知识，请参考第八章的内容。

(2) 创建 ICONVIEW 控件

```

/**
 * @brief      emwin_main(emwin 入口)
 * @param      无
 * @return     无
 */
void emwin_main(void)
{
    int i;

    /* 设置换肤 */
    PROGBAR_SetDefaultSkin(PROGBAR_SKIN_FLEX);
    FRAMEWIN_SetDefaultSkin(FRAMEWIN_SKIN_FLEX);
    PROGBAR_SetDefaultSkin(PROGBAR_SKIN_FLEX);
    BUTTON_SetDefaultSkin(BUTTON_SKIN_FLEX);
    CHECKBOX_SetDefaultSkin(CHECKBOX_SKIN_FLEX);
    DROPDOWN_SetDefaultSkin(DROPDOWN_SKIN_FLEX);
    SCROLLBAR_SetDefaultSkin(SCROLLBAR_SKIN_FLEX);
    SLIDER_SetDefaultSkin(SLIDER_SKIN_FLEX);
    HEADER_SetDefaultSkin(HEADER_SKIN_FLEX);
    RADIO_SetDefaultSkin(RADIO_SKIN_FLEX);
    MULTIPAGE_SetDefaultSkin(MULTIPAGE_SKIN_FLEX);

    WM_SetCallback(WM_HBKWIN, cb_BkWindow); /* 设置桌面窗口 WM_HBKWIN 的回调函数 */
    /* 建立一个 ICONVIEW 作为主界面 */
    IconviewWin = ICONVIEW_CreateEx( 0,0,          /* 左上角坐标 */
                                     800,250,      /* 小工具的水平和垂直尺寸 */
                                     WM_HBKWIN,     /* 父窗口为背景窗口 */
                                     WM_CF_SHOW|WM_CF_HASTRANS,
                                     ICONVIEW_CF_AUTOSCROLLBAR_V,

```

```

        ID_ICONVIEW_0, /* 小工具 ID */
        100,           /* 图标的水平尺寸为 100 */
        100);         /* 图标的垂直尺寸为 100 */

ICONVIEW_SetFont (IconviewWin, &GUI_Font16_ASCII);
ICONVIEW_SetBkColor (IconviewWin, ICONVIEW_CI_SEL, GUI_WHITE|0X70000000);
ICONVIEW_SetSpace (IconviewWin, GUI_COORD_X, 15);
ICONVIEW_SetSpace (IconviewWin, GUI_COORD_Y, 15);
ICONVIEW_SetIconAlign (IconviewWin, ICONVIEW_IA_HCENTER|ICONVIEW_IA_VCENTER);

for(i = 0; i < (GUI_COUNTOF(BitmapItemTab1)); i++)
{
    ICONVIEW_AddBitmapItem (IconviewWin, BitmapItemTab1[i].pBitmap,
                            BitmapItemTab1[i].pText);
}

while(1)
{
    GUI_Delay(100);
}
}
    
```

上述源码可知：我们调用换肤函数设置 ICONVIEW 控件样式，然后调用 ICONVIEW_CreateEx 创建一个 ICONVIEW 控件，这里并没有使用 DIALOG 的方式，此时的父窗口就是背景窗口了。创建好 ICONVIEW 控件以后就可以初始化 ICONVIEW 并添加图标。下面我们来看一下 ICONVIEW 控件的回调函数 cb_BkWindow 到底实现什么功能，如下源码所示：

(3) ICONVIEW 控件回调函数

```

/**
 * @brief      桌面窗口 WM_HBKWIN 回调函数
 * @param      pMsg:消息结构体
 * @return     无
 */
void cb_BkWindow(WM_MESSAGE *pMsg)
{
    int Id;
    int NCode;
    static int Iconview0_Sel;

    switch (pMsg->MsgId)
    {
        case WM_PAINT:
            GUI_SetBkColor (GUI_BLACK);
            GUI_Clear();
            GUI_DrawBitmap (&bmbgbackground, 0, 0); /* 绘制背景图片 */
            break;
    }
}
    
```

```

case WM_NOTIFY_PARENT:
    Id = WM_GetId(pMsg->hWinSrc);
    NCode = pMsg->Data.v;
    switch(Id)
    {
        case ID_ICONVIEW_0: /* ICONVIEW0 控件 */
            switch(NCode) /* 通知代码 */
            {
                case WM_NOTIFICATION_CLICKED:
                    break;
                case WM_NOTIFICATION_RELEASED:
                    Iconview0_Sel = ICONVIEW_GetSel(pMsg->hWinSrc);
                    switch(Iconview0_Sel)
                    {
                        case 0: /* APP0 */
                            /* 创建 BagAPP */
                            if(BagAPPWin == NULL)CreateBagAPP();
                            break;
                        case 1: /* APP1 */
                            /* 创建 BookAPP */
                            if(BookAPPWin == NULL)CreateBookAPP();
                            break;
                        case 2: /* APP2 */
                            /* 创建 BookshAPP */
                            if(BookshAPPWin == NULL)CreateBookshAPP();
                            break;
                        case 3: /* APP3 */
                            /* 创建 BrowerAPP */
                            if(BrowerAPPWin == NULL)CreateBrowerAPP();
                            break;
                    }
                    break;
                case WM_NOTIFICATION_SEL_CHANGED:
                    break;
            }
            break;
    }
    break;
default:
    WM_DefaultProc(pMsg);
    break;
}
}

```

从上述源码可知：当我们按下 APP0\ APP1\ APP2\ APP3 的图标时，系统会调用相关的函数创建对话框。注意：相关的对话框源码请参考 iconviewdemo.c 文件，由于前面我们已经了解对话框的内容，这里我们就不再讲解了。

22.5.3 下载验证

编译并下载例程到开发板中，LCD 显示如下图所示：

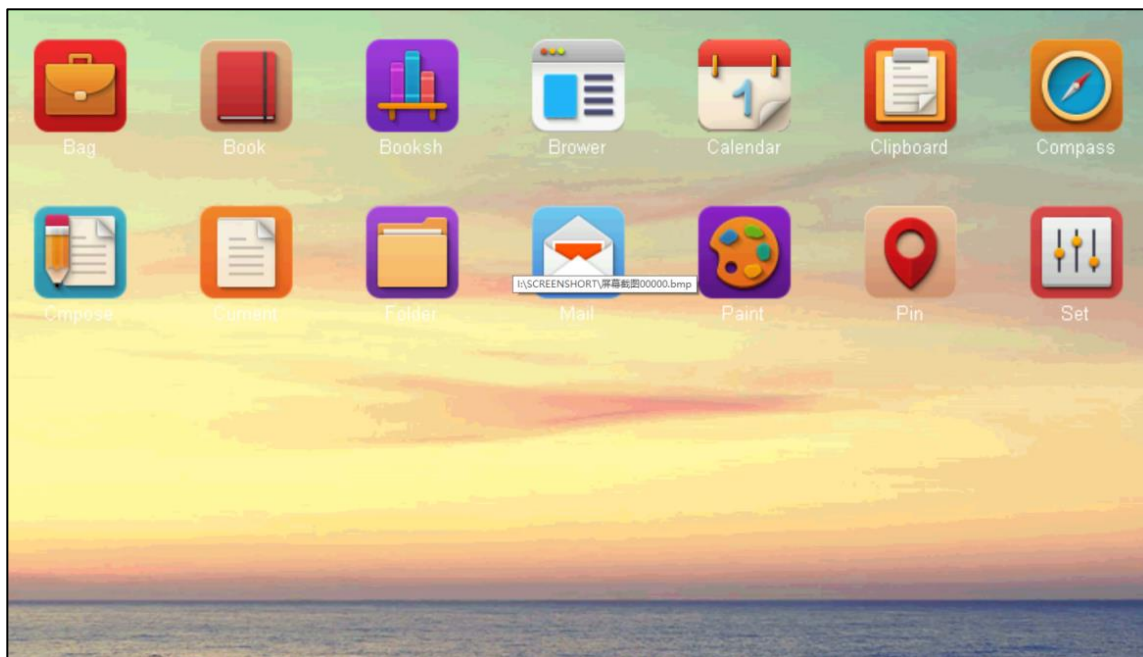


图 22.5.3.1 ICONVIEW 控件显示

第二十三章 SWIPELIST 控件

SWIPELIST 小部件像手机上的微信目录一样，可以滑动查看各个菜单选项。SWIPELIST 滑动列表控件通过在触摸屏或任何其他指针输入设备（PID）上滑动来实现列表的平滑滚动，本章我们就来学习 SWIPELIST 控件的使用。

本章分为以下几个部分：

23.1 SWIPELIST 控件简介

23.2 通知代码

23.3 SWIPELIST 控件 API 函数

23.4 SWIPELIST 控件演示实验

23.1 SWIPELIST 控件简介

SWIPELIST 小控件通过在触摸屏或任何其他指针输入设备(PID)上滑动来实现列表的平滑滚动。每个 SWIPELIST 项可以显示几行文本或一个位图。还可以为每个项分配一个或多个窗口/小部件，如图 23.1.1 所示：

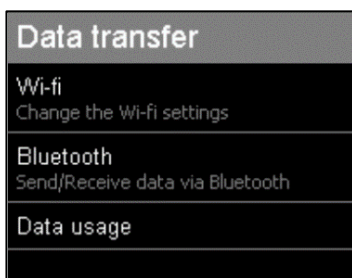


图 23.1.1 SWIPELIST 控件

从上图可知：我们可以在 SWIPELIST 小控件添加多个菜单项，这些菜单项可以使用图标和文本结合显示，当然也可以在这个菜单项添加复选框等小控件。

SWIPELIST 控件的结构

下图显示了 SWIPELIST 控件的详细结构和一个项目的外观：

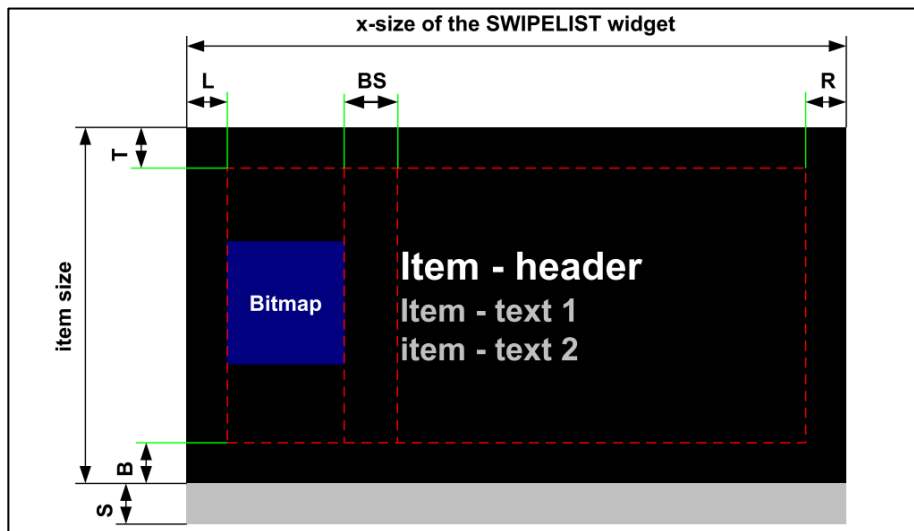


图 23.1.2 SWIPELIST 外观图

上图给出的参数需要用户自己设置，这些参数如下表所示：

参数	描述
B	用于所有项目的下边框（默认值：5 像素）
BS	用于所有项目的位图和文本之间的大小（默认值：5 像素）
L	用于所有项目的左边框（默认值：5 像素）
R	用于所有项目的右边框（默认值：5 像素）
S	可为所有项目分别设置的分隔线（默认值：1 像素）
T	用于所有项目的上边框默认值：5 像素）

表 23.1.1 SWIPELIST 控件参数描述

上述表中的参数由用户设置，当然我们可以选择默认的参数值。下面我们来看一下 SWIPELIST 控件添加多个菜单项的结构示意图，如下图所示：

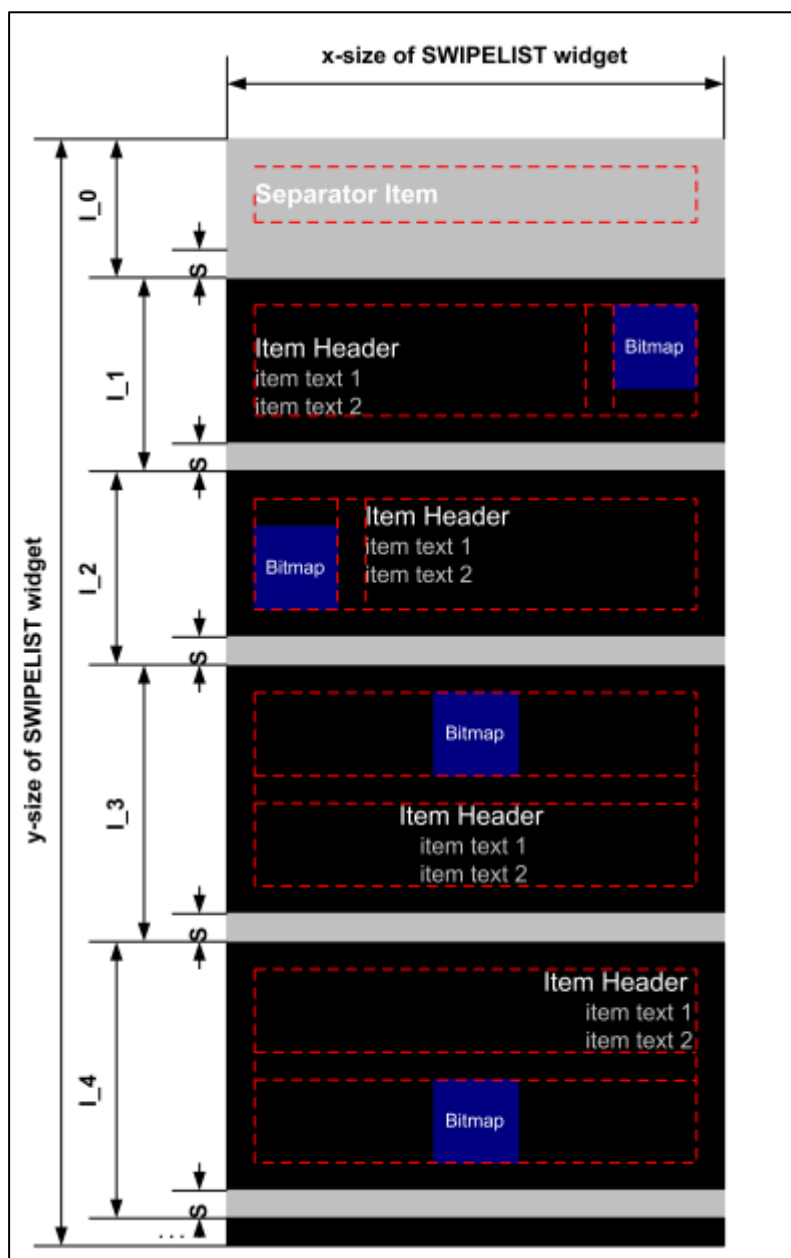


图 23.1.3 SWIPELIST 分隔符项

从上图可知：Bitmap 的位图可以设置的位置具有三种，它们分别为左中右等位置，而文本设置也是一样的操作。虽然 SWIPELIST 小部件可由多个菜单项组成，但是它们的分隔符项是具有不同属性的项，如下表所示：

参数	描述
I0	SWIPELIST 控件的分隔符项目 (ItemIndex = 0) + 文本 (对齐方式：垂直居中，左对齐)
I1	SWIPELIST 控件的项目 (ItemIndex = 1) + 位图 (对齐方式：上、右对齐) + 文本 (对齐方式：下、左对齐)
I2	SWIPELIST 控件的项目 (ItemIndex = 2) + 位图 (对齐方式：下、左对齐) + 文本 (对齐方式：上、左对齐)
I3	SWIPELIST 控件的项目 (ItemIndex = 3) + 位图 (对齐方式：上对齐、水平居中) + 文本 (对齐方式：上对齐、水平居中)
I4	SWIPELIST 控件的项目 (ItemIndex = 4) + 位图 (对齐方式：下对齐、水平居中) + 文本 (对齐方式：下、右对齐)
S	可为所有项目分别设置的分隔线 (默认值：1 像)

表 23.1.2 SWIPELIST 菜单项的描述

从上表可知：SWIPELIST 小控件添加菜单项的种类具有六种，大家可以根据项目的需求来添加那种菜单项。

23.2 通知代码

以下事件将作为 WM_NOTIFY_PARENT 消息的一部分从 SWIPELIST 控件发送至其父窗口，如下表所示：

参数	描述
WM_NOTIFICATION_CLICKED	已点击 SWIPELIST 控件的项目
WM_NOTIFICATION_RELEASED	已释放 SWIPELIST 控件的项目
WM_NOTIFICATION_RELEASED	已点击 SWIPELIST 控件的项目，且指针已经移出

表 23.2.1 SWIPELIST 通知代码

23.3 SWIPELIST 控件 API 函数

下表按字母顺序列出了可用的 emWin 的 SWIPELIST 控件相关函数。这些函数的详细说明如下表所示：

函数	描述
SWIPELIST_AddItem()	添加新项目
SWIPELIST_AddItemText()	添加文本到项目
SWIPELIST_AddSepItem()	添加新分隔符项目
SWIPELIST_CreateEx()	创建 SWIPELIST 控件
SWIPELIST_CreateIndirect()	从资源表条目创建 SWIPELIST 控件
SWIPELIST_CreateUser()	用额外字体作为用户数据创建 SWIPELIST 控件
SWIPELIST_DeleteItem()	删除 SWIPELIST 控件的一个项目
SWIPELIST_GetBitmap()	返回指向位图的指针
SWIPELIST_GetBitmapSpace()	返回文本和位图之间的间距
SWIPELIST_GetBkColor()	返回背景色

SWIPELIST_GetBorderSize()	返回边框的大小
SWIPELIST_GetDefaultBitmapSpace()	返回文本和位图之间的默认间距
SWIPELIST_GetDefaultBkColor()	返回默认背景色
SWIPELIST_GetDefaultBorderSize()	返回默认边框大小
SWIPELIST_GetDefaultFont()	返回指向默认字体的指针
SWIPELIST_GetDefaultSepColor()	返回分隔线的默认颜色
SWIPELIST_GetDefaultSepSize()	返回分隔线的默认大小
SWIPELIST_GetDefaultTextColor()	返回文本的默认颜色
SWIPELIST_GetDefaultTextAlign()	返回文本的默认对齐方式
SWIPELIST_GetDefaultThreshold()	返回默认阈值
SWIPELIST_GetFont()	返回指向字体的指针
SWIPELIST_GetItemSize()	返回项目的大小
SWIPELIST_GetItemUserData()	从特定项目提取以前存储的用户数据
SWIPELIST_GetNumItems()	返回项目数
SWIPELIST_GetNumText()	返回项目中的文本数
SWIPELIST_GetReleasedItem()	返回已释放的项目的项目索引
SWIPELIST_GetScrollPos()	返回控件的滚动位置
SWIPELIST_GetSelItem()	返回当前选定项
SWIPELIST_GetSepColor()	返回分隔线的颜色
SWIPELIST_GetSepSize()	返回分隔线的大小
SWIPELIST_GetText()	返回文本
SWIPELIST_GetTextAlign()	返回文本对齐方式
SWIPELIST_GetTextColor()	返回文本的颜色
SWIPELIST_GetThreshold()	返回控件的阈值
SWIPELIST_GetUserData()	提取用 SWIPELIST_SetUserData 设置的数据
SWIPELIST_ItemAttachWindow()	将窗口附加到项目
SWIPELIST_ItemDetachWindow()	分离窗口
SWIPELIST_OwnerDraw()	绘制 SWIPELIST 控件的默认函数
SWIPELIST_SetAttachedWindowPos()	设置附加窗口的位置
SWIPELIST_SetBitmap()	设置指向位图的指针
SWIPELIST_SetBitmapSpace()	设置文本和位图之间的间距
SWIPELIST_SetBkColor()	设置背景色
SWIPELIST_SetBorderSize()	设置边框的大小
SWIPELIST_SetDefaultBitmapSpace()	设置文本和位图之间的默认间距
SWIPELIST_SetDefaultBkColor()	设置默认背景色
SWIPELIST_SetDefaultBorderSize()	设置边框的默认大小
SWIPELIST_SetDefaultFont()	设置指向用作默认字体的指针
SWIPELIST_SetDefaultSepColor()	设置分隔线的默认颜色
SWIPELIST_SetDefaultSepSize()	设置分隔线的默认大小
SWIPELIST_SetDefaultTextColor()	设置文本的默认颜色
SWIPELIST_SetDefaultTextAlign()	设置文本的默认对齐方式
SWIPELIST_SetDefaultThreshold()	设置默认阈值
SWIPELIST_SetFont()	设置指向所用字体的指针
SWIPELIST_SetItemSize()	设置项目的大小
SWIPELIST_SetItemUserData()	在特定项目中存储用户数据
SWIPELIST_SetOwnerDraw()	设置指向绘制函数的指针
SWIPELIST_SetScrollPos()	设置的 SWIPELIST 控件滚动位置
SWIPELIST_SetScrollPosItem()	设置滚动位置到一个项目

SWIPELIST_SetSepColor()	设置分隔线的颜色
SWIPELIST_SetSepSize()	设置分隔线的大小
SWIPELIST_SetText()	设置文本
SWIPELIST_SetTextAlign()	设置文本的对齐方式
SWIPELIST_SetTextColor()	设置文本的颜色
SWIPELIST_SetThreshold()	设置阈值
SWIPELIST_SetUserData()	设置 SWIPELIST 控件的额外数据

表 23.3.1 SWIPELIST 控件 API 函数

SWIPELIST 控件的 API 函数都比较简单，大家可以看一下 emWin 官方的手册。

23.4 SWIPELIST 控件演示实验

23.4.1 硬件设计

1. 例程功能

本实验主要讲解 SWIPELIST 小工具的使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数主要创建一个 SWIPELIST 小工具并在其上面添加多个菜单项，这些菜单项可以添加位图、文本以及复选框小控件。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 24 SWIPELIST 小工具》。

23.4.2 软件设计

23.4.2.1 程序流程图

本实验的程序流程图，如下图所示：

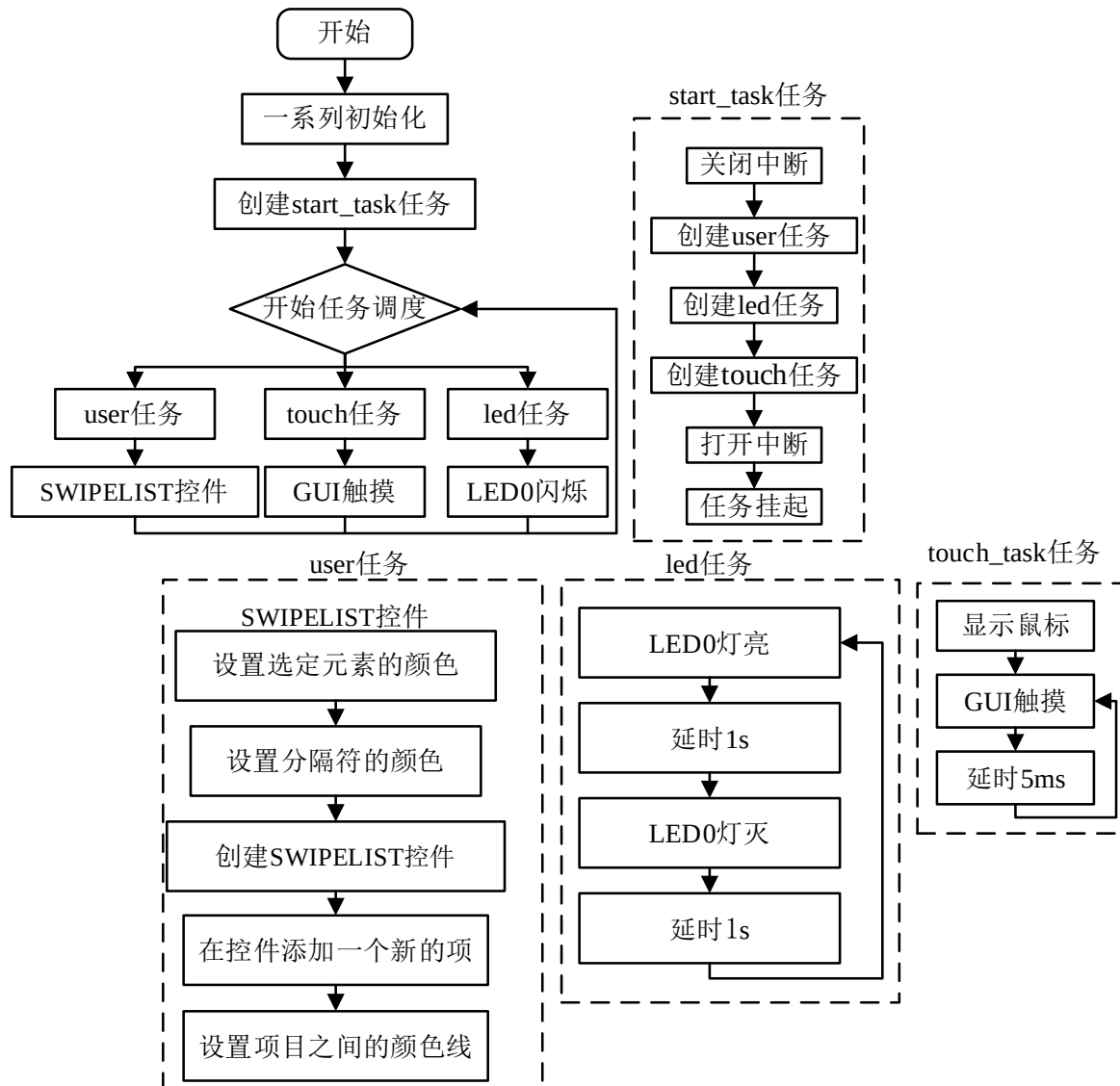


图 23.4.2.1.1 SWIPELIST 小工具实验流程图

23.4.2.2 程序解析

打开 **emWin 实验 24 SWIPELIST 小工具** 工程中的文件 `emwin_demo.c`，这个文件的代码就是本实验的主要源码，这些源码如下所示：

(1) 设置项的信息

```

/* 此数组包含关于每个项的信息 */
static const char *paItemText[3][5] =
{
    {
        "Brightness",
        "Mobile Data",
        "Bluetooth",
        "Hot Spot",
        "Mobil",
    }
}
  
```

```

    },
    {
        "Wifi",
        "music",
        "Idle",
        "Night Mode",
        "Font Size",
    },
    {
        "Account",
        "Sound",
        "Memory",
        "Alert",
        "Clock",
    },
};
};

```

上述源码定义的是菜单项的文本信息。

(2) 设置标题信息

```

/* 关于标题的信息 */
static const char *paSeperatorText[] =
{
    "Display",
    "Network",
    "Settings"
};
};

```

上述源码定义的是菜单项的标题字符串信息。本实验在 SWIPELIST 控件中创建了三个标题，它们分别为"Display"，"Network"和"Settings"标题。下面我们来看一下本实验的演示函数，如下所示：

(3) 演示函数

```

/**
 * @brief      emwin_main(emwin 入口)
 * @param      无
 * @return     无
 */
void emwin_main(void)
{
    WM_HWIN hSwipelists;
    WM_HWIN hItem;
    int i;
    int j;
    int Xsize;
    int Ysize;

```

```

Xsize = LCD_GetXSize(); /* 获取 LCD 的 x 大小 */
Ysize = LCD_GetYSize(); /* 获取 LCD 的 y 大小 */

image_tab[0] = bmwifi; /* 设置位图 */
image_tab[1] = bmmusic;

WM_MULTIBUF_Enable(1); /* 自动使用多缓冲 */

WM_MOTION_Enable(1); /* 使能移动 */
WM_SetBkWindowColor(GUI_BLUE); /* 设置一个默认的背景颜色 */
/* 设置选定项目的颜色 */
SWIPELIST_SetDefaultBkColor(SWIPELIST_CI_BK_ITEM_SEL, COLOR_GREEN);
/* 设置分隔符的颜色 */
SWIPELIST_SetDefaultBkColor(SWIPELIST_CI_BK_SEP_ITEM, GUI_DARKGRAY);

/* 创建一个 swipelists */
hSwipelists = SWIPELIST_CreateEx(10, 10, Xsize/2, Ysize/2, WM_HBKWIN,
                                WM_CF_SHOW, 0, 0);
WM_MOTION_SetMoveable(hSwipelists, WM_CF_MOTION_Y, 1);

for (j = 0; j < NUM_SEPARATORS; j++) /* 在每个分隔符下添加 5 个项目 */
{
    SWIPELIST_AddSepItem(hSwipelists, paSeperatorText[j], 35);

    for (i = 0; i < NUM_ITEMS; i++)
    {
        /* 添加一个新的项 */
        SWIPELIST_AddItem(hSwipelists, paItemText[j][i], 35);
        /* 设置项目之间的颜色线 */
        SWIPELIST_SetSepColor(hSwipelists, i, GUI_RED);
    }
}

for (i = 0; i < 5; i++) /* 将复选框附加到前五个项目 */
{
    hItem = CHECKBOX_Create(0, 0, 15, 15, WM_HBKWIN, 0, WM_CF_SHOW);
    SWIPELIST_ItemAttachWindow(hSwipelists, 1 + i, hItem, Xsize/2-20, 10);
}

for (i = 0; i < 2; i++) /* 设置位图 */
{
    SWIPELIST_SetBitmap(hSwipelists, i+7, SWIPELIST_BA_RIGHT, &image_tab[i]);
}
    
```

```
}  
}
```

在上述的源码中，位图的设置小编就没有必要多说了，请参考 button 位图设置实验，此函数主要调用函数 `SWIPELIST_AddSepItem` 添加五个选项，用户可以在这些选项添加小控件以及添加位图等操作。

23.4.3 下载验证

编译并下载例程到开发板中，LCD 显示如下图所示。



图 23.4.3.1 SWIPELIST 控件显示

第二十四章 TEXT 控件

有时候我们需要在窗口上显示一些提示信息，我们前面讲过可以使用字符串显示函数来显示一些字符信息，但是在使用对话框的时候使用字符串显示函数就不方便了，虽然也可以，但是需要做一些其他的处理。为了解决这个问题 emWin 给我们提供了一个很方便的控件，那就是 TEXT 控件，本章我们就来学习 TEXT 控件的使用。

本章分为以下几个部分：

24.1 TEXT 控件简介

24.2 通知代码

24.3 TEXT 控件 API 函数

24.4 TEXT 控件演示实验

24.1 TEXT 控件简介

TEXT 控件也叫文本控件，它主要用来在对话框或窗口中显示文本字段，当然了其他需要显示字符串的地方都可以用 TEXT 控件，TEXT 控件示意图如下图所示：



图 24.1.1 TEXT 控件

TEXT 控件可以用于其他控件描述，如下图所示：

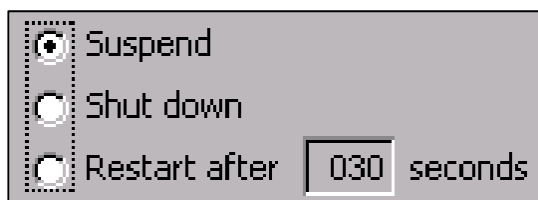


图 24.1.2 TEXT 控件

从上图可知：TEXT 控件一般用于提示小控件信息的文本控件。

24.2 通知代码

TEXT 小工具会作为 WM_NOTIFY_PARENT 消息的一部分发送给父窗口，通知代码如下表所示：

消息	描述
WM_NOTIFICATION_CLICKED	以单击小工具
WM_NOTIFICATION_RELEASED	以释放小工具
WM_NOTIFICATION_MOVED_OUT	以单击小工具，并且指针移出小工具，但是有释放

表 24.2.1 HEADER 通知代码

上表的通知代码是 TEXT 控件在对话框的通知代码，用户可以使用这些代码来判断这个控件的触发类型。

24.3 TEXT 控件 API 函数

表 24.3.1 按字母顺序列出了可用的 TEXT 控件相关 API 函数：

函数	描述
TEXT_Create()	创建 TEXT 小工具（弃用）
TEXT_CreateAsChild()	创建一个 TEXT 小工具作为子窗口（弃用）
TEXT_CreateEx()	创建 TEXT 小工具
TEXT_CreateIndirect()	从资源表条目创建 TEXT 小工具
TEXT_CreateUser()	使用作为用户数据的额外字节来创建 TEXT 小工具
TEXT_GetDefaultFont()	返回文本的默认字体
TEXT_GetNumLines()	返回当前在小工具中所显示的线条数
TEXT_GetUserData()	检索通过 TEXT_SetUserData 设置的数据
TEXT_SetBkColor()	设置文本的背景颜色
TEXT_SetDefaultFont()	设置文本的默认字体
TEXT_SetDefaultTextColor()	设置文本的默认文本颜色
TEXT_SetDefaultWrapMode()	设置新文本小工具的默认覆盖模式
TEXT_SetFont()	设置指定文本小工具所使用的字体
TEXT_SetText()	设置指定文本小工具的文本
TEXT_SetTextAlign()	设置指定文本小工具的文本对齐方式
TEXT_SetTextColor()	设置指定小工具的文本颜色
TEXT_SetUserData()	设置 TEXT 小工具的额外数据
TEXT_SetWrapMode()	设置指定文本小工具的覆盖模式

表 24.3.1 TEXT 控件 API 函数

TEXT 控件的 API 函数比较简单，没有需要特别讲解的，小编就以常用的 TEXT 函数讲解，在使用的时候大家可以参考一下 emWin 官方手册看一下。

(1) TEXT_CreateEx 函数

在指定位置创建指定大小的 TEXT 控件，其函数原型如下源码所示：

```
TEXT_Handle TEXT_CreateEx( int x0,
                           int y0,
                           int xSize,
                           int ySize,
                           WM_HWIN hParent,
                           int WinFlags,
                           int ExFlags,
                           int Id,
                           const char * pText);
```

该函数的形参如下表所示：

参数	描述
x0	控件最左侧的像素（在父坐标中）
y0	控件最顶端的像素（在父坐标中）
xSize	控件的水平尺寸（以像素为单位）
ySize	控件的垂直尺寸（以像素为单位）
hParent	父窗口的句柄。如果为 0，则新创建的 TEXT 控件将作为桌面（顶层窗口）的子窗口
WinFlags	窗口创建标记。为了使控件立即可见，通常为 WM_CF_SHOW
ExFlags	参数 ExFlags 的允许值

	TEXT_CF_LEFT	水平对齐方式：左对齐
	TEXT_CF_RIGHT	水平对齐方式：右对齐
	TEXT_CF_HCENTER	水平对齐方式：居中
	TEXT_CF_TOP	水平对齐方式：居中
	TEXT_CF_BOTTOM	水平对齐方式：居中
	TEXT_CF_VCENTER	垂直对齐方式：居中
Id	TEXT 控件的窗口 ID	
pText	要显示的文本的指针	

表 24.3.2 TEXT_CreateEx 函数相关形参描述

(2) TEXT_SetFont 函数

设置指定 TEXT 控件所使用的字体，其函数原型如下源码所示：

```
void TEXT_SetFont(TEXT_Handle hObj,
                  const GUI_FONT * pFont);
```

该函数的形参如下表所示：

参数	描述
hObj	TEXT 控件的句柄
pFont	指向要使用的字体指针

表 24.3.3 TEXT_SetFont 函数相关形参描述

(3) TEXT_SetText 函数

设置指定 TEXT 控件所使用的文本，其函数原型如下源码所示：

```
int TEXT_SetText(TEXT_Handle hObj,
                 const char * s);
```

该函数的形参如下表所示：

参数	描述
hObj	TEXT 控件的句柄
s	要显示的文本

表 23.3.4 TEXT_SetText 函数相关形参描述

24.4 TEXT 控件演示实验

24.4.1 硬件设计

1. 例程功能

本实验主要讲解 TEXT 小工具的使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。注意：本实验会用到 ADC 以及 RTC 外设，这些外设的相关知识请大家查看正点原子裸机教程，这里小编并不会讲解这些知识。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数主要创建一个 window 小工具并在其上面添加多个 TEXT，它们分别获取 RTC 以及 ADC 的数据并在该控件显示这些数据。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 25 TEXT 小工具》。

24.4.2 软件设计

24.4.2.1 程序流程图

本实验的程序流程图，如下图所示：

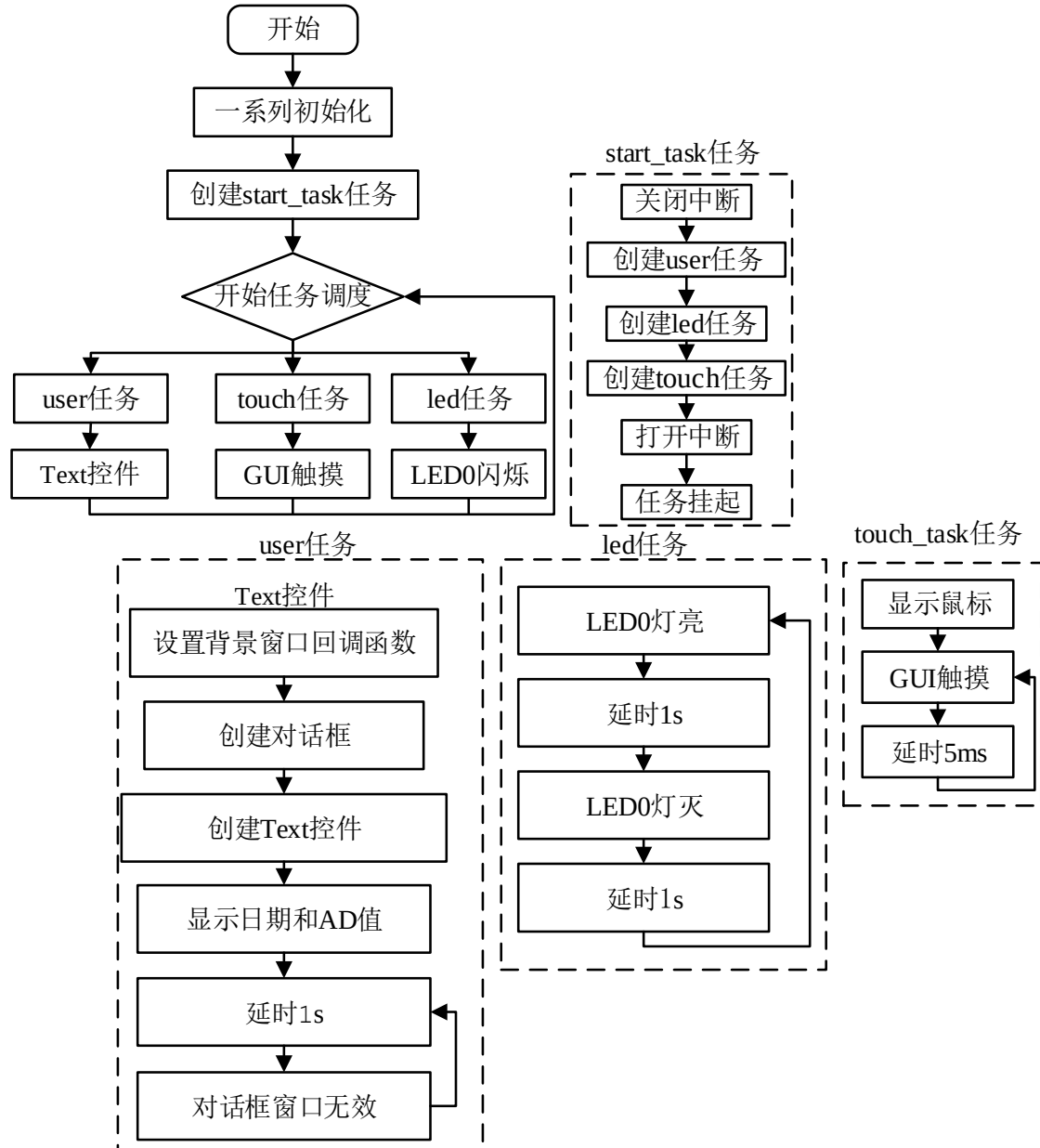


图 24.4.2.1.1TEXT 控件实验流程图

24.4.2.2 程序解析

本章的实验是设计一个用来显示系统内部 AD 值和 RTC 的界面，该实验使用 TEXT 控件来显示信息，ADC 和 RTC 的数据显示使用 GUI_DisStringAt 这样的字符串显示函数。当然了，ADC 和 RTC 肯定可以用 TEXT 控件来显示，主要是很多朋友都问过如何在对话框中调用字符串显示、2D 绘图这些函数，所以在这个例程中顺带着讲一下，打开工程 **emWin 实验 25 TEXT 小工具** 中的文件 emwin_demo.c，函数 emwin_main 就是本例程演示函数，代码如下：

(1) 资源表

```
static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] =
{
{ FRAMEWIN_CreateIndirect, "Framewin", ID_FRAMEWIN_0, 0, 0, 320, 240,
  FRAMEWIN_CF_MOVEABLE, 0x64, 0 },
{ TEXT_CreateIndirect, "AD VALUE:", ID_TEXT_0, 20, 50, 120, 25, 0, 0x64, 0 },
{ TEXT_CreateIndirect, "DATE:", ID_TEXT_1, 68, 80, 69, 20, 0, 0x64, 0 },
{ TEXT_CreateIndirect, "TIME:", ID_TEXT_2, 75, 110, 65, 20, 0, 0x64, 0 },
};
```

使用 GUIBulider 软件创建一个窗口框架和三个 TEXT 控件。

(2) 创建对话框

```
/**
 * @brief      emwin_main(emwin 入口)
 * @param      无
 * @retval     无
 */
void emwin_main(void)
{
    WM_SetCallback(WM_HBKWIN, _cbBkWindow); /*设置背景窗口回调函数*/
    TextDialoghWin = GUI_CreateDialogBox(_aDialogCreate,
                                          GUI_COUNTOF(_aDialogCreate),
                                          _cbDialog, WM_HBKWIN, 0, 0);

    while (1)
    {
        GUI_Delay(1000);
        WM_InvalidateWindow(TextDialoghWin); /*对话框窗口无效*/
    }
}
```

函数 emwin_main 就是设置背景窗口的回调函数并且创建一个对话框，这个没什么好说的，前面大多数例程的套路。重点在于 while(1) 循环函数！while(1) 循环函数里面调用了窗口无效化函数 WM_InvalidateWindow，**讲窗口的时候说过无效化窗口以后窗口就会重绘！**重绘就意味着可以更新数据！这样我们就可以在窗口回调函数的消息 WM_PAINT 中显示那些变化的数据了，比如本章例程中的 ADC 和 RTC 值。而且我们也可以在 WM_PAINT 消息中调用 2D 绘图函数来在窗口中绘制 2D 图形，如线段，圆，椭圆等等。下面我们来看一下背景窗口回调函数的作用，如下所示：

(3) 背景重绘

```
/**
 * @brief      背景窗口 WM_HBKWIN 回调函数
 * @param      pMsg : 消息
 * @retval     无
 */
static void _cbBkWindow(WM_MESSAGE *pMsg)
{
```

```
switch (pMsg->MsgId)
{
    case WM_PAINT:
        GUI_SetBkColor(GUI_BLUE);
        GUI_Clear();
        break;

    default:
        WM_DefaultProc(pMsg);
}
}
```

(4) 对话框的回调函数

```
/**
 * @brief      对话框回调函数
 * @param      pMsg : 消息
 * @retval     无
 */
static void _cbDialog(WM_MESSAGE *pMsg)
{
    WM_HWIN hItem;
    static uint16_t adcvalue = 0;
    uint8_t tbuf[40];

    switch (pMsg->MsgId)
    {
        case WM_PAINT:
            GUI_SetColor(GUI_RED);
            GUI_SetFont(GUI_FONT_24B_ASCII);

            /* 显示AD值 */
            adcvalue = adc_get_result_average(ADC_CHANNEL_5, 10); /* 获取AD值 */
            GUI_DispDecAt(adcvalue, 140, 50, 4);
            rtc_set_alarm(2020, 4, 26, 9, 23, 45); /* 设置一次闹钟 */
            /* 显示日期 */
            rtc_get_time(); /* 获取日期 */
            sprintf((char *)tbuf, "%04d-%02d-%02d", calendar.year,
                    calendar.month,
                    calendar.date);
            GUI_DispStringAt((const char *)tbuf, 140, 80); /* 显示日期 */

            sprintf((char *)tbuf, "%02d:%02d:%02d", calendar.hour, calendar.min,
                    calendar.sec);
            GUI_DispStringAt((const char *)tbuf, 140, 110); /* 显示日期 */
    }
```

```

        sprintf((char *)tbuf, "Week:%d", calendar.week);
        GUI_DispStringAt((const char *)tbuf, 140, 140);
        break;

case WM_INIT_DIALOG:
    /* FRAMWIN 初始化 */
    hItem = pMsg->hWin;
    FRAMEWIN_SetFont(hItem, GUI_FONT_20B_ASCII);
    FRAMEWIN_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
    FRAMEWIN_SetText(hItem, "ALIENTEK TEXT");
    FRAMEWIN_SetTextColor(hItem, 0x0000FFFF);

    /* TEXT 初始化 */
    hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_0);
    TEXT_SetFont(hItem, GUI_FONT_24B_ASCII);
    TEXT_SetText(hItem, "AD VALUE:");

    /* 日期初始化 */
    hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_1);
    TEXT_SetFont(hItem, GUI_FONT_24B_ASCII);
    TEXT_SetText(hItem, "DATE:");

    /* 时间初始化 */
    hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_2);
    TEXT_SetTextColor(hItem, 0x00000000);
    TEXT_SetFont(hItem, GUI_FONT_24B_ASCII);
    TEXT_SetText(hItem, "TIME:");
    break;

default:
    WM_DefaultProc(pMsg);
    break;
}
}

```

此函数就是对话框的回调函数，这个回调函数主要获取 ADC 以及 RTC 的数据并在指定的 TEXT 控件显示这些数据。

24.4.3 下载验证

编译并下载例程到开发板中，LCD 显示如下图所示：

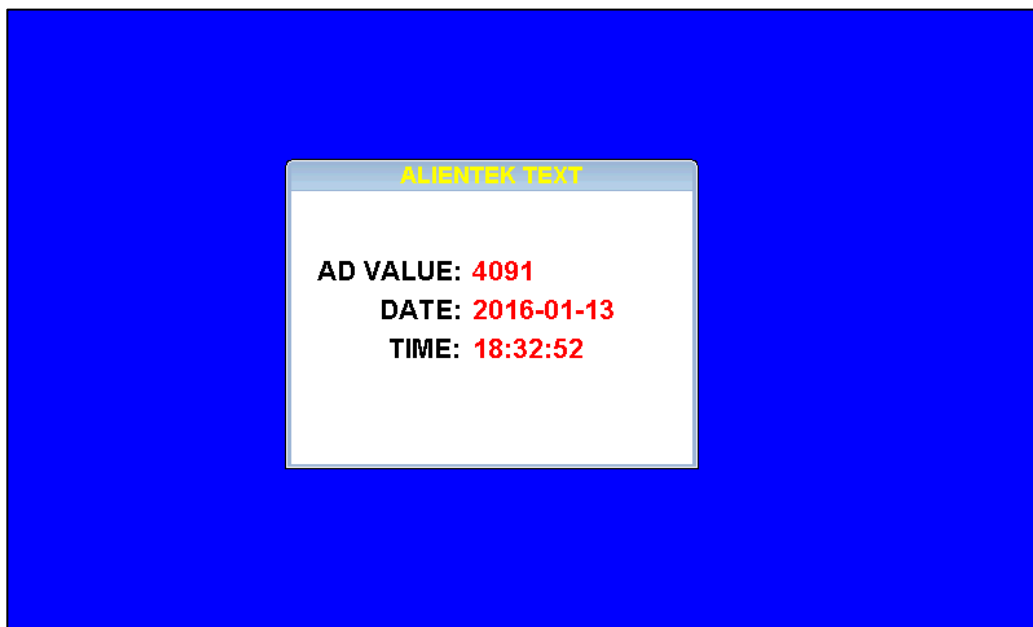


图 24.4.3.1TEXT 控件例程

在图 24.4.3.1 中黑色字符就是用 TEXT 控件显示的，红色的 AD 值和 RTC 值是调用函数 GUI_DispStringAt 显示的。AD 接到 3.3V 上，显示采集到的 AD 原始值是 4091，也就是 3.296V 左右，和实际情况差不多。

第二十五章 LISTBOX 控件

有的时候我们一个产品或者小控件有多种配置选择，这个时候就需要根据实际不同的场景选择不同的配置，本章我们就讲解一个能够在众多选项中选择其中某一个选项的控件，它就是 LISTBOX 控件。

本章分为以下几个部分：

25.1 LISTBOX 控件简介

25.2 通知代码

25.3 键盘反应

25.4 LISTBOX 控件 API 函数

25.5 LISTBOX 控件演示实验

25.1 LISTBOX 控件简介

LISTBOX 控件也叫列表框控件，用于选择列表中的某几个元素。创建的列表框可以没有环绕的框架窗口，也可以作为 FRAMEWIN 小工具的一个子窗口创建，当列表框中的某个项目被选中以后就会突出显示，LISTBOX 控件如下图所示：

有焦点的列表框	没有焦点的列表框
	

图 25.1.1 LISTBOX 控件

注意：LISTBOX 和 DROPDOWN 之间的区别，LISTBOX 一次可以选择多个项目，而 DROPDOWN 只能选择一项，而且 DROPDOWN 选择完成以后是收起来的，而 LISTBOX 是一直都在那里显示着。

25.2 通知代码

LISTBOX 小工具会作为 WM_NOTIFY_PARENT 消息的一部分发送给父窗口，通知代码如下表所示：

消息	描述
WM_NOTIFICATION_CLICKED	列表框已被点击
WM_NOTIFICATION_RELEASED	列表框已被释放
WM_NOTIFICATION_MOVED_OUT	列表框已被点击，且指针已从列表框处移开，没有释放
WM_NOTIFICATION_SCROLL_CHANGED	可选滚动条的滚动位置已更改
WM_NOTIFICATION_SEL_CHANGED	列表框的选择内容已更改

表 25.2.1 LISTBOX 通知代码

25.3 键盘反应

LISTBOX 控件也会对键盘的输入做出反应，如下表所示：

按键	反应
GUI_KEY_SPACE	如果小工具在多选择模式下操作，此键切换当前选定项目的状态
GUI_KEY_RIGHT	如果列表框项目的最大 X 尺寸超过了列表框本身，此键将列表框中的内容滚动至左边
GUI_KEY_LEFT	如果列表框项目的最大 X 尺寸超过了列表框本身，此键将列表框中的内容滚动至右边
GUI_KEY_DOWN	选择栏下移
GUI_KEY_UP	选择栏上移

表 25.3.1 LISTBOX 按键反应

25.4 LISTBOX 控件 API 函数

表 25.4.1 按字母顺序列出了可用的 LISTBOX 控件相关 API 函数：

函数	描述
LISTBOX_AddString()	向列表框添加项目
LISTBOX_Create()	创建 LISTBOX 小工具（弃用）
LISTBOX_CreateAsChild()	创建一个 LISTBOX 小工具作为子窗口（弃用）
LISTBOX_CreateEx()	创建 LISTBOX 小工具
LISTBOX_CreateIndirect()	从资源表项创建 LISTBOX 小工具
LISTBOX_CreateUser()	使用额外字节作为用户数据创建 LISTBOX 小工具
LISTBOX_DecSel()	减小选定范围
LISTBOX_DeleteItem()	删除某元素
LISTBOX_GetDefaultBkColor()	返回 LISTBOX 小工具的默认背景色
LISTBOX_GetDefaultFont()	返回 LISTBOX 小工具的默认字体
LISTBOX_GetDefaultScrollStepH()	返回要水平滚动的默认像素数值
LISTBOX_GetDefaultTextAlign()	返回新列表框的默认文本对齐方式
LISTBOX_GetDefaultTextColor()	返回新列表框的默认文本颜色
LISTBOX_GetFont()	返回列表框的字体
LISTBOX_GetItemDisabled()	返回给定项目的禁用状态
LISTBOX_GetItemSel()	返回 LISTBOX 项目的选取状态
LISTBOX_GetItemText()	返回列表框项目的文本
LISTBOX_GetMulti()	返回表示多选择模式是否激活的值
LISTBOX_GetNumItems()	返回列表框的项目数
LISTBOX_GetScrollStepH()	返回要水平滚动的像素数值
LISTBOX_GetSel()	返回选定的项目数
LISTBOX_GetTextAlign()	返回 LISTBOX 的文本对齐方式
LISTBOX_GetUserData()	检索用 LISTBOX_SetUserData 设置的数据
LISTBOX_IncSel()	增加选定范围
LISTBOX_InsertString()	插入某元素
LISTBOX_InvalidateItem()	使 LISTBOX 的项目失效
LISTBOX_OwnerDraw()	用于绘制 LISTBOX 项目的默认函数
LISTBOX_SetAutoScrollH()	激活自动使用水平滚动条
LISTBOX_SetAutoScrollV()	激活自动使用垂直滚动条
LISTBOX_SetBkColor()	设置背景颜色
LISTBOX_SetDefaultBkColor()	设置 LISTBOX 小工具的默认背景色
LISTBOX_SetDefaultFont()	改变 LISTBOX 小工具的默认字体
LISTBOX_SetDefaultScrollStepH()	设置水平滚动的默认像素数值
LISTBOX_SetDefaultTextAlign()	设置文本对齐方式

LISTBOX_SetDefaultTextColor()	设置 LISTBOX 小工具的默认文本颜色
LISTBOX_SetFont()	选择字体
LISTBOX_SetItemDisabled()	设置给定项目的禁用状态
LISTBOX_SetItemSel()	设置给定项目的选择状态
LISTBOX_SetItemSpacing()	设置项目之间的间距
LISTBOX_SetMulti()	设置多选择模式的开/关
LISTBOX_SetOwnerDraw()	启用列表框为自画
LISTBOX_SetScrollbarColor()	设置可选滚动条的颜色
LISTBOX_SetScrollbarWidth()	设置 LISTBOX 使用的滚动条的宽度
LISTBOX_SetScrollStepH()	设置水平滚动的像素数值
LISTBOX_SetSel()	设置选择的项目
LISTBOX_SetString()	设置元素的文本
LISTBOX_SetTextAlign()	设置 LISTBOX 的文本对齐方式
LISTBOX_SetTextColor()	设置前景颜色
LISTBOX_SetUserData()	设置 LISTBOX 小工具的额外数据

表 25.4.1 LISTBOX 控件 API 函数

下面我们讲解几个常用的比较重点的函数：

(1) LISTBOX_AddString 函数

向列表框中添加新的项目，其函数原型如下源码所示：

```
void LISTBOX_AddString(LISTBOX_Handle hObj, const char * s);
```

该函数的形参如下表所示：

参数	描述
hObj	LISTBOX 控件的句柄
pFont	要显示的文

表 25.4.2 LISTBOX_AddString 函数相关形参描述

返回值：无。

(2) LISTBOX_GetSel 函数

获取列表框中当前选中的项目的索引号，如果使用多重选择模式的话将返回焦点元素的索引，其函数原型如下源码所示：

```
int LISTBOX_GetSel(LISTBOX_Handle hObj)
```

该函数的形参如下表所示：

参数	描述
hObj	LISTBOX 控件的句柄

表 25.4.3 LISTBOX_GetSel 函数相关形参描述

返回值：当前选中的项目的索引，如果未选定任何元素的话将返回-1。

(3) LISTBOX_GetItemText 函数

获取列表框中指定的项目的文本，其函数原型如下源码所示：

```
void LISTBOX_GetItemText(LISTBOX_Handle hObj, unsigned Index,
                        char * pBuffer, int MaxSize);
```

该函数的形参如下表所示：

参数	描述
hObj	列表框的句柄
Index	以零为基准的项目索引
pBuffer	存储项目文本所使用的缓冲区指
MaxSize	缓冲区大小

表 25.4.4 LISTBOX_GetItemText 函数相关形参描述

此函数会将指定的项目的文本复制到 pBuffer 中。

返回值：无。

25.5 LISTBOX 控件演示实验

25.5.1 硬件设计

1. 例程功能

本实验主要讲解 LISTBOX 小工具的使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。注意：本实验会用到 LED 以及 BEEP 外设，这些外设的相关知识请大家查看正点原子裸机教程，这里小编并不会讲解这些知识。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数主要创建一个 window 小工具并在其上面添加一个 LISTBOX 和 “OK”、“CANCEL” 按键，当用户选择 LISTBOX 控件的某一个选项时可以按下 “OK” 按键把这个选项的信息发送到对话框，对话框会根据这用户选择的选项信息做相应的处理，比如关闭和打开 LED 灯；当用户按下 “CANCEL” 按键时，系统隐藏对话框并开启定时器定时 3s，定时时间到来时系统自动显示对话框。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 25 TEXT 小工具》。

25.5.2 软件设计

25.5.2.1 程序流程图

本实验的程序流程图，如下图所示：

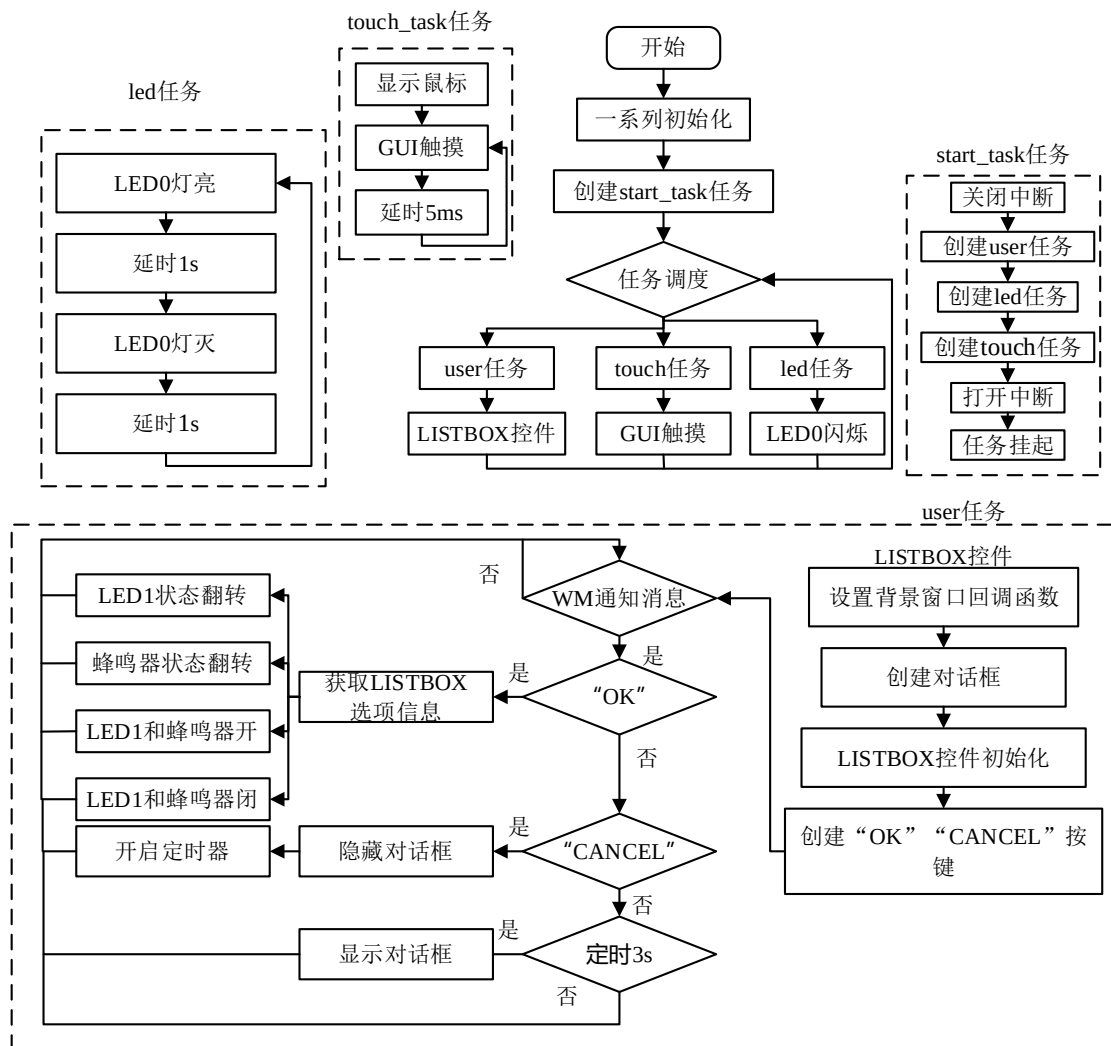


图 25.5.2.1.1LISTBOX 控件实验流程图

25.5.2.2 程序解析

本章我们利用 LISTBOX 控件来控制开发板蜂鸣器和 LED1 的，预先设置好几种控制方案，然后选择不同的控制方案来达到不同的控制效果。我们也可以添加一些其他的功能来学习巩固一下前面学过的控件，比如通过 TEXT 控件显示所选中的项目文本等。打开工程 **emWin 实验 26 LISTBOX 小工具** 中的文件 `emwin_demo.c`，这个文件就是本章实验源码，这个实验源码如下所示：

(1) GUIBulider 生成的资源表

```
/* 对话框资源表 */
static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] =
{
    { FRAMEWIN_CreateIndirect, "Framewin", ID_FRAMEWIN_0, 0, 0, 320, 240,
      FRAMEWIN_CF_MOVEABLE, 0x64, 0 },
    { LISTBOX_CreateIndirect, "Listbox", ID_LISTBOX_0, 21, 20, 115, 128, 0, 0x0, 0 },
    { TEXT_CreateIndirect, "Text", ID_TEXT_0, 170, 70, 133, 35, 0, 0x0, 0 },
    { BUTTON_CreateIndirect, "Button", ID_BUTTON_0, 30, 160, 90, 40, 0, 0x0, 0 },

```

```
{ BUTTON_CreateIndirect, "Button", ID_BUTTON_1, 175, 160, 90, 40, 0, 0x0, 0 },
};
```

(2) 创建对话框

```
/**
 * @brief      emwin_main(emwin 入口)
 * @param      无
 * @return     无
 */
void emwin_main(void)
{
    WM_SetCallback(WM_HBKWIN, _cbBkWindow); /* 设置背景窗口回调函数 */
    GUI_CreateDialogBox(_aDialogCreate, GUI_COUNTOF(_aDialogCreate), _cbDialog,
                        WM_HBKWIN, 0, 0);
}
```

此函数主要调用函数 WM_SetCallback 添加回调函数，然后调用函数 GUI_CreateDialogBox 根据资源表和回调函数 _cbDialog 构建对话框，下面我们来看一下窗口回调函数源码，如下所示：

(3) 重绘回调函数

```
/**
 * @brief      背景窗口 WM_HBKWIN 回调函数
 * @param      pMsg : 传入消息
 * @return     无
 */
static void _cbBkWindow(WM_MESSAGE *pMsg)
{
    static WM_HWIN hWin;

    switch (pMsg->MsgId)
    {
        case WM_PAINT:
            GUI_SetBkColor(GUI_BLUE);
            GUI_Clear();
            GUI_SetColor(GUI_YELLOW);
            GUI_SetFont(&GUI_Font24B_ASCII);
            GUI_DispStringAt("正点原子 LISTBOX DEMO", 0, 0);
            break;

        case WM_DELETEWIN: /* 删除窗口 */
            hWin = pMsg->hWinSrc;
            WM_DeleteWindow(hWin);
            WM_InvalidateWindow(WM_HBKWIN); /* 背景窗口无效 */
            WM_CreateTimer(WM_HBKWIN, 0, 3000, 0); /* 创建定时器 */
            break;
    }
}
```

```

        case WM_TIMER:
            GUI_CreateDialogBox(_aDialogCreate, GUI_COUNTOF(_aDialogCreate),
                                _cbDialog, WM_HBKWIN, 0, 0); /* 创建对话框 */

            break;

        default:
            WM_DefaultProc(pMsg);
    }
}

```

此函数主要处理窗口重绘操作，当 CANCEL 键被按下时，该按键会发送一个消息 WM_DELETEWIN 并设置窗口无效化，这个消息会删除当前窗口并开启定时器 3 秒。下面我们来看一下对话框的回调函数，如下源码所示：

(4) 对话框回调函数

```

/**
 * @brief      对话框回调函数
 * @param      pMsg : 传入消息
 * @return     无
 */
static void _cbDialog(WM_MESSAGE *pMsg)
{
    WM_HWIN hItem;
    int      NCode;
    int      Id;
    static char textbuf[20];
    WM_MESSAGE Msg;

    switch (pMsg->MsgId)
    {
        case WM_INIT_DIALOG:
            /* FRAMWIN 控件初始化 */
            hItem = pMsg->hWin;
            FRAMEWIN_SetFont(hItem, GUI_FONT_20B_ASCII);
            FRAMEWIN_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
            FRAMEWIN_SetText(hItem, "正点原子 LISTBOX");
            FRAMEWIN_SetTextColor(hItem, GUI_YELLOW);

            /* LISTBOX 控件初始化 */
            hItem = WM_GetDialogItem(pMsg->hWin, ID_LISTBOX_0);
            LISTBOX_SetFont(hItem, GUI_FONT_20_ASCII);
            LISTBOX_SetAutoScrollH(hItem, 1); /* 激活自动使用水平滚动条 */
            LISTBOX_SetAutoScrollV(hItem, 1); /* 激活自动使用垂直滚动条 */
            LISTBOX_AddString(hItem, "LED1 ON");
    }
}

```

```

LISTBOX_AddString(hItem, "LED1 OFF");
LISTBOX_AddString(hItem, "BEEP ON");
LISTBOX_AddString(hItem, "BEEP OFF");
LISTBOX_AddString(hItem, "ALL ON");
LISTBOX_AddString(hItem, "ALL OFF");

/* CHECKBOX 控件初始化 */
hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_0);
TEXT_SetTextColor(hItem, GUI_RED);
TEXT_SetFont(hItem, GUI_FONT_24_ASCII);
TEXT_SetText(hItem, "NO SELECT!");

/* BUTTON 控件初始化 */
hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_0);
BUTTON_SetText(hItem, "OK");
BUTTON_SetFont(hItem, GUI_FONT_COMIC18B_ASCII);

hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_1);
BUTTON_SetFont(hItem, GUI_FONT_COMIC18B_ASCII);
BUTTON_SetText(hItem, "CANCEL");
break;

case WM_NOTIFY_PARENT:
    Id = WM_GetId(pMsg->hWinSrc);
    NCode = pMsg->Data.v;

    switch (Id)
    {
        case ID_LISTBOX_0:
            switch (NCode)
            {
                case WM_NOTIFICATION_CLICKED:
                    break;

                case WM_NOTIFICATION_RELEASED:
                    hItem = WM_GetDialogItem(pMsg->hWin, ID_LISTBOX_0);
                    /* 获取选中的项目标号 */
                    listboxitem = LISTBOX_GetSel(hItem);
                    /* 获取选中的条目的内容 */
                    LISTBOX_GetItemText(hItem, listboxitem, textbuf, 10);
                    break;

                case WM_NOTIFICATION_SEL_CHANGED:

```

```

        break;
    }
    break;

case ID_BUTTON_0:
    switch (NCode)
    {
        case WM_NOTIFICATION_CLICKED:
            break;

        case WM_NOTIFICATION_RELEASED:
            switch (listboxitem)
            {
                case 0:
                    LED1(0);    /* LED1 打开 */
                    break;

                case 1:
                    LED1(1);    /* LED1 关闭 */
                    break;

                case 2:
                    BEEP(1);    /* 蜂鸣器打开 */
                    break;

                case 3:
                    BEEP(0);    /* 蜂鸣器关闭 */
                    break;

                case 4:
                    LED1(0);    /* LED1 打开 */
                    BEEP(1);    /* 蜂鸣器打开 */
                    break;

                case 5:
                    LED1(1);    /* LED1 关闭 */
                    BEEP(0);    /* 蜂鸣器关闭 */
                    break;
            }

            if (listboxitem >= 0)
            {
                /* 在 TEXT 控件中显示选中的内容 */
            }
    }
}

```

```

        hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_0);
        TEXT_SetText(hItem, textbuf);
    }

    break;

}

break;

case ID_BUTTON_1:
    switch (NCode)
    {
        case WM_NOTIFICATION_CLICKED:
            break;

        case WM_NOTIFICATION_RELEASED:
            Msg.MsgId = WM_DELETEWIN;
            Msg.hWinSrc = pMsg->hWin;
            WM_SendMessage(WM_HBKWIN, &Msg);
            break;
    }
    break;
}

break;
default:
    WM_DefaultProc(pMsg);
    break;
}
}

```

在上述的源码中，当用户按下 CANCEL 键时，系统会向背景窗口发送消息 WM_DELETEWIN，当背景窗口回调函数收到这个消息以后就会删除对话框；当用户按下“OK”按键时，系统会根据 listboxitem 控件的选项信息触发相应的动作。

25.5.3 下载验证

编译并下载例程到开发板中，LCD 显示如下图所示：

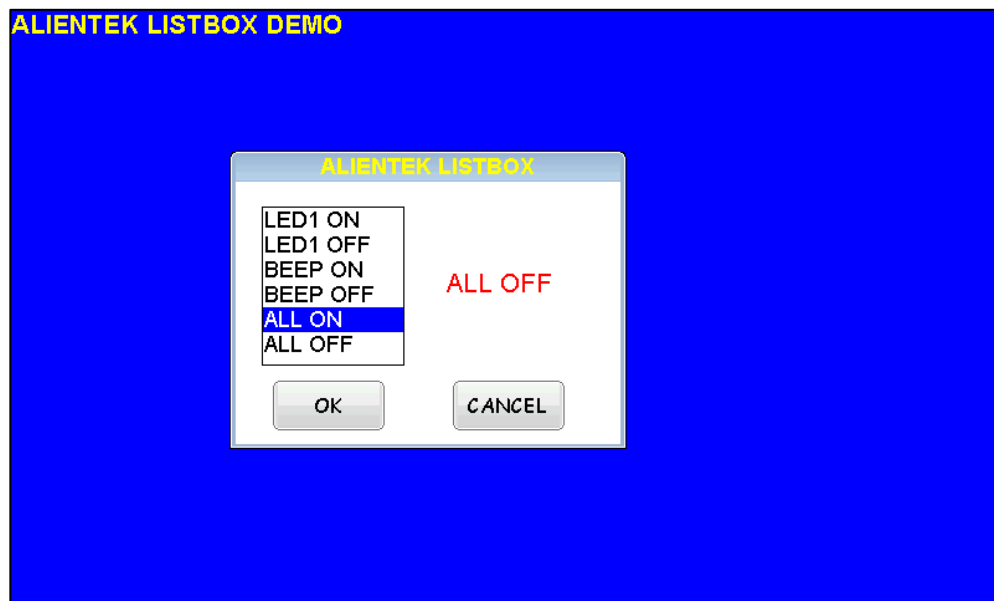


图 25.5.3.1 LISTBOX 控件例程

在图 25.5.3.1 中，当我们选择“ALL ON”并按下 OK 键以后就会同时打开 LED1 和蜂鸣器，当按下 CANCEL 键就会删除对话框，等待 3S 后对话框又会被创建出来。注意图中的 TEXT 控件显示的“ALL OFF”是因为此时只是选中了 LISTBOX 对话框中的“ALL ON”项，但是还没有按下 OK 键，所以此时的 TEXT 控件显示的还是上一次的设置。

第二十六章 LISTVIEW 控件

上一章讲的 LISTBOX 控件只有一列数据，有时候我们需要显示多列数据，比如显示学生成绩的时候需要一列显示学生姓名，一列显示班级，不同的课程还要不同的列来显示其成绩等等，显然，这个时候 LISTBOX 就无能为力了。幸好 emWin 给我们提供了一个解决方法，那就是 LISTVIEW 控件，LISTVIEW 控件是一个类似 excel 的控件，可以很好的帮我们解决需要显示多列信息的应用场合，本章我们就学习 LISTVIEW 控件的使用方法。

本章分为以下几个部分：

26.1 LISTVIEW 控件简介

26.2 通知代码

26.3 键盘反应

26.4 LISTVIEW 控件 API 函数

26.5 LISTVIEW 控件演示实验

26.1 LISTVIEW 控件简介

LISTVIEW 控件也叫做列表控件，LISTVIEW 控件可以在具有多个列的列表中选择某个元素，LISTVIEW 控件包含了一个 HEADER 控件，这样可以通过 HEADER 来管理列。当我们选中 LISTVIEW 中的某行时此行就会突出显示，LISTVIEW 控件如下图所示：

EAN	Order #	Description
623499-0010002	2	Item 2
623499-0010003	3	Item 3
623499-0010004	4	Item 4
623499-0010005	5	Item 5
623499-0010006	6	Item 6
623499-0010007	7	Item 7

图 26.1.1 LISTVIEW 控件

26.2 通知代码

LISTVIEW 小工具会作为 WM_NOTIFY_PARENT 消息的一部分发送给父窗口，通知代码如下表所示：

消息	描述
WM_NOTIFICATION_CLICKED	小工具被点击
WM_NOTIFICATION_RELEASED	小工具已被释放
WM_NOTIFICATION_MOVED_OUT	小工具已被点击，且指针已从列表框处移开，没有释放
WM_NOTIFICATION_SCROLL_CHANGED	可选滚动条的滚动位置已更改
WM_NOTIFICATION_SEL_CHANGED	小工具的选择内容已更改

表 26.2.1 LISTVIEW 通知代码

26.3 键盘反应

LISTVIEW 控件也会对键盘的输入做出反应，如下表所示：

按键	反应
GUI_KEY_UP	选择栏上移
GUI_KEY_DOWN	选择栏下移
GUI_KEY_RIGHT	如果总列宽>列表视图的内部区域，则内容将滚动到左侧
GUI_KEY_LEFT	如果总列宽>列表视图的内部区域，则内容将滚动到右侧

表 26.3.1 LISTVIEW 键盘反应

26.4 LISTVIEW 控件 API 函数

表 26.4.1 按字母顺序列出了可用的 LISTVIEW 控件相关 API 函数：

函数	描述
LISTVIEW_AddColumn()	向 LISTVIEW 添加列
LISTVIEW_AddRow()	向 LISTVIEW 添加行
LISTVIEW_CompareDec()	比较 2 个整数值所使用的比较函数
LISTVIEW_CompareText()	比较 2 个字符串所使用的比较函数
LISTVIEW_Create()	创建 LISTVIEW 小工具（弃用）
LISTVIEW_CreateAttached()	创建附加到窗口的 LISTVIEW 小工具
LISTVIEW_CreateEx()	创建 LISTVIEW 小工具
LISTVIEW_CreateIndirect()	从资源表条目创建 LISTVIEW 小工具
LISTVIEW_CreateUser()	使用额外字节作为用户数据来创建 LISTVIEW 小工具
LISTVIEW_DecSel()	减小选定范围
LISTVIEW_DeleteColumn()	删除指定的列
LISTVIEW_DeleteRow()	删除指定的行
LISTVIEW_DisableRow()	将指定行的状态设置为禁用
LISTVIEW_DisableSort()	禁用 LISTVIEW 排序功能
LISTVIEW_EnableCellSelect()	使能/失能指定单元格
LISTVIEW_EnableRow()	将指定行的状态设置为启用
LISTVIEW_EnableSort()	启用 LISTVIEW 排序功能
LISTVIEW_GetBkColor()	返回 LISTVIEW 的背景颜色
LISTVIEW_GetFont()	返回 LISTVIEW 的字体
LISTVIEW_GetHeader()	返回附加 HEADER 小工具的句柄
LISTVIEW_GetItemRect()	返回指定单元格矩形的坐标
LISTVIEW_GetItemText()	返回指定单元格的文本
LISTVIEW_GetNumColumns()	返回列数
LISTVIEW_GetNumRows()	返回行数
LISTVIEW_GetSel()	返回选定的项目数
LISTVIEW_GetSelUnsorted()	返回未排序状态下选定的项目数
LISTVIEW_GetTextColor()	返回 LISTVIEW 的文本颜色
LISTVIEW_GetUserData()	检索通过 LISTVIEW_SetUserData 所设置的数据
LISTVIEW_GetUserDataRow()	返回指定行的用户数据
LISTVIEW_IncSel()	增加选定范围
LISTVIEW_InsertRow()	在指定位置新插入一行
LISTVIEW_SetAutoScrollH()	启用自动使用水平滚动条
LISTVIEW_SetAutoScrollV()	启用自动使用垂直滚动条
LISTVIEW_SetBkColor()	设置背景颜色
LISTVIEW_SetColumnWidth()	设置列宽
LISTVIEW_SetCompareFunc()	设置指定列的比较函数
LISTVIEW_SetDefaultBkColor()	设置 HEADER 小工具的默认背景颜色

LISTVIEW_SetDefaultFont()	设置 HEADER 小工具的默认字体
LISTVIEW_SetDefaultGridColor()	设置 HEADER 小工具的默认文本颜色
LISTVIEW_SetDefaultTextColor()	设置 HEADER 小工具网格线的颜色
LISTVIEW_SetFixed()	固定指定的列数
LISTVIEW_SetFont()	设置 LISTVIEW 的字体
LISTVIEW_SetGridVis()	设置网格线的可见性标记
LISTVIEW_SetHeaderHeight()	设置标题的高度
LISTVIEW_SetItemBitmap()	将某个位图设置为 LISTVIEW 单元格的背景
LISTVIEW_SetItemBkColor()	设置 LISTVIEW 单元格的背景颜色
LISTVIEW_SetItemText()	设置 LISTVIEW 单元格的文本
LISTVIEW_SetItemTextColor()	设置 LISTVIEW 单元格的文本颜色
LISTVIEW_SetLBorder()	设置左边框所使用的像素数
LISTVIEW_SetRBorder()	设置右边框所使用的像素数
LISTVIEW_SetRowHeight()	设置 LISTVIEW 的行高
LISTVIEW_SetSel()	设置当前选定内容
LISTVIEW_SetSelUnsorted()	设置未排序状态下的当前选定内容
LISTVIEW_SetSort()	设置列以及排序依据的排序顺序
LISTVIEW_SetTextAlign()	设置列的文本对齐方式
LISTVIEW_SetTextColor()	设置文本颜色
LISTVIEW_SetUserData()	设置 LISTVIEW 小工具的额外数
LISTVIEW_SetUserDataRow()	设置指定行的用户数据

表 26.4.1 LISTVIEW 控件 API 函数

下面我们来看几个常用的比较重点的函数：

(1) LISTVIEW_AddColumn 函数

向 LISTVIEW 控件中添加新列，其函数的原型如下源码所示：

```
void LISTVIEW_AddColumn( LISTVIEW_Handle hObj,
                        int Width,
                        const char *s,
                        int Align)
```

该函数的形参如下表所示：

参数	描述
hObj	LISTVIEW 控件句柄
width	新增列的列宽
s	在 HEADER 小工具中将要显示的文本
Align	Align 要设置的文本对齐模式，水平和垂直标记可以通过“OR”操作进行组合
	水平模式：
	GUI_TA_LEFT X 轴方向左对齐
	GUI_TA_HCENTER X 轴方向居中对齐
	GUI_TA_RIGHT X 轴方向右对齐
	垂直模式：
	GUI_TA_TOP Y 轴位置与字符的顶部对齐
	GUI_TA_VCENTER Y 轴方向居中对齐
	GUI_TA_BOTTOM Y 轴位置与字体的底部像素行对齐

表 26.4.2 LISTVIEW_AddColumn 函数相关形参描述

返回值：无。

(2) LISTVIEW_AddRow 函数

向 LISTVIEW 控件添加一行，其函数的原型如下源码所示：

```
void LISTVIEW_AddRow(LISTVIEW_Handle hObj, const GUI_ConstString * ppText)
```

该函数的形参如下表所示：

参数	描述
hObj	LISTVIEW 控件句柄
ppText	包含 LISTVIEW 单元格文本的阵列指针

表 26.4.3 LISTVIEW_AddRow 函数相关形参描述

注意：该函数的 ppText 参数应包含每列的一个项目，如果它包含的项目较少的时候其余单元格会留空。

返回值：无。

(3) LISTVIEW_GetSel 函数

获取当前选中的行索引,其函数的原型如下源码所示：

```
int LISTVIEW_GetSel(LISTVIEW_Handle hObj)
```

该函数的形参如下表所示：

参数	描述
hObj	LISTVIEW 控件句柄

表 26.4.4 LISTVIEW_GetSel 函数相关形参描述

返回值：当前所选中的行索引。

(4) LISTVIEW_GetItemText 函数

获取指定单元格的文本，单元格的文本被复制到指定的缓冲区中，其函数的原型如下源码所示：

```
void LISTVIEW_GetItemText( LISTVIEW_Handle hObj,unsigned Column,unsigned Row,
                           char *pBuffer,unsigned MaxSize);
```

该函数的形参如下表所示：

参数	描述
hObj	小工具的句柄
Column	单元格列的索引（以零为基准）
Row	单元格行的索引（以零为基准）
pBuffer	由例程填充的缓冲区指针
MaxSize	缓冲区大小（以字节为单位）

表 26.4.5 LISTVIEW_GetItemText 函数相关形参描述

如果要复制的文本超过了缓冲区大小 MaxSize 的话就只会复制 MaxSize 字节的数据到缓冲区中。

返回值：无。

26.5 LISTVIEW 控件演示实验

26.5.1 硬件设计

1. 例程功能

本实验主要讲解 LISTVIEW 小工具的使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。注意：本实验会用到 LED 以及 BEEP 外设，这些外设的相关知识请大家查看正点原子裸机教程，这里小编并不会讲解这些知识。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数主要创建一个 window 小工具并在其上面添加 LISTVIEW 控件、“OK” 按键和多个 TXET 控件，我们在 LISTVIEW 控件上添加多个单元格，在这些单元格设置了学生的基本信息，当用户按下某一个单元格并按下“OK” 按键时，系统会获取按下单元格信息并在 TEXT 控件显示这些信息。注意：按下“OK” 按键时，系统会把对话框隐藏起来并定时 3s，当定时时间到了，系统自动弹出对话框。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 27 LISTVIEW 小工具》。

26.5.2 软件设计

26.5.2.1 程序流程图

本实验的程序流程图，如下图所示：

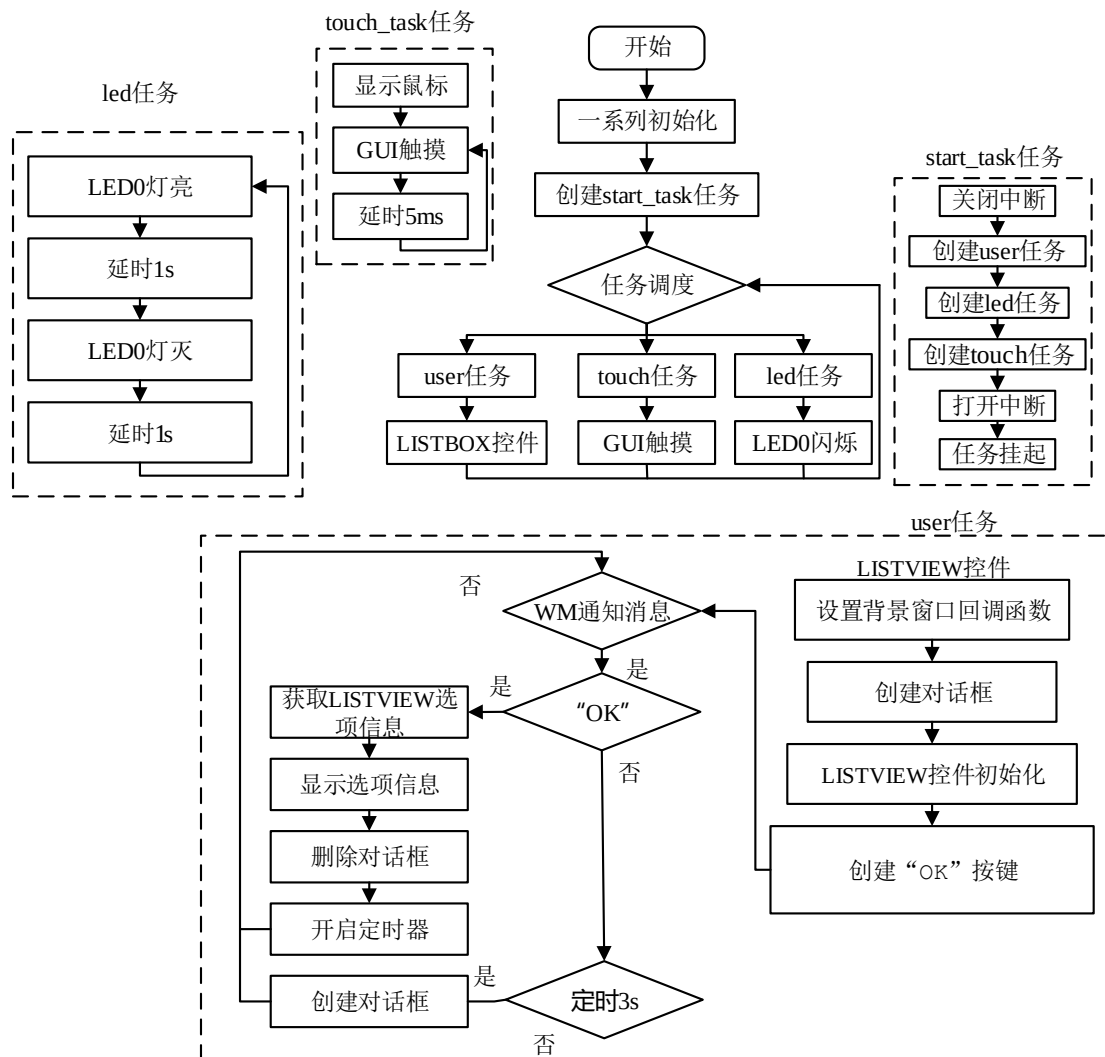


图 26.5.2.1.1LISTVIEW 控件实验流程图

26.5.2.2 程序解析

本章我们使用 LISTVIEW 控件来显示学生考试信息，当选中某个学生的时候点击 OK 键此学生的信息就会显示到背景桌面上，通过本例程可以学习 LISTVIEW 控件常用的用法。打开工程 **emWin 实验 27 LISTVIEW** 中的文件 `emwin_demo.c`，函数 `emwin_main` 用来创建本实验的界面，代码如下：

(1) 资源表

```
/* 资源表 */
static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] =
{
    { FRAMEWIN_CreateIndirect, "Framewin", ID_FRAMEWIN_0, 0, 0, 320, 240,
      FRAMEWIN_CF_MOVEABLE, 0x64, 0 },
    { LISTVIEW_CreateIndirect, "Listview", ID_LISTVIEW_0, 45, 15, 220, 140, 0, 0x0, 0 },
    { BUTTON_CreateIndirect, "Button", ID_BUTTON_0, 100, 160, 100, 40, 0, 0x0, 0 },
};
```

在软件 GUIBulider 创建一个框架窗口、Listview 和 Button。

(2) 创建对话框

```
/**
 * @brief      emwin_main(emwin 入口)
 * @param      无
 * @retval     无
 */
void emwin_main(void)
{
    WM_SetCallback(WM_HBKWIN, _cbBkWindow); /* 设置背景窗口回调函数 */
    GUI_CreateDialogBox(_aDialogCreate, GUI_COUNTOF(_aDialogCreate),
                        _cbDialog, WM_HBKWIN, 0, 0);
}
```

此函数主要调用 `GUI_CreateDialogBox` 函数根据资源表以及回调函数 `_aDialogCreate` 创建对话框，该回调函数 `_aDialogCreate` 函数如下所示：

(3) 对话框的回调函数

```
/**
 * @brief      对话框回调函数
 * @param      pMsg : 消息
 * @retval     无
 */
static void _cbDialog(WM_MESSAGE *pMsg)
{
    WM_HWIN hItem;
    int      NCode;
    int      Id;
    WM_MESSAGE Msg;
    uint8_t i = 0;
```

```

switch (pMsg->MsgId)
{
    case WM_INIT_DIALOG:
        /* 初始化 FRAMWIN */
        hItem = pMsg->hWin;
        FRAMEWIN_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
        FRAMEWIN_SetFont(hItem, GUI_FONT_20B_ASCII);
        FRAMEWIN_SetTextColor(hItem, GUI_YELLOW);
        FRAMEWIN_SetText(hItem, "ALIENTEK LISTVIEW");

        /* 初始化 LISTVIEW */
        hItem = WM_GetDialogItem(pMsg->hWin, ID_LISTVIEW_0);
        LISTVIEW_SetGridVis(hItem, 1);           /* 网格线可见 */
        LISTVIEW_SetAutoScrollH(hItem, 1);       /* 启用自动水平滚动条 */
        LISTVIEW_SetAutoScrollV(hItem, 1);       /* 启用自动垂直滚动条 */
        LISTVIEW_SetHeaderHeight(hItem, 20);     /* 设置标题的高度 */
        LISTVIEW_SetRowHeight(hItem, 15);        /* 设置 LISTVIEW 的行高 */
        LISTVIEW_SetBkColor(hItem, 0, 0X00C080FF); /* 设置背景颜色 */
        /* 向 LISTVIEW 添加列 */
        LISTVIEW_AddColumn(hItem, 50, "Name", GUI_TA_HCENTER |
                           GUI_TA_VCENTER);
        LISTVIEW_AddColumn(hItem, 80, "Num", GUI_TA_HCENTER |
                           GUI_TA_VCENTER);
        LISTVIEW_AddColumn(hItem, 40, "Score", GUI_TA_HCENTER |
                           GUI_TA_VCENTER);
        LISTVIEW_AddColumn(hItem, 35, "Grade", GUI_TA_HCENTER |
                           GUI_TA_VCENTER);

        /* 向 LISTVIEW 添加行 */
        for (i = 0; i < GUI_COUNTOF(ListViewTable); i++)
        {
            LISTVIEW_AddRow(hItem, ListViewTable[i]);
        }

        /* 初始化 BUTTON */
        hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_0);
        BUTTON_SetFont(hItem, GUI_FONT_COMIC18B_ASCII);
        BUTTON_SetText(hItem, "OK");
        break;

    case WM_NOTIFY_PARENT:
        Id = WM_GetId(pMsg->hWinSrc);
        NCode = pMsg->Data.v;

```

```

switch (Id)
{
    case ID_LISTVIEW_0:
        switch (NCode)
        {
            case WM_NOTIFICATION_CLICKED:
                break;

            case WM_NOTIFICATION_RELEASED:
                hItem = WM_GetDialogItem(pMsg->hWin, ID_LISTVIEW_0);
                /* 获取选中的项目编号 */
                listviewitem = LISTVIEW_GetSel(hItem);
                /* 返回指定单元格的文本 */
                LISTVIEW_GetItemText(hItem, 0, listviewitem,
                    (char *)student.name, 20); /* 获取名字 */
                LISTVIEW_GetItemText(hItem, 1, listviewitem,
                    (char *)student.num, 20); /* 获取学号 */
                LISTVIEW_GetItemText(hItem, 2, listviewitem,
                    (char *) (student.score), 4); /* 获取分数 */
                LISTVIEW_GetItemText(hItem, 3, listviewitem,
                    (char *) & (student.grade), 4); /* 获取等级 */
                break;

            case WM_NOTIFICATION_SEL_CHANGED:
                break;
        }
        break;

    case ID_BUTTON_0:
        switch (NCode)
        {
            case WM_NOTIFICATION_CLICKED:
                break;

            case WM_NOTIFICATION_RELEASED:
                Msg.MsgId = WM_DELETEWIN;
                Msg.hWinSrc = pMsg->hWin;
                WM_SendMessage(WM_HBKWIN, &Msg); /* 发送消息 */
                break;
        }

        break;
}

```



```

    }

    break;

    default:
        WM_DefaultProc(pMsg);
        break;
}
}

```

从上述的源码，也不难理解，下面我们分几段来讲解上述的源码，首先我们看一下 ID_LISTVIEW_0 的源码，如以下所示：

```

case ID_LISTVIEW_0:
    switch (NCode)
    {
        case WM_NOTIFICATION_CLICKED:
            break;

        case WM_NOTIFICATION_RELEASED:
            hItem = WM_GetDialogItem(pMsg->hWin, ID_LISTVIEW_0);
            /* 获取选中的项目编号 */
            listviewitem = LISTVIEW_GetSel(hItem);
            /* 返回指定单元格的文本 */
            LISTVIEW_GetItemText(hItem, 0, listviewitem,
                                (char *)student.name, 20); /* 获取名字 */
            LISTVIEW_GetItemText(hItem, 1, listviewitem,
                                (char *)student.num, 20); /* 获取学号 */
            LISTVIEW_GetItemText(hItem, 2, listviewitem,
                                (char *) (student.score), 4); /* 获取分数 */
            LISTVIEW_GetItemText(hItem, 3, listviewitem,
                                (char *) & (student.grade), 4); /* 获取等级 */

            break;

        case WM_NOTIFICATION_SEL_CHANGED:
            break;
    }
break;

```

此函数可以看出，当 ListView 的表格任意一行被按下时，都会进入 WM_NOTIFICATION_RELEASED 里面，系统调用函数 LISTVIEW_GetItemText 获取被按下的列表的信息，然后我们看一下 ID_BUTTON_0 控件的源码，如下所示：

```

case ID_BUTTON_0:
    switch (NCode)
    {
        case WM_NOTIFICATION_CLICKED:
            break;
    }

```

```

        case WM_NOTIFICATION_RELEASED:
            Msg.MsgId = WM_DELETEWIN;
            Msg.hWinSrc = pMsg->hWin;
            WM_SendMessage(WM_HBKWIN, &Msg); /* 发送消息 */
            break;
    }
    break;

```

上述的源码可知：如果 **BUTTON** 按下时，获取 `MsgId = WM_DELETEWIN`（窗口删除）并发送信息，进行重绘操作。

背景窗口回调函数源码，如下所示：

```

/**
 * @brief      背景窗口 WM_HBKWIN 回调函数
 * @param      pMsg : 消息
 * @retval     无
 */
static void _cbBkWindow(WM_MESSAGE *pMsg)
{
    static WM_HWIN hWin;

    switch (pMsg->MsgId)
    {
        case WM_PAINT:
            GUI_SetBkColor(GUI_BLUE);
            GUI_Clear();
            GUI_SetColor(GUI_YELLOW);
            GUI_SetFont(&GUI_Font24B_ASCII);
            GUI_DispStringAt("ALIENTEK LISTVIEW DEMO", 0, 0);
            GUI_SetColor(GUI_BLACK);
            GUI_SetFont(&GUI_Font20_ASCII);
            GUI_DispStringAt(" Name:", 0, 30);
            GUI_DispStringAt(" Num:", 0, 50);
            GUI_DispStringAt("Score:", 0, 70);
            GUI_DispStringAt("Grade:", 0, 90);

            GUI_SetColor(GUI_RED);
            GUI_DispStringAt((char *)student.name, 65, 30);/* 显示选中的学生名字 */
            GUI_DispStringAt((char *)student.num, 65, 50); /* 显示选中的学生学号 */
            GUI_DispStringAt((char *)student.score, 65, 70);/*显示选中的学生分数 */
            /* 显示选中的学生的等级 */
            GUI_DispStringAt((char *) & (student.grade), 65, 90);
            break;
    }
}

```

```

case WM_DELETEWIN:                                /* 删除窗口 */
    hWin = pMsg->hWinSrc;
    WM_DeleteWindow(hWin);
    WM_InvalidateWindow(WM_HBKWIN);                /* 背景窗口无效 */
    WM_CreateTimer(WM_HBKWIN, 0, 3000, 0);          /* 创建定时器 */
    break;

case WM_TIMER:
    GUI_CreateDialogBox(_aDialogCreate, GUI_COUNTOF(_aDialogCreate),
                        _cbDialog, WM_HBKWIN, 0, 0); /* 创建对话框 */
    break;

default:
    WM_DefaultProc(pMsg);
}
}

```

从上述的源码可以看出，如果 Button 按下时，会进入 WM_DELETEWIN 删除窗口并开启定时器 3 秒，当定时时间到了，重新创建对话框。

26.5.3 下载验证

编译并下载例程到开发板中，LCD 显示如下图所示：

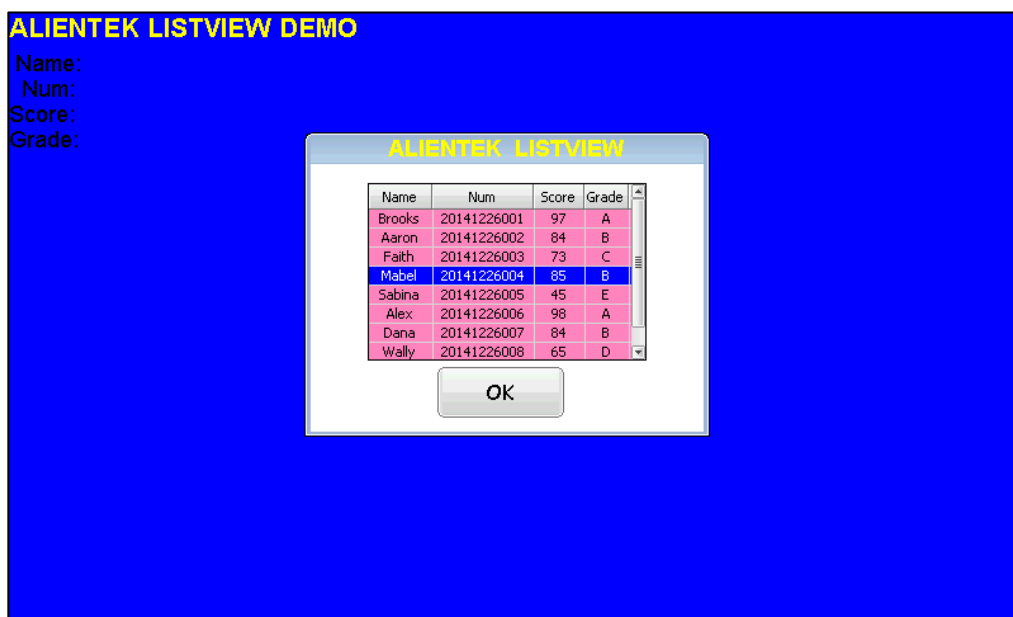


图 26.5.3.1 LISTVIEW 控件例程显示

选择一个学生的信息并且按下 OK 键，这个学生的成绩信息就显示到了背景桌面上，如下图所示。

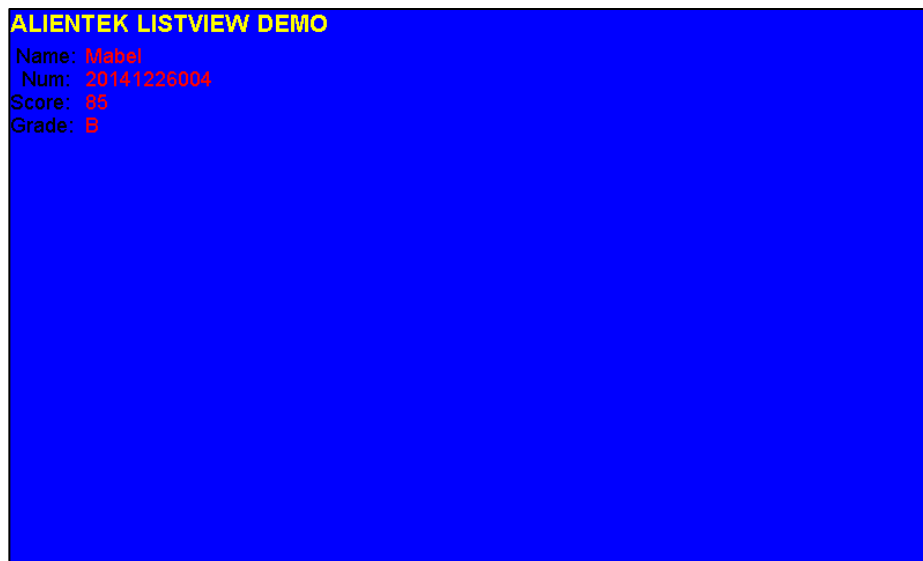


图 26.5.3.2 显示学生成绩信息

第二十七章 LISTWHEEL 控件

很多智能手机在调节时间或者闹钟的时候都是通过滚动一个滚轮一样的东西来选择想要设置的时间，emWin 也提供了一个这样的控件，它就是 LISTWHEEL 控件。我们要注意的，LISTWHEEL 控件是基于指针输入设备(PID)操作的，所以需要有鼠标或者触摸屏这类可以控制指针的设备。

本章分为以下几个部分：

27.1 LISTWHEEL 控件简介

27.2 通知代码

27.3 LISTWHEEL 控件 API 函数

27.4 LISTWHEEL 控件演示实验

27.1 LISTWHEEL 控件简介

LISTWHEEL 控件是从众多的选项中选出某一个，这个和前面讲的 LISTBOX 控件很类似。但是，LISTBOX 除了可以使用触摸屏以外还可以使用键盘来选择所需的选项，而 LISTWHEEL 整个数据区的移动可以通过指针输入设备来实现，当我们从上到下(或者从下到上)触发控件的时候可将数据向下(向上)滚动。如果释放指针的话那么数据区的滚动速度就会放慢，并且最终吸附在一个新的数据项并停下来。另外，数据是以循环的方式显示的，第一个数据项会在最后一项之后继续显示，如同链条一样。因此，数据如同在转轮上一样进行“旋转”，LISTWHEEL 控件如下图所示：



图 27.1.1 LISTWHEEL 控件

注意：图 27.1.1 中所示的 LISTWHEEL 控件是经过复杂的美化换肤以后的效果，默认的效果是远没有这么漂亮的。

27.2 通知代码

LISTWHEEL 小工具会作为 WM_NOTIFY_PARENT 消息的一部分发送给父窗口，通知代码如下表所示：

消息	描述
WM_NOTIFICATION_CLICKED	已单击小工具
WM_NOTIFICATION_RELEASED	已释放小工具
WM_NOTIFICATION_MOVED_OUT	已单击小工具，并且指针已移出小工具，但没有释放
WM_NOTIFICATION_SEL_CHANGED	已在吸附位置吸附一个项

表 27.2.1 LISTWHEEL 通知代码

27.3 LISTWHEEL 控件 API 函数

表 27.3.1 按字母顺序列出了可用的 LISTWHEEL 控件相关 API 函数：

函数	描述
LISTWHEEL_AddString()	添加新字符串
LISTWHEEL_CreateEx()	创建 LISTWHEEL 小工具
LISTWHEEL_CreateIndirect()	从资源表条目创建 LISTWHEEL 小工具
LISTWHEEL_CreateUser()	使用作为用户数据的额外字节来创建 LISTWHEEL 小工具
LISTWHEEL_GetFont()	返回绘制数据所使用的字体
LISTWHEEL_GetItemText()	返回所请求的数据项的文本
LISTWHEEL_GetLBorder()	返回左边框的像素大小
LISTWHEEL_GetLineHeight()	返回某个数据项的高度
LISTWHEEL_GetNumItems()	返回数据项的个数
LISTWHEEL_GetPos()	返回当前所选数据项的索引
LISTWHEEL_GetRBorder()	返回右边框的像素大小
LISTWHEEL_GetSel()	返回当前所选数据项
LISTWHEEL_GetTextAlign()	返回绘制数据项所使用的文本对齐方式
LISTWHEEL_GetUserData()	检索通过 LISTWHEEL_SetUserData 所设置的数据
LISTWHEEL_IsMoving()	检查小工具是否转动
LISTWHEEL_MoveToPos()	将 LISTWHEEL 移到指定位置
LISTWHEEL_OwnerDraw()	绘制小工具所使用的默认函数
LISTWHEEL_SetBkColor()	设置背景所使用的颜色
LISTWHEEL_SetDeceleration()	设置小工具的减速值，速度越大减速效果越好
LISTWHEEL_SetFont()	设置绘制项目文本所使用的字体
LISTWHEEL_SetItemData()	将自定义的无确切类型指针分配给指定数据项
LISTWHEEL_SetLBorder()	设置左边框的像素大小
LISTWHEEL_SetLineHeight()	设置绘制某个数据项所使用的高度
LISTWHEEL_SetOwnerDraw()	设置绘制小工具所使用的自绘函数
LISTWHEEL_SetPos()	将 LISTWHEEL 固定到指定位置
LISTWHEEL_SetRBorder()	设置右边框的像素大小
LISTWHEEL_SetSel()	设置当前所选数据项
LISTWHEEL_SetSnapPosition()	从小工具顶部设置的吸附位置（以像素为单位）
LISTWHEEL_SetText()	设置小工具的内容
LISTWHEEL_SetTextAlign()	设置绘制数据项所使用的对齐方式
LISTWHEEL_SetTextColor()	设置绘制数据项所使用的颜色
LISTWHEEL_SetTimerPeriod()	设置小工具的定时器周期
LISTWHEEL_SetUserData()	设置 LISTWHEEL 小工具的额外数据
LISTWHEEL_SetVelocity()	开始以指定速度移动转轮

表 27.3.1 LISTWHEEL 控件 API 函数

表 27.3.1 是 LISTWHEEL 控件函数相关描述。接下来，我们来讲解 LISTWHEEL 控件常用的函数。

(1) LISTWHEEL_SetSnapPosition 函数

设置“吸附”位置，位置是离 LISTWHEEL 控件顶部的相对位置(像素)，当设置好以后转动滚轮，当滚轮停止的时候就会有一个数据项吸附到此位置，其函数的原型如下源码所示：

```
void LISTWHEEL_SetSnapPosition(LISTWHEEL_Handle hObj, int SnapPosition);
```

该函数的相关形参如下表所示：

参数	描述
----	----

hObj	为 LISTWHEEL 句柄
SnapPosition	是具体控件顶部的距离，在改位置应当吸入数据项，单位是像素

表 27.3.2 LISTWHEEL_SetSnapPosition 函数相关形参描述

返回值：无。

(2) LISTWHEEL_SetOwnerDraw 函数

为控件设置自定义的自绘函数，其函数的原型如下源码所示：

```
void LISTWHEEL_SetOwnerDraw(LISTWHEEL_Handle hObj,
                             WIDGET_DRAW_ITEM_FUNC * pfOwnerDraw);
```

该函数的相关形参如下表所示：

参数	描述
hObj	为 LISTWHEEL 句柄
pfOwnerDraw	指向用户自定义的绘制函数

表 27.3.3 LISTWHEEL_SetOwnerDraw 函数相关形参描述

返回值：无。

该函数为 LISTWHEEL 空间定义一个用户自定义的函数，最常用场景就是给 LISTWHEEL 控件吸附项绘制一个轮廓，用于提示用户滚轮会此处吸附一个数据项，这个用户自定义函数一定要和 LISTWHEEL_SetSnapPosition 配合使用，设置好以后如下图所示：

之前	之后
Friday Saturday Sunday Monday Tuesday	Friday Saturday Sunday Monday Tuesday

图 27.3.1 设置吸附项轮廓

如图 27.3.1 所示，当我们使用函数 LISTWHEEL_SetSnapPosition 设置好吸附位置以后根本看不出吸附位置在哪里，所以我们需要给这个吸附位置设置一个轮廓，好让用户分辨出这个位置在哪里，图 24.3.4 右边部分两条红线就是设置好的轮廓。

(3) LISTWHEEL_AddString 函数

向控件中添加新的数据项，通常为字符串，其函数的原型如下源码所示：

```
void LISTWHEEL_AddString(LISTWHEEL_Handle hObj, const char * s)
```

该函数的相关形参如下表所示：

参数	描述
hObj	为 LISTWHEEL 句柄
s	为要添加的数据项(字符串)

表 27.3.4 LISTWHEEL_AddString 函数相关形参描述

返回值：无。

(4) LISTWHEEL_GetPos 函数

返回当前吸入的数据项索引，其函数的原型如下源码所示：

```
int LISTWHEEL_GetPos(LISTWHEEL_Handle hObj)
```

该函数的相关形参如下表所示：

参数	描述
hObj	为 LISTWHEEL 句柄

表 27.3.5 LISTWHEEL_GetPos 函数相关形参描述

返回值：当前吸入的数据项。

(5) LISTWHEEL_SetSel 函数

设置已选定的项，因为小控件滚动停止以后吸附的那个选项还没有被真正的选定，仅仅只是被吸附了而已，只有调用函数 LISTWHEEL_SetSel 设置这个被吸附的数据项为选定的数据项才会有效，其函数的原型如下源码所示：

```
void LISTWHEEL_SetSel(LISTWHEEL_Handle hObj, int Sel)
```

该函数的相关形参如下表所示：

参数	描述
hObj	为 LISTWHEEL 句柄
Sel	为要选定的数据项索引

表 27.3.6 LISTWHEEL_SetSel 函数相关形参描述

返回值：无。

(6) LISTWHEEL_GetItemText 函数

获取所请求的数据项的文本，其函数的原型如下源码所示：

```
void LISTWHEEL_GetItemText( LISTWHEEL_Handle hObj, unsigned Index,
                             char *pBuffer, int MaxSize);
```

该函数的相关形参如下表所示：

参数	描述
hObj	小工具的句柄
Index	所请求的数据项的索引
pBuffer	存储文本所使用的缓冲区
MaxSize	缓冲区大小（以字节为单位）

表 27.3.7 LISTWHEEL_GetItemText 函数相关形参描述

返回值：无。

27.4 LISTWHEEL 控件演示实验

27.4.1 硬件设计

1. 例程功能

本实验主要讲解 LISTWHEEL 小工具的使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数主要创建一个 window 小工具并在其上面添加三个 LISTWHEEL 控件、一个“OK”按键以及三个 TEXT 控件，这三个 LISTWHEEL 控件分别可滑动选择年、月、日等信息，而“OK”按键的作用是获取这些 LISTWHEEL 控件年月日信息并在指定的 TEXT 控件显示。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 28 LISTWHEEL 小工具》。

27.4.2 软件设计

27.4.2.1 程序流程图

本实验的程序流程图，如下图所示：

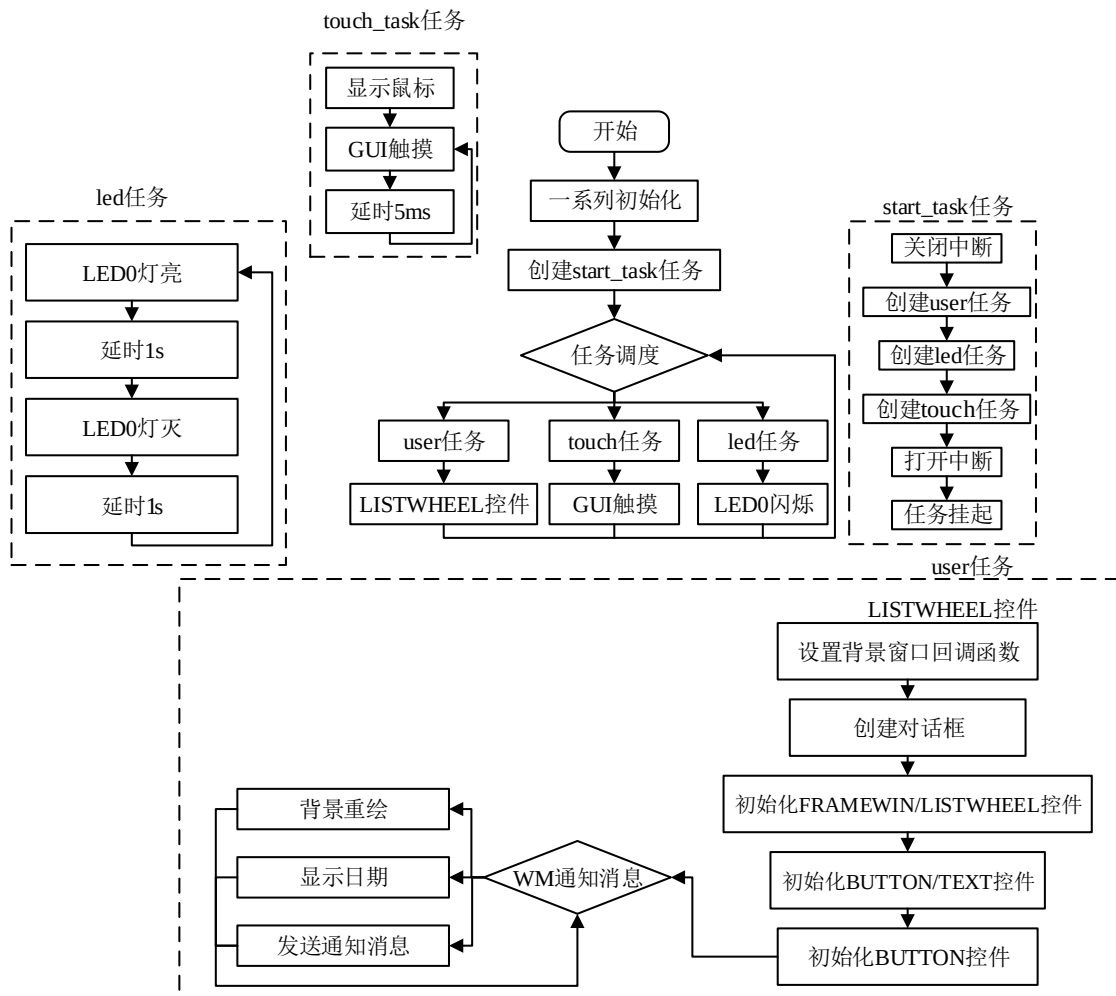


图 27.4.2.1.1 LISTWHEEL 控件实验流程图

27.4.2.2 程序解析

本章设计一个使用 LISTWHEEL 控件选择日期的例程，使用三个 LISTWHEEL 控件来选择年、月、日这三个日期值。界面上还有一个 OK 按键，当按下 OK 键以后选择的日期就生效并显示在对话框中。打开工程 **emWin 实验 28 LISTWHEEL 小工具** 中的文件 emwin_demo.c，其中函数 emwin_main 为实验演示函数，代码如下：

(1) 相关参数

```
/* 日期,默认初始化为 2020 年 1 月 1 日 */
DateStruct datedemo = {"2020", "January", "01"};

uint8_t listwheelindex[3]; /* LISTWHEEL 选中的项目号 */
/* 年 */
const char *_apYear[] =
{
    "1990", "1991", "1992", "1993", "1994", "1995", "1996",
    "1997", "1998", "1999", "2000", "2001", "2002", "2003",
```

```

    "2004", "2005", "2006", "2007", "2008", "2009", "2010",
    "2011", "2012", "2013", "2014", "2015", "2016", "2017",
    "2018", "2019", "2020", "2021", "2022", "2023", "2024",
};

/* 月 */
const char *_apMonth[] =
{
    "January",      "February",    "March",      "April",
    "May",          "June",        "July",       "August",
    "September",    "October",     "November",   "December",
};

/* 日 */
const char *_apDay[] =
{
    "01", "02", "03", "04", "05", "06",
    "07", "08", "09", "10", "11", "12",
    "13", "14", "15", "16", "17", "18",
    "19", "20", "21", "22", "23", "24",
    "25", "26", "27", "28", "29", "30", "31",
};

```

(2) 资源表

```

/* 对话框资源表 */
static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] =
{
    {FRAMEWIN_CreateIndirect, "Framewin", ID_FRAMEWIN_0, 0, 0, 320, 240,
     FRAMEWIN_CF_MOVEABLE, 0x64, 0 },
    {BUTTON_CreateIndirect,   "Button", ID_BUTTON_0, 192, 150, 100, 40, 0, 0x0, 0 },
    {TEXT_CreateIndirect,     "Text",   ID_TEXT_0, 190, 40, 51, 20, 0, 0x64, 0 },
    {TEXT_CreateIndirect,     "Text",   ID_TEXT_1, 178, 60, 66, 20, 0, 0x64, 0 },
    {TEXT_CreateIndirect,     "Text",   ID_TEXT_2, 196, 80, 53, 20, 0, 0x64, 0 },
    {LISTWHEEL_CreateIndirect,"Listwheel",ID_LISTWHEEL_0,10,30,50,150,0,0x0,0 },
    {LISTWHEEL_CreateIndirect,"Listwheel",ID_LISTWHEEL_1,60,30,80,150,0,0x0,0 },
    {LISTWHEEL_CreateIndirect,"Listwheel",ID_LISTWHEEL_2,140,30,30,150,0,0x0,0 },
};

```

(3) 创建对话框

```

/**
 * @brief      背景窗口 WM_HBKWIN 回调函数
 * @param      pMsg : 消息
 * @retval     无
 */
static void _cbBkWindow(WM_MESSAGE *pMsg)

```

```

{
    switch (pMsg->MsgId)
    {
        case WM_PAINT:
            GUI_SetBkColor(GUI_BLUE);
            GUI_Clear();
            GUI_SetColor(GUI_YELLOW);
            GUI_SetFont(&GUI_Font24B_ASCII);
            GUI_DispStringAt("ALIENTEK LISTWHEEL DEMO", 0, 0);

        default:
            WM_DefaultProc(pMsg);
    }
}

/**
 * @brief      emwin_main(emwin 入口)
 * @param      无
 * @retval     无
 */
void emwin_main (void)
{
    WM_SetCallback(WM_HBKWIN, _cbBkWindow); /* 设置背景窗口回调函数 */
    GUI_CreateDialogBox( _aDialogCreate, GUI_COUNTOF(_aDialogCreate),
                        _cbDialog, WM_HBKWIN, 0, 0);
}

```

此函数调用函数 `WM_SetCallback` 设置背景窗口重绘背景，然后调用 `GUI_CreateDialogBox` 根据资源表和回调函数 `_cbDialog` 创建对话框，这个回调函数篇幅过长，大家可以看一下本章节的实验。

27.4.3 下载验证

编译并下载例程到开发板中，LCD 显示如下图所示：



图 27.4.3.1 例程 LCD 显示

从图 27.4.3.1 中可以看出默认情况下三个 LISTWHEEL 控件选中的是索引为 0 的数据项，比如此处就是“1990”、“January”和“01”，而且右侧的红色日期显示和此相符。我们可以波动这三个 LISTWHEEL 控件，重新选择一个日期，如下图所示：



图 27.4.3.2 重新选择日期

从图 27.4.3.2 可以看出此时左侧 LISTWHEEL 控件选择的日期和右侧红色的日期不同，右侧红色日期还是上一次的值为“2020”、“January”和“01”，而左侧选择的日期为“2011”、“December”和“20”。这是因为我们还没有按下 OK 按钮来确认使用新选择的日期，按下 OK 按钮确认日期以后 LCD 显示如下图所示：



图 27.4.3.3 确认日期

从图 27.4.3.3 可以看出，此时右侧红色的日期值已经改变了，和左侧 LISTWHEEL 控件选择的相同，都为“2011”、“Decembe”和“20”。

第二十八章 RADIO 小工具

在一组选项中选择其中某一个选项可以使用 RADIO 控件，也就是单选按钮控件。单选框和复选框是相对的，一个是从一组选项中选择一项，另一个是在一组数据中选择多项。本章就来讲解 RADIO 控件的使用。

本章分为以下几个部分：

28.1 RADIO 控件简介

28.2 通知代码

28.3 键盘反应

28.4 RADIO 控件 API 函数

28.5 RADIO 控件演示例程

28.1 RADIO 控件简介

RADIO 也叫做单选按钮，和复选框一样，单选按钮也可以用来选择选项，但是 RADIO 一次只能选择一个选项。当我们选择 RADIO 中的某个选项的时候就会出现小圆点，一个单选按钮小工具可包含任意数量的按钮，这些按钮都是处于垂直排列的，RADIO 如下图所示：

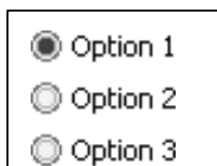


图 28.1.1 RADIO 控件

28.2 通知代码

该部件会作为 WM_NOTIFY_PARENT 消息的一部分发送给父窗口，通知代码如下表所示：

消息	描述
WM_NOTIFICATION_CLICKED	已单击单选按钮
WM_NOTIFICATION_RELEASED	已释放单选按钮
WM_NOTIFICATION_MOVED_OUT	已单击单选按钮并移出单选按钮，但没有释放
WM_NOTIFICATION_VALUE_CHANGED	单选按钮小工具的值（选择内容）已更改

图 28.2.1 控件通知代码

28.3 键盘反应

RADIO 控件也会对键盘的输入做出反应，如下表所示：

按键	反应
GUI_KEY_RIGHT	选定范围增加 1
GUI_KEY_DOWN	选定范围增加 1
GUI_KEY_LEFT	选定范围减小 1
GUI_KEY_UP	选定范围减小 1

表 28.3.1 键盘反应

28.4 RADIO 控件 API 函数

表 28.4.1 按字母顺序列出了可用的 RADIO 控件相关 API 函数：

函数	描述
RADIO_Create()	创建 RADIO 小工具（弃用）
RADIO_CreateEx()	创建 RADIO 小工具
RADIO_CreateIndirect()	从资源表条目创建 RADIO 小工具
RADIO_CreateUser()	使用作为用户数据的额外字节来创建 RADIO 小工具
RADIO_Dec()	将按钮选定范围的值减小 1
RADIO_GetDefaultFont()	返回显示新单选按钮文本所使用的默认字体
RADIO_GetDefaultTextColor()	返回显示新单选按钮文本所使用的默认文本颜色
RADIO_GetText()	返回单选按钮项目文本
RADIO_GetUserData()	检索带 RADIO_SetUserData 的数据集
RADIO_GetValue()	返回当前按钮选定内容
RADIO_Inc()	将按钮选定范围的值增加 1
RADIO_SetBkColor()	设置单选按钮的背景颜色
RADIO_SetDefaultFocusColor()	设置新单选按钮的默认聚焦框颜色
RADIO_SetDefaultFont()	设置显示新单选按钮文本所使用的默认字体
RADIO_SetDefaultImage()	设置新单选按钮将使用的图像
RADIO_SetDefaultTextColor()	设置显示新单选按钮文本所使用的默认文本颜色
RADIO_SetFocusColor()	设置聚焦框的颜色
RADIO_SetFont()	设置显示单选按钮文本所使用的字体
RADIO_SetGroupId()	设置指定单选框小工具的群组 ID
RADIO_SetImage()	设置显示单选按钮所使用的图像
RADIO_SetText()	设置文本
RADIO_SetTextColor()	设置显示单选按钮文本所使用的文本颜色
RADIO_SetUserData()	设置 RADIO 小工具的额外数据
RADIO_SetValue()	设置按钮选定内容

表 28.4.1 RADIO 控件 API 函数

表 28.4.1 是 RADIO 控件函数相关描述。接下来，我们来讲解 RADIO 控件常用的函数。

(1) RADIO_GetValue 函数

获取当前 RADIO 所选中的选项索引号，其函数的原型如下源码所示：

```
int RADIO_GetValue(RADIO_Handle hObj)
```

该函数的相关形参如下表所示：

参数	描述
hObj	RADIO 控件句柄

表 28.4.2 RADIO_GetValue 函数相关形参描述

返回值：当前选中的选项索引号，如果未选中任何按钮则返回-1。

(2) RADIO_GetText 函数

获取指定单选按钮的文本，其函数的原型如下源码所示：

```
int RADIO_GetText(RADIO_Handle hObj, unsigned Index,
                  char *pBuffer, int MaxLen)
```

该函数的相关形参如下表所示：

参数	描述
hObj	小工具的句柄
Index	所需项目的索引

pBuffer	复制文本所在缓冲区的指针
MaxLen	缓冲区大小（以字节为单位）

表 28.4.3 RADIO_GetText 函数形参描述

返回值：复制到缓冲区的文本的长度。

28.5 RADIO 控件演示实验

28.5.1 硬件设计

1. 例程功能

本实验主要讲解 RADIO 小工具的使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数主要创建一个 window 小工具并在其上面添加一个 RADIO 控件、一个“OK”按键控件和 TEXT 控件、我们在 RADIO 控件上添加五个选项，它们设置的文本分别为“RED”、“GREEN”、“BLUE”、“YELLOW”和“WHITE”，当用户选择某一个选项时并按下“OK”按键控件，系统会根据用户选择的选项修改屏幕的背景颜色并在 TEXT 控件显示选项的索引值。注意：按下“OK”按键控件时系统会隐藏当前对话框并开启定时器定时 3s，当定时时间到来时，系统会把对话框显示出来。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 29 RADIO 小工具》。

28.5.2 软件设计

28.5.2.1 程序流程图

本实验的程序流程图，如下图所示：

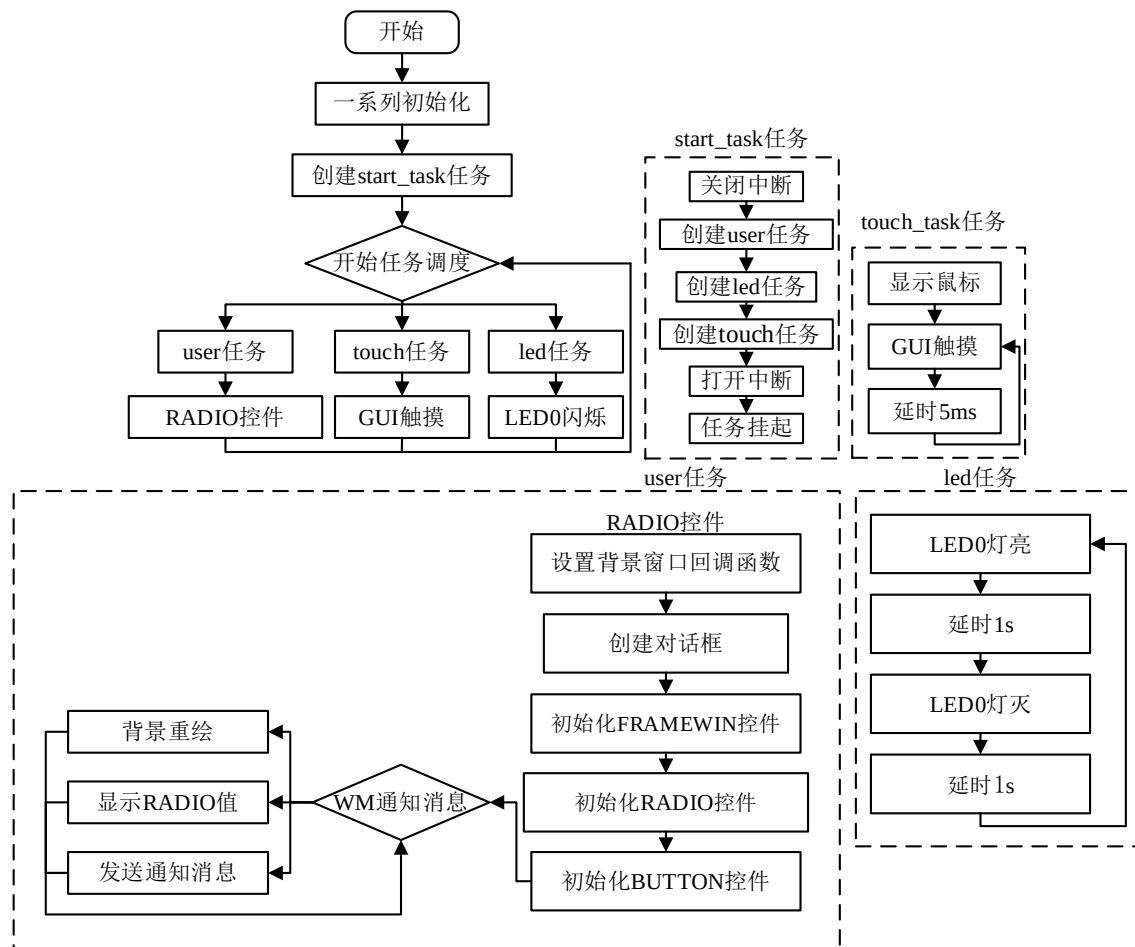


图 28.5.2.1RADIO 控件实验流程图

28.5.2.2 程序解析

本章设计一个利用 RADIO 单选按钮控件来更改背景桌面颜色的例程，在 LCD 上有个对话框，对话框中有一组 RADIO 选项供背景颜色选择。当选择好背景所使用的颜色以后就点击对话框中的 OK 键，然后背景桌面上的颜色就会被刷新为所选中的颜色，背景桌面上也会显示当前 RADIO 控件所选中的索引号。打开工程 **emWin 实验 29 RADIO 小工具** 中的文件 `emwin_demo.c` 源码如下所示：

(1) 定义一个 RADIO 结构体

```
typedef struct
{
    WM_HWIN rdadio;          /* RADIO 窗口句柄 */
    uint8_t value;          /* RADIO 当前所选中的 ITEM */
}radio_struct;
```

该结构体包含了 RADIO 窗口句柄和 RADIO 值。

(2) 资源表

```
/* 背景颜色表 */
int colortbl[] = {GUI_RED, GUI_BLUE, GUI_WHITE, GUI_YELLOW, GUI_GREEN};
radio_struct radio_widg;
/* 对话框资源表 */
```

```
static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] =
{
    {FRAMEWIN_CreateIndirect, "Framewin", ID_FRAMEWIN_0, 0, 0, 320, 240,
        FRAMEWIN_CF_MOVEABLE, 0x64, 0 },
    { RADIO_CreateIndirect, "Radio", ID_RADIO_0, 100, 10, 150, 150, 0, 0x1e05,
        0 },
    { BUTTON_CreateIndirect, "Button", ID_BUTTON_0, 100, 160, 80, 40, 0, 0x0, 0 },
};
```

使用数组定义四个颜色值，它们用在刷新背景颜色的，使用软件GUIBulider创建资源表。

(3) 创建对话框

```
/**
 * @brief      emwin_main(emwin 入口)
 * @param      无
 * @retval     无
 */
void emwin_main (void)
{
    WM_SetCallback(WM_HBKWIN, _BkCallback);

    radio_widg.value = 0;          /* 默认为 0 */
    radio_widg.rradio = GUI_CreateDialogBox(_aDialogCreate,
        GUI_COUNTOF(_aDialogCreate), _cbDialog, WM_HBKWIN, 0, 0);
}
```

此函数主要设置窗口的背景颜色并设置 RADIO 的值为 0，也就是说默认的背景颜色为红色，然后使用资源表以及回调函数_cbDialog 创建对话框，该回调函数如下所示：

(4) 背景回调函数

```
/**
 * @brief      背景窗口的回调函数
 * @param      pMsg : 消息
 * @retval     无
 */
static void _BkCallback(WM_MESSAGE *pMsg)
{
    static WM_HWIN hWin;
    char text[30];

    switch (pMsg->MsgId)
    {
        case WM_PAINT:
            GUI_SetFont(&GUI_Font24_ASCII);
            GUI_SetBkColor(colortbl[radio_widg.value]); /* 设置背景颜色 */
            GUI_Clear();
            GUI_SetColor(GUI_BLACK);                    /* 设置前景色 */
    }
```

```

        GUI_DispStringAt("WIDGET_RADIO - Sample", 0, 0);
        GUI_SetFont(&GUI_Font32_ASCII);
        sprintf(text, "RADIO VALUE:%d", radio_widg.value);
        GUI_DispStringAt(text, 0, 32);
        break;

    case CHANGE_COLOR:
        hWin = pMsg->hWinSrc;
        WM_HideWindow(hWin);                /* 隐藏对话框 */
        WM_InvalidateWindow(WM_HBKWIN);      /* 背景窗口无效 */
        WM_CreateTimer(WM_HBKWIN, 0, 3000, 0);
        break;

    case WM_TIMER:
        WM_ShowWindow(hWin);
        break;

    default:
        WM_DefaultProc(pMsg);
        break;
}
}

```

该回调函数有 3 个消息 ID，WM_PAINT 的消息为重绘背景颜色并显示 RADIO 键值和标题字，我们自定义了一个消息 ID（CHANGE_COLOR）该消息的功能为隐藏对话框，调用函数 WM_InvalidateWindow 使背景窗口无效化（调用该函数可以进入重绘机制），最后创建一个窗口定时器，该定时器设置为 3 秒，如果时间到达，那么进入消息 WM_TIMER 重新显示隐藏的对话框。函数 WM_DefaultProc 为如果全部不满足那么句柄消息的默认函数。

(5) 对话框的回调函数

```

/**
 * @brief      对话框回调函数
 * @param      pMsg : 消息
 * @retval     无
 */
static void _cbDialog(WM_MESSAGE *pMsg)
{
    WM_HWIN hItem;
    int      NCode;
    int      Id;
    WM_MESSAGE Msg;

    switch (pMsg->MsgId)
    {
        case WM_INIT_DIALOG:

```

```

/* 初始化 FRAME */
hItem = pMsg->hWin;
FRAMEWIN_SetText(hItem, "ALIENTEK RADIO");
FRAMEWIN_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
FRAMEWIN_SetFont(hItem, GUI_FONT_24_ASCII);
FRAMEWIN_SetTextColor(hItem, 0x0000FFFF);

/* 初始化 RADIO */
hItem = WM_GetDialogItem(pMsg->hWin, ID_RADIO_0);
RADIO_SetTextColor(hItem, 0x00000000);
RADIO_SetText(hItem, "RED", 0);
RADIO_SetText(hItem, "BLUE", 1);
RADIO_SetText(hItem, "WHITE", 2);
RADIO_SetText(hItem, "YELLOW", 3);
RADIO_SetText(hItem, "GREEN", 4);
RADIO_SetFont(hItem, GUI_FONT_20_ASCII);

/* 初始化 BUTTON0 */
hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_0);
BUTTON_SetText(hItem, "OK");
BUTTON_SetFont(hItem, GUI_FONT_24_ASCII);
break;

case WM_NOTIFY_PARENT:
    Id = WM_GetId(pMsg->hWinSrc);
    NCode = pMsg->Data.v;

    switch (Id)
    {
        case ID_RADIO_0: /* RADIO0 通知代码 */
            switch (NCode)
            {
                case WM_NOTIFICATION_CLICKED:
                    break;

                case WM_NOTIFICATION_RELEASED:
                    break;

                case WM_NOTIFICATION_VALUE_CHANGED:
                    /* 获取 RADIO 的句柄 */
                    hItem = WM_GetDialogItem(pMsg->hWin, ID_RADIO_0);
                    /* 获取 RADIO 值 */
                    radio_widg.value = RADIO_GetValue(hItem);
            }
        }
    }

```

```

        break;
    }
    break;

    case ID_BUTTON_0: /* BUTTON0 通知代码 */
    {
        switch (NCode)
        {
            case WM_NOTIFICATION_CLICKED:
                break;

            case WM_NOTIFICATION_RELEASED:
                Msg.MsgId = CHANGE_COLOR;
                Msg.hWinSrc = pMsg->hWin;
                WM_SendMessage(WM_HBKWIN, &Msg);
                break;
        }
        break;
    }
    break;

    default:
        WM_DefaultProc(pMsg);
        break;
}
}

```

上述的源码也是挺简单理解的，主要分为 2 个消息 ID，分别为 WM_INIT_DIALOG 和 WM_NOTIFY_PARENT。

首先我们看一下消息（WM_INIT_DIALOG）如下源码所示：

```

case WM_INIT_DIALOG:
    /* 初始化 FRAME */
    hItem = pMsg->hWin;
    FRAMEWIN_SetText(hItem, "ALIENTEK RADIO");
    FRAMEWIN_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
    FRAMEWIN_SetFont(hItem, GUI_FONT_24_ASCII);
    FRAMEWIN_SetTextColor(hItem, 0x0000FFFF);

    /* 初始化 RADIO */
    hItem = WM_GetDialogItem(pMsg->hWin, ID_RADIO_0);
    RADIO_SetTextColor(hItem, 0x00000000);
    RADIO_SetText(hItem, "RED", 0);
    RADIO_SetText(hItem, "BLUE", 1);
    RADIO_SetText(hItem, "WHITE", 2);
    RADIO_SetText(hItem, "YELLOW", 3);

```

```

        RADIO_SetText(hItem, "GREEN", 4);
        RADIO_SetFont(hItem, GUI_FONT_20_ASCII);

        /* 初始化 BUTTON0 */
        hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_0);
        BUTTON_SetText(hItem, "OK");
        BUTTON_SetFont(hItem, GUI_FONT_24_ASCII);
break;

```

从上述的源码可以看出，都是一些控件的初始化，设置窗口的标题、字体、颜色等等属性。消息（WM_NOTIFY_PARENT）如下源码所示：

```

case WM_NOTIFY_PARENT:
    Id      = WM_GetId(pMsg->hWinSrc);
    NCode   = pMsg->Data.v;

    switch (Id)
    {
        case ID_RADIO_0: /* RADIO0 通知代码 */
            switch (NCode)
            {
                case WM_NOTIFICATION_CLICKED:
                    break;

                case WM_NOTIFICATION_RELEASED:
                    break;

                case WM_NOTIFICATION_VALUE_CHANGED:
                    /* 获取 RADIO 的句柄 */
                    hItem = WM_GetDialogItem(pMsg->hWin, ID_RADIO_0);
                    /* 获取 RADIO 值 */
                    radio_widg.value = RADIO_GetValue(hItem);
                    break;
            }
            break;

        case ID_BUTTON_0: /* BUTTON0 通知代码 */
            switch (NCode)
            {
                case WM_NOTIFICATION_CLICKED:
                    break;

                case WM_NOTIFICATION_RELEASED:
                    Msg.MsgId = CHANGE_COLOR;
                    Msg.hWinSrc = pMsg->hWin;

```

```
WM_SendMessage(WM_HBKWIN, &Msg);

break;

}

break;

}

break;
```

从上述的源码中，我们可以看出有两个控件，第一个是 RADIO 控件，主要是获取选中的索引号，第二个 BUTTON 控件，该控件主要发送消息到背景重绘回调函数处理。

28.5.3 下载验证

编译并下载例程到开发板中，LCD 显示如下图所示：

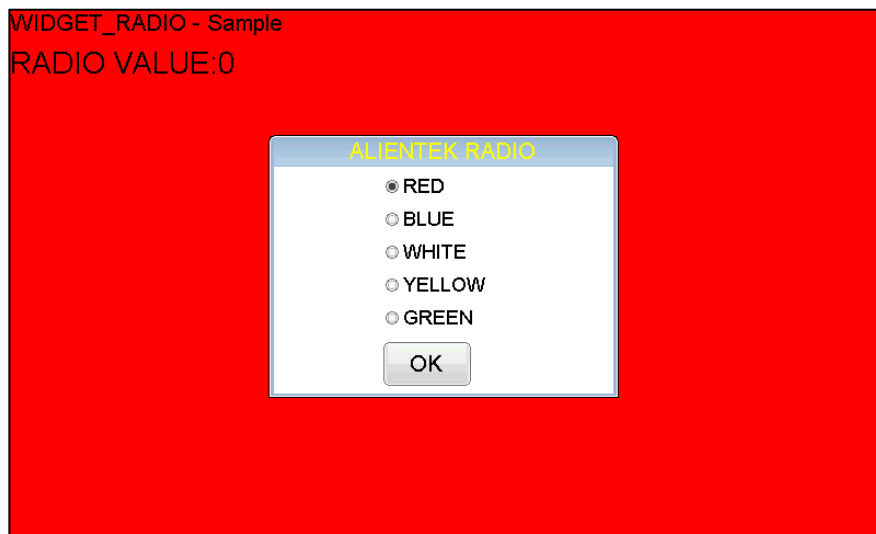


图 28.5.3.1 例程 LCD 显示界面

点击对话框中的 RADIO 控件，随便选择一个要设置的色值，比如绿色“GREEN”，然后点击 OK 键，此时 LCD 背景颜色变为绿色，并且显示 RADIO 控件选中的选项索引号为 4，如下图所示：



图 28.5.3.2 设置 LCD 背景颜色

第二十九章 MULTIEDIT 小工具

前面我们讲解过文本小工具 EDIT，我们知道 EDIT 小工具只能编辑一行字符信息，当我们想要编辑多行文本的时候 EDIT 控件就不适用了。这个时候就该 MULTIEDIT 小工具上场了，看名字就很容易猜到，“MULTI EDIT”是多行文本！在 MULTIEDIT 控件中就可以像我们在电脑上操作记事本那样来编写一些文档之类的东西。

本章分为以下几个部分：

29.1 MULTIEDIT 控件简介

29.2 通知代码

29.3 键盘反应

29.4 MULTIEDIT 控件 API 函数

29.5 MULTIEDIT 空间演示例程

29.1 MULTIEDIT 控件简介

我们可以通过 MULTIEDIT 控件来编辑多行文本，它就是一个简易的文本编辑器，也可以用来显示静态文本。MULTIEDIT 小工具支持带滚动条和不带滚动条的滚动，MULTIEDIT 小工具如下图所示：



图 29.1.1 MULTIEDIT 控件

29.2 通知代码

MULTIEDIT 小工具会作为 WM_NOTIFY_PARENT 消息的一部分发送给父窗口，通知代码如下表所示：

消息	描述
WM_NOTIFICATION_CLICKED	已单击小工具
WM_NOTIFICATION_RELEASED	已释放小工具
WM_NOTIFICATION_MOVED_OUT	已单击小工具，并且指针已移出小工具，但没有释放
WM_NOTIFICATION_SCROLL_CHANGED	可选滚动条的滚动位置已更改
WM_NOTIFICATION_VALUE_CHANGED	小工具的文本已更改

表 29.2.1 控件通知代码

29.3 键盘反应

MULTIEDIT 控件可以对键盘做出反应，如下表所示：

按键	反应
GUI_KEY_UP	将光标向上移动一行
GUI_KEY_DOWN	将光标向下移动一行

GUI_KEY_RIGHT	将光标向右移动一个字符
GUI_KEY_LEFT	将光标向左移动一个字符
GUI_KEY_END	将光标移到当前行的末尾
GUI_KEY_HOME	将光标移到当前行的开头
GUI_KEY_BACKSPACE	如果小工具在读/写模式下工作，此按键将删除光标之前的字符
GUI_KEY_DELETE	如果小工具在读/写模式下工作，此按键将删除光标之下的字符
GUI_KEY_INSERT	在插入模式和覆盖模式之间切换。
GUI_KEY_ENTER	如果小工具在读/写模式下工作，此按键将在当前位置插入一个新行（' \n' ）
GUI_KEY_PGUP	将 MULTIEDIT 按页向上滚动
GUI_KEY_PGDOWN	将 MULTIEDIT 按页向下滚动

表 29.3.1 键盘反应

29.4 MULTIEDIT 控件 API 函数

表 29.4.1 按字母顺序列出了可用的 MULTIEDIT 控件相关 API 函数：

函数	描述
MULTIEDIT_AddKey()	按键输入例程
MULTIEDIT_AddText()	在当前光标位置添加更多文本
MULTIEDIT_Create()	创建 MULTIEDIT 小工具（弃用）
MULTIEDIT_CreateEx()	创建 MULTIEDIT 小工具
MULTIEDIT_CreateIndirect()	从资源表条目创建 MULTIEDIT 小工具
MULTIEDIT_CreateUser()	使用作为用户数据的额外字节来创建小工具
MULTIEDIT_EnableBlink()	启用/禁用闪烁光标
MULTIEDIT_GetCursorCharPos()	返回光标位置的字符编号
MULTIEDIT_GetCursorPixelPos()	返回光标的像素位置
MULTIEDIT_GetPrompt()	返回提示符的文本
MULTIEDIT_GetText()	返回文本
MULTIEDIT_GetTextSize()	返回当前文本所使用的缓冲区大小
MULTIEDIT_GetUserData()	检索通过 MULTIEDIT_SetUserData 设置的数据
MULTIEDIT_SetAutoScrollH()	激活自动使用水平滚动条
MULTIEDIT_SetAutoScrollV()	激活自动使用垂直滚动条
MULTIEDIT_SetBkColor()	设置背景颜色
MULTIEDIT_SetBufferSize()	设置用于文本和提示符的缓冲区大小
MULTIEDIT_SetCursorOffset()	将光标固定到指定字符
MULTIEDIT_SetFont()	设置字体
MULTIEDIT_SetInsertMode()	启用/禁用插入模式
MULTIEDIT_SetMaxNumChars()	设置最大字符（包括提示符）数
MULTIEDIT_SetPasswordMode()	启用/禁用密码模式
MULTIEDIT_SetPrompt()	设置提示文本
MULTIEDIT_SetReadOnly()	启用/禁用只读模式
MULTIEDIT_SetText()	设置文本
MULTIEDIT_SetTextAlign()	设置文本对齐方式
MULTIEDIT_SetTextColor()	设置文本颜色
MULTIEDIT_SetUserData()	设置 MULTIEDIT 小工具的额外数据
MULTIEDIT_SetWrapWord()	启用/禁用字词换行

MULTIEDIT_SetWrapNone()	启用/禁用无换行模式
MULTIEDIT_ShowCursor()	启用/禁用光标

表 29.4.1 MULTIEDIT 控件 API 函数

下面我们来看几个常用的比较重点的函数：

(1) MULTIEDIT_AddText 函数

在当前光标位置添加指定文本，其函数的原型如下源码所示：

```
int MULTIEDIT_AddText(MULTIEDIT_HANDLE hObj, const char * s);
```

该函数的相关形参如下表所示：

参数	描述
hObj	MULTIEDIT 控件的句柄
s	要添加的文本（以空终止符结尾）的指针

表 29.4.2 MULTIEDIT_AddText 函数的相关形参描述

注意：如果字符数超出通过函数 MULTIEDIT_SetMaxNumChars 所设置的最大限值，则该函数将仅添加适合该控件的文本字符。

返回值：无。

(2) MULTIEDIT_SetMaxNumChars 函数

设置文本和提示符所使用的最大字符数，其函数的原型如下源码所示：

```
void MULTIEDIT_SetMaxNumChars(MULTIEDIT_HANDLE hObj, unsigned MaxNumChars);
```

该函数的相关形参如下表所示：

参数	描述
hObj	MULTIEDIT 控件的句柄
MaxNumChars	为最大字符数

表 29.4.3 MULTIEDIT_SetMaxNumChars 函数的相关形参描述

返回值：无。

(3) MULTIEDIT_EnableBlink 函数

启用/禁止闪烁光标，其函数的原型如下源码所示：

```
void MULTIEDIT_EnableBlink(EDIT_Handle hObj, int Period, int OnOff);
```

该函数的相关形参如下表所示：

参数	描述
hObj	编辑字段的句柄
Period	闪烁周期
OnOff	1 表示启用闪烁；0 表示禁用闪烁

表 29.4.4 MULTIEDIT_EnableBlink 函数相关参数描述

返回值：无。

29.5 MULTIEDIT 控件演示实验

29.5.1 硬件设计

1. 例程功能

本实验主要讲解 MULTIEDIT 小工具的使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数主要创建一个 window 小工具并在其上面添加 MULTIEDIT 控件、“SEND” 按键和 EDIT 控件，其中 MULTIEDIT 控件可接收串口 1 的数据，EDIT 控件已经写入 www.openedv.com 字符串，当我们按下“SEND” 按键时系统会把 www.openedv.com 字符串发送到串口 1，大家可以在串口调试助手接收这个字符串。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 30 MULTIEDIT 小工具》。

29.5.2 软件设计

29.5.2.1 程序流程图

本实验的程序流程图，如下图所示：

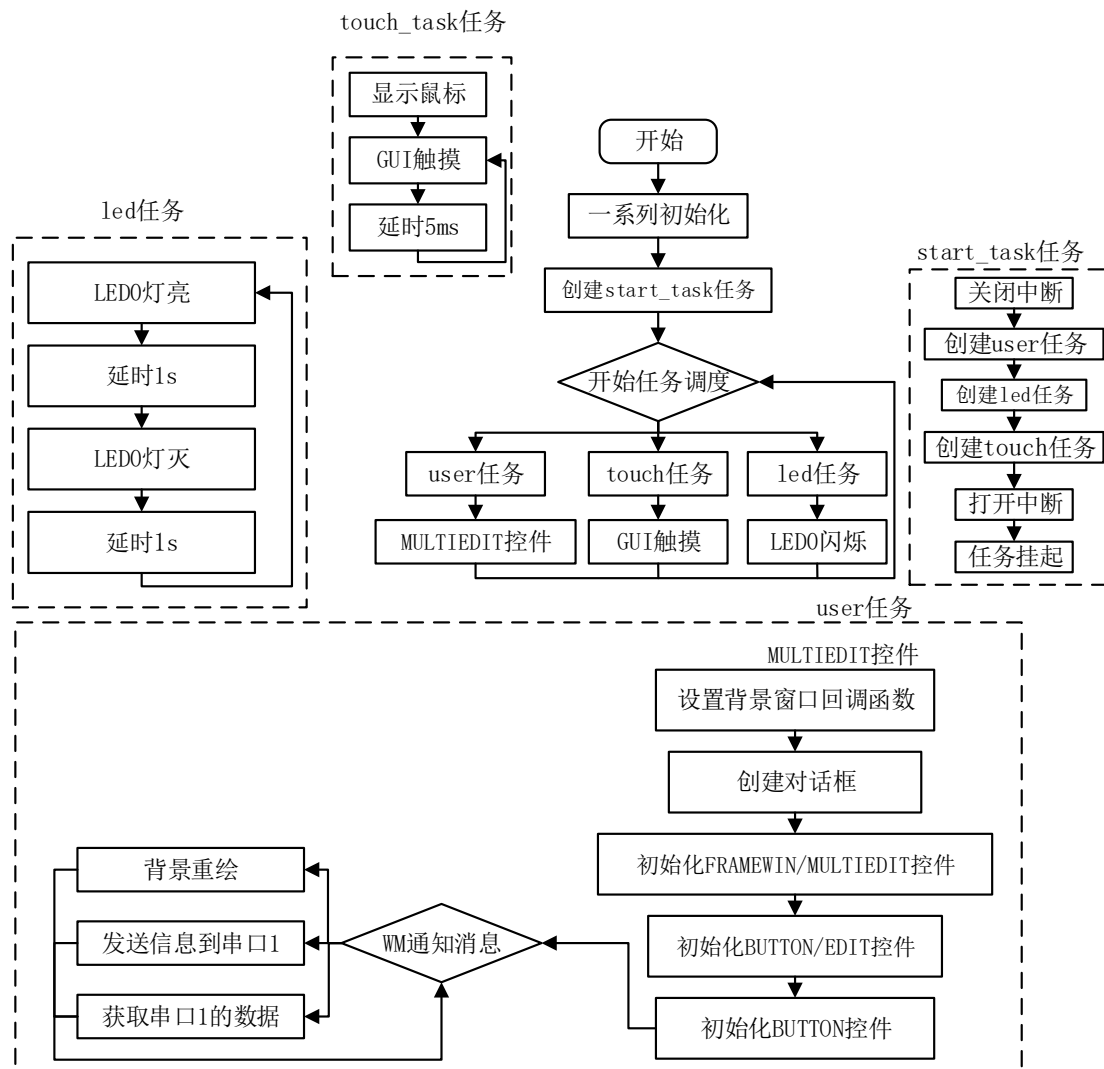


图 29.5.2.1 MULTIEDIT 控件实验流程图

29.5.2.2 程序解析

本章我们使用 MULTIEDIT 控件设计一个简单的串口收发程序，当开发板接收到串口调试助手发送过来的字符串就将其显示在 MULTIEDIT 控件上。在设置一个 SEND 按钮用来给 PC 发送数据，当按下 SEND 按钮开发板就会向 PC 发送指定的字符串。打开工程 **emWin 实验 30 MULTIEDIT 小工具** 中的文件 emwin_demo.c 源码如下：

(1) 资源表

```
/* 对话框资源表 */
static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] = {
{ FRAMEWIN_CreateIndirect, "Framewin", ID_FRAMEWIN_0, 0, 0, 320, 240,
  FRAMEWIN_CF_MOVEABLE, 0x64, 0 },
{ MULTIEDIT_CreateIndirect, "Multiedit", ID_MULTIEDIT_0, 20, 35, 130, 150, 0, 0x0, 0 },
{ EDIT_CreateIndirect, "Edit", ID_EDIT_0, 170, 61, 133, 26, 0, 0x64, 0 },
{ BUTTON_CreateIndirect, "Button", ID_BUTTON_0, 188, 129, 92, 46, 0, 0x0, 0 },
{ TEXT_CreateIndirect, "Text", ID_TEXT_0, 30, 5, 107, 20, 0, 0x64, 0 },
{ TEXT_CreateIndirect, "Text", ID_TEXT_1, 198, 5, 80, 20, 0, 0x64, 0 },
};
```

(2) 创建对话框

```
/**
 * @brief      创建对话框
 * @param      无
 * @retval     无
 */
void emwin_main(void)
{
    WM_HWIN hWin;
    WM_HWIN hItem;
    uint8_t len;

    WM_SetCallback(WM_HBKWIN, _cbBkWindow); /* 设置背景窗口回调函数 */
    hWin = GUI_CreateDialogBox(_aDialogCreate, GUI_COUNTOF(_aDialogCreate),
        _cbDialog, WM_HBKWIN, 0, 0);
    hItem = WM_GetDialogItem(hWin, ID_MULTIEDIT_0); /* 获取 MULTIEDIT 控件句柄 */

    while (1)
    {
        if (g_usart_rx_sta & 0x8000) /* 收到数据 */
        {
            len = g_usart_rx_sta & 0x3fff; /* 得到此次接收到的数据长度 */
            g_usart_rx_buf[len] = '\r'; /* 添加换行符 */
            g_usart_rx_buf[len + 1] = '\n';
            /* 在 MULTIEDIT 中显示数据 */
            MULTIEDIT_AddText(hItem, (char *)g_usart_rx_buf);
            g_usart_rx_sta = 0;
        }
        else
```

```

    {
        GUI_Delay(100);
    }
}
}

```

上面的源码可以看出：我们设置了背景窗口的回调函数并创建对话框，在 `while(1)` 里面判断串口是否接收到数据，如果接收到了数据调用函数 `MULTIEDIT_AddText` 显示接收的数据。注意！我们在向 `MULTIEDIT` 控件添加接收到的字符串的时候添加了换行符，就是“`\r\n`”。

(3) 对话框的回调函数

```

/**
 * @brief      对话框回调函数
 * @param      pMsg : 消息
 * @retval     无
 */
static void _cbDialog(WM_MESSAGE *pMsg)
{
    WM_HWIN hItem;
    int      NCode;
    int      Id;
    uint8_t databuff[50];
    int datalen = 0;

    switch (pMsg->MsgId)
    {
        case WM_PAINT:
            /* 绘制垂直梯度填充矩形 */
            GUI_DrawGradientV(162, 10, 165, 200, GUI_RED, GUI_BLUE);
            GUI_SetColor(GUI_RED);          /* 设置为红色 */
            GUI_SetFont(&GUI_Font8x18);     /* 设置字体 */
            break;

        case WM_INIT_DIALOG:
            /* FRAMWIN 初始化 */
            hItem = pMsg->hWin;
            FRAMEWIN_SetFont(hItem, GUI_FONT_20_ASCII);
            FRAMEWIN_SetTextColor(hItem, 0x0000FFFF);
            FRAMEWIN_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
            FRAMEWIN_SetText(hItem, "正点原子 MULTIEDIT");

            /* MULTIEDIT 初始化 */
            hItem = WM_GetDialogItem(pMsg->hWin, ID_MULTIEDIT_0);
            MULTIEDIT_SetFont(hItem, GUI_FONT_8X18);
            MULTIEDIT_SetTextColor(hItem, MULTIEDIT_CI_EDIT, 0x00FF0000);
    }
}

```

```

MULTIEDIT_SetAutoScrollH(hItem, 1);          /* 激活自动水平滚动条 */
MULTIEDIT_SetAutoScrollV(hItem, 1);          /* 激活自动垂直滚动条 */
MULTIEDIT_EnableBlink(hItem, 500, 1);        /* 开启光标闪烁 */

/* EDIT 初始化 */
hItem = WM_GetDialogItem(pMsg->hWin, ID_EDIT_0);
EDIT_SetTextAlign(hItem, GUI_TA_LEFT | GUI_TA_VCENTER);
EDIT_SetFont(hItem, GUI_FONT_16B_ASCII);
EDIT_SetText(hItem, "www.openedv.com");
EDIT_EnableBlink(hItem, 500, 1);            /* 开启光标闪烁 */

/* BUTTON 初始化 */
hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_0);
BUTTON_SetFont(hItem, GUI_FONT_COMIC18B_ASCII);
BUTTON_SetText(hItem, "SEND");

/* TEXT 初始化 */
hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_0);
TEXT_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
TEXT_SetFont(hItem, GUI_FONT_16B_ASCII);
TEXT_SetTextColor(hItem, 0x000000FF);
TEXT_SetText(hItem, "Receive Area");

hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_1);
TEXT_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
TEXT_SetFont(hItem, GUI_FONT_16B_ASCII);
TEXT_SetTextColor(hItem, 0x000000FF);
TEXT_SetText(hItem, "Send Area");
break;

case WM_NOTIFY_PARENT:
    Id = WM_GetId(pMsg->hWinSrc);
    NCode = pMsg->Data.v;

    switch (Id)
    {
        case ID_MULTIEDIT_0:
            switch (NCode)
            {
                case WM_NOTIFICATION_CLICKED:
                    break;

                case WM_NOTIFICATION_RELEASED:

```

```

        break;

        case WM_NOTIFICATION_VALUE_CHANGED:
            break;
    }
    break;

case ID_EDIT_0:
    switch (NCode)
    {
        case WM_NOTIFICATION_CLICKED:
            break;

        case WM_NOTIFICATION_RELEASED:
            break;

        case WM_NOTIFICATION_VALUE_CHANGED:
            break;
    }
    break;

case ID_BUTTON_0:
    switch (NCode)
    {
        case WM_NOTIFICATION_CLICKED:
            break;

        case WM_NOTIFICATION_RELEASED:
            hItem = WM_GetDialogItem(pMsg->hWin, ID_EDIT_0);
            /* 获取 EDIT 中的文本长度，注意要加一个字节的结束符'\0' */
            datalen = EDIT_GetNumChars(hItem) + 1;
            /* 获取 EDIT 中的文本 */
            EDIT_GetText(hItem, (char *)databuff, datalen);
            printf("%s\r\n", databuff); /* 通过串口发送出去 */
            break;
    }
    break;
}
break;
default:
    WM_DefaultProc(pMsg);
    break;
}

```

```
}
```

上面的源码，我们可以知道当按下 SEND（ID_BUTTON_0）控件时，会调用函数 EDIT_GetText 来获取该控件的数据，然后调用函数 printf() 发送到串口调试助手。

(4) 背景窗口回调函数

```
/**
 * @brief      背景窗口 WM_HBKWIN 回调函数
 * @param      pMsg : 消息
 * @retval     无
 */
static void _cbBkWindow(WM_MESSAGE *pMsg)
{
    switch (pMsg->MsgId)
    {
        case WM_PAINT:
            GUI_SetBkColor(GUI_BLUE);
            GUI_Clear();
            GUI_SetColor(GUI_YELLOW);
            GUI_SetFont(&GUI_Font24B_ASCII);
            GUI_DispStringAt("正点原子 MULIEDT DEMO", 0, 0);

        default:
            WM_DefaultProc(pMsg);
    }
}
```

上述的源码为重绘背景颜色。

29.5.3 下载验证

编译并下载例程到开发板中，LCD 显示如下图所示：

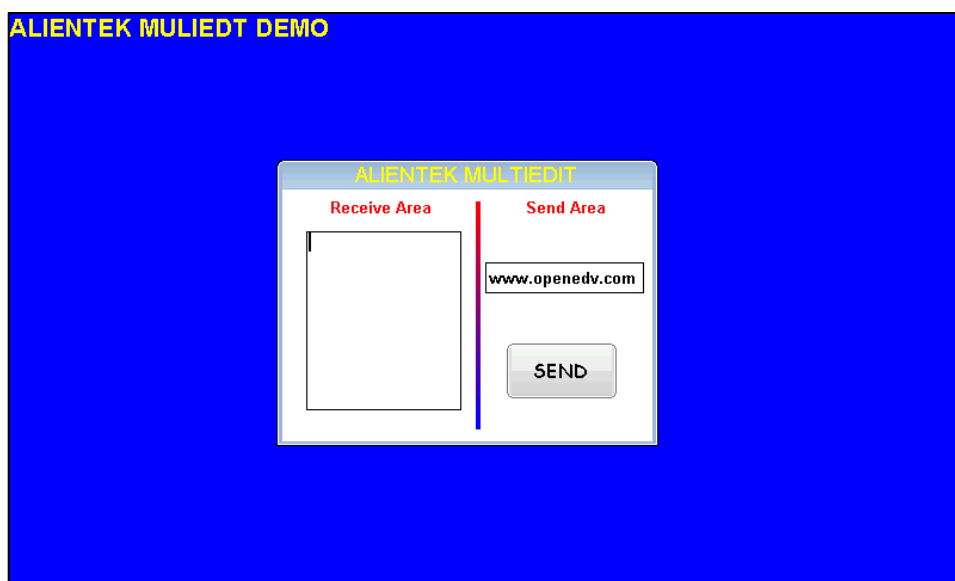


图 29.5.3.1 LCD 显示界面

从图 29.5.3.1 可以看出，此时接收区的 MULTIEDIT 控件中没有任何数据，发送区 EDIT 控件中要发送的数据为：www.openedv.com。打开串口调试助手给开发板发送数据：Multiedit Test！，此时 LCD 如下图所示：

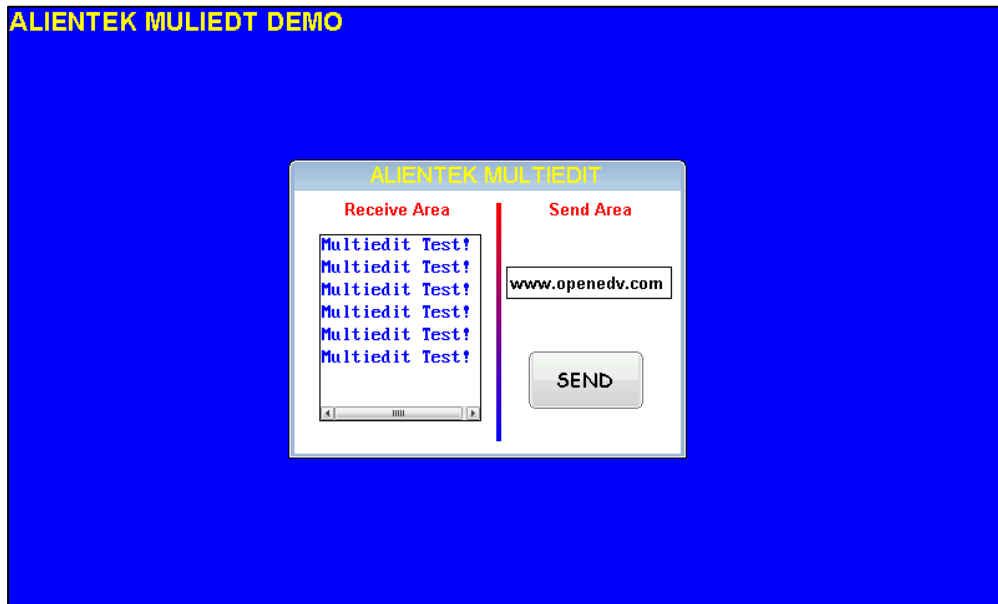


图 29.5.3.2 LCD 接收到数据

从图 29.5.3.2 可以看出开发板接收到数据：Multiedit Test！并且显示在了 MULTIEDIT 控件中。点击 SEND 键发送数据到串口调试助手上，串口调试助手如下图所示，可以看出串口调试助手接收到开发板发送来的数据。

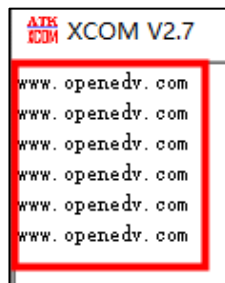


图 29.5.3.3 串口调试助手显示

第三十章 SLIDER 小工具

很多应用中需要更改一些数据的参数，比如电机转速、LCD 背光、音量等等，我们有很多种方法来实现这个功能要求，比如通过按键来改变数值、通过上位机控制等等。本章我们讲解 emWin 提供的一个用来修改数值的控件—SLIDER 控件。

本章分为以下几个部分：

- 30.1 SLIDER 控件简介
- 30.2 通知代码
- 30.3 键盘反应
- 30.4 SLIDER 控件 API 函数
- 30.5 SLIDER 控件演示实验

30.1 SLIDER 控件简介

SLIDER 控件也叫滑块小工具，此控件通常用来修改各项数值。SLIDER 控件包含滑动条和滑动条旁边的刻度标记，在拖动滑动条的时候这些刻度标记可规定光标按指定间距移动，SLIDER 控件如图 30.1.1 所示：

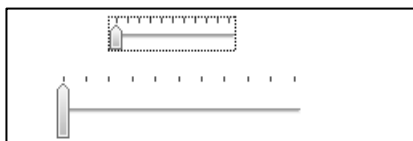


图 30.1 SLIDER 控件

30.2 通知代码

SLIDER 小工具会作为 WM_NOTIFY_PARENT 消息的一部分发送给父窗口，通知代码如下表所示：

消息	描述
WM_NOTIFICATION_CLICKED	已单击滑块小工具
WM_NOTIFICATION_RELEASED	已释放滑块小工具
WM_NOTIFICATION_VALUE_CHANGED	滑块小工具的值可以通过移动缩略图进行更改

表 30.2.1 控件通知代码

30.3 键盘反应

SLIDER 控件也会对键盘的输入做出反应，如下表所示：

按键	反应
GUI_KEY_RIGHT	将滑动条的当前值增加一个项目
GUI_KEY_LEFT	将滑动条的当前值减小一个项目

表 30.3.1 控件键盘反应

30.4 SLIDER 控件 API 函数

表 30.4.1 按字母顺序列出了可用的 SLIDER 控件相关 API 函数：

函数	描述
----	----

SLIDER_Create()	创建 SLIDER 小工具（弃用）
SLIDER_CreateEx()	创建 SLIDER 小工具
SLIDER_CreateIndirect()	从资源表条目创建 SLIDER 小工具
SLIDER_CreateUser()	使用作为用户数据的额外字节来创建 SLIDER 小工具
SLIDER_Dec()	减小滑动条的值
SLIDER_GetUserData()	检索通过 SLIDER_SetUserData 设置的数据
SLIDER_GetValue()	返回滑动条的当前值
SLIDER_Inc()	增加滑动条的值
SLIDER_SetBkColor()	设置滑动条的背景颜色
SLIDER_SetDefaultFocusColor()	设置新滑动条的默认聚焦框颜色
SLIDER_SetFocusColor()	设置聚焦框的颜色
SLIDER_SetNumTicks()	设置滑动条的刻度标记数
SLIDER_SetRange()	设置滑块值的范围
SLIDER_SetUserData()	设置 SLIDER 小工具的额外数据
SLIDER_SetValue()	设置滑动条的当前值
SLIDER_SetWidth()	设置滑动条的宽度

图 30.4.1 SLIDER 控件 API 函数表

表 30.4.1 是 SLIDER 控件函数相关描述。接下来，我们来讲解 SLIDER 控件常用的函数。

(1) SLIDER_GetValue 函数

获取当前滑块的数值，其函数的原型如下源码所示：

```
int SLIDER_GetValue(SLIDER_Handle hObj);
```

该函数的相关形参如下表所示：

参数	描述
hObj	滑块句柄

表 30.4.2 SLIDER_GetValue 函数相关形参描述

返回值：滑块当前值。

(2) SLIDER_SetRange 函数

设置滑块可滑动范围，其函数的原型如下源码所示：

```
void SLIDER_SetRange(SLIDER_Handle hObj, int Min, int Max);
```

该函数的相关形参如下表所示：

参数	描述
hObj	滑块小工具的句柄
Min	最小值
Max	最大值

表 30.4.3 SLIDER_SetRange 函数形参描述

返回值：无。

其他信息：SLIDER 控件创建成功以后默认范围是 0~100。

(3) SLIDER_SetNumTicks 函数

设置滑动条的刻度标记数，其函数的原型如下源码所示：

```
void SLIDER_SetNumTicks(SLIDER_Handle hObj, int NumTicks);
```

该函数的相关形参如下表所示：

参数	描述
hObj	滑块小工具的句柄
NumTicks	是刻度标记数量，SLIDER 控件默认刻度标记数量为 10

表 30.4.4 SLIDER_SetNumTicks 函数形参描述

返回值：无。

(4) SLIDER_SetValue 函数

设置滑动条的当前值，其函数的原型如下源码所示：

```
void SLIDER_SetValue(SLIDER_Handle hObj, int v);
```

该函数的相关形参如下表所示：

参数	描述
hObj	滑块小工具的句柄
v	要设置的值

表 30.4.5 SLIDER_SetValue 函数形参描述

返回值：无。

30.5 SLIDER 控件演示实验

30.5.1 硬件设计

1. 例程功能

本实验主要讲解 MULTIEDIT 小工具的使用，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数主要创建一个 Frame win 小工具并在其上面添加四个 Slider 控件和三个 Edit 控件，其中三个 Edit 控件的背景颜色是由这些 RED、GREEN 和 BLUE 的 Slider 控件滑动值而定，最后根据这三个 Slider 控件的值修改某个区域的颜色。注意：第四个 Slider 控件没有其他作用。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 31 SLIDER 小工具》。

注意：本实验不支持 Mini 开发板运行。

30.5.2 软件设计

30.5.2.1 程序流程图

本实验的程序流程图，如下图所示：

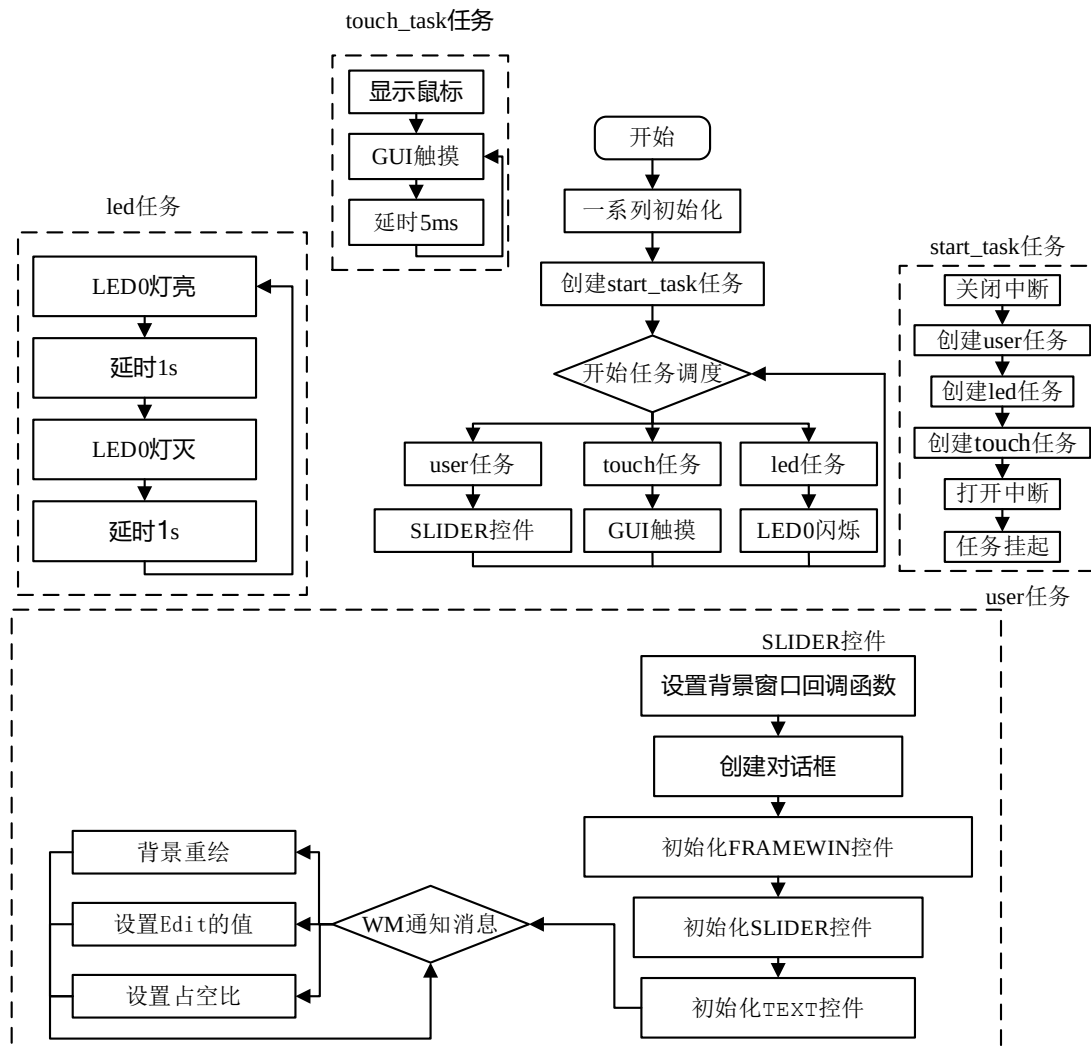


图 30.5.2.1 SLIDER 控件实验流程图

30.5.2.2 程序解析

本章我们使用 SLIDER 控件来调节 LCD 屏幕上一个色块的颜色，使用三个 SLEDER 控件调整色块的 R(红色)、G(绿色)和 B(蓝色)三种颜色成分的分量，这样色块的最终颜色就会随之变化，打开工程 **emWin 实验 31 SLIDER 小工具** 中的文件 emwin_demo.c，代码如下：

(1) 定义一个 slider 结构体

```

/* slider 参数集 */
typedef struct
{
    WM_HWIN    slider;          /* SLIDER 句柄 */
    uint16_t    slider_value[4]; /* SLIDER 滑块值 */
} slider_structer;
    
```

该结构体包含 SLIDER 句柄和 SLIDER 数值。

(2) 资源表

```

/* 对话框资源表 */
static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] =
    
```

```
{
{ FRAMEWIN_CreateIndirect, "Framewin", ID_FRAMEWIN_0, 0,0,800,480,0,0x64, 0 },
{ SLIDER_CreateIndirect, "Slider", ID_SLIDER_0, 90,100,400,35,0,0x0, 0 },
{ SLIDER_CreateIndirect, "Slider", ID_SLIDER_1, 90,165,400, 35,0,0x0, 0 },
{ SLIDER_CreateIndirect, "Slider", ID_SLIDER_2, 90,230,400, 35,0,0x0, 0 },
{ SLIDER_CreateIndirect, "Slider", ID_SLIDER_3, 90,360,595, 32,0,0x0, 0 },
{ EDIT_CreateIndirect, "Edit", ID_EDIT_0, 500, 100, 55,35,0,0x64, 0 },
{ EDIT_CreateIndirect, "Edit", ID_EDIT_1, 500, 165, 55,35, 0,0x64, 0 },
{ EDIT_CreateIndirect, "Edit", ID_EDIT_2, 500, 230, 55,35,0,0x64, 0 },
{ TEXT_CreateIndirect, "Text", ID_TEXT_0, 311, 47,202,29,0,0x64, 0 },
{ TEXT_CreateIndirect, "Text", ID_TEXT_1, 268, 314,279,31,0,0x64, 0 },
{ TEXT_CreateIndirect, "Text", ID_TEXT_2, 31, 109,51,20,0,0x64, 0 },
{ TEXT_CreateIndirect, "Text", ID_TEXT_3, -2, 175,92,20,0,0x64, 0 },
{ TEXT_CreateIndirect, "Text", ID_TEXT_4, 12, 239,80,20,0,0x64, 0 },
{ TEXT_CreateIndirect, "Text", ID_TEXT_5, 23, 366,66,20,0,0x64, 0 },
};
```

(3) 创建对话框

```
/**
 * @brief      emwin_main(emwin 入口)
 * @param      无
 * @retval     无
 */
void emwin_main(void)
{
    slider_dev.slider_value[0] = 0;
    slider_dev.slider_value[1] = 127;
    slider_dev.slider_value[2] = 255;
    slider_dev.slider_value[3] = 300;
    WM_SetCallback(WM_HBKWIN, _cbBkWindow);
    slider_dev.slider = GUI_CreateDialogBox(_aDialogCreate,
                                           GUI_COUNTOF(_aDialogCreate),
                                           _cbDialog, WM_HBKWIN, 0, 0);
}
```

从上述的源码可以看出，首先设置结构体中的 slider_value 默认值，调用 WM_SetCallback 函数设置窗口回调函数，最后调用函数 GUI_CreateDialogBox 根据资源表和 _cbDialog 回调函数创建对话框。

(4) 窗口回调函数

```
/**
 * @brief      窗口回调函数
 * @param      pMsg:消息
 * @retval     无
 */
static void _cbBkWindow(WM_MESSAGE *pMsg)
```

```
{
    switch (pMsg->MsgId)
    {
        case WM_PAINT:
            GUI_SetBkColor(GUI_BLUE);
            GUI_Clear();

        default:
            WM_DefaultProc(pMsg);
    }
}
```

从上述的源码可以看出，设置背景颜色为蓝色。

(5) 对话框回调函数

```
/**
 * @brief      对话框回调函数
 * @param      pMsg:消息
 * @retval     无
 */
static void _cbDialog(WM_MESSAGE *pMsg)
{
    WM_HWIN hItem;
    int      NCode;
    int      Id;
    uint8_t  i;

    switch (pMsg->MsgId)
    {
        case WM_PAINT:
            DrawColorBlock();
            return;

        case WM_INIT_DIALOG:
            /* 初始化 FRAMEWIN */
            hItem = pMsg->hWin;
            FRAMEWIN_SetText(hItem, "ALIENTEK SLIDER");
            FRAMEWIN_SetFont(hItem, GUI_FONT_24_ASCII);
            FRAMEWIN_SetTextColor(hItem, 0x0080FFFF);
            FRAMEWIN_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);

            /* 初始化 TEXT0 */
            hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_0);
            TEXT_SetText(hItem, "RGB BLOCK CONTROL");
            TEXT_SetFont(hItem, GUI_FONT_COMIC18B_ASCII);
    }
}
```

```

TEXT_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
/* 初始化 TEXT1 */
hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_1);
TEXT_SetText(hItem, "LED1 BRIGHTNESS CONTROL");
TEXT_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
TEXT_SetFont(hItem, GUI_FONT_COMIC18B_ASCII);
/* 初始化 TEXT2 */
hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_2);
TEXT_SetText(hItem, "RED:");
TEXT_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
TEXT_SetFont(hItem, GUI_FONT_24_ASCII);
/* 初始化 TEXT3 */
hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_3);
TEXT_SetText(hItem, "GREEN:");
TEXT_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
TEXT_SetFont(hItem, GUI_FONT_24_ASCII);
/* 初始化 TEXT4 */
hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_4);
TEXT_SetText(hItem, "BLUE:");
TEXT_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
TEXT_SetFont(hItem, GUI_FONT_24_ASCII);
/* 初始化 TEXT5 */
hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_5);
TEXT_SetText(hItem, "LED:");
TEXT_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
TEXT_SetFont(hItem, GUI_FONT_24_ASCII);

/* 初始化 EDIT0 */
for (i = 0; i < 3; i++) /* 设置 3 个 EDIT 控件 */
{
    hItem = WM_GetDialogItem(pMsg->hWin, ID_EDIT_0 + i);
    /* 设置十进制模式最大值为 255 */
    EDIT_SetDecMode(hItem, slider_dev.slider_value[i], 0, 255, 0, 0);
    /* EDIT 的对齐模式 */
    EDIT_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
    EDIT_SetFont(hItem, GUI_FONT_24_ASCII); /* 设置字体 */
}

/* 初始化 SLIDER */
for (i = 0; i < 3; i++) /* 设置 3 个 SLIDER 控件 */
{
    /* 获取控件的句柄 */
    hItem = WM_GetDialogItem(pMsg->hWin, ID_SLIDER_0 + i);

```



```

        SLIDER_SetRange(hItem, 0, 255); /* 设置控件的最大值为 255 */
        /* 设置滑块的位置 */
        SLIDER_SetValue(hItem, slider_dev.slider_value[i]);
    }

    /* 获取 ID_SLIDER_3 控件句柄 */
    hItem = WM_GetDialogItem(pMsg->hWin, ID_SLIDER_3);
    SLIDER_SetRange(hItem, 0, 300); /* 设置最大值为 300 */
    SLIDER_SetValue(hItem, 300); /* 设置滑块当前值为 300，此时 LED0 最亮 */
    break;

case WM_NOTIFY_PARENT:
    Id = WM_GetId(pMsg->hWinSrc);
    NCode = pMsg->Data.v;

    switch (Id)
    {
        case ID_SLIDER_0:
        case ID_SLIDER_1:
        case ID_SLIDER_2:
            switch (NCode)
            {
                case WM_NOTIFICATION_CLICKED:
                    break;

                case WM_NOTIFICATION_RELEASED:
                    break;

                case WM_NOTIFICATION_VALUE_CHANGED:
                    SetEditValue(pMsg->hWin, Id);
                    break;
            }
        case ID_SLIDER_3:
            switch (NCode)
            {
                case WM_NOTIFICATION_CLICKED:
                    break;

                case WM_NOTIFICATION_RELEASED:
                    break;

                case WM_NOTIFICATION_VALUE_CHANGED:
                    hItem = WM_GetDialogItem(pMsg->hWin, ID_SLIDER_3);

```

```

        slider_dev.slider_value[3]=SLIDER_GetValue(hItem);
        break;
    }

}

default:
    WM_DefaultProc(pMsg);
    break;
}
}

```

上面的源码中，最主要的两个函数为 `emwin_set_edit_value` 和 `emwin_draw_color_block`，如下源码所示：

```

/**
 * @brief      根据 SLIDER 的值设置 EDIT 的值
 * @param      hDlg: 窗口句柄
 * @param      Id: 小工具 ID
 * @retval     无
 */
void emwin_set_edit_value(WM_HWIN hDlg, int Id)
{
    WM_HWIN hSlider;
    WM_HWIN hEdit;
    uint8_t index;
    index = Id - ID_SLIDER_0;
    /* 获取那个滑块在移动的句柄 */
    hSlider = WM_GetDialogItem(hDlg, ID_SLIDER_0 + index);
    /* 获取滑块对应的 EDIT 区域 */
    hEdit = WM_GetDialogItem(hDlg, ID_EDIT_0 + index);
    /* 获取滑块的值并存入结构体 slider_value */
    slider_dev.slider_value[index] = SLIDER_GetValue(hSlider);
    EDIT_SetValue(hEdit, slider_dev.slider_value[index]); /* 设置 EDIT 值 */
    /* 最后使客户端窗口无效 */
    WM_InvalidateWindow(WM_GetClientWindow(hDlg));
}

```

此函数就是获取滑块的控件句柄并设置 EDIT 值，最后需要调用函数 `WM_InvalidateWindow` 使窗口无效化。

```

/**
 * @brief      绘制颜色框
 * @param      无
 * @retval     无
 */
void emwin_draw_color_block(void)
{

```

```
GUI_SetColor/slider_dev.slider_value[0]);
GUI_FillRect(560, 100, 595, 135);
GUI_SetColor/slider_dev.slider_value[1] << 8);
GUI_FillRect(560, 165, 595, 200);
GUI_SetColor(((uint32_t)slider_dev.slider_value[2] << 16);
GUI_FillRect(560, 230, 595, 265);
GUI_SetColor/slider_dev.slider_value[0] |
                (((uint32_t)slider_dev.slider_value[1]) << 8) |
                (((uint32_t)slider_dev.slider_value[2]) << 16));
GUI_FillRect(600, 100, 765, 265);
}
```

上述的源码可知：系统根据 R(红色)、G(绿色)和 B(蓝色)的滑块控件的滑动值来填充颜色框的颜色值。

30.5.3 下载验证

编译并下载例程到开发板中，LCD 显示如下图所示：

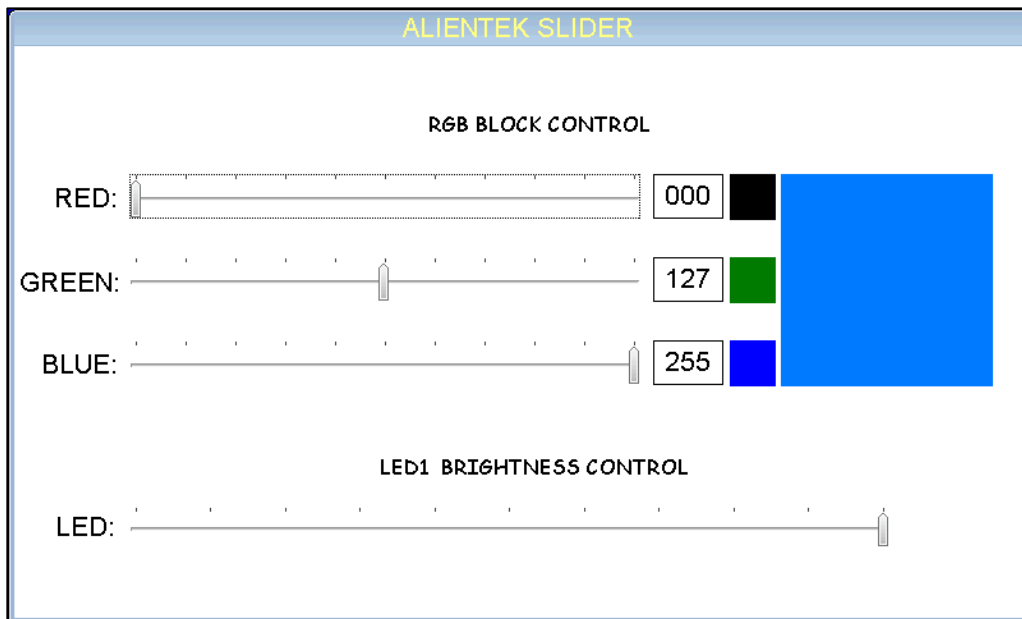


图 30.5.3.1 LCD 显示

滑动 RED、GREEN 和 BLUE 这三个滑块既可调整右侧色块的颜色，当这三个分量都为 0 的时候色块就为黑色，如下图所示：

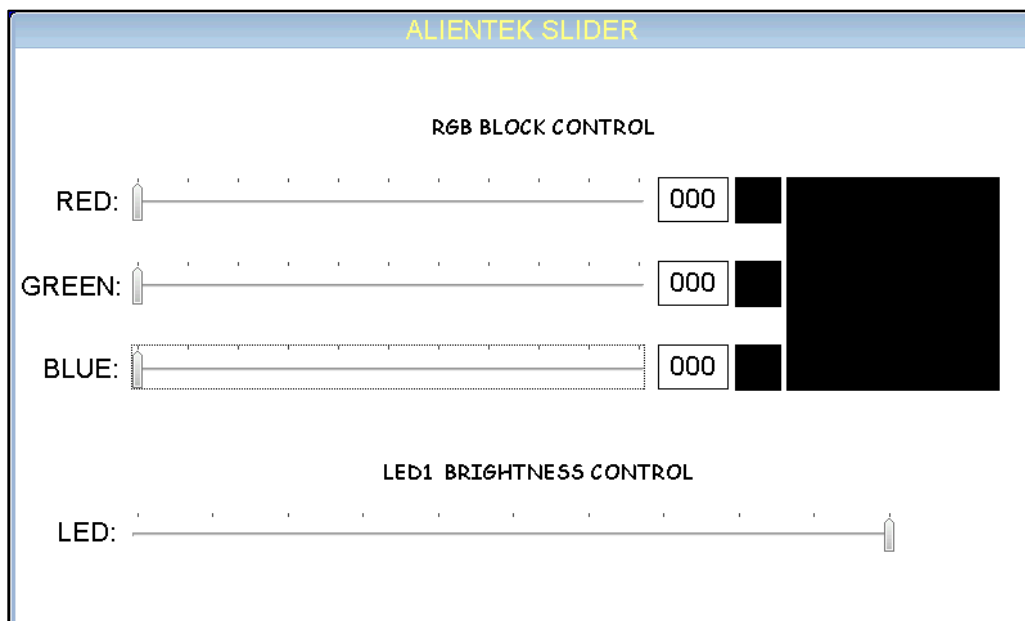


图 30.5.3.2 LCD 色块调节

第三十一章 字体

在前面的所有章节中都没有使用汉字，在接下来的几章将会详细的讲解如何在 emWin 中使用汉字，在此之前我们先简单了解一下有关 emWin 字体的基础知识。

本章分为以下几个部分：

31.1 字体简介

31.2 字体类型

31.3 字体格式

31.4 声明自定义字体

31.5 选择字体

31.6 字体常用 API 函数

31.1 字体简介

字体支持的第一种方法就是使用字体定义中含每个字符的 1bpp 像素信息位图的 C 文件。这种字体支持仅限于使用应用编译的字体。随着时间推移，字体支持在字体质量、ROM 要求、性能、可扩展性和运行时添加更多字体的能力方面进行了改进。同时 emWin 字体包含抗锯齿、复合字符的图样（例如泰语中的要求）、位于外部非可寻址介质中的字体以及 TrueType 支持。除 TrueType 字体（矢量字体）格式外，所有其他字体均为位图字体。

31.2 字体类型

emWin 支持自行定义的内部类型不同的字体以及常用的 TrueType 字体。

✧ 等宽位图字体

等宽位图字体的每个字符大小相同。在比例字体中，每个字符有自己的宽度，而等宽字体的宽度只需定义一次。像素信息保存为 1bpp，涵盖整个字符区域。

✧ 比例位图字体

比例位图字体的每个字符高度相同、宽度可能不同。像素信息为 1bpp，涵盖整个字符区域。

✧ 抗锯齿字体，包含 2bpp 抗锯齿信息

每个字符高度相同、宽度可能不同。像素信息为 2bpp 抗锯齿，涵盖整个字符区域。

✧ 抗锯齿字体，包含 4bpp 抗锯齿信息

每个字符高度相同、宽度可能不同。像素信息为 4bpp 抗锯齿，涵盖整个字符区域。

✧ 扩展比例位图字体

扩展比例位图字体的每个字符拥有自己的高度和宽度。像素信息保存为 1bpp，涵盖字形位图区域。

✧ 扩展比例位图字体，包含 2bpp 抗锯齿信息

每个字符高度相同、宽度可能不同。像素信息为 2bpp 抗锯齿，涵盖字形位图区域。

✧ 扩展比例位图字体，包含 4bpp 抗锯齿信息

每个字符高度相同、宽度可能不同。像素信息为 4bpp 抗锯齿，涵盖字形位图区域。

✧ 扩展比例位图字体，带边框

在某些情况下（例如编译时背景颜色未知），可以使用带边框的字体。带边框的字体始终在透明模式下绘制，与当前设置无关。字符像素按当前所选的前景颜色绘制，边框按背景颜色

绘制。前景颜色和背景颜色之间明显的对比可以确保能够阅读文本，而与背景无关。请注意，这种类型的字体不适用于复合字符，例如泰语，另外，也不适用于 Arabic 字体。

✧ TrueType 矢量字体

emWin 对于矢量字体是支持的，后面章节中会讲解到矢量字体的使用，下图列出了各字体之间的差异。

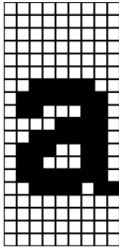
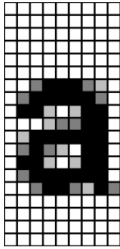
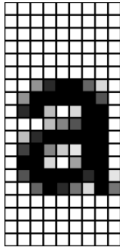
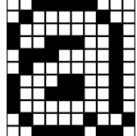
比例位图字体	比例位图字体， AA2	比例位图字体， AA2	扩展比例位图字体	扩展比例位图字体， 带边框
				

图 31.2.1 各字体类型之间的差异

31.3 字体格式

31.3.1 C 文件格式

这是最常见的字体使用方式。使用 C 文件格式的字体时，建议作为库模块来编译所有可用的字体和链接它们，或将所有字体对象文件置于可以链接到应用的库中。这样，可确保实际只链接了您的应用所需的字体。

✧ 何时使用

在编译时字体已知以及对于字体数据拥有足够的可寻址内存的情况下，可以使用这种格式。

✧ 要求

为了能够在应用中使用字体 C 文件，必须满足以下要求：

1. 字体文件为与 emWin 兼容的 C 文件、对象文件或库格式。
2. 字体文件与您的应用链接。
3. 应用中包含字体声明。

✧ 格式说明

字体 C 文件首先包含字体所含全部字符的像素信息，之后，是有关每个字符大小信息的字符信息表。此表之后是字体文件中所含各个字符相邻区域的范围信息结构，而每个结构均指向下一个结构。请注意，如果使用许多单独的字符，这种方法可以大幅扩展字体文件。在范围信息结构之后，是 GUI_FONT 结构，其中包含字体的类型、像素大小等主要信息。

31.3.2 系统独立字体（SIF）格式

系统独立字体是包含字体信息的二进制数据块，可以使用字体转换器来创建系统独立字体。后面章节会对字体转换器进行简单的介绍。

✧ 何时使用

在编译时字体未知以及对于字体数据拥有足够的可寻址内存的情况下，可以使用这种格式。

✧ 要求

为了能够在应用中使用 SIF 字体文件，需要在可寻址存储器（ROM 或 RAM）中保存整个文件。

◇ 格式说明

SIF 文件的结构与 C 文件大致相同，它在二进制格式中包含相同的信息。文件要素的顺序反之亦然：常规字体信息，之后是范围信息结构、字符信息表和所有字符的最小像素信息。

31.3.3 外部位图字体（XBF）格式

同 SIF 字体一样，XBF 字体也是包含字体信息的二进制数据块，并且可以使用字体转换器来创建 XBF 文件。与其他字体不同的是，XBF 字体在使用时不必位于存储器中，而 emWin 的所有其他字体均需要完全驻存在存储器中。XBF 字体文件在使用时可保存在任意外部介质中，通过“GetData”回调函数可以进行数据访问，而所有其他字体都需要驻存在可寻址存储器（RAM 或 ROM）中。XBF 字体的优势在于，可以在内存很小的系统上使用非常大的字体。

◇ 何时使用

如果对于字体数据没有足够的可寻址内存，并且可以使用任何其他种类的外部介质来存储字体的情况下，可以使用这种格式。

◇ 要求

为了能够在应用中使用 XBF 字体，需要使用“GetData”回调函数来负责获取字体数据。

◇ 格式说明

这种格式总体上与 SIF 和 C 文件格式有所不同。首先，它包含小块常规字体信息，其中包含最低和最高的字符代码。之后，是包含最低和最高字符代码之间每个字符的偏移和数据大小信息的访问表。如果不存在字符，则相应字符的此信息为零。访问表之后是所有字符的字符信息，包含像素数据和字符大小信息。

31.3.4 TrueType 字体(TTF)格式

TrueType 是 Apple Computer 开发的轮廓字体标准。它为字体开发人员提供对在各种字体高度下字体显示方式的高度控制。与位图字体（基于每个字符的位图）不同，TrueType 字体基于矢量图形。矢量表示的优势在于无损的可扩展性。这意味着，每个字符在绘制前需要光栅化为位图。为避免每次绘制字符时都进行光栅化，通常用字体引擎缓存位图数据。这要求 CPU 速度快、RAM 足够。默认的 emWin 源码中不包含 TTF 包。该包可在 www.segger.com/link/emwin_freetype.zip 下免费获得。

◇ 许可

emWin 对 TTF 支持的实施基于来自 David Turner、Robert Wilhelm 和 Werner Lemberg 的 FreeType 字体库，该库可在 www.freetype.org 下免费获得。emWin 对该库的使用符合 GUITrueType\FTL.txt 下的 FreeType 许可。emWin 对该库进行了少许改编，添加了带有 GUI 函数的“粘贴”层。

◇ 何时使用

如果在运行时需要对字体进行缩放，可使用这种格式。

◇ 要求

1. CPU: TTF 仅适用于 32 位 CPU。
2. ROM: TTF 引擎的 ROM 要求大约为 250 K，确切大小取决于 CPU、编译器以及编译器的优化水平。
3. RAM: 该库的 RAM 要求主要取决于使用的字体，TTF 引擎的基本 RAM 要求大约为 50 K。在使用 GUI_TTF_CreateFont 创建 GUI 字体时，字体引擎会加载生成字符所需的 TTF 文件中定义的所有字体表，不同字体之间的表大小有所差异。创建字体额外要求的 RAM 量可能介于几个 KB 到 1MB 以上之间，一般字体需要 80-300 kb。取决于使用的字体文件需要多少 RAM，

至少，TTF 引擎需要位图缓存。默认情况下，引擎使用 200 K 的缓存就足够大多数应用使用。TTF 引擎通过非 emWin 函数 malloc 和 free 分配内存。使用 TTF 引擎之前，必须确保能够运行这些函数。

◇ 格式说明

有关 TTF 格式的详细信息，请参阅 www.apple.com 下可用的信息。

31.4 声明自定义字体

声明自定义字体原型最推荐的方法是将它们置于应用定义的头文件中。使用这些字体的每个应用源文件中均应包含此内容。如以下示例所示：

```
#include "GUI.H"

extern GUI_CONST_STORAGE GUI_FONT GUI_FontApp1;
extern GUI_CONST_STORAGE GUI_FONT GUI_FontApp2;
```

请注意，如果这些字体要用于 BUTTON_FONT_DEFAULT 或类似的 emWin 配置宏，这种声明原型不起作用。在这种情况下，字体需要在配置文件 GUIConf.h 中声明。这种情况下的声明如以下示例所示：

```
typedef struct GUI_FONT GUI_FONT;
extern const GUI_FONT GUI_FontApp1;
#define BUTTON_FONT_DEFAULT &GUI_FontApp1
#define EDIT_FONT_DEFAULT &GUI_FontApp1
```

由于前面 emWin 包含 GUIConf.h 的位置未定义 GUI_FONT 结构，所以需要类型定义符（typedef）。

31.5 选择字体

emWin 提供不同的字体，但是始终选择其中之一。通过调用函数 GUI_SetFont 或 GUI_XX X_CreateFont 函数之一可更改此选项，它们将选择用于当前任务所有文本输出的字体。

如果应用总没有选择任何字体，则将使用默认字体，默认字体在 GUIConf.h 中配置，且可以更改。因此应该确保默认字体是在应用中实际使用的字体。

31.6 字体常用 API 函数

表 31.6.1 列出了有关字体的常用 API 函数。

函数	描述
GUI_SetDefaultFont()	设置默认字体
GUI_SetFont()	设置当前字体
GUI_GetCharDistX()	返回当前字体中指定字符的宽度像素（X 大小）
GUI_GetFont()	返回当前选择的字体的指针
GUI_GetFontDistY()	返回当前字体的 Y 间距
GUI_GetFontInfo()	返回包含字体信息的结构
GUI_GetFontSizeY()	返回当前字体的高度像素（Y 大小）
GUI_GetLeadingBlankCols()	返回给定字符的前导空格像素列数
GUI_GetStringDistX()	返回使用当前字体的文本的 X 大小
GUI_GetTextExtend()	评估使用当前字体的文本的大小
GUI_GetTrailingBlankCols()	返回给定字符的后导空格像素列数
GUI_GetYDistOffFont()	返回特定字体的 Y 间距
GUI_GetYSizeOffFont()	返回特定字体的 Y 大小

GUI_IsInFont()	评估特定字体中是否存在指定的字符
GUI_SetDefaultFont()	设置 GUI_Init()之后使用的默认字体

表 31.6.1 字体常用 API 函数

(1) GUI_GetCharDistX 函数

获取显示当前所选字体中指定字符的宽度像素（X 大小），该函数的原型如下源码所示：

```
int GUI_GetCharDistX(U16 c);
```

该函数的相关形参如下表所示：

参数	描述
c	计算宽度的字符

表 31.6.2 GUI_GetCharDistX 函数相关形参描述

返回值：返回当前字符的宽度。

(2) GUI_GetDefaultFont 函数

返回当前默认设置的默认字体，该函数的原型如下源码所示：

```
const GUI_FONT * GUI_GetDefaultFont(void);
```

返回值：该函数返回指向当前设置的默认字体的指针。

(3) GUI_GetFont 函数

获取当前选择的字体的指针，该函数的原型如下源码所示：

```
const GUI_FONT * GUI_GetFont(void);
```

返回值：返回当前选择的字体的指针。

(4) GUI_GetFontDistY 函数

获取当前所选字体的 Y 间距，该函数的原型如下源码所示：

```
int GUI_GetFontDistY(void);
```

注意：Y 间距是两行相邻文本之间的垂直距离（像素）。返回的值是为当前所选字体输入的 YDist 值。返回的值对比例字体和等宽字体都有效。

返回值：返回当前所选字体的 Y 间距。

(5) GUI_GetFontDistY 函数

计算特定字体的 GUI_FONTINFO 结构的指针，该函数的原型如下源码所示：

```
void GUI_GetFontInfo(const GUI_FONT * pFont, GUI_FONTINFO * pfi);
```

该函数的相关形参如下表所示：

参数	描述
pFont	指向该字体
pfi	指向 GUI_FONTINFO 结构

表 31.6.3 GUI_GetFontInfo 函数相关形参描述

返回值：无。

GUI_FONTINFO 结构的定义如下所示：

```
typedef struct {
    U16 Flags;
} GUI_FONTINFO;
```

成员变量 Flags 可采用以下值：

```
GUI_FONTINFO_FLAG_PROP
GUI_FONTINFO_FLAG_MONO
GUI_FONTINFO_FLAG_AA
GUI_FONTINFO_FLAG_AA2
GUI_FONTINFO_FLAG_AA4
```

(6) GUI_GetFontSizeY 函数

返回当前所选字体的高度像素（Y 大小），该函数的原型如下源码所示：

```
int GUI_GetFontSizeY(void);
```

返回值：是为当前所选字体输入的 YSize 值。

(7) GUI_GetLeadingBlankCols 函数

获取当前所选字体中给定字符的前导空格像素列数，该函数的原型如下源码所示：

```
int GUI_GetLeadingBlankCols(U16 c);
```

该函数的相关形参如下表所示：

参数	描述
c	要使用的字符

表 31.6.4 GUI_GetLeadingBlankCols 函数相关形参描述

返回值：返回当前所选字体中给定字符的前导空格像素列数。

(8) GUI_GetStringDistX 函数

用于获取显示当前所选字体中指定字符串的 X 大小，该函数的原型如下源码所示：

```
int GUI_GetStringDistX(const char * s);
```

该函数的相关形参如下表所示：

参数	描述
c	指向字符串的指针

表 31.6.5 GUI_GetStringDistX 函数相关形参描述

返回值：返回显示当前所选字体中指定字符串的 X 大小。

(9) GUI_GetTextExtend 函数

返回用于显示当前所选字体中指定字符串的 X 大小，该函数的原型如下源码所示：

```
void GUI_GetTextExtend(GUI_RECT * pRect, const char * s, int Len);
```

该函数的相关形参如下表所示：

参数	描述
pRect	指向存储结果的 GUI_RECT-structure
s	指向字符串的指针
Len	字符串的字符数

表 31.6.6 GUI_GetTextExtend 函数相关形参描述

返回值：无。

(10) GUI_GetTrailingBlankCols 函数

获取当前所选字体中给定字符的后导空格像素列数，该函数的原型如下源码所示：

```
int GUI_GetTrailingBlankCols(U16 c);
```

该函数的相关形参如下表所示：

参数	描述
c	要使用的字符

表 31.6.7 GUI_GetTrailingBlankCols 函数相关形参描述

返回值：返回当前所选字体中给定字符的后导空格像素列数。

(11) GUI_GetYDistOfFont 函数

获取特定字体的 Y 间距，该函数的原型如下源码所示：

```
int GUI_GetYDistOfFont(const GUI_FONT * pFont);
```

该函数的相关形参如下表所示：

参数	描述
pFont	指向字体的指针

表 31.6.8 GUI_GetYDistOffont 函数相关形参描述

返回值：返回特定字体的 Y 间距。

(12) GUI_GetYSizeOfFont 函数

获取特定字体的 Y 大小，该函数的原型如下源码所示：

```
int GUI_GetYSizeOfFont(const GUI_FONT * pFont);
```

该函数的相关形参如下表所示：

参数	描述
pFont	指向字体的指针

表 31.6.9 GUI_GetYSizeOfFont 函数相关形参描述

返回值：返回特定字体的 Y 大小。

(13) GUI_IsInFont 函数

评估特定字体中是否包含指定的字符，该函数的原型如下源码所示：

```
char GUI_IsInFont(const GUI_FONT * pFont, U16 c);
```

该函数的相关形参如下表所示：

参数	描述
pFont	指向字体的指针
c	要搜索的字符

表 31.6.10 GUI_IsInFont 函数相关形参描述

返回值：1, 如果找到该字符；0, 如果没有找到该字符。

(14) GUI_SetDefaultFont 函数

设置默认用于文本输出的字体，该函数的原型如下源码所示：

```
void GUI_SetDefaultFont(const GUI_FONT * pFont);
```

该函数的相关形参如下表所示：

参数	描述
pFont	指向选作默认字体的指针

表 31.6.11 GUI_SetDefaultFont 函数相关形参描述

返回值：无。

注意：该函数要在 GUI_X_Config 中使用。定义 GUI_DEFAULT_FONT 不再是强制性的。如果既没有定义 GUI_DEFAULT_FONT，也没有调用 GUI_SetDefaultFont，GUI_Font6x8 将被设置为默认字体。如果没有使用任何一个 emWin 字体，GUI_DEFAULT_FONT 必须定义为 NULL，并且需要用这个函数将自定义字体设置为默认字体。

(15) GUI_SetFont 函数

设置用于文本输出的字体，该函数的原型如下源码所示：

```
const GUI_FONT * GUI_SetFont(const GUI_FONT * pNewFont);
```

该函数的相关形参如下表所示：

参数	描述
pNewFont	指向选择并使用的字体的指针

表 31.6.12 GUI_SetFont 函数相关形参描述

返回值：返回指向先前选择的字体的指针，以便进行缓冲。

第三十二章 XBF 格式字体显示

XBF 字体也是包含字体信息的二进制数据块，并且可以使用字体转换器来创建 XBF 文件，在 LCD 上显示汉字是很重要的功能，emWin 支持多种显示汉字的方法，本章讲解一下如何在 emWin 中使用 XBF 格式的汉字。

本章分为以下几个部分：

32.1 XBF 格式字库制作

32.2 XBF 字体相关 API 函数

32.3 XBF 格式字库实验

32.1 XBF 格式字库制作

与其他字体不同的是，XBF 字体在使用时不必位于内存中，而 emWin 的所有其他字体均需要完全驻存在内存中。XBF 字体文件在使用时可保存在任意外部介质中。通过“GetData”回调函数可以进行数据访问。XBF 字体的优势在于，可以在内存很小的系统上使用非常大的字体。在使用包含许多不按顺序直接跟随字符的字体时，XBF 字体提供了性能上优势。这种字符集将使得字体转换器创建一个 C 文件字体，包含许多 GUI_FONT_PROP 结构，该结构有一个指向下一对应结构的指针。一个字体中存在的 GUI_FONT_PROP 结构越多，显示字符时所用的时间就越长。XBF 字体仅使用内存偏移，这样每个字符可以用相同的时间找到。

首先我们要用字体转换器制作所需的 XBF 字体，我们下载的 emWin 源码中已经包含了字体转换器：FontCvtST.exe，它和 GUIBulider 在同一目录下，下面我们使用这个软件来生成 XBF 字库，FontCvtST.exe 安装完成以后打开字体转换器如下图所示：

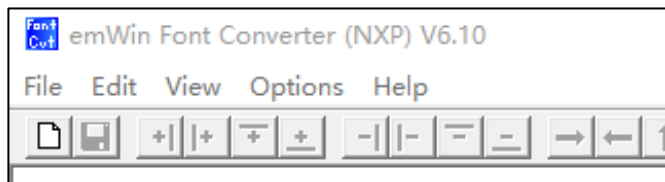


图 32.1.1 字体转换器

接下来我们制作 4 个 XBF 字库，字体都选择为宋体，大小分别为 12,16,24 和 36，字体类型为“Extended,antialiased, 2bpp”，也就是扩展比例位图字体，包含 2bpp 抗锯齿。下面笔者分为几个步骤来讲解本小节的内容。

✧ 选择字体类型，如下图所示：

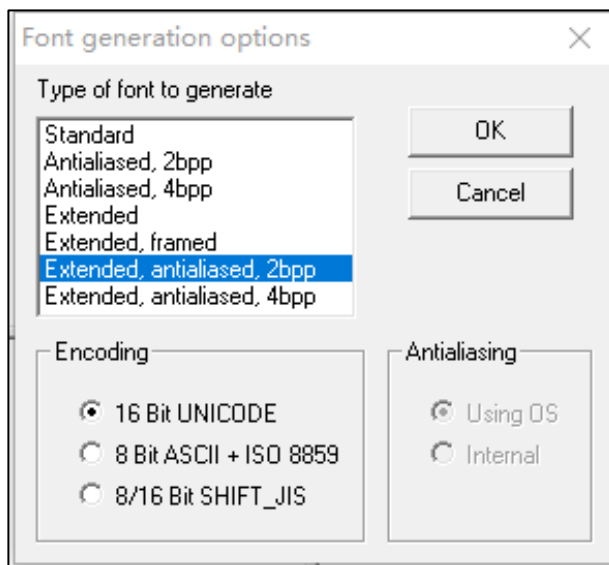


图 32.1.2 字体类型为“Extended,antialiased,2bpp”

✧ 点击图 32.1.2 中的“OK”以后，出现下图所示对话框：

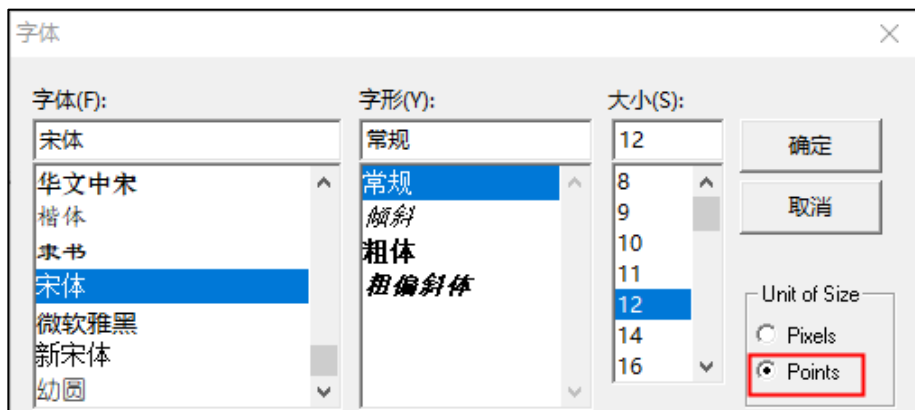


图 32.1.3 设置字体格式

这里，笔者选择字体为“宋体”，字体大小为“12”号，“Unit of Size”选择框为“Points”。

设置完成以后点击“确定”，字体转换器如下图所示：

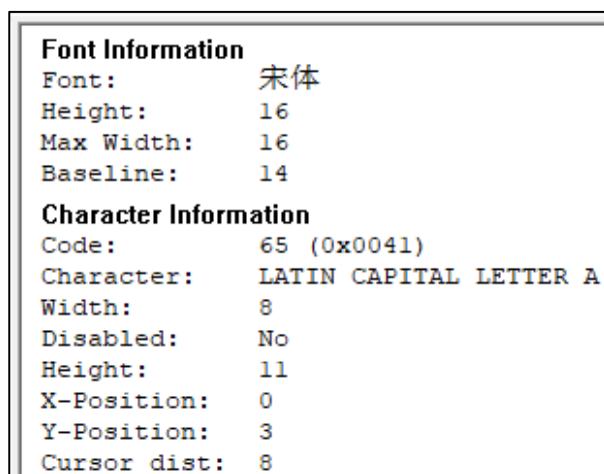


图 32.1.4 设置完字体类型和格式以后的字体转换器

- ✧ 保存字体，点击：File→Save As，出现如下图所示对话框，选择保存类型为.xbf，文件名设置为 XBF12.xbf。

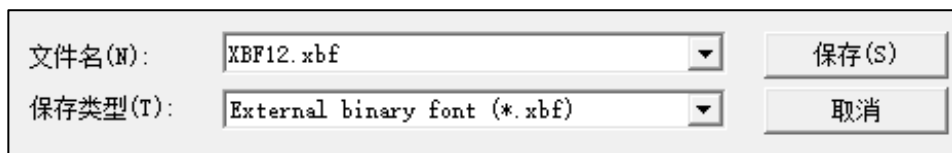


图 32.1.5 设置保存格式

- ✧ 点击图 32.1.5 中的保存以后，等待字库制作完成，使用同样的方法制作其他 16,24 和 36 号字。
- ✧ 将制作好的 XBF 字体拷贝到 SD 卡指定文件目录中：SYSTEM→EMWINFONT→XBF，如下图所示：

名称	修改日期	类型
XBF12.xbf	2020/9/25 16:50	XBF 文件
XBF16.xbf	2020/9/25 16:53	XBF 文件
XBF24.xbf	2020/9/25 16:59	XBF 文件
XBF36.xbf	2020/9/25 17:09	XBF 文件

图 32.1.6 将制作好的字体复制到 SD 卡中

我们已经制作好了 12 号、16 号、24 号和 36 号共四种不同大小的字体，大家可以直接复制到 SD 卡中。在后面的章节中我们还会用到 SD 卡，实验中需要用到的所有文件正点原子都已经准备好了放在文件夹“emWin 实验所需 SD 卡文件”中，大家可以把其中的所有文件拷贝到 SD 卡根目录下。

emWin 实验所需 SD 卡文件文件夹共有三个文件夹：PICTURE、SCREENSHOT 和 SYSTEM，将这三个文件拷贝到 SD 卡根目录中！，如下图所示：

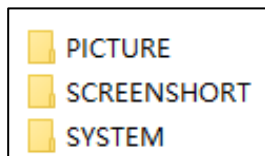


图 32.1.7 实验所需 SD 卡文件

32.2 XBF 字体相关 API 函数

与 XBF 字体相关的函数有两个：GUI_XBF_CreateFont 和 GUI_XBF_DeleteFont，下面我们分别地介绍这两个函数。

(1) GUI_XBF_CreateFont 函数

将指针传递至负责从 XBF 字体文件获取数据的回调函数创建和选择字体，该函数的原型如下源码所示：

```
int GUI_XBF_CreateFont( GUI_FONT          *pFont,
                        GUI_XBF_DATA      *pXBF_Data,
                        const GUI_XBF_TYPE *pFontType,
                        GUI_XBF_GET_DATA_FUNC *pfGetData,
                        void                *pVoid );
```

函数参数含义如下表所示：

参数	描述
pFont	指向 RAM 中由该函数填充的 GUI_FONT 结构
pXBF_Data	指向 RAM 中由该函数填充的 GUI_XBF_DATA 结构
pFontType	元素 pFontType 的允许值
	GUI_XBF_TYPE_PROP 在参数 pFont 指向比例字体时使用。
	GUI_XBF_TYPE_PROP_EXT 在参数 pFont 指向扩展比例字体时使用
	GUI_XBF_TYPE_PROP_FRM 在参数 pFont 指向扩展带边框比例字体时使用
	GUI_XBF_TYPE_PROP_AA2_EXT 在参数 pFont 指向使用 2 bpp 抗锯齿的扩展比例字体时使用
	GUI_XBF_TYPE_PROP_AA4_EXT 在参数 pFont 指向使用 4 bpp 抗锯齿的扩展比例字体时使用
pfGetData	指向负责从字体文件获取数据的回调函数，请参见以下原型
pVoid	传递到“GetData”回调函数的应用定义的指针

表 32.2.1 函数参数描述

返回值：该函数成功返回 0，字体创建失败返回 1。

GUI_XBF_GET_DATA_FUNC 描述

```
int GUI_XBF_GET_DATA_FUNC( U32      Off,
                           U16      NumBytes,
                           void      *pVoid,
                           void      pBuffer);
```

该函数应设置 pBuffer，指向请求数据所在的位置，比如下面演示的“GetData”函数：

```
int GetData(U32 off, U16 NumBytes, void * pVoid, void *pBuffer)
{
    HANDLE * phFile;
    DWORD NumBytesRead;
    phFile = (HANDLE *)pVoid; /* 要读取的文件 */
    /* 检查缓冲区大小 */
    if (NumBytesReq > sizeof(_acBuffer)) NumBytesReq = sizeof(_acBuffer);
    /* 移动文件指针到需要读取的数据处 */
    SetFilePointer(*phFile, Off, 0, FILE_BEGIN);
    /* 将数据从存储器中读取到缓冲区_acBuffer 中 */
    ReadFile(phFile, _pBuffer, NumBytes, NumBytes, NULL, (UINT *)&bread);
    return 0;
}
```

上面函数 GetData 只是一个演示函数，在实际使用过程中需要根据自己所使用的文件系统做出相应的修改。

其他信息

该函数的参数 pfGetData 应指向负责从字体文件获取数据的应用定义的回调函数，在请求字体数据时，参数 pVoid 将传递给回调函数，例如，可使用它将文件句柄传递给回调函数。该函数要求指向 GUI_FONT 结构和 GUI_XBF_DATA 结构，该函数将使用字体信息填充这些结构，这些结构的内容在字体使用期间必须保持有效。该函数不知道必须要创建的 XBF 字体类型，因此必须使用参数 pFontType 告知该函数要创建的字体类型，这样，可避免链接不必要的代码。默认情况下，每个字符的数据字节最大值限制为 200，这应该满足大部分要求，如果加

载使用更多字节的字符，在调试版本中将生成警告，通过向文件 GUIConf.h 添加以下定义可增加默认值：

```
#define GUI_MAX_XBF_BYTES 500 /* 设置最大字节数为 500 */
```

(2) GUI_XBF_DeleteFont 函数

删除 XBF 字体，该函数的原型如下源码所示：

```
void GUI_XBF_DeleteFont(GUI_FONT * pFont);
```

该函数的相关形参如下表所示：

参数	描述
pFont	指向要删除的字体

表 32.2.2 函数 GUI_XBF_DeleteFont() 相关形参描述

返回值：无。

其他信息

使用 GUI_XBF_CreateFont 创建的字体后，如果不再使用该字体，应将其删除。

32.3 XBF 格式字库实验

32.3.1 硬件设计

1. 例程功能

本实验主要讲解 XBF 格式汉字显示实验，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数读取 SD 卡上的 xbf 字库文件，然后对话框的文本根据字库来显示字体。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《实验 32 XBF 格式汉字显示》。

注意：本实验不支持 Mini、DMF407 开发板运行。

32.3.2 软件设计

32.3.2.1 程序流程图

本实验的程序流程图，如下图所示：

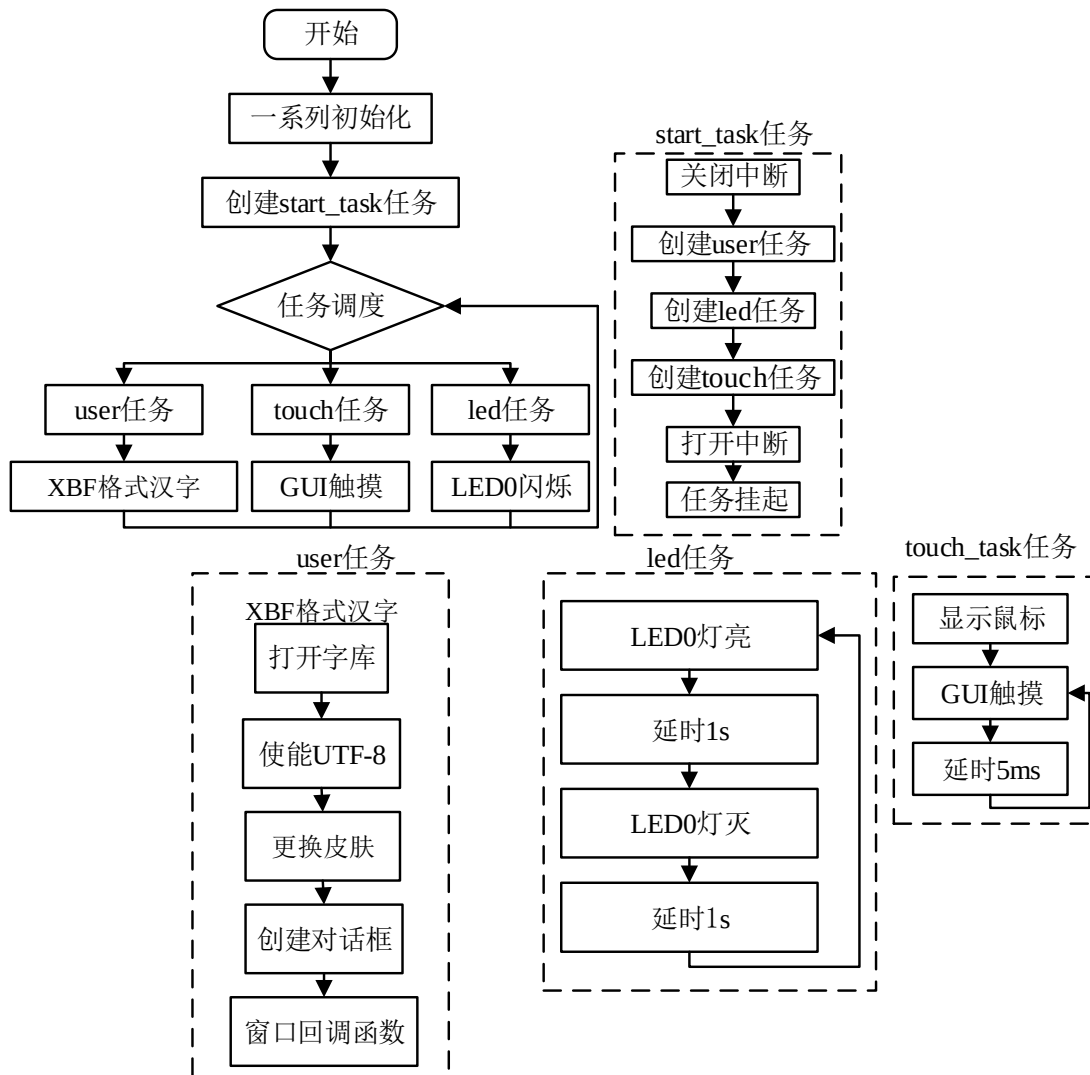


图 32.3.2.1.1 XBF 格式汉字显示实验流程图

32.3.2.2 程序解析

在上一节中我们已经制作好了 XBF 字库，但是此时 emWin 还不能使用，我们还没有制作适合 emWin 调用的字体，本章要使用到 SD 卡和文件系统 FATFS，关于 SD 卡和 FATFS 的使用不属于本教程范围，请参考正点原子的裸机教程中相关章节。本实验完整工程为：[emWin 实验 32 XBF 格式汉字显示](#)。

(1) 制作字体

打开本章对应实验工程中的 emwin_demo.c 文件，程序源码如下：

```

/**
 * @brief      创建 XBF12 字体，共 EMWIN 使用
 * @param      Off:          在 XBF 中偏移(位置)
 * @param      NumBytes:     要读取的字节数
 * @param      pVoid:        要读取的文件
 * @param      pBuff:        读取到的数据的缓冲区
 * @retval     返回值:0，成功；1，失败
 */

```

```
static int _cbGetData( uint32_t Off, uint16_t NumBytes,
                      void *pVoid,
                      void *pBuffer)
{
    int result;
    uint16_t bread;
    FIL *hFile;

    hFile = (FIL *)pVoid;

    /* 设置在文件中的偏移(位置) */
    result = f_lseek(hFile, Off);

    if (result != FR_OK) return 1; /* 返回错误 */

    /* 读取字体数据 */

    result = f_read(hFile, pBuffer, NumBytes, (UINT *)&bread); /* 读取数据 */

    if (result != FR_OK) return 1; /* 返回错误 */

    return 0;
}

/**
 * @brief      创建 XBF12 字体, 共 EMWIN 使用
 * @param      fxpath:XBF 字体文件路径
 * @retval     返回值:0, 成功; 1, 失败
 */
uint8_t emwin_create_xbf12(uint8_t *fxpath)
{
    int result;
    /* 打开字库文件 */
    result = f_open(&XBF12FontFile, (const TCHAR *)fxpath, FA_READ);

    if (result != FR_OK) return 1;

    /* 创建 XBF16 字体 */
    GUI_XBF_CreateFont( &XBF12_Font, /* 指向 GUI_FONT 结构 */
                       &XBF12_Data, /* 指向 GUI_XBF_DATA 结构 */
                       GUI_XBF_TYPE_PROP_AA2_EXT, /* 要创建的字体类型 */
                       (void *)_cbGetData, /* 回调函数 */
                       &XBF12FontFile); /* 窗体给回调函数_cbGetData 的参数 */
}
```

```

        return 0;
    }

    /**
     * @brief      创建 XBF16 字体，共 EMWIN 使用
     * @param      fxpath:XBF 字体文件路径
     * @retval     返回值:0，成功；1，失败
     */
uint8_t emwin_create_xbf16(uint8_t *fxpath)
{
    int result;
    /*打开字库文件 */
    result = f_open(&XBF16FontFile, (const TCHAR *)fxpath, FA_READ);

    if (result != FR_OK) return 1;

    /*创建 XBF16 字体 */
    GUI_XBF_CreateFont( &XBF16_Font,          /* 指向 GUI_FONT 结构 */
                       &XBF16_Data,          /* 指向 GUI_XBF_DATA 结构 */
                       GUI_XBF_TYPE_PROP_AA2_EXT, /* 要创建的字体类型 */
                       (void *)_cbGetData,      /* 回调函数 */
                       &XBF16FontFile); /* 窗体给回调函数_cbGetData 的参数 */

    return 0;
}

    /**
     * @brief      创建 XBF24 字体，共 EMWIN 使用
     * @param      fxpath:XBF 字体文件路径
     * @retval     返回值:0，成功；1，失败
     */
uint8_t emwin_create_xbf24(uint8_t *fxpath)
{
    int result;
    /* 打开字库文件 */
    result = f_open(&XBF24FontFile, (const TCHAR *)fxpath, FA_READ);

    if (result != FR_OK) return 1;

    /* 创建 XBF16 字体 */
    GUI_XBF_CreateFont( &XBF24_Font,          /* 指向 GUI_FONT 结构 */
                       &XBF24_Data,          /* 指向 GUI_XBF_DATA 结构 */
                       GUI_XBF_TYPE_PROP_AA2_EXT, /* 要创建的字体类型 */
                       (void *)_cbGetData,      /* 回调函数 */

```

```

        &XBF24FontFile);    /* 窗体给回调函数_cbGetData 的参数 */

    return 0;
}

/**
 * @brief      创建 XBF36 字体，共 EMWIN 使用
 * @param      fxpath:XBF 字体文件路径
 * @retval     返回值:0，成功；1，失败
 */
uint8_t emwin_create_xbf36(uint8_t *fxpath)
{
    int result;
    /*打开字库文件 */
    result = f_open(&XBF36FontFile, (const TCHAR *)fxpath, FA_READ);

    if (result != FR_OK) return 1;

    /* 创建 XBF16 字体 */
    GUI_XBF_CreateFont( &XBF36_Font,          /* 指向 GUI_FONT 结构 */
                        &XBF36_Data,          /* 指向 GUI_XBF_DATA 结构 */
                        GUI_XBF_TYPE_PROP_AA2_EXT, /* 要创建的字体类型 */
                        (void *)_cbGetData, /* 回调函数 */
                        &XBF36FontFile);    /* 窗体给回调函数_cbGetData 的参数 */

    return 0;
}

```

上述源码主要调用文件系统读取 SD 卡上的字库并创建相关字号的字体。

(2) 创建一个对话框

```

static const char *_ListViewTable[][4] =
{
    {"张三",      "20141226001", "97", "A"},
    {"李四",      "20141226002", "84", "B"},
    {"刘翔",      "20141226003", "73", "C"},
    {"赵子龙",    "20141226004", "85", "B"},
    {"刘备",      "20141226005", "45", "E"},
    {"欧阳夏雨",  "20141226006", "98", "A"},
    {"王子奇",    "20141226007", "84", "B"},
    {"孙菲菲",    "20141226008", "65", "D"},
};

char *MultiEdit_String =
    " 正点原子\r\n      \r\n"
    " 板载:10M/100M 以太网 PHY 芯片\r\n";

```

```

/* 对话框资源列表 */
static const GUI_WIDGET_CREATE_INFO _aDialogCreate[] =
{
    { FRAMEWIN_CreateIndirect, "Framewin", ID_FRAMEWIN_0, 0, 0, 800, 480,
    0, FRAMEWIN_CF_MOVEABLE, 0 },
    { MULTIEDIT_CreateIndirect, "Multiedit", ID_MULTIEDIT_0, 20, 20, 300, 160, 0, 0x0, 0 },
    { BUTTON_CreateIndirect, "BUTTON", ID_BUTTON_0, 398, 370, 140, 50, 0, 0x0, 0 },
    { BUTTON_CreateIndirect, "BUTTON", ID_BUTTON_1, 599, 370, 140, 50, 0, 0x0, 0 },
    { CHECKBOX_CreateIndirect, "Checkbox", ID_CHECKBOX_0, 530, 90, 120, 30, 0, 0x0, 0 },
    { CHECKBOX_CreateIndirect, "Checkbox", ID_CHECKBOX_1, 530, 130, 120, 30, 0, 0x0, 0 },
    { CHECKBOX_CreateIndirect, "Checkbox", ID_CHECKBOX_2, 530, 170, 120, 30, 0, 0x0, 0 },
    { CHECKBOX_CreateIndirect, "Checkbox", ID_CHECKBOX_3, 530, 210, 120, 30, 0, 0x0, 0 },
    { RADIO_CreateIndirect, "Radio", ID_RADIO_0, 650, 119, 80, 87, 0, 0x1404, 0 },
    { LISTVIEW_CreateIndirect, "Listview", ID_LISTVIEW_0, 20, 200, 300, 194, 0, 0x0, 0 },
    { DROPDOWN_CreateIndirect, "Dropdown", ID_DROPDOWN_0, 324, 22, 150, 100, 0, 0x0, 0 },
    { TEXT_CreateIndirect, "Text0", ID_TEXT_0, 327, 180, 150, 20, 0, 0x64, 0 },
    { TEXT_CreateIndirect, "Text1", ID_TEXT_1, 327, 200, 150, 20, 0, 0x64, 0 },
    { TEXT_CreateIndirect, "Text2", ID_TEXT_2, 327, 220, 200, 30, 0, 0x64, 0 },
    { TEXT_CreateIndirect, "Text3", ID_TEXT_3, 327, 250, 300, 50, 0, 0x64, 0 },
    { SCROLLBAR_CreateIndirect, "Scrollbar", ID_SCROLLBAR_0, 482, 310, 260, 40, 0, 0x0, 0 },
    { SLIDER_CreateIndirect, "Slider", ID_SLIDER_0, 513, 18, 259, 38, 0, 0x0, 0 },
};

/* 对话框窗口回调函数 */
static void _cbDialog(WM_MESSAGE *pMsg)
{
    WM_HWIN hItem;
    WM_HWIN hHeader;
    int NCode;
    int Id;
    int i;

    switch (pMsg->MsgId)
    {
        case WM_INIT_DIALOG:

            /* 初始化 FRAMEWIN */
            hItem = pMsg->hWin;
            FRAMEWIN_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
            FRAMEWIN_SetTitleHeight(hItem, 30);
            FRAMEWIN_SetFont(hItem, &XBFL6_Font);
            FRAMEWIN_SetTextColor(hItem, GUI_RED);
            FRAMEWIN_SetText(hItem, "ALIENTEK XBF 汉字显示");
    }
}

```

```
/* 初始化 MULTIEDIT */
hItem = WM_GetDialogItem(pMsg->hWin, ID_MULTIEDIT_0);
MULTIEDIT_SetTextColor(hItem, MULTIEDIT_CI_EDIT, GUI_BLACK);
MULTIEDIT_SetBkColor(hItem, MULTIEDIT_CI_EDIT, GUI_LIGHTCYAN);
MULTIEDIT_SetFont(hItem, &XBF16_Font);
MULTIEDIT_SetText(hItem, MultiEdit_String);

/* 初始化 BUTTON */
hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_0);
BUTTON_SetFont(hItem, &XBF16_Font);
BUTTON_SetText(hItem, "确定");
hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_1);
BUTTON_SetFont(hItem, &XBF16_Font);
BUTTON_SetText(hItem, "取消");

/* 初始化 CHECKBOX */
hItem = WM_GetDialogItem(pMsg->hWin, ID_CHECKBOX_0);
CHECKBOX_SetFont(hItem, &XBF12_Font);
CHECKBOX_SetText(hItem, "复选框 1");
hItem = WM_GetDialogItem(pMsg->hWin, ID_CHECKBOX_1);
CHECKBOX_SetFont(hItem, &XBF12_Font);
CHECKBOX_SetText(hItem, "复选框 2");
hItem = WM_GetDialogItem(pMsg->hWin, ID_CHECKBOX_2);
CHECKBOX_SetFont(hItem, &XBF12_Font);
CHECKBOX_SetText(hItem, "复选框 3");
hItem = WM_GetDialogItem(pMsg->hWin, ID_CHECKBOX_3);
CHECKBOX_SetFont(hItem, &XBF12_Font);
CHECKBOX_SetText(hItem, "复选框 4");

/* 初始化 RADIO */
hItem = WM_GetDialogItem(pMsg->hWin, ID_RADIO_0);
RADIO_SetFont(hItem, &XBF12_Font);
RADIO_SetText(hItem, "选项 1", 0);
RADIO_SetText(hItem, "选项 2", 1);
RADIO_SetText(hItem, "选项 3", 2);
RADIO_SetText(hItem, "选项 4", 3);

/* 初始化 LISTVIEW */
hItem = WM_GetDialogItem(pMsg->hWin, ID_LISTVIEW_0);
hHeader = LISTVIEW_GetHeader(hItem);
LISTVIEW_SetHeaderHeight(hItem, 30);
HEADER_SetFont(hHeader, &XBF12_Font);
```

```

LISTVIEW_AddColumn(hItem, 40, "名字", GUI_TA_HCENTER |
    GUI_TA_VCENTER);
LISTVIEW_AddColumn(hItem, 40, "学号", GUI_TA_HCENTER |
    GUI_TA_VCENTER);
LISTVIEW_AddColumn(hItem, 40, "成绩", GUI_TA_HCENTER |
    GUI_TA_VCENTER);
LISTVIEW_AddColumn(hItem, 40, "等级", GUI_TA_HCENTER |
    GUI_TA_VCENTER);
LISTVIEW_SetGridVis(hItem, 1);
LISTVIEW_SetAutoScrollH(hItem, 1);
LISTVIEW_SetAutoScrollV(hItem, 1);
LISTVIEW_SetRowHeight(hItem, 20);
LISTVIEW_SetFont(hItem, &XBFL6_Font);
LISTVIEW_SetBkColor(hItem, LISTVIEW_CI_UNSEL, GUI_ORANGE);

for (i = 0; i < GUI_COUNTOF(_ListViewTable); i++)
{
    LISTVIEW_AddRow(hItem, _ListViewTable[i]);
}

/* 初始化 DROPDOWN */
hItem = WM_GetDialogItem(pMsg->hWin, ID_DROPDOWN_0);
DROPDOWN_SetAutoScroll(hItem, 1); /* 开启自动滚动条 */
DROPDOWN_SetFont(hItem, &XBFL6_Font);
DROPDOWN_AddString(hItem, "汉语");
DROPDOWN_AddString(hItem, "英语");
DROPDOWN_AddString(hItem, "韩语");
DROPDOWN_AddString(hItem, "马来西亚语");
DROPDOWN_AddString(hItem, "西班牙语");
DROPDOWN_AddString(hItem, "其他语言.....");

/* 初始化 TEXT */
hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_0);
TEXT_SetFont(hItem, &XBFL2_Font);
TEXT_SetTextColor(hItem, GUI_YELLOW);
TEXT_SetText(hItem, "广州星翼电子");
hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_1);
TEXT_SetFont(hItem, &XBFL6_Font);
TEXT_SetTextColor(hItem, GUI_MAGENTA);
TEXT_SetText(hItem, "广州星翼电子");
hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_2);
TEXT_SetFont(hItem, &XBFL24_Font);
TEXT_SetTextColor(hItem, GUI_TRANSPARENT);
TEXT_SetText(hItem, "广州星翼电子");
hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_3);
    
```

```

        TEXT_SetFont(hItem, &XBF36_Font);
        TEXT_SetTextColor(hItem, GUI_BLUE);
        TEXT_SetText(hItem, "广州星翼电子");
        break;
    case WM_NOTIFY_PARENT:
        Id = WM_GetId(pMsg->hWinSrc);
        NCode = pMsg->Data.v;
        switch (Id)
        {
            case ID_MULTIEDIT_0:
                switch (NCode)
                {
                    case WM_NOTIFICATION_CLICKED:
                        break;
                    case WM_NOTIFICATION_RELEASED:
                        break;
                    case WM_NOTIFICATION_VALUE_CHANGED:
                        break;
                }
                break;
            case ID_BUTTON_0:
                switch (NCode)
                {
                    case WM_NOTIFICATION_RELEASED:
                        break;
                }
                break;
            case ID_BUTTON_1:
                switch (NCode)
                {
                    case WM_NOTIFICATION_RELEASED:
                        break;
                }
                break;
        }
        break;
    default:
        WM_DefaultProc(pMsg);
        break;
}

/**

```



```

* @brief      emwin_main(emwin 入口)
* @param      无
* @return     无
*/
void emwin_main (void)
{
    emwin_create_xbf12((uint8_t *)"0:/SYSTEM/EMWINFONT/XBF/XBF12.xbf");
    emwin_create_xbf16((uint8_t *)"0:/SYSTEM/EMWINFONT/XBF/XBF16.xbf");
    emwin_create_xbf24((uint8_t *)"0:/SYSTEM/EMWINFONT/XBF/XBF24.xbf");
    emwin_create_xbf36((uint8_t *)"0:/SYSTEM/EMWINFONT/XBF/XBF36.xbf");
    GUI_UC_SetEncodeUTF8();
    /* 更换皮肤 */
    BUTTON_SetDefaultSkin(BUTTON_SKIN_FLEX);
    CHECKBOX_SetDefaultSkin(CHECKBOX_SKIN_FLEX);
    DROPDOWN_SetDefaultSkin(DROPDOWN_SKIN_FLEX);
    FRAMEWIN_SetDefaultSkin(FRAMEWIN_SKIN_FLEX);
    HEADER_SetDefaultSkin(HEADER_SKIN_FLEX);
    MENU_SetDefaultSkin(MENU_SKIN_FLEX);
    MULTIPAGE_SetDefaultSkin(MULTIPAGE_SKIN_FLEX);
    PROGBAR_SetDefaultSkin(PROGBAR_SKIN_FLEX);
    RADIO_SetDefaultSkin(RADIO_SKIN_FLEX);
    SCROLLBAR_SetDefaultSkin(SCROLLBAR_SKIN_FLEX);
    SLIDER_SetDefaultSkin(SLIDER_SKIN_FLEX);
    SPINBOX_SetDefaultSkin(SPINBOX_SKIN_FLEX);
#ifdef GUI_SUPPORT_MEMDEV
    WM_SetCreateFlags(WM_CF_MEMDEV);
#endif
    hDialog = GUI_CreateDialogBox(_aDialogCreate, GUI_COUNTOF(_aDialogCreate),
                                  _cbDialog, WM_HBKWIN, 0, 0);

    GUI_Exec();
    while (1)
    {
        GUI_Delay(100);
    }
}

```

使用 XBF 格式字体的话需要使能 emWin 的 UTF8 编码格式，我们在 main.c 中的任务函数 user_task 中使能 emWin 的 UTF8 编码格式。

为了编译通过，我们需要对 emwin_demo.c 文件以 UTF-8 格式一下，下面我们分几个步骤来讲解如何对 emwin_demo.c 文件以 UTF-8 进行格式：

第一步：找到本实验的 emwin_demo.c 文件，在这个文件所在的路径下创建文本文档，如下图所示。



图 32.3.2.2.1 记事本打开文件

从上图可知：我们把 emwin_demo.c 文件的内容复制到文本文档当中。

第二步：在上图中点击：文件→另存为，出现如下图所示对话框，修改编码方式为带有 ROMd 的 UTF-8。

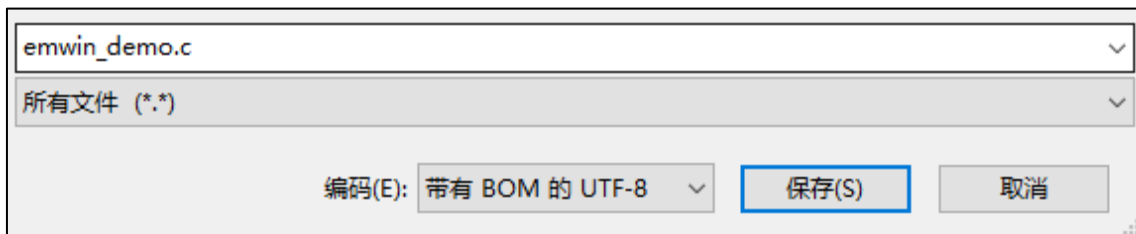


图 32.3.2.2.2 修改编码方式

第三步：修改完成以后点击保存就会替换到原来的文件，新文件的编码方式就改为了 UTF-8/带 ROM 的 UTF-8 了。

第二种方法是在 KEIL 中 Options for Target 'Flash' → C/C++ → Misc Controls 添加 “--locale=english”，注意这个方法适用于 AC5 编译器，而 AC6 编译器不适用。如下图所示：

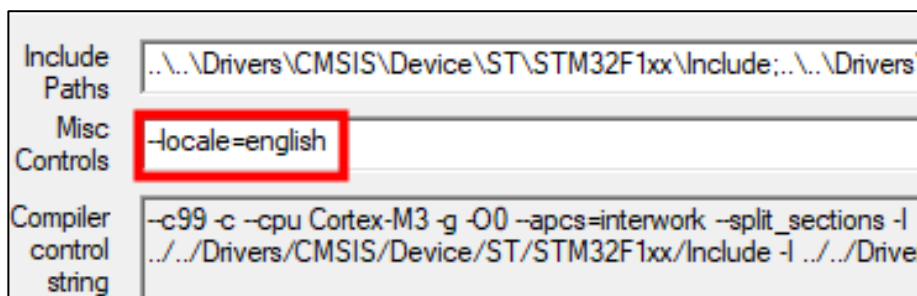


图 32.3.2.2.3 填写--locale=english

32.3.3 下载验证

编译工程并下载到开发板中，注意字库一定要放在 SD 卡中！。

注意：这里使用的是 4.3 寸 800*480 分辨率的屏幕，如果使用其他小于 800*480 的屏幕的话会导致显示不完全！

因为显示汉字的时候需要从 SD 卡中读取字库信息，所以看起来显示的很慢，这个也是 XBF 格式字体的缺点，如下图所示：

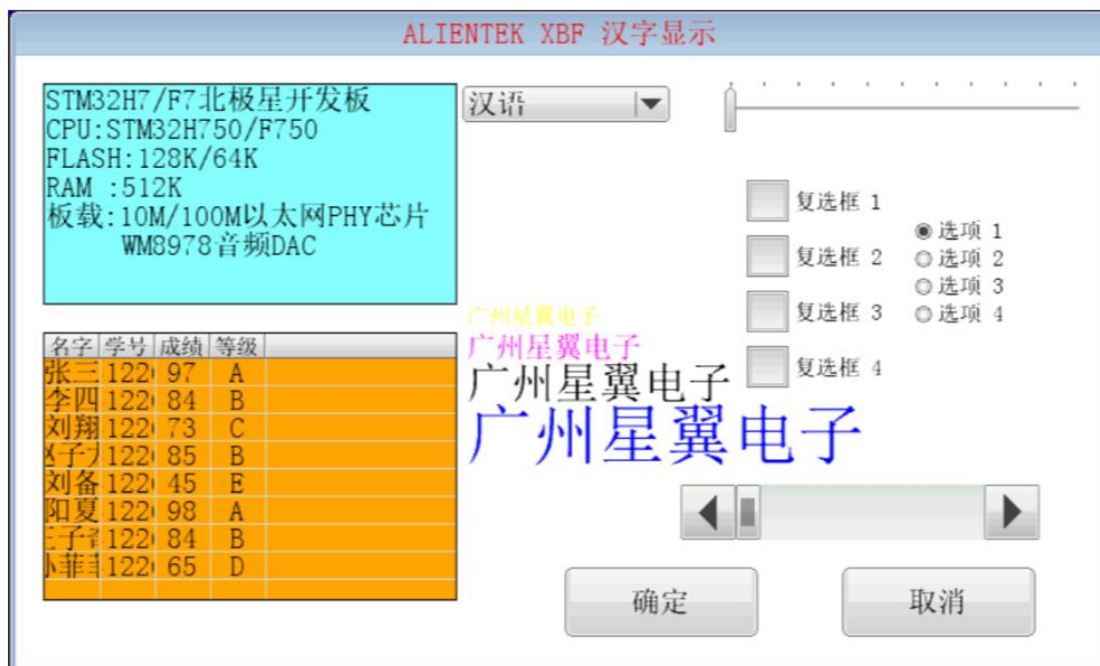


图 32.3.3.1 LCD 显示效果

第三十三章 TTF 格式字体显示

对于 emWin 的功能可以通过使用 TrueType 字体增强。TrueType 字体是 Apple Computer 开发的一种轮廓字体标准。它为字体开发人员提供了高度的控制权，使他们的字体在不同字体高度上显示出来。相反，位图字体是每个字符基于位图的，TrueType 字体是基于矢量图形。向量表示的优势是无损耗的可扩展性也就是矢量字体，emWin 是支持矢量字体的，本章我们就讲解一下如何使用矢量字体。

本章分为以下几个部分：

33.1 TTF 字体简介

33.2 TTF 字体相关 API 函数

33.3 TTF 字体显示实验

33.1 TTF 字体简介

我们平时使用 PDF 阅读文件的时候可以将文档方到很大，但是文档里面的内容确没有变的模糊，相反确很清晰，这是因为 PDF 使用的就是矢量字体，TTF 字体是由 Apple 和 Microsoft 两个公司推出的字体格式。

矢量字体的每个字符都是通过数学曲线描述的，在显示的时候通过读取指定字符的数学矢量并且对其进行一定的数学计算然后将其显示出来，正如前面所说的，这类字体的大小可以任意的缩放，改变颜色。

有三种主流的矢量字体：Type1，TrueType 和 OpenType，Type1 全称 PostScript Type1，是由 Adobe 公司提出的矢量字体标准，Type1 是不免费的，使用 Type1 的话需要向 Adobe 交付使用费。TrueType 是由 Apple 公司与 Microsoft 公司联合提出另一套矢量字标准，Type1 使用三次贝塞尔曲线来描述字形，TrueType 则使用二次贝塞尔曲线来描述字形，所以 TrueType 没有 Type1 的字体美观。但是实际中用的最广泛的是 TrueType，这是因为 Type1 字体方程的复杂，所以在屏幕上显示的时候花费的时间多，另一个原因是 Type1 需要支付高昂的使用费用。OpenType 则是 Type1 与 TrueType 之争的最终产物。是由 Adobe 和 Microsoft 合作开发出来的一种兼容 Type1 和 TrueType，并且真正支持 Unicode 的字体，OpenType 可以嵌入 Type1 和 TrueType，这样就兼有了二者的特点。

emWin 源码中默认不带 TTF 包的，需要我们自行去下载，下载地址为：https://www.segger.com/downloads/emwin/emWin_FreeType，注意这里下载下来的是 TTF 字体的渲染引擎，并不是 TTF 字库，TTF 字库需要我们自行下载，渲染引擎对 TTF 字库进行渲染并且显示在 LCD 上。

33.2 TTF 字体相关 API 函数

与 TTF 字体相关的函数有两个：GUI_TTF_CreateFont 和 GUI_TTF_DeleteFont 函数。下面我们分别地介绍这两个函数原型。

(1) GUI_TTF_CreateFont 函数

为 TTF 字体文件创建和选择 emWin 字体，该函数的原型如下源码所示：

```
int GUI_TTF_CreateFont(GUI_FONT * pFont,  
                      GUI_TTF_CS * pCS);
```

函数参数描述见下表所示：

参数	描述
pFont	指向 RAM 中由该函数填充的 GUI_FONT 结构
pCS	指向包含创建参数的 GUI_TTF_CS 结构

表 33.2.1 函数参数描述

返回值：0 表示成功，1 表示错误。

GUI_TTF_CS 为一个结构体，如下：

```
typedef struct
{
    GUI_TTF_DATA    * pTTF;
    U32              aImageTypeBuffer[4];
    int              PixelHeight;
    int              FaceIndex;
} GUI_TTF_CS;
```

GUI_TTF_CS 结构体中各个元素的含义如下表所示：

数据类型	元素	描述
GUI_TTF_DATA*	pTTF	指向包含要使用的字体文件位置和大小 的 GUI_TTF_DATA 结构
U32	aImageTypeBuffer[]	图片类型结构体缓冲区
int	PixelHeight	新字体的像素高度，表示字形 “g” 和 “f” 之 间周围矩形的高度。请注意，这并非两行文本 之间的距离。换句话说，GUI_GetFontSizeY 返 回的值与此值并不完全相同
int	FaceIndex	有些字体文件可能包含多种字体风格，对于多 种风格，此索引指定使用基于零的风格索引来 创建字体，通常为 0

表 33.2.2 GUI_TTF_CS 结构体元素

GUI_TTF_DATA 的元素如下表所示：

数据类型	元素	描述
const void*	pData	指向可寻址存储区域中的 TTF 字体文件
NumBytes	NumBytes	文件的大小（字节）

表 33.2.3 GUI_TTF_DATA 结构体元素

其他信息：

第一次使用该函数时，它会对 TTF 引擎和内部缓存系统进行初始化。如果缓存需要使用默认定义的其他值，则需要在首次调用此函数之前调用函数 GUI_TTF_SetCacheSize()来进行配置。内部数据缓存管理创建字体和缓存位图数据的整个机制，字体风格在缓存中按参数 pTTF 和 FaceIndex 中给定的地址进行唯一标识，通常为 0。例如，如果应使用相同的字体文件来创建不同大小的字体，则参数 pTTF 应指向 GUI_TTF_DATA 结构的相同位置。参数 PixelHeight 指定字形 “g” 和 “f” 之间周围矩形的高度。请注意 PixelHeight 的值与一行文本到另一行文本之间的偏移量不同。

(2) GUI_TTF_SetCacheSize 函数

设置第一次调用函数 GUI_TTF_CreateFont 时用于创建缓存的大小参数，该函数的原型如下源码所示：

```
void GUI_TTF_SetCacheSize( unsigned MaxFaces,
                           unsigned MaxSizes,
                           U32 MaxBytes);
```

函数参数描述如下表所示：

参数	描述
MaxFaces	缓存应能够同时处理的最大字体风格数，0 选择默认值
MaxSizes	缓存应能够同时处理的最大字体对象数，0 选择默认值
MaxBytes	位图缓存使用的最大字节数，0 选择默认值

表 33.2.4 函数参数描述表

返回值：无。

其他信息：

例如，如果应使用 3 种字体风格，每种有 2 种大小，则缓存应能够处理 6 种字体对象。TTF 引擎使用的默认值是：2 种风格、4 种大小对象和 200 K 位图数据缓存。

33.3 TTF 字体显示实验

33.3.1 硬件设计

1. 例程功能

本实验主要讲解 TTF 格式字体显示实验，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数读取 SD 卡上的 ttf 字体文件，然后 emWin 的文本根据字库来显示字体。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 33 TTF 格式汉字显示》。

注意：本实验不支持 Mini、DMF407、精英开发板运行。

33.3.2 软件设计

33.3.2.1 程序流程图

本实验的程序流程图，如下图所示：

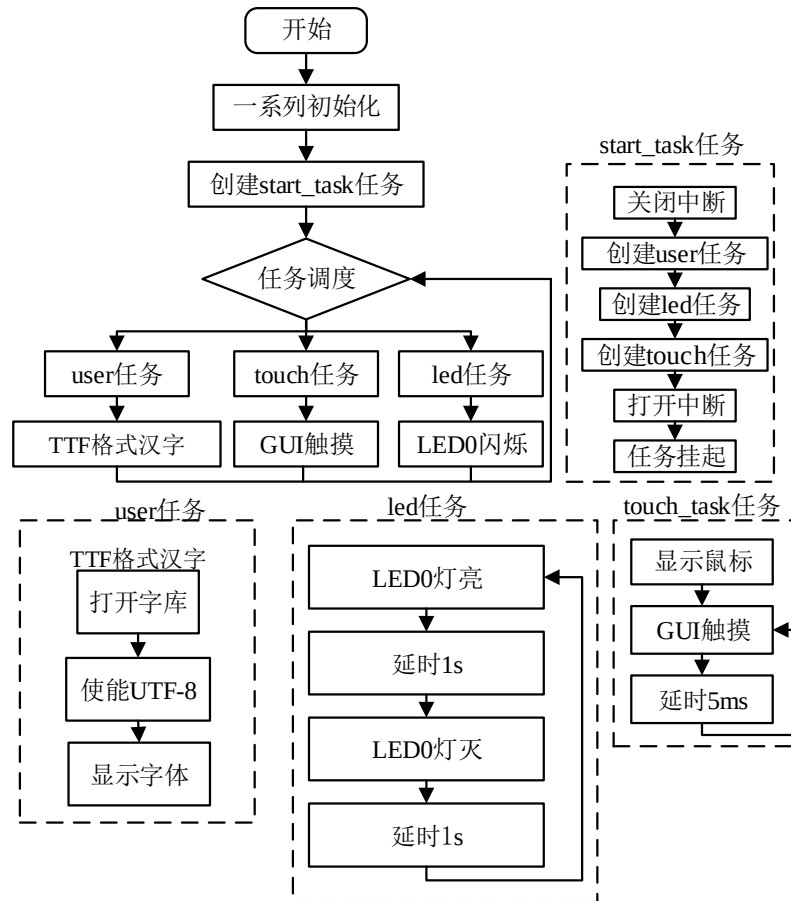


图 33.3.2.1 TTF 格式字体显示实验流程图

33.3.2.2 程序解析

(1) 移植 Freetype 库

将下载下来的 Freetype 库移植到工程中，Freetype 我们已经下载下来放到光盘中，路径：
6，软件资料→3，emWin 学习资料→emwin_freetype，因为我们使用的是 6.10 版本的 EMWIN，
所以这里 freetype 也使用 6.14 版本的，新建 Middlewares/emWin/TRUETYPE 分组，将 Freetype
文件添加进来，添加完成以后如下图所示：

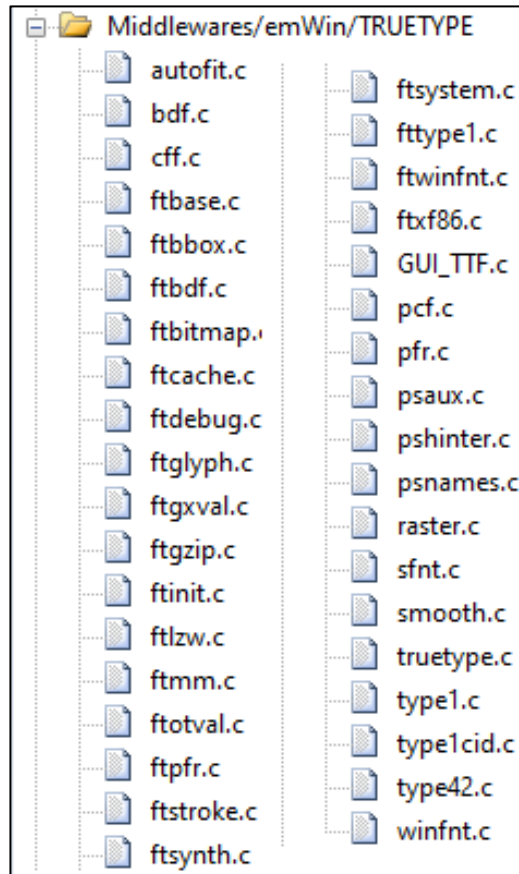


图 33.3.2.2.1 添加 Freetype 库

注意添加 Freetype 文件中相应的头文件路径，如下图所示：

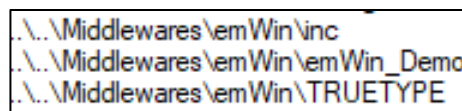


图 33.4.3.2 添加头文件路径

(2) Freetype 内存管理

Freetype 需要使用到 malloc 和 free 函数来申请和释放内存，这里我们需要修改一下 ftstdlib.h 文件，让 Freetype 所使用的内存管理函数映射到正点原子的内存管理函数上，打开 ftstdlib.h 文件，按照下图所示修改：

```
#define ttf_malloc(size) ..... mymalloc(SRAMEX, size)
#define ttf_free(block) ..... myfree(SRAMEX, block)
#define ttf_realloc(block, size) myrealloc(SRAMEX, block, size)

#define ft_scalloc ..... calloc
#define ft_sfree ..... ttf_free
#define ft_smalloc ..... ttf_malloc
#define ft_srealloc ..... ttf_realloc
```

图 33.4.3.3 修改内存管理函数

我们也可以采用另外一种方法，只适合 F4 以上的开发板，F1 只能使用上面的方式，那就是不需要对 ftstdlib.h 做任何修改，让其使用 KEIL 自带的 malloc 函数和 free 函数，只需要将 startup_stm32fxxx.s 中堆改大一点就可以了，如下图所示，将 Heap_Size 改为 0X0000F000。

52	Heap_Size	EQU	0x0000F000
----	-----------	-----	------------

图 33.4.3.4 更改堆大小

(3) 准备 TTF 字库

前面下载的 FreeType 是 TTF 渲染文件，这里我们还需要下载 TTF 字库，我们已经整理了一些.ttf 格式的字库放在光盘中，路径：3、emWin 学习资料→TTF 字库。大家也可以自行下载一些 TTF 字库。由于开发板 RAM 限制，即使是使用外部 SRAM 最大也只有 1M，因此只能使用一些英文和数字的 TTF 字库，不能使用中文 TTF 字库！。

将下载下来的 TTF 字库拷贝到 SD 卡中，在使用的时候需要将字库从 SD 卡读取到 RAM 中，SD 卡的保存路径为：SYSTEM→EMWINFONT→TTF，如下图所示：

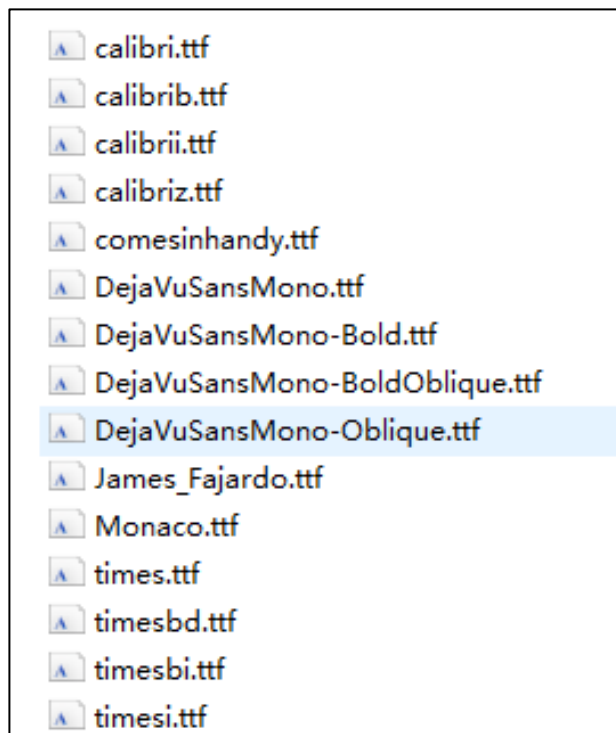


图 33.4.3.5 保存 TTF 字库到 SD 卡中

注意：如果发现有两个错误是指向 GUI_TTF.c 中的 GUI_TTF_DestroyCache 函数和 GUI_TTF_Done 函数的，请把这两个函数的形参 void 去掉即可。

经过上面几步，工程已经修改的差不多了，现在就开始编写代码了，本实验完整工程为：**emWin 实验 33 TTF 格式字体显示**。打开文件 emwin_demo.c，程序如下：

```
/**
 * @brief      创建 TTF 字体，共 EMWIN 使用
 * @param      fxpath:存放 TTF 字库的路径
 * @retval     0,成功；其他值，失败
 */
int Create_TTFFont(uint8_t *fxpath)
{
    CPU_SR_ALLOC();
    int result;
    uint16_t bread;
```

```

char *TtfBuffer;
/* 打开库文件 */
result = f_open(&TTFFontFile, (const TCHAR *)fxpath, FA_READ);
if (result != FR_OK) return 1;
TtfBuffer = mymalloc(SRAMEX, TTFFontFile.obj.objsize); /* 申请内存 */
if (TtfBuffer == NULL)
{
    printf("内存申请失败\r\n");
    return 3; /* 内存申请失败 */
}

/* 读取字体数据 */
#if SYS_SUPPORT_OS
    OS_CRITICAL_ENTER(); /* 进入临界区 */
#endif

result = f_read(&TTFFontFile, TtfBuffer, TTFFontFile.obj.objsize,
                (UINT *)&bread); /* 读取数据 */

if (result != FR_OK)
{
    return 4; /* 文件打开失败, 跳出 */
}

f_close(&TTFFontFile); /* 关闭 TTFFontFile 文件*/
#if SYS_SUPPORT_OS
    OS_CRITICAL_EXIT(); /* 退出临界区 */
#endif

TTFData.pData = TtfBuffer; /* 指向文件地址 */
TTFData.NumBytes = TTFFontFile.obj.objsize; /* 文件大小*/

Cs0.pTTF = &TTFData;
Cs0.PixelHeight = 12;
Cs0.FaceIndex = 0;

Cs1.pTTF = &TTFData;
Cs1.PixelHeight = 18;
Cs1.FaceIndex = 0;

Cs2.pTTF = &TTFData;
Cs2.PixelHeight = 24;
Cs2.FaceIndex = 0;
    
```

```

Cs3.pTTF          = &TTFFData;
Cs3.PixelHeight   = 36;
Cs3.FaceIndex     = 0;

result = GUI_TTF_CreateFont(&TTF12_Font, &Cs0); /* 创建字体 */

if (result) return 5; /* 字体创建失败*/

result = GUI_TTF_CreateFont(&TTF18_Font, &Cs1); /* 创建字体 */

if (result) return 5; /* 字体创建失败*/

result = GUI_TTF_CreateFont(&TTF24_Font, &Cs2); /* 创建字体*/

if (result) return 5; /* 字体创建失败 */

result = GUI_TTF_CreateFont(&TTF36_Font, &Cs3); /* 创建字体*/

if (result) return 5; /* 字体创建失败 */

return 0;
}

```

在函数 `emwin_main` 中我们将调用函数 `Create_TTFFont` 创建并使用 TTF 字库，程序入门函数如下所示：

```

/**
 * @brief      emwin_main(emwin 入口)
 * @param      无
 * @return     无
 */
void emwin_main(void)
{
    int result;
    result = Create_TTFFont((uint8_t *)
        "0:/SYSTEM/EMWINFONT/TTF/comesinhandy.ttf");

    if(result)
    {
        printf("TTF 字体创建失败,%d\r\n", result);
    }

    GUI_CURSOR_Show();
    GUI_SetBkColor(GUI_CYAN);
}

```

```
GUI_SetColor(GUI_RED);
GUI_UC_SetEncodeUTF8();
GUI_Clear();
GUI_SetFont(&TTF12_Font);
GUI_DispStringAt("ALIENTEK STemWin TTF Font Test 0123456789!",5,10);
GUI_SetFont(&TTF18_Font);
GUI_DispStringAt("ALIENTEK STemWin TTF Font Test 0123456789!",5,25);
GUI_SetFont(&TTF24_Font);
GUI_DispStringAt("ALIENTEK STemWin TTF Font Test 0123456789!",5,45);
GUI_SetFont(&TTF36_Font);
GUI_DispStringAt("ALIENTEK STemWin TTF Font Test 0123456789!",5,70);
}
```

33.3.3 下载验证

编译工程并下载到开发板中，注意 TTF 字库一定要放在 SD 卡中指定位置！下载完成以后 LCD 显示如下图所示，TTF 字体显示正常。

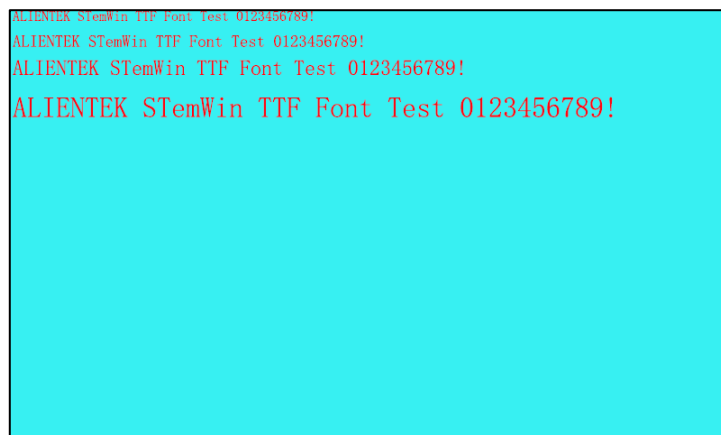


图 33.3.3.1 LCD 显示效果图

注意：只能加载小于 500k 的 TTF 字库，汉字 TTF 字库一般都比较大（大于 2M），所以只能显示一些字母、数字和字符。

第三十四章 SIF 格式字体显示

前面章节中，我们讲了 emWin 的 XBF 格式汉字的显示，由于要从 SD 卡读取字库信息，所以使用起来效果很差，速度很慢，很卡。那么有没有其他方式让字符的显示速度快一点，肯定有的！那就是本章要讲的 SIF 格式字体，SIF 也是 emWin 官方推出的另一种字体。SIF 也叫做系统独立字体格式，SIF 就是为了解决 XBF 显示速度慢的问题而推出的！但是 SIF 是需要大量的 RAM 的，因为 SIF 是将整个字库加载到 RAM 中，而 XBF 的字库是在 SD 卡中的，所以 SIF 显示肯定要比 XBF 快，但是你的板子得有很大的 RAM。本章我们就来讲解 SIF 格式字体的显示。

本章分为以下几个部分：

34.1 SIF 格式字库制作

34.2 SIF 格式字体相关 API 函数

34.3 SIF 格式字体实验

34.1 SIF 格式字库制作

在使用 SIF 格式字体之前肯定要先制作好 SIF 格式字体的字库，同样 XBF 格式字体显示一样，使用工具字体转换器来完成字库的制作。下面我们分为几个步骤来讲解 SIF 格式字库制作。

第一步：打开软件 File→New，选择“Antialiased 2bpp”，如下图所示：

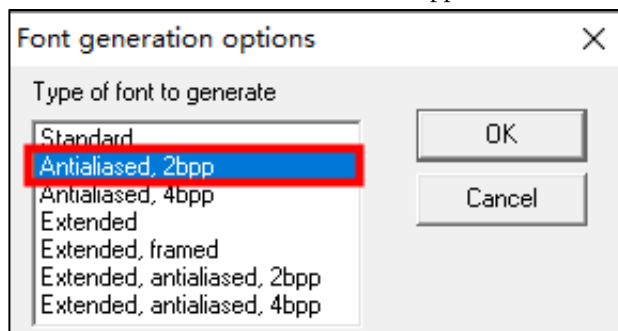


图 34.1.1 创建 SIF 字库

第二步：选择要制作的字库的字体，字形和大小，如下图所示：

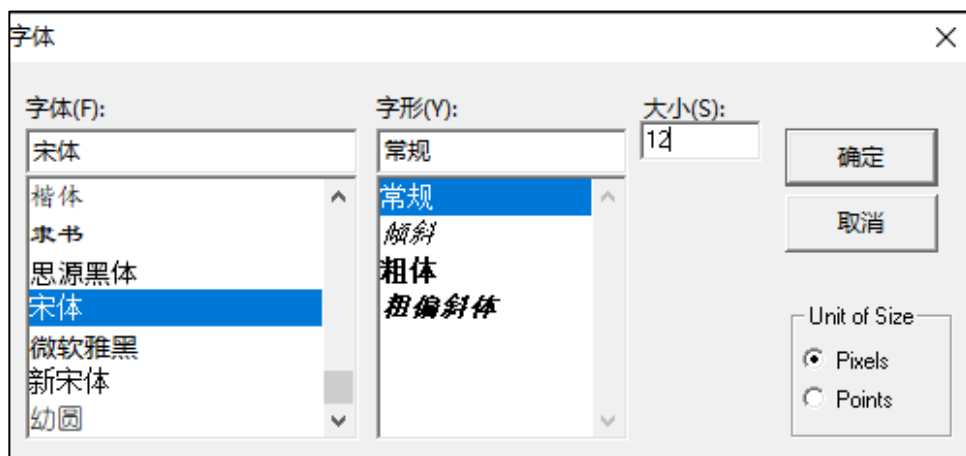


图 34.1.2 选择要制作的字库信息

第三步：设置好要制作的字库信息，可以保存生成.sif 结尾的字库文件了，如下图所示：

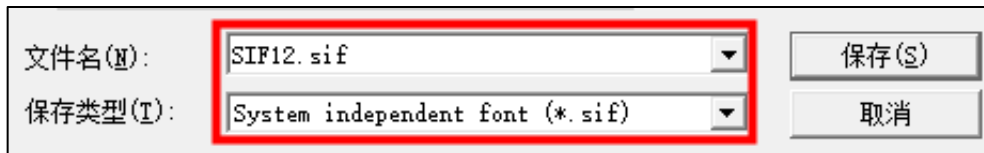


图 33.1.3 保存字库

第四步：等待字库制作完成，然后同样的方式接着制作 16，24，36 号的字库。

34.2 SIF 格式字体相关 API 函数

1. GUI_SIF_CreateFont 函数

创建 SIF 格式的字体，该函数的原型如下源码所示：

```
void GUI_SIF_CreateFont(void*pFontData,
                        GUI_FONT*pFont,
                        const GUI_SIF_TYPE*pFontType);
```

函数参数描述见下表所示。

参数	描述
pFontData	指向系统独立字体数据
pFont	指向 RAM 中由该函数填充的 GUI_FONT 结构
pFontType	可选值
	GUI_SIF_TYPE_PROP 比例字体时使用
	GUI_SIF_TYPE_PROP_EXT 扩展比例字体时使用
	GUI_SIF_TYPE_PROP_FRM 扩展带边框比例字体时使用
	GUI_SIF_TYPE_PROP_AA2 2 bpp 抗锯齿的比例字体时使用
	GUI_SIF_TYPE_PROP_AA4 4 bpp 抗锯齿的比例字体时使用
	GUI_SIF_TYPE_PROP_AA2_EXT 2 bpp 抗锯齿的扩展比例字体时使用
	GUI_SIF_TYPE_PROP_AA4_EXT 4 bpp 抗锯齿的扩展比例字体时使用

表 34.2.1 GUI_SIF_CreateFont 函数参数描述

返回值：无。

2. GUI_SIF_DeleteFont 函数

删除指定 SIF 的字体，该函数的原型如下源码所示：

```
void GUI_SIF_DeleteFont(GUI_FONT * pFont)
```

函数参数描述见下表所示。

参数	描述
pFont	删除的字体

表 34.2.2 GUI_SIF_DeleteFont 函数参数描述

34.3 SIF 格式字体实验

34.3.1 硬件设计

1. 例程功能

本实验主要讲解 SIF 格式汉字显示实验，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数读取 SD 卡上的 sif 字库文件，然后对话框的文本根据字库来显示字体。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 34 SIF 格式汉字显示》。

注意：本实验不支持 Mini、精英、战舰、DMF407、探索者开发板运行。

34.3.2 软件设计

34.3.2.1 程序流程图

本实验的程序流程图，如下图所示：

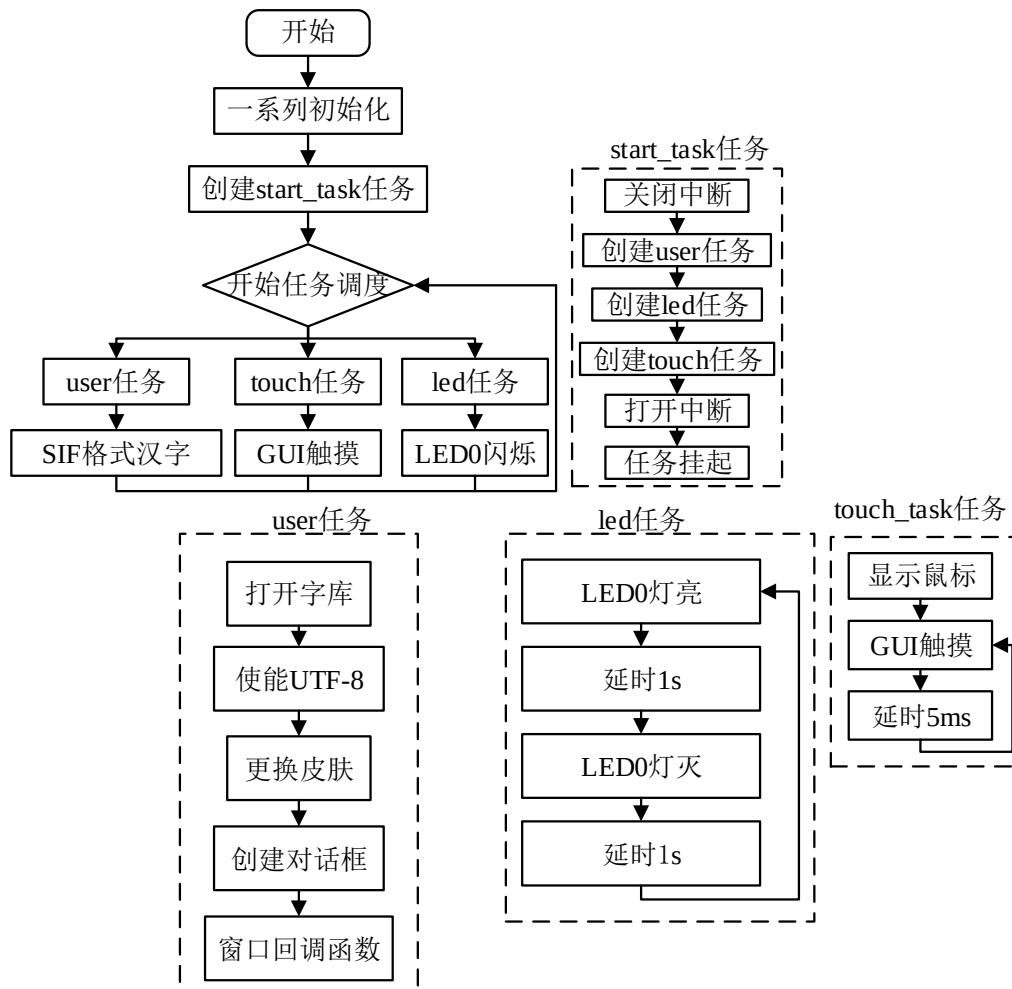


图 34.3.2.1.1 SIF 格式汉字显示实验流程图

34.3.2.2 程序解析

打开本章对应实验工程中的 emwin_demo.c 文件，程序源码如下：

(1) SD 卡获取字库文件

```

/**
 * @brief      SD 卡获取字库文件
 * @param      fxpath:字库路径

```

```

* @param      files:文件
* @retval      返回值:0, 失败; 其他, 得到的字库文件
*/
uint8_t *emwin_get_font_info(uint8_t *fxpath, FIL files)
{
    int result;
    uint16_t bread;
    uint8_t *FontBuffer;
    #if SYSTEM_SUPPORT_UCOS
        OS_CPU_SR cpu_sr;
    #endif

    result = f_open(&files, (const TCHAR*) fxpath, FA_READ);    /* 打开字库文件*/
    FontBuffer = mymalloc(SRAMEX, files.obj.objsize); /* 从外部 SDRAM 申请内存*/

    if(FontBuffer == NULL)
    {
        printf("内存申请失败\r\n");
        FontBuffer = NULL;    /* 内存申请失败*/
    }

    /* 读取字体数 */
    #if SYSTEM_SUPPORT_UCOS
        OS_ENTER_CRITICAL();    /* 临界区*/
    #endif
    /* 读取数据*/
    result = f_read(&files, FontBuffer, files.obj.objsize, (UINT *) &bread);
    if(result != FR_OK) FontBuffer = NULL;
    #if SYSTEM_SUPPORT_UCOS
        OS_EXIT_CRITICAL();    /* 退出临界区*/
    #endif
    return FontBuffer;
}

```

调用 `f_open()` 打开 SIF 字库文件，申请内存主要为了保存 SIF 格式字库，前面说了 SIF 格式的字库是加载到 RAM 中，因此我们需要申请一个很大的 RAM 来保存字库。因为我们的开发板有一个 32M 的 SDRAM，所以这里从 SDRAM 中申请，然后调用 `f_read` 函数读取字库数据并保存缓冲区中。

(2) 创建 SIF 格式字体

```

/**
* @brief      创建 SIF12 字体
* @param      fxpath:字库路径
* @retval      无
*/
void Create_SIF12(uint8_t *fxpath)

```



```

{
    SIF12FontBuff = emwin_get_font_info(fxpath, SIF12FontFile);

    if(SIF12FontBuff != NULL)
    {
        /* 创建 SIF 格式字体 */
        GUI_SIF_CreateFont(SIF12FontBuff, &SIF12_Font, GUI_SIF_TYPE_PROP_AA2);
    }
}

/**
 * @brief      创建 SIF16 字体
 * @param      fxpath: 字库路径
 * @retval     无
 */
void Create_SIF16(uint8_t *fxpath)
{
    SIF16FontBuff = emwin_get_font_info(fxpath, SIF16FontFile);

    if(SIF16FontBuff != NULL)
    {
        /* 创建 SIF 格式字体 */
        GUI_SIF_CreateFont(SIF16FontBuff, &SIF16_Font, GUI_SIF_TYPE_PROP_AA2);
    }
}

/**
 * @brief      创建 SIF24 字体
 * @param      fxpath: 字库路径
 * @retval     无
 */
void Create_SIF24(uint8_t *fxpath)
{
    SIF24FontBuff = emwin_get_font_info(fxpath, SIF24FontFile);

    if(SIF24FontBuff != NULL)
    {
        /* 创建 SIF 格式字体 */
        GUI_SIF_CreateFont(SIF24FontBuff, &SIF24_Font, GUI_SIF_TYPE_PROP_AA2);
    }
}

/**

```

```

* @brief      创建 SIF36 字体
* @param      fxpath: 字库路径
* @retval     无
*/
void Create_SIF36(uint8_t *fxpath)
{
    SIF36FontBuff = emwin_get_font_info(fxpath, SIF36FontFile);

    if(SIF36FontBuff != NULL)
    {
        /* 创建 SIF 格式字体 */
        GUI_SIF_CreateFont(SIF36FontBuff, &SIF36_Font, GUI_SIF_TYPE_PROP_AA2);
    }
}

```

调用 GUI_SIF_CreateFont() 创建 SIF 格式字体 12、16、24 以及 36 号字体。

(3) 创建对话框

演示例程就使用上一章 XBF 格式字体的演示例程，只是将其中使用的 XBF 格式字体更换为 SIF 格式的，部分代码如下：

```

/* 对话框窗口回调函数 */
static void _cbDialog(WM_MESSAGE *pMsg)
{
    WM_HWIN hItem;
    WM_HWIN hHeader;
    int NCode;
    int Id;
    int i;

    switch (pMsg->MsgId)
    {
        case WM_INIT_DIALOG:

            /* 初始化 FRAMEWIN */
            hItem = pMsg->hWin;
            FRAMEWIN_SetTextAlign(hItem, GUI_TA_HCENTER | GUI_TA_VCENTER);
            FRAMEWIN_SetTitleHeight(hItem, 30);
            FRAMEWIN_SetFont(hItem, &SIF16_Font);
            FRAMEWIN_SetTextColor(hItem, GUI_RED);
            FRAMEWIN_SetText(hItem, "ALIENTEK XBF 汉字显示");

            /* 初始化 MULTIEDIT */
            hItem = WM_GetDialogItem(pMsg->hWin, ID_MULTIEDIT_0);
            MULTIEDIT_SetTextColor(hItem, MULTIEDIT_CI_EDIT, GUI_BLACK);
            MULTIEDIT_SetBkColor(hItem, MULTIEDIT_CI_EDIT, GUI_LIGHTCYAN);
    }
}

```

```
MULTIEDIT_SetFont(hItem, &SIF16_Font);
MULTIEDIT_SetText(hItem, MultiEdit_String);

/* 初始化 BUTTON */
hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_0);
BUTTON_SetFont(hItem, &SIF16_Font);
BUTTON_SetText(hItem, "确定");
hItem = WM_GetDialogItem(pMsg->hWin, ID_BUTTON_1);
BUTTON_SetFont(hItem, &SIF16_Font);
BUTTON_SetText(hItem, "取消");

/* 初始化 CHECKBOX */
hItem = WM_GetDialogItem(pMsg->hWin, ID_CHECKBOX_0);
CHECKBOX_SetFont(hItem, &SIF12_Font);
CHECKBOX_SetText(hItem, "复选框 1");
hItem = WM_GetDialogItem(pMsg->hWin, ID_CHECKBOX_1);
CHECKBOX_SetFont(hItem, &SIF12_Font);
CHECKBOX_SetText(hItem, "复选框 2");
hItem = WM_GetDialogItem(pMsg->hWin, ID_CHECKBOX_2);
CHECKBOX_SetFont(hItem, &SIF12_Font);
CHECKBOX_SetText(hItem, "复选框 3");
hItem = WM_GetDialogItem(pMsg->hWin, ID_CHECKBOX_3);
CHECKBOX_SetFont(hItem, &SIF12_Font);
CHECKBOX_SetText(hItem, "复选框 4");

/* 初始化 RADIO */
hItem = WM_GetDialogItem(pMsg->hWin, ID_RADIO_0);
RADIO_SetFont(hItem, &SIF12_Font);
RADIO_SetText(hItem, "选项 1", 0);
RADIO_SetText(hItem, "选项 2", 1);
RADIO_SetText(hItem, "选项 3", 2);
RADIO_SetText(hItem, "选项 4", 3);

/* 初始化 LISTVIEW */
hItem = WM_GetDialogItem(pMsg->hWin, ID_LISTVIEW_0);
hHeader = LISTVIEW_GetHeader(hItem);
LISTVIEW_SetHeaderHeight(hItem, 30);
HEADER_SetFont(hHeader, &SIF12_Font);
LISTVIEW_AddColumn(hItem, 40, "名字", GUI_TA_HCENTER |
                    GUI_TA_VCENTER);
LISTVIEW_AddColumn(hItem, 40, "学号", GUI_TA_HCENTER |
                    GUI_TA_VCENTER);
LISTVIEW_AddColumn(hItem, 40, "成绩", GUI_TA_HCENTER |
```

```

        GUI_TA_VCENTER);

    LISTVIEW_AddColumn(hItem, 40, "等级", GUI_TA_HCENTER |
        GUI_TA_VCENTER);

    LISTVIEW_SetGridVis(hItem, 1);
    LISTVIEW_SetAutoScrollH(hItem, 1);
    LISTVIEW_SetAutoScrollV(hItem, 1);
    LISTVIEW_SetRowHeight(hItem, 20);
    LISTVIEW_SetFont(hItem, &SIF16_Font);
    LISTVIEW_SetBkColor(hItem, LISTVIEW_CI_UNSEL, GUI_ORANGE);

    for (i = 0; i < GUI_COUNTOF(_ListViewTable); i++)
    {
        LISTVIEW_AddRow(hItem, _ListViewTable[i]);
    }

    /* 初始化 DROPDOWN */
    hItem = WM_GetDialogItem(pMsg->hWin, ID_DROPDOWN_0);
    DROPDOWN_SetAutoScroll(hItem, 1); /* 开启自动滚动条 */
    DROPDOWN_SetFont(hItem, &SIF16_Font);
    DROPDOWN_AddString(hItem, "汉语");
    DROPDOWN_AddString(hItem, "英语");
    DROPDOWN_AddString(hItem, "韩语");
    DROPDOWN_AddString(hItem, "马来西亚语");
    DROPDOWN_AddString(hItem, "西班牙语");
    DROPDOWN_AddString(hItem, "其他语言.....");

    /* 初始化 TEXT */
    hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_0);
    TEXT_SetFont(hItem, &SIF12_Font);
    TEXT_SetTextColor(hItem, GUI_YELLOW);
    TEXT_SetText(hItem, "广州星翼电子");
    hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_1);
    TEXT_SetFont(hItem, &SIF16_Font);
    TEXT_SetTextColor(hItem, GUI_MAGENTA);
    TEXT_SetText(hItem, "广州星翼电子");
    hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_2);
    TEXT_SetFont(hItem, &SIF24_Font);
    TEXT_SetTextColor(hItem, GUI_TRANSPARENT);
    TEXT_SetText(hItem, "广州星翼电子");
    hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_3);
    TEXT_SetFont(hItem, &SIF36_Font);
    TEXT_SetTextColor(hItem, GUI_BLUE);
    TEXT_SetText(hItem, "广州星翼电子");
    break;

case WM_NOTIFY_PARENT:

```

```
Id = WM_GetId(pMsg->hWinSrc);
NCode = pMsg->Data.v;

switch (Id)
{
    case ID_MULTIEDIT_0:
        switch (NCode)
        {
            case WM_NOTIFICATION_CLICKED:
                break;

            case WM_NOTIFICATION_RELEASED:
                break;

            case WM_NOTIFICATION_VALUE_CHANGED:
                break;
        }
        break;

    case ID_BUTTON_0:
        switch (NCode)
        {
            case WM_NOTIFICATION_RELEASED:
                break;
        }
        break;

    case ID_BUTTON_1:
        switch (NCode)
        {
            case WM_NOTIFICATION_RELEASED:
                break;
        }
        break;

    default:
        WM_DefaultProc(pMsg);
        break;
}
}
```

上述函数就是对话框回调函数，在初始化对话框中的小工具的时候设置所使用的字体为 SIF 格式，其他的就都和上一章例程相同的。

(4) 启用 UTF8 编码

```
/**
 * @brief      emwin_main(emwin 入口)
 * @param      无
 * @return     无
 */
void emwin_main (void)
{
    Create_SIF12("0:/SYSTEM/EMWINFONT/SIF/SIF12.sif");
    Create_SIF16("0:/SYSTEM/EMWINFONT/SIF/SIF16.sif");
    Create_SIF24("0:/SYSTEM/EMWINFONT/SIF/SIF24.sif");
    Create_SIF36("0:/SYSTEM/EMWINFONT/SIF/SIF36.sif");
    GUI_UC_SetEncodeUTF8();

    /* 更换皮肤 */
    BUTTON_SetDefaultSkin(BUTTON_SKIN_FLEX);
    CHECKBOX_SetDefaultSkin(CHECKBOX_SKIN_FLEX);
    DROPDOWN_SetDefaultSkin(DROPDOWN_SKIN_FLEX);
    FRAMEWIN_SetDefaultSkin(FRAMEWIN_SKIN_FLEX);
    HEADER_SetDefaultSkin(HEADER_SKIN_FLEX);
    MENU_SetDefaultSkin(MENU_SKIN_FLEX);
    MULTIPAGE_SetDefaultSkin(MULTIPAGE_SKIN_FLEX);
    PROGBAR_SetDefaultSkin(PROGBAR_SKIN_FLEX);
    RADIO_SetDefaultSkin(RADIO_SKIN_FLEX);
    SCROLLBAR_SetDefaultSkin(SCROLLBAR_SKIN_FLEX);
    SLIDER_SetDefaultSkin(SLIDER_SKIN_FLEX);
    SPINBOX_SetDefaultSkin(SPINBOX_SKIN_FLEX);

#ifdef GUI_SUPPORT_MEMDEV
    WM_SetCreateFlags(WM_CF_MEMDEV);
#endif

    hDialog = GUI_CreateDialogBox(_aDialogCreate, GUI_COUNTOF(_aDialogCreate),
                                   _cbDialog, WM_HBKWIN, 0, 0);

    GUI_Exec();

    while (1)
    {
        GUI_Delay(100);
    }
}
```

和使用 XBF 格式字体一样，SIF 格式字体也需要使能 emWin 的 UTF8 编码格式，使能 emWin 的 UTF8 编码格式

为了编译通过，我们需要将 emwin_demo.c 文件 UTF-8 格式一下。修改完成以后点击保存就会替换到原来的文件，新文件的编码方式就改为了 UTF-8/带 ROM 的 UTF-8 了。

第二种方法是在 KEIL 中 Options for Target 'Flash' → C/C++ → Misc Controls 添加 “--locale=english”，注意这个方法适用于 AC5 编译器，而 AC6 不适用。如下图所示：

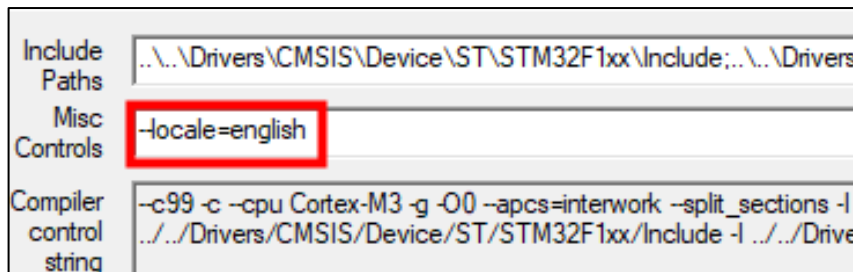


图 34.3.2.2.1 填写--locale=english

34.3.3 下载验证

编译工程并下载到开发板中，注意字库一定要放在 SD 卡中！下载完成以后 LCD 显示如下图所示，各个控件中汉字显示正常。



图 34.3.3.1 LCD 显示效果

注意：LCD 上电以后会黑屏几秒钟，这是因为要把 SD 卡中的 SIF 格式字库加载到 RAM 中，所以需要一点时间。有时候可能会无法显示汉字，串口提示“内存申请失败”，这是因为 SIF 格式比较大，导致可以使用的 SDRAM 不够，我们可以打开文件看一下 4 个 SIF 格式字库大小为 16.8M 字节，如下图所示。



图 34.3.3.2 文件占用大小

从图 34.3.3.2 可以看出，SDRAM 可用内存至少要有 17M 左右，如果不够的话肯定会在加载字库的时候出错，例程中 malloc 可分配的 SDRAM 内存设置为 20M，而给分给 emWin 的就有 8M，剩余的只有 20-8=12M，肯定不够用！这里修改文件 GUIConf.c，使其分配给 emWin 的内存为 2M。

第三十五章 汉字显示（外部存储器）

前面讲了 XBF 字体和 TTF 字体的使用，XBF 在显示汉字的时候需要不断的从存储器中读取字库所以显示比较慢；TTF 受限于开发板 RAM 大小，不能加载汉字 TTF 字库，因此只能显示一些字母或者数字和字符，而 Segger 官方推荐的另外一种字体文件——SIF，更是因为生成的字库信息太大而不能使用。这里我们讲解一个显示速度快而且也不会占用大量 ram 的汉字显示方法，那就是将字库放在外部存储器中，比如外部 SPI Flash 等，这个也是我们常用的显示汉字的方法。

本章分为以下几个部分：

35.1 汉字显示简介

35.2 实验工程准备

35.3 字库制作

35.4 汉字显示实验

35.1 汉字显示简介

在正点原子裸机教程中有讲解如何在外部 SPI FLASH 中制作汉字字库，并且在 LCD 上显示出来，本章我们就讲解一下如何在 emWin 中使用同样的方法显示汉字，至于这种方法的原理性知识本章就不讲解了，请参考正点原子裸机教程汉字显示实验章节，本章默认大家已经掌握了在不使用 emWin 时在 LCD 上显示汉字的方法。

35.2 实验工程准备

本实验要将基础实验中的汉字显示部分的文件和代码移植过来，这个很简单，就不详细讲解了，移植完成以后工程如下图所示：

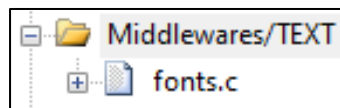


图 35.2.1 准备工程

35.3 字库制作

35.3.1 汉字字库制作

我们制作三个汉字字库，字体都为宋体，字库点阵大小为 12*12、16*16 和 24*24，使用易木雨软件工作室设计的点阵字库生成器 V3.8 来制作字库，软件设置如图 35.3.1.1 所示，注意这里字库取模的方式选择的是“横向取模方式一”，在裸机教程中是按照“纵向取模方式二”的，一定要注意这个差别，要不然会导致汉字显示出现异常！



图 35.3.1.1 汉字字库制作

这里字体大小和汉字点阵大小公式为：

$$fsize = dsize * 6 / 8$$

其中 fsize 是字体大小，dsize 是点阵字高和字宽(字高和字宽一样)，也就是字宽和字高。通过计算可以得到，12*12 点阵对应着 9 号字体，16*16 点阵对应着 12 号字体，24*24 点阵对应 18 号字体，字库点阵大小和字体大小对应关系一定要设置好，否则制作出的字库将会显示不正常！

字库制作完成以后放到 SD 卡中，路径为：SYSTEM→FONT，如图 35.3.1.2 所示：

名称	修改日期	类型
GBK12.FON	2014/12/30 17:38	字体文件
GBK16.FON	2014/12/30 17:10	字体文件
GBK24.FON	2014/12/30 22:54	字体文件
UNIGBK.BIN	2014/8/23 12:14	BIN 文件

图 35.3.1.2 字库

由于基础例程中的汉字显示和 emWin 中汉字显示的字符取模方式不同，因此一旦在 SD 卡中放入为 emWin 制作的字库并且更新以后会导致基础例程和综合实验中的汉字显示出现问题，这个时候需要重新将字库更新为基础例程中字库。

35.3.2 英文字符字库制作

由于 emWin 的取模方式和正点原子裸机教程中显示字符时的取模方式不同，所以英文、数字和符号的字模也需要重新制作一遍，使用 PCtoLCD2002 软件制作英文字模。对应的我们需要制作点阵大小为 12*12、16*16 和 24*24，但是我们制作的是英文、数字和符号，实际上做出来的大小为 12*6、16*8 和 24*12，取模方式设置如图 35.3.2.1 所示，取模方式选择为逐行式，格式为 C51。

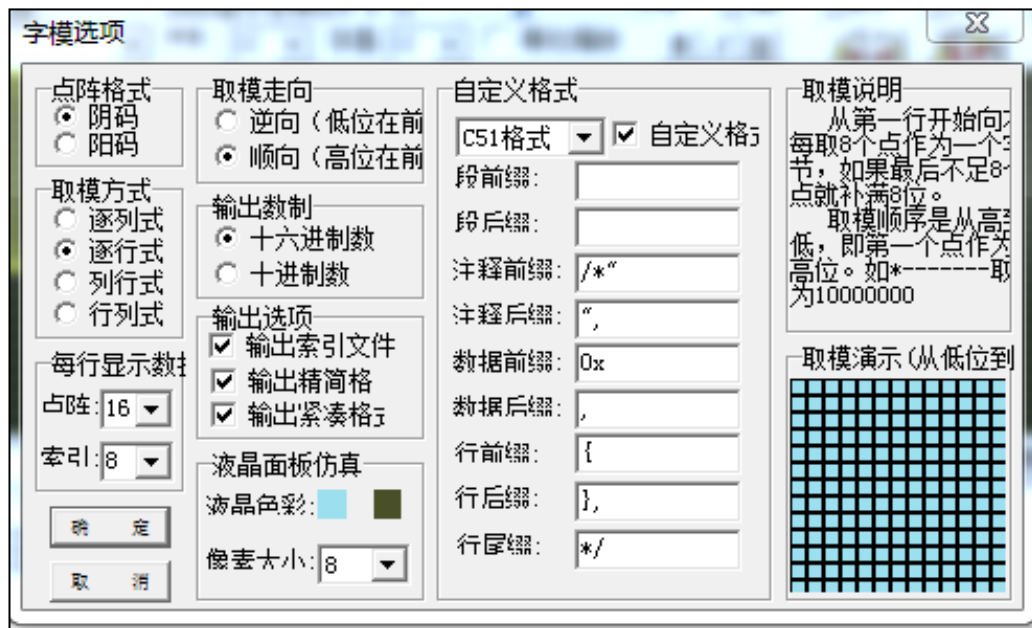


图 35.3.2.1 PCtoLCD2002 取模方式设置

最终我们要定义 3 个数组来存放 12*6、16*8 和 24*12 这 3 个不同点阵大小的字模，将制作好的字模数组放到 emWin_ASCII_Font.h 头文件中，对应的数组分别为：emwin_asc2_1206、emwin_asc2_1608 和 emwin_asc2_2412。

35.4 汉字显示实验

35.4.1 硬件设计

1. 例程功能

本实验主要讲解汉字显示(FLASH 外置字库)实验，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数调用了 GUI_SetFont 函数设置字体，然后调用函数 GUI_DispStringAt 设置文本。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 34 汉字显示(FLASH 外置字库)》。

注意：本实验不支持 Mini、DMF407 开发板运行。

35.4.2 软件设计

35.4.2.1 程序流程图

本实验的程序流程图，如下图所示：

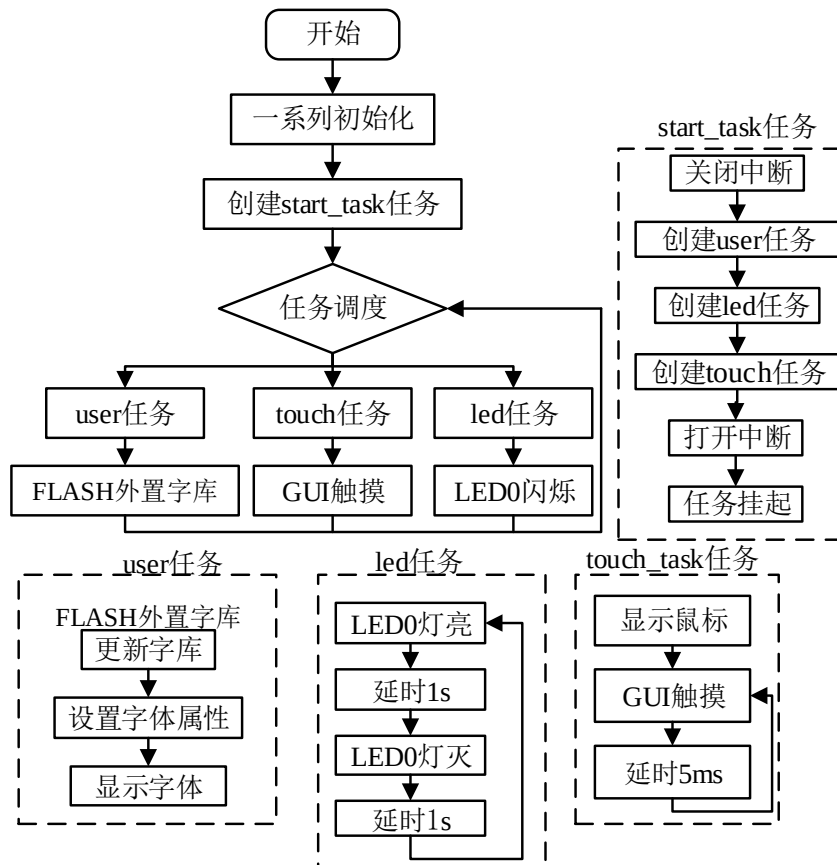


图 35.4.2.1.1 TTF 格式字体显示实验流程图

35.4.2.2 程序解析

(1) 汉字字体制作

本实验完整工程为：**emWin 实验 34 汉字显示(FLASH 外置字库)**。font.c 和 font.h 文件是完成字库更新和初始化的，在正点原子 STM32 系列的开发指南里面有详细的讲解，这里就不做讲解了。打开实验工程的 Middlewares/emWin/STemWinHz 分组，如下图所示：

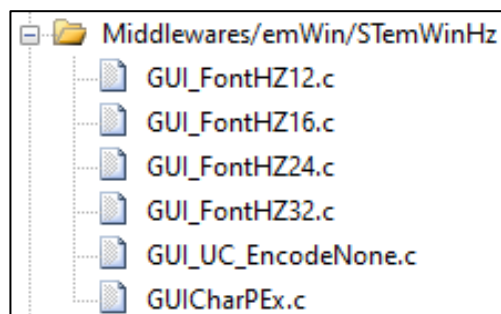


图 35.4.2.2.1 STemWinHZ 分组

在 emWin/STemWinHz 分组中有 5 个.c 文件，分别为 GUICharPEx.c、GUI_FontHZ12.c、GUI_FontHZ16.c、GUI_FontHZ24.c 和 GUI_UC_EncodeNone.c，打开 GUICharPEx.c 文件，代码如下：

```
#define BYTES_PER_FONT 1024 /* 32*32=1024，最大显示 32*32 点阵字库 */
U8 GUI_FontDataBuf[BYTES_PER_FONT];
```

```

/* 获取字模数据 */
static void GUI_GetDataFromMemory(const GUI_FONT_PROP GUI_UNI_PTR *pProp, U16P c)
{
    unsigned char qh, ql;
    unsigned char i;
    unsigned long foffset;
    unsigned char t;
    unsigned char *mat;
    uint8_t size, csize; /* 字体大小 */
    U16 BytesPerFont;
    GUI_FONT EMWINFONT;
    EMWINFONT = *GUI_pContext->pAFont;
    /* 每个字模的数据字节数 */
    BytesPerFont = GUI_pContext->pAFont->YSize*pProp->paCharInfo->BytesPerLine;
    if (BytesPerFont > BYTES_PER_FONT) BytesPerFont = BYTES_PER_FONT;

    /* 判断字体的大小 */
    /* 12 字体 */
    if (memcmp(&EMWINFONT, &GUI_FontHZ12, sizeof(GUI_FONT)) == 0) size = 12;
    /* 16 字体 */
    else if (memcmp(&EMWINFONT, &GUI_FontHZ16, sizeof(GUI_FONT)) == 0) size = 16;
    /* 24 字体 */
    else if (memcmp(&EMWINFONT, &GUI_FontHZ24, sizeof(GUI_FONT)) == 0) size = 24;

    /* 得到字体一个字符对应点阵集所占的字节数 */
    csize = (size / 8 + ((size % 8) ? 1 : 0)) * (size);
    if (c < 0x80) /* 英文字符地址偏移算法 */
    {
        switch (size)
        {
            case 12:
                for (t = 0; t < 12; t++)
                    GUI_FontDataBuf[t] = emwin_asc2_1206[c - 0x20][t];
                break;

            case 16:
                for (t = 0; t < 16; t++)
                    GUI_FontDataBuf[t] = emwin_asc2_1608[c - 0x20][t];
                break;

            case 24:
                for (t = 0; t < 48; t++)
                    GUI_FontDataBuf[t] = emwin_asc2_2412[c - 0x20][t];
        }
    }
}

```

```

        break;
    }
}
else
{
    ql = c / 256;
    qh = c % 256;

    if (qh < 0x81 || ql < 0x40 || ql == 0xff || qh == 0xff) /* 非常用汉字 */
    {
        for (i = 0; i < (size * 2); i++) *mat++ = 0x00; /* 填充满格 */

        return; /* 结束访问 */
    }

    if (ql < 0x7f) ql -= 0x40;
    else ql -= 0x41;

    qh -= 0x81;
    /* 得到字库中的字节偏移量 */
    foffset = ((unsigned long)190 * qh + ql) * csize;

    switch (size)
    {
        case 12:
            norflash_read(GUI_FontDataBuf, foffset + ftinfo.fl12addr, csize);
            break;

        case 16:
            norflash_read(GUI_FontDataBuf, foffset + ftinfo.fl16addr, csize);
            break;

        case 24:
            norflash_read(GUI_FontDataBuf, foffset + ftinfo.fl24addr, csize);
            break;
    }
}
}

/*****
*
*      Public code
*
*****/

```

```

*/
/*****
*
*      GUIPROP_Dispatch
*
* Purpose:
*   This is the routine that displays a character. It is used by all
*   other routines which display characters as a subroutine.
*/
void GUIPROP_X_Dispatch(U16P c)
{
    int BytesPerLine;
    GUI_DRAWMODE DrawMode = GUI_pContext->TextMode;
    const GUI_FONT_PROP GUI_UNI_PTR *pProp = GUI_pContext->pAFont->pProp;

    /* 搜索定位字库数据信息 */
    for (; pProp; pProp = pProp->pNext)
    {
        if ((c >= pProp->First) && (c <= pProp->Last)) break;
    }

    if (pProp)
    {
        GUI_DRAWMODE OldDrawMode;
        const GUI_CHARINFO GUI_UNI_PTR *pCharInfo = pProp->pCharInfo;
        GUI_GetDataFromMemory(pProp, c); /* 取出字模数据 */
        BytesPerLine = pCharInfo->BytesPerLine;
        OldDrawMode = LCD_SetDrawMode(DrawMode);
        LCD_DrawBitmap(GUI_pContext->DispPosX, GUI_pContext->DispPosY,
                       pCharInfo->XSize, GUI_pContext->pAFont->YSize,
                       GUI_pContext->pAFont->XMag, GUI_pContext->pAFont->YMag,
                       1, /* Bits per Pixel */
                       BytesPerLine,
                       &GUI_FontDataBuf[0],
                       &LCD_BKCOLORINDEX);

        /* Fill empty pixel lines */
        if (GUI_pContext->pAFont->YDist > GUI_pContext->pAFont->YSize)
        {
            int YMag = GUI_pContext->pAFont->YMag;
            int YDist = GUI_pContext->pAFont->YDist * YMag;
            int YSize = GUI_pContext->pAFont->YSize * YMag;

            if (DrawMode != LCD_DRAWMODE_TRANS)

```

```

        {
            LCD_COLOR OldColor = GUI_GetColor();
            GUI_SetColor(GUI_GetBkColor());
            LCD_FillRect(GUI_pContext->DispPosX, GUI_pContext->DispPosY +
                        YSize, GUI_pContext->DispPosX + pCharInfo->XSize,
                        GUI_pContext->DispPosY + YDist);
            GUI_SetColor(OldColor);
        }
    }

    LCD_SetDrawMode(OldDrawMode); /* Restore draw mode */
    GUI_pContext->DispPosX += pCharInfo->XDist * GUI_pContext->pAFont->XMag;
}

/*****
 *
 *      GUIPROP_GetCharDistX
 */
int GUIPROP_X_GetCharDistX(U16P c)
{
    const GUI_FONT_PROP GUI_UNI_PTR *pProp = GUI_pContext->pAFont->p.pProp;
    for (; pProp; pProp = pProp->pNext)
    {
        if ((c >= pProp->First) && (c <= pProp->Last)) break;
    }
    return (pProp) ? (pProp->paCharInfo->XSize * GUI_pContext->pAFont->XMag : 0;
}

```

上面计算汉字字符在 SPI FLASH 中的偏移量的公式和方法在正点原子裸机教程中有详细的讲解，不懂的请查阅正点原子裸机教程中汉字显示实验章节。

(2) emwin_main 函数

```

/**
 * @brief      emwin_main
 * @param      无
 * @retval     无
 */
void emwin_main()
{
    GUI_SetBkColor(GUI_BLUE);
    GUI_Clear();
    GUI_SetColor(GUI_RED);
    GUI_SetFont(&GUI_FontHZ12);
    GUI_DispStringAt("12 号字体: 广州星翼电子 ALIENTEK!", 10, 10);
}

```



```
GUI_SetFont(&GUI_FontHZ16);
GUI_DispStringAt("16 号字体: 广州星翼电子 ALIENTEK!",10,30);
GUI_SetFont(&GUI_FontHZ24);
GUI_DispStringAt("24 号字体: 广州星翼电子 ALIENTEK!",10,50);

while(1)
{
    GUI_Delay(100);
}
}
```

此函数调用了 GUI_SetFont 函数设置字体，然后调用函数 GUI_DispStringAt 设置文本。

注意：必须在 GUI_Type.h 文件声明一种新的字体结构，如下源码所示：

```
/* 声明一种新的字体结构 */
#define GUI_FONTTYPE_PROP_USER \
    GUIPROP_X_DispChar, \
    (GUI_GETCHARDISTX*)GUIPROP_X_GetCharDistX, \
    GUIMONO_GetFontInfo, \
    GUIMONO_IsInFont, \
    (GUI_GETCHARINFO *)0, \
    (tGUI_ENC_APIList*)0
```

35.4.3 下载验证

编译并下载到开发板中，下载完成以后首先按 KEY_UP 键更新字库，字库更新完成以后 LCD 上会显示提示信息“Font Updata finshed,Please Restart! ”。字库更新完成以后根据提示复位一次开发板，复位完成后 LCD 就可以显示出汉字了，如下图所示：



图 35.4.3.1 汉字显示

第三十六章 BMP 图片显示

emWin 支持 BMP 图片解码，可以直接从外部存储器（如 SD 卡）中读取 BMP 图片文件在 LCD 上显示，本章就讲解如何使用 emWin 的 BMP 解码函数在 LCD 上显示 BMP 图片。

本章分为以下几个部分：

36.1 支持格式

36.2 BMP 解码 API 函数

36.3 BMP 显示实验

36.1 支持格式

BMP 文件格式由 Microsoft 定义，下表显示了许多不同的格式：

每像素位数	索引	压缩	支持
1	是	否	是
4	是	否	是
4	是	是	是
8	是	否	是
8	是	是	是
16	否	否	是
24	否	否	是
32	否	否	是

表 36.1.1 BMP 格式

36.2 BMP 解码 API 函数

表 36.2.1 列出了关于 BMP 图片显示的 API 函数。

函数	描述
GUI_BMP_Draw()	绘制已加载到存储器的 BMP 文件
GUI_BMP_DrawEx()	绘制无需加载到存储器的 BMP 文件
GUI_BMP_DrawScaled()	绘制已加载到存储器的带比例的 BMP 文件
GUI_BMP_DrawScaledEx()	绘制无需加载到存储器的带比例的 BMP 文件
GUI_BMP_EnableAlpha()	为带 32bpp 颜色深度的 BMP 文件启用半透明
GUI_BMP_GetXSize()	返回加载到存储器的 BMP 文件的 X 大小
GUI_BMP_GetXSizeEx()	返回无需加载到存储器的 BMP 文件的 X 大小
GUI_BMP_GetYSize()	返回加载到存储器的位图的 Y 大小
GUI_BMP_GetYSizeEx()	返回无需加载到存储器的 Y 大小
GUI_BMP_Serialize()	创建 BMP 文件
GUI_BMP_SerializeEx()	基于给定的矩形创建 BMP 文件

表 36.2.1 BMP 文件 API 函数

(1) GUI_BMP_Draw 函数

在当前窗口中的指定位置绘制已加载到内存的 bmp 文件，其函数的原型如下源码所示：

```
int GUI_BMP_Draw(const void * pFileData, int x0, int y0);
```

该函数的相关形参如下表所示：

参数	描述
pFileData	指向 bmp 文件所在的存储器区域的起始位置的指针

x0	显示器中位图左上角的 X 位置
y0	显示器中位图左上角的 Y 位置

表 36.2.2 GUI_BMP_Draw 函数的相关形参描述

返回值：返回内存大小。

(2) GUI_BMP_DrawEx 函数

在当前窗口中的指定位置绘制不必加载到内存的 bmp 文件，其函数的原型如下源码所示：

```
int GUI_BMP_DrawEx(GUI_GET_DATA_FUNC * pfGetData, void * p, int x0, int y0);
```

该函数的相关形参如下表所示：

参数	描述
pFileData	指向用于获取数据而调用的函数的指针
p	传递给由 pfGetData 指向的函数的 Void 指针
x0	显示器中位图左上角的 X 位置
y0	显示器中位图左上角的 Y 位置

表 36.2.3 GUI_BMP_DrawEx 函数的相关形参描述

返回值：0 成功，非零值函数失败。

(3) GUI_BMP_DrawScaled 函数

在当前窗口中的指定位置通过缩放比例绘制已加载到内存的 bmp 文件，其函数的原型如下源码所示：

```
int GUI_BMP_DrawScaled(const void * pFileData, int x0,int y0,int Num,int Denom);
```

该函数的相关形参如下表所示：

参数	描述
pFileData	指向用于获取数据而调用的函数的指针
x0	显示器中位图左上角的 X 位置
y0	显示器中位图左上角的 Y 位置
Num	用于缩放比例的分子
Denom	用于缩放比例的分母

表 36.2.4 GUI_BMP_DrawScaled 函数的相关形参描述

返回值：0 成功，非零值函数失败。

注意：该函数通过使用给定的分子和分母构建分数来缩放图像。例如，如果图像应缩减至 2/3 大小，则 Num 应为 2，Denom 应为 3。

(4) GUI_BMP_DrawScaledEx 函数

在当前窗口中的指定位置通过缩放比例绘制不必加载到内存的 bmp 文件，其函数的原型如下源码所示：

```
int GUI_BMP_DrawScaledEx(GUI_GET_DATA_FUNC * pfGetData, void * p,  
                          int x0, int y0,int Num, int Denom);
```

该函数的相关形参如下表所示：

参数	描述
pfGetData	指向用于获取数据而调用的函数的指针
p	传递给由 pfGetData 指向的函数的 void 指针
x0	显示器中位图左上角的 X 位置
y0	显示器中位图左上角的 Y 位置
Num	用于缩放比例的分子
Denom	用于缩放比例的分

表 36.2.5 GUI_BMP_DrawScaledEx 函数的相关形参描述

返回值：0 成功，非零值函数失败。

注意：该函数通过使用给定的分子和分母构建分数来缩放图像。例如，如果图像应缩减至 2/3 大小，则 Num 应为 2， Denom 应为 3。

(5) GUI_BMP_EnableAlpha 函数

为带 32bpp 颜色深度的 BMP 文件启用半透明，其函数的原型如下源码所示：

```
void GUI_BMP_EnableAlpha(void);
```

返回值：无。

(6) GUI_BMP_GetXSize 函数

返回已加载到内存的指定位图的 X 大小，其函数的原型如下源码所示：

```
int GUI_BMP_GetXSize(const void * pFileData);
```

该函数的相关形参如下表所示：

参数	描述
pFileData	指向 bmp 文件所在的内存区域的起始位置的指针

表 36.2.6 GUI_BMP_GetXSize 函数的相关形参描述

返回值：位图的 X 大小。

(7) GUI_BMP_GetXSizeEx 函数

返回不必加载到内存的指定 bmp 文件的 X 大小，其函数的原型如下源码所示：

```
int GUI_BMP_GetXSizeEx(GUI_GET_DATA_FUNC * pfGetData, void * p);
```

该函数的相关形参如下表所示：

参数	描述
pfGetData	指向用于获取数据而调用的函数的指针
p	传递给由 pfGetData 指向的函数的 Void 指针

表 36.2.7 GUI_BMP_GetXSizeEx 函数的相关形参描述

返回值：位图的 X 大小。

(8) GUI_BMP_GetYSize 函数

返回已加载到存储器的指定位图的 Y 大小，其函数的原型如下源码所示：

```
int GUI_BMP_GetYSize(const void * pFileData);
```

该函数的相关形参如下表所示：

参数	描述
pFileData	指向 bmp 文件所在的内存区域的起始位置的指针

表 36.2.8 GUI_BMP_GetYSize 函数的相关形参描述

返回值：位图的 Y 大小。

(9) GUI_BMP_GetYSizeEx 函数

返回不必加载到存储器的指定 bmp 文件的 X 大小，其函数的原型如下源码所示：

```
int GUI_BMP_GetYSizeEx(GUI_GET_DATA_FUNC * pfGetData, void * p);
```

该函数的相关形参如下表所示：

参数	描述
pfGetData	指向 bmp 文件所在的内存区域的起始位置的指针
p	传递给由 pfGetData 指向的函数的 Void 指针

表 36.2.9 GUI_BMP_GetYSizeEx 函数的相关形参描述

返回值：位图的 Y 大小。

(10) GUI_BMP_Serialize 函数

创建包含 LCD 完整内容的 BMP 文件。BMP 文件使用 emWin 中使用的颜色深度来创建，最大为 24bpp。在使用小于 8bpp 的颜色深度情况下，BMP 文件颜色深度为 8bpp。当前选定的设备用于读取像素数据。如果选择内存设备，其内容将写入文件。

该函数不支持存储压缩的 BMP 数据，其函数的原型如下源码所示：

```
void GUI_BMP_Serialize(GUI_CALLBACK_VOID_U8_P * pfSerialize, void * p);
```

该函数的相关形参如下表所示：

参数	描述
pfSerialize	指向初始化函数的指针
p	指向传递给初始化函数的用户定义的数据的指针

表 36.2.10 GUI_BMP_Serialize 函数的相关形参描述

返回值：无。

(11) GUI_BMP_SerializeEx 函数

创建包含给定数据区的 BMP 文件。BMP 文件使用 emWin 中使用的颜色深度来创建，最大为 24bpp。在使用小于 8bpp 的颜色深度情况下，BMP 文件颜色深度为 8bpp。当前选定的设备用于读取像素数据。如果选择内存设备，其内容将写入文件。该函数不支持存储压缩的 BMP 数据，其函数的原型如下源码所示：

```
void GUI_BMP_SerializeEx(GUI_CALLBACK_VOID_U8_P * pfSerialize,
                        int x0, int y0, int xSize, int ySize, void * p);
```

该函数的相关形参如下表所示：

参数	描述
pfSerialize	指向初始化函数的指针
x0	用于创建 BMP 文件的 X 起始位置
y0	用于创建 BMP 文件的 Y 起始位置
xSize	X 大小
ySize	Y 大小
p	指向传递给初始化函数的用户定义的数据的指针

表 36.2.10 GUI_BMP_Serialize 函数的相关形参描述

返回值：无。

36.3 BMP 显示实验

36.3.1 硬件设计

1. 例程功能

本实验主要讲解 BMP 图片显示实验，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数利用循环的形式加载 BMP 图片，当用户按下 KEY0 按键时，我们会看到 LED1 灯由灭到亮，此时系统把当前界面进行截图并保存到 SD 卡指定路径下，直到 LED1 由亮到灭才表示截图结束；当用户按下 KEY_UP 按键时，系统会更新当前字库。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 35 BMP 图片显示》。

注意：本实验不支持 Mini、DMF407 开发板运行。

36.3.2 软件设计

36.3.2.1 程序流程图

本实验的程序流程图，如下图所示：

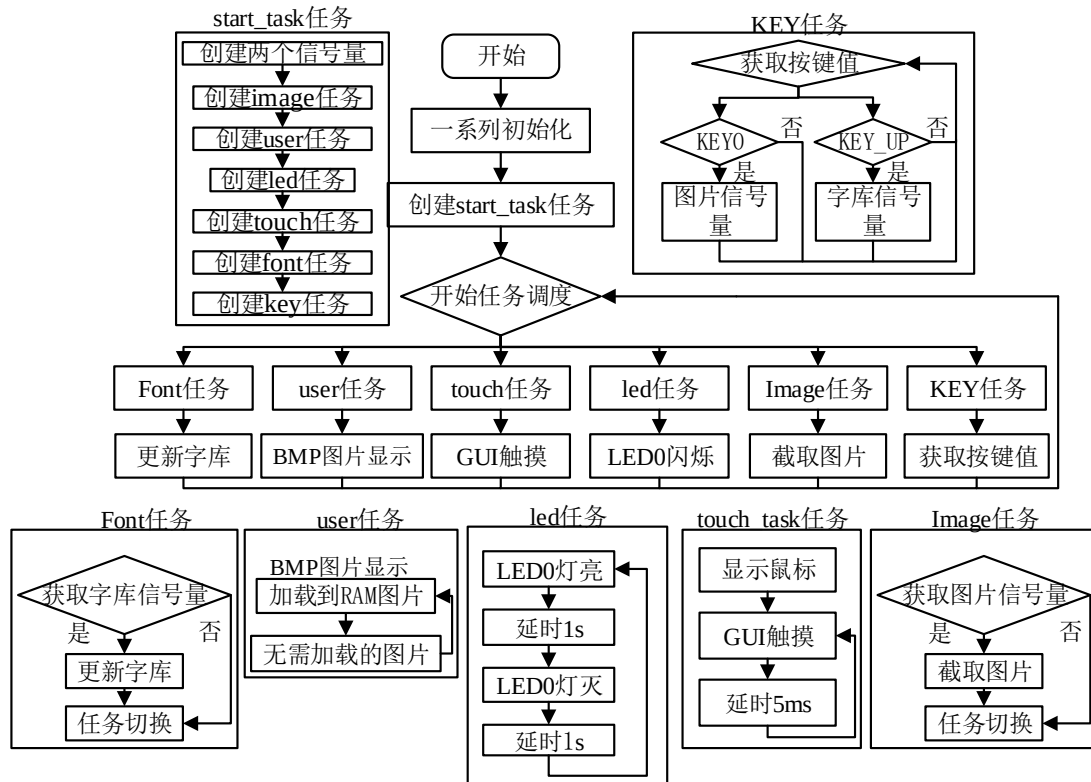


图 36.3.2.1.1 BMP 图片显示实验流程图

36.3.2.2 程序解析

本实验完整工程为：**emWin 实验 35 BMP 图片显示**。打开文件 `emwin_demo.c`，程序源码如下：注意，由于精英版没有外部 SRAM，所以不能先将 `bmp` 图片加载到 `ram` 中，然后在 LCD 上显示，因此也就没有提供相应的 API 函数。

(1) key_task 任务函数

```

/**
 * @brief      key_task
 * @param      p_arg : 传入参数(未用到)
 * @retval     无
 */
void key_task(void *p_arg)
{
    OS_ERR err;
    p_arg = p_arg;
    uint8_t key;

    while(1)
  
```

```

{
    key = key_scan(0);

    switch(key)
    {
        case KEY0_PRES:
            printf("发送截图信号量\r\n");
            OSSemPost (&IMAGE_SEM, OS_OPT_POST_1, &err); /* 发送截图信号量 */
            break;

        case WKUP_PRES:
            printf("更新字体信号量\r\n");
            OSSemPost (&FONT_SEM, OS_OPT_POST_1, &err); /* 发送更新字体信号量 */
            break;
    }

    OSTimeDlyHMSM(0,0,0,10,OS_OPT_TIME_PERIODIC,&err); /* 延时 10ms */
}
}

```

上述的源码，我们可以得知 KEY0_PRES 按键为释放截图的信号量，而 WKUP_PRES 按键为释放更新字体信号量。

(2) Font_task 任务函数

```

/**
 * @brief      font_task
 * @param      p_arg : 传入参数(未用到)
 * @retval     无
 */
void font_task(void *p_arg)
{
    OS_ERR err;
    p_arg = p_arg;

    while (1)
    {
        /* 请求截图信号量 */
        OSSemPend(&FONT_SEM, 0, OS_OPT_PEND_BLOCKING, 0, &err);
        GUI_COLOR forecolor, bkcolor;
        forecolor = GUI_GetColor(); /* 获取当前前景色 */
        bkcolor = GUI_GetBkColor(); /* 获取当前背景色 */
        OSSchedLock(&err);
        lcd_show_string(10,50,250,30,16,"Font Updataing,Please Wait...",RED);
        fonts_update_font(10,70,16,(uint8_t *) "0:", RED);
        lcd_show_string(10,50,280,30,16,"Font Updata finshed,Please
                                Restart!",RED);
    }
}

```

```

        OSSchedUnlock(&err);

        GUI_SetColor(forecolor);    /* 恢复前景色 */
        GUI_SetBkColor(bkcolor);    /* 恢复背景色 */
        OSTimeDlyHMSM(0,0,0,10,OS_OPT_TIME_PERIODIC,&err); /* 延时 10ms */
    }
}

```

首先运行这个程序，必须获取一个信号量，该信号量在 `key_task` 任务中释放的，调用函数 `GUI_GetColor` 和函数 `GUI_GetBkColor` 获取更新字体之前的背景颜色和前景颜色，这样我们在更新字体之后再调用 `GUI` 的设置背景函数和设置前景函数这样可以使图片还原操作。更新字体必须调用操作系统的 `OSSchedLock` 调度锁，这样可以不被其他任务打扰字体更新操作，字体更新完毕必须调用 `OSSchedUnlock` 调度解锁函数，不然其他任务无法执行。

(3) image_task 任务函数

```

/**
 * @brief      image_task
 * @param      p_arg : 传入参数(未用到)
 * @retval     无
 */
void image_task(void *p_arg)
{
    OS_ERR err;
    p_arg = p_arg;

    while (1)
    {
        /* 请求截图信量 */
        OSSemPend(&IMAGE_SEM, 0, OS_OPT_PEND_BLOCKING, 0, &err);
        OSSchedLock(&err);

        LED1(0);

        create_bmppicture((uint8_t *) "0:/SCREENSHORT/屏幕截图 0.bmp"
                           ,0,0,lcddev.width,lcddev.height); /* 创建 BMP 截图 */

        LED1(1);
        OSSchedUnlock(&err);
        OSTimeDlyHMSM(0,0,0,10,OS_OPT_TIME_PERIODIC,&err); /* 延时 10ms */
    }
}

```

`image_task` 任务和 `Font_task` 任务是差不多的，主要也是获取信号量，开启任务调度锁和关闭任务锁，其中间的截图程序如下源码所示：

```

/**
 * @brief      此函数被 GUI_BMP_Serialize() 调用，用来向文件写入字节
 * @param      无
 * @retval     无
 */
static void _WriteByte2File(U8 Data, void *p)

```



```

{
    UINT nWritten;
    f_write((FIL *)p, &Data, 1, &nWritten);
}

/**
 * @brief      生成一个 BMP 图片，保存到 SD 卡中, 实现屏幕截图功能！
 * @param      x0: 创建 BMP 文件的 X 起始位置
 * @param      y0: 创建 BMP 文件的 Y 起始位置
 * @param      Xsize: X 的大小
 * @param      Ysize: Y 的大小
 * @retval     无
 */
void create_bmppicture(uint8_t *filename, int x0, int y0, int Xsize, int Ysize)
{
    static FIL hFile;

    GUI_COLOR forecolor, bkcolor;
    forecolor = GUI_GetColor(); /* 获取当前前景色 */
    bkcolor = GUI_GetBkColor(); /* 获取当前背景色 */
    /* 创建一个文件，路径为 filename, 如果文件原本就存在的话会被新建的文件覆盖掉！ */
    f_open(&hFile, (const TCHAR *)filename, FA_READ | FA_WRITE |
                                                FA_CREATE_ALWAYS);
    GUI_BMP_SerializeEx(_WriteByte2File, x0, y0, Xsize, Ysize, &hFile);
    f_close(&hFile); /* 关闭文件 */
    GUI_SetColor(forecolor); /* 恢复前景色 */
    GUI_SetBkColor(bkcolor); /* 恢复背景色 */
}

```

我们也知道，使用文件系统是三步走，打开文件—读写文件—关闭文件等操作。首先也是获取背景颜色和前景颜色，调用系统文件函数 `f_open` 打开文件的路径，调用 `emwin` 的 `GUI_BMP_SerializeEx` 绘制图片函数，该函数的回调函数调用函数 `f_write` 写入数据，然后关闭文件。

(4) emwin_main 函数

```

/**
 * @brief      emwin_main(emwin 入口)
 * @param      无
 * @retval     无
 */
void emwin_main (void)
{
    GUI_SetBkColor(GUI_BLUE);
    GUI_SetColor(GUI_RED);
    GUI_SetFont(&GUI_FontHZ16);
    GUI_Clear();
}

```

```

while (1)
{
    GUI_DispStringHCenterAt("在指定位置显示一张加载到 RAM 中的 BMP 图片", 400, 5);
    dispbmp((uint8_t *) "0:/PICTURE/BMP/正点原子 LOGO.bmp", 0, 10, 78, 1, 1);
    GUI_Delay(1000);
    GUI_Clear();

    GUI_DispStringHCenterAt("在 LCD 中间显示一张缩小 1/2 的加载到 RAM 中的 BMP 图片",
                            400, 5);
    dispbmp((uint8_t *) "0:/PICTURE/BMP/ALIENTEKLOGO.bmp", 1, 0, 0, 1, 2);
    GUI_Delay(1000);
    GUI_Clear();

    GUI_DispStringHCenterAt("LCD 中间显示一张扩大 4/3 倍的加载到 RAM 中的 BMP 图片",
                            400, 5);
    dispbmp((uint8_t *) "0:/PICTURE/BMP/ ALIENTEKLOGO.bmp", 1, 0, 0, 4, 3);
    GUI_Delay(1000);
    GUI_Clear();

    GUI_DispStringHCenterAt("在指定位置显示一张无需加载的 BMP 图片", 400, 5);
    dispbmpex((uint8_t *) "0:/PICTURE/BMP/ ALIENTEKLOGO.bmp", 0, 10, 8, 1, 1);
    GUI_Delay(1000);
    GUI_Clear();

    GUI_DispStringHCenterAt("在 LCD 中间显示一张缩小 1/2 的无需加载的 BMP 图片",
                            400, 5);
    dispbmpex((uint8_t *) "0:/PICTURE/BMP/ ALIENTEKLOGO.bmp", 1, 0, 0, 1, 2);
    GUI_Delay(1000);
    GUI_Clear();

    GUI_DispStringHCenterAt("在 LCD 中间显示一张扩大 4/3 倍的无需加载的 BMP 图片",
                            400, 5);
    dispbmpex((uint8_t *) "0:/PICTURE/BMP/ ALIENTEKLOGO.bmp", 1, 0, 0, 4, 3);
    GUI_Delay(1000);
    GUI_Clear();
}
}
    
```

上述的源码，其实也是很简单的，主要就是循环刷新不同的图片模式，最主要的函数是有两个，它们分别为 `dispbmp()` 和 `dispbmpex()`，接下来笔者逐一讲解这两个函数的实现原理。

/**

* @brief 在指定位置显示加载到 RAM 中的 JPEG 图片

 BMPFileName: 图片在 SD 卡或者其他存储设备中的路径(需文件系统支持!)

```

* @param      mode:显示模式
    @arg      0 在指定位置显示, 有参数 x,y 确定显示位置
    @arg      1 在 LCD 中间显示图片, 当选择此模式的时候参数 x,y 无效。
* @param      x:图片左上角在 LCD 中的 x 轴位置 (当参数 mode 为 1 时, 此参数无效)
* @param      y:图片左上角在 LCD 中的 y 轴位置 (当参数 mode 为 1 时, 此参数无效)
* @param      member: 缩放比例的分子项
* @param      denom:缩放比例的分母项
* @retval     0 显示正常,其他 失败
*/
int dispbmp(uint8_t *BMPFileName, uint8_t mode, uint32_t x, uint32_t y,
            int member,
            int denom)
{
    uint16_t bread;
    char *bmpbuffer;
    char result;
    int XSize, YSize;
    float Xflag, Yflag;
    GUI_COLOR forecolor, bkcolor;

    forecolor = GUI_GetColor(); /* 获取当前前景色 */
    bkcolor = GUI_GetBkColor(); /* 获取当前背景色 */
    /* 打开文件 */
    result = f_open(&BMPFile, (const TCHAR *)BMPFileName, FA_READ);

    /* 文件打开错误或者文件大于 BMPMEMORYSIZE */
    if ((result != FR_OK) || (BMPFile.obj.objsize > BMPMEMORYSIZE)) return 1;

    bmpbuffer = mymalloc(SRAMEX, BMPFile.obj.objsize); /* 申请内存 */

    if (bmpbuffer == NULL) return 2;

    /* 读取数据 */
    result = f_read(&BMPFile, bmpbuffer, BMPFile.obj.objsize, (UINT *)&bread);

    if (result != FR_OK) return 3;

    switch (mode)
    {
        case 0: /* 在指定位置显示图片 */
            if ((member == 1) && (denom == 1)) /* 无需缩放, 直接绘制 */
            {
                GUI_BMP_Draw(bmpbuffer, x, y); /* 在指定位置显示 BMP 图片 */
            }
    }
}

```

```

    }

    else /* 否则图片需要缩放 */
    {
        GUI_BMP_DrawScaled(bmpbuffer, x, y, member, denom);
    }

    break;

case 1: /* 在 LCD 中间显示图片 */
    XSize = GUI_BMP_GetXSize(bmpbuffer); /* 获取图片的 x 轴大小 */
    YSize = GUI_BMP_GetYSize(bmpbuffer); /* 获取图片的 y 轴大小 */

    if ((member == 1) && (denom == 1)) /* 无需缩放, 直接绘制 */
    {
        /* 在 LCD 中间显示图片 */
        GUI_BMP_Draw(bmpbuffer, (lcddev.width - XSize) / 2 - 1,
                      (lcddev.height - YSize) / 2 - 1);
    }
    else /* 否则图片需要缩放 */
    {
        Xflag = (float)XSize * ((float)member / (float)denom);
        Yflag = (float)YSize * ((float)member / (float)denom);
        XSize = (lcddev.width - (int)Xflag) / 2 - 1;
        YSize = (lcddev.height - (int)Yflag) / 2 - 1;
        GUI_BMP_DrawScaled(bmpbuffer, XSize, YSize, member, denom);
    }
    break;
}

f_close(&BMPFile); /* 关闭 BMPFile 文件 */
myfree(SRAMEX, bmpbuffer); /* 释放内存 */
GUI_SetColor(forecolor); /* 恢复前景色 */
GUI_SetBkColor(bkcolor); /* 恢复背景色 */
return 0;
}

```

上述的源码也是挺简单的，也是和上面的讲述一下，文件的操作，图片的放大还是缩小，最主要的一点就是该函数是使用 RAM 来加载图片的，函数 `dispbmp()` 用于显示 BMP 图片，该函数首先将 BMP 图片的所有数据加载到 RAM 中，然后调用 EMWIN 中的相应函数在 LCD 上显示加载到 RAM 中的 BMP 图片，由于需要将整副 BMP 图片加载到 RAM 中，所以需要大量的 RAM，但是显示速度快！此函数可以设置要显示的 BMP 图片的位置和放大倍数。

函数 `dispbmpex` 如下源码所示：

```
/**
```

```

* @brief      在指定位置显示无需加载到 RAM 中的 BMP 图片 (需文件系统支持! 对于小 RAM, 推荐使用此方法!)
               BMPFileName: 图片在 SD 卡或者其他存储设备中的路径
* @param      mode: 显示模式
               @arg      0 在指定位置显示, 有参数 x, y 确定显示位置
               @arg      1 在 LCD 中间显示图片, 当选择此模式的时候参数 x, y 无效。
* @param      x: 图片左上角在 LCD 中的 x 轴位置 (当参数 mode 为 1 时, 此参数无效)
* @param      y: 图片左上角在 LCD 中的 y 轴位置 (当参数 mode 为 1 时, 此参数无效)
* @param      member: 缩放比例的分子项
* @param      denom: 缩放比例的分母项
* @retval     0 显示正常, 其他 失败
*/
    
```

```

int dispbmpex( uint8_t *BMPFileName, uint8_t mode, uint32_t x, uint32_t y,
               int member,
               int denom)
{
    char result;
    int XSize, YSize;
    float Xflag, Yflag;
    GUI_COLOR forecolor, bkcolor;
    forecolor = GUI_GetColor(); /* 获取当前前景色 */
    bkcolor = GUI_GetBkColor(); /* 获取当前背景色 */
    /* 打开文件 */
    result = f_open(&BMPFile, (const TCHAR *)BMPFileName, FA_READ);

    /* 文件打开错误 */
    if (result != FR_OK)    return 1;

    switch (mode)
    {
        case 0: /* 在指定位置显示图片 */
            if ((member == 1) && (denom == 1)) /* 无需缩放, 直接绘制 */
            {
                /* 在指定位置显示 BMP 图片 */
                GUI_BMP_DrawEx(BmpGetData, &BMPFile, x, y);
            }
            else /* 否则图片需要缩放 */
            {
                GUI_BMP_DrawScaledEx(BmpGetData, &BMPFile, x, y, member, denom);
            }
            break;

        case 1: /* 在 LCD 中间显示图片 */
    
```

```

/* 获取图片的 x 轴大小 */
XSize = GUI_BMP_GetXSizeEx(BmpGetData, &BMPFile);
/* 获取图片的 y 轴大小 */
YSize = GUI_BMP_GetYSizeEx(BmpGetData, &BMPFile);

if ((member == 1) && (denom == 1)) /* 无需缩放, 直接绘制 */
{
    /* 在 LCD 中间显示图片 */
    GUI_BMP_DrawEx(BmpGetData, &BMPFile,
                    (lcddev.width - XSize) / 2 - 1,
                    (lcddev.height - YSize) / 2 - 1);
}
else /* 否则图片需要缩放 */
{
    Xflag = (float)XSize * ((float)member / (float)denom);
    Yflag = (float)YSize * ((float)member / (float)denom);
    XSize = (lcddev.width - (int)Xflag) / 2 - 1;
    YSize = (lcddev.height - (int)Yflag) / 2 - 1;
    GUI_BMP_DrawScaledEx(BmpGetData, &BMPFile, XSize, YSize,
                          member, denom);
}
break;
}

f_close(&BMPFile); /* 关闭 BMPFile 文件 */
GUI_SetColor(forecolor); /* 恢复前景色 */
GUI_SetBkColor(bkcolor); /* 恢复背景色 */
return 0;
}

```

函数 `dispbmpex()` 也是显示 BMP 图片的, 与函数 `dispbmp()` 不同的是, `dispbmpex()` 不需要先将 BMP 图片加载到 RAM 中, 可以直接从外部存储器中读取 BMP 图片并显示, 这种方法需要的 RAM 小, 但是速度相比 `dispbmp()` 要慢, 此函数也可以设置要显示的 BMP 图片的位置和放大倍数。

36.3.3 下载验证

编译并下载到开发板中, LCD 轮流着按照指定的显示方法显示图片, 如下图所示:

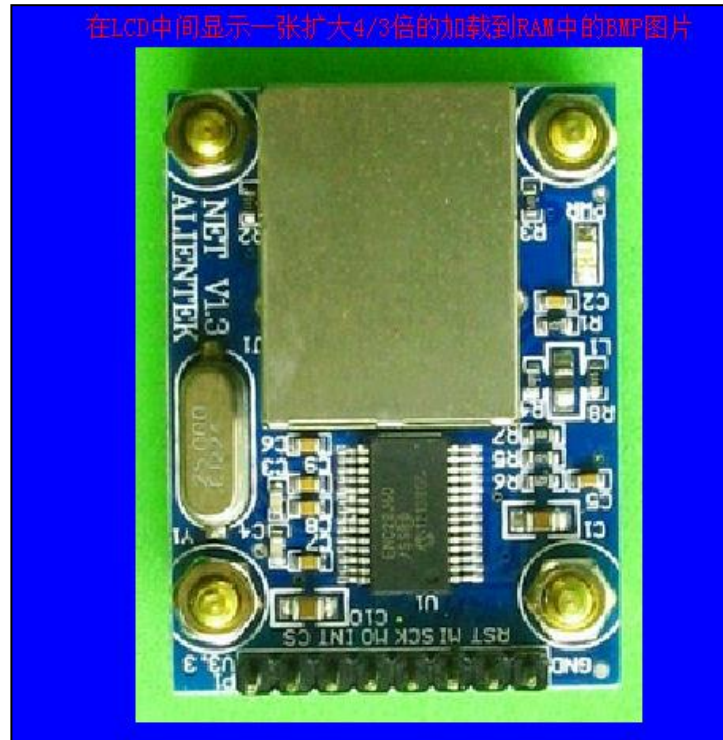


图 36.3.3.1 显示 BMP 图片

图 36.3.3.1 是在屏幕中间显示一张加载到 RAM 中并且扩大了 4/3 倍的 BMP 图片，按下 KEY0 键可以进行屏幕截图，下图显示的就是屏幕截图。

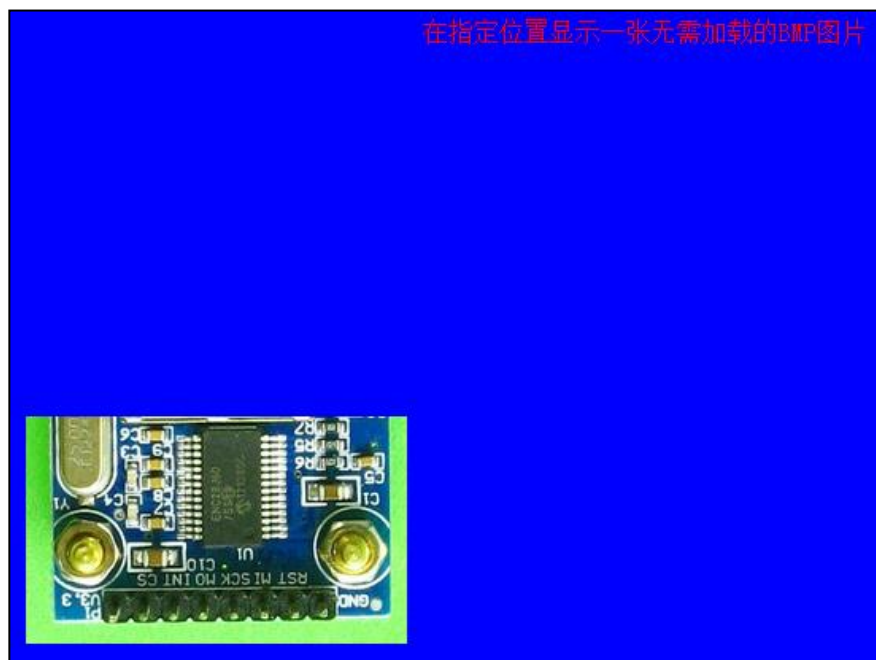


图 36.3.3.2 屏幕截图

图 36.3.3.2 是正在显示的时候进行了屏幕截图，可以看出此时图片显示并未完成就进行了屏幕截图。

注意！：进行屏幕截图的时候通过观察 LED1 来判断截图是否完成，LED1 点亮表示开始截图，LED1 熄灭表明截图完成。

第三十七章 JPEG 图片显示

JPEG 是全彩和灰度图像的标准压缩方法，JPEG 用于压缩“真实世界”的景象、线条画、卡通，其他非现实图像并不是其强项。JPEG 会有损耗，意指输出图像与输入图像并不完全相同，因此，如果需要达到完全相同的输出位，则不能使用 JPEG。不过，对于常见的照片图像，可以得到非常好的压缩级别，看不出变化。emWin 支持 JPEG 图片显示，可以直接从外部存储器（如 SD 卡）中读取 JPEG 图片文件在 LCD 上显示，本章就讲解如何使用 emWin 的 JPEG 显示函数在 LCD 上显示 JPEG 图片。

本章分为以下几个部分：

- 37.1 支持的 JPEG 压缩方法
- 37.2 支持的 JPEG 压缩方法
- 37.3 JPEG 解码 API 函数
- 37.4 JPEG 显示实验

37.1 支持的 JPEG 压缩方法

此软件执行 JPEG 基准、可扩展顺序以及渐进式的压缩流程。尽管尚未实施某些不常见的参数设置，但它支持这些流程的所有变体。出于法律原因，不得分发 JPEG 算术编码变体的代码。JPEG 规范的算术编码选项似乎属于归 IBM、AT&T 和 Mitsubishi 所有的专利。因此，从法律上讲，如未获得一个或多个许可，则不能使用算术编码。JPEG 文件支持不包含提供标准中定义的层次式或无损流程。

37.2 RAM 使用要求

JPEG 解压缩大约需要 33 Kb RAM 用于与图像大小无关的解压缩和依赖大小的字节量。RAM 要求可按以下方式计算：

$$\text{大约 RAM 要求} = \text{图像的 X 大小} * 80 \text{ 字节} + 33 \text{ 千字节}$$

公式中 X 大小的取决于 JPEG 文件的压缩类型，下表列出了部分类型：

压缩	图像大小(像素)	RAM 使用 [千字节]	RAM 使用, 大小 依赖量[千字节]
H1V1	160x120	45	12
H2V2	160x120	46	13
GRAY	160x120	38	4

表 37.2.1 RAM 使用量

JPEG 图片显示时解压缩所需的 RAM 由 emWin 存储器管理系统自动分配，绘制 JPEG 图片后就会释放掉 RAM。

将 JPEG 文件转换为 C 源码

某些情况下，将 JPEG 文件作为 C 文件添加到项目中非常有用。这时，首先需要将 JPEG 文件转换为 C 文件。使用 emWin 随附的工具 Bin2C.exe 可完成此任务。在“Tools”子文件夹中可找到该工具。它可以将给定的二进制文件（在本例中为 JPEG 文件）转换为 C 文件。C 文件的文件名与二进制文件名称相同，采用文件扩展名“.c”。

以下步骤将显示如何使用 Bin2C 嵌入 JPEG 文件：

- ✧ 启动 Bin2C.exe，并选择要转换为 C 文件的 JPEG 文件（例如“Image.jpeg”），然后将其转换为 C 文件。
- ✧ 将该 C 文件添加到项目中。

下面的示例可以是 JPEG、GIF、PNG 格式都可以的，如下源码所示：

```
#include "GUI.h"

#include "Image.c" /* 包括转换后的 c 文件 */

void emwin_main(void)
{
    GUI_Init();
    GUI_JPEG_Draw(acImage, sizeof(acImage), 0, 0);
    ...
}
```

37.3 JPEG 解码 API 函数

表 37.3.1 列出了关于 BMP 图片显示的 API 函数。

函数	描述
GUI_JPEG_Draw	绘制已加载到存储器的 JPEG 文件
GUI_JPEG_DrawEx	绘制无需加载到存储器的 JPEG 文件
GUI_JPEG_DrawScaled	绘制已加载到存储器的带比例的 JPEG 文件
GUI_JPEG_DrawScaledEx	绘制无需加载到存储器的带比例的 JPEG 文件
GUI_JPEG_GetInfo	基于已加载到存储器的 JPEG 文件填充 UGL_JPEG_INFO 结构
GUI_JPEG_GetInfoEx	基于无需加载到存储器的 JPEG 文件填充 GUI_JPEG_INFO 结构

表 37.3.1 JPEG 文件 API 函数

(1) GUI_JPEG_Draw 函数

在当前窗口中的指定位置绘制已加载到内存的 jpeg 文件，其函数的原型如下源码所示：

```
int GUI_JPEG_Draw(const void * pFileData,
                  int DataSize, int x0, int y0);
```

该函数的相关形参如下表所示：

参数	描述
pFileData	指向 jpeg 文件所在的存储器区域的起始位置
DataSize	jpeg 文件的字节数
x0	显示器中位图左上角的 X 位置
y0	显示器中位图左上角的 Y 位置

表 37.3.2 GUI_JPEG_Draw 函数的相关形参描述

返回值：0 成功，!= 0 错误。

(2) GUI_JPEG_DrawEx 函数

在当前窗口中的指定位置绘制不必加载到内存的 jpeg 文件，其函数的原型如下源码所示：

```
int GUI_JPEG_DrawEx(GUI_GET_DATA_FUNC * pfGetData,
                    void * p,
                    int x0,
                    int y0);
```

该函数的相关形参如下表所示：

参数	描述
pfGetData	指向用于获取数据而调用的函数的指针

p	传递给由 pfGetData 指向的函数的 Void 指针
x0	显示器中位图左上角的 X 位置
y0	显示器中位图左上角的 Y 位置

表 37.3.3 GUI_JPEG_Draw 函数的相关形参描述

返回值：0 成功, != 0 错误。

(3) GUI_JPEG_DrawScaled 函数

在当前窗口中的指定位置通过缩放比例绘制已加载到内存的 jpeg 文件，其函数的原型如下源码所示：

```
int GUI_JPEG_DrawScaled(const void * pFileData, int DataSize,
                        int x0, int y0, int Num, int Denom);
```

该函数的相关形参如下表所示：

参数	描述
pFileData	指向 jpeg 文件所在的存储器区域的起始位置
DataSize	Jpeg 文件的字节数
x0	显示器中位图左上角的 X 位置
y0	显示器中位图左上角的 Y 位置
Num	用于缩放比例的分子
Denom	用于缩放比例的分母

表 37.3.4 GUI_JPEG_DrawScaled 函数的相关形参描述

返回值：0 成功, != 0 错误。

注意：该函数通过使用给定的分子和分母构建分数来缩放图像。例如，如果图像应缩减至 2/3 大小，则 Num 应为 2，Denom 应为 3。

(4) GUI_JPEG_DrawScaledEx 函数

在当前窗口中的指定位置通过缩放比例绘制不必加载到内存的 jpeg 文件，其函数的原型如下源码所示：

```
int GUI_JPEG_DrawScaledEx(GUI_GET_DATA_FUNC * pfGetData, void * p,
                          int x0, int y0, int Num, int Denom);
```

该函数的相关形参如下表所示：

参数	描述
pfGetData	指向用于获取数据而调用的函数的指针
p	传递给由 pfGetData 指向的函数的 void 指针
x0	显示器中位图左上角的 X 位置
y0	显示器中位图左上角的 Y 位置
Num	用于缩放比例的分子
Denom	用于缩放比例的分母

表 37.3.5 GUI_JPEG_DrawScaledEx 函数的相关形参描述

返回值：0 成功, != 0 错误。

注意：该函数通过使用给定的分子和分母构建分数来缩放图像。例如，如果图像应缩减至 2/3 大小，则 Num 应为 2，Denom 应为 3。

(5) GUI_JPEG_GetInfo 函数

利用已加载到内存的 jpeg 文件的相关信息填充 GUI_JPEG_INFO 结构，其函数的原型如下源码所示：

```
int GUI_JPEG_GetInfo(const void *pFileData, int DataSize, GUI_JPEG_INFO *pInfo);
```

该函数的相关形参如下表所示：

参数	描述
----	----

pFileData	指向 jpeg 文件所在的存储器区域的起始位置	
DataSize	Jpeg 文件的字节数	
pInfo	指向该函数要填充的 GUI_JPEG_INFO 结构。	
	XSize(int)	图像的 X 像素大小
	YSize(int)	图像的 Y 像素大小

表 37.3.6 GUI_JPEG_GetInfo 函数的相关形参描述

返回值：0 成功, != 0 错误。

(6) GUI_JPEG_GetInfoEx 函数

利用不必加载到内存的 jpeg 文件的相关信息填充 GUI_JPEG_INFO 结构，其函数的原型如下源码所示：

```
int GUI_JPEG_GetInfoEx( GUI_GET_DATA_FUNC *pfGetData, void *p,
                        GUI_JPEG_INFO *pInfo);
```

该函数的相关形参如下表所示：

参数	描述	
pfGetData	指向用于获取数据而调用的函数的指针	
p	传递给由 pfGetData 指向的函数的 void 指针	
pInfo	指向该函数要填充的 GUI_JPEG_INFO 结构。	
	XSize(int)	图像的 X 像素大小
	YSize(int)	图像的 Y 像素大小

表 37.3.7 GUI_JPEG_GetInfoEx 函数的相关形参描述

返回值：0 成功, != 0 错误。

37.4 JPEG 显示实验

37.4.1 硬件设计

1. 例程功能

本实验主要讲解 JPEG 图片显示实验，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数利用循环的形式加载 JPEG 图片，当用户按下 KEY0 按键时，我们会看到 LED1 灯由灭到亮，此时系统把当前界面进行截图并保存到 SD 卡指定路径下，直到 LED1 由亮到灭才表示截图结束；当用户按下 KEY_UP 按键时，系统会更新当前字库。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 36 JPEG 图片显示》。

注意：本实验不支持 Mini、精英、DMF407 开发板运行。

37.4.2 软件设计

37.4.2.1 程序流程图

本实验的程序流程图，如下图所示：

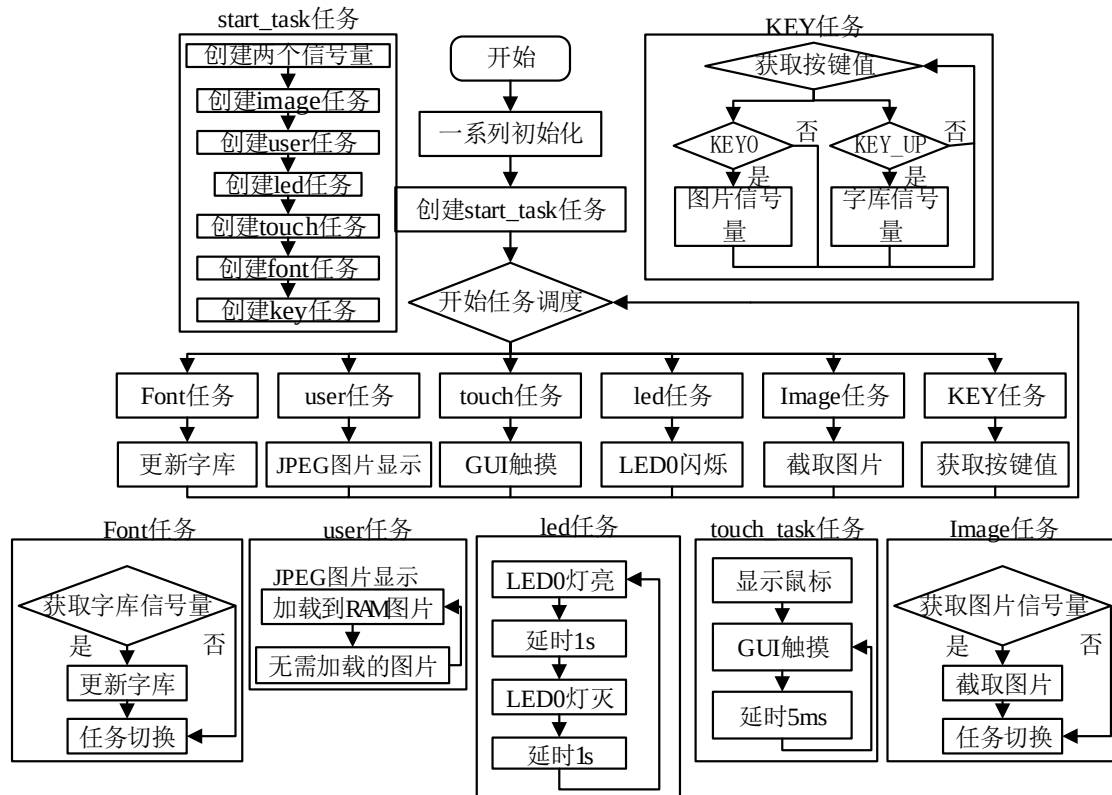


图 37.4.2.1.1 JPEG 图片显示实验流程图

37.4.2.2 程序解析

本实验完整工程为：**emWin 实验 36 JPEG 图片显示**。打开文件 `emwin_demo.c`，程序源码如下：

(1) key_task 任务函数

```

/**
 * @brief      key_task
 * @param      p_arg : 传入参数(未用到)
 * @retval     无
 */
void key_task(void *p_arg)
{
    OS_ERR err;
    p_arg = p_arg;
    uint8_t key;

    while(1)
    {
        key = key_scan(0);

        switch(key)
        {

```

```

        case KEY0_PRES:
            printf("发送截图信号量\r\n");
            OSSemPost (&IMAGE_SEM, OS_OPT_POST_1, &err); /* 发送截图信号量 */
            break;

        case WKUP_PRES:
            printf("更新字体信号量\r\n");
            OSSemPost (&FONT_SEM, OS_OPT_POST_1, &err); /* 发送更新字体信号量 */
            break;
    }

    OSTimeDlyHMSM(0,0,0,10,OS_OPT_TIME_PERIODIC,&err); /* 延时 10ms */
}
}

```

上述的源码，我们可以得知 KEY0_PRES 按键为释放截图的信号量，而 WKUP_PRES 按键为释放更新字体信号量。

(2) Font_task 任务函数

```

/**
 * @brief      font_task
 * @param      p_arg : 传入参数(未用到)
 * @retval     无
 */
void font_task(void *p_arg)
{
    OS_ERR err;
    p_arg = p_arg;

    while (1)
    {
        /* 请求截图信号量 */
        OSSemPend(&FONT_SEM, 0, OS_OPT_PEND_BLOCKING, 0, &err);
        GUI_COLOR forecolor, bkcolor;
        forecolor = GUI_GetColor(); /* 获取当前前景色 */
        bkcolor = GUI_GetBkColor(); /* 获取当前背景色 */
        OSSchedLock(&err);
        lcd_show_string(10,50,250,30,16,"Font Updataing,Please Wait...",RED);
        fonts_update_font(10,70,16,(uint8_t *) "0:", RED);
        lcd_show_string(10,50,280,30,16,"FontUpdata finshed,PleaseRestart!",RED);
        OSSchedUnlock(&err);
        GUI_SetColor(forecolor); /* 恢复前景色 */
        GUI_SetBkColor(bkcolor); /* 恢复背景色 */
        OSTimeDlyHMSM(0,0,0,10,OS_OPT_TIME_PERIODIC,&err); /* 延时 10ms */
    }
}

```

首先运行这个程序，必须获取一个信号量，该信号量在 key_task 任务中释放的，调用函数 GUI_GetColor 和函数 GUI_GetBkColor 获取更新字体之前的背景颜色和前景颜色，这样我们在更新字体之后再调用 GUI 的设置背景函数和设置前景函数这样可以使图片还原操作。更新字体必须调用操作系统的 OSSchedLock 调度锁，这样可以不被其他任务打扰字体更新操作，字体更新完毕必须调用 OSSchedUnlock 调度解锁函数，不然其他任务无法执行。

(3) Image_task 任务函数

```
/**
 * @brief      image_task
 * @param      p_arg : 传入参数(未用到)
 * @retval     无
 */
void image_task(void *p_arg)
{
    OS_ERR err;
    p_arg = p_arg;

    while (1)
    {
        /* 请求截图信 量 */
        OSSemPend(&IMAGE_SEM, 0, OS_OPT_PEND_BLOCKING, 0, &err);
        OSSchedLock(&err);
        LED1(0);
        create_bmppicture((uint8_t *) "0:/SCREENSHORT/屏幕截图 0.bmp", 0, 0,
                          lcddev.width, lcddev.height); /* 创建 BMP 截图 */
        LED1(1);
        OSSchedUnlock(&err);
        OSTimeDlyHMSM(0, 0, 0, 10, OS_OPT_TIME_PERIODIC, &err); /* 延时 10ms */
    }
}
```

image_task 任务和 font_task 任务是差不多的，主要也是获取信号量，开启任务调度锁和关闭任务锁，其中间的截图程序如下源码所示：

```
/**
 * @brief      此函数被 GUI_BMP_Serialize() 调用，用来向文件写入字节
 * @param      无
 * @retval     无
 */
static void _WriteByte2File(U8 Data, void *p)
{
    UINT nWritten;
    f_write((FIL *)p, &Data, 1, &nWritten);
}

/**
```

```

* @brief      生成一个 BMP 图片，保存到 SD 卡中，实现屏幕截图功能！
* @param      x0:创建 BMP 文件的 X 起始位置
* @param      y0:创建 BMP 文件的 Y 起始位置
* @param      Xsize:X 的大小
* @param      Ysize:Y 的大小
* @retval     无
*/

void create_bmppicture(uint8_t *filename, int x0, int y0, int Xsize, int Ysize)
{
    static FIL hFile;

    GUI_COLOR forecolor, bkcolor;
    forecolor = GUI_GetColor();    /* 获取当前前景色 */
    bkcolor = GUI_GetBkColor();    /* 获取当前背景色 */
    /* 创建一个文件，路径为 filename,如果文件原本就存在的话会被新建的文件覆盖掉! */
    f_open(&hFile, (const TCHAR *)filename, FA_READ | FA_WRITE |
        FA_CREATE_ALWAYS);
    GUI_BMP_SerializeEx(_WriteByte2File, x0, y0, Xsize, Ysize, &hFile);
    f_close(&hFile);              /* 关闭文件 */
    GUI_SetColor(forecolor);       /* 恢复前景色 */
    GUI_SetBkColor(bkcolor);       /* 恢复背景色 */
}

```

我们也知道，使用文件系统是三步走，打开文件—读写文件—关闭文件等操作。首先也是获取背景颜色和前景颜色，调用系统文件函数 `f_open` 打开文件的路径，调用 `emwin` 的 `GUI_BMP_SerializeEx` 绘制图片函数，该函数的回调函数调用函数 `f_write` 写入数据，然后关闭文件。

(4) JPEG 图片显示程序

在显示 JPEG 图片程序中有三个函数是非常重要的。

1. `emwin_jpeg_get_data` 函数，如下源码所示：

```

static int emwin_jpeg_get_data ( void *p, const uint8_t **ppData,
                                unsigned NumBytesReq,
                                uint32_t Off)
{
    static int readaddress = 0;
    FIL *phFile;
    UINT NumBytesRead;

    phFile = (FIL *)p;

    if (NumBytesReq > sizeof(jpegBuffer))
    {
        NumBytesReq = sizeof(jpegBuffer);
    }
}

```

```

/*移动指针到应该读取的位置 */
if (Off == 1)
{
    readaddress = 0;
}
else
{
    readaddress = Off;
}

f_lseek(phFile, readaddress);

/* 读取数据到缓冲区中 */
f_read(phFile, jpegBuffer, NumBytesReq, &NumBytesRead);

*ppData = (uint8_t *)jpegBuffer;
return NumBytesRead; /* 返回读取到的字节数 */
}
    
```

函数 `emwin_jpeg_get_data` 用于从外部存储器中获取 JPEG 图片数据，在函数 `emwin_displayjpegex` 显示 JPEG 图片的时候做为参数传递给 `GUI_JPEG_GetInfoEx`。

2. `emwin_displyjpeg` 函数，如下源码所示：

```

/**
 * @brief          在指定位置显示加载到 RAM 中的 JPEG 图片
 *                  JPEGFileName:图片在 SD 卡或者其他存储设备中的路径(需文件系统支持!)
 * @param          mode:显示模式
 *                  @arg      0 在指定位置显示，有参数 x,y 确定显示位置
 *                  @arg      1 在 LCD 中间显示图片，当选择此模式的时候参数 x,y 无效。
 * @param          x:图片左上角在 LCD 中的 x 轴位置(当参数 mode 为 1 时，此参数无效)
 * @param          y:图片左上角在 LCD 中的 y 轴位置(当参数 mode 为 1 时，此参数无效)
 * @param          member: 缩放比例的分子项
 * @param          denom:缩放比例的分母项
 * @retval         0 显示正常,其他 失败
 */
int emwin_displyjpeg(uint8_t *JPEGFileName, uint8_t mode, uint32_t x, uint32_t y,
                    int member, int denom)
{
    uint16_t bread;
    char *jpegbuffer;
    char result;
    int XSize, YSize;
    GUI_JPEG_INFO JpegInfo;
    float Xflag, Yflag;
    
```



```

/* 打开文件 */
result = f_open(&JPEGFile, (const TCHAR *)JPEGFileName, FA_READ);

/*文件打开错误或者文件大于 JPEGMEMORYSIZE*/
if ((result != FR_OK) || (JPEGFile.obj.objsize > JPEGMEMORYSIZE)) return 1;

jpegbuffer = mymalloc(SRAMEX, JPEGFile.obj.objsize); /* 申请内存 */

if (jpegbuffer == NULL) return 2;

/* 读取数据 */
result = f_read(&JPEGFile, jpegbuffer, JPEGFile.obj.objsize,
                (UINT *)&bread);

if (result != FR_OK) return 3;

/* 获取 JEGP 图片信息 */
GUI_JPEG_GetInfo(jpegbuffer, JPEGFile.obj.objsize, &JpegInfo);
XSize = JpegInfo.XSize; /* 获取 JPEG 图片的 X 轴大小 */
YSize = JpegInfo.YSize; /* 获取 JPEG 图片的 Y 轴大小 */

switch (mode)
{
    case 0: /*在指定位置显示图片*/
        if ((member == 1) && (denom == 1)) /*无需缩放, 直接绘制*/
        {
            /*在指定位置显示 JPEG 图片*/
            GUI_JPEG_Draw(jpegbuffer, JPEGFile.obj.objsize, x, y);
        }
        else /*否则图片需要缩放*/
        {
            GUI_JPEG_DrawScaled(jpegbuffer, JPEGFile.obj.objsize, x, y, member,
                                denom);
        }
        break;

    case 1: /* 在 LCD 中间显示图片 */
        if ((member == 1) && (denom == 1)) /* 无需缩放, 直接绘制 */
        {
            /* 在 LCD 中间显示图片 */
            GUI_JPEG_Draw(jpegbuffer, JPEGFile.obj.objsize, (lcddev.width -
                                                                XSize) / 2 - 1, (lcddev.height - YSize) / 2 - 1);
        }
}
    
```

```

else /* 否则图片需要缩放 */
{
    Xflag = (float)XSize * ((float)member / (float)denom);
    Yflag = (float)YSize * ((float)member / (float)denom);
    XSize = (lcddev.width - (int)Xflag) / 2 - 1;
    YSize = (lcddev.height - (int)Yflag) / 2 - 1;
    GUI_JPEG_DrawScaled(jpegbuffer, JPEGFile.obj.objsize, XSize,
                        YSize, member, denom);
}
break;
}

f_close(&JPEGFile); /* 关闭 JPEGFile 文件 */
myfree(SRAMEX, jpegbuffer); /* 释放内存 */
return 0;
}

```

函数 `emwin_displyjpeg` 用于显示 JPEG 图片，该函数首先将 JPEG 图片的所有数据加载到 RAM 中，然后调用 EMWIN 中的相应函数在 LCD 上显示加载到 RAM 中的 JPEG 图片，由于需要将整副 JPEG 图片加载到 RAM 中，所以需要大量的 RAM，但是显示速度快！此函数可以设置要显示的 JPEG 图片的位置和放大倍数。

3. `emwin_displayjpegex` 函数，如下源码所示：

```

/**
 * @brief      在指定位置显示无需加载到 RAM 中的 BMP 图片 (需文件系统支持！对于小 RAM，
              推荐使用此方法！)
              JPEGFileName: 图片在 SD 卡或者其他存储设备中的路径
 * @param      mode: 显示模式
              @arg      0 在指定位置显示，有参数 x, y 确定显示位置
              @arg      1 在 LCD 中间显示图片，当选择此模式的时候参数 x, y 无效。
 * @param      x: 图片左上角在 LCD 中的 x 轴位置 (当参数 mode 为 1 时，此参数无效)
 * @param      y: 图片左上角在 LCD 中的 y 轴位置 (当参数 mode 为 1 时，此参数无效)
 * @param      member: 缩放比例的分子项
 * @param      denom: 缩放比例的分母项
 * @retval     0 显示正常, 其他 失败
 */
int emwin_displayjpegex(uint8_t *JPEGFileName, uint8_t mode, uint32_t x,
                        uint32_t y, int member, int denom)
{
    char result;
    int XSize, YSize;
    float Xflag, Yflag;
    GUI_JPEG_INFO JpegInfo;

    /* 打开文件 */
    result = f_open(&JPEGFile, (const TCHAR *)JPEGFileName, FA_READ);

```

```

/* 文件打开错误 */
if (result != FR_OK)    return 1;
GUI_JPEG_GetInfoEx((GUI_GET_DATA_FUNC*) atk_JpegGetData, &JPEGFile,
                    &JpegInfo);
XSize = JpegInfo.XSize; /* JPEG 图片 X 大小 */
YSize = JpegInfo.YSize; /* JPEG 图片 Y 大小 */
switch (mode)
{
    case 0:    /*在指定位置显示图片*/
        if ((member == 1) && (denom == 1))    /*无需缩放, 直接绘制*/
        {
            /*在指定位置显示 BMP 图片*/
            GUI_JPEG_DrawEx((GUI_GET_DATA_FUNC *) atk_JpegGetData,
                            &JPEGFile, x, y);
        }
        else    /*否则图片需要缩放*/
        {
            GUI_JPEG_DrawScaledEx((GUI_GET_DATA_FUNC *) atk_JpegGetData,
                                   &JPEGFile, x, y, member, denom);
        }
        break;
    case 1:    /* 在 LCD 中间显示图片 */
        if ((member == 1) && (denom == 1)) /* 无需缩放, 直接绘制 */
        {
            /* 在 LCD 中间显示图片 */
            GUI_JPEG_DrawEx((GUI_GET_DATA_FUNC *) atk_JpegGetData,
                            &JPEGFile, (lcddev.width - XSize) / 2 - 1,
                            (lcddev.height - YSize) / 2 - 1);
        }
        else /* 否则图片需要缩放 */
        {
            Xflag = (float)XSize * ((float)member / (float)denom);
            Yflag = (float)YSize * ((float)member / (float)denom);
            XSize = (lcddev.width - (int)Xflag) / 2 - 1;
            YSize = (lcddev.height - (int)Yflag) / 2 - 1;
            GUI_JPEG_DrawScaledEx((GUI_GET_DATA_FUNC *) atk_JpegGetData,
                                   &JPEGFile, XSize, YSize, member, denom);
        }
        break;
}
f_close(&JPEGFile); /* 关闭 BMPFile 文件 */
return 0;
}
    
```

函数 `emwin_displayjpegex` 也是显示 JPEG 图片的，与函数 `emwin_displyjpeg` 不同的是，`emwin_displayjpegex` 不需要将 JPEG 图片加载到 RAM 中，可以直接从外部存储器中读取 JPEG 图片并显示，这种方法需要的 RAM 小，但是速度相比 `emwin_displyjpeg` 要慢，此函数也可以设置要显示的 JPEG 图片的位置和放大倍数。

37.4.3 下载验证

编译并下载到开发板中，LCD 轮流着按照指定的显示方法显示图片，下图是先将 JPEG 图片加载到 RAM 中，然后再显示出来，注意，图片扩大了 2 倍！



图 37.4.3.1 显示加载到 RAM 中的 JPEG 图片

图 37.4.3.1 是直接从 SD 卡中读取 JPEG 图片，并不需要事先将整幅 JPEG 读取到 RAM 中，下图中的图片缩小为原来的 1/2。

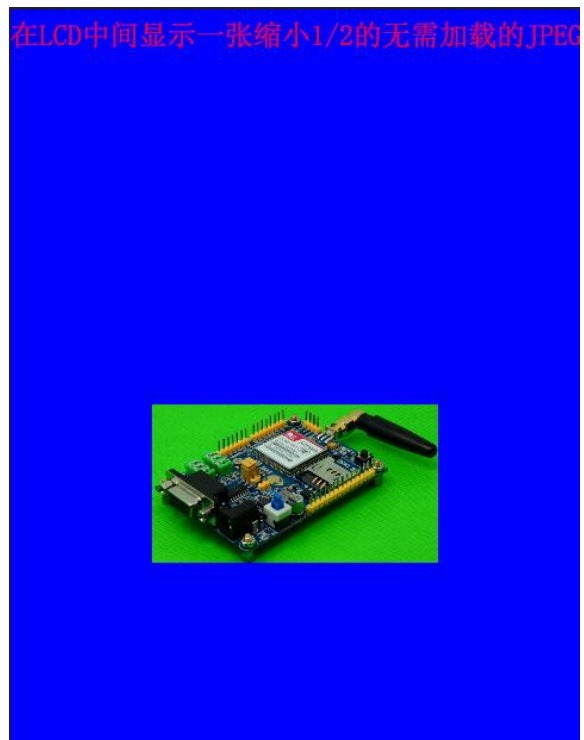


图 37.4.3.2 直接从 SD 卡读取显示 JPEG 图片

第三十八章 GIF 图片显示

GIF 图片用于跨数据网络传输，GIF 标准支持隔行扫描、透明、应用定义数据、动画以及原始文本渲染，emWin 将忽略原始文本或应用特定数据等不受支持的数据。GIF 文件使用 LZW (Lempel-Zif-Welch) 文件压缩方法来压缩图像数据，这种压缩方法运行起来不会丢失数据。输出图像与输入图像完全相同。本章就讲解如何使用 emWin 中的相关 API 函数在 LCD 上显示 GIF 图片。

本章分为以下几个部分：

38.1 GIF 文件支持

38.2 GIF 解码 API 函数

38.3 GIF 显示实验

38.1 GIF 文件支持

图形库首先对图形信息进行解码，在显示 GIF 图像的时候，其中解码流程将花费相当长的时间。如果在窗口管理器经常调用的回调函数中使用 GIF 文件，同样，解码过程会花费相当长的时间。通过使用存储设备可缩短计算时间，最好的方法是先将图像绘制到存储设备中，在这种情况下，将只进行一次解压缩。

emWin 的 GIF 解压缩过程大约需要 16K 字节动态分配的 RAM 进行解压缩，绘制图像后，将释放在用于解压缩的 RAM。

38.2 GIF 解码 API 函数

表 38.2.1 列出了关于 GIF 图片显示的 API 函数。

函数	描述
GUI_GIF_Draw()	绘制已加载到存储器的 GIF 文件的第一个图像
GUI_GIF_DrawEx()	绘制无需加载到存储器的 GIF 文件的第一个图像
GUI_GIF_DrawSub()	绘制已加载到存储器的 GIF 文件的给定子图像
GUI_GIF_DrawSubEx()	绘制无需加载到存储器的 GIF 文件的给定子图像
GUI_GIF_DrawSubScaled()	绘制已加载到存储器的带比例的 GIF 文件的给定子图像
GUI_GIF_DrawSubScaledEx()	绘制无需加载到存储器的带比例的 GIF 文件的给定子图像
GUI_GIF_GetComment()	返回已加载到存储器的 GIF 文件给定的注释
GUI_GIF_GetCommentEx()	返回无需加载到存储器的 GIF 文件的给定注释
GUI_GIF_GetImageInfo()	返回已加载到存储器的 GIF 文件的相关信息
GUI_GIF_GetImageInfoEx()	返回无需加载到存储器的 GIF 文件的相关信息
GUI_GIF_GetInfo()	返回已加载到存储器的 GIF 文件的相关信息
GUI_GIF_GetInfoEx()	返回无需加载到存储器的 GIF 文件的相关信息
GUI_GIF_GetXSize()	返回加载到存储器的位图的 X 大小
GUI_GIF_GetXSizeEx()	返回无需加载到存储器的位图的 X 大小
GUI_GIF_GetYSize()	返回加载到存储器的位图的 Y 大小
GUI_GIF_GetYSizeEx()	返回无需加载到存储器的位图的 Y 大小

表 38.2.1 GIF 文件 API 函数

(1) GUI_GIF_Draw 函数

在当前窗口中的指定位置绘制已加载到内存的 gif 文件的第一个图像，其函数的原型如下源码所示：

```
int GUI_GIF_Draw(const void * pGIF, U32 NumBytes, int x0, int y0);
```

该函数的相关形参如下表所示：

参数	描述
pGIF	指向 gif 文件所在的内存区域的起始位置
NumBytes	Gif 文件的字节数
x0	显示器中位图左上角的 X 位置
y0	显示器中位图左上角的 Y 位置

表 38.2.2 GUI_GIF_Draw 函数的相关形参描述

返回值：0 成功, != 0 错误.

(2) GUI_GIF_DrawEx 函数

在前窗口中的指定位置绘制不必加载到存储器的 gif 文件，其函数的原型如下源码所示：

```
int GUI_GIF_DrawEx(GUI_GET_DATA_FUNC * pfGetData, void * p, int x0, int y0);
```

该函数的相关形参如下表所示：

参数	描述
pfGetData	指向用于获取数据而调用的函数的指针
p	传递给由 pfGetData 指向的函数的 Void 指针
x0	显示器中位图左上角的 X 位置
y0	显示器中位图左上角的 Y 位置

表 38.2.3 GUI_GIF_DrawEx 函数的相关形参描述

返回值：0 成功, != 0 错误.

(3) GUI_GIF_DrawSub 函数

在当前窗口中的指定位置绘制已加载到存储器的 gif 文件的给定子图像，其函数的原型如下源码所示：

```
int GUI_GIF_DrawSub(const void * pGIF, U32 NumBytes,
                    int x0, int y0, int Index);
```

该函数的相关形参如下表所示：

参数	描述
pGIF	指向 gif 文件所在的存储器区域的起始位置
NumBytes	Gif 文件的字节数
x0	显示器中位图左上角的 X 位置
y0	显示器中位图左上角的 Y 位置
Index	要显示的子图像基于 0 的索引

表 38.2.4 GUI_GIF_DrawSub 函数的相关形参描述

返回值：0 成功, != 0 错误.

(4) GUI_GIF_DrawSubEx 函数

在当前窗口中的指定位置绘制不必加载到存储器的 gif 文件的给定子图像，其函数的原型如下源码所示：

```
int GUI_GIF_DrawSubEx(GUI_GET_DATA_FUNC * pfGetData,
                      void * p, int x0, int y0, int Index);
```

该函数的相关形参如下表所示：

参数	描述
pfGetData	指向用于获取数据而调用的函数的指针
p	传递给由 pfGetData 指向的函数的 Void 指针

x0	显示器中位图左上角的 X 位置
y0	显示器中位图左上角的 Y 位置
Index	要显示的子图像基于 0 的索引

表 38.2.5 GUI_GIF_DrawSubEx 函数的相关形参描述

返回值：0 成功, != 0 错误.

(5) GUI_GIF_DrawSubScaled 函数

在当前窗口中的指定位置绘制已加载到存储器的 gif 文件的给定子图像，其函数的原型如下源码所示：

```
int GUI_GIF_DrawSubScaled(const void * pGIF, U32 NumBytes,
                          int x0, int y0,
                          int Index, int Num, int Denom);
```

该函数的相关形参如下表所示：

参数	描述
pGIF	指向 gif 文件所在的存储器区域的起始位置
NumBytes	Gif 文件的字节数
x0	显示器中位图左上角的 X 位置
y0	显示器中位图左上角的 Y 位置
Index	要显示的子图像基于 0 的索引
Num	用于缩放比例的分子
Denom	用于缩放比例的分母

表 38.2.6 GUI_GIF_DrawSubScaled 函数的相关形参描述

返回值：0 成功, != 0 错误.

注意：该函数通过使用给定的分子和分母构建分数来缩放图像。例如，如果图像应缩减至 2/3 大小，则 Num 应为 2，Denom 应为 3。

(6) GUI_GIF_DrawSubScaledEx 函数

在当前窗口中的指定位置通过缩放比例绘制已加载到存储器的 gif 文件的给定子图像，其函数的原型如下源码所示：

```
int GUI_GIF_DrawSubScaledEx(GUI_GET_DATA_FUNC * pfGetData, void * p,
                             int x0, int y0,
                             int Index, int Num, int Denom);
```

该函数的相关形参如下表所示：

参数	描述
pfGetData	指向用于获取数据而调用的函数的指针
p	传递给由 pfGetData 指向的函数的 void 指针
x0	显示器中位图左上角的 X 位置
y0	显示器中位图左上角的 Y 位置
Index	要显示的子图像基于 0 的索引
Num	用于缩放比例的分子
Denom	用于缩放比例的分母

表 38.2.7 GUI_GIF_DrawSubScaledEx 函数的相关形参描述

返回值：0 成功, != 0 错误.

注意：该函数通过使用给定的分子和分母构建分数来缩放图像。例如，如果图像应缩减至 2/3 大小，则 Num 应为 2，Denom 应为 3。

(7) GUI_GIF_GetComment 函数

获取已加载到存储器的 GIF 图像的给定注释，其函数的原型如下源码所示：


```
int GUI_GIF_GetComment(const void * pGIF, U32 NumBytes,
                      U8 * pBuffer, int MaxSize, int Index);
```

该函数的相关形参如下表所示:

参数	描述
pGIF	指向 gif 文件所在的存储器区域的起始位置
NumBytes	Gif 文件的字节数
pBuffer	指向填充了注释的缓冲区
MaxSize	缓冲区的大小
Index	要返回的注释基于 0 的索引

表 38.2.8 GUI_GIF_GetComment 函数的相关形参描述

返回值: 0 成功, != 0 错误.

(8) GUI_GIF_GetCommentEx 函数

获取不必加载到存储器的 GIF 图像的给定注释, 其函数的原型如下源码所示:

```
int GUI_GIF_GetCommentEx(GUI_GET_DATA_FUNC * pfGetData, void * p,
                        U8 * pBuffer, int MaxSize, int Index);
```

该函数的相关形参如下表所示:

参数	描述
pfGetData	指向用于获取数据而调用的函数的指针
p	传递给由 pfGetData 指向的函数的 void 指针
pBuffer	指向填充了注释的缓冲区
MaxSize	缓冲区的大小
Index	要返回的注释基于 0 的索引

表 38.2.9 GUI_GIF_GetCommentEx 函数的相关形参描述

返回值: 0 成功, != 0 错误.

(9) GUI_GIF_GetImageInfo 函数

获取已加载到存储器的 GIF 文件给定子图像的相关信息, 其函数的原型如下源码所示:

```
int GUI_GIF_GetImageInfo(const void * pGIF,
                        U32 NumBytes, GUI_GIF_IMAGE_INFO * pInfo,
                        int Index);
```

该函数的相关形参如下表所示:

参数	描述
pGIF	指向 gif 文件所在的存储器区域的起始位置
NumBytes	Gif 文件的字节数
pInfo	指向该函数要填充的 GUI_GIF_IMAGE_INFO 结构
Index	子图像基于 0 的索引

表 38.2.10 GUI_GIF_GetImageInfo 函数的相关形参描述

返回值: 0 成功, != 0 错误.

GUI_GIF_IMAGE_INFO 结构的成员如下表所示:

数据类型	成员	描述
Int	xPos	上次所绘制图像的 X 位置
Int	yPos	上次所绘制图像的 Y 位置
Int	xSize	上次所绘制图像的 X 大小
Int	ySize	上次所绘制图像的 Y 大小
Int	Delay	图像应在电影中显示的时间 (1/100 秒)

表 38.2.11 GUI_GIF_IMAGE_INFO 结构的成员描述

(10) GUI_GIF_GetImageInfoEx 函数

获取无需加载到存储器的 GIF 文件给定子图像的相关信息，其函数的原型如下源码所示：

```
int GUI_GIF_GetImageInfoEx(GUI_GET_DATA_FUNC * pfGetData,
                           void * p,
                           GUI_GIF_IMAGE_INFO * pInfo,
                           int Index);
```

该函数的相关形参如下表所示：

参数	描述
pfGetData	指向用于获取数据而调用的函数的指针
p	传递给由 pfGetData 指向的函数的 void 指针
pInfo	指向该函数要填充的 GUI_GIF_IMAGE_INFO 结构
Index	子图像基于 0 的索引

表 38.2.12 GUI_GIF_GetImageInfoEx 函数的相关形参描述

返回值：0 成功, != 0 错误.

(11) GUI_GIF_GetInfo 函数

获取已加载到存储器的给定 GIF 文件的子图像的信息结构，包含子图像的大小和数量等相关信息，其函数的原型如下源码所示：

```
int GUI_GIF_GetInfo(const void * pGIF,
                   U32 NumBytes,
                   GUI_GIF_INFO * pInfo);
```

该函数的相关形参如下表所示：

参数	描述
pGIF	指向 gif 文件所在的存储器区域的起始位置
NumBytes	Gif 文件的字节数
pInfo	指向该函数要填充的 GUI_GIF_INFO 结构。
	XSize 图像的 X 像素大小
	YSize 图像的 Y 像素大小
	YSize 文件中的子图像数

表 38.2.13 GUI_GIF_GetInfo 函数的相关形参描述

(12) GUI_GIF_GetInfoEx 函数

返回无需加载到存储器的给定 GIF 文件的子图像的信息结构，包含子图像的大小和数量等相关信息，其函数的原型如下源码所示：

```
int GUI_GIF_GetInfoEx(GUI_GET_DATA_FUNC * pfGetData, void * p,
                      GUI_GIF_INFO * pInfo);
```

该函数的相关形参如下表所示：

参数	描述
pfGetData	指向用于获取数据而调用的函数的指针
p	传递给由 pfGetData 指向的函数的 void 指针
pInfo	指向该函数要填充的 GUI_GIF_INFO 结构。
	XSize 图像的 X 像素大小
	YSize 图像的 Y 像素大小
	YSize 文件中的子图像数

表 38.2.14 GUI_GIF_GetInfoEx 函数的相关形参描述

返回值：0 成功, != 0 错误.

(13) GUI_GIF_GetXSize 函数

获取已加载到存储器的指定 GIF 图像的 X 大小，其函数的原型如下源码所示：

```
int GUI_GIF_GetXSize(const void * pGIF);
```

该函数的相关形参如下表所示：

参数	描述
pGIF	指向 gif 文件所在的存储器区域的起始位置

表 38.2.15 GUI_GIF_GetXSize 函数的相关形参描述

返回值：GIF 图像的 X 大小。

(14) GUI_GIF_GetXSizeEx 函数

获取无需加载到存储器的 GIF 图像的 X 大小，其函数的原型如下源码所示：

```
int GUI_GIF_GetXSizeEx(GUI_GET_DATA_FUNC * pfGetData, void * p);
```

该函数的相关形参如下表所示：

参数	描述
pfGetData	指向用于获取数据而调用的函数的指针
p	传递给由 pfGetData 指向的函数的 Void 指针

表 38.2.16 GUI_GIF_GetXSizeEx 函数的相关形参描述

返回值：GIF 图像的 X 大小。

(15) GUI_GIF_GetYSize 函数

返回已加载到存储器的指定 GIF 图像的 Y 大小，其函数的原型如下源码所示：

```
int GUI_GIF_GetYSize(const void * pGIF);
```

该函数的相关形参如下表所示：

参数	描述
pGIF	指向 gif 文件所在的存储器区域的起始位置

表 38.2.17 GUI_GIF_GetYSize 函数的相关形参描述

返回值：GIF 图像的 Y 大小。

(16) GUI_GIF_GetYSizeEx 函数

返回无需加载到存储器的 GIF 图像的 Y 大小，其函数的原型如下源码所示：

```
int GUI_GIF_GetYSizeEx(GUI_GET_DATA_FUNC * pfGetData, void * p);
```

该函数的相关形参如下表所示：

参数	描述
pfGetData	指向用于获取数据而调用的函数的指针
p	传递给由 pfGetData 指向的函数的 Void 指针

表 38.2.18 GUI_GIF_GetYSizeEx 函数的相关形参描述

返回值：GIF 图像的 Y 大小。

38.3 GIF 显示实验

38.3.1 硬件设计

1. 例程功能

本实验主要讲解 GIF 图片显示实验，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数利用循环的形式加载 GIF 图片，当用户按下 KEY0 按键时，我们会看到 LED1 灯由灭到亮，此时系统把当前界面进行截图并保存到 SD 卡指定路径下，直到 LED1 由亮到灭才表示截图结束；当用户按下 KEY_UP 按键时，系统会更新当前字库。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 37 GIF 图片显示》。

注意：本实验不支持 Mini、DMF407 开发板运行。

38.3.2 软件设计

38.3.2.1 程序流程图

本实验的程序流程图，如下图所示：

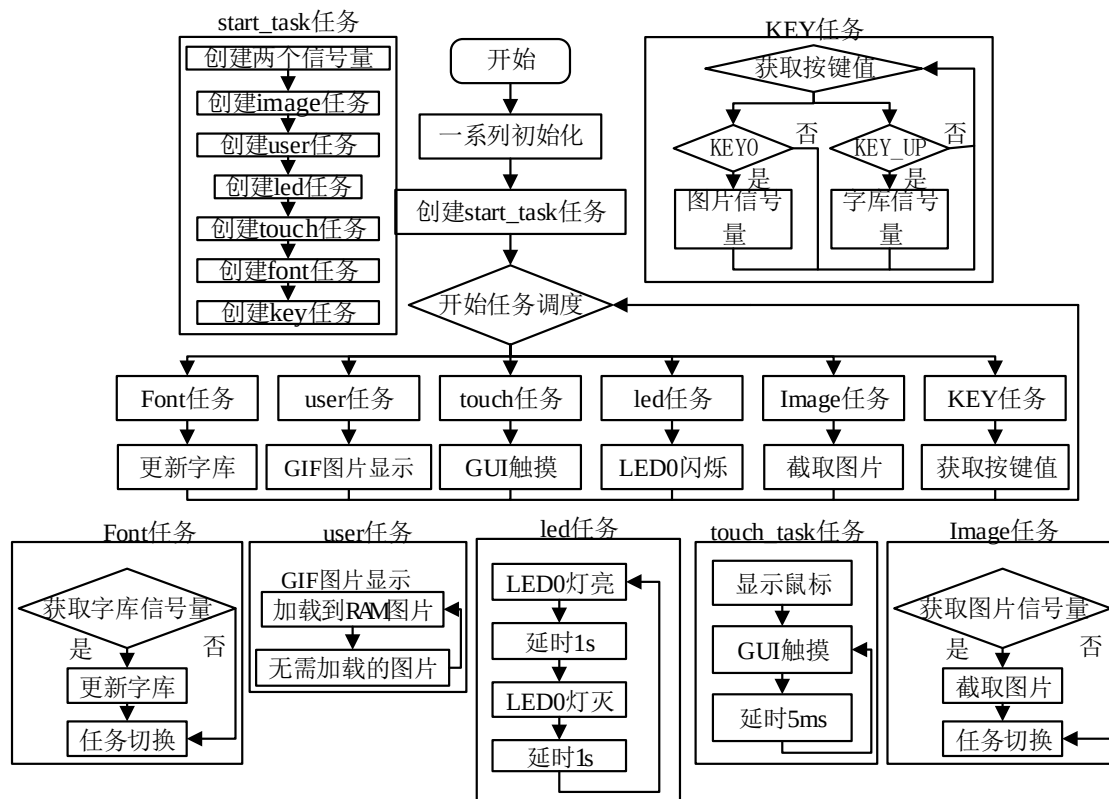


图 38.3.2.1.1 GIF 显示实验流程图

38.3.2.2 程序解析

本实验完整工程为：emWin 实验 37 GIF 图片显示，打开文件 emwin_demo.c，程序源码如下：注意，由于精英版没有外部 SRAM，所以不能先将 bmp 图片加载到 ram 中，然后在 LCD 上显示，因此也就没有提供相应的 API 函数。

(1) key_task 任务函数

```
/**
 * @brief      key_task
 * @param      p_arg : 传入参数(未用到)
```

```

    * @retval      无
    */
void key_task(void *p_arg)
{
    OS_ERR err;
    p_arg = p_arg;
    uint8_t key;

    while(1)
    {
        key = key_scan(0);

        switch(key)
        {
            case KEY0_PRES:
                printf("发送截图信号量\r\n");
                OSSemPost (&IMAGE_SEM, OS_OPT_POST_1, &err);/* 发送截图信号量 */
                break;

            case WKUP_PRES:
                printf("更新字体信号量\r\n");
                OSSemPost (&FONT_SEM, OS_OPT_POST_1, &err);/* 发送更新字体信号量 */
                break;

        }

        OSTimeDlyHMSM(0,0,0,10,OS_OPT_TIME_PERIODIC,&err);/* 延时 10ms */
    }
}

```

上述的源码，我们可以得知 KEY0_PRES 按键为释放截图的信号量，而 WKUP_PRES 按键为释放更新字体信号量。

(2) Font_task 任务函数

```

/**
 * @brief      font_task
 * @param      p_arg : 传入参数(未用到)
 * @retval      无
 */
void font_task(void *p_arg)
{
    OS_ERR err;
    p_arg = p_arg;

    while (1)
    {

```

```

    /* 请求截图信号量 */
    OSSemPend(&FONT_SEM, 0, OS_OPT_PEND_BLOCKING, 0, &err);
    GUI_COLOR forecolor, bkcolor;
    forecolor = GUI_GetColor(); /* 获取当前前景色 */
    bkcolor = GUI_GetBkColor(); /* 获取当前背景色 */
    OSSchedLock(&err);
    lcd_show_string(10,50,250,30,16,"Font Updataing,Please Wait...",RED);
    fonts_update_font(10,70,16,(uint8_t *)"0:", RED);
    lcd_show_string(10,50,280,30,16,"Font Updata finshed,Please
                        Restart!",RED);
    OSSchedUnlock(&err);
    GUI_SetColor(forecolor); /* 恢复前景色 */
    GUI_SetBkColor(bkcolor); /* 恢复背景色 */
    OSTimeDlyHMSM(0,0,0,10,OS_OPT_TIME_PERIODIC,&err); /* 延时 10ms */
}
}

```

首先运行这个程序，必须获取一个信号量，该信号量在 `key_task` 任务中释放的，调用函数 `GUI_GetColor` 和函数 `GUI_GetBkColor` 获取更新字体之前的背景颜色和前景颜色，这样我们在更新字体之后再调用 `GUI` 的设置背景函数和设置前景函数这样可以使图片还原操作。更新字体必须调用操作系统的 `OSSchedLock` 调度锁，这样可以不被其他任务打扰字体更新操作，字体更新完毕必须调用 `OSSchedUnlock` 调度解锁函数，不然其他任务无法执行。

(3) Image_task 任务函数

```

/**
 * @brief      image_task
 * @param      p_arg : 传入参数(未用到)
 * @retval     无
 */
void image_task(void *p_arg)
{
    OS_ERR err;
    p_arg = p_arg;

    while (1)
    {
        /* 请求截图信号量 */
        OSSemPend(&IMAGE_SEM, 0, OS_OPT_PEND_BLOCKING, 0, &err);
        OSSchedLock(&err);
        LED1(0);
        create_bmppicture((uint8_t *)"0:/SCREENSHORT/屏幕截图 0.bmp"
                        ,0,0,lcddev.width,lcddev.height); /* 创建 BMP 截图 */
        LED1(1);
        OSSchedUnlock(&err);
        OSTimeDlyHMSM(0,0,0,10,OS_OPT_TIME_PERIODIC,&err); /* 延时 10ms */
    }
}

```

```

    }
}

```

Image_task 任务和 Font_task 任务是差不多的，主要也是获取信号量，开启任务调度锁和关闭任务锁，其中间的截图程序如下源码所示：

```

/**
 * @brief      此函数被 GUI_BMP_Serialize() 调用，用来向文件写入字节
 * @param      无
 * @retval     无
 */
static void _WriteByte2File(U8 Data, void *p)
{
    UINT nWritten;
    f_write((FIL *)p, &Data, 1, &nWritten);
}

/**
 * @brief      生成一个 BMP 图片，保存到 SD 卡中，实现屏幕截图功能！
 * @param      x0:创建 BMP 文件的 X 起始位置
 * @param      y0:创建 BMP 文件的 Y 起始位置
 * @param      Xsize:X 的大小
 * @param      Ysize:Y 的大小
 * @retval     无
 */
void create_bmppicture(uint8_t *filename, int x0, int y0, int Xsize, int Ysize)
{
    static FIL hFile;

    GUI_COLOR forecolor, bkcolor;
    forecolor = GUI_GetColor();    /* 获取当前前景色 */
    bkcolor = GUI_GetBkColor();    /* 获取当前背景色 */
    /* 创建一个文件，路径为 filename，如果文件原本就存在的话会被新建的文件覆盖掉！ */
    f_open(&hFile, (const TCHAR *)filename, FA_READ | FA_WRITE |
        FA_CREATE_ALWAYS);
    GUI_BMP_SerializeEx(_WriteByte2File, x0, y0, Xsize, Ysize, &hFile);
    f_close(&hFile);              /* 关闭文件 */
    GUI_SetColor(forecolor);       /* 恢复前景色 */
    GUI_SetBkColor(bkcolor);       /* 恢复背景色 */
}

```

我们也知道，使用文件系统是三步走，打开文件—读写文件—关闭文件等操作。首先也是获取背景颜色和前景颜色，调用系统文件函数 `f_open` 打开文件的路径，调用 `emwin` 的 `GUI_BMP_SerializeEx` 绘制图片函数，该函数的回调函数调用函数 `f_write` 写入数据，然后关闭文件。

(4) GIF 显示程序

1. emwin_gif_get_data 函数

```
/**
 * @brief      用于将数据指针设置为至少具有有效数据位置
 * @param      p:          指向应用程序定义的数据
 * @param      NumBytesReq: 请求字节数
 * @param      ppData:      数据指针的指针。这个指针应该设置为有效位置。
 * @param      StartOfFile: 如果该标志为 1，则数据指针应设置为数据流的开始
 * @retval     可用数据字节数。
 */
static int emwin_gif_get_data(void *p, const U8 **ppData, unsigned NumBytesReq,
                               U32 Off)
{
    static int readaddress = 0;
    FIL *phFile;
    UINT NumBytesRead;

    phFile = (FIL *)p;

    if (NumBytesReq > sizeof(gifBuffer))
    {
        NumBytesReq = sizeof(gifBuffer);
    }

    /* 移动指针到应该读取的位置 */
    if (Off == 1) readaddress = 0;
    else readaddress = Off;

    f_lseek(phFile, readaddress);

    /* 读取数据到缓冲区中 */
    f_read(phFile, gifBuffer, NumBytesReq, &NumBytesRead);

    *ppData = (U8 *)gifBuffer;
    return NumBytesRead; /* 返回读取到的字节数 */
}
```

函数 `atk_GifGetData` 用于从外部存储器中获取 JPEG 图片数据，在函数 `emwin_displaygifex` 显示 GIF 图片的时候做为参数传递给 `GUI_GIF_GetImageInfoEx` 函数。

2. emwin_displaygif 函数

```
/**
 * @brief      在指定位置显示加载到 RAM 中的 GIF 图片
 *             GIFFileName: 图片在 SD 卡或者其他存储设备中的路径(需文件系统支持!)
```



```

* @param      mode:显示模式
    @arg      0 在指定位置显示, 有参数 x,y 确定显示位置
    @arg      1 在 LCD 中间显示图片, 当选择此模式的时候参数 x,y 无效
* @param      x:图片左上角在 LCD 中的 x 轴位置 (当参数 mode 为 1 时, 此参数无效)
* @param      y:图片左上角在 LCD 中的 y 轴位置 (当参数 mode 为 1 时, 此参数无效)
* @param      member: 缩放比例的分子项
* @param      denom:缩放比例的分母项
* @retval     0 显示正常,其他 失败
*/
int emwin_displaygif(char *GIFFileName, uint8_t mode, uint32_t x, uint32_t y,
                    int member, int denom)
{
    int i;
    uint16_t bread;
    char *gifbuffer;
    char result;
    int XSize, YSize;
    GUI_GIF_INFO GifInfo;
    GUI_GIF_IMAGE_INFO ImageInfo;
    float Xflag, Yflag;

    /* 打开文件 */
    result = f_open(&GIFFile, (const TCHAR *)GIFFileName, FA_READ);

    /* 文件打开错误或者文件大于 JPEGMEMORYSIZE */
    if ((result != FR_OK) || (GIFFile.obj.objsize > GIFMEMORYSIZE)) return 1;

    gifbuffer = mymalloc(SRAMEX, GIFFile.obj.objsize);

    if (gifbuffer == NULL) return 2;

    /* 读取数据 */
    result = f_read(&GIFFile, gifbuffer, GIFFile.obj.objsize, (UINT *)&bread);

    if (result != FR_OK) return 3;

    /* 获取 GIF 图片信息 */
    GUI_GIF_GetInfo(gifbuffer, GIFFile.obj.objsize, &GifInfo);

    XSize = GifInfo.xSize; /* 获取 GIF 图片的 X 轴大小 */
    YSize = GifInfo.ySize; /* 获取 GIF 图片的 Y 轴大小 */

    switch (mode)

```

```

{
    case 0: /* 在指定位置显示图片 */
        if ((member == 1) && (denom == 1)) /* 无需缩放，直接绘制 */
        {
            /* 在指定位置显示 JPEG 图片 */
            for (i = 0; i < GifInfo.NumImages; i++)
            {
                GUI_GIF_DrawSub(gifbuffer, GIFFFile.obj.objsize, x, y, i);
                GUI_GIF_GetImageInfo(gifbuffer, GIFFFile.obj.objsize,
                                    &ImageInfo, i);
                GUI_Delay(ImageInfo.Delay ? ImageInfo.Delay * 10 : 100);
            }
        }
        else /* 否则图片需要缩放 */
        {
            for (i = 0; i < GifInfo.NumImages; i++)
            {
                GUI_GIF_DrawSubScaled(gifbuffer, GIFFFile.obj.objsize, x, y, i,
                                     member, denom);
                GUI_GIF_GetImageInfo(gifbuffer, GIFFFile.obj.objsize,
                                    &ImageInfo, i);
                GUI_Delay(ImageInfo.Delay ? ImageInfo.Delay * 10 : 100);
            }
        }
        break;

    case 1: /* 在 LCD 中间显示图片 */
        if ((member == 1) && (denom == 1)) /* 无需缩放，直接绘制 */
        {
            /* 在 LCD 中间显示图片 */
            for (i = 0; i < GifInfo.NumImages; i++)
            {
                GUI_GIF_DrawSub(gifbuffer, GIFFFile.obj.objsize,
                                (lcddev.width - XSize) / 2 - 1,
                                (lcddev.height - YSize) / 2 - 1, i);
                GUI_GIF_GetImageInfo(gifbuffer, GIFFFile.obj.objsize,
                                    &ImageInfo, i);
                GUI_Delay(ImageInfo.Delay ? ImageInfo.Delay * 10 : 100);
            }
        }
        else /* 否则图片需要缩放 */
        {
            Xflag = (float)XSize * ((float)member / (float)denom);

```

```

        Yflag = (float)YSize * ((float)member / (float)denom);
        XSize = (lcddev.width - (int)Xflag) / 2 - 1;
        YSize = (lcddev.height - (int)Yflag) / 2 - 1;

        for (i = 0; i < GifInfo.NumImages; i++)
        {
            GUI_GIF_DrawSubScaled(gifbuffer, GIFFFile.obj.objsize, XSize,
                                   YSize, i, member, denom);

            GUI_GIF_GetImageInfo(gifbuffer, GIFFFile.obj.objsize,
                                   &ImageInfo, i);

            GUI_Delay(ImageInfo.Delay ? ImageInfo.Delay * 10 : 100);
        }
    }
    break;
}

f_close(&GIFFFile);          /* 关闭 JPEGFile 文件 */
myfree(SRAMEX, gifbuffer);    /* 释放内存 */
return 0;
}

```

函数 `emwin_displaygif` 用于显示 GIF 图片，该函数首先将 GIF 图片的所有数据加载到 RAM 中，然后调用 `emWin` 中的相应函数在 LCD 上显示加载到 RAM 中的 GIF 图片，由于需要将整副 GIF 图片加载到 RAM 中，所以需要大量的 RAM，但是显示速度快！此函数可以设置要显示的 GIF 图片的位置和放大倍数。

3. `emwin_displaygifex` 函数

```

/**
 * @brief 指定位置显示无需加载到RAM中的GIF图片(需文件系统支持！对于小RAM，推荐使用此方法！)
 *        GIFFFileName:图片在SD卡或者其他存储设备中的路径
 * @param mode:显示模式
 *        @arg      0 在指定位置显示，有参数x,y确定显示位置
 *        @arg      1 在LCD中间显示图片，当选择此模式的时候参数x,y无效
 * @param x:图片左上角在LCD中的x轴位置(当参数mode为1时，此参数无效)
 * @param y:图片左上角在LCD中的y轴位置(当参数mode为1时，此参数无效)
 * @param member:  缩放比例的分子项
 * @param denom:缩放比例的分母项
 * @retval 0 显示正常,其他 失败
 */
int emwin_displaygifex(char *GIFFFileName, uint8_t mode, uint32_t x, uint32_t y,
                       int member,
                       int denom)
{
    char result;
    int i;

```

```

int XSize, YSize;
float Xflag, Yflag;
GUI_GIF_INFO GifInfo;
GUI_GIF_IMAGE_INFO ImageInfo;
/* 打开文件 */
result = f_open(&atk_GIFFile, (const TCHAR *)GIFFFileName, FA_READ);

/* 文件打开错误 */
if (result != FR_OK)    return 1;

GUI_GIF_GetInfoEx(GifGetData, &GIFFile, &GifInfo);
XSize = GifInfo.xSize; /* GIF 图片 X 大小 */
YSize = GifInfo.ySize; /* GIF 图片 Y 大小 */

switch (mode)
{
    case 0: /* 在指定位置显示图片 */
        if ((member == 1) && (denom == 1)) /* 无需缩放, 直接绘制 */
        {
            /* 在指定位置显示 BMP 图片 */
            for (i = 0; i < GifInfo.NumImages; i++)
            {
                GUI_GIF_DrawSubEx(atk_GifGetData, &GIFFile, x, y, i);
                GUI_GIF_GetImageInfoEx(atk_GifGetData, &GIFFile, &ImageInfo, i);
                GUI_Delay(ImageInfo.Delay ? ImageInfo.Delay * 10 : 100);
            }
        }
        else /* 否则图片需要缩放 */
        {
            for (i = 0; i < GifInfo.NumImages; i++)
            {
                GUI_GIF_DrawSubScaledEx(emwin_gif_get_data, &GIFFile, x, y, i,
                                         member, denom);
                GUI_GIF_GetImageInfoEx(emwin_gif_get_data, &GIFFile,
                                         &ImageInfo, i);
                GUI_Delay(ImageInfo.Delay ? ImageInfo.Delay * 10 : 100);
            }
        }
        break;

    case 1: /* 在 LCD 中间显示图片 */
        if ((member == 1) && (denom == 1)) /* 无需缩放, 直接绘制 */
        {

```

```

/* 在 LCD 中间显示图片 */
for (i = 0; i < GifInfo.NumImages; i++)
{
    GUI_GIF_DrawSubEx(emwin_gif_get_data, &GIFFile,
                      (lcddev.width - XSize)
                      / 2 - 1, (lcddev.height - YSize) / 2 - 1, i);
    GUI_GIF_GetImageInfoEx(emwin_gif_get_data,
                           &GIFFile, &ImageInfo, i);
    GUI_Delay(ImageInfo.Delay ? ImageInfo.Delay * 10 : 100);
}
}
else /* 否则图片需要缩放 */
{
    Xflag = (float)XSize * ((float)member / (float)denom);
    Yflag = (float)YSize * ((float)member / (float)denom);
    XSize = (lcddev.width - (int)Xflag) / 2 - 1;
    YSize = (lcddev.height - (int)Yflag) / 2 - 1;
    for (i = 0; i < GifInfo.NumImages; i++)
    {
        GUI_GIF_DrawSubScaledEx(emwin_gif_get_data,
                                &GIFFile, XSize, YSize,
                                i, member, denom);
        GUI_GIF_GetImageInfoEx(emwin_gif_get_data,
                                &GIFFile, &ImageInfo, i);
        GUI_Delay(ImageInfo.Delay ? ImageInfo.Delay * 10 : 100);
    }
}
break;
}

f_close(&GIFFile); /* 关闭 GIFFile 文件 */
return 0;
}

```

函数 `emwin_displaygifex` 也是显示 GIF 图片的，与函数 `ewin_displygif` 不同的是，`emwin_displaygifex` 不需要将 GIF 图片加载到 RAM 中，可以直接从外部存储器中读取 GIF 图片并显示，这种方法需要的 RAM 小，但是速度相比 `displygif` 要慢，此函数也可以设置要显示的 GIF 图片的位置和放大倍数。

38.3.3 下载验证

编译并下载到开发板中，LCD 轮流着按照指定的显示方法显示图片，图 38.3.3.1 是先将 GIF 图片加载到 RAM 中，然后在指定的位置显示出来。

在显示一些较为复杂的 GIF 图片的时候效果非常差，可以非常明显的看出刷屏的过程！

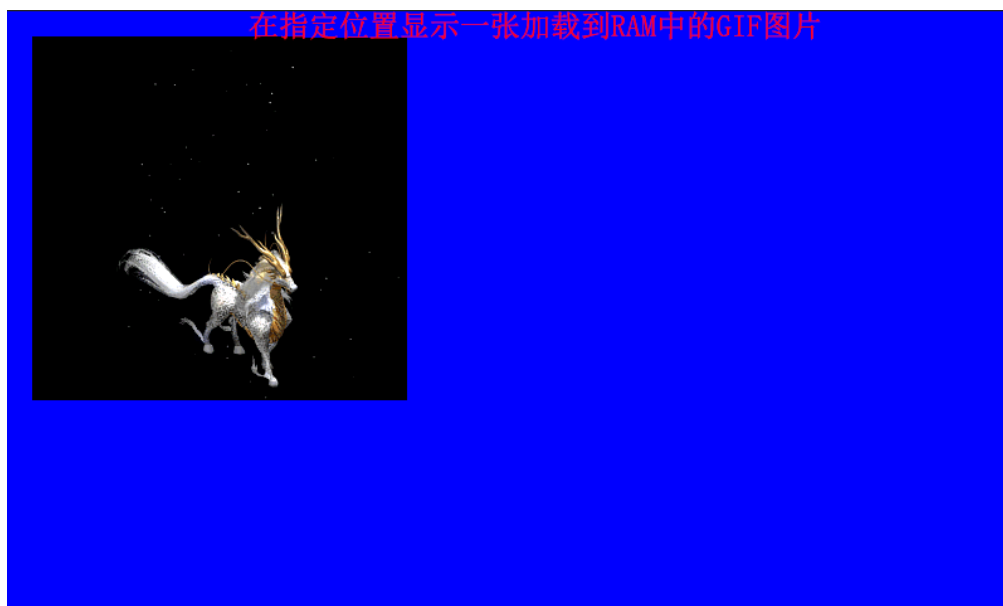


图 38.3.3.1 显示加载到 RAM 中的 GIF 图片

图 38.3.3.2 是直接从 SD 卡中读取 GIF 图片，并不需要事先将整幅 GIF 读取到 RAM 中，图 38.3.3.2 中的图片扩大了 2 倍。



图 38.3.3.2 直接从 SD 卡读取显示 GIF 图片

第三十九章 PNG 图片显示

PNG（可移植的网络图形）格式是一种图像格式，它利用非专利的数据压缩方法提供无损的数据压缩和 Alpha 混合，其目的是试图替代 GIF 和 TIFF 文件格式，同时增加一些 GIF 文件格式所不具备的特性。PNG 用来存储灰度图像时，灰度图像的深度可多到 16 位，存储彩色图像时，彩色图像的深度可多到 48 位，并且还可存储多到 16 位的 α 通道数据。emWin 是支持 PNG 解码的，本章我们就讲解一下如何在 LCD 上显示 PNG 图片。

本章分为以下几个部分：

39.1 基础工程准备

39.2 RAM 使用要求

39.3 PNG 解码 API 函数

39.4 PNG 显示实验

39.1 基础工程准备

emWin 源码中默认是不带 PNG 图片解码库的，我们需要在 segger 官方下载 PNG 解码库：libpng，下载地址为：www.segger.com/link/emwin_png.zip 获得，我们已经下载好放到光盘中，路径为：6、软件资料→3、EMWIN 学习资料→emwin_png，下载下来有多个版本的 libpng，如下图所示，由于 emWin 是 6.10 版本的，所以 libpng 库我们选择 emWin_V6.14 版本的。

 V528	2017/12/13 19:16	文件夹
 V530	2017/12/13 19:16	文件夹
 V532	2017/12/13 19:16	文件夹
 V534	2017/12/13 19:16	文件夹
 V536	2017/12/13 19:16	文件夹
 V538	2017/12/13 19:16	文件夹
 V540	2017/12/13 19:16	文件夹
 V542	2017/12/13 19:16	文件夹
 V544	2017/12/13 19:16	文件夹
 V546	2017/12/13 19:17	文件夹
 V548	2018/6/13 17:42	文件夹
 V550	2019/6/12 16:42	文件夹
 V614	2020/6/3 17:15	文件夹

图 39.1.1 libpng 库

将 libpng 库添加到工程中，如下图所示：

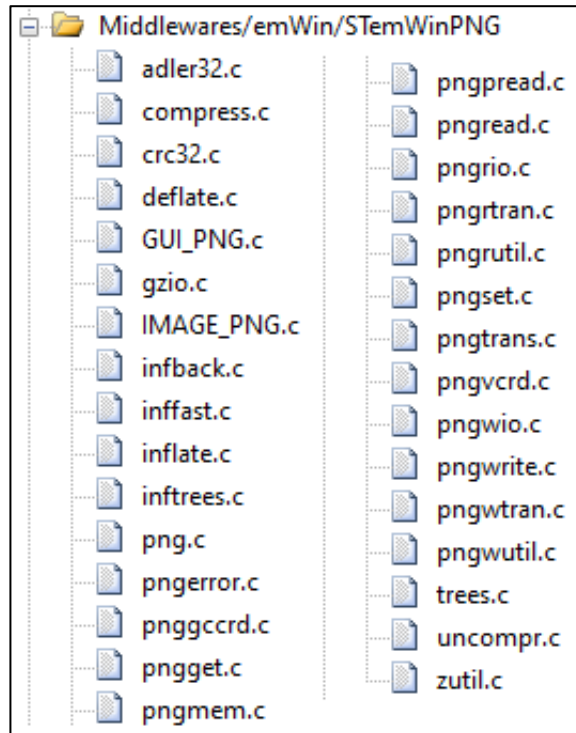


图 39.1.2 向工程中添加 libpng 库

39.2 RAM 使用要求

PNG 解压缩大约需要 21 Kb RAM 用于与图像大小无关的解压缩和依赖大小的字节量。RAM 要求可按以下方式计算：

$$\text{大约 RAM 要求} = (\text{X-Size} + 1) * \text{Y 大小} * 4 + 21\text{Kbytes}$$

这里我们给 emWin 分配 500k 的外部 SRAM 内存，打开 GUIConf.c，下图就是为 emWin 分配的内存大小。

```
#define USE_EXRAM ..... 1 ..... /* 不使用外部 SRAM */
#define GUI_NUMBYTES (500*1024) ... /* 设置 EMWIN 内存大小 */
#define GUI_BLOCKSIZE 0X80 ..... /* 块大小 */
```

图 39.2.1 emWin 内存大小

39.3 PNG 解码 API 函数

表 39.3.1 列出了关于 PNG 图片显示的 API 函数。

函数	描述
GUI_PNG_Draw()	绘制已加载到存储器的 PNG 文件
GUI_PNG_DrawEx()	绘制无需加载到存储器的 PNG 文件
GUI_PNG_GetXSize()	返回加载到存储器的位图的 X 大小
GUI_PNG_GetXSizeEx()	返回无需加载到存储器的位图的 X 大小
GUI_PNG_GetYSize()	返回加载到存储器的位图的 Y 大小
GUI_PNG_GetYSizeEx()	返回无需加载到存储器的位图的 Y 大小

表 39.3.1 PNG 文件 API 函数

(1) GUI_PNG_Draw 函数

在当前窗口中的指定位置绘制已加载到存储器的 png 文件，其函数的原型如下源码所示：

```
int GUI_PNG_Draw(const void * pFileData, int FileSize, int x0, int y0);
```


该函数的相关形参如下表所示：

参数	描述
pFileData	指向 png 文件所在的存储器区域的起始位置
FileSize	png 文件的字节数
x0	显示器中位图左上角的 X 位置
y0	显示器中位图左上角的 Y 位置

表 39.3.2 GUI_PNG_Draw 函数的相关形参描述

返回值：0 表示成功，非 0 表示该函数失败（当前的实施始终返回 0）。

(2) GUI_PNG_DrawEx 函数

在当前窗口中的指定位置绘制不必加载到存储器的 png 文件，其函数的原型如下源码所示：

```
int GUI_PNG_Draw(const void * pFileData, int FileSize, int x0, int y0);
```

该函数的相关形参如下表所示：

参数	描述
pfGetData	指向用于获取数据而调用的函数的指针
p	传递给由 pfGetData 指向的函数的 void 指针
x0	显示器中位图左上角的 X 位置
y0	显示器中位图左上角的 Y 位置

表 39.3.3 GUI_PNG_DrawEx 函数的相关形参描述

返回值：0 表示成功，非 0 表示该函数失败。

(3) GUI_PNG_GetXSize 函数

获取已加载到存储器的指定 PNG 图像的 X 大小，其函数的原型如下源码所示：

```
int GUI_PNG_GetXSize(const void * pFileData, int FileSize);
```

该函数的相关形参如下表所示：

参数	描述
pFileData	指向 png 文件所在的存储器区域的起始位置
FileSize	png 文件的字节数

表 39.3.4 GUI_PNG_GetXSize 函数的相关形参描述

返回值：PNG 图像的 X 大小。

(4) GUI_PNG_GetXSizeEx 函数

获取无需加载到存储器的 PNG 图像的 X 大小，其函数的原型如下源码所示：

```
int GUI_PNG_GetXSizeEx(GUI_GET_DATA_FUNC * pfGetData,
                        void * p);
```

该函数的相关形参如下表所示：

参数	描述
pfGetData	指向用于获取数据而调用的函数的指针
p	传递给由 pfGetData 指向的函数的 void 指针

表 39.3.5 GUI_PNG_GetXSizeEx 函数的相关形参描述

返回值：PNG 图像的 X 大小。

(5) GUI_PNG_GetYSize 函数

获取已加载到存储器的指定 PNG 图像的 Y 大小，其函数的原型如下源码所示：

```
int GUI_PNG_GetYSize(const void * pFileData,
                     int FileSize);
```

该函数的相关形参如下表所示：

参数	描述
pFileData	指向 png 文件所在的存储器区域的起始位置

FileSize	png 文件的字节数
----------	------------

表 39.3.6 GUI_PNG_GetYSize 函数的相关形参描述

返回值：PNG 图像的 Y 大小。

(6) GUI_PNG_GetYSizeEx 函数

获取无需加载到存储器的 PNG 图像的 Y 大小，其函数的原型如下源码所示：

```
int GUI_PNG_GetYSizeEx(GUI_GET_DATA_FUNC * pfGetData, void * p);
```

该函数的相关形参如下表所示：

参数	描述
pfGetData	指向用于获取数据而调用的函数的指针
p	传递给由 pfGetData 指向的函数的 void 指针

表 39.3.7 GUI_PNG_GetYSizeEx 函数的相关形参描述

返回值：PNG 图像的 Y 大小。

39.4 PNG 显示实验

39.4.1 硬件设计

1. 例程功能

本实验主要讲解 GIF 图片显示实验，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数利用循环的形式加载 PNG 图片，当用户按下 KEY0 按键时，我们会看到 LED1 灯由灭到亮，此时系统把当前界面进行截图并保存到 SD 卡指定路径下，直到 LED1 由亮到灭才表示截图结束；当用户按下 KEY_UP 按键时，系统会更新当前字库。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 38 PNG 图片显示》。

注意：本实验不支持 Mini、精英、DMF407 开发板运行。

39.4.2 软件设计

39.4.2.1 程序流程图

本实验的程序流程图，如下图所示：

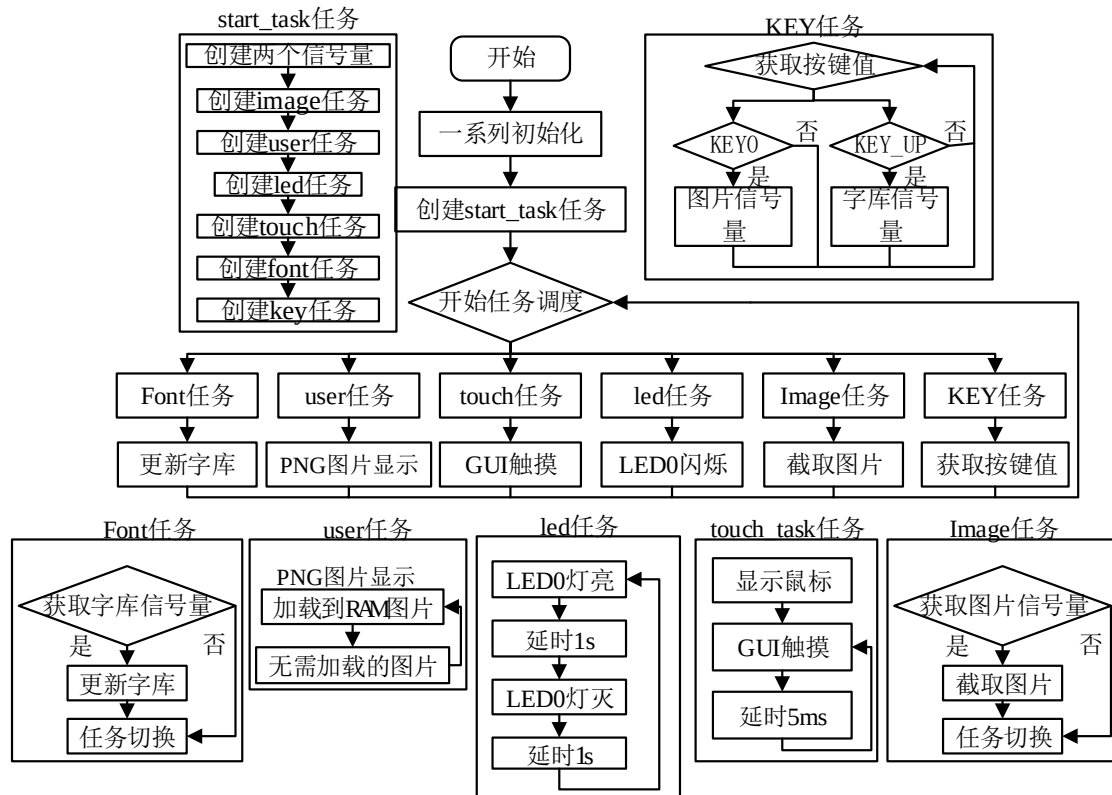


图 39.4.2.1.1 PNG 图片显示实验流程图

39.4.2.2 程序解析

本实验完整工程为：**emWin 实验 27 PNG 图片显示**，打开文件 emwin_demo.c，程序源码如下：

(1) key_task 任务函数

```

/**
 * @brief      key_task
 * @param      p_arg : 传入参数(未用到)
 * @retval     无
 */
void key_task(void *p_arg)
{
    OS_ERR err;
    p_arg = p_arg;
    uint8_t key;
    while(1)
    {
        key = key_scan(0);

        switch(key)
        {
            case KEY0_PRES:
        }
    }
}
    
```

```

        printf("发送截图信号量\r\n");
        OSSemPost (&IMAGE_SEM, OS_OPT_POST_1, &err);    /* 发送截图信号量 */
        break;

    case WKUP_PRES:
        printf("更新字体信号量\r\n");
        OSSemPost (&FONT_SEM, OS_OPT_POST_1, &err); /* 发送更新字体信号量 */
        break;
    }

    OSTimeDlyHMSM(0,0,0,10,OS_OPT_TIME_PERIODIC,&err); /* 延时 10ms */
}
}

```

上述的源码，我们可以得知 KEY0_PRES 按键为释放截图的信号量，而 WKUP_PRES 按键为释放更新字体信号量。

(2) Font_task 任务

```

/**
 * @brief      font_task
 * @param      p_arg : 传入参数(未用到)
 * @retval     无
 */
void font_task(void *p_arg)
{
    OS_ERR err;
    p_arg = p_arg;

    while (1)
    {
        /* 请求截图信号量 */
        OSSemPend(&FONT_SEM, 0, OS_OPT_PEND_BLOCKING, 0, &err);
        GUI_COLOR forecolor, bkcolor;
        forecolor = GUI_GetColor(); /* 获取当前前景色 */
        bkcolor = GUI_GetBkColor(); /* 获取当前背景色 */
        OSSchedLock(&err);
        lcd_show_string(10,50,250,30,16,"Font Updataing,Please Wait...",RED);
        fonts_update_font(10,70,16,(uint8_t *)"0:", RED);
        lcd_show_string(10,50,280,30,16,"FontUpdata finshed,PleaseRestart",RED);
        OSSchedUnlock(&err);
        GUI_SetColor(forecolor); /* 恢复前景色 */
        GUI_SetBkColor(bkcolor); /* 恢复背景色 */
        OSTimeDlyHMSM(0,0,0,10,OS_OPT_TIME_PERIODIC,&err); /* 延时 10ms */
    }
}

```

首先运行这个程序，必须获取一个信号量，该信号量在 key_task 任务中释放的，调用函数 GUI_GetColor 和函数 GUI_GetBkColor 获取更新字体之前的背景颜色和前景颜色，这样我们在更新字体之后再调用 GUI 的设置背景函数和设置前景函数这样可以使图片还原操作。更新字体必须调用操作系统的 OSSchedLock 调度锁，这样可以不被其他任务打扰字体更新操作，字体更新完毕必须调用 OSSchedUnlock 调度解锁函数，不然其他任务无法执行。

(3) Image_task 任务

```
/**
 * @brief      image_task
 * @param      p_arg : 传入参数(未用到)
 * @retval     无
 */
void image_task(void *p_arg)
{
    OS_ERR err;
    p_arg = p_arg;

    while (1)
    {
        /* 请求截图信量 */
        OSSemPend(&IMAGE_SEM, 0, OS_OPT_PEND_BLOCKING, 0, &err);
        OSSchedLock(&err);
        LED1(0);
        create_bmppicture((uint8_t *) "0:/SCREENSHORT/屏幕截图
        0.bmp", 0, 0, lcddev.width, lcddev.height); /* 创建 BMP 截图 */
        LED1(1);
        OSSchedUnlock(&err);
        OSTimeDlyHMSM(0, 0, 0, 10, OS_OPT_TIME_PERIODIC, &err); /* 延时 10ms */
    }
}
```

image_task 任务和 Font_task 任务是差不多的，主要也是获取信号量，开启任务调度锁和关闭任务锁，其中间的截图程序如下源码所示：

```
/**
 * @brief      此函数被 GUI_BMP_Serialize() 调用，用来向文件写入字节
 * @param      无
 * @retval     无
 */
static void _WriteByte2File(U8 Data, void *p)
{
    UINT nWritten;
    f_write((FIL *)p, &Data, 1, &nWritten);
}

/**
```

```

* @brief      生成一个 BMP 图片，保存到 SD 卡中，实现屏幕截图功能！
* @param      x0:创建 BMP 文件的 X 起始位置
* @param      y0:创建 BMP 文件的 Y 起始位置
* @param      Xsize:X 的大小
* @param      Ysize:Y 的大小
* @retval     无
*/

void create_bmppicture(uint8_t *filename, int x0, int y0, int Xsize, int Ysize)
{
    static FIL hFile;

    GUI_COLOR forecolor, bkcolor;
    forecolor = GUI_GetColor();    /* 获取当前前景色 */
    bkcolor = GUI_GetBkColor();    /* 获取当前背景色 */
    /* 创建一个文件，路径为 filename,如果文件原本就存在的话会被新建的文件覆盖掉！ */
    f_open(&hFile, (const TCHAR *)filename, FA_READ | FA_WRITE |
        FA_CREATE_ALWAYS);
    GUI_BMP_SerializeEx(_WriteByte2File, x0, y0, Xsize, Ysize, &hFile);
    f_close(&hFile);                /* 关闭文件 */
    GUI_SetColor(forecolor);        /* 恢复前景色 */
    GUI_SetBkColor(bkcolor);       /* 恢复背景色 */
}

```

我们也知道，使用文件系统是三步走，打开文件—读写文件—关闭文件等操作。首先也是获取背景颜色和前景颜色，调用系统文件函数 `f_open` 打开文件的路径，调用 `emwin` 的 `GUI_BMP_SerializeEx` 绘制图片函数，该函数的回调函数调用函数 `f_write` 写入数据，然后关闭文件。

(4) PNG 显示程序

1. `emwin_png_get_data` 函数

```

/**
* @brief      用于将数据指针设置为至少具有有效数据位置一个有效字节
* @param      p:            指向应用程序定义的数据
* @param      NumBytesReq: 请求字节数
* @param      ppData:       数据指针的指针。这个指针应该设置为有效位置
* @param      StartOfFile: 如果此标志为 1，则数据指针应设置为数据流的开始
* @retval     可用数据字节数
*/

static int emwin_png_get_data(void *p,
                                const U8 **ppData,
                                unsigned NumBytesReq,
                                U32 Off)
{
    static int readaddress = 0;
    FIL *phFile;
    U8 *pData;

```

```

UINT NumBytesRead;

pData = (U8 *)*ppData;
phFile = (FIL *)p;

/* 移动指针到应该读取的位置 */
if (Off == 1) readaddress = 0;
else readaddress = Off;

f_lseek(phFile, readaddress);

/* 读取数据到缓冲区中 */
f_read(phFile, pData, NumBytesReq, &NumBytesRead);

return NumBytesRead; /* 返回读取到的字节数 */
}

```

函数 `emwin_png_get_data` 用于从外部存储器中获取 PNG 图片数据，在函数 `emwin_displaypngex` 显示 PNG 图片的时候做为参数传递给 `GUI_PNG_DrawEx` 和 `GUI_PNG_GetXSizeEx` 这两个函数。

2. emwin_displaypng 函数

```

/**
 * @brief      在指定位置显示加载到 RAM 中的 GIF 图片
 *              GIFFileName: 图片在 SD 卡或者其他存储设备中的路径 (需文件系统支持!)
 * @param      mode: 显示模式
 *              @arg      0 在指定位置显示, 有参数 x, y 确定显示位置
 *              @arg      1 在 LCD 中间显示图片, 当选择此模式的时候参数 x, y 无效
 * @param      x: 图片左上角在 LCD 中的 x 轴位置 (当参数 mode 为 1 时, 此参数无效)
 * @param      y: 图片左上角在 LCD 中的 y 轴位置 (当参数 mode 为 1 时, 此参数无效)
 * @param      member: 缩放比例的分子项
 * @param      denom: 缩放比例的分母项
 * @retval     0 显示正常, 其他 失败
 */
int emwin_displaypng(char *PNGFileName, uint8_t mode, uint32_t x, uint32_t y)
{
    uint16_t bread;
    char *pngbuffer;
    char result;
    int XSize, YSize;
    /* 打开文件 */
    result = f_open(&PNGFile, (const TCHAR *)PNGFileName, FA_READ);

    /* 文件打开错误或者文件大于 JPEGMEMORYSIZE */
    if ((result != FR_OK) || (PNGFile.obj.objsize > PNGMEMORYSIZE)) return 1;

```

```

pngbuffer = mymalloc(SRAMEX, PNGFile.obj.objsize);

if (pngbuffer == NULL) return 2;
/* 读取数据 */
result = f_read(&PNGFile, pngbuffer, PNGFile.obj.objsize, (UINT *)&bread);

if (result != FR_OK) return 3;
/* 获取 PNG 图片的 x 轴大小 */
XSize = GUI_PNG_GetXSize(pngbuffer, PNGFile.obj.objsize);
/* 获取 PNG 图片的 y 轴大小 */
YSize = GUI_PNG_GetYSize(pngbuffer, PNGFile.obj.objsize);

switch (mode)
{
    case 0: /* 在指定位置显示图片 */
        GUI_PNG_Draw(pngbuffer, PNGFile.obj.objsize, x, y); /* 显示 PNG 图片 */
        break;

    case 1: /* 在 LCD 中间显示图片 */
        GUI_PNG_Draw(pngbuffer, PNGFile.obj.objsize,
                      (lcddev.width - XSize) / 2 - 1,
                      (lcddev.height - YSize) / 2 - 1);
        break;
}

f_close(&PNGFile); /* 关闭 PNGFile 文件 */
myfree(SRAMEX, pngbuffer);
return 0;
}

```

emwin_displaypng 函数用于显示 PNG 图片，该函数首先将 PNG 图片的所有数据加载到 RAM 中，然后调用 emWin 中的相应函数在 LCD 上显示加载到 RAM 中的 PNG 图片，由于需要将整副 PNG 图片加载到 RAM 中，所以需要大量的 RAM，但是显示速度快！此函数可以设置要显示的 PNG 图片的位置。

3. emwin_displaypngex 函数

```

/**
 * @brief      指定位置显示无需加载到 RAM 中的 GIF 图片(需文件系统支持！对于小 RAM，
               GIFFileName:图片在 SD 卡或者其他存储设备中的路径
 * @param      mode:显示模式
 *               @arg      0 在指定位置显示，有参数 x,y 确定显示位置
 *               @arg      1 在 LCD 中间显示图片，当选择此模式的时候参数 x,y 无效
 * @param      x:图片左上角在 LCD 中的 x 轴位置(当参数 mode 为 1 时，此参数无效)
 * @param      y:图片左上角在 LCD 中的 y 轴位置(当参数 mode 为 1 时，此参数无效)

```



```

* @param      member:  缩放比例的分子项
* @param      denom:缩放比例的分母项
* @retval     0 显示正常,其他 失败
*/
int emwin_displaypngex(char *PNGFileName, uint8_t mode, uint32_t x, uint32_t y)
{
    char result;
    int XSize, YSize;
    /* 打开文件 */
    result = f_open(&PNGFile, (const TCHAR *)PNGFileName, FA_READ);

    /* 文件打开错误 */
    if (result != FR_OK) return 1;

    XSize = GUI_PNG_GetXSizeEx(emwin_png_get_data, &PNGFile); /* PNG 图片 x 大小 */
    YSize = GUI_PNG_GetYSizeEx(emwin_png_get_data, &PNGFile); /* PNG 图片 y 大小 */

    switch (mode)
    {
        case 0: /* 在指定位置显示图片 */
            GUI_PNG_DrawEx(emwin_png_get_data, &PNGFile, x, y);
            break;

        case 1: /* 在 LCD 中间显示图片 */
            GUI_PNG_DrawEx(emwin_png_get_data, &PNGFile,
                (lcddev.width - XSize) / 2 - 1, (lcddev.height - YSize) / 2 - 1);
            break;
    }

    f_close(&PNGFile); /* 关闭 PNGFile 文件 */
    return 0;
}

```

函数 emwin_displaypngex 也是显示 PNG 图片的，与函数 emwin_displaypng 不同的是，emwin_displaypngex 不需要将 PNG 图片加载到 RAM 中，可以直接从外部存储器中读取 GIF 图片并显示，这种方法需要的 RAM 小，但是速度相比 emwin_displaypng 函数要慢，此函数也可以设置要显示的 PNG 图片的位置。

39.4.3 下载验证

编译并下载到开发板中，LCD 轮流着按照指定的显示方法显示图片，图 39.4.3.1 是先将 PNG 图片加载到 RAM 中，然后在指定的位置显示出来。



图 39.4.3.1 显示加载到 RAM 中的 PNG 图片

图 39.4.3.2 为直接从 SD 卡中读取 PNG 图片然后直接显示在 LCD 上的。

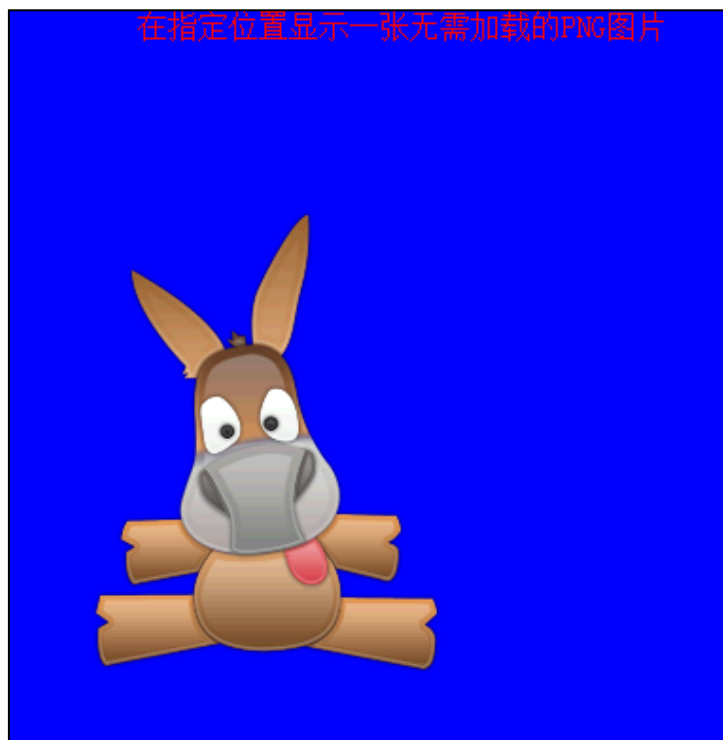


图 39.4.3.2 直接显示 PNG 图片

第四十章 软键盘

有时候我们需要输入一些信息，比如编辑文本、数字和字符等，在电脑上我们可以使用键盘输入，手机的话可以使用手机自带的软键盘。但是 emWin 并没有给我们提供软键盘控件，所以我们需要自己编写实现一个软键盘输入法。本章就讲解如何利用 emWin 现有的这些控件来设计一个软键盘，实现中文、英文大小写、数字、字符等输入。

本章分为以下几个部分：

40.1 软键盘组成

40.2 软键盘设计

40.3 软键盘实验

40.1 软键盘组成

因为设计的软键盘涉及到字母大小写输入、数字输入、中文输入和符号输入，所以我们可以将整个软键盘拆分为四个分键盘，分别为字母输入键盘、数字输入键盘、中文输入键盘和符号输入键盘，如下图所示：

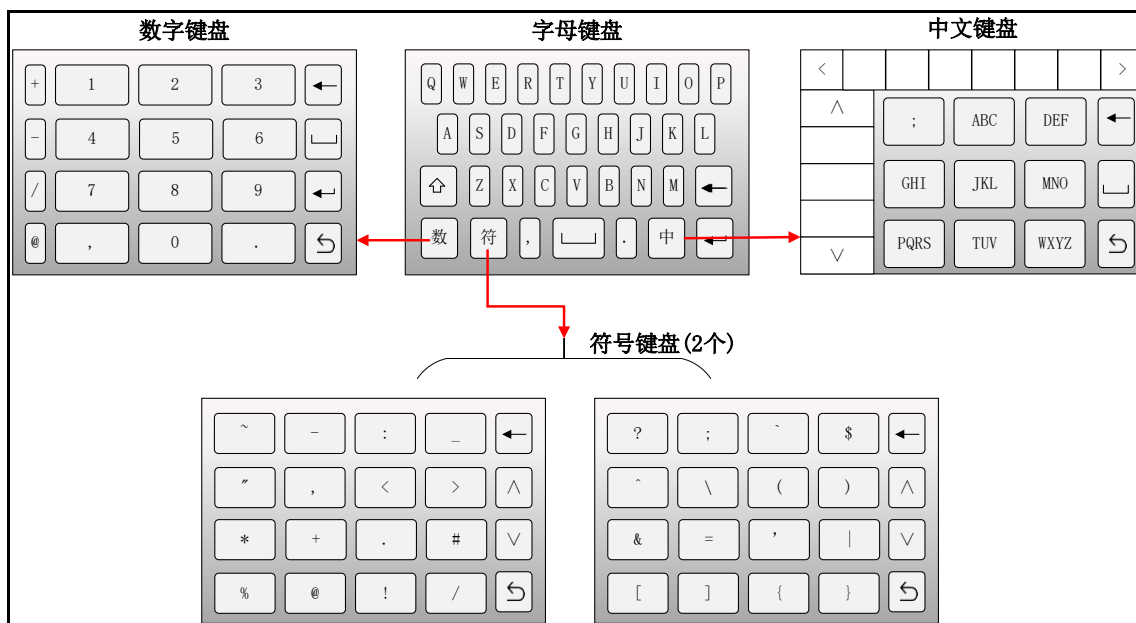


图 40.1.1 软键盘组成

从图 40.1.1 中就能清晰的看出软键盘的组成是由 5 个分键盘组成的，其中符号键盘有两个，因为符号太多了。字母键盘是主键盘，即打开软键盘以后默认显示出来的，其他三种键盘均可从主键盘进入，点击相应的按钮即可。每个分键盘均有返回键“↶”，用于返回主键盘。那么每个分键盘的组成部分是什么呢？BUTTON 控件！每个分键盘都是有很多个 BUTTON 按钮构成的。这样将一个庞大的软键盘拆分以后是不是思路就清晰多了，剩下的就是一个键盘一个键盘的去实现，设置每个键盘 BUTTON 按钮的功能，注意！中文键盘使用的是九宫格输入法。

40.2 软键盘设计

40.2.1 软键盘数据结构

Segger 官方给我们提供了一个叫做 SKINNING_Notepad 的例程，此例程实现了一个可以输入英文字符的软键盘，主键盘就是参考这个例程改的。打开工程 **emWin 实验 39 软键盘实验** 中的文件 `emwin_demo.h` 和 `emwin_demo.c`，`emwin_demo.h` 中有定义了一个枚举类型和一个结构体如下源码所示：

```
/* 键盘类型，包括英文键盘，数字键盘，符号键盘，九宫格中文输入法键盘 */
typedef enum
{
    ENGLISH_KEYPAD,          /* 英文键盘*/
    NUMBER_KEYPAD,           /* 数字键盘*/
    SIGN_KEYPAD,             /* 符号键盘*/
    CHINESE_KEYPAD,          /* 九宫格中文输入法键盘*/
    KEYPAD_NUM               /* 键盘数量*/
} keypad_type;

/* keypad 配置结构体*/
typedef struct
{
    keypad_type padtype;     /* 键盘类型*/
    uint16_t xpos;          /* 软键盘 x 坐标*/
    uint16_t ypos;          /* 软键盘 y 坐标*/
    uint16_t width;         /* 软键盘宽度*/
    uint16_t height;        /* 软键盘高度*/
    WM_HWIN hKeypad;        /* 软键盘窗口句柄*/
    WM_HWIN EngButton_Handle[35]; /* 英文键盘上按钮句柄*/
    WM_HWIN NumButton_Handle[20]; /* 数字键盘上按钮句柄*/
    WM_HWIN SignButton_Handle[20]; /* 符号键盘上按钮句柄*/
    WM_HWIN ChineseButton_Handle[25]; /* 中文键盘上按钮句柄*/
    uint8_t inputstr[7];    /* 输入字符串，最大 6 个字符+结束符*/
    uint8_t inputlen;       /* 输入长度*/
    uint8_t cur_index;      /* 当前拼音索引*/
    uint8_t result_num;     /* 拼音匹配数*/
    uint8_t pypagenum;      /* 拼音页数*/
    uint8_t pynowpage;      /* 当前选中的拼音页*/
    uint8_t dispagenum;     /* 汉字显示总页数*/
    uint8_t dispagecur;     /* 当前汉字显示页 */
    uint8_t *disstr;        /* 横向显示的字符串指针 */
    uint8_t *cur_hz;        /* 当前选中的汉字*/
    uint8_t hznum;          /* 选中的拼音对应的汉字数量*/
    uint8_t disbuf[6][3];   /* 横向按钮缓存*/
    uint8_t *butname[6];    /* 按钮名字*/
    uint8_t sta;            /* 记录输入法状态*/
}
```

```
/* bit7: 0,第一次进入中文输入法;1,前面已经进入了中文输入法*/
uint8_t LowerCase; /* 0 大写;1 小写*/
uint8_t signpad_flag; /* 符号键盘索引*/
uint8_t signpad_num; /* 符号键盘的数量*/
} KEYPADStructTypeDef;
```

其中枚举类型 keypad_type 用来标记键盘类型的，结构体 KEYPADStructTypeDef 用来记录软键盘的各个参数。文件 keypad.h 还有一些宏定义和位图数组，宏定义都是一些用户自定义消息、颜色和键盘上按钮的位置信息，位图数组是要在 BUTTON 上显示的位图文件，比如大小写切换键、回车键、删除键和返回键。

emwin_demo.c 文件中定义了一个结构体 BUTTON_DATA 来保存分键盘上每个 BUTTON 的位置和字符信息，BUTTON_DATA 结构体如下：

```
typedef struct
{
    int      xPos; /* 按钮 x 位置 */
    int      yPos; /* 按钮 y 位置 */
    int      xSize; /* 按钮宽度 */
    int      ySize; /* 按钮高度 */
    const char *acLabel; /* 按钮所代表的字符 */
    void (* pfDraw)(WM_HWIN hWin); /* 按钮的绘制函数 */
    const char *ButString; /* 按钮上显示的字符 */
} BUTTON_DATA;
```

每个分键盘上都有大量的按钮，这些按钮的位置、大小、所代表的字符和显示在 BUTTON 上的字符都用 BUTTON_DATA 来定义，本例程我们需要 5 个结构体数组来定义这些 BUTTON 的信息，如表 40.2.1 所示：

结构体数组	描述
_aEngButtonData[]	英文键盘各个 BUTTON 信息
_aNumButtonData[]	数字键盘各个 BUTTON 信息
_aSinButtonData[]	符号键盘各个 BUTTON 信息，注意！这是一个二维数组。
_aChineseButtonData1[] _aChineseButtonData2[]	中文键盘各个 BUTTON 信息，中文键盘虽然只有一个，但是由于其拼音输入部分和拼音汉字选择部分的 BUTTON 属性不同，因此要分开处理

表 40.2.1 键盘按钮信息结构体数组

40.2.2 分键盘的绘制

不同的分键盘肯定需要不同的绘制函数，emwin_demo.c 中一共有 5 个键盘绘制函数，如表 40.2.2.1 所示：

函数	描述
emwin_drawenglish_pad()	绘制英文键盘
emwin_drawingnumber_pad()	绘制数字键盘
emwin_drawsign_pad()	绘制符号键盘
emwin_drawchinese1_pad() emwin_drawchinese2_pad()	绘制中文键盘

表 40.2.2.1 分键盘绘制函数

可以看出中文键盘绘制函数有两个，那么肯定是负责不同的部分的，那么问题来了！中文键盘分为了哪两部分？中文键盘不是只有一个的吗？请看下图所示：

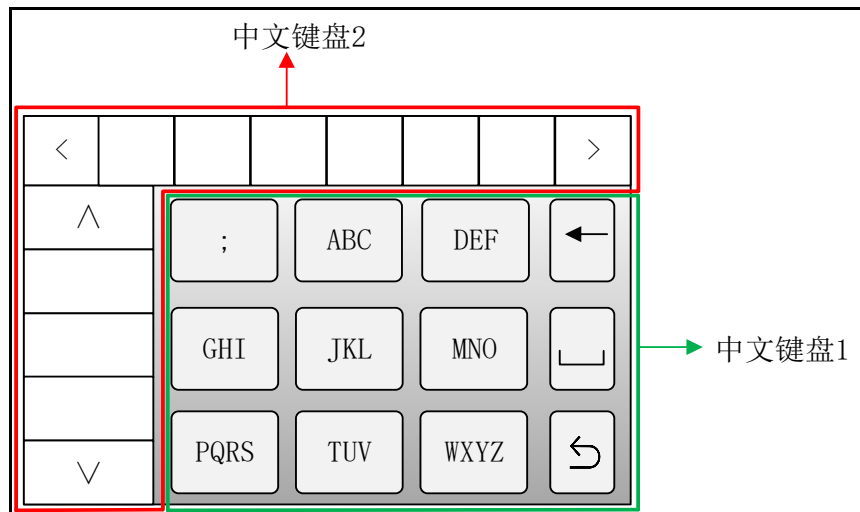


图 40.2.2.2 中文键盘分布

从图 40.2.2.2 可以看出，中文键盘分为了两部分：中文键盘 1 和中文键盘 2，中文键盘 1 是用来输入拼音的，中文键盘 2 用来选择的拼音和最终需要的汉字。这两个键盘的功能和外观是不同的，所以需要两个函数来分别绘制。

分键盘的绘制大同小异，我们以英文键盘为例来讲解，函数 `emwin_drawenglish_pad` 源码如下：

```
void emwin_drawenglish_pad(WM_HWIN hWin)
{
    uint16_t i;

    /* 设置 BUTTON 所要使用的皮肤，不使用皮肤的话可以不要这一句 */
    BUTTON_SetDefaultSkin(_DrawSkinFlex_BUTTON);

    for (i = 0; i < GUI_COUNTOF(_aEngButtonData); i++)
    {
        /* 调用函数 BUTTON_CreateEx() 来创建英文键盘上的每个 BUTTON 按钮，
           这些按钮的参数就是使用的结构体数组_aEngButtonData() */
        keypad_dev.EngButton_Handle[i]=BUTTON_CreateEx(_aEngButtonData[i].xPos,
                                                         _aEngButtonData[i].yPos,
                                                         _aEngButtonData[i].xSize,
                                                         _aEngButtonData[i].ySize,
                                                         hWin,
                                                         WM_CF_SHOW | WM_CF_HASTRANS, 0,
                                                         ID_BUTTON + i);

        /* 设置按钮的字体 */
        BUTTON_SetFont(keypad_dev.EngButton_Handle[i], &GUI_FontHZ24);

        /* 设置按钮的文本 */
        BUTTON_SetText(keypad_dev.EngButton_Handle[i],
```

```

        _aEngButtonData[i].ButString);

    /* 所有的 BUTTON 默认都不是焦点 */
    BUTTON_SetFocussable(keypad_dev.EngButton_Handle[i], 0);
}

changepcase(keypad_dev.LowerCase, _aEngButtonData);
}

```

键盘绘制完成以后还需要根据当前是大写键盘还是小写键盘来修改键盘上的字符显示，此功能由函数 `emwin_changepcase` 来完成，函数 `emwin_changepcase` 源码如下：

```

void emwin_changepcase(uint8_t change_flag, BUTTON_DATA *buttondata)
{
    uint16_t i;
    char NumToStr;
    char buffer[5];

    if (keypad_dev.LowerCase) /* 小写,按钮修改为小写 */
    {
        /* 更改按钮的字符 */
        for (i = 0; i < 27; i++)
        {
            /* 第 19 个 BUTTON 是大小写切换键“个”，所以跳过不设置 */
            if (i == 19) continue;

            /* 当前为小写输入，每个键盘上的字母都必须显示为小写，
            所以需要根据大写字符得到小写字符 */
            NumToStr = (char)((int) * (buttondata[i].acLabel) + 0x20);
            sprintf(buffer, "%c", NumToStr); /* 转换为字符 */
            BUTTON_SetFont(keypad_dev.EngButton_Handle[i], &GUI_FontHZ24);
            /* 得到的小写字符写入到 BUTTON 上去，这样所有的 BUTTON 就显示小写字符了 */
            BUTTON_SetText(keypad_dev.EngButton_Handle[i], buffer);
            BUTTON_SetFocussable(keypad_dev.EngButton_Handle[i], 0);
        }
    }

    else /* 大写，按钮上的字符改为大写 */
    {
        /* 更改按钮的字符 */
        for (i = 0; i < 27; i++)
        {
            if (i == 19) continue;

            BUTTON_SetFont(keypad_dev.EngButton_Handle[i], &GUI_FontHZ24);
            /* 结构体数组_aEngButtonData[]中的 ButString 默认就是大写的，
            所以不需要经过小写转换大写的过程，可以直接显示 */
            BUTTON_SetText(keypad_dev.EngButton_Handle[i],

```

```

        buttontdata[i].acLabel);

    BUTTON_SetFocussable(keypad_dev.EngButton_Handle[i], 0);

}

}

}
    
```

40.2.3 分键盘处理过程

键盘按下的是什么东西？其代表什么字符？该如何处理？等等，当一个 BUTTON 被按下以后肯定是需要处理过程的，不同的键盘肯定需要不同的处理过程。四个分键盘有四个处理函数，如下表所示：

函数	描述
emwin_engkeypad_process()	英文键盘处理过程
emwin_numkeypad_process()	数字键盘处理过程
emwin_signkeypad_process()	符号键盘处理过程
emwin_chinesekeypad_process()	中文键盘处理过程

表 40.2.3.1 分键盘处理过程

同样，以英文键盘处理函数 emwin_engkeypad_process 为例来分析一下处理过程，函数 emwin_engkeypad_process()源码如下：

```

void emwin_engkeypad_process(BUTTON_DATA *buttontdata, int Id, WM_MESSAGE *pMsg)
{
    char        c;
    WM_MESSAGE   Msg;
    uint8_t      KeyID;

    /* 获取 BUTTON 在分键盘上的序号，默认从 0 开始的 */
    KeyID = Id - ID_BUTTON;

    if (buttontdata[Id - ID_BUTTON].acLabel)
    {
        /* 获取某个 BUTTON 所代表的字符，比如英文键盘上的 Q 键就代表字符"Q" */
        c = buttontdata[Id - ID_BUTTON].acLabel[0];

        if (KeyID == 19)                                /* 大小写切换 */
        {
            keypad_dev.LowerCase ^= 1;                  /* 对 LowerCase 进行异或操作 */
            /* 19 号键被按下，说明要切换大小写，调用函数 changecase() 更改大小写 */
            changecase(keypad_dev.LowerCase, buttontdata);
        }
        else if (KeyID == 28)                            /* 切换到数字键盘 */
        {
            keypad_dev.padttype = NUMBER_KEYPAD;         /* 数字键盘 */
            Msg.MsgId = CHANGE_ENGLISHPADTO_NUMPAD;
            Msg.hWinSrc = pMsg->hWin;
            /* 28 号键被按下，说明要切换到数字键盘，这里我们不直接切换，
    
```


而是给软键盘的父窗口发送切换键盘的消息，
 然后在父窗口的回调函数中完成分键盘的切换，
 软键盘的父窗口回调函数后面会讲解 */

```
WM_SendMessage(keypad_dev.hKeypad, &Msg);
}
else if (KeyID == 29)                                /* 切换到符号键盘*/
{
    keypad_dev.padtype = SIGN_KEYPAD;
    Msg.MsgId = CHANGE_ENGLISHPADTO_SIGNPAD1;
    Msg.hWinSrc = pMsg->hWin;
    /* 29 号键被按下，说明要切换到符号键盘，发送切换消息 */
    WM_SendMessage(keypad_dev.hKeypad, &Msg);
}
else if (KeyID == 33)                                /* 切换到中文键盘 */
{
    keypad_dev.padtype = CHINESE_KEYPAD;
    Msg.MsgId = CHANGE_ENGLISHPADTO_CHINESEPAD;
    Msg.hWinSrc = pMsg->hWin;
    /* 33 号键被按下，说明要切换到中文键盘，发送切换消息 */
    WM_SendMessage(keypad_dev.hKeypad, &Msg);
}
else
{
    if (keypad_dev.LowerCase && (c >= 0x41) && (c <= 0x5A)) /* 小写 */
    {
        /* 程序如果到了这里说明被按下的不是切换键盘类的 BUTTON，
        此时字符类的 BUTTON 被按下，并且还是小写，
        所以需要对 c 加上一个 0x20，从大写转换为小写字母 */
        c += 0x20;
    }

    /* 将获得的字符添加到键盘缓冲器中，
    这样获得焦点的并且具有输入功能的控件就可以显示这个字符了，
    比如 EDIT 或者 MULTIEDIT 控件。注意！如果是中文键盘的话，
    首先获得要输入的汉字，然后调用相应控件的添加字符的函数来添加！！
    比如 MULTIEDIT 控件就使用函数 MULTIEDIT_AddText() 来添加这个汉字到控件中 */
    GUI_StoreKeyMsg(c, 1); /* 把消息存进键盘缓冲器，按下状态 */
    GUI_StoreKeyMsg(c, 0); /* 把消息存进键盘缓冲器，松开状态 */
}
}
```

40.2.4 软键盘父窗口

上面说的这些分键盘他们肯定要有有一个父窗口的，这个父窗口的回调函数也要做一些很重要的事情，本例程的父窗口回调函数如下：

```
/* keypad 窗口调用的回调函数 */
static void _cbKeyPad(WM_MESSAGE *pMsg)
{
    WM_HWIN    hWin;
    int        Id;
    int        NCode;
    int        xSize;
    int        ySize;
    uint16_t    slen;
    unsigned    i;

    hWin = pMsg->hWin;

    switch (pMsg->MsgId)
    {
        case WM_CREATE: /* 此消息在窗口创建后立即发送，是窗口可以初始化并创建任何子窗口 */
            /* 默认创建英文键盘，即主键盘为英文键盘 */
            drawenglish_pad(hWin); /* 创建英文键盘 */
            break;

        case WM_PAINT:
            /* 绘制背景 */
            xSize = WM_GetWindowSizeX(hWin);
            ySize = WM_GetWindowSizeY(hWin);
            GUI_SetColor(COLOR_BORDER);
            GUI_DrawRect(0, 0, xSize - 1, ySize - 1);
            GUI_DrawGradientV(1, 1, xSize - 2, ySize - 2, COLOR_KEYPAD0,
                              COLOR_KEYPAD1);
            break;

        case WM_NOTIFY_PARENT: /* 告诉父窗口，子窗口发生了改变 */
            /* 想当前焦点窗口发送按键消息 */
            Id = WM_GetId(pMsg->hWinSrc);
            NCode = pMsg->Data.v;

            switch (NCode)
            {
                /* 键盘有键被按下，根据不同的键盘来调用不同的键盘处理函数 */
                case WM_NOTIFICATION_RELEASED: /* 按钮被释放 */
                    if (keypad_dev.padtype == ENGLISH_KEYPAD) /* 英文键盘 */

```

```

        {
            engkeypad_process(_aEngButtonData, Id, pMsg);
        }
        else if (keypad_dev.padtype == NUMBER_KEYPAD) /* 数字键盘 */
        {
            numkeypad_process(_aNumButtonData, Id, pMsg);
        }
        else if (keypad_dev.padtype == SIGN_KEYPAD)
        {
            signkeypad_process(
                _aSinButtonData[keypad_dev.signpad_flag], Id, pMsg);
        }
        else if (keypad_dev.padtype == CHINESE_KEYPAD)
        {
            chinesekeypad_process(_aChineseButtonData1, Id, pMsg);
        }
        break;
    }
    break;

/* 收到从英文键盘切换到数字键盘的消息 */
case CHANGE_ENGLISHPADTO_NUMPAD: /* 从英文键盘切换到数字键盘 */
    /* 首先删除英文键盘上的所有 BUTTON */
    for (i = 0; i < GUI_COUNTOF(_aEngButtonData); i++)
        WM_DeleteWindow(keypad_dev.EngButton_Handle[i]);

    /* keypad_dev 中保存的所有的 BUTTON 句柄清零 */
    for (i = 0; i < GUI_COUNTOF(_aEngButtonData); i++)
        keypad_dev.EngButton_Handle[i] = 0;

    /* 调用函数 drawnumber_pad() 创建数字键盘 */
    drawnumber_pad(hWin);
    break;

case CHANGE_NUMPADTO_ENGLISHPAD: /* 从数字键盘切换到英文键盘 */
    for (i = 0; i < GUI_COUNTOF(_aNumButtonData); i++)
        WM_DeleteWindow(keypad_dev.NumButton_Handle[i]);
    for (i = 0; i < GUI_COUNTOF(_aEngButtonData); i++)
        keypad_dev.NumButton_Handle[i] = 0;
    drawenglish_pad(hWin);
    break;

case CHANGE_ENGLISHPADTO_SIGNPAD1: /* 从英文键盘切换到符号键盘 */

```

```

        for (i = 0; i < GUI_COUNTOF(_aEngButtonData); i++)
            WM_DeleteWindow(keypad_dev.EngButton_Handle[i]);

        for (i = 0; i < GUI_COUNTOF(_aEngButtonData); i++)
            keypad_dev.SignButton_Handle[i] = 0;

        drawsign_pad(hWin);
        break;

case CHANGE_SIGNPADTO_ENGLISHPAD: /* 从符号键盘切换到英文键盘 */
    for(i=0;i<GUI_COUNTOF(_aSinButtonData[keypad_dev.signpad_flag]);i++)
        WM_DeleteWindow(keypad_dev.SignButton_Handle[i]);
    for(i=0;i<GUI_COUNTOF(_aSinButtonData[keypad_dev.signpad_flag]);i++)
        keypad_dev.SignButton_Handle[i] = 0;

    drawenglish_pad(hWin);
    break;

case CHANGE_SIGNPAD: /* 切换符号键盘 */
    for(i=0;i<GUI_COUNTOF(_aSinButtonData[keypad_dev.signpad_flag]);i++)
        WM_DeleteWindow(keypad_dev.SignButton_Handle[i]);
    for(i=0;i<GUI_COUNTOF(_aSinButtonData[keypad_dev.signpad_flag]);i++)
        keypad_dev.SignButton_Handle[i] = 0;

    drawsign_pad(hWin);
    break;

case CHANGE_ENGLISHPADTO_CHINESEPAD: /* 从英文键盘切换到中文键盘 */
    for (i = 0; i < GUI_COUNTOF(_aEngButtonData); i++)
        WM_DeleteWindow(keypad_dev.EngButton_Handle[i]);
    for (i = 0; i < GUI_COUNTOF(_aEngButtonData); i++)
        keypad_dev.EngButton_Handle[i] = 0;

    drawchinese1_pad(hWin);
    drawchinese2_pad(hWin);
    break;

case CHANGE_CHINESEPADTO_ENGLISHPAD: /* 从中文键盘切换到英文键盘 */
    for (i = 0; i < 25; i++)
        WM_DeleteWindow(keypad_dev.ChineseButton_Handle[i]);

    for (i = 0; i < 25; i++) keypad_dev.ChineseButton_Handle[i] = 0;
    
```

```

memset(keypad_dev.inputstr, 0, 7); /* 清零 */
keypad_dev.cur_index = 0;
keypad_dev.inputlen = 0;
keypad_dev.result_num = 0;
keypad_dev.pypagenum = 0;
keypad_dev.pynowpage = 0;
keypad_dev.dispagecur = 0;
keypad_dev.dispagenum = 0;
keypad_dev.sta = 0;
drawenglish_pad(hWin);
break;

case CHANGE_CHINESE_PY: /* 显示九宫格输入的拼音选项 */
    if (keypad_dev.cur_index>0) /* keypad_dev.curindex>0 说明有拼音选中 */
    {
        /* 获得当前码表 */
        keypad_dev.disstr = t9.pymb[keypad_dev.cur_index - 1]->pymb;
        /* 得到当前选中的拼音对应的汉字字符串的长度 */
        slen = strlen((const char *)keypad_dev.disstr);
        keypad_dev.hznum = slen / 2; /* 因为是中文，每个字视为 2 个字符。*/
    }
    else /* 没有拼音选中 */
    {
        keypad_dev.disstr = NULL;
        slen = 0;
        keypad_dev.hznum = 0;
    }
    /* 得到汉字字符串的总页数 */
    keypad_dev.dispagenum = keypad_dev.hznum / 6 +
        (keypad_dev.hznum % 6 ? 1 : 0);

    /* 得到拼音页数 */
    keypad_dev.pypagenum = keypad_dev.result_num / 3 +
        (keypad_dev.result_num % 3 ? 1 : 0);

    if (keypad_dev.inputstr[0] != NULL)
    {
        for (i = 0; i < 3; i++)
        {
            BUTTON_SetFont(keypad_dev.ChineseButton_Handle[21 + i],
                &GUI_FontHZ24);

            if ((i + 3 * (keypad_dev.pynowpage)) >
                (keypad_dev.result_num - 1))
            {

```

```

        BUTTON_SetText(keypad_dev.ChineseButton_Handle[21 + i], " ");
    }
    else
    {
        BUTTON_SetText(keypad_dev.ChineseButton_Handle[21 + i],
            (const char *)t9.pymb[i + 3 * (keypad_dev.pynowpage)]->py);
    }
}

/* 更新横向按钮名字 */
keypad_hbtn_updc(keypad_dev.disstr + (keypad_dev.dispagecur * 6 * 2));
/* 绘制横向按钮 */
keypad_draw_hbtn(keypad_dev.ChineseButton_Handle);

}

else /* 没有输入任何拼音 */
{
    for (i = 0; i < 3; i++)
    {
        BUTTON_SetFont(keypad_dev.ChineseButton_Handle[21 + i],
            &GUI_FontHZ24);
        BUTTON_SetText(keypad_dev.ChineseButton_Handle[21 + i], " ");
    }

    for (i = 0; i < 6; i++)
    {
        keypad_dev.disbuf[i][0] = ' ';
        keypad_dev.disbuf[i][1] = ' ';
        keypad_dev.disbuf[i][2] = '\\0';
        keypad_dev.butname[i] = keypad_dev.disbuf[i];
    }

    /* 绘制横向按钮 */
    keypad_draw_hbtn(keypad_dev.ChineseButton_Handle);
}

break;
}
}

```

其他键盘切换处理过程基本上都是一样的。

40.3 软键盘实验

40.3.1 硬件设计

1. 例程功能

本实验主要讲解软键盘实验，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数根据多个数组创建软键盘，该软键盘可以切换多种类型键盘，如中文键盘、英文键盘以及数字键盘等，这些键盘的输入字符都会在 multiedit 控件中显示。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 39 软键盘实验》。

注意：本实验不支持 Mini、DMF407 开发板运行。

40.3.2 软件设计

40.3.2.1 程序流程图

本实验的程序流程图，如下图所示：

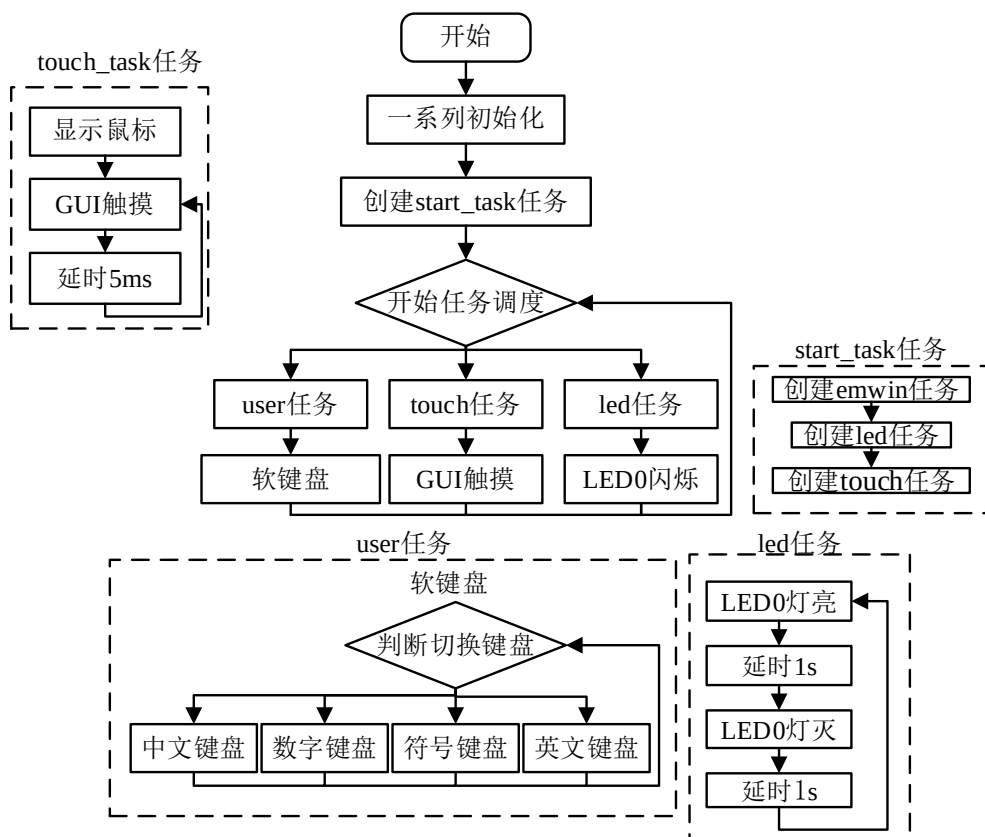


图 40.3.2.1.1 软键盘实验流程图

40.3.2.2 程序解析

emwin_main 函数即为本章例程的演示例程，函数源码如下：

```
void emwin_main(void)
{
```

```

WM_HWIN hFrame;

WM_SetCallback(WM_HBKWIN, _cbBk);      /* 是指背景窗口回调函数 */

keypad_dev.xpos = 0;
keypad_dev.ypos = (lcddev.height * 6) / 10;
keypad_dev.width = lcddev.width;
keypad_dev.height = lcddev.height - ((lcddev.height * 6) / 10);
keypad_dev.padtype = ENGLISH_KEYPAD;    /* 默认为英文键盘 */
keypad_dev.signpad_flag = 0;
keypad_dev.signpad_num = 2;
keypad_dev.inputlen = 0;
keypad_dev.pynowpage = 0;
keypad_dev.cur_index = 0;
keypad_dev.sta = 0;

/* 设置 keypad 所使用的按钮的皮肤 */
BUTTON_SetDefaultSkin(_DrawSkinFlex_BUTTON); /* 设置 BUTTON 的皮肤 */

/* 创建软键盘，其实本质上是创建一个窗口，这个窗口是软键盘的父窗口 */
keypad_dev.hKeypad = WM_CreateWindowAsChild(keypad_dev.xpos,
                                              keypad_dev.ypos,
                                              keypad_dev.width,
                                              keypad_dev.height,
                                              WM_HBKWIN,
                                              WM_CF_SHOW | WM_CF_STAYONTOP,
                                              _cbKeyPad, 0);

/* 设置 notepad 属性 */
BUTTON_SetDefaultSkin(BUTTON_SKIN_FLEX);
FRAMEWIN_SetDefaultSkin(FRAMEWIN_SKIN_FLEX);
FRAMEWIN_SetDefaultTextAlign(GUI_TA_HCENTER);

/* 创建一个 FrameWin 框架结构，这个 FrameWin 框架结构里面会有
   一个 multitedit 控件用来解释软键盘输入的字符和汉字 */
hFrame = FRAMEWIN_CreateEx(0, 0, 480, 480, WM_HBKWIN, WM_CF_SHOW, 0, 0,
                           "ALIENTEK NOTEPAD", 0);

FRAMEWIN_SetTextColor(hFrame, GUI_YELLOW);
FRAMEWIN_SetFont(hFrame, &GUI_Font20_ASCII);
FRAMEWIN_SetClientColor(hFrame, GUI_WHITE);

/* 创建的 FrameWin 框架上新建一个 multitedit 控件 */
hMulti = MULTIEDIT_CreateEx(0, 0, 0, 0, WM_GetClientWindow(hFrame),

```



```
WM_CF_SHOW, 0, GUI_ID_MULTIEDIT0, 100, NULL);

MULTIEDIT_EnableBlink(hMulti, 500, 1); /* 开启光标,周期 500ms */
MULTIEDIT_SetInsertMode(hMulti, 1);    /* 开启插入模式 */
MULTIEDIT_SetFont(hMulti, &GUI_FontHZ24);

/* 设置焦点在 multiedit 控件上 */
WM_SetFocus(hMulti);

while (1)
{
    GUI_Delay(100);
}
}
```

此函数调用函数 WM_CreateWindowAsChild 创建软键盘窗口，然后创建 FrameWin 框架，我们在该框架上创建 multiedit 控件，这个控件主要显示软键盘按下的字符。

40.3.3 下载验证

编译并下载例程到开发板中，默认界面如下图所示：

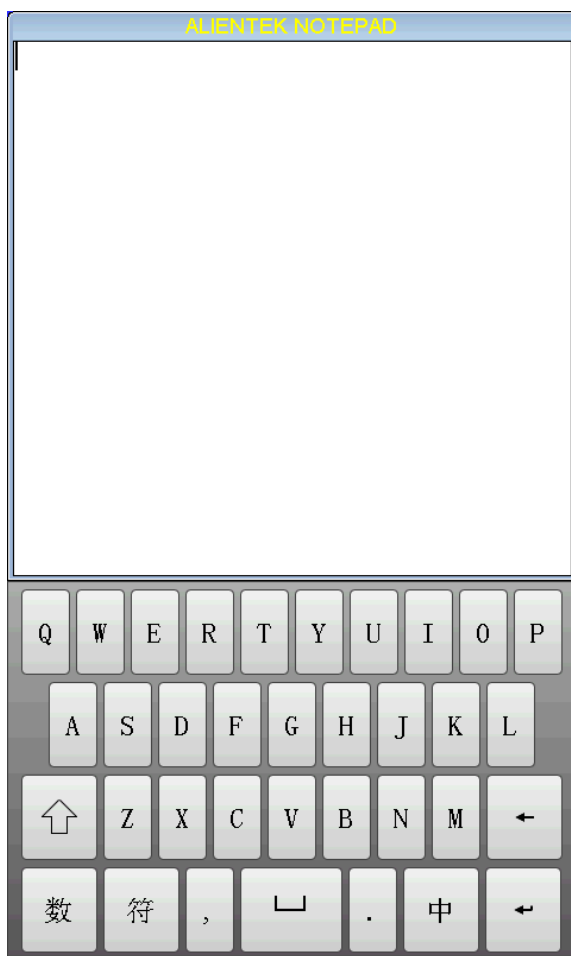


图 40.3.3.1 软键盘界面

可以在键盘上输入一些信息测试一下软键盘的各个分键盘工作是否正常，如下图所示：

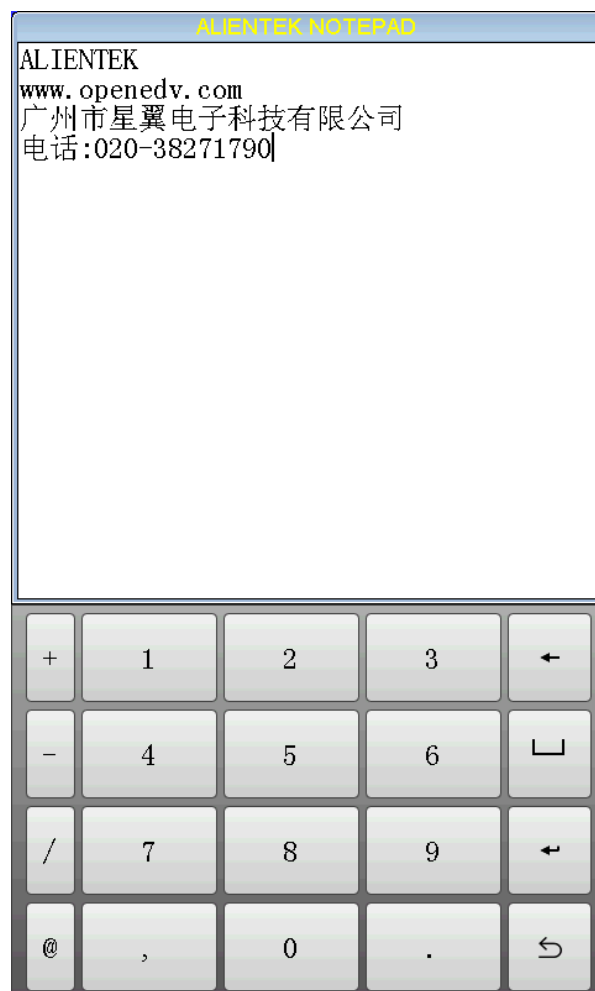


图 40.3.3.2 软键盘输入测试

第四十一章 多层显示(仅限 RGB)

emWin 本身是支持多层显示的，了解过 LTDC 接口的朋友应该对多层显示很熟悉，因为 LTDC 接口就支持双层。利用多层显示的功能可以制作很多有趣的 UI 界面，而且也可以尽可能的发挥 LTDC 接口功能。新版本的 emWin 是支持软件模拟多层显示，本章我们不讲解软件模拟的情况，对软件模拟多层显示感兴趣的可以看 emWin6.22 的官方手册。注意：本章节只适合具有 LTDC 接口的 MCU。

本章分为以下几个部分：

41.1 多层显示简介

41.2 多层 API 函数

41.3 多层演示例程

41.1 多层显示简介

如果 RGB 屏开启了多层显示功能以后，窗口可以放在任何层中，绘图操作也可以应用于任何层中。各个层的显示均使用相同的方法处理(使用相同的 API 函数)，多层显示中的每个层都可以有自己的颜色设置、尺寸设置和显示驱动。通过宏 GUI_NUM_LAYERS 来确定层数，由于 LTDC 接口最多支持两层，因此宏 GUI_NUM_LAYERS 我们设置为 2。多层显示中高索引图层位于低索引图层的上方，图层之间可以使用透明像素，如下图所示：

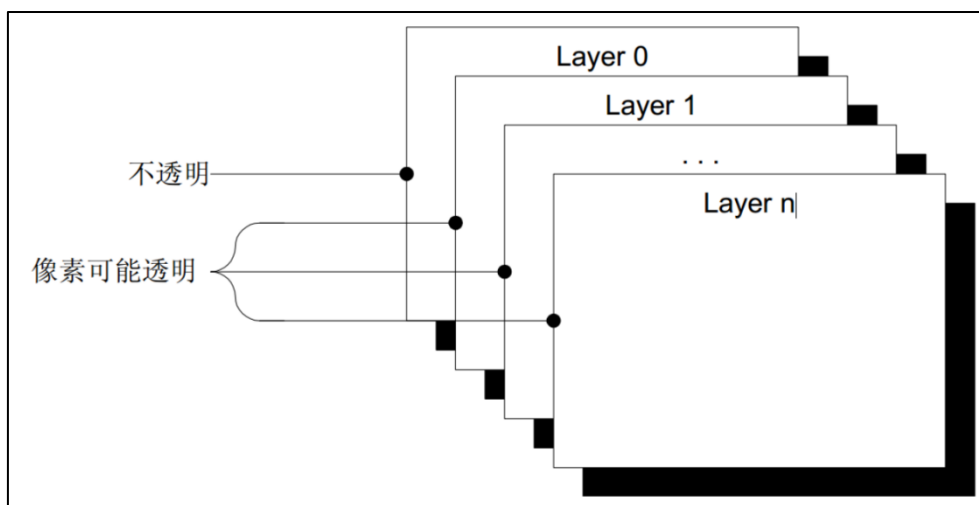


图 41.1.1 图层示意图

注意：LTDC 接口对于高分辨率 LCD 的多层支持不是很友好，经过测试可以开启双层显示的最大分辨率为 640*480，在往上就有可能出问题，比如 800*480 的话开启双层以后就会出现屏幕花屏、抖动等现象。有一些方法可以缓解屏幕花屏、抖动这些现象，如每层降低颜色格式(ARGB8888 降低为 RGB565)，像素时钟尽可能的降低，通过这些方法 800*480 分辨率的 LCD 或许能够开启双层。但是基本上也是极限了，而且也会带来其他一些问题，如无节制的降低像素时钟会导致屏幕出现水波纹等。总之一句话！800*480 以上的 RGB 接口屏不建议开启双层！多层显示支持透明，所谓的透明就是在大于 0 层中的其他层中当某个像素的颜色值设置为透明以后就可以看到底层的颜色了。目前支持透明的颜色格式只有 GUICC_M1555I 和 GUICC_8666_1，也就是说大于 0 层的其他层要想使用透明功能的话颜色格式只能设置为 GUICC_M1

555I 或 GUICC_8666_1，我们的所有的例程中第一层的颜色格式都 GUICC_M1555I。关于透明这个概念我们可以通过 segger 官方的一个例程来看一下，例程代码如下：

```
void emwin_main_demo(void)
{
    /* 第 0 层显示 R、G、B 三条颜色条 */
    GUI_SelectLayer(0);
    GUI_SetColor(GUI_RED);
    GUI_FillRect(100, 50, 380, 107);
    GUI_SetColor(GUI_GREEN);
    GUI_FillRect(100, 108, 380, 165);
    GUI_SetColor(GUI_BLUE);
    GUI_FillRect(100, 166, 380, 223);
    /* 第一层显示两个矩形和一个圆形，使用透明颜色 */
    GUI_SelectLayer(1);
    GUI_SetBkColor(GUI_WHITE);
    GUI_Clear();
    GUI_SetColor(GUI_TRANSPARENT);
    GUI_FillCircle(240, 137, 75);
    GUI_FillRect(110, 62, 155, 211);
    GUI_FillRect(325, 62, 370, 211);
}
```

函数 TransDemo 在第 0 层上使用红、绿、蓝色绘制了三个矩形，第 1 层首先使用白色填充背景，然后在使用透明颜色绘制了两个矩形了一个圆形色块，由于使用的是透明色那么肯定可以看到第 0 层的三个颜色块，这个过程如下图所示：



图 41.1.2 多层透明显示

41.2 多层 API 函数

有关多层显示的 API 函数如下表所示：

函数	描述
通用多层函数	
GUI_AssignCursorLayer()	指定管理硬件游标所使用的层
GUI_SelectLayer()	选择输出操作的 layer/display
GUI_SetLayerAlphaEx()	设置层 alpha 混合
GUI_SetLayerPosEx()	设置给定层的位置
GUI_SetLayerSizeEx()。	设置给定层的尺寸
GUI_SetLayerVisEx()	设置给定层的可见性
LCD_GetNumLayers()	返回层数

软件多层函数	
GUI_SOFTLAYER_Enable()	使能软件多层
GUI_SOFTLAYER_Refresh()	刷新软件层
GUI_SOFTLAYER_SetCompositeColor()	设置混合颜色
GUI_SOFTLAYER_MULTIBUF_Enable()	软件层使用多缓冲

表 41.2.1 多层 API 函数

在表 40.2.1 中我们使用最多的就只有两个函数 GUI_SelectLayer 和 GUI_SetLayerVisEx，这两个函数分别是选择某层和设置某层可见性的。

41.3 多层演示例程

41.3.1 硬件设计

1. 例程功能

本实验主要讲解软键盘实验，在 uc-os3_demo.c 文件中设计了四个任务，它们分别为 start_task、user_task、touch_task 和 led_task 任务，下面我们分别地讲解这些任务到底实现什么功能。

start_task 任务：主要负责创建 user_task、touch_task 和 led_task 任务，最后删除自身。

user_task 任务：主要调用函数 emwin_main 执行 emWin 程序，该函数运行 emWin 的多层显示程序。

touch_task 任务：主要调用 emWin 的 GUI_TOUCH_Exec 函数，该函数主要刷新触摸坐标。

led_task 任务：主要测试 LED0 是否在闪烁，如果 LED0 在运行，则代表系统再运行当中。

该实验的实验工程，请参考《emWin 实验 41 多层显示实验》。

注意：本实验不支持 Mini、精英、战舰、探索者、DMF407 和 Mini Pro H750 开发板运行。

41.3.2 软件设计

41.3.2.1 程序流程图

根据上述的例程功能分析，程序流程图如下图所示：

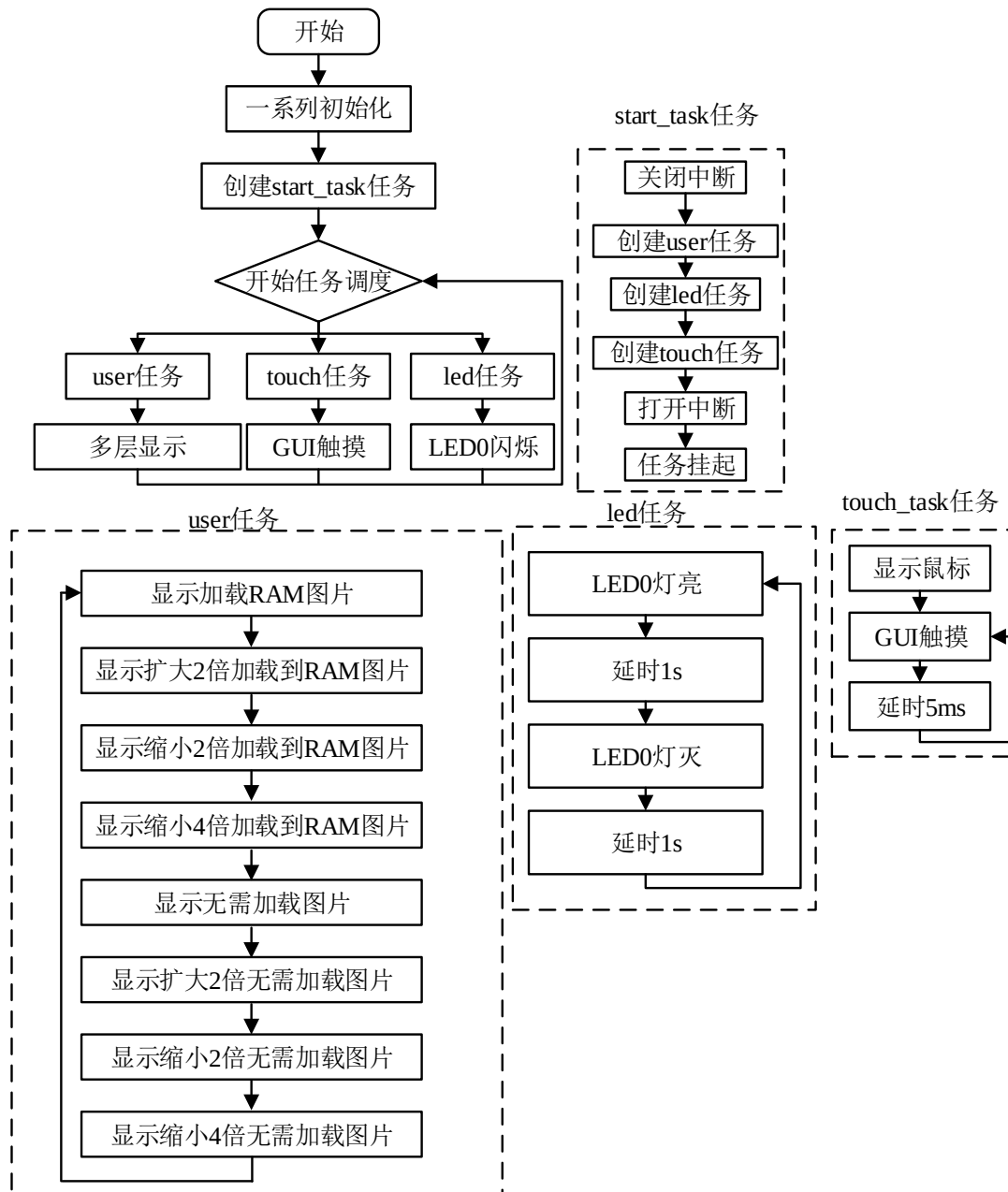


图 41.3.2.1.1 多层显示实验流程图

41.3.2.2 程序解析

本章我们利用 emWin 自带的多层显示功能来设计一个电子相框，本例程是在 Segger 官方的例程 MULTILAYER_PictureViewer.c 上修改而来的。本例程的电子相框轮流播放 SD 卡中的 JPEG 格式图片，使用过场动画进行图片切换，而这个过场动画是通过多层显示来实现的。打开工程 **emWin 试验 41 多层显示(仅限 RGB 屏)** 中的文件 pictureview.c，其中函数 EffectLineV0、EffectLineV1、EffectLineH0、EffectLineH1、EffectLineHV0 和 EffectLineHV1 这里六个函数是图片更新的时候使用的过场动画。他们的处理过程基本类似，都是通过绘制透明线条来将整个第 1 层都设置为透明色，这样就可以看到第 0 层了，而第 0 层就是下一张要显示的图片。函数 FillCurrentLayer 是使用图片填充当前活动层，代码如下：

(1) 图片刷新函数

```

/**
 * @brief 垂直刷新(从上往下绘制水平线)
 * @param xSize :屏幕宽度
 * @param ySize :屏幕高度
 * @param Delay :延时
 * @retval 无
 */
static void EffectLineV0(int xSize,int ySize,int Delay)
{
    int i;
    int j;

    GUI_SetColor(GUI_TRANSPARENT);          /* 设置颜色为透明 */
    for(i = 0;i < 32;i ++)
    {
        for (j = i;j < ySize;j += 32)
        {
            GUI_DrawHLine(j,0,xSize - 1); /* 绘制水平线 */
        }
        GUI_X_Delay(Delay);
    }
}

/**
 * @brief 垂直刷新(从下往上绘制水平线)
 * @param xSize :屏幕宽度
 * @param ySize :屏幕高度
 * @param Delay :延时
 * @retval 无
 */
static void EffectLineV1(int xSize,int ySize,int Delay)
{
    int i;
    int j;

    GUI_SetColor(GUI_TRANSPARENT);          /* 设置颜色为透明 */
    for(i = 31; i >= 0; i --)
    {
        for(j = i;j < ySize;j += 32)
        {
            GUI_DrawHLine(j,0,xSize - 1); /* 绘制水平线 */
        }
        GUI_X_Delay(Delay);
    }
}

```

```

    }
}

/**
 * @brief 水平更新 (从左往右绘制垂直线)
 * @param xSize :屏幕宽度
 * @param ySize :屏幕高度
 * @param Delay :延时
 * @retval 无
 */
static void EffectLineH0(int xSize,int ySize,int Delay)
{
    int i;
    int j;

    GUI_SetColor(GUI_TRANSPARENT);      /* 设置颜色为透明 */
    for(i = 0;i < 32;i ++)
    {
        for(j = i;j < xSize;j += 32)
        {
            GUI_DrawVLine(j,0,ySize - 1); /* 绘制垂直线 */
        }
        GUI_X_Delay(Delay);
    }
}

/**
 * @brief 绘制垂直线 (从右往左绘制垂直线)
 * @param xSize :屏幕宽度
 * @param ySize :屏幕高度
 * @param Delay :延时
 * @retval 无
 */
static void EffectLineH1(int xSize,int ySize,int Delay)
{
    int i;
    int j;

    GUI_SetColor(GUI_TRANSPARENT);      /* 设置颜色为透明 */

    for(i = 31;i >= 0;i --)
    {
        for (j = i;j < xSize;j += 32)
        {

```



```

        GUI_DrawVLine(j,0,ySize - 1); /* 绘制垂直线 */
    }
    GUI_X_Delay(Delay);
}

/**
 * @brief  矩形更新，即同时进行水平和垂直更新(从左上角向右下角绘制)
 * @param  xSize :屏幕宽度
 * @param  ySize :屏幕高度
 * @param  Delay :延时
 * @retval 无
 */
static void EffectLineHV0(int xSize, int ySize, int Delay)
{
    int i;
    int j;

    GUI_SetColor(GUI_TRANSPARENT); /* 设置颜色为透明 */
    for(i = 0;i < 32;i ++)
    {
        for(j = i;j < ySize;j += 32)
        {
            GUI_DrawHLine(j,0,xSize - 1); /* 绘制水平线 */
        }
        for(j = i;j < xSize;j += 32)
        {
            GUI_DrawVLine(j,0,ySize - 1); /* 绘制垂直线 */
        }
        GUI_X_Delay(Delay);
    }
}

/**
 * @brief  矩形更新，即同时进行水平和垂直更新(从右小角向左上角绘制)
 * @param  xSize :屏幕宽度
 * @param  ySize :屏幕高度
 * @param  Delay :延时
 * @retval 无
 */
static void EffectLineHV1(int xSize,int ySize,int Delay)
{
    int i;

```

```

int j;

GUI_SetColor(GUI_TRANSPARENT);      /* 设置颜色为透明 */
for(i = 31; i >= 0; i --)
{
    for(j = i; j < ySize; j += 32)
    {
        GUI_DrawHLine(j, 0, xSize - 1); /* 绘制水平线 */
    }
    for(j = i; j < xSize; j += 32)
    {
        GUI_DrawVLine(j, 0, ySize - 1); /* 绘制垂直线 */
    }
    GUI_X_Delay(Delay);
}
}

/**
 * @brief  图片更新函数数组
 * @param  xSize :屏幕宽度
 * @param  ySize :屏幕高度
 * @param  Delay :延时
 * @retval 无
 */
static void (* _apFunc[])(int xSize, int ySize, int Delay)
{
    EffectLineV0,
    EffectLineV1,
    EffectLineH0,
    EffectLineH1,
    EffectLineHV0,
    EffectLineHV1,
};

```

上述源码可知：我们定义了 6 个图片刷新函数，最好调用图片更新函数数组来调用这些函数。

(2) 显示图片

```

/**
 * @brief  填充当前层
 * @param  picname :要显示的图片
 * @retval 无
 */
static void FillCurrentLayer(uint8_t *picname)
{

```

```

uint8_t err=0;
GUI_MULTIBUF_Begin();
err=displyjpeg(picname,1,0,0,1,1); /* 在 LCD 中间显示图片*/
if(err) printf("图片%s 显示出错, 错误码%d\r\n",picname,err);
GUI_MULTIBUF_End();
}

/**
 * @brief  图片显示
 * @param  pFunc:图片更新函数
 * @param  Index:层索引
 * @param  Delay:延时
 * @retval 无
 */
static void ShowPicture(void (* pFunc)(int xSize, int ySize, int Delay),
                        int8_t *picname,int Delay)
{
    int xSize;
    int ySize;
    xSize=LCD_GetXSize();      /* 获取屏幕宽度*/
    ySize=LCD_GetYSize();      /* 获取屏幕高度*/
    /* 绘制第 0 层的图片, 不可见! 因为被第 1 层覆盖了*/
    GUI_SelectLayer(0);
    GUI_SetBkColor(GUI_WHITE);
    GUI_Clear();
    FillCurrentLayer(picname);
    /* 第 1 层用透明色填充, 这样第 1 层的图片就可以看到了*/
    GUI_SelectLayer(1);
    GUI_SetColor(GUI_TRANSPARENT);
    pFunc(xSize,ySize,Delay);
    /* 第 1 层也显示这张图片, 只有第一张也显示了第 0 层的图片*/
    /* 才会在显示下一张照片的时候有更新的效果出现。*/
    FillCurrentLayer(picname);
}

/**
 * @brief  得到 path 路径下, 目标文件的总个数
 * @param  path:路径
 * @retval 总有效文件数
 */
uint16_t pic_get_tnum(uint8_t *path)
{
    uint8_t res;

```

```
uint16_t rval=0;
DIR tdir;          /* 临时目录 */
FILINFO *tfileinfo; /* 临时文件信息 */
tfileinfo=(FILINFO*)mymalloc(SRAMIN,sizeof(FILINFO)); /* 申请内存 */
res = f_opendir(&tdir,(const TCHAR*)path);          /* 打开目录 */
if(res == FR_OK&&tfileinfo)
{
    while(1)          /* 查询总的有效文件数 */
    {
        res = f_readdir(&tdir,tfileinfo);          /* 读取目录下的一个文件 */
        if(res!=FR_OK||tfileinfo->fname[0]==0)break; /* 错误了/到末尾了,退出 */
        res = f_typedtell((uint8_t*)tfileinfo->fname);
        if((res&0XF0) == 0X50) /* 取高四位,看看是不是图片文件 */
        {
            rval++;      /* 有效文件数增加 1 */
        }
    }
}
myfree(SRAMIN,tfileinfo); /* 释放内存 */
return rval;
}
```

上述源码可知：ShowPicture 函数主要作用是在那个图层显示图片，而显示图片的函数为 FillCurrentLayer，该函数主要调用 emWin 的 jpeg 解码库函数。

41.3.3 下载验证

编译并下载例程到开发板中，默认界面如下图所示：



图 41.3.3.1 图片预览

第四十二章 AppWizard 设计器

AppWizard 界面开发是 emWin 新推出的 GUI 设计器，在早期的时候，我们一般使用 GUIBuilder 设计界面，由于该设计器功能比较少等原因。所以 emWin 推出了新一代 GUI 设计器 AppWizard。

本章分为以下几个部分：

42.1 AppWizard 使用

42.2 创建 AppWizard 工程

42.3 移植到工程

42.1 AppWizard 使用

42.1.1 AppWizard 下载与安装

AppWizard 设计器安装步骤如下所示：

第一步：AppWizard 设计器安装包在 emWin 软件包下 Tool 文件夹中，如下图所示：

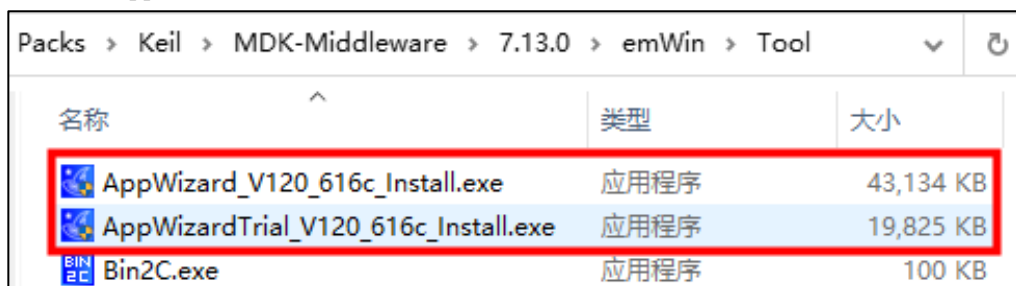


图 42.1.1.1 AppWizard 设计器安装包

第二步：双击 AppWizard_V120_616c_install.exe，如下图所示：

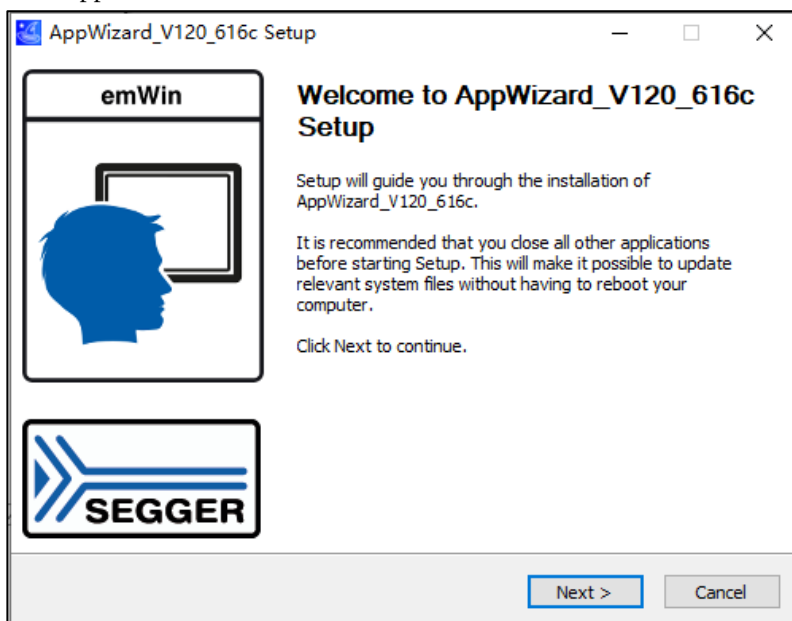


图 42.1.1.2 安装提示

第三步：点击 Next，最后点击 Finish，如下图所示：

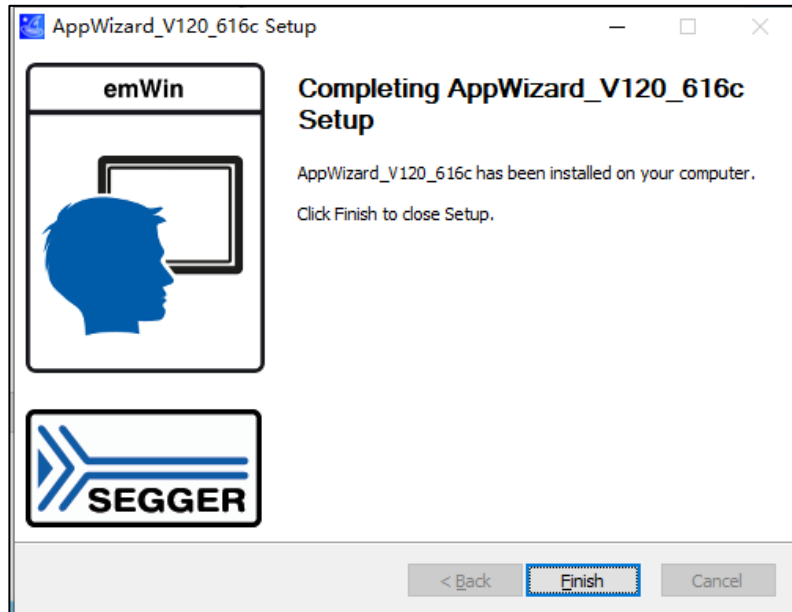


图 42.1.1.3 安装完成

42.1.2 AppWizard 框架介绍

AppWizard 设计器框架主要由 7 个区域组成，如以下所示：

1. 菜单栏：主要分为四个部分，如以下所示：

- 文件(新建项目、关闭项目、保存、打开、另存为、导出并保存、退出、最近的文件)。
- 编辑(撤销、重做、剪切、复制、粘贴、删除、全选、首选项)。
- 项目(编辑选项，播放 F5，启动模拟 F6)。
- 资源(编辑文本、编辑字体、编辑图像、编辑变量)。

AppWizard 菜单栏分布在设计器的左上方，如下图所示：

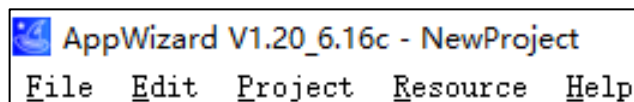


图 42.1.2.1 菜单栏

2. 添加对象窗口：主要把 emWin 控件添加编辑窗口上，如下图所示：

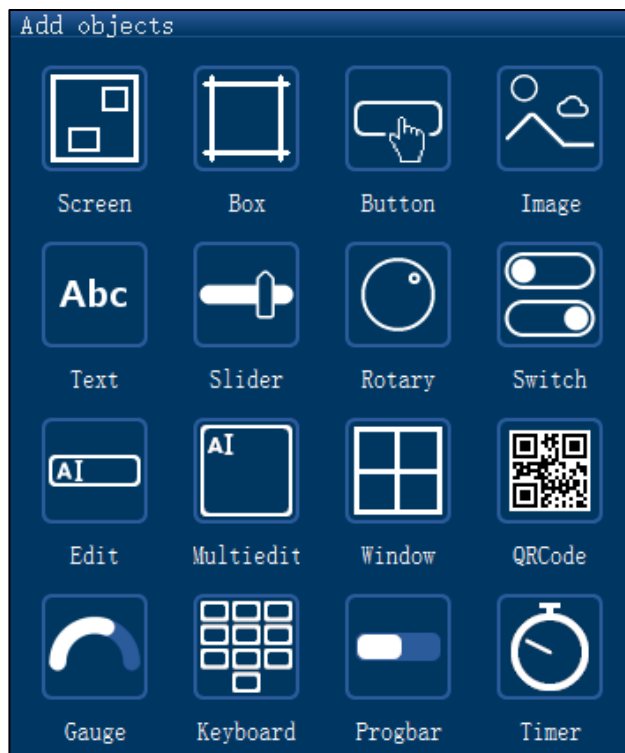


图 42.1.2.2 添加对象窗口

3. 分层的树视图：层次树视图提供了当前存在对象的快速概览。它允许在每次拖放时改变子类和父类的节点的相对位置，如下图所示：

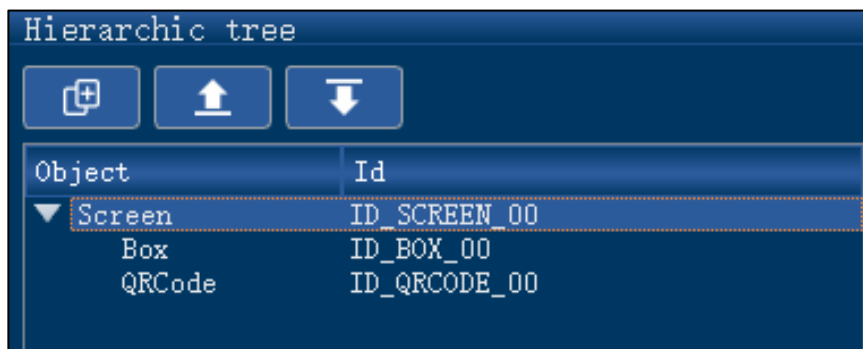


图 42.1.2.3 分层的树视图

4. 文本、字体、变量、动画和图像资源窗口：添加图片和动画等资源，如下图所示：

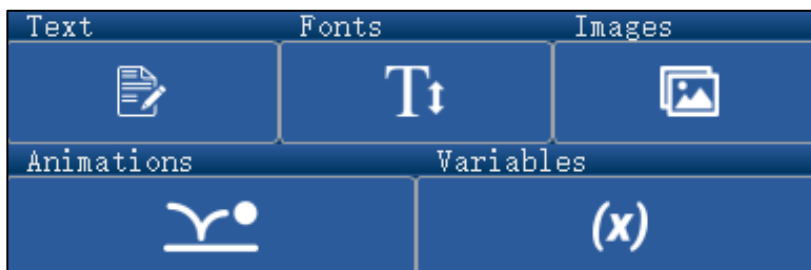


图 42.1.2.4 资源窗口

5. 编辑窗口：把控件添加到编辑器中，可调正位置、大小属性等，如下图所示：

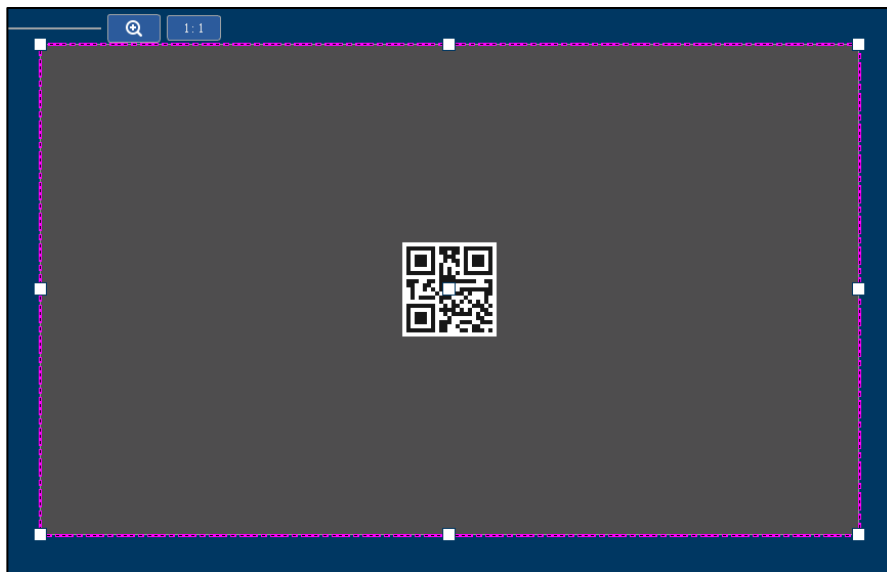


图 42.1.2.5 编辑窗口

6. 交互窗口：每个交互控件都有自己的发射器、信号、工作和接收器，如下图所示：

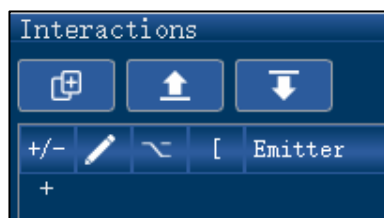


图 42.1.2.6 交互窗口

7. 属性窗口：主要设置控件的属性，例如大小、位置、颜色等属性，如下图所示：

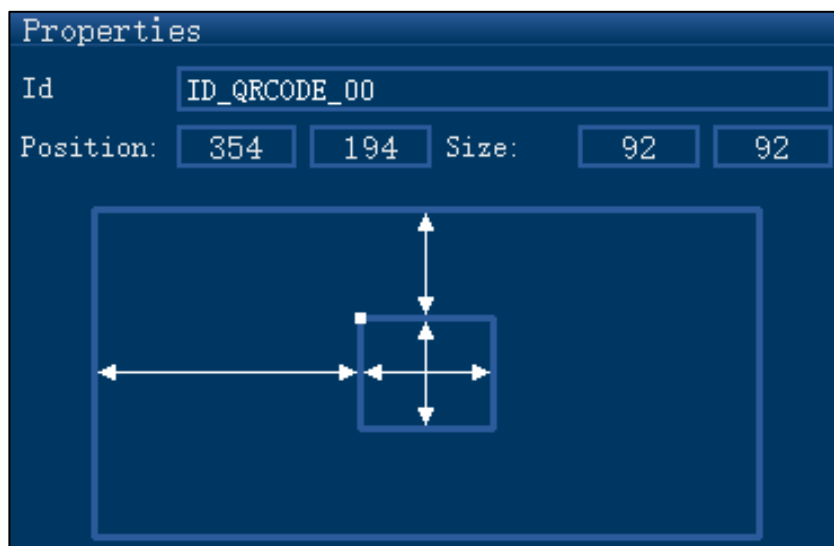


图 42.1.2.7 属性窗口

42.2 创建 AppWizard 工程流程

第一步：打开 AppWizard 软件，并新建工程如下图所示：

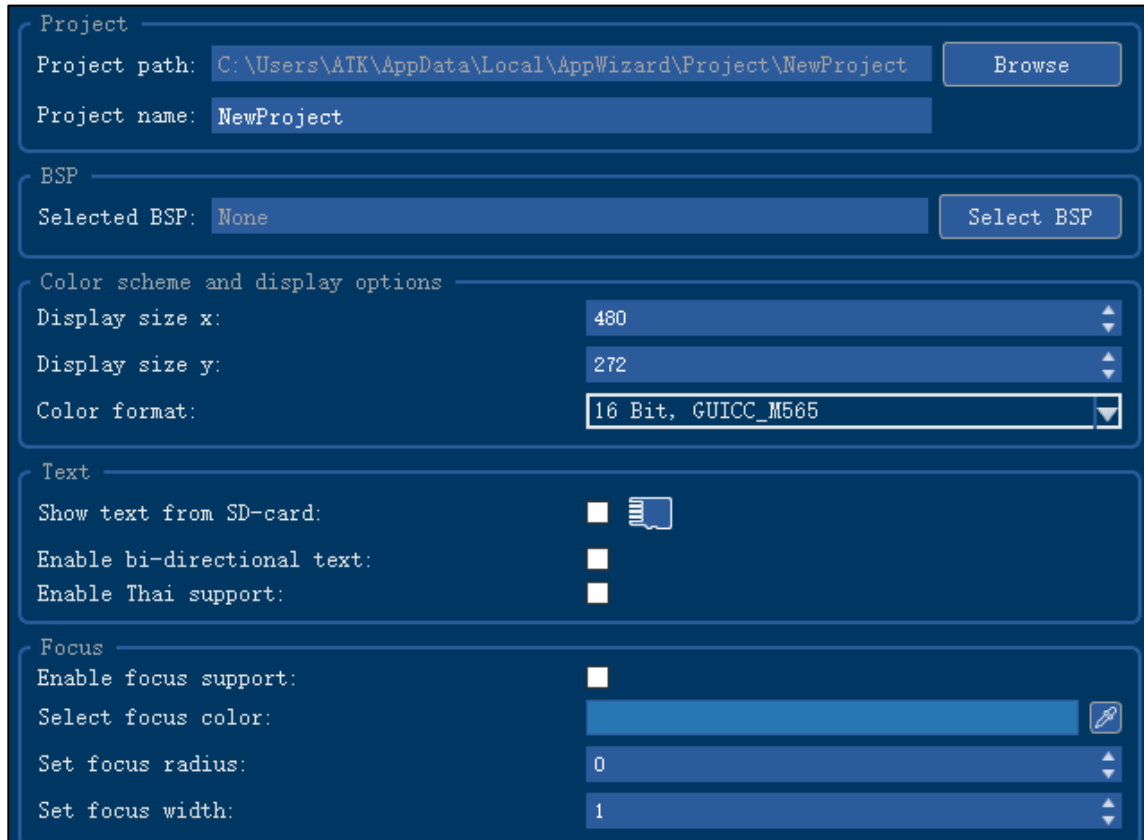


图 42.2.1 创建 AppWizard 工程

上图中，我们着重设置“Display size x”、“Display size y”以及“Color format”选项，其他的选项自行填写。注意：Color format 必须设置 16 Bit, GUICC_M565，这个选项根据自己的显示屏参数选择。

第二步：在对象窗口先添加 Screen 控件到编辑器上，然后添加 Box 控件到编辑器上，最后添加其他控件，如下图所示：

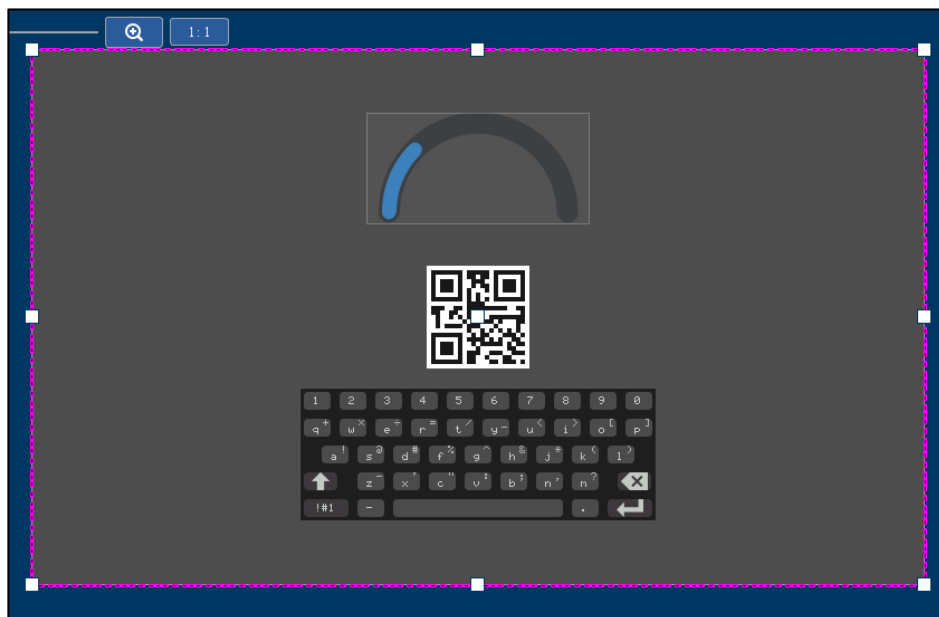


图 42.2.2 主界面

第三步：点击编辑器右上方的按钮，开启模拟器，如下图所示：



图 42.2.3 模拟 UI

为了能把代码移植到工程当中，请点击 Edit→Preferences→Manage default bitmap formats... 选项卡，然后选择 High color with alpha(565)，如下图所示：

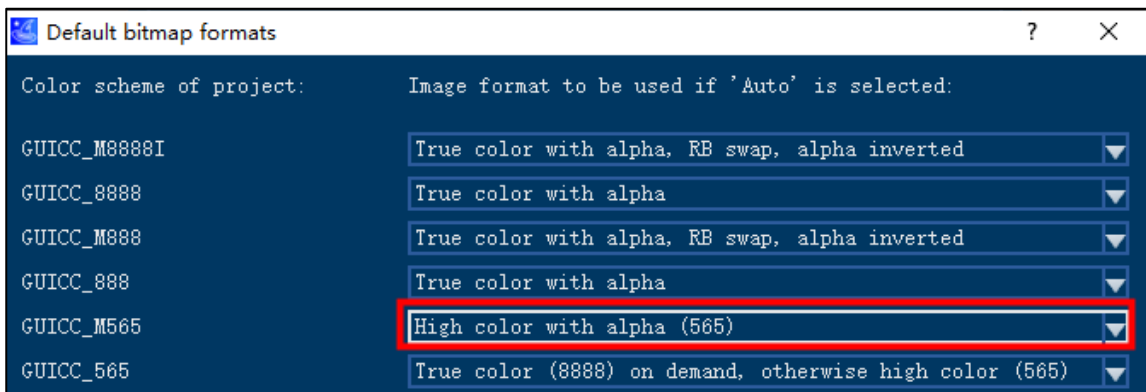


图 42.2.4 设置颜色

42.3 移植到工程

AppWizard 生成界面代码后，将其复制到工程中，这些过程我们必须注意几个事项，如下所示：

第一步：AppWizard 设计器导出代码，点击 File→Export&Save，如下图所示：

APP > NewProject		
名称	类型	大小
Resource	文件夹	
Simulation	文件夹	
Source	文件夹	
FileList.txt	文本文档	1 KB
NewProject.AppWizard	APPWIZARD 文件	4 KB

图 42.3.1 AppWizard 导出代码

第二步：准备基础工程，注意：该工程必须是 AppWizard 版本所对应的 emWin 软件包。

第三步：把图 42.3.1 中 Source 和 Resource 文件夹复制到 Middlewares/emWin/emWin_Demo 文件夹下，如下图所示：

EMWIN实验41 AppWizard > Middlewares > emWin > emWin_Demo		
名称	类型	大小
Resource	文件夹	
Source	文件夹	

图 42.3.2 复制文件夹

第四步：把图 42.3.1 中 Simulation/ GUI_Lib 的所有.h 文件复制基础工程/emWin/inc 文件夹下，如下图所示：

AppWizard.h	C++ Header file	54 KB
BUTTON.h	C++ Header file	10 KB
BUTTON_Private.h	C++ Header file	4 KB
CALENDAR.h	C++ Header file	7 KB
CHECKBOX.h	C++ Header file	9 KB
CHECKBOX_Private.h	C++ Header file	5 KB
CHOOSECOLOR.h	C++ Header file	5 KB
CHOOSEFILE.h	C++ Header file	6 KB
DIALOG.h	C++ Header file	4 KB
DIALOG_Type.h	C++ Header file	4 KB
DROPDOWN.h	C++ Header file	10 KB
DROPDOWN_Private.h	C++ Header file	5 KB
EDIT.h	C++ Header file	10 KB
EDIT_Private.h	C++ Header file	5 KB
FRAMEWIN.h	C++ Header file	12 KB
FRAMEWIN_Private.h	C++ Header file	8 KB

图 42.3.3 覆盖文件

第五步：打开基础工程，导入 emWin_Demo 文件夹下的全部文件，如下图所示：

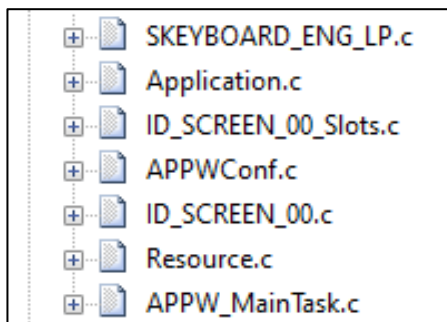


图 42.3.5 添加文件

第六步：添加头文件路径，如下图所示：

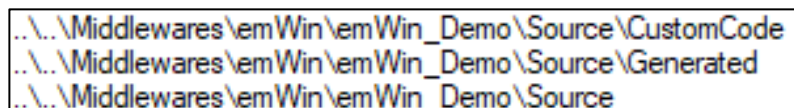


图 42.3.6 添加文件路径

第七步：修改 APPWConf.c 文件，该文件中设置_MultibufEnable 宏定义为 1，表示开始多缓冲，然后把 APPW_X_FS_Init 函数注释，最后在 main 函数调用 MainTask 函数并编译下载，如下图所示：



图 42.3.7 显示屏显示效果